心理学統計実習

Exercises in Psychological Statistics with R/RStudio

Koji Kosugi

Table of contents

はじめに 11
ライセンス等 . . . 11

第 1 章 はじめよう R/RStudio 13
1.1 環境の準備 . . . 13
1.1.1 R のインストール . . . 13
1.1.2 RStudio のインストール . . . 14
1.1.3 環境の準備に関する導入サイト . . . 14
1.2 RStudio の基礎（4つのペイン） . . . 15
1.2.1 領域 1；エディタ・ペイン . . . 16
1.2.2 領域 2；コンソール・ペイン . . . 17
1.2.3 領域 3；環境ペイン . . . 17
1.2.4 領域 4；ファイルペイン . . . 17
1.2.5 そのほかのタブ . . . 18
1.3 R のパッケージ . . . 18
1.4 RStudio のプロジェクト . . . 20
1.5 課題 . . . 21

第 2 章 R の基礎 23
2.1 R で計算 . . . 23
2.2 オブジェクト . . . 24
2.3 関数 . . . 25
2.4 変数の種類 . . . 26
2.5 オブジェクトの型 . . . 26
2.5.1 ベクトル . . . 27
2.5.2 行列 . . . 28
2.5.3 リスト型 . . . 29
2.5.4 データフレーム型 . . . 30
2.6 外部ファイルの読み込み . . . 32
2.7 おまけ；スクリプトの清書 . . . 34
2.8 課題 . . . 34

第 3 章 R によるデータハンドリング 37
3.1 tidyverse の導入 . . . 37
3.2 パイプ演算子 . . . 38
3.3 課題 1. パイプ演算子 . . . 39
3.4 列選択と行選択 . . . 39
3.4.1 列選択 . . . 39
3.4.2 行選択 . . . 42
3.5 変数を作る・再割り当てする . . . 43
3.6 課題 2.select, filter, mutate . . . 44
3.7 ロング型とワイド型 . . . 45
3.8 グループ化と要約統計量 . . . 48
3.9 課題 3. データの整形 . . . 51

第 4 章 R によるレポートの作成 53
4.1 Rmd/Quarto の使い方 . . . 53
4.1.1 概略 . . . 53
4.1.2 ファイルの作成と knit . . . 54
4.1.3 マークダウンの記法 . . . 58
4.2 プロットによる基本的な描画 . . . 59
4.3 ggplot による描画 . . . 60
4.4 幾何学的オブジェクト geom . . . 62
4.5 描画 tips . . . 65
4.5.1 ggplot オブジェクトを並べる . . . 66
4.5.2 ggplot オブジェクトの保存 . . . 68
4.5.3 テーマの変更（レポートに合わせる） . . . 69
4.6 課題 . . . 70

第 5 章 R でプログラミング 73
5.1 代入 . . . 73
5.2 反復 . . . 74
5.2.1 for 文 . . . 74
5.2.2 while 文 . . . 76
5.3 条件分岐 . . . 77
5.3.1 `if` 文の基本的な構文 . . . 77
5.4 反復と条件分岐に関する練習問題 . . . 78
5.5 関数を作る . . . 79
5.5.1 基本的な関数の作り方 . . . 79
5.5.2 複数の戻り値 . . . 80
5.6 課題 . . . 81

第 6 章 確率とシミュレーション 83
6.1 確率の考え方と使い所 . . . 83

6.2 確率分布の関数 . . . 84
6.3 乱数 . . . 87
6.3.1 乱数のつかいかた . . . 88
6.4 練習問題；乱数を用いて . . . 91
6.5 母集団と標本 . . . 92
6.6 一致性 . . . 94
6.7 不偏性 . . . 95
6.8 信頼区間 . . . 96
6.8.1 正規母集団分布の母分散が明らかな場合の信頼区間 . . . 98
6.8.2 正規母集団分布の母分散が不明な場合の信頼区間 . . . 99
6.9 課題 . . . 100

第 7 章 統計的仮説検定 101
7.1 帰無仮説検定の理屈と手続き . . . 101
7.1.1 帰無仮説検定の目的 . . . 101
7.1.2 帰無仮説検定の手続き . . . 102
7.2 相関係数の検定 . . . 102
7.3 標本相関係数の分布と検定 . . . 105
7.4 2 種類の検定のエラー確率 . . . 108
7.5 課題 . . . 111

第 8 章 平均値差の検定 113
8.1 一標本検定 . . . 113
8.2 二標本検定 . . . 115
8.3 二標本検定 (ウェルチの補正) . . . 117
8.3.1 効果量の算出 . . . 118
8.4 対応のある二標本検定 . . . 120
8.4.1 仮想データの組成 . . . 120
8.4.2 検定の方向性 . . . 123
8.5 課題 . . . 123

第 9 章 多群の平均値差の検定 125
9.1 分散分析の基礎 . . . 125
9.2 分散分析のステップ . . . 126
9.3 ANOVA 君を使う . . . 126
9.3.1 ANOVA 君の入力とデータ . . . 127
9.4 Between デザイン . . . 127
9.4.1 1way-ANOVA . . . 127
9.4.2 2way-ANOVA . . . 131
9.5 Within デザイン . . . 134
9.6 課題 . . . 139

第 10 章 疑わしき研究実践とサンプルサイズ設計 141
10.1 疑わしき研究実践 Questionable Research Practicies . . . 141
10.1.1 検定の繰り返し . . . 141
10.1.2 ボンフェロー二の方法 . . . 143
10.1.3 N 増し問題 . . . 144
10.1.4 サンプルサイズを事前に決めないことの問題 . . . 147
10.2 サンプルサイズ設計 . . . 149
10.2.1 対応のない t 検定 . . . 149
10.2.2 シミュレーションによるサンプルサイズ設計 . . . 152
10.3 課題 . . . 153

第 11 章 重回帰分析の基礎 155
11.1 回帰分析の基礎 . . . 155
11.2 回帰分析の特徴 . . . 156
11.2.1 パラメータリカバリ . . . 156
11.2.2 残差の正規性と相関関係 . . . 158
11.3 重回帰分析の特徴 . . . 162
11.3.1 回帰係数と偏回帰係数 . . . 162
11.3.2 多重共線性 . . . 164
11.3.3 変数の投入順序 . . . 165
11.4 係数の標準誤差と検定 . . . 166
11.4.1 係数の検定 . . . 166
11.4.2 モデル適合度の検定 . . . 168
11.5 サンプルサイズ設計 . . . 170
11.6 まとめ . . . 170
11.7 課題 . . . 171

第 12 章 ベイズ統計の活用 173
12.1 ベイズ統計の位置付け . . . 173
12.2 MCMC 法 . . . 174
12.2.1 ベイズの定理 . . . 174
12.2.2 ベイズの定理の実践史 . . . 175
12.2.3 Markov Chain Monte Carlo 法 . . . 176
12.3 ツールの導入；Stan と brms . . . 177
12.4 ベイズ法による推定の例 . . . 178
12.4.1 パラメータリカバリによる確認と結果の読み取り . . . 178
12.4.2 MCMC を評価する . . . 183
12.4.3 可視化してモデルの評価をみる . . . 184
12.4.4 MCMC サンプルを確認する . . . 185
12.4.5 brms パッケージのオプション . . . 186
12.4.6 MCMC サンプリングの設定 . . . 188

12.5 課題 . . . 189

第 13 章 線型代数の基礎 191
13.1 スカラー・ベクトル・行列 . . . 191
13.1.1 スカラー . . . 191
13.1.2 ベクトル . . . 191
13.1.3 行列 . . . 192
13.2 特殊な行列 . . . 193
13.3 行列の演算 . . . 194
13.3.1 加法・減法 . . . 194
13.3.2 スカラー倍 . . . 195
13.3.3 行列の積 . . . 195
13.3.4 転置 . . . 196
13.3.5 逆行列 . . . 196
13.3.6 トレース . . . 197
13.4 データの行列表現 . . . 197
13.4.1 平均ベクトルと平均偏差行列 . . . 197
13.4.2 R での確認 . . . 198
13.5 距離行列 . . . 199
13.6 固有値と固有ベクトル . . . 200
13.6.1 固有値の重要な性質 . . . 200
13.6.2 スペクトル分解 . . . 201
13.6.3 固有値分解と主成分分析 . . . 201
13.7 用語のまとめ . . . 202
13.8 課題 . . . 203

第 14 章 線型モデルの展開 205
14.1 一般線型モデル (General Linear Model) . . . 205
14.2 一般化線型モデル (GLM) . . . 206
14.2.1 ベルヌーイ分布に対する線型モデル；ロジスティック回帰分析 . . . 207
14.2.2 ポアソン分布に対する線型モデル；ポアソン回帰 . . . 212
14.3 一般化線型混合モデル (GLMM) . . . 217
14.3.1 個人差の分布を「混ぜる」 . . . 217
14.3.2 ランダム切片モデル . . . 218
14.3.3 ランダム傾きモデル . . . 228
14.3.4 ランダム切片ランダム傾きモデル . . . 233
14.4 階層線型モデル (HLM) . . . 239
14.4.1 2 レベル階層線型モデル . . . 239
14.4.2 モデルの図解 . . . 241
14.4.3 具体例 . . . 242
14.5 課題 . . . 258

14.5.1 基本問題：パラメータリカバリ . . . 258
14.5.2 応用問題：野球データの階層モデリング . . . 260

第 15 章 多変量解析 (その 1) 263
15.1 多変量解析の包括的な説明 . . . 263
15.1.1 データキューブと断面 . . . 264
15.2 因子分析 . . . 266
15.2.1 探索的因子分析 . . . 267
15.2.2 確認的因子分析 . . . 275
15.2.3 SEM の応用 . . . 282
15.3 項目反応理論 . . . 282
15.3.1 ロジスティックモデル . . . 283
15.3.2 IRT モデルの展開 . . . 294
15.4 まとめ . . . 296
15.5 課題 . . . 297
15.5.1 課題 1：探索的因子分析の実践 . . . 297
15.5.2 課題 2：確認的因子分析の実践 . . . 297
15.5.3 課題 3：多段階項目反応理論の実践 . . . 297
15.5.4 課題 4: 多次元多段階項目反応理論の実践 . . . 298

第 16 章 多変量解析 (その 2) 299
16.1 距離行列を用いるもの . . . 299
16.1.1 心理学における距離データ . . . 300
16.1.2 クラスター分析 . . . 300
16.1.3 多次元尺度構成法 . . . 313
16.2 共頻行列を用いるもの . . . 320
16.2.1 テキストマイニングとは . . . 320
16.3 偏相関行列を用いるもの . . . 325
16.4 課題 . . . 331
16.4.1 課題 1: クラスター分析による消費者セグメンテーション . . . 331
16.4.2 課題 2: 多次元尺度構成法による都市間類似性の可視化 . . . 331
16.4.3 課題 3: ネットワーク分析による心理尺度の構造解析 . . . 332

第 17 章 ベイジアンモデリング 333
17.1 ベイジアンモデリングの学習方法 . . . 333
17.1.1 学習のステップ 1; 確率的プログラミング言語 Stan の学習 . . . 333
17.1.2 学習のステップ 2; これまでの分析方法を書き直してみる . . . 336
17.2 正規分布を使ったモデル . . . 338
17.2.1 分散を推定する . . . 338
17.2.2 欠測のあるデータを有効に使う . . . 343
17.3 正規分布以外の分布を使う . . . 347
17.3.1 項目反応理論 . . . 347

17.3.2 再捕獲法による全体の推論 . . . 351
17.4 分布を混ぜる . . . 357
17.4.1 変化点を検出する . . . 357
17.4.2 異質な群を分けて考察する . . . 362

第 18 章 ベイズの観点から見た平均値差の検定 369
18.1 検定からモデリングへ . . . 369
18.1.1 多重比較の問題が生じない . . . 369
18.2 分散分析の組成を思い出す . . . 370
18.3 二群の平均値差をベイズで（t 検定） . . . 370
18.3.1 Stan によるモデルの記述 . . . 372
18.3.2 推定と結果の読み取り . . . 373
18.4 多群の平均値差をベイズで（分散分析） . . . 375
18.4.1 Stan によるモデルの記述 . . . 376
18.4.2 推定と全ペアの比較 . . . 377
18.5 意思決定の三つの道具 . . . 381
18.5.1 ROPE . . . 381
18.5.2 ベイズファクター . . . 383
18.5.3 事後予測分布 . . . 387
18.6 まとめ . . . 390
18.7 課題 . . . 390

第 19 章 演習問題 393
19.1 最終課題 . . . 394

References 397

第 20 章 あとがき 401

第 21 章 インストールガイド 403

はじめに

この資料は，授業「心理学統計演習」についてのものです。演習という授業名にあるように，理論的な解説で「理解して進む」ことよりも，「手を動かして理解する」ことを目的にしています。

この資料を活用する人は，理論的な (いわゆる座学の) 心理学統計を履修済みであることを前提にしています。また，資料集という位置付けですので，行間の説明が省略されていることが多くあります。その点は講義時間中の講話で補完していくつもりですので，不明な点があれば授業時間中に質問してください。

ライセンス等

この資料は Creative Commons BY-SA(CC BY-SA) ライセンス Version 4.0 に基づいて提供されています。著者に適切なクレジットを与える限り，この本を再利用，再編集，保持，改訂，再頒布 (商用利用を含む) をすることができます。もし再編集したり，このオープンなテキストを変更したい場合，すべてのバージョンにわたってこれと同じライセンス，CC BY-SA を適用しなければなりません。

This article is published under a Creative Commons BY-SA license (CC BY-SA) version 4.0. This means that this book can be reused, remixed, retained, revised and redistributed (including commercially) as long as appropriate credit is given to the authors. If you remix, or modify the original version of this open textbook, you must redistribute all versions of this open textbook under the same license - CC BY-SA.

第 1 章

はじめよう R/RStudio

「R」。この一文字で表現されるがゆえに，検索しにくいことこの上ないが，これは統計に特化したプログラミング言語であり，心理学はもちろん統計に関する学問領域で多岐にわたって利用されているものである。フリーソフトウェア，すなわち自由で開かれているソフトウェアであるから，ソースコードに至るまで公開されており，誰でも無償で利用できる。無償すなわち無料ではない。補償がないので無償なのだが，逆に金銭で計算をはじめ科学的真実性が保証されるわけではない，という至極まともな考え方は理解できるだろう。科学はもちろん，ソフトウェアも人類の共有財産として，オープンに育んでいこう。

R はコミュニティ活動も盛んで，Tokyo.R を中心に日本の各地で R ユーザからなる自主的な勉強会が開催されている[*1]。また R 自体がインターネットを通じて公開されているように，導入から応用までさまざまな資料がオンラインで活用できる。以下では導入から解説していくが，頻繁にアップデートされるものでもあるので，必要に応じて検索し，なるべく時系列的に近い情報を吟味して活用することを薦める。

1.1 環境の準備

1.1.1 R のインストール

R のインストールに関して，初心者でも利用可能な資料がオンラインで公開されている。

R は The Comprehensive R Archive Network，通称 CRAN[*2]というネットワークで公開されている。CRAN のトップページにはダウンロードリンクが用意されており，自分のプラットフォームにあった最新版をダウンロードしよう[*3]。

[*1] 2024 年 1 月現在で，Tokyo だけでなく Fukuoka，Sapporo，Yamaguchi，Iruma などで地方コミュニティがあり，参加者みんなで楽しまれている。

[*2] CRAN は「しーらん」，あるいは「くらん」と発音される。筆者はしーらん派。

[*3] この授業のために自身の PC に R をインストールしたとして，次に使うときに半年以上間隔が空いたのなら，改めて最新版をチェックし，バージョンが上がっていたら旧版をアンインストールして最新版をインストールするところから始めた方が良い。R で利用するパッケージなどが新しい版にしか対応していないことなどもある。R と畳は新しい方が良い。

1.1.2 RStudio のインストール

R のインストールが終われば，次は RStudio をインストールしよう。RStudio は総合開発環境 (IDE) と呼ばれるものである。R は単体で，統計の分析や関数の描画など，専門的な利用に耐えうる分析機能を有している。その本質はもちろん計算機能であって，計算を実行する命令文 (スクリプト) を与えれば，必要な返答をあたえてくれる。このように分析の本質が計算機能であったとしても，実際の分析活動に際しては，スクリプトの下書きと清書，入出力データや描画ファイルの生成・管理，パッケージ (後述) の管理など，分析にまつわるさまざまな周辺活動が含まれる。喩えるなら料理の本質が包丁・まな板・コンロによる加工であったとしても，実際の調理に際しては，広い調理スペースや使いやすいシンク，ボウルやタッパーなどの補助的な調理器具があった方がスムーズにことが進む。いわば，R 単体で分析をするのは飯盒炊爨のような必要最低限かつワイルドな調理法であり，RStudio は総合的な調理環境を提供してくれるものなのである。

繰り返しになるが，本質的には R 単体で作業が可能である。なるべく単純な環境を維持したいというのであれば R 単体での利用を否定するものではないが，RStudio はエディタや文書作成ソフトとしても有用であるので，本授業では RStudio を使うことを前提とする[*4]。

1.1.3 環境の準備に関する導入サイト

以下に執筆時点 (2025 年 4 月) で参照可能な，導入に関する Web 教材を挙げておく。自分に合ったものを適宜参照し，R と RStudio を自身の PC 環境に導入してほしい。もちろん自身で「R RStudio インストール」などとして検索しても良いし，chatGPT に相談しても良い。

1.1.3.1 For Windows

- 東京大学・大学院農学生命科学研究科アグリバイオインフォマティクス教育研究プログラムによる PDF 資料
- 初心者向け R のインストールガイド
- 関西学院大学商学部土方ゼミ資料
- 多摩大学情報社会研究所・応用統計学室資料
- 奥村晴彦先生のページ

1.1.3.2 For Macintosh

- 東京大学・大学院農学生命科学研究科アグリバイオインフォマティクス教育研究プログラムによる PDF 資料
- note の記事
- いちばんやさしい，医療統計記事

[*4] VSCode のようなエディタから使うことも可能であるし，Jupyter Notebook の計算エンジンを R にすることも可能。最近では分析ソフトウェアを個々人で準備せず，環境として提供することも一般的になってきており，例えば Google Colaboratory のエンジンを R にすることもできるようになっている。ローカル PC に自前の環境を作るということが，時代遅れになる日も近いかもしれない。

なお，Mac の場合は Homebrew などのパッケージ管理ソフトを使って導入することもできる (し，そのほうがいい)。その場合は以下の資料を参照。

- 群馬大学大学院医学系研究科機能形態学の記事
- コアラさばお氏の note 記事
- Ryu Takahashi 氏の Qiita 記事
- Yuhki Yano 氏の Qiita 記事

1.2　RStudio の基礎（４つのペイン）

ここまでで，R および RStudio を利用する準備が整っているものとする。

さて，RStudio を起動すると大きくわけて 4 つの領域に分かれた画面が出てくる。この領域のことを**ペイン**と呼ぶ。図中の「領域 1」がないように見えるときもあるが，下のペインが最大化され折りたたまれているだけなので，ペイン上部のサイズ変更ボタンを操作することで出てくるだろう。

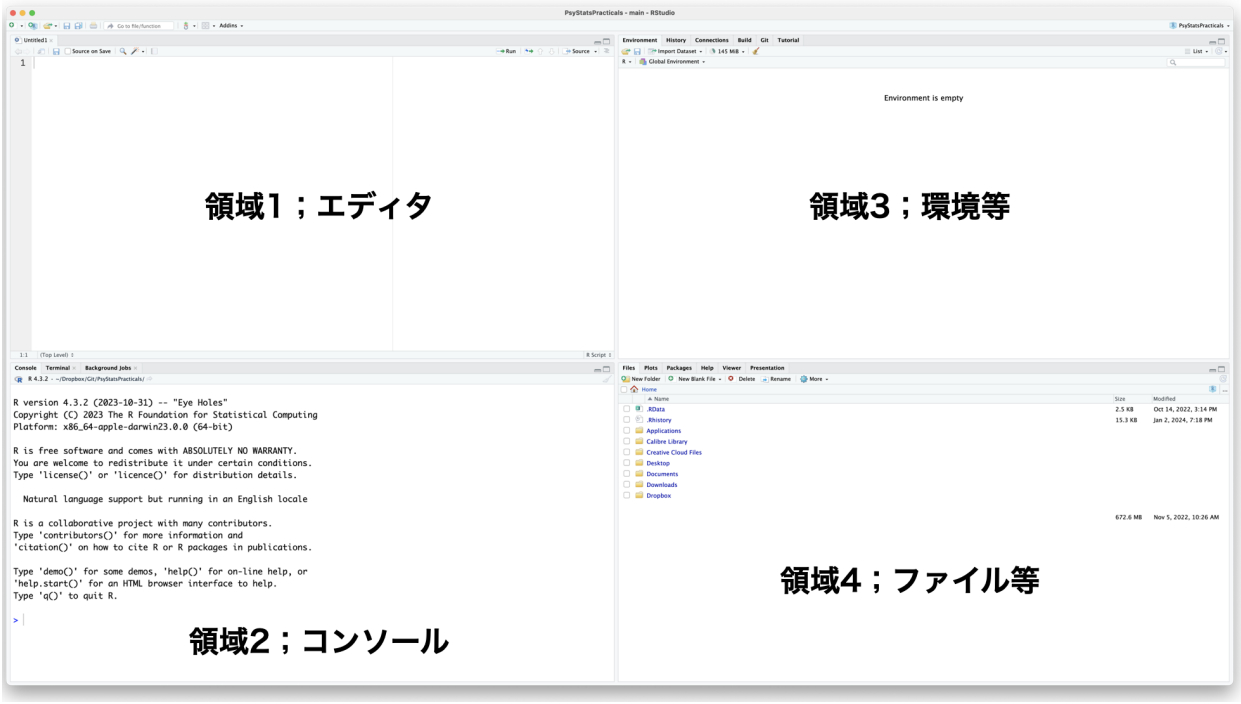


Figure1.1: RStudio の初期画面

このペインのレイアウトは，メニューの Tools > Global Options... > Pane Layout から変更することもできる。基本的に 4 分割であることに変わりはないが，自分が利用しやすい位置にレイアウトを変更するとよい。

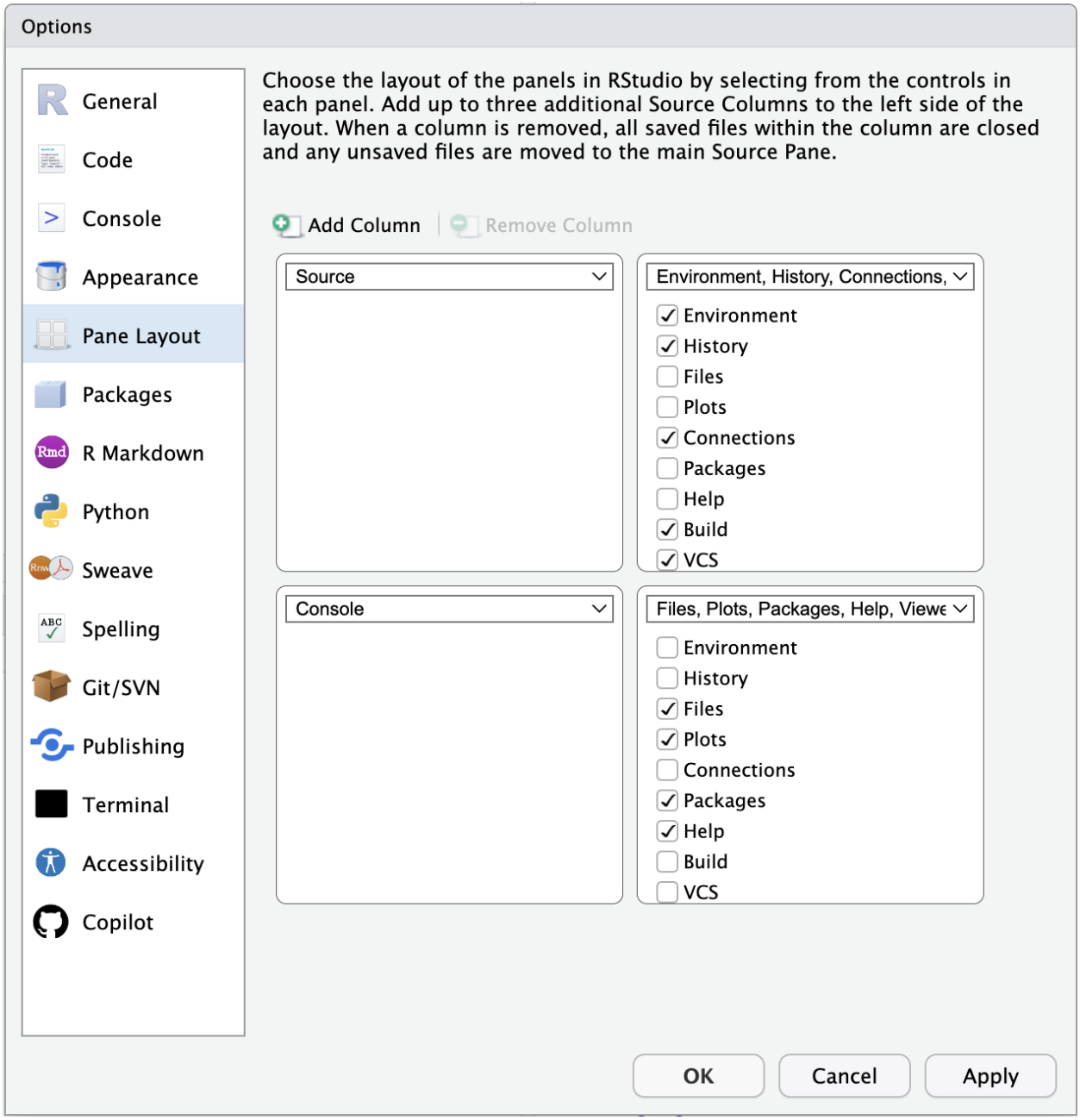


Figure1.2: レイアウト変更画面。このほかにも背景色などを変えることもできる

以下，各ペイン (領域) が何をするところかを簡単に解説する。

1.2.1　領域 1；エディタ・ペイン

エディタ領域。R のスクリプトはもちろん，レポートの文章など，基本的に入力するときはこのペインに書く。ここで作業するファイルの種類は，File > New File から見ると明らかなように，R 言語だけでなく C 言語，Python 言語などのスクリプトや，Rmd，md,Qmd,HTML などのマークアップ言語，Stan や SQL など特殊な言語など

にも対応している。ペインの右下に現在開かれているファイルの種類が表示されているのを確認しておこう。

R 言語でスクリプトを書く例で解説しよう。R は命令を逐次実行していくインタプリタ形式であり，ここに記述された R コードを，右上の Run ボタンでコンソールに送って計算を実行するように使う。一回の命令をコマンド，コマンドが積み重ねられた全体をスクリプト，あるいはプログラムと呼ぶ。複数のコマンドを実行したい場合は，エディタ領域で複数行選択して Run ボタンを，スクリプトファイル全体を実行したいときは Run ボタンのとなりにある Source を押す。CTRL+Enter(Mac の場合はコマンド +Enter) で Run ボタンのショートカットになる。

1.2.2 領域 2；コンソール・ペイン

R 単体で利用する場合は，ここのペインだけを利用するようなものである。すなわち，ここに示されているのが R 本体というか，R の計算機能そのものである。ここに「>」の記号が表示されているところをプロンプトといい，プロンプトが表示されているときは R が入力待ちの状態である。

R は逐次的に計算を行うので，プロンプトのある状態でコマンドを入力すると計算結果が返される。ここに直接コマンドを書いて行っても良いが，書き間違えたりすることもあるし，コマンドが複数行に渡ることが一般的になってくるので，エディタ領域に清書するつもりで記述していったほうがよい。ごくたまに，一時的に確認したいことがある時だけ，直接コンソールを触るようにすると良い。

なお，コンソールを綺麗にしたいときは右上の箒ボタンをおすとよい。

1.2.3 領域 3；環境ペイン

基本的にこのペインと次の領域 4 のペインは複数のタブが含まれる。Pane Layout でどちらにどのタブを含めるかを自分好みにカスタマイズすることもできる。ここでは代表的な 2 つのタブについてのみ言及する。

Environment タブは，R の実行メモリ内に保管されている変数や関数などが表示されている。「変数や関数など」をまとめて**オブジェクト**というが，ここで内容や構造を GUI で確認することができる。

History タブは履歴である。これまでコンソールに送られてきたコマンドが順に記録されている。History タブからエディタ，コンソールにコマンドを送ることも可能であり，「さっきの命令をもう一度実行したい」といったときに参照すると良い。

1.2.4 領域 4；ファイルペイン

ここでも代表的なタブについてのみ解説する。

Files タブは Mac でいう Finder，Windows でいうエクスプローラーのような，ファイル操作画面である。フォルダの作成，ファイルの削除，リネーム，コピーなどの操作が可能である。

Plot タブは R コマンドで描画命令が出された時の結果がここに表示される。RStudio の利点の一つは，この Plot から図をファイルに Export することが可能であり，その際にファイルサイズやファイル形式を指定できるところにある。

Packages タブは読み込まれているパッケージ，(読み込まれていないが) 保管しているパッケージのリストが表示されている。新しくパッケージを導入するときも，ここの install ボタンから可能であり，保管しているパッケージのアップデートもボタンひとつで可能である。なお，パッケージについては後ほど言及する。

Help タブは R コマンドでヘルプを表示する命令 (`help` 関数) が実行された時の結果が表示される領域である。ヘルプを使うことで関数の引数，戻り値，使用例などを参照できる。

1.2.5　そのほかのタブ

そのほか，表示の有無もオプションになっているようないくつかのタブについて，簡単に解説しておく。

Connections タブは R を外部データベースなどに繋げるときに参照する。大規模データをローカルにすべて取り込むことなく，SQL で必要なテーブルだけ取り出すといった操作をする際は必要になってくるだろう。

Git タブは R，とくに R プロジェクト (後述) のバージョンを管理するときに利用する。Git とは複数のプログラマによって同時並行的にプログラムを作っていく時の管理システムである。時系列的な差分の記録を得意とするシステムなので，レポートの作成時などに応用すればラボノートの記録としても利用できる。

Build タブは R パッケージや Web サイトを構築するときに利用する。なおこの資料も RStudio を利用して作られており，資料を生成 (原稿から HTML や PDF にする) ときにはこのタブを利用している。

Tutorial タブはチュートリアルツアーを楽しむ時のタブである。

Viewer タブは RStudio で作られた HTML や PDF などを見るためのタブである。

Presentation タブは RStudio で作られたプレゼンテーションを見るためのタブである。

Terminal タブは Windows/Mac でいう Terminal，Linux でいう端末についてのタブであり，R に限らず，コマンドラインを通じて OS に命令するときに使う。

Background Jobs タブはその名の通りバックグラウンドで作業をさせるときに利用する。R は基本的にシングルコアで計算が実行されるが，このタブを使ってスクリプトファイルをバックグラウンドで実行することで並列的に作業が可能になる。

1.3　R のパッケージ

R は単体でも線型モデルなどの基本的な分析は可能であるが，より進んだ統計モデルを利用したい場合は専門の**パッケージ**を導入することになる。パッケージとは関数群のことであり，これも CRAN や Github などインターネットを介して提供されている。ちなみに提供されているパッケージは，CRAN で公開されているものだけで 23,512 件あり[*5]，Github[*6]で公開されているものなど，CRAN を介さないパッケージも少なくない。

[*5] 2026 年 04 月 02 日調べ

[*6] Git はバージョン管理システムであるが，これをインターネット上のサーバ (レポジトリ) で行うものを Github という。RStudio は Github とも連携しており，プロジェクトを Github と紐づけることで簡単にバージョン管理ができる。しかもここで言及しているように，Github 上でパッケージを公開することもできるので，最近は CRAN の校閲を待たずに公開できる Github が好まれている側面もある。

パッケージを利用する際は，まずローカルにパッケージファイルをインストールしなければならない。その上で，R を起動するごとに (セッションごとに)，関数 `library` でパッケージを呼び出して利用する。インストールを毎回行う必要はないことに注意。

インストールは R のコマンドでも可能だが，RStudio の Packages ペインを使って導入するのが簡単だろう。以下に，一部の有名かつ有用なパッケージ名とその簡単な説明を挙げる。本講義の中で使うものもあるので，事前に準備しておくことが望ましい。

- *tidyverse* パッケージ (Wickham et al., 2019)；R が飛躍的に使いやすくなったのは，この tidyverse パッケージ導入以後のことである。開発者の Hadley Wickham は R 業界で神と崇められており，R 業界に与えたインパクトは大きい。このパッケージは「パッケージ群」「パッケージのパッケージ」であり，tidyverse とは tidy な (整然とした)verse(世界) というような意味合いである。このパッケージは統計分析モデルを提供するものではなく，その前のデータの**前処理**に関する便利な関数を提供する[*7]。このパッケージをインストールすると，関連する依存パッケージが次々取り込まれるので，少々時間がかかる。
- *psych* パッケージ (Revelle, 2021)；名前の通り，心理学統計に関する統計モデルの多くが含まれている。特に特殊な相関係数や，因子分析モデルなどは非常に便利なので，インストールしておいて間違いない。
- *GPArotation* パッケージ (Bernaards & Jennrich, 2005)；因子分析における因子軸の回転に使うパッケージ。
- *styler* パッケージ；スタイルを整えてくれるパッケージ。スクリプトの清書に便利。
- *lavaan* パッケージ (Rosseel, 2012)；潜在変数を含んだモデル (LAtent VAriable ANalysis) の分析，要するに構造方程式モデリング (Structural Equation Modeling;SEM，共分散構造分析ともいう) を実行するパッケージ。
- *ctv* パッケージ (Zeileis, 2005); CRAN Task Views の略で，膨大に膨れ上がった CRAN から必要なパッケージを見つけ出すのは困難であることから，ある程度のジャンルごとに関連しそうなパッケージをまとめて導入してくれるのがこのパッケージ。例えば，このパッケージをインストールした後で，`install.views("Psychometrics")` とすると，心理統計関係の多くのパッケージを次々導入してくれる。
- *cmdstanr* パッケージ (Gabry et al., 2023)；複雑な統計モデルで利用される，確率的プログラミング言語 stan を R から使うことができるようになるパッケージ。導入にはこのパッケージの他にも stan やコンパイル環境の準備が必要なので，公式の導入サイトも参考にしてほしい。
- *pacman* パッケージ (Rinker & Kurkiewicz, 2018)；パッケージマネージャ，というパッケージ。パッケージは `library` で呼び出して使うが，インストールされていないと `install.packages` でインストールしてから使うことになる。また，パッケージを A,B,C…と複数呼び出す時は，`library` 関数を数行にわたって書く必要がある。`pacman` の `p_load` 関数を使うと，もし環境にそのパッケージが含まれていなければ自動的にインストールが始まるし，`p_load(A,B,C)` のように並べて書くこともできる。この資料は基本的に `pacman::p_load` でパッケージを呼び出すようにしているので，`pacman` パッケージだけは先に入れておいてほしい。

[*7] 実は統計データの解析にかかる時間のほとんどが，解析に適切な形にデータを整形する「前処理」に費やされる。前処理，別名データハンドリングをいかに上手く，素早く，直感的にできるかは，その後の分析にも影響するほど重要な手順であるため，tidyverse パッケージの登場はありがたかった。これを使ったデータハンドリングだけの専門書松村他 (2021) が重宝されるほどである。

1.4　RStudio のプロジェクト

実際に R を使っていく前に，最後の準備として RStudio におけるプロジェクトについて解説しておく。

みなさんも，PC をつかって文書を作ったり保管したりするときに，フォルダにまとめて入れておくことがあるだろう。フォルダは例えば「文書」>「心理学」>「心理学統計演習」のように階層的に整理することが一般的で，そうしておくことで必要なファイルをすぐに取り出すことができる。

逆に言えば，こうしたフォルダ管理をしておかなければファイルが PC のなかで散乱してしまい，必要な情報を得るために逐一 PC の中身を検索しなければならないだろう。

R/RStudio をつかった分析実践の場合も同様で，一回のテーマについて複数のファイル (スクリプトファイル，データファイル，画像ファイル，レポートなど文書ファイル等々) があり，シーンに合わせて (例えば「授業」「卒論」など) フォルダで管理することになる。

さらに，PC 環境には作業フォルダ (Working Directory)[*8]という概念がある。たとえば R/RStudio を起動・実行しているときに，R が「今どこで」実行されているか，どこを管理場所としているか，を表す概念である。例えばこの作業フォルダの中に `sample.csv` というファイルがあって，それをスクリプト上から読み込みたい，というコマンドを実行するのであれば，そのままファイル名を書けば良い。しかし別の場所にそのファイルが保存されているのなら，作業フォルダから見た相対的な位置を含めて指示してやるか (相対パス)，あるいは PC 環境全体からみた絶対的な位置を含めて (絶対パス) 指示してやる必要がある。相対・絶対パスの違いは，「ここから二つ目の角を右」のように指示するか，住所で指示するかの違いであると考えれば良い。

ともあれ，この作業フォルダがどこに設定されているかは，実行するときに常に気にしていなければならない。ちなみにこの作業フォルダは，RStudio のファイルペイン・Files タブでひらいているところとは**限らない**ことに注意してほしい。GUI 上でエクスプローラ/Finder で開いたからといって，作業フォルダが自動的に切り替わるようにはなっていない。

そこで RStudio のプロジェクトである。RStudio には「プロジェクト」という概念があり，作業フォルダや環境の設定などをそこで管理することができる。新しくプロジェクトを始めるときは File>New Project，すでに一度プロジェクトを作っているときは File > Open Project としてプロジェクトファイル (拡張子が.proj のファイル) を開くようにする。そうすると，作業フォルダが当該フォルダに設定される。プロジェクトを Git に連携しておくとバージョン管理などもフォルダ単位で行える。

以後，本講義で外部ファイルを参照する場合，プロジェクトフォルダの中にそのファイルがあるものとして (パスを必要としない形で) 論じるので注意されたし。

[*8] ここでは，フォルダとディレクトリは同じ意味であると思ってもらって良い。一般に，CUI ではディレクトリ，GUI ではフォルダという用語が好まれる。語幹 direct にあるように，ファイルやアクセス先など具体的な指し示す先を強調しているのがディレクトリであり，それにファイル群などまとまった容れもの，という意味を付加したのがフォルダである。フォルダの方が言葉としてわかりやすいし。

1.5 課題

- R の最新版を CRAN からダウンロードし，自分の PC にインストールしてください。
- RStudio の Desktop 版を Posit 社のサイトからダウンロードし，自分の PC にインストールしてください。
- RStido を起動し，ペインレイアウトをデフォルトではない状態に並べ直してみてください。ソースペインを 3 列にするのも良いでしょう。
- コンソールペインに書かれている文字を全て消去してみてください。
- ファイルペインにある Files タブをつかって，色々なフォルダを開けてみたり，不要なファイルを削除したり，ファイル名を変更したりしてみてください。
- ファイルペインにある Files タブを開き，`More` のところから `Go To Working Directory` を選択・実行してください。何か起こったでしょうか。
- この授業のために，新しいプロジェクトを作成してください。プロジェクトは新しいフォルダでも，既存のフォルダでも構いません。
- プロジェクトが開いた状態のとき，RStudio のウィンドウ・タブのどこかに「プロジェクト名」が表示されているはずです。確認してください。
- またファイルペインの Files タブから, 色々なファイル操作をした上で, 改めて `Go To Working Directory` をしてください。プロジェクトフォルダの中に戻ってこれたら成功です。
- 新しい R スクリプトファイルを開き，空白のままで結構ですからファイル名をつけて保存してください。
- RStudio を終了あるいは最小化させ，OS のエクスプローラ/Finder から，プロジェクトフォルダに移動してください。先ほど作ったファイルが保存されていることを確認してください。
- プロジェクトフォルダには，プロジェクト名 +`.proj` というファイルが存在するはずです。これを開いて，RStudio のプロジェクトを開いてください。
- RStudio の File > Close Project からプロジェクトを閉じてください。画面の細部でどこが変わったか，確認してください。
- RStudio を終了し，再び RStudio を起動してください。起動の方法はプロジェクトファイルからでも，アプリケーションの起動でも構いません。起動後に，プロジェクトを開いてください (あるいはプロジェクトが開かれていることを確認してください。)。

第 2 章

R の基礎

ここから実際に R/RStudio をつかった演習に入る。前回すでに言及したように，この講義用のプロジェクトを準備し，RStudio はプロジェクトが開かれた状態であることを前提に話を進める。

2.1 R で計算

まずは R を使った計算である。R スクリプトファイルを開き，最初の行に次の 4 行を入力してみよう。各行を実行 (Run ボタン，あるいは ctrl+enter) し，コンソールの結果を確認しよう。

```
1 + 2
```

```
[1] 3
```

```
2 - 3
```

```
[1] -1
```

```
3 * 4
```

```
[1] 12
```

```
6 / 3
```

```
[1] 2
```

それぞれ加減乗除の計算結果が正しく出ていることを確認してほしい。なお，出力のところに [1] とあるのは，R がベクトルを演算の基本としているからで，回答ベクトルの第 1 要素を返していることを意味する。

四則演算の他に，次のような演算も可能である。

```
# 整数の割り算
8 %/% 3
```

```
[1] 2
```

```
# 余り
7 %% 3
```

```
[1] 1
```

```
# 冪乗
2^3
```

```
[1] 8
```

ここで，#から始まる行は**コメントアウト**されたものとして，実際にコンソールに送られても計算されないことに注意しよう。スクリプトが単純なものである場合はコメントをつける必要はないが，複雑な計算になったり，他者と共有するときは「今どのような演算をしているか」を逐一解説するようにすると便利である。

実践上のテクニックとして，複数行を一括でコメントアウトしたり，アンコメント (コメントアウトを解除する) したりすることがある。スクリプトを複数行選択した上で，Code メニューから `Comment/Uncomment Lines` を押すとコメント/アンコメントを切り替えられるので試してみよう。また，ショートカットキーも確認し，キーからコメント/アンコメントができるように慣れておくと良い (Ctrl+ ↑ +C/Cmd+ ↑ +C)。

One more tip. コメントではなく，大きな段落的な区切り (セクション区切り) が欲しいこともあるかもしれない。Code メニューの一番上に「Insert Section」があるのでこれを選んでみよう。ショートカットキーから入力しても良い (Ctrl+ ↑ +R/Cmd+ ↑ +R)。セクション名を入力するボックスに適当な命名をすると，スクリプトにセクションが挿入される。次に示すのがセクションの例である。

```
# 計算 ----------------------------------------------------------------
```

これはもちろん実行に影響を与えないが，ソースが長くなった場合はこのセクション単位で移動したり (スクリプトペインの左下)，アウトラインを確認したり (スクリプトペインの右上にある横三本線) できるので，活用して欲しい。

2.2　オブジェクト

R では変数，関数などあらゆるものを**オブジェクト**としてあつかう。オブジェクトには任意の名前をつけることができる (数字から始まる名前は不可)。オブジェクトを作り，そこにある値を**代入**する例は次の通りである。

```
a <- 1
b <- 2
A <- 3
a + b # 1 + 2におなじ
```

```
[1] 3
```

```
A + b # 3 + 2におなじ
```

```
[1] 5
```

ここでは数字をオブジェクトに保管し，オブジェクトを使って計算をしている。大文字と小文字が区別されてるため，計算結果が異なることに注意。

代入に使った記号<-は「小なり」と「ハイフン」であるが，左矢印のイメージである。次のように，=や->を使うこともできる。

```
B <- 5
7 -> A
```

ここで，二行目に 7 -> A を行った。先ほど A <- 3 としたが，その後に A には 7 を代入し直したので，値は上書きされる。

```
A + b # 7 + 2におなじ
```

```
[1] 9
```

このように，オブジェクトに代入を重ねると，警告などなしに上書きされることに注意して欲しい。似たようなオブジェクト名を使い回していると，本来意図していたものと違う値・状態を保管していることになりかねないからである。

ちなみに，オブジェクトの中身を確認するためには，そのままオブジェクト名を入力すれば良い。より丁寧には，print 関数を使う。

```
a
```

```
[1] 1
```

```
print(A)
```

```
[1] 7
```

あるいは，RStudio の Environment タブをみると，現在 R が保持しているオブジェクトが確認でき，単一の値の場合は Value セクションにオブジェクト名と値を見ることができる。

注意点として，オブジェクト名として，次の名前は使うことができない。> break, else, for, if, in, next, function, repeat, return, while, TRUE, FALSE.

これらは R で特別な意味を持つ**予約語**と呼ぶ。特に TRUE と FALSE は真・偽を表すもので，大文字の T,F でも代用できるため，この一文字だけをオブジェクト名にするのは避けた方が良い。また，T と F は予約語ではないため，オブジェクト名として使用可能だが，混乱を避けるため使用は推奨されない。

2.3 関数

関数は一般に $y = f(x)$ と表されるが，要するに x を与えると y に形が変わる作用のことを指す。プログラミング言語では一般に，x を**引数 (ひきすう,argument)**，y を**戻り値 (もどりち,value)** という。以下，関数の使用例を挙げる。

```
sqrt(16)
```

```
[1] 4
```

```
help("sqrt")
```

最初の例は平方根 square root を取る関数 `sqrt` であり，引数として数字を与えるとその平方根が返される。第二の例は関数の説明を表示させる関数 `help` であり，これを実行するとヘルプペインに関数の説明が表示される。

2.4　変数の種類

先ほどの `help` 関数に与えた引数`"sqrt"`は文字列である。文字列であることを明示するためにダブルクォーテーション (") で囲っている (シングルクォーテーションで囲っても良い)。このように，R が扱う変数は数字だけではない。変数の種類は数値型 (numeric)，文字型 (character)，論理値 (logical) の 3 種類がある。

```
obj1 <- 1.5
obj2 <- "Hello"
obj3 <- TRUE
```

数値型は整数 (integer)，実数 (double) を含み[*1]，そのほか，複素数型 (complex)，欠測値を表す `NA`，非数値を表す `NaN`(Not a Number)，無限大を表す `Inf` などがある。

文字型はすでに説明した通りで，対になるクォーテーションが必要であることに注意してほしい。終わりを表すクォーテーションがなければ，R は続く数字や文字も含めた「語」として処理する。この場合，enter キーを押しても文字入力が閉じられていないため，コンソールには「+」の表示が出る (この記号は前の行から入力が続いており，プロンプト状態ではないことを表している)。

また，文字型は当然のことながら四則演算の対象にならない。ただし，論理型の `TRUE/FALSE` はそれぞれ 1,0 に対応しているため，計算結果が表示される。次のコードを実行してこのことを確認しよう。

```
obj1 + obj2
obj1 + obj3
```

2.5　オブジェクトの型

ここまでみてきたように，数値や文字など (まとめて**リテラル**という) にも種類があるが，これをストックしておくものは全て**オブジェクト**である。オブジェクトとは変数のこと，と理解しても良いが，関数もオブジェクトに含まれる。

[*1] 実数は real number じゃないのか，という指摘もあろうかとおもう。ここでは電子計算機上の数値の分類である，倍精度浮動小数点数 (double-precision floating-point number) の意味である。倍精度とは単精度の倍を意味しており，単精度は 32 ビットを，倍精度は 64 ビットを単位として一つの数字を表す仕組みのことである。

2.5.1 ベクトル

R のオブジェクトは単一の値しか持たないものではない。むしろ，複数の要素をセットで持つことができるのが特徴である。次に示すのは，**ベクトル**オブジェクトの例である。

```
vec1 <- c(2, 4, 5)
vec2 <- 1:3
vec3 <- 7:5
vec4 <- seq(from = 1, to = 7, by = 2)
vec5 <- c(vec2, vec3)
```

それぞれのオブジェクトの中身を確認しよう。最初の `c()` は結合 combine 関数である。また，コロン (`:`) は連続する数値を与える。`seq` 関数は複数の引数を取るが，初期値，終了値，その間隔を指定した連続的なベクトルを生成する関数である。

ベクトルの計算は要素ごとに行われる。次のコードを実行し，どのように振る舞うか確認しよう。

```
vec1 + vec2
```

```
[1] 3 6 8
```

```
vec3 * 2
```

```
[1] 14 12 10
```

```
vec1 + vec5
```

```
[1]  3  6  8  9 10 10
```

最後の計算でエラーが出なかったことに注目しよう。たとえば `vec1 + vec4` はエラーになるが，ここでは計算結果が示されている (=エラーにはなっていない)。数学的には，長さの違うベクトルは計算が定義されていないのだが，`vec1` の長さは 3，`vec5` の長さは 6 であった。**R はベクトルを再利用する**ので，長いベクトルが短いベクトルの定数倍になるときは反復して利用される。すなわち，ここでは

$$(2,4,5,2,4,5)+(1,2,3,7,6,5)=(3,6,8,9,10,10)$$

の計算がなされた。この R の仕様については，意図せぬ挙動にならぬよう注意しよう。

ベクトルの要素にアクセスするときは大括弧 (`[ ]`) を利用する。特に第二・第三行目のコードの使い方を確認しておこう。大括弧の中は，要素番号でも良いし，真/偽の判断でも良いのである。この真偽判断による指定の方法は，条件節 (`if` 文) をつかって要素を指定できるため，有用である。

```
vec1[2]
```

```
[1] 4
```

```
vec2[c(1, 3)]
```

```
[1] 1 3
```

```
vec2[c(TRUE, FALSE, TRUE)]
```

```
[1] 1 3
```

ここまで，ベクトルの要素は数値で説明してきたが，文字列などもベクトルとして利用できる。

```
words1 <- c("Hello!", "Mr.", "Monkey", "Magic", "Orchestra")
words1[3]
```

```
[1] "Monkey"
```

```
words2 <- LETTERS[1:10]
words2[8]
```

```
[1] "H"
```

ここで LETTERS はアルファベット 26 文字が含まれている予約語ベクトルである。

ベクトルを引数に取る関数も多い。たとえば記述統計量である，平均，分散，標準偏差，合計などは，次のようにして計算する。

```
dat <- c(12, 18, 23, 35, 22)
mean(dat) # 平均
```

```
[1] 22
```

```
var(dat) # 分散
```

```
[1] 71.5
```

```
sd(dat) # 標準偏差
```

```
[1] 8.455767
```

```
sum(dat) # 合計
```

```
[1] 110
```

他にも最大値 max や最小値 min，中央値 median などの関数が利用可能である。

2.5.2 行列

数学では線型代数でベクトルを扱うが，同時にベクトルが複数並んだ二次元の行列も扱うだろう。R でも行列のように配置したオブジェクトを利用できる。

次のコードで作られる行列 A,B がどのようなものか確認しよう。

```
A <- matrix(1:6, ncol = 2)
B <- matrix(1:6, ncol = 2, byrow = T)
```

行列を作る関数 matrix は，引数として要素，列数 (ncol)，行数 (nrow)，要素配列を行ごとにするかどうかの指定 (byrow) をとる。ここでは要素を 1:6 としており，1 から 6 までの連続する整数をあたえている。ncol で 2 列であることを明示しているので，nrow で行数を指定してやる必要はない。byrow の有無でどのように数字が変わっているかは表示させれば一目瞭然であろう。

与える要素が行数 × 列数に一致しておらず，ベクトルの再利用も不可能な場合はエラーが返ってくる。

また，ベクトルの要素指定のように，行列も大括弧を使って要素を指定することができる。行，列の順に指定し，行だけ，列だけの指定も可能である。

```
A[2, 2]
```

```
[1] 5
```

```
A[1, ]
```

```
[1] 1 4
```

```
A[, 2]
```

```
[1] 4 5 6
```

2.5.3 リスト型

行列はサイズの等しいベクトルのセットであるが，サイズの異なる要素をまとめて一つのオブジェクトとして保管しておきたいときはリスト型をつかう。

```
Obj1 <- list(1:4, matrix(1:6, ncol = 2), 3)
```

このオブジェクトの第一要素 ([[1]]) はベクトル，第二要素は行列，第三要素は要素 1 つのベクトル (スカラー) である。オブジェクトの要素の要素 (ex. 第二要素の行列の 2 行 3 列目の要素) にどのようにアクセスすれば良いか，考えてみよう。

このリストは要素へのアクセスの際に [[1]] など数字が必要だが，要素に名前をつけることで利便性が増す。

```
Obj2 <- list(
  vec1 = 1:5,
  mat1 = matrix(1:10, nrow = 5),
  char1 = "YMO"
)
```

この名前付きリストの要素にアクセスするときは，$記号を用いることができる。

```
Obj2$vec1
```

```
[1] 1 2 3 4 5
```

これを踏まえて，名前付きリストの要素の要素にアクセスするにはどうすれば良いか，考えてみよう。

リスト型はこのように，要素のサイズ・長さを問わないため，いろいろなものを保管しておくことができる。統計関数の結果はリスト型で得られることが多く，そのような場合，リストの要素も長くなりがちである。リストがどのような構造を持っているかを見るために，`str` 関数が利用できる。

```
str(Obj2)
```

```
List of 3
 $ vec1 : int [1:5] 1 2 3 4 5
 $ mat1 : int [1:5, 1:2] 1 2 3 4 5 6 7 8 9 10
 $ char1: chr "YMO"
```

`str` 関数の返す結果と同じものが，RStudio の Environment タブからオブジェクトを見ることでも得られる。また，リストの要素としてリストを持つ，すなわち階層的になることもある。そのような場合，必要としている要素にどのようにアクセスすれば良いか，確認しておこう。

```
Obj3 <- list(Obj1, Second = Obj2)
str(Obj3)
```

```
List of 2
 $       :List of 3
  ..$ : int [1:4] 1 2 3 4
  ..$ : int [1:3, 1:2] 1 2 3 4 5 6
  ..$ : num 3
 $ Second:List of 3
  ..$ vec1 : int [1:5] 1 2 3 4 5
  ..$ mat1 : int [1:5, 1:2] 1 2 3 4 5 6 7 8 9 10
  ..$ char1: chr "YMO"
```

2.5.4 データフレーム型

リスト型は要素のサイズを問わないことはすでに述べた。しかしデータ解析を行うときは得てして，2 次元スプレッドシートのような形式である。すなわち一行に 1 オブザベーション，各列は変数を表すといった具合である。このように矩形かつ，列に変数名を持たせることができる特殊なリスト型を**データフレーム型**という。以下はそのようなオブジェクトの例である。

```
df <- data.frame(
  name = c("Ishino", "Pierre", "Marin"),
  origin = c("Shizuoka", "Shizuoka", "Hokkaido"),
```

```
  height = c(170, 180, 160),
  salary = c(1000, 20, 800)
)
# 内容を表示させる
df
```

```
    name   origin height salary
1 Ishino Shizuoka    170   1000
2 Pierre Shizuoka    180     20
3  Marin Hokkaido    160    800
```

```
# 構造を確認する
str(df)
```

```
'data.frame':   3 obs. of  4 variables:
 $ name  : chr  "Ishino" "Pierre" "Marin"
 $ origin: chr  "Shizuoka" "Shizuoka" "Hokkaido"
 $ height: num  170 180 160
 $ salary: num  1000 20 800
```

ところで，心理統計の初歩として Stevens の尺度水準 (Stevens, 1946) について学んだことと思う。そこでは数値が，その値に許される演算のレベルをもとに，名義，順序，間隔，比率尺度水準という 4 つの段階に分類される。間隔・比率尺度水準の数値は数学的な計算を施しても良いが，順序尺度水準や名義尺度水準の数字はそのような計算が許されない (ex.2 番目に好きな人と 3 番目に好きな人が一緒になっても，1 番好きな人に敵わない。)

R には，こうした尺度水準に対応した数値型がある。間隔・比率尺度水準は計算可能なので `numeric` 型でよいが，名義尺度水準は `factor` 型 (要因型，因子型とも呼ばれる)，順序尺度水準は `ordered.factor` 型と呼ばれるものである。

factor 型の変数の例を挙げる。すでに文字型として入っているものを factor 型として扱うよう変換するためには，`as.factor` 関数が利用できる。

```
df$origin <- as.factor(df$origin)
df$origin
```

```
[1] Shizuoka Shizuoka Hokkaido
Levels: Hokkaido Shizuoka
```

要素を表示させて見ると明らかなように，値としては `Shizuoka,Shizuoka,Hokkaido` の 3 つあるが，レベル (水準) は `Shizuoka,Hokkaido` の 2 つである。このように factor 型にしておくと，カテゴリとして使えて便利である。

次に示すのは順序つき factor 型変数の例である。

```
# 順序付き要因型の例
ratings <- factor(c("低い", "高い", "中程度", "高い", "低い"),
```

```
  levels = c("低い", "中程度", "高い"),
  ordered = TRUE
)
# ratingsの内容と型を確認
print(ratings)
```

```
[1] 低い     高い     中程度 高い     低い
Levels: 低い < 中程度 < 高い
```

集計の際などは factor 型と違わないため，使用例は少ないかもしれない。しかし R は統計モデルを適用する時に，尺度水準に対応した振る舞いをするものがあるので，データの尺度水準を丁寧に設定しておくのも良いだろう。

データフレームの要素へのアクセスは，基本的に変数名を介してのものになるだろう。たとえば先ほどのオブジェクト df の数値変数に統計処理をしたい場合は，次のようにすると良い。

```
mean(df$height)
```

```
[1] 170
```

```
sum(df$salary)
```

```
[1] 1820
```

また，データフレームオブジェクトを一括で要約する関数もある。

```
summary(df)
```

```
       name        origin      height        salary
 Length   :3   Hokkaido:1   Min.   :160   Min.   :  20.0
 N.unique :3   Shizuoka:2   1st Qu.:165   1st Qu.: 410.0
 N.blank  :0                Median :170   Median : 800.0
 Min.nchar:5                Mean   :170   Mean   : 606.7
 Max.nchar:6                3rd Qu.:175   3rd Qu.: 900.0
                            Max.   :180   Max.   :1000.0
```

2.6　外部ファイルの読み込み

解析の実際では，データセットを手入力することはなく，データベースから取り出してくるか，別ファイルから読み込むことが一般的であろう。

統計パッケージの多くは独自のファイル形式を持っており，R にはそれぞれに対応した読み込み関数も用意されているが，ここでは最もプレーンな形でのデータである CSV 形式からの読み込み例を示す。

サンプルデータ Baseball.csv を読み込むことを考える。なおこのデータは UTF-8 形式で保存されている[*2]。これ

[*2] UTF-8 というのは文字コードの一種で，0 と 1 からなる機械のデータを人間語に翻訳するためのコードであり，世界的にもっとも一般

を読み込むには，R がデフォルトで持っている関数 read.csv が使える。

```
dat <- read.csv("Baseball.csv")
head(dat)
```

```
      Year       Name team salary bloodType height weight UniformNum position
1 2011年度 永川　勝浩 Carp  12000       O型    188     97         20     投手
2 2011年度 前田　健太 Carp  12000       A型    182     73         18     投手
3 2011年度 栗原　健太 Carp  12000       O型    183     95          5   内野手
4 2011年度 東出　輝裕 Carp  10000       A型    171     73          2   内野手
5 2011年度   シュルツ Carp   9000      不明    201    100         70     投手
6 2011年度   大竹　寛 Carp   8000       B型    183     90         17     投手
  Games AtBats Hit HR Win Lose Save Hold
1    19     NA  NA NA   1    2    0    0
2    31     NA  NA NA  10   12    0    0
3   144    536 157 17  NA   NA   NA   NA
4   137    543 151  0  NA   NA   NA   NA
5    19     NA  NA NA   0    0    0    9
6     6     NA  NA NA   1    1    0    0
```

```
str(dat)
```

```
'data.frame':   7944 obs. of  17 variables:
 $ Year      : chr  "2011年度" "2011年度" "2011年度" "2011年度" ...
 $ Name      : chr  "永川　勝浩" "前田　健太" "栗原　健太" "東出　輝裕" ...
 $ team      : chr  "Carp" "Carp" "Carp" "Carp" ...
 $ salary    : int  12000 12000 12000 10000 9000 8000 8000 7500 7000 6600 ...
 $ bloodType : chr  "O型" "A型" "O型" "A型" ...
 $ height    : int  188 182 183 171 201 183 177 173 176 188 ...
 $ weight    : int  97 73 95 73 100 90 82 73 80 97 ...
 $ UniformNum: int  20 18 5 2 70 17 31 6 1 43 ...
 $ position  : chr  "投手" "投手" "内野手" "内野手" ...
 $ Games     : int  19 31 144 137 19 6 110 52 52 40 ...
 $ AtBats    : int  NA NA 536 543 NA NA 299 192 44 149 ...
 $ Hit       : int  NA NA 157 151 NA NA 60 41 11 35 ...
 $ HR        : int  NA NA 17 0 NA NA 4 2 0 1 ...
 $ Win       : int  1 10 NA NA 0 1 NA NA NA NA ...
 $ Lose      : int  2 12 NA NA 0 1 NA NA NA NA ...
 $ Save      : int  0 0 NA NA 0 0 NA NA NA NA ...
 $ Hold      : int  0 0 NA NA 9 0 NA NA NA NA ...
```

的な文字コードである。しかし WindowsOS はいまだにデフォルトで Shift-JIS というローカルな文字コードにしているため，このファイルを一度 Windows 機の Excel などで開くと文字化けし，以下の手続が正常に作用しなくなることがよくある。本講義で使う場合は，ダウンロード後に Excel などで開くことなく，直接 R から読み込むようにされたし。

ここで head 関数はデータフレームなどオブジェクトの冒頭部分 (デフォルトでは 6 行分) を表示させるものである。また，str 関数の結果から明らかなように，読み込んだファイルが自動的にデータフレーム型になっている。

ちなみに，サンプルデータにおいて欠測値に該当する箇所には NA の文字が入っていた。read.csv 関数では，欠測値はデフォルトで文字列”NA” としている。しかし，実際は別の文字 (ex. ピリオド) や，特定の値 (ex.9999) の場合もあるだろう。その際は，オプション na.strings で「欠測値として扱う値」を指示すれば良い。

2.7　おまけ；スクリプトの清書

さて，ここまでスクリプトを書いてきたことで，そこそこ長いスクリプトファイルができたことと思う。スクリプトの記述については，もちろん「動けばいい」という考え方もあるが，美しくかけていたほうがなお良いだろう。「美しい」をどのように定義するかは異論あるだろうが，一般に「コード規約」と呼ばれる清書方法がある。ここでは細部まで言及しないが，RStudio の Code メニューから Reformat Code を実行してみよう。スクリプトファイルが綺麗に整ったように見えないだろうか?

美しいコードはデバッグにも役立つ。時折 Reformat することを心がけよう。

2.8　課題

- R を起動し，新しいスクリプトファイルを作成してください。そのファイル内で，2 つの整数を宣言し，足し算を行い，結果をコンソールに表示してください。
- スクリプトに次の計算を書き，実行してください。
  - $\frac{5}{6} + \frac{1}{3}$
  - $9.6 \div 4$
  - $2.3 + \frac{1}{2}$
  - $3 \times (2.2 + \frac{4}{5})$
  - $(-2)^4$
  - $2\sqrt{2} \times \sqrt{3}$
  - $2 \log_e 25$
- R のスクリプトファイル内で，ベクトルを作成してください。ベクトルには 1 から 10 までの整数を格納してください。その後，ベクトルの要素の合計と平均を計算してください。ベクトルを合計する関数は sum，平均は mean です。
- 次の表をリスト型オブジェクト Tbl にしてください。

Name	Pop	Area	Density
Tokyo	1,403	2,194	6,397
Beijing	2,170	16,410	1,323
Seoul	949	605	15,688

- 先ほど作った `Tbl` オブジェクトの，東京 (Tokyo) の面積 (Area) の値を表示させてください (リスト要素へのアクセス)
- `Tbl` オブジェクトの人口 (Pop) 変数の平均を計算してください。
- `Tbl` オブジェクトをデータフレーム型オブジェクト `df2` に変換してください。新たに作り直しても良いですし，`as.data.frame` 関数を使っても良い。
- R のスクリプトを使用して，別のサンプルデータ Baseball2022.csv ファイルを読み込み，データフレーム `dat` に格納してください。ただし，このファイルの欠測値は 999 という数値になっています。
- 読み込んだ `dat` の冒頭の 10 行を表示してみてください。
- 読み込んだ `dat` に `summary` 関数を適用してください。
- このデータセットの変数 `team` は名義尺度水準です。Factor 型にしてください。他にも Factor 型にすべき変数が 2 つありますので，それらも同様に型を変換してください。
- このデータセットの変数の中で，数値データに対して平均，分散，標準偏差，最大値，最小値，中央値をそれぞれ算出してください。
- 課題を記述したスクリプトファイルに対して，Reformat などで整形してください。

第 3 章

R によるデータハンドリング

心理学を始め，データを扱うサイエンスでは，データ収集の計画，実行と，データに基づいた解析結果，それを踏まえてのコミュニケーションとの間に，「データをわかりやすい形に加工し，可視化し，分析する」という手順がある。このデータの加工を**データハンドリング**という。統計といえば「分析」に注目されがちだが，実際にはデータハンドリングと可視化のステップが最も時間を必要とし，重要なプロセスである。

3.1 tidyverse の導入

本講義では tidyverse をつかったデータハンドリングを扱う。tidyverse は，データに対する統一的な設計方針を表す概念でもあり，具体的にはそれを実装したパッケージ名でもある。まずは tidyverse パッケージをインストール (ダウンロード) し，次のコードで R に読み込んでおく。

```
pacman::p_load(tidyverse)
```

Attaching core tidyverse packages, と表示され，複数のパッケージ名にチェックマークが入っていたものが表示されただろう。tidyverse パッケージはこれらの下位パッケージを含むパッケージ群である。これに含まれる dplyr,tidyr パッケージはデータの整形に，readr はファイルの読み込みに，forcats は Factor 型変数の操作に，stringr は文字型変数の操作に，lubridate は日付型変数の操作に，tibble はデータフレーム型オブジェクトの操作に，purrr はデータに適用する関数に，ggplot2 は可視化に特化したパッケージである。

続いて Conflicts についての言及がある。tidyverse パッケージに限らず，パッケージを読み込むと表示されることのあるこの警告は，「関数名の衝突」を意味している。ここまで，R を起動するだけで，sqrt,mean などの関数が利用できた。これは R の基本関数であるが，具体的には base パッケージに含まれた関数である。R は起動時に base などいくつかのパッケージを自動的に読み込んでいるのである。これに別途パッケージを読み込むとき，あとで読み込まれたパッケージが同名の関数を使っていることがある。このとき，関数名は後から読み込んだもので上書きされる。そのことについての警告が表示されているのである。具体的にみると，dplyr::filter() masks stats::filter() とあるのは，最初に読み込んでいた stats パッケージの filter 関数は，(tidyverse パッケージに含まれる)dplyr パッケージのもつ同名の関数で上書きされ，今後はこちらが優先的に利用されるよ，ということを示している。

このような同音異字関数は，関数を特定するときに混乱を招くかもしれない。あるパッケージの関数であること

を明示したい場合は，この警告文にあるように，パッケージ名:: 関数名，という書き方にすると良い。

3.2　パイプ演算子

続いてパイプ演算子について解説する。パイプ演算子は tidyverse パッケージに含まれていた magrittr パッケージで導入されたもので，これによってデータハンドリングの利便性が一気に向上した。そこで R も ver 4.2 からこの演算子を導入し，特段パッケージのインストールを必要としなくとも使えるようになった。この R 本体のパイプ演算子のことを，tidyverse のそれと区別して，ネイティブパイプと呼ぶこともある。

ともあれこのパイプ演算子がいかに優れたものであるかを解説しよう。次のスクリプトは，あるデータセットの標準偏差を計算するものである[*1]。数式で表現すると次の通り。ここで $\bar{x}$ はデータベクトル x の算術平均。

$$v = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

```
dat <- c(10, 13, 15, 12, 14) # データ
M <- mean(dat) # 平均
dev <- dat - M # 平均偏差
pow <- dev^2 # 平均偏差の2乗
variance <- mean(pow) # 平均偏差の2乗の平均が分散
standardDev <- sqrt(variance) # 分散の正の平方根が標準偏差
```

ここでは，標準偏差オブジェクト standardDev を作るまでに平均オブジェクト M，平均偏差ベクトル dev，その 2 乗したもの pow，分散 variance と 4 つものオブジェクトを作って答えに到達している。また，作られるオブジェクトが左側にあり，その右側にどのような演算をしているかが記述されているため，頭の中では「オブジェクトを作る，次の計算で」と読んでいったことだろう。

パイプ演算子はこの思考の流れをそのまま具現化する。パイプ演算子は %>% と書き，左側の演算結果をパイプ演算子の右側に来る関数の第一引数として右側に渡す役目をする。これを踏まえて上のスクリプトを書き直してみよう。ちなみにパイプ演算子はショートカット Ctrl(Cmd)+Shift+M で入力できる。

```
dat <- c(10, 13, 15, 12, 14)
standardDev <- dat %>%
  {
    . - mean(.)
  } %>%
  {
    .^2
  } %>%
  mean() %>%
  sqrt()
```

[*1] もちろん sd(dat) の一行で済む話だが，ここでは説明のために各ステップを書き下している。もっとも，sd 関数で計算されるのは $n-1$ で割った不偏分散の平方根であり，標本標準偏差とは異なるものである。

ここでピリオド (.) は，前の関数から引き継いだもの (プレイスホルダー) であり，二行目は`{dat - mean(dat)}`，すなわち平均偏差の計算を意味している。それを次のパイプで二乗し，平均し，平方根を取っている。平均や平方根を取るときにプレイスホルダーが明示されていないのは，引き受けた引数がどこに入るかが明らかなので省略しているからである。

この例に見るように，パイプ演算子を使うと，データ → 平均偏差$→ 2乗 → 平均 →$平方根，という計算の流れと，スクリプトの流れが一致しているため，理解しやすくなったのではないだろうか。

また，ここでの計算は，次のように書くこともできる。

```
standardDev <- sqrt(mean((dat - mean(dat))^2))
```

この書き方は，関数の中に関数がある入れ子状態になっており，$y = h(g(f(x)))$ のような形式である。これも対応するカッコの内側から読み解いていく必要があり，思考の流れと逆転しているため理解が難しい。パイプ演算子を使うと，`x %>% f() %>% g() %>% h() -> y` のように記述できるため，苦労せずに読むことができる。

以下はこのパイプ演算子を使った記述で進めていくので，この表記法 (およびショートカット) に慣れていこう。

3.3 課題 1. パイプ演算子

- `sqrt`,`mean` 関数が `base` パッケージに含まれることをヘルプで確認してみましょう。どこを見れば良いでしょうか。`filter`,`lag` 関数はどうでしょうか。
- `tidyverse` パッケージを読み込んだことで，`filter` 関数は `dplyr` パッケージのものが優先されることになりました。`dplyr` パッケージの `filter` 関数をヘルプで見てみましょう。
- 上書きされる前の `stats` パッケージの `filter` 関数に関するヘルプを見てみましょう。
- 先ほどのデータを使って，平均値絶対偏差 (MeanAD) および中央絶対偏差 (MAD) をパイプ演算子を使って算出してみましょう。なお平均値絶対偏差，中央値絶対偏差は次のように定義されるものです。また絶対値を計算する R 関数は `abs` です。

$$MeanAD = \frac{1}{n}\sum_{i=1}^{n}|x_i - \bar{x}|$$

$$MAD = median(|x_1 - median(x)|, \cdots, |x_n - median(x)|)$$

3.4 列選択と行選択

ここからは `tidyverse` を使ったより具体的なデータハンドリングについて言及する。まずは特定の列および行だけを抜き出すことを考える。データの一部にのみ処理を加えたい場合に重宝する。

3.4.1 列選択

列選択は `select` 関数である。これは `tidyverse` パッケージ内の `dplyr` パッケージに含まれている。`select` 関数は `MASS` パッケージなど，他のパッケージに同名の関数が含まれることが多いので注意が必要である。

例示のために，R がデフォルトで持つサンプルデータ，iris を用いる。なお，iris データは 150 行あるので，以下ではデータセットの冒頭を表示する head 関数を用いているが，演習の際には head を用いなくても良い。

```
# irisデータの確認
iris %>% head()
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

```
# 一部の変数を抜き出す
iris %>%
  select(Sepal.Length, Species) %>%
  head()
```

```
  Sepal.Length Species
1          5.1  setosa
2          4.9  setosa
3          4.7  setosa
4          4.6  setosa
5          5.0  setosa
6          5.4  setosa
```

逆に，一部の変数を除外したい場合はマイナスをつける。

```
iris %>%
  select(-Species) %>%
  head()
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1          5.1         3.5          1.4         0.2
2          4.9         3.0          1.4         0.2
3          4.7         3.2          1.3         0.2
4          4.6         3.1          1.5         0.2
5          5.0         3.6          1.4         0.2
6          5.4         3.9          1.7         0.4
```

```
# 複数変数の除外
iris %>%
  select(-c(Petal.Length, Petal.Width)) %>%
```

```
  head()
```

```
  Sepal.Length Sepal.Width Species
1          5.1         3.5  setosa
2          4.9         3.0  setosa
3          4.7         3.2  setosa
4          4.6         3.1  setosa
5          5.0         3.6  setosa
6          5.4         3.9  setosa
```

これだけでも便利だが，select 関数は適用時に抜き出す条件を指定してやればよく，そのために便利な以下のような関数がある。

- starts_with()
- ends_with()
- contains()
- matches()

使用例を以下に挙げる。

```
# starts_withで特定の文字から始まる変数を抜き出す
iris %>%
  select(starts_with("Petal")) %>%
  head()
```

```
  Petal.Length Petal.Width
1          1.4         0.2
2          1.4         0.2
3          1.3         0.2
4          1.5         0.2
5          1.4         0.2
6          1.7         0.4
```

```
# ends_withで特定の文字で終わる変数を抜き出す
iris %>%
  select(ends_with("Length")) %>%
  head()
```

```
  Sepal.Length Petal.Length
1          5.1          1.4
2          4.9          1.4
3          4.7          1.3
4          4.6          1.5
5          5.0          1.4
```

```
6          5.4          1.7
```

```
# containsで部分一致する変数を取り出す
iris %>%
  select(contains("etal")) %>%
  head()
```

```
  Petal.Length Petal.Width
1          1.4         0.2
2          1.4         0.2
3          1.3         0.2
4          1.5         0.2
5          1.4         0.2
6          1.7         0.4
```

```
# matchesで正規表現による選択をする
iris %>%
  select(matches(".t.")) %>%
  head()
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1          5.1         3.5          1.4         0.2
2          4.9         3.0          1.4         0.2
3          4.7         3.2          1.3         0.2
4          4.6         3.1          1.5         0.2
5          5.0         3.6          1.4         0.2
6          5.4         3.9          1.7         0.4
```

ここで触れた**正規表現**とは，文字列を特定するためのパターンを指定する表記ルールであり，R 言語に限らずプログラミング言語一般で用いられるものである。書誌検索などでも用いられることがあり，任意の文字列や先頭・末尾の語などを記号 (メタ文字) を使って表現するものである。詳しくは正規表現で検索すると良い (たとえばこちらのサイトなどがわかりやすい。)

3.4.2　行選択

一般にデータフレームは列に変数が並んでいるので，`select` 関数による列選択とは変数選択とも言える。これに対し，行方向にはオブザベーションが並んでいるので，行選択とはオブザベーション (ケース，個体) の選択である。行選択には `dplyr` の `filter` 関数を使う。

```
# Sepal.Length変数が6以上のケースを抜き出す
iris %>%
  filter(Sepal.Length > 6) %>%
  head()
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1          7.0         3.2          4.7         1.4 versicolor
2          6.4         3.2          4.5         1.5 versicolor
3          6.9         3.1          4.9         1.5 versicolor
4          6.5         2.8          4.6         1.5 versicolor
5          6.3         3.3          4.7         1.6 versicolor
6          6.6         2.9          4.6         1.3 versicolor
```

```
# 特定の種別だけ抜き出す
iris %>%
  filter(Species == "versicolor") %>%
  head()
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1          7.0         3.2          4.7         1.4 versicolor
2          6.4         3.2          4.5         1.5 versicolor
3          6.9         3.1          4.9         1.5 versicolor
4          5.5         2.3          4.0         1.3 versicolor
5          6.5         2.8          4.6         1.5 versicolor
6          5.7         2.8          4.5         1.3 versicolor
```

```
# 複数指定の例
iris %>%
  filter(Species != "versicolor", Sepal.Length > 6) %>%
  head()
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
1          6.3         3.3          6.0         2.5 virginica
2          7.1         3.0          5.9         2.1 virginica
3          6.3         2.9          5.6         1.8 virginica
4          6.5         3.0          5.8         2.2 virginica
5          7.6         3.0          6.6         2.1 virginica
6          7.3         2.9          6.3         1.8 virginica
```

ここで==とあるのは一致しているかどうかの判別をするための演算子である。=ひとつだと「オブジェクトへの代入」と同じになるので，判別条件の時には重ねて表記する。同様に，!=とあるのは not equal，つまり不一致のとき真になる演算子である。

3.5　変数を作る・再割り当てする

既存の変数から別の変数を作る，あるいは値の再割り当ては，データハンドリング時に最もよく行う操作のひとつである。たとえば連続変数をある値を境に「高群・低群」というカテゴリカルな変数に作り変えたり，単位を変換するために線型変換したりすることがあるだろう。このように，変数を操作するときに「既存の変数を加工して

特徴量を作りだす」というときの操作は，基本的に dplyr の mutate 関数を用いる。次の例をみてみよう。

```
mutate(iris, Twice = Sepal.Length * 2) %>% head()
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species Twice
1          5.1         3.5          1.4         0.2  setosa  10.2
2          4.9         3.0          1.4         0.2  setosa   9.8
3          4.7         3.2          1.3         0.2  setosa   9.4
4          4.6         3.1          1.5         0.2  setosa   9.2
5          5.0         3.6          1.4         0.2  setosa  10.0
6          5.4         3.9          1.7         0.4  setosa  10.8
```

新しく Twice 変数ができたのが確認できるだろう。この関数はパイプ演算子の中で使うことができる (というかその方が主な使い方である)。次の例は，Sepal.Length 変数を高群と低群の 2 群に分けるものである。

```
iris %>%
  select(Sepal.Length) %>%
  mutate(Sepal.HL = ifelse(Sepal.Length > mean(Sepal.Length), 1, 2)) %>%
  mutate(Sepal.HL = factor(Sepal.HL, label = c("High", "Low"))) %>%
  head()
```

```
  Sepal.Length Sepal.HL
1          5.1      Low
2          4.9      Low
3          4.7      Low
4          4.6      Low
5          5.0      Low
6          5.4      Low
```

ここでもちいた ifelse 関数は，**if(条件判断，真のときの処理，偽のときの処理)** という形でもちいる条件分岐関数であり，ここでは平均より大きければ 1，そうでなければ 2 を返すようになっている。mutate 関数でこの結果を Sepal.HL 変数に代入 (生成) し，次の mutate 関数では今作った Sepal.HL 変数を Factor 型に変換して，その結果をまた Sepal.HL 変数に代入 (上書き) している。このように，変数の生成先を生成元と同じにしておくと上書きされるため，たとえば変数の型変換 (文字型から数値型へ，数値型から Factor 型へ，など) にも用いることができる。

3.6　課題 2.select, filter, mutate

- Baseball.csv を読み込んで、データフレーム df に代入しましょう。
- df には複数の変数が含まれています。変数名の一覧は names 関数で確認できます。df オブジェクトに含まれる変数名を確認しましょう。
- df には多くの変数がありますが、必要なのは年度 (Year)、選手名 (Name)、所属球団 (team)、身長 (height)、体重 (weight)、年俸 (salary)、守備位置 (position) だけです。これらの変数だけを選択して、df2 オブジェ

クトを作成しましょう。

- df2 には数年分のデータが含まれています。2020 **年度**のデータだけを分析したいので、選別してみましょう。
- 同じく、2020 **年度**の**阪神タイガース**に関するデータだけを選別してみましょう。
- 同じく、2020 **年度**の**阪神タイガース**以外のデータセットはどのようにして選別できるでしょうか。
- 選手の身体的特徴を表す BMI 変数を作成しましょう。なお、BMI は体重 (kg) を身長 (m) の二乗で除したものです。変数 height の単位が cm であることに注意しましょう。
- 投手と野手を区別する新しい変数 position2 を作成しましょう。これは Factor 型にします。なお、野手は投手でないもの、すなわち内野手、外野手、捕手のいずれかです。
- 日本プロ野球界は大きく分けてセリーグ (Central League) とパリーグ (Pacific League) に分かれています。セリーグに所属する球団は Giants, Carp, Tigers, Swallows, Dragons, DeNA であり、パ・リーグはそれ以外です。df2 を加工して、所属するリーグの変数 League を作成しましょう。この変数も Factor 型にしておきましょう。
- 変数 Year は語尾に「年度」という文字が入っているため文字列型になっています。実際に使うときは不便なので、「年度」という文字を除外し、数値型変数に変換しましょう。

3.7 ロング型とワイド型

ここまでみてきたデータは行列の 2 次元に，ケース × 変数の形で格納されていた。この形式は，人間が見て管理するときにわかりやすい形式をしているが，計算機にとっては必ずしもそうではない。たとえば「神エクセル」と揶揄されることがあるように，稀に表計算ソフトを方眼紙ソフトあるいは原稿用紙ソフトと勘違いしたかのような使い方がなされる場合がある。人間にとってはわかりやすい (見て把握しやすい) かもしれないが，計算機にとって構造が把握できないため，データ解析に不向きである。巷には，こうした分析しにくい電子データがまだまだたくさん存在する。

これをうけて 2020 年 12 月，総務省により機械判読可能なデータの表記方法の統一ルールが策定された (総務省, 2020)。それには次のようなチェック項目が含まれている。

- ファイル形式は Excel か CSV となっているか
- 1 セル 1 データとなっているか
- 数値データは数値属性とし，文字列を含まないこと
- セルの結合をしていないか
- スペースや改行等で体裁を整えていないか
- 項目名を省略していないか
- 数式を使用している場合は，数値データに修正しているか
- オブジェクトを使用していないか
- データの単位を記載しているか
- 機種依存文字を使用していないか
- データが分断されていないか
- 1 シートに複数の表が掲載されていないか

データの入力の基本は，**1 行に 1 ケースの情報が入っている，過不足のない 1 つのデータセットを作ること**とい

えるだろう。

同様に，計算機にとって分析しやすいデータの形について，Wickham (2014) が提唱したのが**整然データ (Tidy Data)** という考え方である。整然データとは，次の 4 つの特徴を持ったデータ形式のことを指す。

- 個々の変数 (variable) が 1 つの列 (column) をなす。
- 個々の観測 (observation) が 1 つの行 (row) をなす。
- 個々の観測の構成単位の類型 (type of observational unit) が 1 つの表 (table) をなす。
- 個々の値 (value) が 1 つのセル (cell) をなす。

この形式のデータであれば，計算機が変数と値の対応構造を把握しやすく，分析しやすいデータになる。データハンドリングの目的は，混乱している雑多なデータを，利用しやすい整然データの形に整えることであると言っても過言ではない。さて，ここでよく考えてみると，変数名も一つの変数だと考えることに気づく。一般に，行列型のデータは次のような書式になっている。

Table3.1: ワイド型データ

	午前	午後	夕方	深夜
東京	晴	晴	雨	雨
大阪	晴	曇	晴	晴
福岡	晴	曇	曇	雨

ここで，たとえば大阪の夕方の天気を見ようとすると「晴れ」であることは明らかだが，この時の視線の動きは大阪行の，夕方列，という参照の仕方である。言い方を変えると，大阪・夕方の「晴れ」を参照するときに，行と列の両方のラベルを参照する必要がある。

ここで同じデータを次のように並べ替えてみよう。

Table3.2: ロング型データ

地域	時間帯	天候
東京	午前	晴
東京	午後	晴
東京	夕方	雨
東京	深夜	雨
大阪	午前	晴
大阪	午後	曇
大阪	夕方	晴
大阪	深夜	晴
福岡	午前	晴
福岡	午後	曇
福岡	夕方	曇

地域	時間帯	天候
福岡	深夜	雨

このデータが表す情報は同じだが，大阪・夕方の条件を絞り込むことは行選択だけでよく，計算機にとって使いやすい。この形式をロング型データ，あるいは「縦持ち」データという。これに対して前者の形式をワイド型データ，あるいは「横持ち」データという。

ロング型データにする利点のひとつは，欠測値の扱いである。ワイド型データで欠測値が含まれる場合，その行あるいは列全体を削除するのは無駄が多く，かと言って行・列両方を特定するのは技術的にも面倒である。これに対しロング型データの場合は，当該行を絞り込んで削除するだけで良い。

tidyverse には (正確には tidyr には)，このようなロング型データ，ワイド型データの変換関数が用意されている。実例とともに見てみよう。まずはワイド型データをロング型に変換する pivot_longer である。

```
iris %>% pivot_longer(-Species)
```

```
# A tibble: 600 x 3
   Species name         value
   <fct>   <chr>        <dbl>
 1 setosa  Sepal.Length   5.1
 2 setosa  Sepal.Width    3.5
 3 setosa  Petal.Length   1.4
 4 setosa  Petal.Width    0.2
 5 setosa  Sepal.Length   4.9
 6 setosa  Sepal.Width    3
 7 setosa  Petal.Length   1.4
 8 setosa  Petal.Width    0.2
 9 setosa  Sepal.Length   4.7
10 setosa  Sepal.Width    3.2
# i 590 more rows
```

ここでは元の iris データについて，Species セルを軸として，それ以外の変数名と値を name,value に割り当てて縦持ちにしている。

逆に，ロング型のデータをワイド型に持ち替えるには，pivot_wider を使う。実例は以下の通りである。

```
iris %>%
  select(-Species) %>%
  rowid_to_column("ID") %>%
  pivot_longer(-ID) %>%
  pivot_wider(id_cols = ID, names_from = name, values_from = value)
```

```
# A tibble: 150 x 5
```

```
      ID Sepal.Length Sepal.Width Petal.Length Petal.Width
   <int>        <dbl>       <dbl>        <dbl>       <dbl>
 1     1          5.1         3.5          1.4         0.2
 2     2          4.9         3            1.4         0.2
 3     3          4.7         3.2          1.3         0.2
 4     4          4.6         3.1          1.5         0.2
 5     5          5           3.6          1.4         0.2
 6     6          5.4         3.9          1.7         0.4
 7     7          4.6         3.4          1.4         0.3
 8     8          5           3.4          1.5         0.2
 9     9          4.4         2.9          1.4         0.2
10    10          4.9         3.1          1.5         0.1
# i 140 more rows
```

今回は Species 変数を除外し，別途 ID 変数として行番号を変数に付与した。この行番号をキーに，変数名は names 列から，その値は value 列から持ってくることでロング型をワイド型に変えている[*2]。

3.8　グループ化と要約統計量

データをロング型にすることで，変数やケースの絞り込みが容易になる。その上で，ある群ごとに要約した統計量を算出したい場合は，group_by 変数によるグループ化と，summarise あるいは reframe がある。実例を通して確認しよう。

```
iris %>% group_by(Species)
```

```
# A tibble: 150 x 5
# Groups:   Species [3]
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <fct>
 1          5.1         3.5          1.4         0.2 setosa
 2          4.9         3            1.4         0.2 setosa
 3          4.7         3.2          1.3         0.2 setosa
 4          4.6         3.1          1.5         0.2 setosa
 5          5           3.6          1.4         0.2 setosa
 6          5.4         3.9          1.7         0.4 setosa
 7          4.6         3.4          1.4         0.3 setosa
 8          5           3.4          1.5         0.2 setosa
 9          4.4         2.9          1.4         0.2 setosa
```

[*2] Species 変数を除外したのは，これをキーにしたロング型をワイド型に変えることができない (Species は 3 水準しかない) からで，個体を識別する ID が別途必要だったからである。Species 情報が欠落することになったが，これはロング型データの value 列が char 型と double 型の両方を同時に持てないからである。この問題を回避するためには，Factor 型のデータを as.numeric() 関数で数値化することなどが考えられる。

```
10          4.9         3.1          1.5         0.1 setosa
# i 140 more rows
```

上のコードでは，一見したところ表示されたデータに違いがないように見えるが，出力時に Species[3] と表示されていることがわかる。ここで，Species 変数の 3 水準で群分けされていることが示されている。これを踏まえて，summarise してみよう。

```
iris %>%
  group_by(Species) %>%
  summarise(
    n = n(),
    Mean = mean(Sepal.Length),
    Max = max(Sepal.Length),
    IQR = IQR(Sepal.Length)
  )
```

```
# A tibble: 3 x 5
  Species        n  Mean   Max   IQR
  <fct>      <int> <dbl> <dbl> <dbl>
1 setosa        50  5.01   5.8 0.400
2 versicolor    50  5.94   7   0.7
3 virginica     50  6.59   7.9 0.675
```

ここではケース数 (n)，平均 (mean)，最大値 (max)，四分位範囲 (IQR)[*3]を算出した。

また，ここでは Sepal.Length についてのみ算出したが，他の数値型変数に対しても同様の計算がしたい場合は，across 関数を使うことができる。

```
iris %>%
  group_by(Species) %>%
  summarise(across(
    c(Sepal.Length, Sepal.Width, Petal.Length),
    ~ mean(.x)
  ))
```

```
# A tibble: 3 x 4
  Species    Sepal.Length Sepal.Width Petal.Length
  <fct>             <dbl>       <dbl>        <dbl>
1 setosa             5.01        3.43         1.46
2 versicolor         5.94        2.77         4.26
3 virginica          6.59        2.97         5.55
```

ここで，~mean(.x) の書き方について言及しておく。チルダ (tilda,~) で始まるこの式を，R では特にラムダ関

[*3] 四分位範囲 (Inter Quartile Range) とは，データを値の順に 4 分割した時の上位 1/4 の値から，上位 3/4 の値を引いた範囲である。

数とかラムダ式と呼ぶ。これはこの場で使う即席関数の作り方である。別の方法として，正式に関数を作る関数 function を使って次のように書くこともできる。

```
iris %>%
  group_by(Species) %>%
  summarise(across(
    c(Sepal.Length, Sepal.Width, Petal.Length),
    function(x) {
      mean(x)
    }
  ))
```

```
# A tibble: 3 x 4
  Species    Sepal.Length Sepal.Width Petal.Length
  <fct>             <dbl>       <dbl>        <dbl>
1 setosa             5.01        3.43         1.46
2 versicolor         5.94        2.77         4.26
3 virginica          6.59        2.97         5.55
```

ラムダ関数や自作関数の作り方については，後ほどあらためて触れるとして，ここでは複数の変数に関数をあてがう方法を確認して欲しい。across 関数で変数を選ぶ際は，select 関数の時に紹介した starts_with なども利用できる。次に示す例は，複数の変数を選択し，かつ，複数の関数を適用する例である。複数の関数を適用するために，ラムダ関数をリストで与えることができる。

```
iris %>%
  group_by(Species) %>%
  summarise(across(starts_with("Sepal"),
    .fns = list(
      M = ~ mean(.x),
      Q1 = ~ quantile(.x, 0.25),
      Q3 = ~ quantile(.x, 0.75)
    )
  ))
```

```
# A tibble: 3 x 7
  Species    Sepal.Length_M Sepal.Length_Q1 Sepal.Length_Q3 Sepal.Width_M
  <fct>               <dbl>           <dbl>           <dbl>         <dbl>
1 setosa               5.01            4.8             5.2          3.43
2 versicolor           5.94            5.6             6.3          2.77
3 virginica            6.59            6.22            6.9          2.97
# i 2 more variables: Sepal.Width_Q1 <dbl>, Sepal.Width_Q3 <dbl>
```

3.9 課題 3. データの整形

- 上で作った `df2` オブジェクトを利用します。環境に `df2` オブジェクトが残っていない場合は、もう一度上の課題に戻って作り直しておきましょう。
- 年度 (Year) でグルーピングし、年度ごとの登録選手数 (データの数)、平均年俸を見てみましょう。
- 年度 (Year) とチーム (team) でグルーピングし、同じく年度ごとの登録選手数 (データの数)、平均年俸を見てみましょう。
- 続いて、一行に 1 年度分、列に各チームと変数の組み合わせが入った、ワイド型データを作りたいと思います。`pivot_wider` を使って上のオブジェクトをワイド型にしてみましょう。
- ワイド型になったデータを、`Year` 変数をキーにして `pivot_longer` でロング型データに変えてみましょう。

第 4 章

R によるレポートの作成

4.1 Rmd/Quarto の使い方

4.1.1 概略

今回は RStudio を使った文書作成法について解説する。皆さんは、これまで作成と言えば、基本的に Microsoft Word のような文書作成ソフトを使ってきたものと思う。また，統計解析といえば R(やその他のソフト)，図表の作成は Excel といったように，用途ごとに異なるアプリケーションを活用するのが一般的であろう。

このやり方は，統計解析の数値結果を表計算ソフトに，そこで作った図表を文書作成ソフトにコピー＆ペーストする，という転記作業が何度も発生する。ここで転記ミス・貼り付け間違いが生じると，当然ながら出来上がる文書は間違ったものになる。こうした転記ミスのことを「コピペ汚染」と呼ぶこともある。

問題はこの環境をまたぐ作業にあるわけで，計算・作図・文書が一つの環境で済めばこうした問題が起こらない。これを解決するのが R Markdown や Quarto という書式・ソフトウェアなのである。

Rmarkdown の `markdown` とは書式の一種である。マークアップ言語と呼ばれる書き方の一種で，なかでも R との連携に特化したのが Rmarkdown である。マークアップ言語とは，言語の中に専門の記号を埋め込み，その書式に対応した読み込みアプリで，表示の際に書式を整える方式のことを指す。有名なマークアップ言語としては，数式に特化した LaTeX や，インターネットウェブサイトで用いられている HTML などがある。Markdown は，特に読みやすさと書きやすさを重視した軽量なマークアップ言語で，テキストエディタで直接編集できる形式である。

Rmarkdown は markdown の書式を踏襲しつつ，R での実行結果を文中に埋め込むコマンドを有している。R のコマンドで計算したり図表を作成しつつ，その結果を埋め込む場所をマークアップ言語で指定する。最終的に文書を閲覧する場合は，マークアップ言語を出力ファイルに変換する (コンパイルする，ニットするという) 必要があり，その時 R での計算が実行される。コンパイルのたびに計算されるので，同じコードでも乱数を使ったコードを書いていたり，一部読み込みファイルを変更するだけで，出力される結果は変わる。しかしコピペ汚染のように，間違った値・図表が含まれるものではないので，研究の再現性にも一役買うことになる。再現可能な文書の作成について、詳しくは高橋（2018）を参考にすると良い。

Quarto は Rmarkdown をさらに拡張させたもので，RStudio を提供している Posit 社が今もっとも注力している

ソフトのひとつである。Rmarkdown は R と markdown の連携であったが，Quarto は R だけでなく，Python や Julia といった他の言語にも対応しているし，これら複数の計算言語の混在も許す。すなわち，一部は R で計算し，その結果を Python で検算して Julia で描画する，といったことを一枚のファイルで書き込むことも可能である。

なおこの授業資料も Quarto で作成している。このように Quarto はプレゼンテーション資料やウェブサイトも作成できるし，出力形式もウェブサイトだけでなく PDF や ePUB(電子書籍の形式) にすることが可能である。なおこの授業の資料もウェブサイトと同時に PDF 形式と，ePUB 形式で出力されている。Quarto について専門の解説書はまだないが，インターネットに充実したドキュメントがあるので検索するといいだろう。まだ新しい技術なので，公式を第一に参照すると良い。

4.1.2　ファイルの作成と knit

Rmarkdown は RStudio との相性がよく，RStudio の File > New File から `R Markdown` を選ぶと Rmarkdown ファイルがサンプルとともに作成される。作成時に文書のタイトル，著者名，作成日時や出力フォーマットが指定できるサブウィンドゥが開き，作成するとサンプルコードが含まれた R markdown ファイルが表示されるだろう。

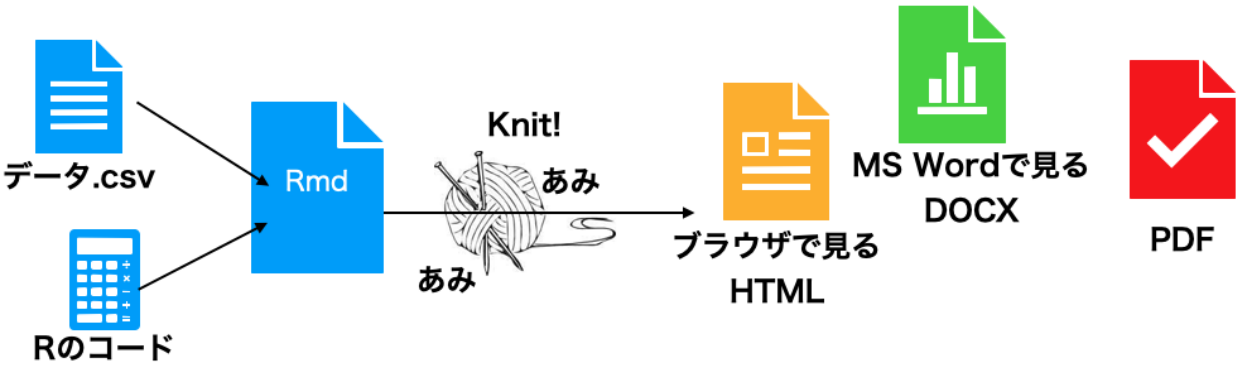


Figure4.1: Rmarkdown を knit するイメージ

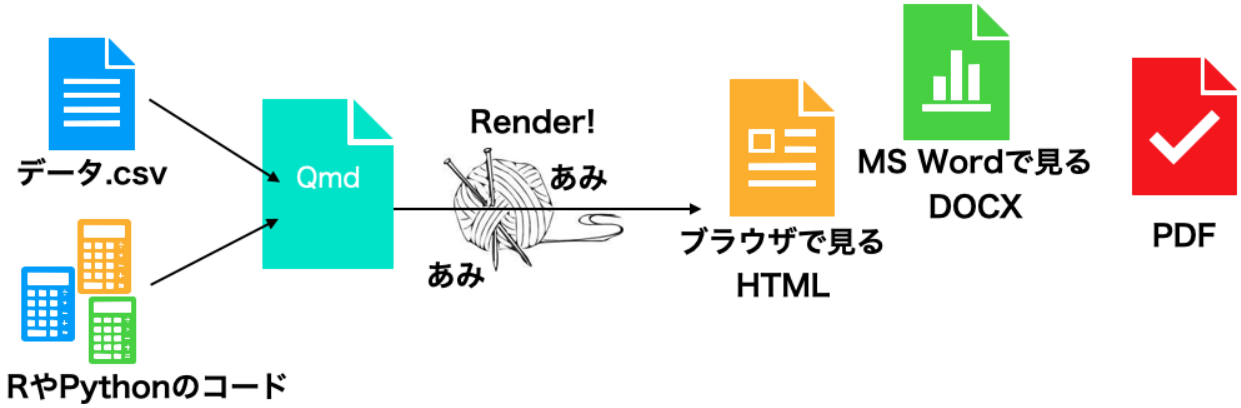


Figure4.2: Quarto ファイルを Rendering するイメージ

Quarto も同様に，RStudio の File > New File から `Quarto Document` を選ぶことで新しいファイル画面が開く。なお Rmarkdown ファイルの拡張子は `Rmd` とすることが一般的であり，Quarto は `Qmd` とすることが一般的である。もっとも，Quarto は RStudio 以外のエディタから利用することも考えられていて，例えば VS Code などの一般的なエディタで作成し，コマンドライン経由でコンパイルすることも可能である。

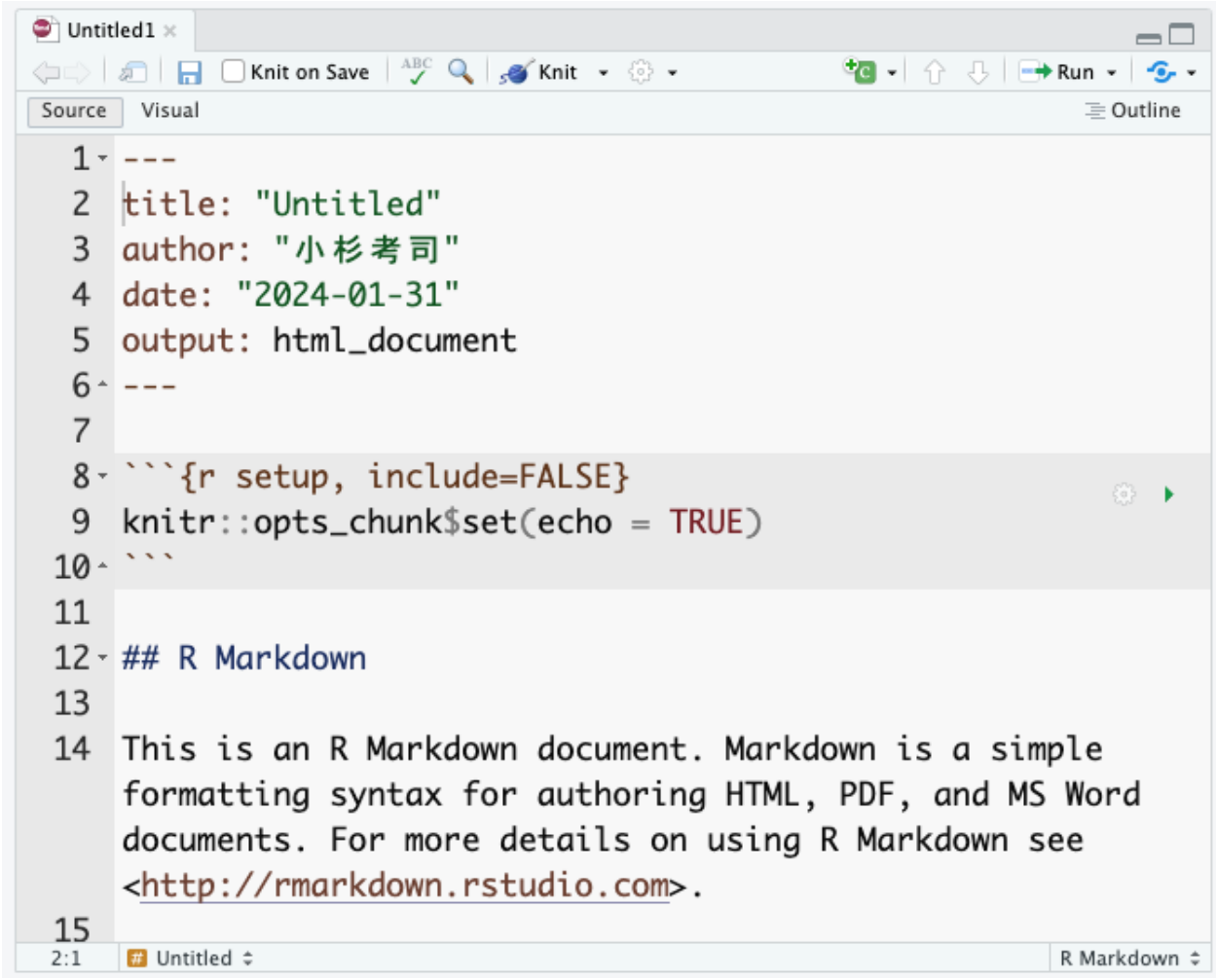


Figure4.3: Rmarkdown ファイルのサンプル

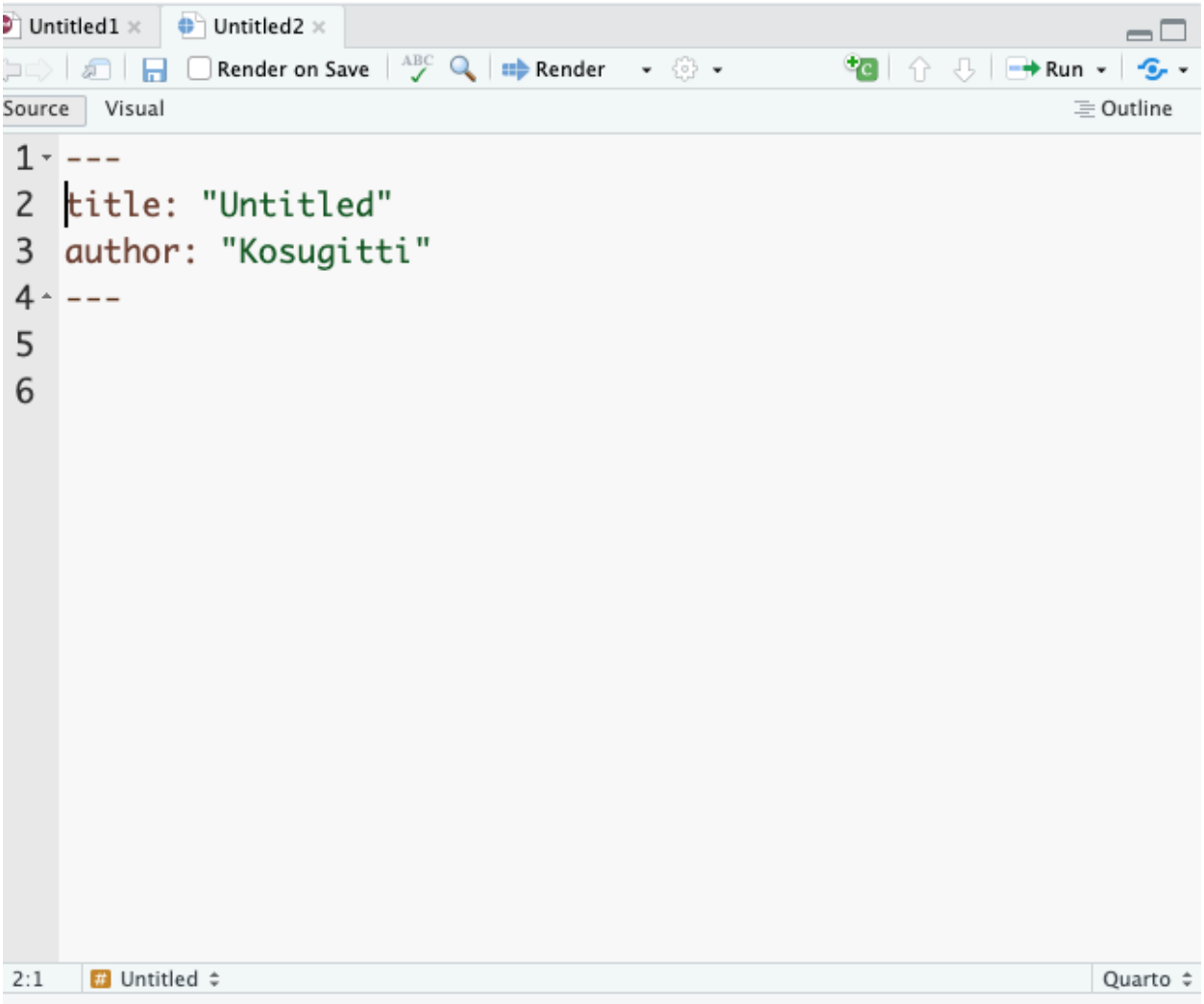


Figure4.4: Quarto ファイルのサンプル

Rmarkdown，Quarto ともに，ファイルの冒頭に 4 つのハイフンで囲まれた領域が見て取れるだろう。これは YAML ヘッダ (YAML は Yet Another Markup Language の略。ここはまだマークアップ領域じゃないよということ) と呼ばれる，文書全体に対する設定をする領域のことである。

この領域を一瞥すると，タイトルや著者名，出力形式などが記載されていることが見て取れる。YAML ヘッダはインデントに敏感で，また正しくない記述が含まれているとエラーになって出力ファイルが作られないことが少なくないため，ここを手動で書き換えるときは注意が必要である。とはいえ，ここを自在に書き換えることができるようになると，様々な応用が効くので興味があるものは調べて色々トライしてもらいたい。

さて，Rmd/Qmd のファイル上部に Knit あるいは Render と書かれたボタンがあるだろう。これをクリックすると，表示用ファイルへの変換が実行される。[*1]Rmarkdown の場合は，すでにサンプルコードが含まれているので，数値および図表のはいった HTML ドキュメントが表示されるだろう。以下はこのサンプルコードを例に説明する

[*1] もし新しく開いているファイルに名前がつけられていないのなら (Untitled のままになっているようであれば), ファイル名の指定画面が開く。また環境によっては，初回実行時にコンパイルに必要な関連パッケージのダウンロードが求められることがある。

ため，Rmarkdown とそのコンパイル (knit)) を一度試してもらいたい。その上で，元の Rmd ファイルと出来上がったファイルとの対応関係を確認してみよう。

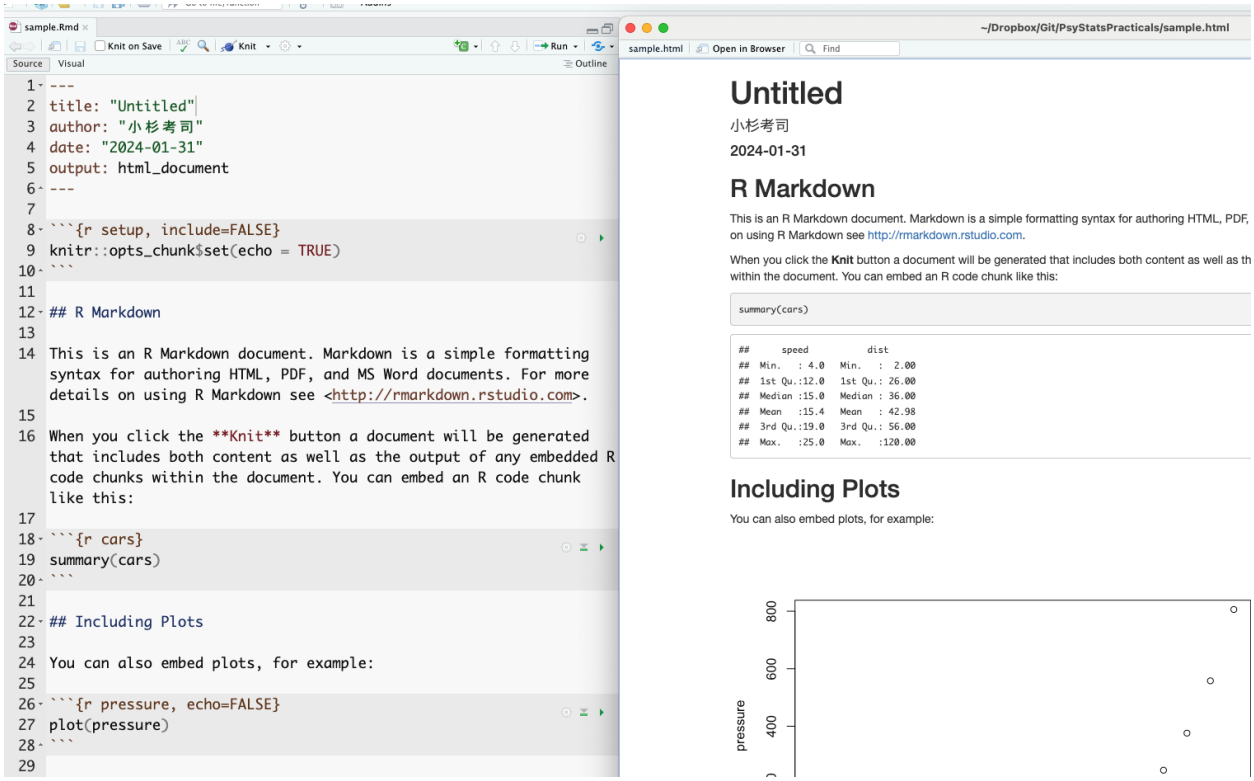


Figure4.5: Rmd ファイルと出力結果の対応

おおよそ，何がどのように変換されているかの対応が推察できるだろう。出力ファイルの冒頭には，YAML で設定したタイトル，著者名，日付などが表示されているし，#の印がついていた一行は見出しとして強調されている。

特に注目したいのが，元のファイルで 3 つのクォーテーションで囲まれた灰色の領域である。この領域のことを特に**チャンク**といい，ここに書かれた R スクリプトが変換時に実行され，結果として出力される。出力ファイルをみると，`summary(cars)` というチャンクで指定された命令文があり，その結果 (cars というデータセットの要約) が出力されているのが見て取れる。繰り返しになるが，ポイントは原稿ファイルには計算を指示するスクリプトが書かれているだけで，出力結果を書いていないことにある。原稿は指示だけなのである。こうすることで，コピー＆ペーストのミスがなくなるし，同じ Rmd/Qmd 原稿とデータを持っていれば，ことなる PC 上でも同じ出力が得られる。環境を統合することで，ミスの防止と再現可能性に貢献しているのがわかるだろう。

今回は `cars` という R がデフォルトでもっているサンプルデータの例なので，どの環境でも同じ結果が出力されている。しかしもちろん，個別のデータファイルであっても，同じファイルで同じ読み込み方，同じ加工をしていれば，環境が違っても追跡可能である。注意して欲しいのは，コンパイルするときは新しい環境から行われるという点ある。すなわち，**原稿ファイルにないオブジェクトの利用はできない**のである。これは再現性を担保するという意味では当然のことで，「事前に別途処理しておいたデータ」から分析を始められても，その事前処理が適切だったかどうかがチェックできないからである。Rmd/Qmd ファイルと，CSV ファイルなどの素データが共有されていれば再現できる，という利点を活かすため，データハンドリングを含めた前処理も全てチャンクに書き込

み，新しい環境で最初からトレースできるようにする必要がある。不便に感じるところもあるかもしれないが，科学的営みとして重要な手続きであることを理解してもらいたい。[*2]

RStudi では，ビジュアルモードやアウトライン表示，チャンク挿入ボタンやチャンクごとの実行・設定など Rmd/Qmd ファイルの編集に便利な機能も複数用意されているので，高橋（2018）などを参考にいろいろ試してみるといいだろう。

4.1.3　マークダウンの記法

以下では，マークダウン記法について基本的な利用法を解説する。

4.1.3.1　見出しと強調

すでに見たように，マークダウンでは#記号で見出しを作ることができる。#の数が見出しレベルに対応し，#はトップレベル，本でいうところの「章,chapter」，HTML でいうところの `H1` に相当する。#記号の後ろに半角スペースが必要なことに注意されたし。以下，##で「節,section」あるいは `H2`，###で小節 (subsection,`H3`)，####で小小節 (subsubsection,`H4`)…と続く。

心理学を始め，科学論文の書き方としての「パラグラフライティング」を既にみしっていることだろう。文章をセクション，サブセクション，パラグラフ，センテンスのように階層的に分割し，それぞれの区分が 4 つの下位区分を含むような文章構造である。心理学の場合は特に「問題，方法，結果，考察」の 4 セクションで一論文が構成されるのが基本である。こうしたアウトラインを意識した書き方は読み手にも優しく，マークダウンの記法ではそれが自然と実装できるようになっている。

これとは別に，一部を太字や斜体で強調したいこともあるだろう。そのような場合はアスタリスクを 1 つ，あるいは 2 つつけて**強調**したり**強調**したりできる。

4.1.3.2　図表とリンク

文中に図表を挿入したいこともあるだろう。表の挿入は，マークダウン独自の記法があり，縦棒|やハイフン-を駆使して以下のように表記する。

```
| Header 1 | Header 2 | Header 3 |
| -------- | -------- | -------- |
| Row 1    | Data 1   | Data 2   |
| Row 2    | Data 3   | Data 4   |
```

R のコードの中には分析結果をマークダウン形式で出力してくれる関数もあるし，表計算ソフトなどでできた表があるなら，chatGPT など生成 AI を利用するとすぐに書式変換してくれるので，そういったツールを活用すると良い。

[*2] もっとも，R のバージョンやパッケージのバージョンによっては同じ計算結果が出ない可能性がある。より本質的な計算過程に違いがあるかもしれないのである。そのため，R 本体やパッケージのバージョンごとパッキングして共有する工夫も考えられている。Docker と呼ばれるシステムは，解析環境ごと保全し共有するシステムの例である。

図の挿入は，マークダウンでは図のファイルへのリンクと考えると良い。次のように，大括弧で括った文字がキャプション，つづく小括弧で括ったものが図へのリンクとなる。実際に表示されるときは図が示される。

![**図のキャプション**]（**図へのリンク**）

同様に，ウェブサイトへのリンクなども，[**表示名**]（**リンク先**）の書式で対応できる。

4.1.3.3 リスト

並列的に箇条書きを示したい場合は，プラスあるいはマイナスでリストアップする。注意すべきは，リストの前後に改行を入れておくべきことである。

ここまで前の文

```
+  list item 1
+  list item 2
+  list item 3
    - sub item 1
    - sub item 2
```

ここから次の文

4.1.3.4 チャンク

既に述べたように，チャンク (chunk) と呼ばれる領域は実行されるコードを記載するところである。チャンクはまず，バックスラッシュ 3 つ繋げることでコードブロックであることを示し，次に `r` と書くことで計算エンジンが R であることを明示する。ここに Juilia や Python など他の計算エンジンを指定することも可能である。

可能であれば，チャンク名をつけておくと良い。次の例は，チャンク名として「chunksample」を与えたものである。チャンク名をつけておくと，RStudio では見出しジャンプをつかって移動することもできるので，編集時に便利である。

“‘{r chunksample,echo = FALSE} summary(cars) “‘

さらに，`echo = FALSE` のようにチャンクオプションを指定することができる。`echo=FALSE` は入力したスクリプトを表示せず，結果だけにするオプションである。そのほか「計算結果を含めない」「表示せずに計算は実行する」等様々な指定が可能である。

なお Quarto ではこのチャンクオプションを次のように書くこともできる。

“‘{r} #| echo: FALSE #| include: FALSE summary(cars) “‘

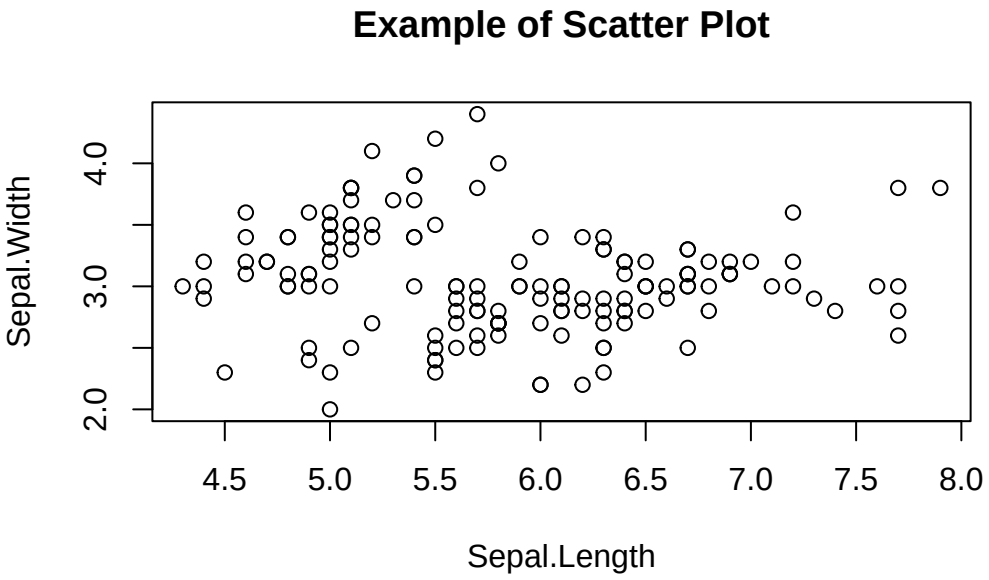
4.2 プロットによる基本的な描画

再現可能な文書という観点から，図表もスクリプトによる記述で表現することは重要である。

データはまず可視化するものと心がけよ。可視化は，数値の羅列あるいはまとめられた統計量では把握しきれない多くの情報を提供し，潜在的な関係性を直観的に見つけ出せる可能性がある。なので，取得したあらゆる**データはまず可視化するもの**，と思っておいて間違いはない。大事なことなので二度言いました。可視化の重要性については心理学の知見にも触れているキーラン・ヒーリー（2021）も参考にしてほしい。

さて，R には基本的な作図環境も整っており，plot という関数に引数として，x 軸，y 軸に相当する変数を与えるだけで，簡単に散布図を書いてくれる。

```
plot(iris$Sepal.Length, iris$Sepal.Width,
  main = "Example of Scatter Plot",
  xlab = "Sepal.Length",
  ylab = "Sepal.Width"
)
```



この関数のオプションとして，タイトルを与えたり，軸に名前を与えたりできる。またプロットされるピンの形，描画色，背景色など様々な操作が可能である。特段のパッケージを必要とせずとも，基本的な描画機能は備えていると言えるだろう。

4.3　ggplot による描画

ここでは，tidyverse に含まれる描画専用のパッケージである，ggplot2 パッケージを用いた描画を学ぶ。R の基本描画関数でもかなりのことができるのだが，この ggplot2 パッケージをもちいた図の方が美しく，直観的に操作できる。というのも ggplot の gg とは The Grammar of Graphics(描画の文法) のことであり，このことが示すようにロジカルに図版を制御できるからである。ggplot2 の形で記述された図版のスクリプトは可読性が高く，視覚的にも美しいため，多くの文献で利用されている。

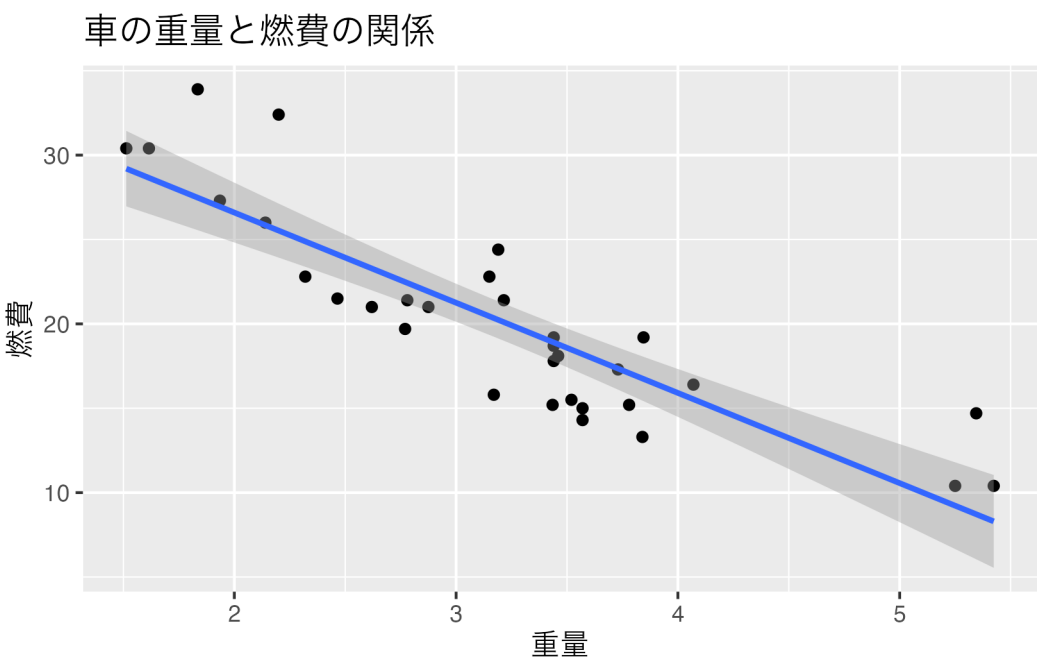
ggplot2 パッケージの提供する描画環境の特徴は，レイア (Layer) の概念である。図版は複数のレイアの積み重ねとして表現される。まず土台となるキャンバスがあり，そこにデータセット，幾何学的オブジェクト (点，線，

バーなど)，エステティックマッピング (色，形，サイズなど)，凡例やキャプションを重ねていく，という発想である。そして図版全体を通したテーマを手強することで，カラーパレットの統一などの仕上げをすれば，すぐにでも論文投稿可能なレベルの図版を描くことができる。

以下に ggplot2 における描画のサンプルを示す。サンプルデータ mtcars を用いた。

```
pacman::p_load(ggplot2)

ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth(method = "lm", formula = "y ~ x") +
  labs(title = "車の重量と燃費の関係", x = "重量", y = "燃費")
```



まずは出来上がる図版の美しさと，コードのイメージを把握してもらいたい。最初の pacman::p_load(ggplot2) はパッケージを読み込んでいるところである。今回は明示的に ggplot2 を読み込んでいるが，tidyverse パッケージを読み込むと同時に読み込まれているので，R のスクリプトの冒頭に pacman::p_load(tidyverse) と書く癖をつけておけば必要ない。

続いて ggplot の関数が 4 行にわたって書いてあるが，それぞれが + の記号で繋がれていることがわかるだろう。これがレイアを重ねるという作業に相当する。まずは，図を書くためのキャンバスを用意し，その上にいろいろ重ねていくのである。

次のコードは，キャンバスだけを描画した例である。

```
g <- ggplot()
print(g)
```

ここでは g というオブジェクトを ggplot 関数でつくり，それを表示させた。最初はこのようにプレーンなキャンバスだが，ここに次々と上書きしていくことになる。

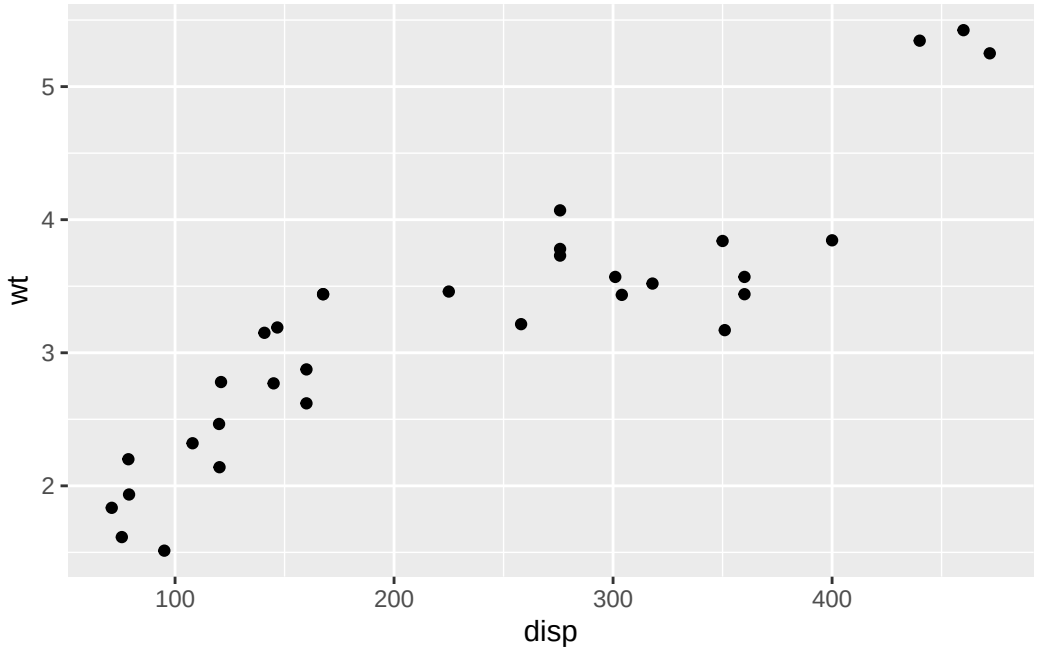
4.4　幾何学的オブジェクト geom

幾何学的オブジェクト (geometric object) とは，データの表現方法の指定であり，ggplot には様々なパターンが用意されている。以下に一例を挙げる。

- **geom_point()**: 散布図で使用され、データ点を個々の点としてプロットする。
- **geom_line()**: 折れ線グラフで使用され、データ点を線で結んでプロットする。時系列データなどによく使われる。
- **geom_bar()**: 棒グラフで使用され、カテゴリごとの量を棒で表示する。データの集計（カウントや合計など）に適している。
- **geom_histogram()**: ヒストグラムで使用され、連続データの分布を棒で表示する。データの分布を理解するのに役立つ。
- **geom_boxplot()**: 箱ひげ図で使用され、データの分布（中央値、四分位数、外れ値など）を要約して表示する。
- **geom_smooth()**: 平滑化曲線を追加し、データのトレンドやパターンを可視化する。線型回帰やローパスフィルタなどの方法が使われる。

これらの幾何学的オブジェクトに，データおよび軸との対応を指定するなどして描画する。次に挙げるのは geom_point による点描画，つまり散布図である。

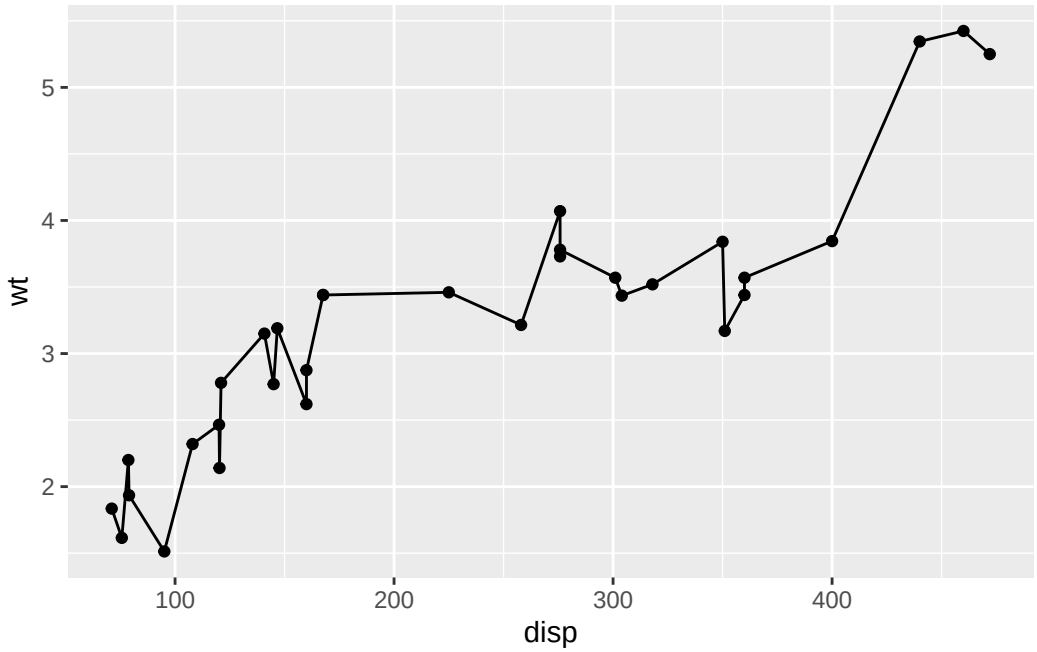
```
ggplot() +
  geom_point(data = mtcars, mapping = aes(x = disp, y = wt))
```



一行目でキャンバスを用意し，そこに `geom_point` で点を打つようにしている。このとき，データは `mtcars` であり，x 軸に変数 `disp` を，y 軸に変数 `wt` をマッピングしている。マッピング関数の `aes` は aesthetic mappings の意味で，データによって変わる値 (x 座標，y 座標，色，サイズ，透明度など) を指定することができる。

レイアは次々と重ねることができる。以下の例を見てみよう。

```
g <- ggplot()
g1 <- g + geom_point(data = mtcars, mapping = aes(x = disp, y = wt))
g2 <- g1 + geom_line(data = mtcars, mapping = aes(x = disp, y = wt))
print(g2)
```



重ねることを強調するために，g オブジェクトを次々作るようにしたが，もちろん 1 つのオブジェクトでまとめて書いてもいいし，g オブジェクトとして保管せずとも，最初の例のように直接出力することもできる。また，ここでは点描画オブジェクトに線描画オブジェクトを重ねているが，データやマッピングは全く同じである。異なるデータを一枚のキャンバスに書く場合は，このように幾何学オブジェクトごとの指定が可能であるが，図版は得てして一枚のキャンバスに一種類のデータになりがちである。そのような場合は，以下に示すようにキャンバスの段階から基本となるデータセットとマッピングを与えることが可能である。

```
ggplot(data = mtcars, mapping = aes(x = disp, y = wt)) +
  geom_point() +
  geom_line()
```

また，この用例の場合 ggplot 関数の第一引数はデータセットなので，パイプ演算子で渡すことができる。

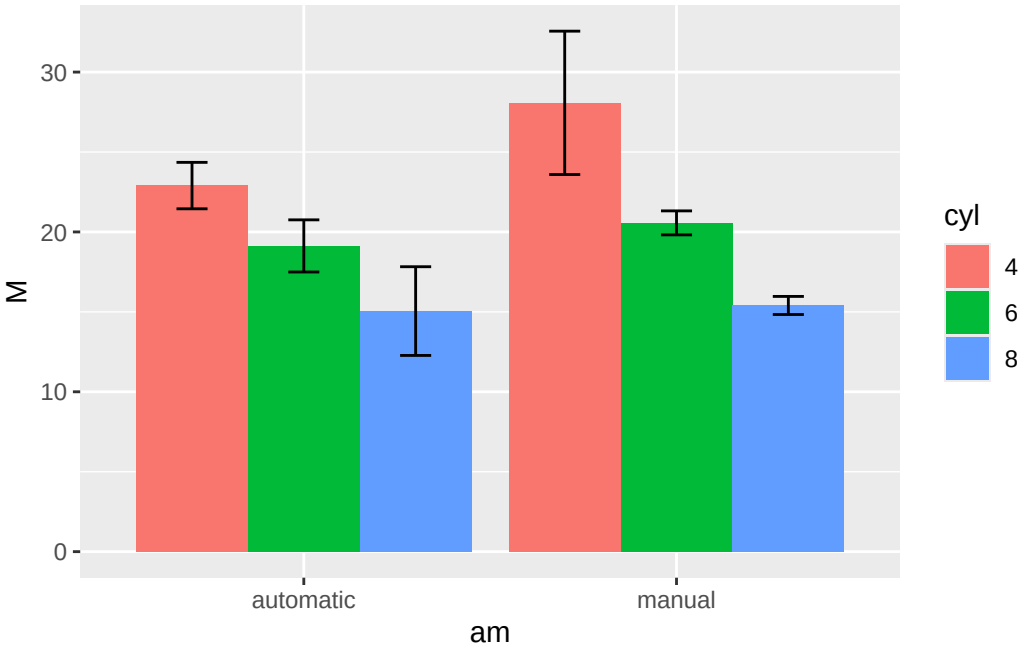
```
mtcars %>%
  ggplot(mapping = aes(x = disp, y = wt)) +
  geom_point() +
  geom_line()
```

パイプ演算子を使うことで，素データをハンドリングし，必要な形に整えて可視化する，という流れがスクリプト上で読むように表現できるようになる。慣れてくると，データセットから可視化したい要素を特定し，最終的にどのように成形すれば ggplot に渡しやすくなるかを想像して加工していくようになる。そのためには到達目標となる図版のイメージを頭に描き，その図の x 軸，y 軸は何で，どのような幾何学オブジェクトが上に乗っているのか，といったように図版のリバースエンジニアリング，あるいは図版の作成手順の書き下しができる必要がある。たとえるなら，食べたい料理に必要な材料を集め，大まかな手順 (下ごしらえからの調理) を組み立てられるかどうか，である。実際にレシピに書き起こす際は生成 AI の力を借りると良いが，その際も最終的な目標と，全体的な設計方針から指示し，微調整を追加していくように指示すると効率的である。

以下に，データハンドリングと描画の一例を示す。各ステップにコメントをつけたので，文章を読むように加工と描画の流れを確認し，出力結果と照らし合わせてみよう。

```
# mtcarsデータセットを使用
mtcars %>%
  # 変数選択
  select(mpg, cyl, wt, am) %>%
  mutate(
    # 変数am,cylをFactor型に変換
    am = factor(am, labels = c("automatic", "manual")),
    cyl = factor(cyl)
  ) %>%
  # 水準ごとにグループ化
  group_by(am, cyl) %>%
  summarise(
    M = mean(mpg), # 各グループの平均燃費（M）を計算
```

```
  SD = sd(mpg), # 各グループの燃費の標準偏差（SD）を計算
  .groups = "drop" # summarise後の自動的なグルーピングを解除
) %>%
# x軸にトランスミッションの種類、y軸に平均燃費，塗りつぶしの色はcyl
ggplot(aes(x = am, y = M, fill = cyl)) +
# 横並びの棒グラフ
geom_bar(stat = "identity", position = "dodge") +
# ±1SDのエラーバーを追加
geom_errorbar(
  # エラーバーのマッピング
  aes(ymin = M - SD, ymax = M + SD),
  # エラーバーの位置を棒グラフに合わせる
  position = position_dodge(width = 0.9),
  width = 0.25 # エラーバーの幅を設定
)
```



繰り返しになるが，このコードは慣れてくるまでいきなり書けるものではない。重要なのは「出力結果をイメージ」することと，それを「要素に分解」,「手順に沿って並べる」ことができるかどうかである。[*3]

4.5 描画 tips

最後に，いくつかの描画テクニックを述べておく。これらについては，必要な時に随時ウェブ上で検索したり，生成 AI に尋ねることでも良いが，このような方法がある，という基礎知識を持っておくことも重要だろう。なお描

[*3] 実際コードは chatGPTver4 に指示して生成した。いきなり全体像を描くのではなく，徐々に追記していくと効果的である。

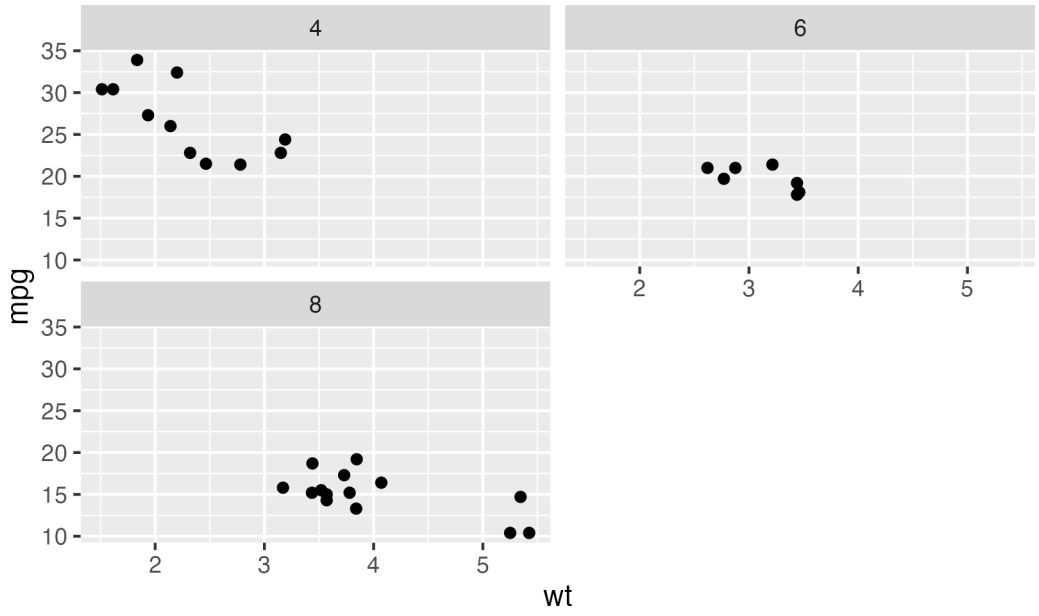
画について詳しくは松村他（2021）の 4 章を参照すると良い。

4.5.1　ggplot オブジェクトを並べる

複数のプロットを一枚のパネルに配置したい，ということがあるかもしれない。先ほどの mtcars データの例でいえば，am 変数にオートマチック車かマニュアル車かの 2 水準があるが，このようなサブグループごとに図を分割したいという場合である。

このような時には，facet_wrap や facet_grid という関数が便利である。前者はある変数について，後者は 2 つの変数について図を分割する。

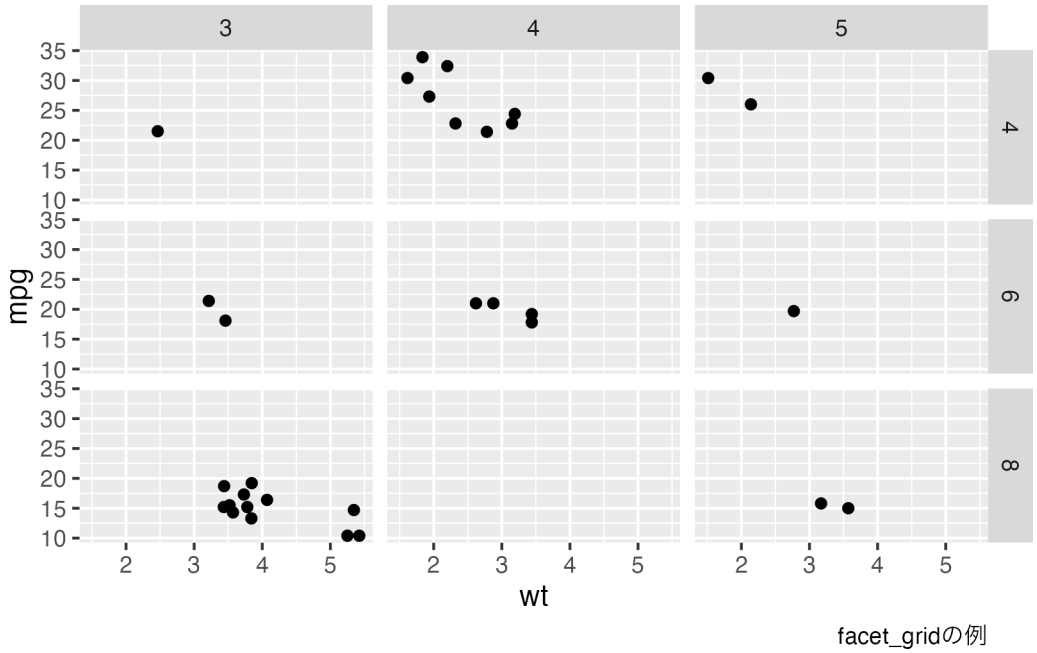
```
mtcars %>%
  # 重さwtと燃費mpgの散布図
  ggplot(aes(x = wt, y = mpg)) +
  geom_point() +
  # シリンダ数cylで分割
  facet_wrap(~cyl, nrow = 2) +
  # タイトルをつける
  labs(caption = "facet_wrapの例")
```



facet_wrapの例

```
mtcars %>%
  ggplot(aes(x = wt, y = mpg)) +
  geom_point() +
  # シリンダ数cylとギア数gearで分割
  facet_grid(cyl ~ gear) +
  # キャプションをつける
```

```
  labs(caption = "facet_gridの例")
```



一枚の図をサブグループに分けるのではなく，異なる図を一枚の図として配置したいこともあるかもしれない。そのような場合は，patchwork パッケージを使うと便利である。

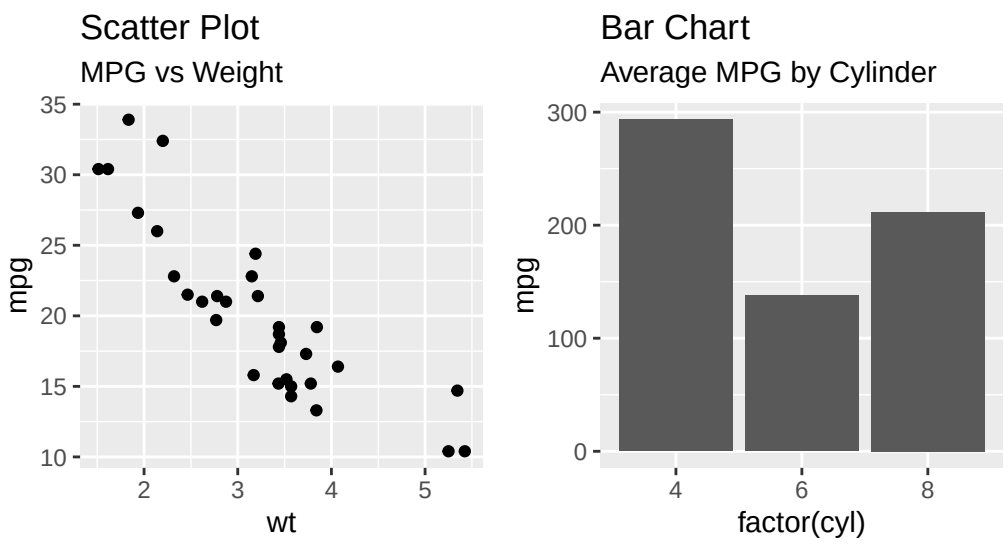
```
pacman::p_load(patchwork)

# 散布図の作成
g1 <- ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  # 散布図のタイトルとサブタイトル
  ggtitle("Scatter Plot", "MPG vs Weight")

# 棒グラフの作成
g2 <- ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_bar(stat = "identity") +
  # 棒グラフのタイトルとサブタイトル
  ggtitle("Bar Chart", "Average MPG by Cylinder")

# patchworkを使用して2つのグラフを組み合わせる
combined_plot <- g1 + g2 +
  plot_annotation(
    title = "Combined Plots",
    subtitle = "Scatter and Bar Charts"
  )
```

```
# プロットを表示
print(combined_plot)
```



4.5.2　ggplot オブジェクトの保存

Rmd や Quarto で文書を作るときは，図が自動的に生成されるので問題ないが，図だけ別のファイルとして利用したい，保存したいということがあるかもしれない。その時は ggsave 関数で ggplot オブジェクトを保存するとよい。

```
# 散布図を作成
p <- ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
ggsave(
  filename = "my_plot.png", # 保存するファイル名。
  plot = p, # 保存するプロットオブジェクト。
  device = "png", # 保存するファイル形式。
  path = "path/to/directory", # ファイルを保存するディレクトリのパス
  scale = 1, # グラフィックスの拡大縮小比率
  width = 5, # 保存するプロットの幅（インチ）
  height = 5, # 保存するプロットの高さ（インチ）
  dpi = 300, # 解像度（DPI: dots per inch）
)
```

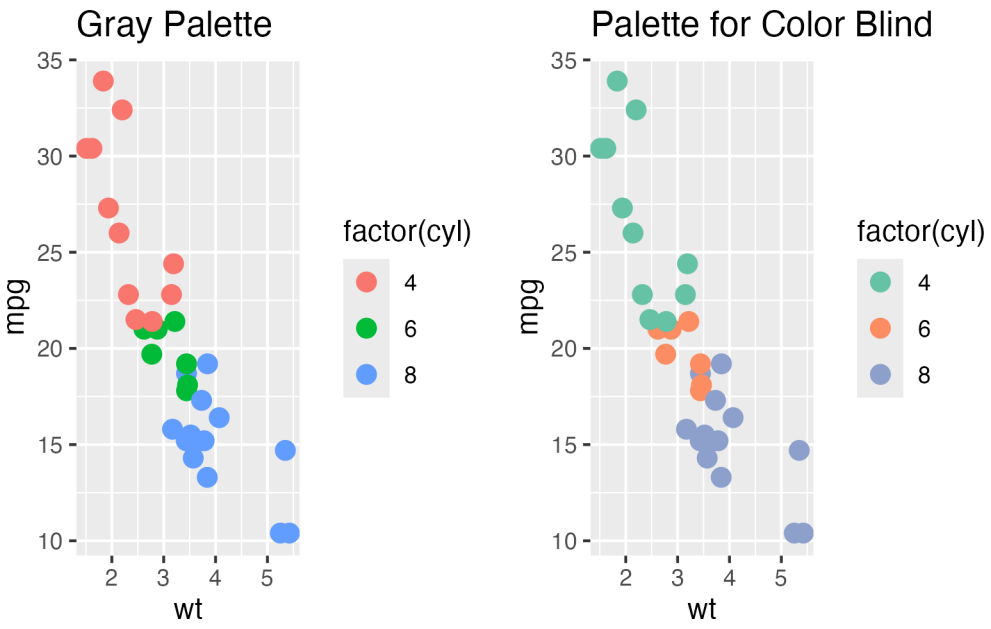
4.5.3 テーマの変更（レポートに合わせる）

レポートや論文などの提出次の条件として，図版をモノクロで表現しなければならないことがあるかもしれない。ggplot では自動的に配色されるが，その背後ではデフォルトの絵の具セット (**パレット**という) が選択されているからである。このセットを変更すると，同じプロットでも異なる配色で出力される。モノクロ (グレイスケール) で出力したい時のパレットは Grays である。

```
# グレースケールのプロット
p1 <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 3) +
  scale_fill_brewer(palette = "Greys") +
  ggtitle("Gray Palette")

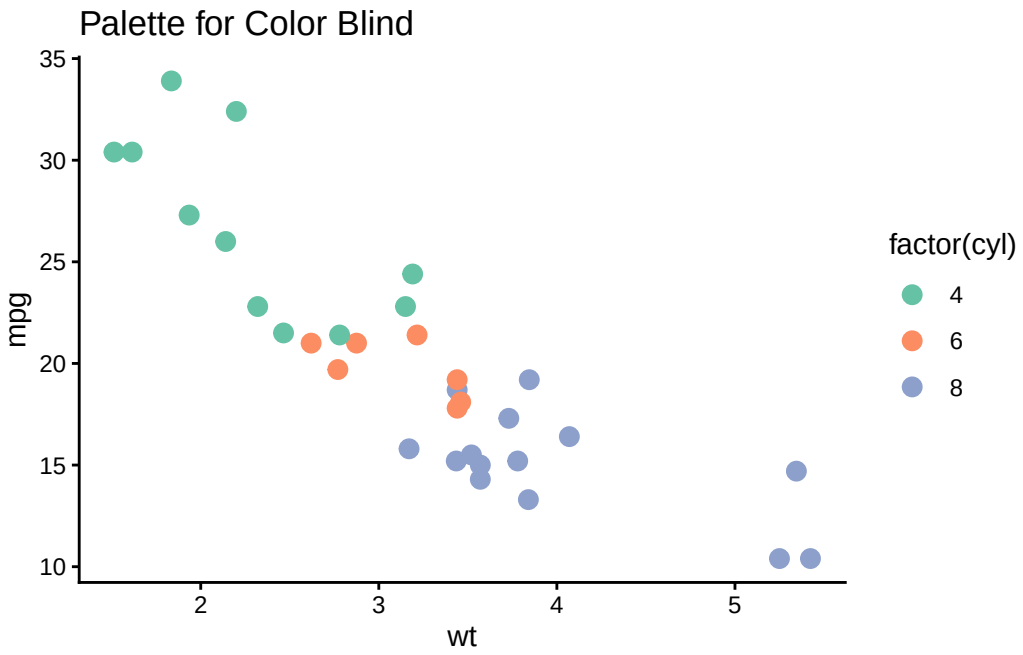
# カラーパレットが多く含まれているパッケージの利用
pacman::p_load(RColorBrewer)
# 色覚特性を考慮したカラーパレット
p2 <- ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 3) +
  scale_color_brewer(palette = "Set2") + # 色覚特性を考慮したカラーパレット
  ggtitle("Palette for Color Blind")

# 両方のプロットを並べて表示
combined_plot <- p1 + p2 + plot_layout(ncol = 2)
print(combined_plot)
```



また，ggplot2 のデフォルト設定では，背景色が灰色になっている。これは全体のテーマとして theme_gray() が設定されているからである。しかし日本心理学会の執筆・投稿の手引きに記載されているグラフの例を見ると，背景は白色とされている。このような設定に変更するためには，theme_classic() や theme_bw() を用いる。

```
p2 + theme_classic()
```



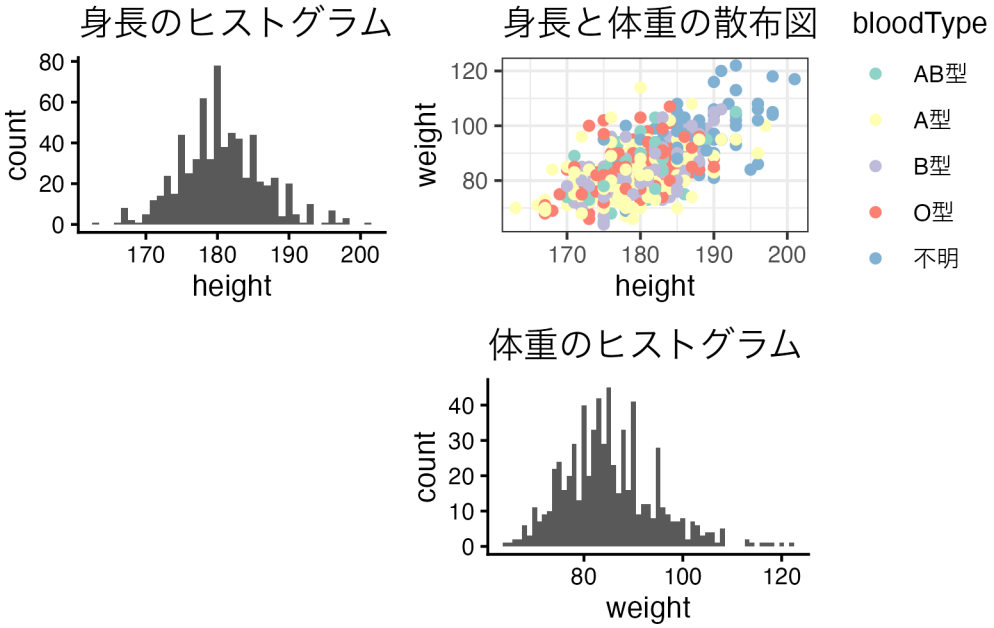
このほかにも，様々な描画上の工夫は考えられる。目標となる図版のレシピを書き起こせるように，要素に分解ができれば，殆どのケースにおいて問題を解決することができるだろう。

4.6　課題

- 今日の課題は Rmarkdown で記述してください。著者名に学籍番号と名前を含め、適宜見出しをつくり、平文で以下に挙げる課題を記載することでどの課題に対する回答のコード（チャンク）であるかわかるようにしてください。

1. Baseball.csv を読み込み、2020 年度のデータセットに限定し、以下の操作に必要であれば変数の変換を済ませたデータセット、dat.tb を用意してください。
2. dat.tb の身長変数を使って、ヒストグラムを描いてください。この時、テーマを theme_classic にしてください。
3. dat.tb の身長変数と体重変数を使って、散布図を描いてください。この時、テーマを theme_bw にしてください。
4. (承前) 散布図の各点を血液型で塗り分けてください。この時、カラーパレットを Set3 に変えてください。
5. (承前) 散布図の点の形を血液型で変えてください。
6. dat.tb の身長と体重についての散布図を、チームごとに分割してください。
7. (承前) geom_smooth() でスムーズな線を引いてください。特に method を指定する必要はありません。
8. (承前) geom_smooth() で直線関数を引いてください。method="lm"と指定するといいでしょう。

9. x 軸は身長、y 軸は体重の平均値をプロットしてください。方法はいろいろありますが、要約統計量を計算した別のデータセット `dat.tb2` を作るか、幾何学オブジェクトの中で、次のように関数を適用することもできます。ヒント：`geom_point(stat="summary", fun=mean)`。
10. 課題 2, 4 および体重のヒストグラムを使って下の図を描き、`ggsave` 関数を使って保存するコードを書いてください。ファイル名やその他オプションは任意です。



第 5 章

R でプログラミング

ここではプログラミング言語としての R について解説する。なお副読本として小杉他（2023）を挙げておく。また，プログラミングのより専門的な理解のために，ランダー，J.P.（2018）, 2017, 2016 なども参考にすると良い。

プログラミング言語は，古くは C や Java，最近では Python や Julia などがよく用いられている。R も統計パッケージというよりプログラミング言語として考えるのが適切かもしれない。R は他のプログラミング言語に比べて，変数の型宣言を事前にしなくても良いことや，インデントなど書式についておおらかなところは，初心者にとって使いやすいところだろう。一方で，ベクトルの再利用のところで注意したように (Section 2.5.1)，不足分を補うために先回りして補填されたり，この後解説する関数の作成時に明示的な指定がなければ環境変数を参照する点など，親切心が空回りするところがある。より厳格な他言語になれていると，こうした点はかえって不便に思えるところもあるかもしれない。総じて，R 言語は初心者向けであるといえるだろう。

さて，世にプログラミング言語は多くあれど[*1]，その全てに精通する必要はないし，不可能である。それよりも，プログラミング言語一般に通底する基本的概念を知り，あとは各言語による「方言」がある，と考えた方が生産的である。その基本的概念を 3 つ挙げるとすれば，「代入」「反復」「条件分岐」になるだろう。

5.1 代入

代入は，言い換えればオブジェクト (メモリ) に保管することを指す。これについては既に Chapter 2 で触れた通りであり，ここでは言及しない。オブジェクトや変数の型，常に上書きされる性質に注意しておけば十分だろう。

一点だけ追加で説明しておくと，次のような表現がなされることがある。

```
a <- 0
a <- a + 1
print(a)
```

```
[1] 1
```

ここではあえて，代入記号として=を使った。2 行目に `a = a + 1` とあるが，これを見て数式のように解釈しよ

[*1] シ（2016）には 117 種もの計算機言語が紹介されている。

うとすると混乱する。数学的には明らかにおかしな表現だが，これは上書きと代入というプログラミング言語の特徴を使ったもので，「(いま保持している)a の値に 1 を加えたものを，(新しく同じ名前のオブジェクト)a に代入する (=上書きする)」という意味である。この方法で，a をカウンタ変数として用いることがある。誤読の可能性を下げるため，この授業においては代入記号を<-としている。

このオブジェクトを上書きするという特徴は多くの言語に共通したものであり，間違いを避けるためには，オブジェクトを作る時に初期値を設定することが望ましい。先の例では，代入の直上で a <- 0 としており，オブジェクト a に 0 を初期値として与えている。この変数の初期化作業がないと，以前に使っていた値を引き継いでしまう可能性があるので，今から新しく使う変数を作りたいというときは，このように明示しておくといいだろう。

なお，変数をメモリから明示的に削除する場合は，remove 関数を使う。

```
remove(a)
```

これを実行すると，RStudio の Environment タブからオブジェクト a が消えたことがわかるだろう。メモリの一斉除去は，同じく RStudio の Environment タブにある箒マークをクリックするか，remove(list=ls()) とすると良い[*2]。

5.2　反復

5.2.1　for 文

電子計算機の特徴は，電源等のハードウェア的問題がなければ疲労することなく計算を続けられるところにある。人間は反復によって疲労が溜まったり，集中力が欠如するなどして単純ミスを生成するが，電子計算機にそういったところはない。

このように反復計算は電子計算機の中心的特徴であり，細々した計算作業を指示した期間反復させ続けることができる。反復の代表的なコマンドは for であり，for ループなどと呼ばれる。for ループはプログラミングの基本的な制御構造であり，R 言語の for ループの基本的な構文は次のようになる：

```
for (value in sequence) {
    # 実行するコード
}
```

ここの value は各反復で sequence の次の要素を取る反復インデックス変数である。。sequence は一般にベクトルやリストなどの配列型のデータであり，「#実行するコード」はループ体内で実行される一連の命令になる。

以下は for 文の例である。

```
for (i in 1:5) {
  cat("現在の値は", i, "です。\n")
}
```

```
現在の値は 1 です。
```

[*2] ls() 関数は list objects の意味で，メモリにあるオブジェクトのリストを作る関数

```
現在の値は 2 です。
現在の値は 3 です。
現在の値は 4 です。
現在の値は 5 です。
```

for 文は続く小括弧のなかである変数を宣言し (ここでは i)，それがどのように変化するか (ここでは 1:5，すなわち 1,2,3,4,5) を指定する。続く中括弧の中で，反復したい操作を記入する。今回は cat 文によるコンソールへの文字力の出力を行っている。ここでのコマンドは複数あってもよく，中括弧が閉じられるまで各行のコマンドが実行される。

次に示すは，sequence にあるベクトルが指定されているので，反復インデックス変数が連続的に変化しない例である。

```
for (i in c(2, 4, 12, 3, -6)) {
  cat("現在の値は", i, "です。\n")
}
```

```
現在の値は 2 です。
現在の値は 4 です。
現在の値は 12 です。
現在の値は 3 です。
現在の値は -6 です。
```

また，反復はネスト (入れ子) になることもできる。次の例を見てみよう。

```
# 2次元の行列を定義
A <- matrix(1:9, nrow = 3)

# 行ごとにループ
for (i in 1:nrow(A)) {
  # 列ごとにループ
  for (j in 1:ncol(A)) {
    cat("要素 [", i, ", ", j, "]は ", A[i, j], "\n")
  }
}
```

```
要素 [ 1 ,  1 ]は  1
要素 [ 1 ,  2 ]は  4
要素 [ 1 ,  3 ]は  7
要素 [ 2 ,  1 ]は  2
要素 [ 2 ,  2 ]は  5
要素 [ 2 ,  3 ]は  8
要素 [ 3 ,  1 ]は  3
要素 [ 3 ,  2 ]は  6
```

要素 [3 , 3]は 9

ここで，反復インデックス変数が i と j というように異なる名称になっていることに注意しよう。例えば今回，ここで両者を i にしてしまうと，行変数なのか列変数なのかわからなくなってしまう。また少し専門的になるが，R 言語は for 文で宣言されるたびに，内部で反復インデックス変数を新しく生成している (異なるメモリを割り当てる) ためにエラーにならないが，他言語の場合は同じ名前のオブジェクトと判断されることが一般的であり，その際は値が終了値に到達せず計算が終わらないといったバグを引き起こす。反復に使う汎用的な変数名として i,j,k がよく用いられるため，自身のスクリプトの中でオブジェクト名として単純な一文字にすることは避けた方がいいだろう。

5.2.2　while 文

while ループはプログラミングの基本構造であり，特定の条件が真（True）である間，繰り返し一連の命令を実行する。「“while”（〜する間)」という名前から直感的に理解できるだろう。

R 言語の while ループの基本的な構文は次のようになる：

```
while (condition) {
    # 実行するコード
}
```

ここで，「condition」はループが終了するための条件である。「# 実行するコード」はループ体内で実行される一連の指示である。たとえば，1 から 10 までの値を出力する while ループは以下のように書くことができる：

```
i <- 1
while (i <= 5) {
  print(i)
  i <- i + 1
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

このコードでは，「i」が 5 以下である限りループが続く。「print(i)」で「i」の値が表示され，「i <- i + 1」で「i」の値が 1 ずつ増加する。これにより，「i」の値が 5 を超えると条件が偽 (False) となり，ループが終了する。

while ループを使用する際の一般的な注意点は，無限ループ（終わらないループ）を避けることである。これは，condition が常に真 (True) である場合に発生する。そのような状況を避けるためには，ループ内部で何らかの形で condition が最終的に偽 (False) となるようにコードを記述することが必要である。

また，R 言語は他の多くのプログラミング言語と異なり，ベクトル化された計算を効率的に行う設計がされている。したがって，可能な限り for ループや while ループを使わずに，ベクトル化した表現を利用すれば計算速度

を上げることができる。

5.3 条件分岐

条件分岐はプログラム内で特定の条件を指定し，その条件が満たされるかどうかによって異なる処理を行うための制御構造である。R 言語では `if-else` を用いて条件分岐を表現する。

5.3.1 `if` 文の基本的な構文

以下が `if` 文の基本的な構文になる：

```
if (条件) {
    # 条件が真である場合に実行するコード
}
```

`if` の後の小括弧内に条件を指定する。この条件が真 (TRU) であれば，その後の中括弧`{}` 内のコードが実行される。さらに，`else` を使用して，条件が偽（FALSE）の場合の処理を追加することもできる：

```
if (条件) {
    # 条件が真である場合に実行するコード
} else {
    # 条件が偽である場合に実行するコード
}
```

以下に具体的な使用例を示そう：

```
x <- 10

if (x > 0) {
  print("x is positive")
} else {
  print("x is not positive")
}
```

```
[1] "x is positive"
```

このコードでは，変数 `x` が正の場合とそうでない場合で異なるメッセージを出力する。

条件は論理式（例：`x > 0`, `y == 1`）や論理値（TRUE/FALSE）を返す関数・操作（例：`is.numeric(x)`）などで指定する。また，複数の条件を組み合わせる際には論理演算子 (`&&`, `||`) を使用する。

この例では，`x` が正と `y` が負の場合に特定のメッセージを出力する。それ以外の場合は，「Other case」と出力される。`x` や `y` の値を色々変えて，試してみて欲しい。

```
x <- 10
y <- -3

if (x > 0 && y < 0) {
  print("x is positive and y is negative")
} else {
  print("Other case")
}
```

```
[1] "x is positive and y is negative"
```

5.4　反復と条件分岐に関する練習問題

1. 1 から 20 までの数字で，偶数だけをプリントするプログラムを書いてください。
2. 1 から 40 までの数値をプリントするプログラムを書いてください。ただしその数値に 3 がつく (1 か 10 の位の値が 3 である) か，3 の倍数の時だけ，数字の後ろに「サァン！」という文字列をつけて出力してください。
3. ベクトル `c(1, -2, 3, -4, 5)` の各要素について，正なら “positive”，負なら “negative” をプリントするプログラムを書いてください。
4. 次の行列 A と B の掛け算を計算するプログラムを書いてください。なお，R で行列の積は `%*%` という演算子を使いますが，ここでは `for` 文を使ったプログラムにしてください。出来上がる行列の i 行 j 列目の要素 c_{ij} は，行列 A の第 i 行の各要素と，行列 B の第 j 列目の各要素の積和，すなわち

$$c_{ij} = \sum_k a_{ik} b_{kj}$$

   になります。検算用のコードを下に示します。

```
A <- matrix(1:6, nrow = 3)
B <- matrix(3:10, nrow = 2)
## 課題になる行列
print(A)
```

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

```
print(B)
```

```
     [,1] [,2] [,3] [,4]
[1,]    3    5    7    9
[2,]    4    6    8   10
```

```
## 求めるべき答え
C <- A %*% B
print(C)
```

```
     [,1] [,2] [,3] [,4]
[1,]   19   29   39   49
[2,]   26   40   54   68
[3,]   33   51   69   87
```

5.5 関数を作る

複雑なプログラムも，ここまでの代入、反復，条件分岐の組み合わせからなる。回帰分析や因子分析のような統計モデルを実行するときに，統計パッケージのユーザとしては，統計モデルを実現してくれる関数にデータを与えて答えを受け取るだけであるが，そのアルゴリズムはこれらプログラミングのピースを紡いでつくられているのである。

ここでは関数を自分で作ることを考える。といっても身構える必要はない。表計算ソフトウェアで同じような操作を繰り返すときにマクロに記録するように，R 上で同じようなコードを何度も書く機会があるならば，それを関数という名のパッケージにしておこう，ということである。関数化しておくことで手続きをまとめることができ，小単位に分割できるため並列して開発したり，バグを見つけやすくなるという利点がある。

5.5.1 基本的な関数の作り方

関数が受け取る値のことを**引数** (ひきすう，argument) といい，また関数が返す値のことを**戻り値** (もどりち,value) という。$y = f(x)$ という式は，引数が x で戻り値が y な関数 f，と言い換えることができるだろう。

R の関数を書く基本的な構文は以下のようになる。

```
function_name <- function(argument) {
   # function body
   return(value)
}
```

ここで function body とあるのは計算本体である。例えば与えられた数字に 3 を足して返す関数，add3 を作ってみよう。プログラムは以下のようになる。

```
add3 <- function(x) {
  x <- x + 3
  return(x)
}
# 実行例
add3(5)
```

```
[1] 8
```

また，2 つの値を足し合わせる関数は次のようになる。

```
add_numbers <- function(a, b) {
  sum <- a + b
  return(sum)
}
# 実行例
add_numbers(2, 5)
```

```
[1] 7
```

ここで示したように，引数は複数取ることも可能である。また，既定値 default value を設定することも可能である。次の例を見てみよう。

```
add_numbers2 <- function(a, b = 1) {
  sum <- a + b
  return(sum)
}
# 実行例
add_numbers2(2, 5)
```

```
[1] 7
```

```
add_numbers2(4)
```

```
[1] 5
```

関数を作るときに，(a,b=1) としているのは，b に既定値として 1 を与えていて，特に指定がなければこの値を使うよう指示しているということである。実行例において，引数が 2 つ与えられている場合はそれらを使った計算をし (2+5)，1 つしか与えられていない場合は第一引数 a に与えられた値を，第二引数 b は既定値を使った計算をする (4+1)，という挙動になる。

ここから推察できるように，われわれユーザが使う統計パッケージの関数にも実は多くの引数があり，既定値が与えられているということだ。これらの引数は選択的に指定することができるものであるが，通常は一般的に使われる値や計算の細かな設定に関するものであり，開発者がユーザの手間を省くために提供しているものである。関数のヘルプを見ると指定可能な引数の一覧が表示されるので，ぜひ興味を持って見てもらいたい。

5.5.2　複数の戻り値

R での戻り値は 1 つのオブジェクトでなければならない。しかし，複数の値を返したいということがあるだろう。そのような場合は，返すオブジェクトを list などでひとまとめにして作成すると良い。以下に簡単な例を示す。

```
calculate_values <- function(a, b) {
  sum <- a + b
```

```
  diff <- a - b
  # 戻り値として名前付きリストを作成
  result <- list("sum" = sum, "diff" = diff)
  return(result)
}
# 実行例
result <- calculate_values(10, 5)
# 結果を表示
print(result)
```

```
$sum
[1] 15

$diff
[1] 5
```

5.6 課題

1. ある値を与えたとき，正の値なら”positive”，負の値なら”negative”，0 のときは”Zero” と表示する関数を書いてください。
2. ある 2 組の数字を与えた時，和，差，積，商を返す関数を書いてください。
3. あるベクトルを与えた時，算術平均，中央値，最大値，最小値，範囲を返す関数を書いてください。
4. あるベクトルを与えた時，標本分散を返す関数を書いてください。なお R の分散を返す関数 `var` は不偏分散 $\hat{\sigma}$ を返しており，標本分散 `v` とは計算式が異なります。念のため，計算式を以下に示します。

$$\hat{\sigma} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

$$v = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

第 6 章

確率とシミュレーション

6.1 確率の考え方と使い所

統計と確率は密接な関係がある。まずデータをたくさん集めると，個々のケースでは見られない全体的な傾向が見られるようになり，それを表現するのに確率の考え方を使う，というのがひとつ。次にデータがそれほどたくさんなくとも，大きな全体の中から一部を取り出した標本 Sample と考えられるとき，標本は全体の性質をどのように反映しているかを考えることになる。ここで全体の傾向から一部を取り出した偶然性を表現するときに確率の考え方を使うことになる。最後に，理論的・原理的に挙動がわかっている機械のようなものでも，現実的・実践的には系統だったズレが生じたり，偶然としか考えられない誤差が紛れ込むことがある。前者は機械の調整で対応できるが，後者は偶然が従う確率を考える必要がある。

心理学は人間を対象に研究を行うが，あらゆる人間を一度に調べるわけにはいかないので，サンプルを取り出して調査したり実験したりする (第 2 のケース)。データサイエンスでは何万レコードというおおきなデータセットになるが，心理学の場合は数件から数十件しかないことも多い。また，心理学的傾向を理論立ててモデル化できたとしても，実際の行動には誤差が含まれている可能性が高い (第 3 のケース)。このことから，心理学で得られるデータは確率変数として考えられ，小標本から母集団の性質を推測する**推測統計**と共に利用される。

厳密に数学的な意味での**確率**は，集合，積分，測度といった緻密な概念の積み重ねから定義される[*1]。ここではその詳細に分け入らず，単に「特定の結果が生じる可能性について，0 から 1 の間の実数でその大小を表現したもの」とだけ理解しておいて欲しい。この定義からは，「全ての可能な組み合わせのうち当該事象の成立する割合」という解釈も成り立つし，「主観的に重みづけた真実味の強さに関する信念の度合い」という解釈も成り立つ。[*2]これまで学んできた確率は順列・組み合わせを全て書き出す退屈なもの，と思っていたかもしれないが，「十中八九まちがいないね (80-90% ほど確からしいと考えている)」という数字も確率の一種として扱えるので，非常に身近で適用範囲の広い概念である。理解を進めるポイントの 1 つとして，確率を面積として考えると良いかもしれない。ありうる状況の全体の空間に対して，事象の成立する程度がどの程度の面積がどの程度の割合であるかを表現したのが確率という量である，と考えるのである (平岡・堀 (2009) は書籍の中で一貫して面積で説明してい

[*1] 詳しくは吉田 (2021) , 河野 (1999) , 佐藤 (1994) などを参照のこと。

[*2] 前者の解釈は高校までの数学で学ぶ確率であり，頻度主義的確率と呼ばれることがある。一方後者の解釈は，降水確率 X% のように日常でも使うものであり，主観確率と呼ばれることがある。こうした解釈の違いを，主義主張の対立であって数学的ではない，と批判する向きもあるが，実際コルモゴロフの公理はどちらの立場でも成立するように整えられており，筆者個人的にはユーザが理解しやすく計算できればどちらでも良いと考えている。

る。この説明だと，条件付き確率などの理解がしやすい。)。

ただし注意して区別しておいて欲しいのが，確率変数とその実現値の違いである。データセットやスプレッドシートに含まれる値は，あくまでも**確率変数の実現値**（実際に観測された値）というのであって，**確率変数**はその不確実な状態を有した変数そのものを指す言葉である。サイコロは確率変数だが，サイコロの出目は確率変数の実現値である。心理変数は確率変数だが，手に入れたデータはその実現値である。実現値を通じて変数の特徴を知り，全体を推測するという流れである。

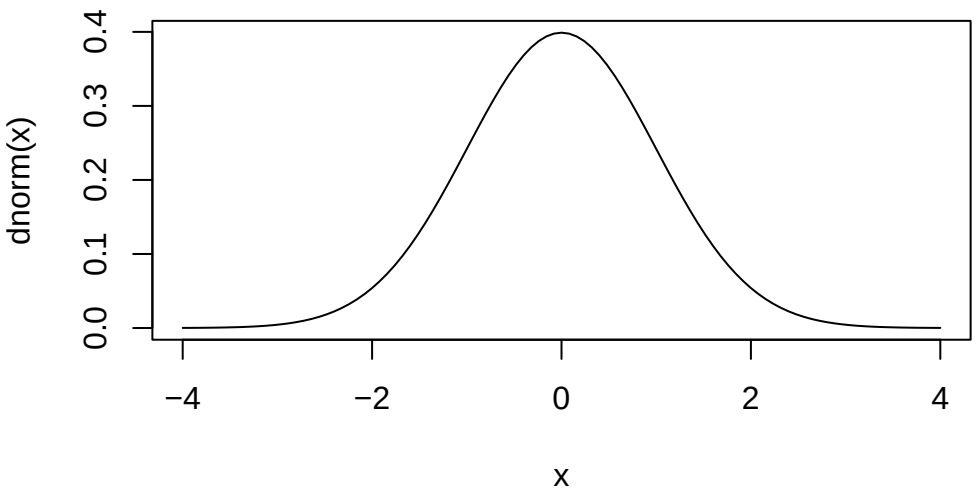
目の前のデータを超えて，抽象的な実体で議論を進めることが難しく感じられるかもしれない。実は誰しもそうなのであって，確率の正確な理解は非常に難易度が高い。しかし R など計算機言語に実装されている関数を通じて，より具体的に，操作しながら理解することで徐々に理解していこう。

6.2　確率分布の関数

確率変数の実現値は，**確率分布**に従う。確率分布とは，その実現値がどの程度生じやすいかを全て表した総覧であり，一般的に関数で表現される。実現値が連続的か離散的かによって名称が異なるが，連続的な確率分布関数は**確率密度関数 (Probability Density Function)**，離散的な確率分布関数は**確率質量関数 (Probability Mass Function)** という。

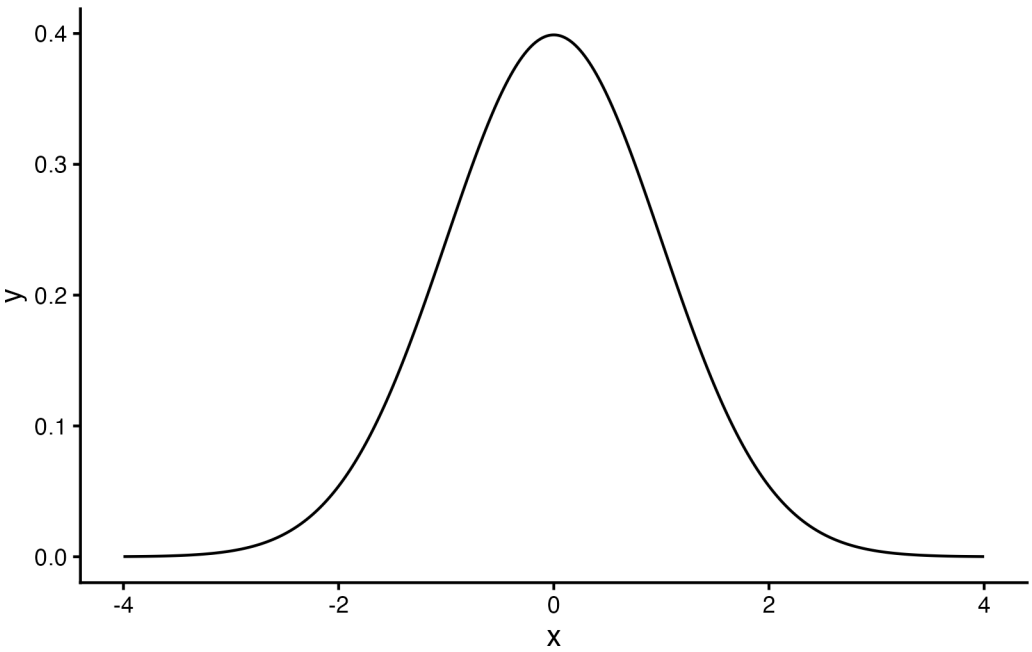
R には最初から確率に関する関数がいくつか準備されている。最も有名な確率分布である**正規分布**について，次のような関数がある。

```
# 標準のプロット関数，curve
curve(dnorm(x), from = -4, to = 4)
```




```
# ggplot2を使ってカッコよく
pacman::p_load(tidyverse)
data.frame(x = seq(-4, 4, by = 0.01)) %>%
  mutate(y = dnorm(x)) %>%
  ggplot(aes(x = x, y = y)) +
  geom_line() +
```

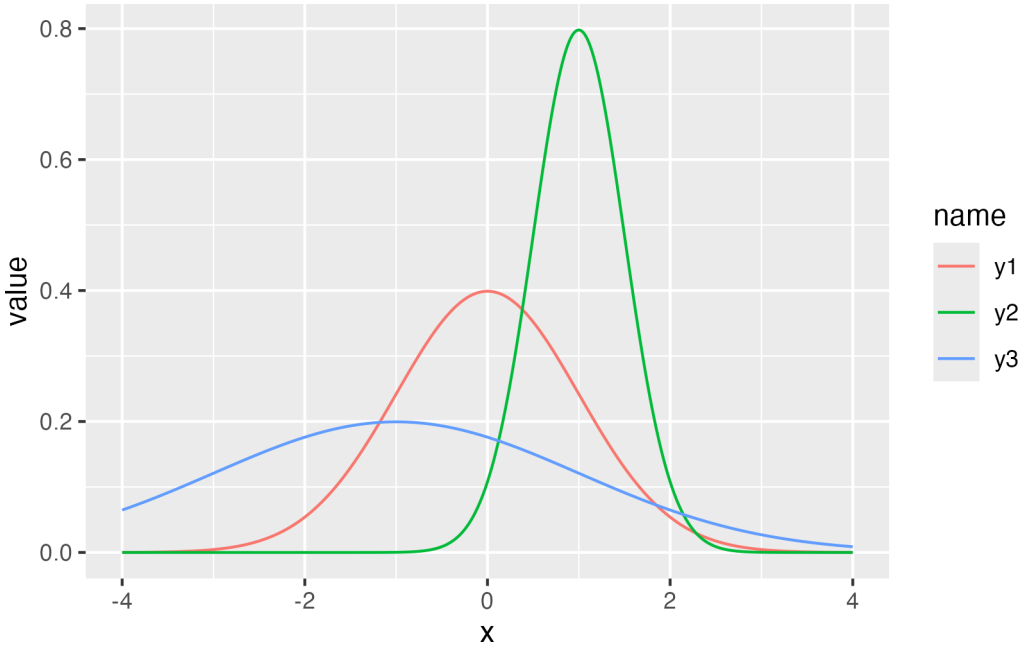
```
  theme_classic()
```



ここで dnorm という関数を使っているが，d は Density(確率密度) の頭文字であり，norm は Normal Distribution(正規分布) の一部である。このように，R では確率分布の名前を表す名称 (ここでは norm) と，それに接頭文字ひとつ (d) で関数を構成する。この接頭文字は他に p,q,r があり，dpois(ポアソン分布 poisson distribution の確率密度関数)，pnorm(正規分布 normal distribution の累積分布関数),rbinom(二項分布 binomial distribution からの乱数生成) のように使う。

ここでは正規分布を例に説明を続けよう。正規分布は平均 μ と標準偏差 σ でその形状が特徴づけられる。これらの確率分布の特徴を表す数字のことを**母数 parameter** という。たとえば，次の 3 つの曲線はパラメータが異なる正規分布である。

```
data.frame(x = seq(-4, 4, by = 0.01)) %>%
  mutate(
    y1 = dnorm(x, mean = 0, sd = 1),
    y2 = dnorm(x, mean = 1, sd = 0.5),
    y3 = dnorm(x, mean = -1, sd = 2)
  ) %>%
  pivot_longer(-x) %>%
  ggplot(aes(x = x, y = value, color = name)) +
  geom_line()
```



平均は位置母数，標準偏差はスケール母数とも呼ばれ，分布の位置と幅を変えていることがわかる。言い換えると，データになるべく当てはまるように正規分布の母数を定めることもできるわけで，左右対称で単峰の分布という特徴があれば，正規分布でかなり様々なパターンを表せる。

さて，上の例で用いた関数はいずれも d を頭に持つ dnorm であり，確率分布の密度の高さを表現していた。では p や q が表すのは何であろうか。数値と図の例を示すので，その対応関係を確認してもらいたい。

```
# 累積分布関数
pnorm(1.96, mean = 0, sd = 1)
```

```
[1] 0.9750021
```

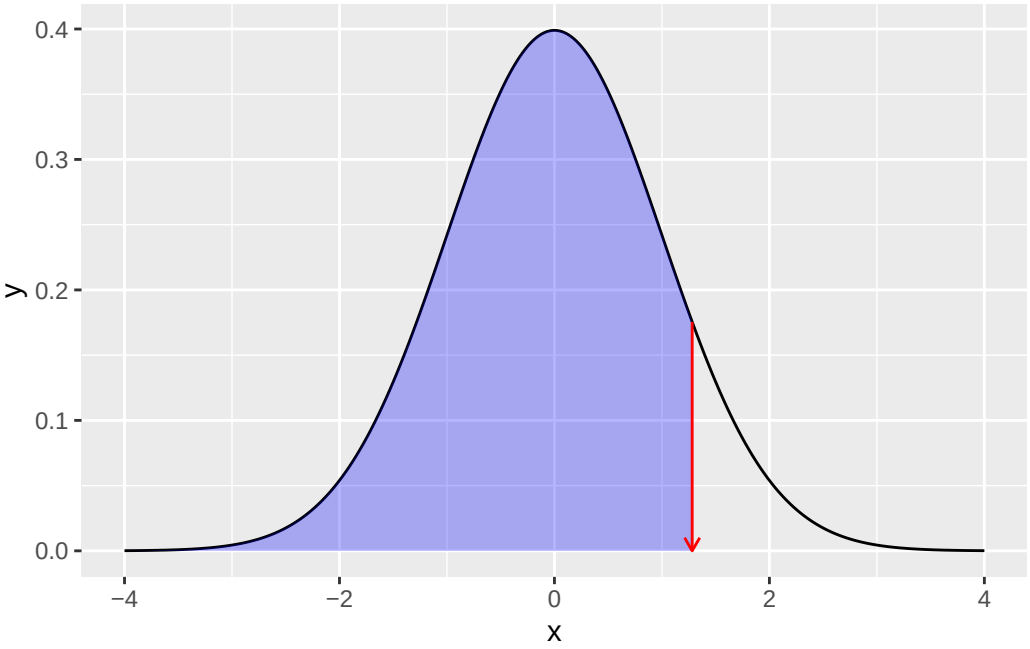
```
# 累積分布の逆関数
qnorm(0.975, mean = 0, sd = 1)
```

```
[1] 1.959964
```

数値で直感的にわかりにくい場合，次の図を見て確認しよう。pnorm 関数は x 座標の値を与えると，そこまでの面積 (以下のコードで描かれる色付きの領域) すなわち確率を返す。qnorm 関数は確率 (=面積) を与えると，確率密度関数のカーブの下領域を積分してその値になるときの x 座標の値を返す。

```
# 描画
prob <- 0.9
## 全体の正規分布カーブ
df1 <- data.frame(x = seq(from = -4, 4, by = 0.01)) %>%
  mutate(y = dnorm(x, mean = 0, sd = 1))
## qnorm(0.975)までのデータ
df2 <- data.frame(x = seq(from = -4, qnorm(prob), by = 0.01)) %>%
```

```
  mutate(y = dnorm(x, mean = 0, sd = 1))
## データセットの違いに注意
ggplot() +
  geom_line(data = df1, aes(x = x, y = y)) +
  geom_ribbon(data = df2, aes(x = x, y = y, ymin = 0, ymax = y), fill = "blue", alpha = 0.3) +
  ## 以下装飾
  geom_segment(
    aes(x = qnorm(prob), y = dnorm(qnorm(prob)), xend = qnorm(prob), yend = 0),
    arrow = arrow(length = unit(0.2, "cm")), color = "red"
  )
```



d,p,q,r といった頭の文字は，他の確率分布関数にも付く。では次に r について説明しよう。

6.3 乱数

乱数とは何であるかを説明するのは，「ランダムである (確率変数である) とは如何なることか」を説明するのと同じように難しい。カンタンに説明するなら，規則性のない数列という意味である。しかし計算機はアルゴリズムに沿って正しく数値を計算するものだから，ランダムに，規則性がない数字を示すということは厳密にはあり得ない。計算機が出す乱数は，乱数生成アルゴリズムに沿って出される数字であり，ランダムに見えて実は規則性があるので，疑似乱数というのが正しい。

とはいえ，人間が適当な数字を思いつきで誦じていく[*3]よりは，よほど規則性がない数列を出すので，疑似的とはいえ十分に役に立つ。たとえばアプリなどで「ガチャ」を引くというのは，内部で乱数によって数値を出し，それ

[*3] 厳密なエビデンスは示せないが，俗に「嘘のゴサンパチ」というように人間が適当に数字を述べると 5,3,8 が使われる率がチャンスレベルより高いと言われている。

に基づいてあたり・ハズレ等の判定をしている。他にも，RPG などで攻撃する時に一定の確率で失敗するとか，一定の確率で「会心の一撃」を出すというのも同様である。ここで大事なのは，そうしたゲームへの実装において規則性のない数字に基づくプログラムにしたとしても，その統計的な性質，すなわち実現値の出現確率はある程度制御したいのである。

そこで，ある確率分布に基づく乱数を生成したい，ということになる。幸いにして，一様乱数 (全ての実現値が等しい確率で生じる) を関数で変換することで，正規分布ほか様々な確率分布に従う乱数を作ることができる。R にはその基本関数として幾つかの確率分布に従う乱数が実装されている。たとえば次のコードは，平均 50，SD10 の正規分布に従う乱数を 10 個出現させるものである。

```
rnorm(n = 10, mean = 50, sd = 10)
```

```
 [1] 64.86061 62.45877 71.94247 48.74745 44.17199 50.71921 48.45129 51.47301
 [9] 57.23399 42.29301
```

たとえば諸君が心理統計の練習問題を作ろうとして，適当な数列が欲しければこのようにすれば良いかもしれない。しかし，同じ問題をもう一度作ろうとすると，乱数なのでまた違う数字が出てしまう。

```
rnorm(n = 10, mean = 50, sd = 10)
```

```
 [1] 48.72109 56.07326 37.65196 37.18492 35.26374 49.84208 57.60022 61.24018
 [9] 44.24463 20.34663
```

疑似乱数に過ぎないのだから，再現性のある乱数を生じさせたいと思うかもしれない。そのような場合は，`set.seed` 関数を使う。疑似乱数は内部の乱数生成の種 (seed) から計算して作られているため，その数字を固定してやると同じ乱数が再現できる。

```
# seedを指定
set.seed(12345)
rnorm(n = 3)
```

```
[1]  0.5855288  0.7094660 -0.1093033
```

```
# 同じseedを再設定
set.seed(12345)
rnorm(n = 3)
```

```
[1]  0.5855288  0.7094660 -0.1093033
```

6.3.1　乱数のつかいかた

乱数の使い方のひとつは，先に述べたように，プログラムが偶然による振る舞いをしているように仕掛けたいとき，ということだろう。

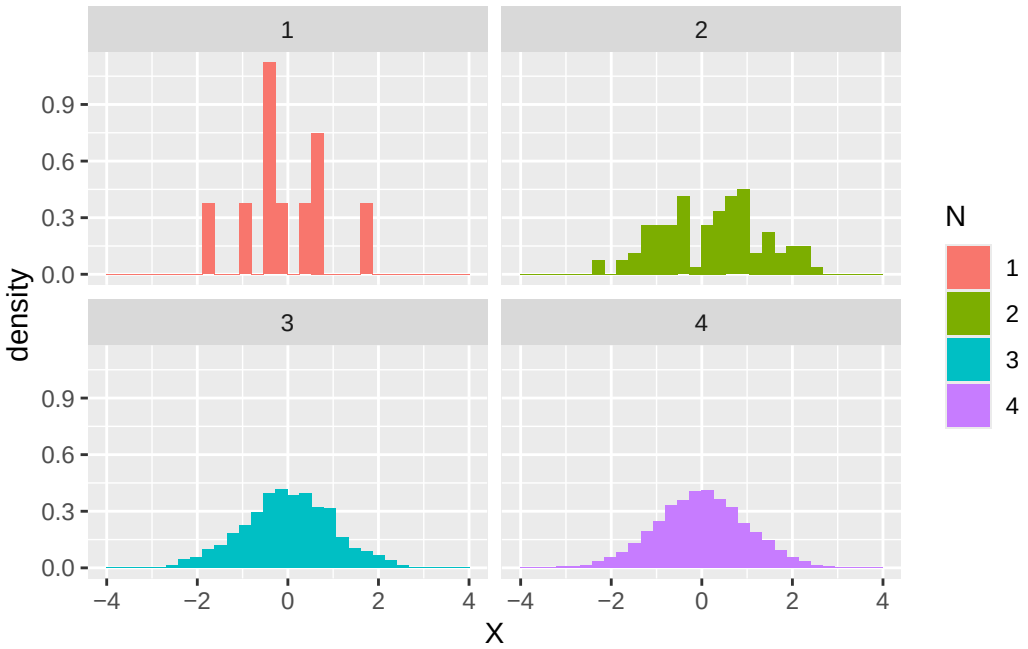
実は他にも使い道がある。それは確率分布を具体的に知りたいときである。次に示すのは，標準正規分布から $n = 10, 100, 1000, 10000$ とした時のヒストグラムである。

```
rN10 <- rnorm(10)
rN100 <- rnorm(100)
rN1000 <- rnorm(1000)
rN10000 <- rnorm(10000)

data.frame(
  N = c(
    rep(1, 10), rep(2, 100),
    rep(3, 1000), rep(4, 10000)
  ),
  X = c(rN10, rN100, rN1000, rN10000)
) %>%
  mutate(N = as.factor(N)) %>%
  ggplot(aes(x = X, fill = N)) +
  # 縦軸を相対頻度に
  geom_histogram(aes(y = ..density..)) +
  facet_wrap(~N)
```

```
Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
i Please use `after_stat(density)` instead.
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



これを見ると，最初の 10 個程度のヒストグラムは不規則な分布に見えるが，100,1000 と増えるに従って徐々に正規分布の理論的形状に近似していくところがみて取れる。

R にはポアソン分布や二項分布などに加え，統計に馴染みの深い t 分布や F 分布，χ^2 分布などの確率分布関数も実装されている。これらの分布はパラメタの値を聞いてもイメージしにくいところがあるかもしれないが，そのような時はパラメタを指定した上で乱数を大量に生成し，そのヒストグラムを描けば確率分布関数の形が眼に見えてくるため，より具体的に理解できるだろう。

実際，ベイズ統計学が昨今隆盛している一つの理由は，計算機科学の貢献によるところが大きい。**マルコフ連鎖モンテカルロ法** (MCMC 法) と呼ばれる乱数発生技術は，明確な名前を持たないモデルによって作られる事後分布からでも，乱数を生成できる技術である。この分布は解析的に示すことは困難であるが，そこから乱数を生成し，そのヒストグラムを見ることで，形状を可視化できるのである。

また，この乱数利用法の利点は可視化だけではない。標準正規分布において，ある範囲の面積 (=確率) が知りたいとする。たとえば，確率点-1.5 から +1.5 までの範囲の面積を求めたいとしよう。正規分布の数式はわかっているので，次のようにすればその面積は求められる。

$$p = \int_{-1.5}^{+1.5} \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} dx$$

もちろん我々は pnorm 関数を知っているので，次のようにして数値解を得ることができる。

```
pnorm(+1.5, mean = 0, sd = 1) - pnorm(-1.5, mean = 0, sd = 1)
```

```
[1] 0.8663856
```

同様のことは乱数を使って，次のように近似解を得ることができる。

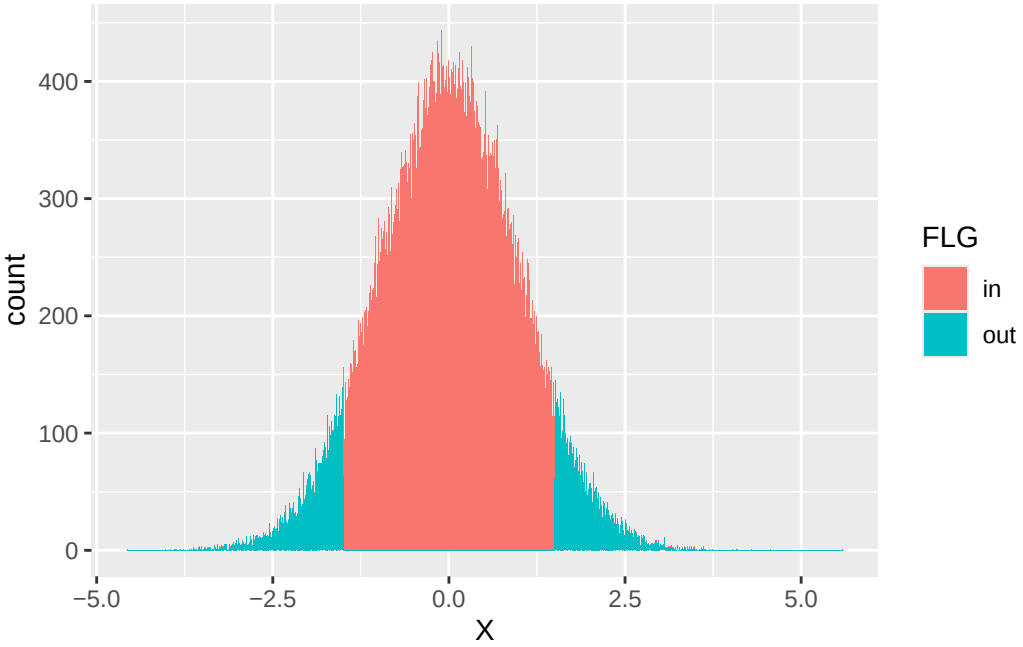
```
x <- rnorm(100000, mean = 0, sd = 1)
df <- data.frame(X = x) %>%
  # 該当する範囲かどうかを判定する変数を作る
  mutate(FLG = ifelse(X > -1.5 & X < 1.5, 1, 2)) %>%
  mutate(FLG = factor(FLG, labels = c("in", "out")))
## 計算
df %>%
  group_by(FLG) %>%
  summarise(n = n()) %>%
  mutate(prob = n / 100000)
```

```
# A tibble: 2 x 3
  FLG       n  prob
  <fct> <int> <dbl>
1 in    86642 0.866
2 out   13358 0.134
```

ここでは乱数を 10,000 個生成し，指定の範囲内に入るかどうか (入れば 1, 入らなければ 2) を示す factor 型変数 FLG を作った。この変数ごとに群分けして数を数え，総数で割ることで相対度数にする。確率は全体の中に占める相対的な面積の割合であり，今回当該領域の値が 0.866 と pnorm 関数で算出した解とほぼ同等の値変えられている。

なお，次のようにすれば範囲の可視化も容易い。

```
## 可視化
df %>%
  ggplot(aes(x = X, fill = FLG)) +
  geom_histogram(binwidth = 0.01)
```



繰り返すが，確率分布の形がイメージできなかったり，解析的にその式を書き表すことが困難であった場合でも，具体的な数値にすることでヒストグラムで可視化でき，また近似的に確率計算ができている。

あくまでも近似に過ぎないのでその精度が信用できない，というひとは生成する乱数の数を 10 倍，100 倍にすれば良い。昨今の計算機の計算能力において，その程度の増加はさほど計算料の負担にならない。複雑な積分計算が記述統計量 (数え上げ) の問題になる点で，具体的に理解できるという利点は大きい。

さらに思いを馳せてほしいのだが，心理学者は心理学実験や調査によって，データを得る。しかしそれらは個人差や誤差を考え，確率変数だとされている。目の前の数件から数十件のデータであっても，正規分布に従うと仮定して統計的処理をおこなう。これは「乱数によって生成したデータ」に対して行うとしても本質的には同じである。すなわち，調査実験を行う前に，乱数によってシミュレーションしておくことができるのである。調査実験の本番一発勝負をする前に，自分の取ろうとしているデータがどのような性質を持ちうるかを具体的に確かめておくことは重要な試みであろう。

6.4　練習問題；乱数を用いて

正規乱数を用いて，次の値を近似計算してみよう。なお設定や解析的に算出した「真の値」と少数以下 2 位までの精度が得られるように工夫しよう。

1. 平均 100, 標準偏差 8 の正規分布の期待値。なお連続確率変数の期待値は次の式で表されます。

$$E[X] = \int_{-\infty}^{\infty} x f(x)\, dx$$

ここで x は確率変数を表し，$f(x)$ は確率密度関数であり，確率密度関数の全定義域を積分することで得られます。正規分布の期待値は，平均パラメータに一致しますので，今回の真値は設定した 100 になります。

2. 平均 100, 標準偏差 3 の正規分布の分散を計算してみよう。なお連続確率変数の分散は次の式で表されます。

$$\sigma^2 = \int_{-\infty}^{\infty} (x-\mu)^2 f(x)\, dx$$

ここで μ は確率変数の期待値であり，正規分布の分散は，標準偏差パラメータの二乗に一致しますので，今回の真値は $3^2 = 9$ です。

3. 平均 65，標準偏差 10 の正規分布に従う確率変数 X の，$90 < X < 110$ の面積。解析的に計算した結果は次の通りです。

```
pnorm(110, mean = 65, sd = 10) - pnorm(90, mean = 65, sd = 10)
```

```
[1] 0.006206268
```

4. 平均 10，標準偏差 10 の正規分布において，実現値が 7 以上になる確率。解析的に計算した結果は次の通りです。

```
1 - pnorm(7, mean = 10, sd = 10)
```

```
[1] 0.6179114
```

5. 確率変数 X, Y があります。X は平均 10,SD10 の正規分布，Y は平均 5，SD8 の正規分布に従うものとします。ここで，X と Y が独立であるとしたとき，和 $Z = X + Y$ の平均と分散が，もとの X, Y の平均の和，分散の和になっていることを，乱数を使って確認してください。

6.5　母集団と標本

ここまで確率分布の性質を見るために乱数を利用する方法を見てきた。ここからは，推測統計学における確率分布の利用を考える。推測統計では，知りたい集団全体のことを**母集団 population**，そこから得られた一部のデータを**標本 sample** と呼ぶのであった。標本の統計量を使って，母集団の性質を推論するのが推測統計/統計的推測である。母集団の特徴を表す統計量は**母数 parameter** と呼ばれ，母平均，母分散など「母」の字をつけて母集団の情報であることを示す。同様に，標本の平均や分散も計算できるが，この時は標本平均，標本分散など「標本」をつけて明示的に違いを強調することもある。

乱数を使って具体的な例で見てみよう。ここに 100 人から構成される村があったとする。この村の人々の身長を測ってデータにしたとしよう。100 個の適当な数字を考えるのは面倒なので，乱数で生成してこれに代える。

```
set.seed(12345)
# 100人分の身長データをつくる。小数点以下2桁を丸めた
Po <- rnorm(100, mean = 150, sd = 10) %>% round(2)
print(Po)
```

```
  [1] 155.86 157.09 148.91 145.47 156.06 131.82 156.30 147.24 147.16 140.81
 [11] 148.84 168.17 153.71 155.20 142.49 158.17 141.14 146.68 161.21 152.99
 [21] 157.80 164.56 143.56 134.47 134.02 168.05 145.18 156.20 156.12 148.38
 [31] 158.12 171.97 170.49 166.32 152.54 154.91 146.76 133.38 167.68 150.26
 [41] 161.29 126.20 139.40 159.37 158.54 164.61 135.87 155.67 155.83 136.93
 [51] 144.60 169.48 150.54 153.52 143.29 152.78 156.91 158.24 171.45 126.53
 [61] 151.50 136.57 155.53 165.90 144.13 131.68 158.88 165.93 155.17 137.04
 [71] 150.55 142.15 139.51 173.31 164.03 159.43 158.26 141.88 154.76 160.21
 [81] 156.45 160.43 146.96 174.77 159.71 168.67 156.72 146.92 155.37 158.25
 [91] 140.36 141.45 168.87 146.08 140.19 156.87 144.95 171.58 144.00 143.05
```

この 100 人の村が母集団なので，母平均や母分散は次のようにして計算できる。

```
M <- mean(Po)
V <- mean((Po - M)^2)
# 母平均
print(M)
```

```
[1] 152.4521
```

```
# 母分散
print(V)
```

```
[1] 123.0206
```

さて，この村からランダムに 10 人の標本を得たとしよう。ベクトルの前から 10 人でも良いが，R にはサンプリングをする関数 sample があるのでこれを活用する。

```
s1 <- sample(Po, size = 10)
s1
```

```
 [1] 164.61 155.86 136.93 143.29 160.43 168.87 151.50 155.17 153.71 135.87
```

この s1 が手元のデータである。心理学の実験でデータを得る，というのはこのように全体に対してごく一部だけ取り出したものになる。このサンプルの平均や分散は標本平均，標本分散である。

```
m1 <- mean(s1)
v1 <- mean((s1 - mean(s1))^2)
# 標本平均
print(m1)
```

```
[1] 152.624
```

```
# 標本分散
print(v1)
```

```
[1] 110.2049
```

今回，母平均は 152.4521 で標本平均は 152.624 である。実際に知りうる値は標本の値だけなので，標本平均 152.624 を得たら，母平均も 152.624 に近い値だろうな，と推測するのはおかしなことではないだろう。しかし標本平均は，標本の取り方によって毎回変わるものである。試しにもう一つ，標本をとったとしよう。

```
s2 <- sample(Po, size = 10)
s2
```

```
 [1] 154.76 135.87 143.05 171.45 136.57 170.49 156.87 158.25 155.17 155.20
```

```
m2 <- mean(s2)
v2 <- mean((s2 - mean(s2))^2)
# 標本平均その2
print(m2)
```

```
[1] 153.768
```

今回の標本平均は 153.768 になった。このデータが得られたら，諸君は母平均が「153.768 に近い値だろうな」と推測するに違いない。標本 1 の 152.624 と標本 2 の 153.768 を比べると，前者の方が正解 152.4521 に近い (その差はそれぞれ-0.1719 と-1.3159 である)。つまり，標本の取り方によっては当たり外れがあるということである。データをとって研究していても，仮説を支持する結果なのかそうでないのかは，こうした確率的揺らぎの下にある。

つまり，**標本は確率変数であり，標本統計量も確率的に変わりうる**ものである。標本統計量でもって母数を推定するときは，標本統計量の性質や標本統計量が従う確率分布を知っておく必要がある。以下では母数の推定に望ましい性質を持つ推定量の望ましい性質をみていこう。

6.6　一致性

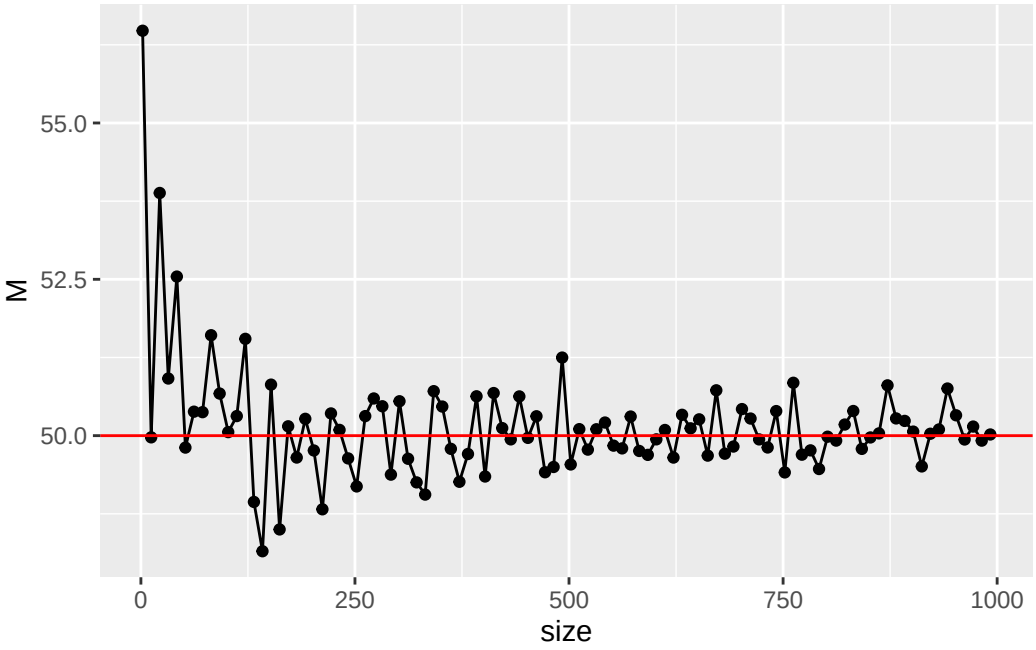
最も単純には，標本統計量が母数に近ければ近いほど，できれば一致してくれれば喜ばしい。先ほどの例では 100 人の村から 10 人しか取り出さなかったが，もし 20 人，30 人とサンプルサイズが大きくなると母数に近づいていくことが予想できる。この性質のことを**一致性** consistency といい，推定量が持っていてほしい性質のひとつである。幸い，標本平均は母平均に対して一致性を持っている。

このことを確認してみよう。サンプルサイズを様々に変えて計算してみれば良い。例として，平均 50,SD10 の正規分布からサンプルサイズを 2 から 1000 まで増やしていくことにしよう。サンプルを取り出すことを，乱数生成に置き換えてその平均を計算していくこととする。

```
set.seed(12345)
sample_size <- seq(from = 2, to = 1000, by = 10)
# 平均値を格納するオブジェクトを初期化
sample_mean <- rep(0, length(sample_size))
# 反復
for (i in 1:length(sample_size)) {
  sample_mean[i] <- rnorm(sample_size[i], mean = 50, sd = 10) %>%
```

```
    mean()
}

# 可視化
data.frame(size = sample_size, M = sample_mean) %>%
  ggplot(aes(x = size, y = M)) +
  geom_point() +
  geom_line() +
  geom_hline(yintercept = 50, color = "red")
```



このようにサンプルサイズが増えていくにつれて，真値の 50 に近づいていくことが見て取れる。母集団分布の形状やパラメータ，サンプルサイズなどを変えて確認してみよう。

6.7　不偏性

推定量は確率変数であり，確率分布でその性質を記述することができる。標本統計量の従う確率分布のことを**標本分布**と呼ぶが，標本分布の確率密度関数がわかっているなら，その期待値や分散も計算できるだろう。推定量の期待値 (平均) が母数に一致することも，推定量の望ましい性質の一つであり，この性質のことを**不偏性** unbiasedness という。

心理統計を学ぶ時に初学者を苛立たせるステップの一つとして，分散の計算の時にサンプルサイズ n ではなく $n-1$ で割る，という操作がある。これは不偏分散といって標本分散とは違うのだが，前者が不偏性を持っているのに対し，後者がそうでないからである。これを乱数を使って確認してみよう。

平均 50，SD10(母分散 $10^2 = 100$) の母集団から，サンプルサイズ $n = 20$ の標本を繰り返し得る。これはサイズ 20 の乱数生成で行う。各標本に対して標本分散と不偏分散を計算し，その平均 (標本統計量の期待値) を計算して

みよう。

```
iter <- 5000
vars <- rep(0, iter)
unbiased_vars <- rep(0, iter)

## 乱数の生成と計算
set.seed(12345)
for (i in 1:iter) {
  sample <- rnorm(n = 20, mean = 50, sd = 10)
  vars[i] <- mean((sample - mean(sample))^2)
  unbiased_vars[i] <- var(sample)
}

## 期待値
mean(vars)
```

```
[1] 95.08531
```

```
mean(unbiased_vars)
```

```
[1] 100.0898
```

標本分散を計算したオブジェクト vars の平均すなわち期待値は 95.0853144 であり，設定した値 (真値) の 100 からは幾分はなれている。これに対して，R の埋め込み関数である var をつかった不偏分散の平均すなわち期待値は 100.0898047 であり，母分散の推定量としてはこちらの方が好ましいことがわかる。このように標本分散にはバイアスが生じることがわかっているので，あらかじめバイアスを補正するために元の計算式を修正していたのである。この説明で，苛立ちを感じていた人の溜飲が下がればよいのだが。

他にも推定量の望ましい性質として有効性 efficacy があるが，詳細は小杉他 (2023) を参照してほしい。この本には正規分布以外の例や，相関係数など他の標本統計量の例なども載っているが，いずれも乱数生成による近似で理解を進めるものである。諸君も数理統計的な説明に疲れたなら，ぜひ参考にしてもらいたい。

6.8　信頼区間

標本統計量は確率変数であり，標本を取るたびに変わる。標本を取るときに入る確率的ゆらぎによるからで，標本平均は一致性，不偏性という望ましい性質を持ってはいるが，標本平均 = 母平均とはならない。

標本平均という確率変数の実現値一点でもって，母平均を推測することは，母平均を推測する上ではほぼ確実に外れるギャンブルである。そこで母数に対してある幅でもって推定することを考えよう。

たとえば平均 50，標準偏差 10 の標準正規分布を母集団分布とし，サンプルサイズ 10 の標本をとり，その標本平均を母平均の推定値としよう (点推定)。同時に，その推定値に少し幅を持たせ，たとえば標本平均 ± 5 の**区間推定**をしたとする。この時，真値 0 を正しく推測できる確率を，反復乱数生成のシミュレーションで確かめてみよう。

```r
iter <- 10000
n <- 10
mu <- 50
SD <- 10

# 平均値を格納しておくオブジェクト
m <- rep(0, iter)

set.seed(12345)
for (i in 1:iter) {
  # サンプリングし，標本統計量を保存
  sample <- rnorm(n, mean = mu, sd = SD)
  m[i] <- mean(sample)
}

result.df <- data.frame(m = m) %>%
  # 推定が一致するとTRUE,外れるとFALSEになる変数を作る
  mutate(
    point_estimation = ifelse(m == mu, TRUE, FALSE),
    interval_estimation = ifelse(m - 5 <= mu & mu <= m + 5, TRUE, FALSE)
  ) %>%
  summarise(
    n1 = sum(point_estimation),
    n2 = sum(interval_estimation),
    prob1 = mean(point_estimation),
    prob2 = mean(interval_estimation)
  ) %>%
  print()
```

```
  n1   n2 prob1 prob2
1  0 8880     0 0.888
```

結果からわかるように，点推定値は一度も正しく母数を当てていない。これは当然で，実数でやる以上小数点以下どこかでズレてしまうことがあるからで，精度を無視すると一致することはあり得ないのである。これに対して幅を持った予測の場合は，10^4 回の試行のうち 8880 回はその区間内に真値を含んでおり，その正答率は 88.8% である。

区間推定において正答率を 100% にするためには，その区間を無限に広げなければならない (母平均の推定の場合)。これは実質的に何も推定していないことに等しいので，5% 程度の失敗を認めよう，95% の正答率で区間推定しようというのが習わしになっている。この区間のことを 95% の**信頼区間** confidence interval という。

6.8.1　正規母集団分布の母分散が明らかな場合の信頼区間

上のシミュレーションを応用して，区間推定が正当する確率が 95% になるまで区間を調整して行ってもよいが，さすがにそれは面倒なので，推測統計学によって明らかになっている性質を紹介しよう。

母集団が正規分布に従い，その母平均が μ，母分散が σ^2 であることがわかっている場合，標本平均の従う分布は平均 μ, 分散 $\frac{\sigma^2}{n}$(標準偏差 $\frac{\sigma}{\sqrt{n}}$) の正規分布であることがわかっている。

標準正規分布の 95% 区間は，次の通り約 ± 1.96 である。

```
# 両端から2.5%ずつ取り除くと
qnorm(0.025)
```

```
[1] -1.959964
```

```
qnorm(0.975)
```

```
[1] 1.959964
```

これらを合わせると，標本平均が $\bar{X}$ であったとき，95% 信頼区間は標準偏差を 1.96 倍して，次のようになる。

$$\bar{X} - 1.96\frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X} + 1.96\frac{\sigma}{\sqrt{n}}$$

先ほどの数値例を応用して，これを確かめてみよう。95 %ちかい割合で，区間内に真値が含んでいることがわかる。

```
interval <- 1.96 * SD / sqrt(n)
result.df2 <- data.frame(m = m) %>%
  # 推定が一致するとTRUE,外れるとFALSEになる変数を作る
  mutate(
    interval_estimation = ifelse(m - interval <= mu & mu <= m + interval, TRUE, FALSE)
  ) %>%
  summarise(
    prob = mean(interval_estimation)
  ) %>%
  print()
```

```
    prob
1 0.9498
```

6.8.2 正規母集団分布の母分散が不明な場合の信頼区間

先ほどの例では母分散がわかっている場合の例であったが，母平均や母分散がわかっていれば推測する必要はないわけで，実践的には母分散がわからない場合の推定が必要になってくる。幸いにしてそのような場合，すなわち母分散を不偏分散 (標本統計量) で置き換えた場合は，標本平均が自由度 $n-1$ の t 分布に従うことがわかっている。(詳細は小杉他 (2023) を参照) ただその場合，標準正規分布のように 95% 区間が ± 1.96 に限らず，サンプルサイズに応じて t 分布の形が変わるから，それを考慮して以下の式で信頼区間を算出する。

$$\bar{X} + T_{0.025}\frac{U}{\sqrt{n}} \leq \mu \leq \bar{X} + T_{0.975}\frac{U}{\sqrt{n}}$$

ここで $T_{0.025}$ は t 分布の 2.5 パーセンタイル，$T_{0.975}$ は 97.5 パーセンタイルを指す。t 分布は (平均が 0 であれば) 左右対称なので，$T_{0.025} = -T_{0.975}$ と考えても良い。また U^2 は不偏分散である (U はその平方根)。

これも乱数による近似計算で確認しておこう。同じく 95 %ちかい割合で，区間内に真値が含まれていることがわかる。

```
# シミュレーションの設定
iter <- 10000
n <- 10
mu <- 50
SD <- 10

# 平均値を格納しておくオブジェクト
m <- rep(0, iter)
interval <- rep(0, iter)

set.seed(12345)
for (i in 1:iter) {
  # サンプリングし，標本統計量を保存
  sample <- rnorm(n, mean = mu, sd = SD)
  m[i] <- mean(sample)
  U <- sqrt(var(sample)) # sd(sample)でも同じ
  interval[i] <- qt(p = 0.975, df = n - 1) * U / sqrt(n)
}

result.df <- data.frame(m = m, interval = interval) %>%
  # 推定が一致するとTRUE,外れるとFALSEになる変数を作る
  mutate(
    interval_estimation = ifelse(m - interval <= mu & mu <= m + interval, TRUE, FALSE)
  ) %>%
  summarise(
```

```
    prob = mean(interval_estimation)
  ) %>%
  print()
```

```
    prob
1 0.9482
```

6.9　課題

1. 算術平均 $M = \frac{1}{n}\sum x_i$ が一致推定量であることが示されましたが，調和平均 $HM = \frac{n}{\sum \frac{1}{x_i}}$ や幾何平均 $GM = (\prod x_i)^{\frac{1}{n}} = \exp(\frac{1}{n}\sum \log(x_i)))$ はどうでしょうか。シミュレーションで確かめてみましょう。
2. サンプルサイズ n が大きくなるほど，標本平均が母平均に近づくという性質は正規分布以外でも成立するでしょうか。自由度 $\nu = 3$ の t 分布を使って，シミュレーションで確認してみましょう。なお t 分布の乱数は `rt()` で生成でき，非心度パラメータ `ncp` を指定しなければその平均は 0 です。
3. t 分布の自由度 ν が極めて大きい時は，標準正規分布に一致することがわかっています。`rt()` 関数を使って自由度が 10,50,100 のときの乱数を 1000 個生成し，ヒストグラムを書いてその形状を確認しましょう。また乱数の平均と標本標準偏差を計算し，標準正規分布に近づくことを確認しましょう。
4. 平均が 50，標準偏差が 10 の正規分布からサンプルサイズ 20 の乱数を 10000 個生成し，quantile 関数をつかって 95 %信頼区間をシミュレートしてください。理論値と比較して確認してください。
5. 平均が 100，標準偏差が 15 の正規分布から抽出された標本について，サンプルサイズを 10，100，1000 と変えたときの標本平均の 95% 信頼区間の幅を比較してください。

第 7 章

統計的仮説検定

帰無仮説検定は，心理学における統計の利用シーンの代表的なものだろう。その手順は形式化されており，統計パッケージによってはデータの種類を指定するだけで自動的に結果の記述までしてくれるものもあるほどである。誰がやっても同じ結果になり，また，機械的に手続きを自動化できることは大きな利点ではある。欠点は，初学者がそのメカニズムを十分に理解せずに誤った結果を得たり，悪意のある利用者が自分に都合の良い数字を出させたりすることにある。科学的営みは悪意をもった実践者を想定しておらず，もしそのような悪例が露見した場合には事後的に摘発・対処するしかない。しかし残念なことながら，初学者の浅慮や意図せぬ悪用も多くみられる。

心理学において，先行研究の結果が再現しないことを再現性問題というが，そのひとつは統計的手法の誤った使い方にあるとされる (池田・平石，2016)。改めて，丁寧に帰無仮説検定の手続きやロジックを見ていくことにしよう。

7.1　帰無仮説検定の理屈と手続き

7.1.1　帰無仮説検定の目的

帰無仮説検定は，実験や調査で得たデータから得られた知見が意味のあるものかどうか，母集団の性質として一般化可能かどうかを判定するための枠組みである。手法と判断基準が明確なゲームの一種だと考えたよう。というのも，帰無仮説検定は**有意水準**という基準を設けて，**帰無仮説**と**対立仮説**という 2 つの考え方 (モデル) を対決させ，勝敗を決するものだからである。勝敗を決するとしたのは，帰無仮説と対立仮説は排他的な関係にあるからであり，どちらも正しいとかどちらも間違っているという結末にはならないからである。ただし，あくまでも推測統計的なロジックに基づく判定であるから，判定結果にも確率的な要素が含まれる。本当は帰無仮説が正しい時に，間違って「対立仮説が正しい」と判定してしまう確率はゼロではない。逆に帰無仮説が正しくない時に，間違って「対立仮説が正しくない (帰無仮説が正しい)」と判定してしまう可能性もある。前者を**タイプ 1 エラー**，後者を**タイプ 2 エラー**という。どちらの確率もゼロであってほしいが，そうはならないので，前者を α，後者を β としたときに，それぞれを一定の水準以下に抑えたい。この目的のために手順を整え，一般化したのが帰無仮説検定である。なお，先に述べた有意水準は，この α の許容される水準であり，心理学では一般に 5% に設定する。

このように帰無仮説検定という考え方は，エラーの統制が本来の狙いであるから，「有意になるように工夫する」という発想は根本的に間違っている。また，統計的推定という数学的手続きに，人間が納得しやすい判定を下すと

いう人為的手続きが組み合わさったものであるから，帰無仮説検定の結果に過剰な意味を持たせたり一喜一憂したりすることがないように注意しよう。

7.1.2　帰無仮説検定の手続き

帰無仮説検定の手続きを一般化すれば，次のようになる。

1. 帰無仮説と対立仮説を設定する。
2. 検定統計量を選択する。
3. 判定基準を決定する。
4. 検定統計量を計算する。
5. 判定する。

帰無仮説検定は，群間の平均値に差があるかどうか，相関係数に統計的な意味があるかどうかといった事例に対して適用される。当然のことながら，これは標本から母集団を推定するという文脈における話で，物理学的な真偽を理論的に判断するとか，全数調査のように母集団全体の情報が手に入る場合といった場合の話ではない。また，標本のサンプルサイズが小さく，標本統計量の信頼区間が大きいことから，枠組みなしには判定できないという背景があることも再確認しておこう。

母集団の状態がわからないので，仮説を設定する。帰無仮説 Null Hypothesis は空っぽの仮説という意味で，母平均差がない (差がゼロ，$\mu_1 - \mu_2 = 0$) とか，母相関がゼロ ($\rho = 0$) である，とされる。対立仮説 Alternative Hypothesis は帰無仮説と排他的な関係にある仮説としてつくられるから，「差が無くはない ($\mu_1 - \mu_2 \neq 0$)」「相関がゼロではない ($\rho \neq 0$)」という表現になる。なぜ帰無仮説がゼロであることから始められるかといえば，ふたつの排他的な仮説を考えた時にゼロでない状態というのは無数にあり得るので，仮説として特定できないからである (差が 1 のとき，1.1 のとき，1.11 のとき・・・と延々と検定し続けるわけにもいくまい)。

検定統計量の選択は，二群の平均値差のときは t，三群以上の時は F，相関係数の検定も t，と天下り的に示されることが一般的である。もちろんこれらの統計量が選ばれるのは，数理統計的な論拠に基づいている。判定基準は 5% 水準とすることが一般的だし，検定統計量の計算はアルゴリズムに沿って機械的に可能である。判定は客観的な指標に基づいて行われるから，「どの状況でどのような帰無仮説をおくか」が類型化できれば，この手続き全体が自動的に進められる。

しかしここでは改めて，丁寧に手順を追いながらみてみよう。

7.2　相関係数の検定

ここでは相関係数の検定を例に取り上げる。俗に「無相関検定」と呼ばれるように，相関がどれほど大きいとかどれほど意味があるということをチェックするのでは無く，無相関ではない，ということをチェックする。もちろん標本相関は計算してゼロでなければ，それは無相関ではない。ここで考えたいのは，母相関がゼロではないということである。言い換えると，母相関がゼロの状態であっても，標本相関がゼロでないことは，小標本のサンプリングという背景のもとでは当然のことである。

確認してみよう。まず，無相関なデータセットを作ることを考える。R の `MASS` パッケージを使い，多変量正規分

布の確率分布関数から乱数を生成しよう。

```
pacman::p_load(MASS)
set.seed(12345)
N <- 100000
X <- mvrnorm(N,
  mu = c(0, 0),
  Sigma = matrix(c(1, 0, 0, 1), ncol = 2),
  empirical = TRUE
)
head(X)
```

```
           [,1]        [,2]
[1,] -0.4070308 -0.72271139
[2,] -0.5774631 -0.57075167
[3,]  0.2312929 -0.42458994
[4,]  0.6242499 -0.55522146
[5,] -0.7791585  0.55004824
[6,]  1.8995860 -0.04899946
```

ここでは 10^5 個の乱数を生成した。つくられたオブジェクト X は表示されているように，2 変数からなる。ここでは相関のある 2 変数を想定しており，各変数がそれぞれ標準正規分布に従っているという設定である。rnorm 関数を 2 つ使って 2 変数をつくっても良いのだが，2 変数セットで取り出すことを考えると多変量正規分布をかんがえることになる。多変量正規分布は，ひとつひとつの変数については正規分布として平均と SD をもち，かつ，変数の組み合わせとして共分散をもつものである。mvrnorm の引数をみると，mu は平均ベクトルであり，Sigma が分散共分散行列である。分散共分散行列とは，ここでは 2×2 の正方行列であり，対角項に分散を，非対角項に共分散をもつ行列である。共分散は標準偏差と相関係数の積で表される。

分散

$$s_x^2 = \frac{1}{n}\sum (x_i - \bar{x})^2 = \frac{1}{n}\sum (x_i - \bar{x})(x_i - \bar{x})$$

標準偏差

$$s_x = \sqrt{s_x^2} = \sqrt{\frac{1}{n}\sum (x_i - \bar{x})^2}$$

共分散

$$s_{xy} = \frac{1}{n}\sum (x_i - \bar{x})(y_i - \bar{y})$$

相関係数

$$r_{xy} = \frac{s_{xy}}{s_x s_y} = \frac{\frac{1}{n}\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\frac{1}{n}\sum(x_i - \bar{x})^2}\sqrt{\frac{1}{n}\sum(y_i - \bar{y})^2}}$$

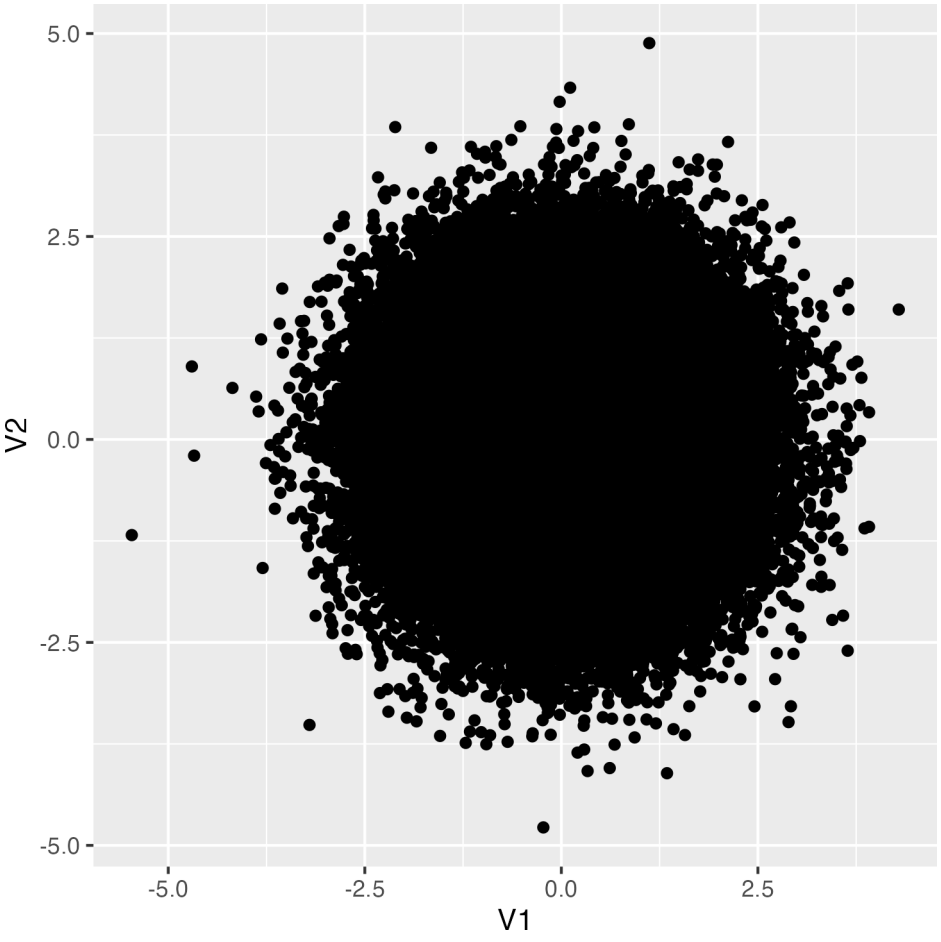
分散共分散行列

$$\Sigma = \begin{pmatrix} s_x^2 & s_{xy} \\ s_{yx} & s_y^2 \end{pmatrix} = \begin{pmatrix} s_x^2 & r_{xy}s_x s_y \\ r_{xy}s_x s_y & s_y^2 \end{pmatrix}$$

今回 `Sigma = matrix(c(1,0,0,1),ncol = 2)` としたのは，この 2 変数が無相関であること (SD はそれぞれ 1 であること) を指定している。ちなみに `empirical = TRUE` のオプションは，生成された乱数が設定した分散共分散行列のもつ相関係数と一致するように補正することを意味している。

可視化しておこう。つくられた乱数が無相関であることを，散布図を使って確認する。

```
pacman::p_load(tidyverse)
X %>%
  as.data.frame() %>%
  ggplot(aes(x = V1, y = V2)) +
  geom_point()
```



数値的にも確認しておこう。

```
cor(X) %>% round(5)
```

```
     [,1] [,2]
[1,]    1    0
[2,]    0    1
```

つくられた乱数が無相関であることが確認できた。さてこれが母集団であったとして，ここからたとえば n = 20 のサンプルをとったとする。この時の相関はどうなるだろうか。R で計算してみよう。sample 関数をつかって抜き出す行を決めて，該当する行だけ s1 オブジェクトに代入する。その上で相関係数を計算してみよう。

```
selected_row <- sample(1:N, 20)
print(selected_row)
```

```
 [1]  9647 80702 57543 93179 99032 82624 32672 53670 69698 42383 23801 69303
[13]  9816 61803 69464 23107 76958 44447    10 27292
```

```
s1 <- X[selected_row, ]
cor(s1)
```

```
          [,1]      [,2]
[1,] 1.0000000 0.1431698
[2,] 0.1431698 1.0000000
```

今回の相関係数は 0.1431698 となった。母集団の相関係数が 0 であっても，適当に抜き出した 20 点が相関係数を持ってしまう (0 でない) ことはあり得ることなのである。問題は，これがどの程度あり得ることなのか，である。いいかえると，研究者が $n = 20$ のサンプルをとって相関を得た時，それが $r = 0.14$ であったとしても，母相関 $\rho = 0.0$ からのサンプルである可能性がどれぐらいあるか，ということである。

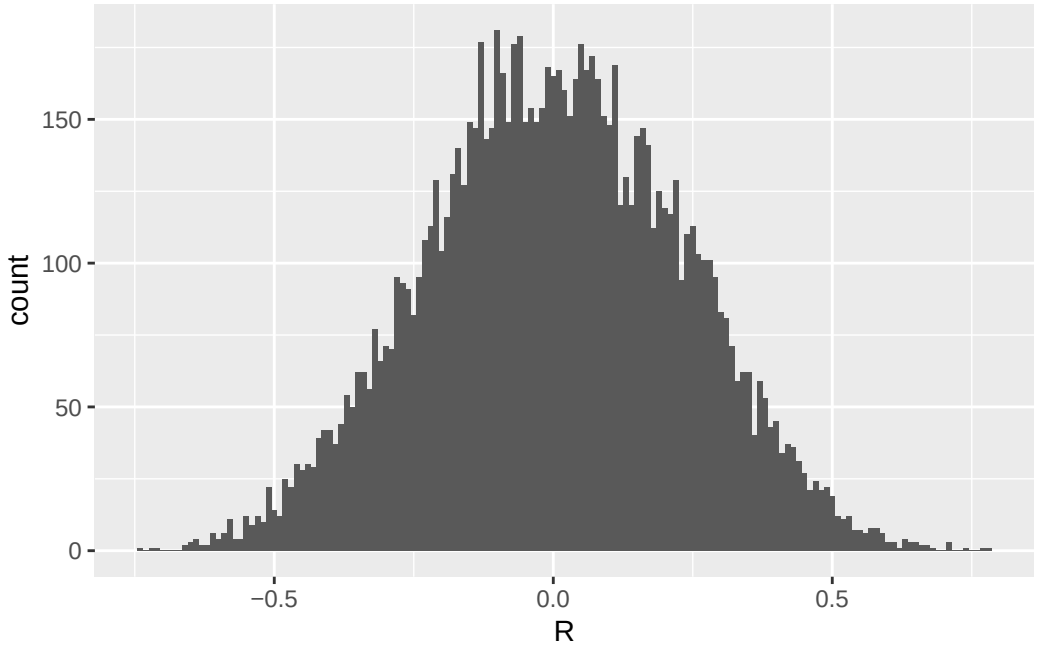
7.3 標本相関係数の分布と検定

標本相関係数は確率変数なので，毎回標本を取る度に値が変わるし，どの実現値がどの程度出現するかは標本分布で表現できる。ではどのような標本分布に従うのだろうか。先ほどのサンプリングを繰り返して，乱数によって近似してみよう[*1]。

```
iter <- 10000
samples <- c()
for (i in 1:iter) {
  selected_row <- sample(1:N, 20)
  s_i <- X[selected_row, ]
  cor_i <- cor(s_i)[1, 2]
```

[*1] このような二度手間を取らず，mvrnorm からサンプルサイズ 20 の乱数を反復生成しても良い。母集団を具体的なものとしてイメージするために，母相関が 0 の母集団からサンプリングを繰り返す方法をとった。

```
  samples <- c(samples, cor_i)
}
df <- data.frame(R = samples)
# ヒストグラムの描画
g <- df %>%
  ggplot(aes(x = R)) +
  geom_histogram(binwidth = 0.01)
print(g)
```

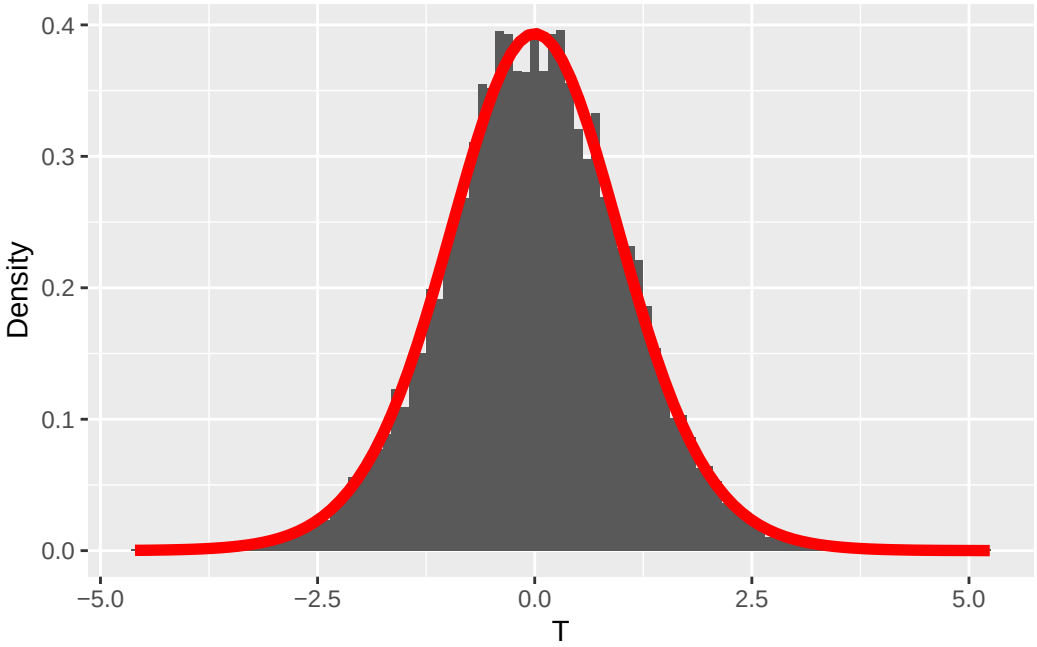


ヒストグラムを見ると，サンプルサイズが 20 の場合，母相関係数 $\rho = 0.0$ であっても $r = 0.3$ や $r = 0.4$ 程度の標本相関が出現することはある程度みられることである。

また，標本分布は左右対称の何らかの理論分布に従っていそうだ。数理統計学の知見から，相関係数の場合，標本相関係数を次の式によって変換することで，自由度が $n - 2$ の t 分布に従うことが知られている。

$$t = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}}$$

```
df %>%
  mutate(T = R * sqrt(18) / sqrt(1 - R^2)) %>%
  ggplot(aes(x = T)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 0.1) +
  # 自由度18のt分布の確率密度関数を追加
  stat_function(fun = dt, args = list(df = 18), color = "red", linewidth = 2) +
  # Y軸のラベルを変更
  ylab("Density")
```



これを利用して相関係数の検定が行われる。以下，サンプルサイズ 20 で標本相関係数が $r = 0.5$ だったとして，手順に沿って解説する。

1. 帰無仮説は母相関 $\rho = 0.0$ とする。対立仮説は $\rho \neq 0.0$ である。
2. 検定統計量は相関係数 r を変換した t とする。
3. 判定基準として，$\alpha = 0.05$ とする。すなわち，母相関が 0 であるという仮説を棄却して間違える確率を 5% 以下に制御したい。
4. 検定統計量を計算する。$n = 20, r = 0.5$ より，

$$t = \frac{0.5 \times (\sqrt{18})}{\sqrt{1 - 0.5^2}} = 2.449$$

5. 標本相関係数の**絶対値が** 0.5 **を超える**確率は，t 分布の理論値から，次のように計算できる。あるいは，t 分布の両端 5% を切り出す**臨界値**を次のように計算できる。

```
(1 - pt(0.5 * sqrt(18) / sqrt(1 - 0.5^2), df = 18)) * 2
```

```
[1] 0.02476956
```

```
qt(0.975, df = 18)
```

```
[1] 2.100922
```

ここで注意してほしい点は，今回の検定の目的が「母相関が 0 であるという帰無仮説を棄却できるかどうか」であり，相関係数の符号については関心がなく絶対値で考える点である。pt 関数は，ある確率点までの累積面積であるから，1 から引くことでその確率点以上の値がでる確率が示される。t 分布は左右対称の分布なので，これを 2 倍した値が絶対値で考えた時の出現確率である。これが 5% よりも小さければ，有意であると判断できる。今回は，統計的に有意であるといって良い。

なお，表現上の細かい注意点になるが，この確率は今回の実現値「以上」のより極端な値が出る確率であり，この

実現値が出る確率という言い方はしない。確率は面積であり，点に対する面積はないからである。

qt 関数で示されるのは確率点なので，これ以上の値を今回の実現値が出していたら，統計的に有意であると判断できる。今回の実現値から算出した値は $t(18) = 2.449$ であり，臨界値の 2.100 よりも大きな値なので，有意であると判断できる。

7.4　2 種類の検定のエラー確率

上では丁寧に計算過程をみてきたが，実践場面ではサンプルはひとつであり，標本統計量もひとつ算出されるだけである。自分の大切なデータであるから，標本分布から得られた特定のケースにすぎないことが直感的にわかりにくいかもしれない。

相関係数の検定をするときは，R の関数 cor.test を使って次のように行う。ここでは mvrnorm 関数を使って，相関係数 0.5 の仮想データを作っている。

```
set.seed(17)
n <- 20
sampleData <- mvrnorm(n,
  mu = c(0, 0),
  Sigma = matrix(c(1, 0.5, 0.5, 1), ncol = 2),
  empirical = TRUE
)
cor.test(sampleData[, 1], sampleData[, 2])
```

```
    Pearson's product-moment correlation

data:  sampleData[, 1] and sampleData[, 2]
t = 2.4495, df = 18, p-value = 0.02477
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.07381057 0.77176071
sample estimates:
cor
0.5
```

結果として示されている，t の値や自由度，p 値が先ほど示した例と対応していることを確認できる。さらに，相関係数の信頼区間や標本相関係数そのものも示されている。この信頼区間が 0 を跨いでいないことからも，帰無仮説が棄却されることが見て取れるだろう。

われわれは既に，母相関が 0 のデータセットの一部を取り出すと，その相関係数が 0 ではなく 0.5 のような数字になることも知っている。もちろん母相関が 0 であれば標本相関も 0 近い値が出やすいとしても，である。つまり標本から得られた値をあまり大事に考えすぎない方が良い (もちろん一般化を念頭においている時は，である)。

また，帰無仮説は「母相関が 0 である」なので，これが**棄却**されたとしても「母相関が 0 であるとは言えない」のに過ぎない。ここから，母相関も $r = 0.5$ 付近にあるはずだとか，p 値が 2.4% なので 5% よりもずいぶん低いのは証拠の重要さを物語っているのだ，と論じるのは適切ではない。母相関が 0 という仮想的な状況のもとでの話であって，母相関が実際にどの程度なのかを検討しているわけではない。この点が誤解されやすいので特に注意してほしい。

ここに来るとタイプ 1 エラー，タイプ 2 エラーがより具体的に理解できるようになってきたではないだろうか。タイプ 1 エラーはこの帰無仮説が正しい時に，標本相関から計算した統計量で判断する確率であるから，上の手続きで見たことそのものである。

別の角度で見てみよう。`cor.test` をつかうと標本統計量の信頼区間が算出できる。この信頼区間が母相関 – ここでは帰無仮説である $\rho = 0$ を「正しく」含んでいる割合を見てみよう。`cor.test` 関数が返すオブジェクトには，`conf.int` という名前のものがあり，デフォルトではここで 95% の信頼区間が含まれている。シミュレーションに先立って，結果を格納する 2 列のデータフレームを作っておき，シミュレーション後に `ifelse` 関数で母相関が含まれているかどうかの判定をした。

```
set.seed(42)
iter <- 10000
intervals <- data.frame(matrix(NA, nrow = iter, ncol = 2))
names(intervals) <- c("Lower", "Upper")
for (i in 1:iter) {
  selected_row <- sample(1:N, 20)
  s_i <- X[selected_row, ]
  cor_i <- cor.test(s_i[, 1], s_i[, 2])
  intervals[i, ] <- cor_i$conf.int[1:2]
}
#
df <- intervals %>%
  mutate(FLG = ifelse(Lower <= 0 & Upper >= 0, 1, 0)) %>%
  summarise(type1error = mean(FLG)) %>%
  print()
```

```
  type1error
1       0.95
```

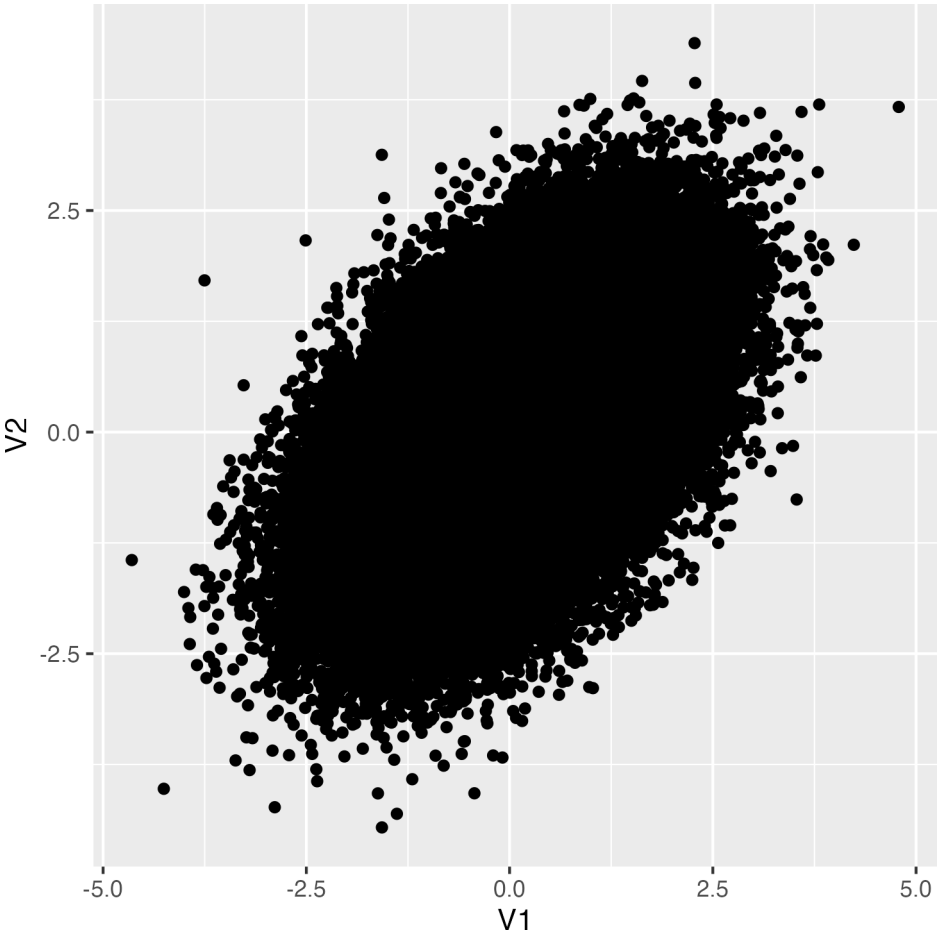
今回の例では，95% の割合で正しく判断できていた。言い換えると，エラーが生じる割合は 5% だったので，タイプ 1 エラー確率を 5% 以下にするという目的はしっかり達成できていたことが確認できた。

同様に，タイプ 2 エラーは，帰無仮説が正しくないときに帰無仮説を採択する確率だから，シミュレーションするなら次のようになる。まず母相関が 0 でない状況を作り出そう。今回は母相関が 0.5 であるとして，母集団分布を描いてみよう。

```
set.seed(12345)
N <- 100000
```

```r
X <- mvrnorm(N,
  mu = c(0, 0),
  Sigma = matrix(c(1, 0.5, 0.5, 1), ncol = 2),
  empirical = TRUE
)

X %>%
  as.data.frame() %>%
  ggplot(aes(x = V1, y = V2)) +
  geom_point()
```



今度は，ここからサンプルサイズ 20 のデータセットを取り出し，検定することにしよう。検定の結果，有意になれば 1，ならなければ 0 というオブジェクトを作って，判定の正しさを考えてみることにする。

```r
iter <- 10000
judges <- c()
for (i in 1:iter) {
  selected_row <- sample(1:N, 20)
  s_i <- X[selected_row, ]
```

```
  cor_i <- cor.test(s_i[, 1], s_i[, 2])
  judges <- c(judges, cor_i$p.value)
}
df <- data.frame(p = judges) %>%
  mutate(FLG = ifelse(p <= 0.05, 1, 0)) %>%
  summarise(
    sig = sum(FLG == 1),
    non.sig = sum(FLG == 0),
    type2error = non.sig / iter
  ) %>%
  print()
```

```
   sig non.sig type2error
1 6442    3558     0.3558
```

今回は母相関が 0.5 であり，帰無仮説は棄却されて然るべきなのだが，有意でないと判断された割合が 35.58% あったことになる。心理学の研究などでは，この確率 β が 0.2 未満，逆にいうと検出が 0.8 以上あることが望ましいとされているので，今回のこの事例では十分な件出力がなかった，と言えるだろう。

もちろん実際には，母相関がどれぐらいなのかわからない。0.3 なのかもしれないし，−0.5 であるかもしれない。つまりタイプ 2 エラーは研究者が制御できるところではなく，せいぜい大きな相関が見込めそうな変数について標本を取ろうと心がけるだけである。

タイプ 1,2 エラーの確率は，サンプルサイズや効果量 (ここでは母相関の大きさ) の関数である。サンプルサイズは研究者が決定することができるので，効果を見積もり，制御したいエラー確率の基準を決めて，合理的にサンプルサイズを決めるべきである。

7.5 課題

1. 母相関が 0 の母集団から，サンプルサイズ 10 の標本を取り出して標本相関を見た時の標本分布を，乱数のヒストグラムで近似してみましょう。
2. 同じく，サンプルサイズ 50 の標本を取り出して標本相関を見た時の標本分布を，乱数のヒストグラムで近似してみましょう。サンプルサイズが 20 や 10 の時と比べてどういう違いがあるでしょうか。
3. サンプルサイズ 50 の標本相関が $r = -0.3$ のとき，統計的に有意と言えるでしょうか。`cor.test` をつかって検定し，検定結果と判断結果を記述してください。
4. 標本相関が $r = -0.3$ だとします。サンプルサイズが 10,20,50,1000 のとき，統計的に有意と言えるでしょうか。`cor.test` を使って検定し，検定結果を一覧にしてみましょう。ここから何がわかるでしょうか。
5. 母相関が $\rho = -0.3$ だったとします。サンプルサイズ 20 のとき，どの程度の検出力があると見込めるでしょうか。シミュレーションで近似してください。

第 8 章

平均値差の検定

平均値差の検定は，実験計画の結論を出すために用いられる手段である。無作為割り当てによって個人差や背景要因が相殺され，平均的な因果効果を検証することができるからである。その結果を一般化するためには，やはり推測統計学の知見が必要であり，サンプルサイズやタイプ 1,2 エラーが関わってくることに変わりはない。

8.1 一標本検定

まず配置標本検定の例から始める。母平均がわかっている，あるいは理論的に仮定される特定の値に対して，標本平均が統計的に有意に異なっていると言って良いかどうかの判断をするときに用いる。たとえば 7 件法のデータを取ったときに，ある項目の平均が中点 4 より有意に離れていると言って良いかどうか，といった判定をするときに用いる。かりに，サンプルサイズ 10 で 7 件法のデータが得られたとしよう。ここでは平均 4,SD1 の正規乱数を 10 件生成することで表現する。実際にはこの値を，人に対する尺度カテゴリへの反応として得ているはずである。

```
pacman::p_load(tidyverse)
set.seed(17)
n <- 10
mu <- 4
X <- rnorm(n, mean = mu, sd = 1)
print(X)
```

```
 [1] 2.984991 3.920363 3.767013 3.182732 4.772091 3.834388 4.972874 5.716534
 [9] 4.255237 4.366581
```

今回，標本平均は 4.177 であり，これより極端な値が $\mu = 4$ の母集団から得られるかどうかを検定する。帰無仮説検定の手順にそって進めていくと，以下のようになる。

1. 帰無仮説は母平均が理論的な値 (ここでは尺度の中点 4) であること，すなわち $\mu = 4$ であり，対立仮説は $\mu \neq 4$ である。
2. 検定統計量は，正規母集団から得られる標本平均が従う標本分布であり，母分散が未知の場合の区間推定に用いた T 統計量になる。
3. 判断基準は心理学の慣例に沿って 5% とする。

このあと，検定統計量の計算と判定である。これを R は t.test 関数で一気に処理できる。

```
result <- t.test(X, mu = mu)
print(result)
```

```

    One Sample t-test

data:  X
t = 0.6776, df = 9, p-value = 0.5151
alternative hypothesis: true mean is not equal to 4
95 percent confidence interval:
 3.585430 4.769131
sample estimates:
mean of x
 4.177281
```

結果として，今回の検定統計量の実現値は 0.678 であり，自由度 9 の t 分布からこれ以上の値が出てくる確率は，0.515 であることがわかる。これは 5% 水準と見比べてより大きいので，レアケースではないと判断できる。つまり，母平均 4 の正規母集団から，4.177 の標本平均が得られることはそれほど珍しいものではなく，統計的に有意に異なっていると判断するには及ばない，ということである。

レポートなどに記載するときは，これら実現値や p 値を踏まえて「$t(9) = 0.66776, p = 0.5151, n.s.$」などとする。ここで n.s. は not significant の略である。

さてこの例では，母平均 4 の正規乱数を生成し，その平均が 4 と異なるとはいえない，と結論づけた。これは一見，当たり前のことのようであり，無意味な行為におもえるかもしれない。しかし次の例を見てみよう。

```
n <- 3
mu <- 4
X <- rnorm(n, mean = mu, sd = 1)
mean(X) %>%
  round(3) %>%
  print()
```

```
[1] 5.04
```

```
result <- t.test(X, mu = mu)
print(result)
```

```

    One Sample t-test

data:  X
t = 5.1723, df = 2, p-value = 0.03541
```

```
alternative hypothesis: true mean is not equal to 4
95 percent confidence interval:
 4.174825 5.904710
sample estimates:
mean of x
 5.039768
```

ここではサンプルサイズ $n = 3$ であり，標本平均が 5.04 であった。このとき t 値は 5% 臨界値を上回っており，「母平均 4 のところから得られる値にしては極端」であるから，統計的に有意に異なる，と判断することになる。乱数生成時は平均を確かに 4 に設定したが，母平均から取り出したごく一部が，そこから大きく離れてしまうことはあり得るのである。

8.2 二標本検定

続いて二標本の検定について考えよう。実験群と統制群のように，無作為割り当てをすることで平均因果効果をみる際に行われるのが，この検定である。帰無仮説は「群間差はない」であり，対立仮説はその否定である。また，正規母集団からの標本を仮定するので，検定統計量はここでも t 分布に従う値になる。帰無仮説検定の手順に沿って，改めて確認しておこう。

1. 帰無仮説は「二群の母平均に差がない」である。二群の母平均をそれぞれ μ_1, μ_2 とすると，帰無仮説は $\mu_1 = \mu_2$，あるいは $\mu_1 - \mu_2 = 0$ と表される。対立仮説は $\mu_1 \neq \mu_2$ あるいは $\mu_1 - \mu_2 \neq 0$ である。
2. 検定統計量は，正規母集団から得られる標本平均が従う標本分布であり，母分散が未知の場合の区間推定に用いた T 統計量になる。
3. 判断基準は心理学の慣例に沿って 5% とする。

これを検証するために，サンプルデータを乱数で生成しよう。まず，各群のサンプルサイズを `n1,n2` とする。ここでは話を簡単にするため，サンプルサイズは両群ともに 10 とした。つぎに両群の母平均だが，群 1 の母平均を μ_1，群 2 の母平均を $\mu_2 = \mu_1 + \delta$ で表現した。この δ は差分であり，これが $\delta = 0$ であれば母平均が等しいこと，$\delta \neq 0$ であれば母平均が異なることになる。最後に両群の母 SD を設定した。

ここでの検定は，この差分 d が母平均 0 の母集団から得られたと判断して良いかどうか，という形で行われる。検定統計量 T は，次式で算出されるものである。

$$T = \frac{d - \mu_0}{\sqrt{U_p^2 / \frac{n_1 n_2}{n_1 + n_2}}}$$

ここで d は二群の標本平均の差であり，U_p^2 はプールされた不偏分散と呼ばれ，二群を合わせて計算された全体の母分散推定量である。各群の標本分散をそれぞれ S_1^2, S_2^2 とすると，次式で算出される。

$$U_p^2 = \frac{n_1 S_1^2 + n_2 S_2^2}{n_1 + n_2 - 2}$$

これらの式はつまり，サンプルサイズの違いを考慮するため，一旦両群の標本分散に各サンプルサイズを掛け合わせ，プールした全体のサンプルサイズから各々 −1 をすることで全体として不偏分散にしている。

これを踏まえて，具体的な数字で見ていこう。その上で乱数でデータを生成し，その標本平均を確認した上で，t.test 関数によって検定を行っている。

```
n1 <- 10
n2 <- 10
mu1 <- 4
sigma <- 1
delta <- 1
mu2 <- mu1 + (sigma * delta)

set.seed(42)
X1 <- rnorm(n1, mean = mu1, sd = sigma)
X2 <- rnorm(n2, mean = mu2, sd = sigma)

X1 %>%
  mean() %>%
  round(3) %>%
  print()
```

```
[1] 4.547
```

```
X2 %>%
  mean() %>%
  round(3) %>%
  print()
```

```
[1] 4.837
```

```
result <- t.test(X1, X2, var.equal = TRUE)
print(result)
```

```

    Two Sample t-test

data:  X1 and X2
t = -0.49924, df = 18, p-value = 0.6237
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -1.506473  0.927980
sample estimates:
mean of x mean of y
```

```
 4.547297  4.836543
```

今回の母平均は $\mu_1 = 4, \mu_2 = 4 + 1$ にしているが，標本平均は 4.547 と 4.837 であり，標本上では大きな差が見られなかった。結果として，t 値は 0.4992369 であり，自由度 18 のもとでの p 値は 0.6236593 である。5% 水準を上回る値であるから，結論としては対立仮説を採択するには至らない，差があるとはいえない，である。

今回の設定では母平均に差があるはず $(4 \neq 4 + 1)$ なのだから，これは誤った判断で，タイプ 2 エラーが生じているケースということになる。研究実践場面では，母平均やその差については知り得ないのだから，このような判断ミスが生じていたかどうかは分かり得ないことに留意しよう。

なお，ここではわかりやすく 2 群であることを示すために `X1`,`X2` と 2 つのオブジェクトを用意したが，実践的にはデータフレームの中で群わけを示す変数があり，`formula` の形で次のように書くことが多いだろう。

```
dataSet <- data.frame(group = c(rep(1, n1), rep(2, n2)), value = c(X1, X2)) %>%
  mutate(group = as.factor(group))
t.test(value ~ group, data = dataSet, var.equal = TRUE)
```

```

    Two Sample t-test

data:  value by group
t = -0.49924, df = 18, p-value = 0.6237
alternative hypothesis: true difference in means between group 1 and group 2 is not equal to 0
95 percent confidence interval:
 -1.506473  0.927980
sample estimates:
mean in group 1 mean in group 2
       4.547297        4.836543
```

8.3　二標本検定 (ウェルチの補正)

先ほどの t.test 関数には，`var.equal = TRUE` というオプションが追加されていた。これは 2 群の分散が等しいと仮定した場合の検定になる。t 検定は歴史的にこちらが先に登場しているが，2 群の分散が等しいかどうかはいきなり前提できるものでもない。等分散性の検定は，Levene 検定を行うのが一般的であり，R においては，`car` パッケージや `lawstat` パッケージが対応する関数を持っている。ここでは `car` パッケージの `leveneTest` 関数を用いる例を示す。

```
pacman::p_load(car)
leveneTest(value ~ group, data = dataSet, center = mean)
```

```
Levene's Test for Homogeneity of Variance (center = mean)
      Df F value Pr(>F)
group  1  2.9405 0.1035
      18
```

この結果を見ると，p 値から明らかなように，2 群の分散が等しいという帰無仮説が棄却**できなかった**ので，等しいと考えて t 検定に進むことができる。もしこれが棄却されてしまったら，2 群の分散が等しいという帰無仮説が成り立たないのだから，等分散性の仮定を外す必要がある。実行は簡単で，`var.equal` を `FALSE` にすれば良い。

```
result2 <- t.test(value ~ group, data = dataSet, var.equal = FALSE)
print(result2)
```

```
    Welch Two Sample t-test

data:  value by group
t = -0.49924, df = 13.421, p-value = 0.6257
alternative hypothesis: true difference in means between group 1 and group 2 is not equal to 0
95 percent confidence interval:
 -1.5369389  0.9584459
sample estimates:
mean in group 1 mean in group 2
       4.547297        4.836543
```

よく見ると，タイトルが Welch Two Sample t-test に変わっている。Welch の補正が入った t 検定という意味である。また自由度が実数 (13.421) になっているが，このように t 分布の自由度を調整することで等分散性の仮定から逸脱した場合の補正となる。もちろん報告する際は「$t($ 13.421 $)$ = -0.499, $p = 0.626$」のように書くことになるから，自由度が実数であれば補正済みであると考えられるだろう。

しかし，分散が等しいという仮定は，等しくない場合の特殊な場合であるから，最初から Welch の補正がはいった検定だけで十分である。このような考え方から，R における `t.test` 関数のデフォルトでは `var.equal = FALSE` となっており，特段の指定をしなければ等分散性の仮定をしない。こちらの方が検定を重ねることがないので，より望ましい。

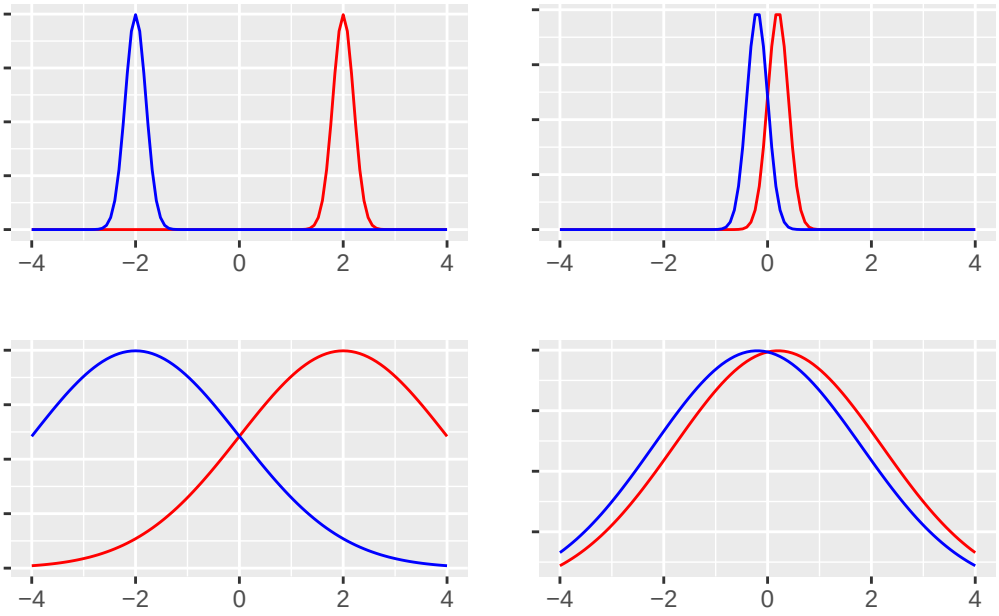
8.3.1　効果量の算出

今回の例は，仮想データとして $\mu_1 = 4, \mu_2 = \mu_1 + \sigma d$ であり，明らかに $\mu_1 \neq \mu_2$ なのだが，有意差を検出するには至らなかった。統計的な有意差はあくまでも「統計的な」観点からのものであり，我々が現実に検証したいのは本当に差があるかどうか，いわば「実質的な差」があるかどうかであるのだから，統計的な有意差を得ることを目的にするのははっきりと不適切な目標設定であると言えるだろう[*1]。

ところで，統計的に差があるとはっきり言えるのはどのような時だろうか。これは次の 4 つのデータの分布を見

[*1] たとえば物理学などのシーンでは，測定の精度が高く，単一の物理世界を対象にした検証を行うのだから，予測が真であるか偽であるかを確率的に考えるような必要はない。そのような世界における検証 – あえて理論的な正しさが明確な世界，と表現するが – であれば，統計的な差があるかどうかの情報はあくまでも理論を支持するおまけ情報にすぎない。いわば統計的検定の結果を報告するのは，論文を書くためのレトリックである。例えば，ニュートンの運動法則やアインシュタインの相対性理論などの物理法則の検証では，測定誤差の範囲内で理論値と実験値が一致するかどうかが重要であり，統計的な有意性は二次的な情報に過ぎない。翻って，人間を対象にした小サンプルの科学である心理学は，統計的な判断に頼らざるを得ないという側面はあるだろう。しかしだからと言って，実質的な差が本質的であることを忘れてしまっては本末転倒である。

てもらうとわかりやすい。



左列は平均差が大きいデータ，右列は小さいデータである。上段は分散が小さいデータ，下段は大きいデータである。この 4 つそれぞれのシーンにおいて，「差がある」と判断しやすいのはどれかを考えてみるとよい。当然，左上のシーンが最も明確に差があると言えるであろう。なぜなら，両群が明確に分かれており，群間の重複がないからである。左下は同じ平均値差であっても，群内の広がりが大きいから群間の重複がみられるため，「差がある」という判断を受けても各群の中には該当しないケースがちらほらみられることだろう。右上パネルのようなケースでは，重複は少ないが差が小さいため，「差がある」と判断できるかどうかが微妙である。右下に至っては，差も小さく分布の重複も大きいから，「差がある」と判断しても該当しないケースが多くなる。たとえば「男性は女性よりも力が強い (体力・筋力に差がある)」というデータがあったとしても，「女性より非力な男性」もかなり多く存在するだろう。そういう反例が多くみられるような場合，統計的に差があるという結果が示されたとしても，受け入れられないのではないだろうか。

ここから明らかなように，差の判断には平均値差だけでなく分散も関わってくる。そこで平均値差を標準偏差で割った，**標準化された差**が重要になってくるのであり，これが**効果量**と呼ばれるものである[*2]。

今回 2 群の差のデータを作る時に，σd としたが，平均値差の効果量 es は，

$$es = \frac{\mu_1 - \mu_2}{\sigma}$$

で表現されるから，d が効果量を表していたのである。もちろん我々は母平均，母 SD などを知り得ないのでこれもデータから推定する他ない。幸い R には `effsize` パッケージなど，効果量を算出するものが用意されている。

```
pacman::p_load(effsize)
cohen.d(value ~ group, data = dataSet)
```

[*2] 統計的な有意差よりも効果量，効果量よりも実質的な差のほうが意味のある差であることを忘れてはならない。詳しくは豊田 (2009) を参照。

```
Cohen's d

d estimate: -0.2232655 (small)
95 percent confidence interval:
    lower     upper
-1.165749  0.719218
```

```
cohen.d(value ~ group, data = dataSet, hedges.correction = TRUE)
```

```
Hedges's g

g estimate: -0.2138318 (small)
95 percent confidence interval:
     lower      upper
-1.1162608  0.6885973
```

平均値差の検定の後は，ここに示した Cohen の d や Hedges の g といった効果量を添えて報告することが一般的である。

8.4　対応のある二標本検定

実験群と統制群のように異なる 2 群ではなく，プレポスト実験のように対応がある 2 群の場合は，t 検定の定式化が異なる。対応がない t 検定の場合は，群平均の差 $\mu_1 - \mu_2$ の分布を考えたが，対応がある場合は個々の測定の差，つまり $X_{i1} - X_{i2} = D_i$ を考える。この一つの標本統計量を検定するのだから，一標本検定の一種であるとも言える。またこの D の分布の標準誤差は，標本標準誤差 U_D を使った $U_D/\sqrt{n}$ を使って推定する[*3]。検定統計量 T は，次式で算出される。

$$T = \frac{\bar{D}}{U_D/\sqrt{n}} = \frac{\sum D_i/n}{\sqrt{\frac{\frac{1}{n-1}\sum(D_i-\bar{D})^2}{n}}}$$

検定にあたっては，`t.test` 関数の引数 `paired` を `TRUE` にするだけで良い。

8.4.1　仮想データの組成

仮想データを作って演習してみよう。データの組成については，2 種類のアプローチで説明が可能である。ひとつは次のシミュレーションで表されるような形である。

[*3] 対応があるケースを考えているので，当然 n は前後の群で同数である。

```r
n <- 10
mu1 <- 4
sigma <- 1
d <- 1
X1 <- rnorm(n, mu1, sigma)
X2 <- X1 + sigma * d + rnorm(n, 0, sigma)
t.test(X1, X2, paired = TRUE)
```

```
    Paired t-test

data:  X1 and X2
t = -1.8036, df = 9, p-value = 0.1048
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -1.4339112  0.1617193
sample estimates:
mean difference
     -0.6360959
```

すなわち，第一の群が μ_1 を平均にばらついた実現値として得られ，第二の群はその実現値に一定の効果 $\sigma * d$ が加わり，その測定にさらに誤差がつく形である。この方法は具体的なデータ生成プロセスをそのまま模したような形でデータを作っているが，測定誤差を二重に計上している点が気になるかもしれない。

もう一つの考え方は，プレポスト型のデータに限らず，何らかの形で「対応がある」ことも表現できるものである。対応があるということは，2 つのデータがそれぞれ独立した一変数正規分布から得られているのではなく，二変数正規分布から得られると考えるのである。二変数正規分布は，それぞれの変数は正規分布しているが，両者の間に相関があると考えるものである。変数が一つだけの正規分布は

$$X \sim N(\mu, \sigma)$$

で表現されているのに対し，複数の変数を同時に生成する多変数 (多次元) 正規分布 Multivariate Normal Distribution は，以下のように表現される。

$$\mathbf{X} \sim MVN(,)$$

ここで $\mathbf{X}$ や は n 次元ベクトルであり， は分散共分散行列を表している。二変数の場合は以下のように書くことができる。

$$= \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{pmatrix} = \begin{pmatrix} \sigma_1^2 & \rho_{12}\sigma_1\sigma_2 \\ \rho_{21}\sigma_2\sigma_1 & \sigma_2^2 \end{pmatrix}$$

共分散 σ_{ij} は相関係数 ρ_{ij} を用いて書けることからわかるように，変数間に相関があることを想定してデータを生成するのである。この組成に従った仮想データの作成は以下のとおりである。

```
pacman::p_load(MASS) # 多次正規乱数を生成するのに必要
n <- 10
mu1 <- 4
sigma <- 1
d <- 1
mu <- c(mu1, mu1 + sigma * d)
rho <- 0.4
SIG <- matrix(c(sigma^2, rho * sigma * sigma, rho * sigma * sigma, sigma^2), ncol = 2, nrow = 2)
X <- mvrnorm(n, mu, SIG)
t.test(X[, 1], X[, 2], paired = TRUE)
```

```
    Paired t-test

data:  X[, 1] and X[, 2]
t = -2.4313, df = 9, p-value = 0.0379
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -1.96934592 -0.07095313
sample estimates:
mean difference
       -1.02015
```

効果量については，対応のない t 検定の場合と同じで良い。

```
cohen.d(X[, 1], X[, 2])
```

```
Cohen's d

d estimate: -1.04088 (large)
95 percent confidence interval:
      lower       upper
-2.04204357 -0.03971697
```

```
cohen.d(X[, 1], X[, 2], hedges.correction = TRUE)
```

```
Hedges's g

g estimate: -0.9968994 (large)
95 percent confidence interval:
     lower      upper
```

```
-1.9510179 -0.0427809
```

8.4.2 検定の方向性

ここまでの検定では，主に「差があるかどうか」といった仮説に対応するものを扱ってきた。差があるかどうか，というのはその差がプラスの方向にでているのか，マイナスの方向に出ているのかといったことを問題にしていない。そこで検定統計量の分布についても，分布の両裾を考えて有意水準を設定していた。

しかしプレポスト実験などでは，効果が「上がった」のか「下がった」のか，ということが大きな関心時でもあることが多いだろう。効果がある，ただし逆効果である，というのでは意味がないからである。このように方向性をもった仮説を検証する場合は，検定統計量の分布も一方向だけ考えればよく，`t.test` 関数には `alternative` オプションをつかって表現する。

`t.test(x,y,alternatives = "less")` とすると $x < y$ の帰無仮説を検証することになるし，`alternatives = "greater"`とすると $x > y$ の帰無仮説を検証することになる。デフォルトでは `alternatives = "two.sided"` であり，両側検定が選ばれている。

ただし，両裾から片裾にかわるということは，検定統計量が超えるかどうかの判断をする**臨界値**が小さくなることでもある。必然的に，片裾 (片側検定) のほうが緩やかな基準で検定をしていることにもなる。デフォルトで普段から厳しく検定しているから大丈夫だろう，というのも一つの考え方だが，やはり本来の研究仮説に適した帰無仮説の設定をするべきだろう。

8.5 課題

1. 平均が 50、標準偏差が 10 の正規分布からランダムに選んだ 30 個のサンプルを用意し，このサンプルの平均が母集団の平均と異なるかどうかを検定してください。検定結果を，心理学のフォーマット (心理学会編「論文執筆投稿の手引き」) に準拠した書き方で，結果を記述してください。

2. 以下のデータセットを使用して，2 つの独立した群の平均に差があるかどうかを t 検定してください。検定結果を，心理学のフォーマット (心理学会編「論文執筆投稿の手引き」) に準拠した書き方で，結果を記述してください。

$$group1 = \{45, 50, 55, 60, 65\}$$

$$group2 = \{57, 60, 62, 77, 75\}$$

3. 多次元正規分布を用いた仮想データ生成方で，対応のある t 検定の練習をしましょう。サンプルサイズを $n = 20$ とし, 平均ベクトル $\mu = (12, 15)$, 分散共分散行列 $\Sigma = \begin{pmatrix} 4 & 2.8 \\ 2.8 & 4 \end{pmatrix}$ の多次元正規分布から作られた乱数を使って，対応のある t 検定をしてください。検定結果を，心理学のフォーマット (心理学会編「論文執筆投稿の手引き」) に準拠した書き方で，結果を記述してください。

4. 自由度が 10, 20, 30 の t 分布のグラフを，標準正規分布のグラフとともに描画してください。自由度が増えると t 分布がどのように変化するでしょうか。

5. 自由度が 15 の t 分布において、有意水準 5% の片側検定と両側検定の臨界値 (検定の判断基準となる理論値) を求めてください。

第 9 章

多群の平均値差の検定

心理学実験においては，古典的に分散分析モデルが多用されてきた。平均値差を見ることで実験の効果，因果関係を明らかにできるように，巧妙に実験デザインが組み立てられる。その精緻さは理論的一貫性という意味である種の美しさを持ち，多くの研究者が魅了されてきた。

分散分析にのせることを目的にした実験計画であり，実験デザインの不自由さ (とにかく分散分析をしなければならない!) が批判的に論じられることもあるが，心理学が測定している対象が平均値差以上の精度で議論できる性質でないという反論もあるだろう。

今や分散分析を超えたより高度な統計モデルがあり，現在の研究においては分散分析はもはや過去のものにすぎないかもしれないが，以後のモデルも分散分析を基本としたその発展系であるので，改めて基本を押さえておくことも重要である。

9.1 分散分析の基礎

分散分析は「分散」の分析であるかのような名称であるが，平均値差を検定するためのものである。なぜ「分散」を冠するかといえば，効果量のところで見たように，平均値差の判断には群内分散の情報が必要だからである。

多群の平均値の差，その散らばりを群間分散といい，群に含まれるデータの散らばりを群内分散とよぶ。分散分析は群内分散に対する群間分散の比が十分に大きいと考えられる場合，群間に統計的な有意差があると判断する。分散の比を表す確率分布は F 分布と呼ばれる。F 分布は群間・群内それぞれの自由度を母数にもつ。

また，実験計画は Between デザインと Within デザインに区分される。t 検定でみたような，対応のない独立した群を対象にしたデザインが Between，群間に相関が想定される対応のある群を対象にしたデザインが Within である。Within デザインは同じ個体から複数回の反応を得る (ex. period 1-2-3…) ため，反復測定デザイン Repeated measured design ともよばれることがある。この場合，群内分散から個人内の分散すなわち個人差を取り出すことができるため，これが分離できない Between デザインよりも基本的に Within デザインのほうが目的となる変動を捉えやすい。ただし，反復測定による個体への負担を考えると，毎回 Within デザインでいいというわけにもいかないところが難点である。

$$BetweenDesign：\text{全変動} = \text{群間変動} + \text{群内変動 (誤差)}$$

$$WithinDesign：全変動 = 群間変動 + 個人差変動 + 誤差$$

分散分析は要因が複数ある場合も考えられるから，要因 A が Between，要因 B が Within といった場合は混合計画と呼ばれることがある。慣例的に，要因 Factor とその要因に含まれる水準 Level を同時に表現し，間2 × 間3 の分散分析 (二要因の分散分析で，いずれも Between デザインであり，水準数がそれぞれ 2 と 3)，といった言い方をすることがある。

9.2　分散分析のステップ

t 検定において等分散性の仮定が成立するかどうかが事前に問題になったように，Between デザインにおいても分散の等質性は仮定されており，Levene の検定などで事前に検証しておくべきである。また Within デザインにおいては，データの組成に関わる分散共分散行列の非対角要素が全て等しいことが望ましいが，実践的にはそこまでの仮定が成立しているとは考えにくい。ただし分散分析としては，等分散性の仮定よりも，より緩やかな球状性の仮定が成立していればよいとされており，これを事前に検定することが一般的である。Welch の補正のように，球状性の仮定が成立していない場合は，自由度を補正することで検定の精度が維持される。

分散分析は多要因・多水準の平均値差の検定である。各水準ごとに t 検定を繰り返せば良いのではないか，というアイデアは誰しも思いつくことであろうが，この方法は検定の目的である α 水準の制御ができなくなるという問題を含む。そこで多水準の場合は分散分析を行うことで，すべての要因・水準の母平均が同じであるという帰無仮説を検定し，効果の有無をまず明確にする。この帰無仮説が棄却されたらどこかに差があるわけだから，以後は慎重に α 水準を制御しつつ事後的な検定にすすむ。

水準間の差をみるための事後的な検定は，下位検定とも呼ばれる。その方法は多岐に渡り，ゴールドスタンダードは存在せず，往々にして分析者が利用しているソフトウェアが対応する手法が選択される。要因・水準が多くなると検証すべき組み合わせも多くなり，下位検定の手続きも非常に煩雑になる。統計ソフトウェアはそれこそ機械的に，幾重にも細かく分散分析表を分解して下位検定をつづけていってくれるが，いくら制御されているとはいえ検定を繰り返していることに変わりはないし，各下位検定の結果を一貫した総合的解釈をするのは困難である。実験計画はシンプルであるほうが望ましいし，複雑なモデルになるようであれば分散分析を超えた，階層線型モデルやベイジアンアプローチなどを取る方が良いだろう。

9.3　ANOVA 君を使う

分散分析を R で実行するには，基本関数である `aov` や car パッケージなどを用いることができる。もっとも，その出力は必ずしも親切ではないし，下位検定や効果量の算出などは別のパッケージ，別の関数を用いる必要がある。

筆者がお勧めするのは，大正大学の井関龍太が開発した anovakun である。パッケージ化されていないので，リンク先からソースコードを読み込んで anovakun 関数を実行する必要があるが，さまざまな実験デザインに対応し，また下位検定や効果量，球状性の補正などおよそ分散分析で必要な手法は網羅されている。以下ではこれを用いた実践を行う。

anovakun の読み込みは，ソースコードをプロジェクトフォルダにダウンロードして `source` 関数で読み込むか，

インターネットに繋がっている状態でリンク先から直接ソースファイル (anovakun_489.txt)[*1]を source 関数で読み込むといいだろう。

```
source("https://riseki.cloudfree.jp/?plugin=attach&refer=ANOVA%E5%90%9B&openfile=anovakun_489.txt")
```

読み込みが終わると Environ タブに anovakun 関数が含まれていることを確認しよう。

9.3.1 ANOVA 君の入力とデータ

ANOVA 君は伝統的にワイド型データから読み込むようになっている。すなわち，一行に 1 オブザベーション入っている形式である。Between 計画の場合は，データの前に水準数を表すインデックスと最終的な従属変数の形に整形したデータが必要である。Within 計画の場合は 1 行に 1Obs. なのだから，反復した水準の数だけ右にデータを入れていく形に整形する。

しかし Chapter 3.7 で述べたように，昨今は計算機にとって優しい型，ロング型での入力もおおく，ANOVA 君も version 4.4.0 からロング型での入力も許すようになった。その場合はオプション long=TRUE とロング型であることを明記する必要がある。

ANOVA 君を使う時は，関数 anovakun に，データ，要因計画の型，各要因の水準の順で入力する。ここで要因計画の型とは，文字列で Between/Within の違いを明示することになる。被験者のラベルを表す小文字の s を挟んで，左側に間 (Between) 要因，右側に内 (Within) 要因を入れる。例えば一要因 Between 計画の場合は"As"，二要因 Within 計画の場合は"sAB"，間 1 内 2 の混合計画であれば"AsBC"のようにする。

続いて入力する水準数は，要因の数だけ必要である。ただし，ロング型で入力した場合は自動的に水準数が計算されるので入力の必要がない。

このテキストでは，データの持ち替えについてすでに触れているので，色々扱いやすいロング型に整形して利用していくものとする。

9.4 Between デザイン

9.4.1 1way-ANOVA

もっとも単純な一要因 3 水準，Between 計画の例から始めよう。仮想データの生成を行うことで，分散分析のメカニズムと共に見ていくことにする。

```
set.seed(123)
# 各群のサンプルサイズ
n1 <- 5
n2 <- 4
n3 <- 6
# 母平均，効果量，母SD
```

[*1] 2024.03.17 時点での最新バージョンが 4.8.9 である。リンク先 URL は，公式サイトからソースファイルのリンクをコピーして貼り付けると良い。

```
mu <- 10
delta <- 1
sigma <- 3
# 群平均
mu1 <- mu - (delta * sigma)
mu2 <- mu
mu3 <- mu + (delta * sigma)
# データセット
X1 <- rnorm(n1, mu1, sigma)
X2 <- rnorm(n2, mu2, sigma)
X3 <- rnorm(n3, mu3, sigma)
## 組み上げる
dat <- data.frame(
  ID = 1:(n1 + n2 + n3),
  group = as.factor(rep(LETTERS[1:3], c(n1, n2, n3))),
  value = c(X1, X2, X3)
)
## データの確認
dat
```

```
   ID group     value
1   1     A  5.318573
2   2     A  6.309468
3   3     A 11.676125
4   4     A  7.211525
5   5     A  7.387863
6   6     B 15.145195
7   7     B 11.382749
8   8     B  6.204816
9   9     B  7.939441
10 10     C 11.663014
11 11     C 16.672245
12 12     C 14.079441
13 13     C 14.202314
14 14     C 13.332048
15 15     C 11.332477
```

```
### 実行
anovakun(dat, "As", long = TRUE, peta = TRUE)
```

```
[ As-Type Design ]

This output was generated by anovakun 4.8.9 under R version 4.6.1.
It was executed on Wed Sep 16 15:26:31 2026.


<< DESCRIPTIVE STATISTICS >>

------------------------------
 group   n     Mean    S.D.
------------------------------
     A   5   7.5807  2.4331
     B   4  10.1681  3.9548
     C   6  13.5469  1.9483
------------------------------


<< ANOVA TABLE >>

== This data is UNBALANCED!! ==
== Type III SS is applied. ==

-------------------------------------------------------------
 Source       SS  df      MS  F-ratio  p-value      p.eta^2
-------------------------------------------------------------
  group  98.3840   2 49.1920   6.5897   0.0117 *     0.5234
  Error  89.5804  12  7.4650
-------------------------------------------------------------
  Total 187.9644  14 13.4260
                 +p < .10, *p < .05, **p < .01, ***p < .001


<< POST ANALYSES >>

< MULTIPLE COMPARISON for "group" >

== Shaffer's Modified Sequentially Rejective Bonferroni Procedure ==
== The factor < group > is analysed as independent means. ==
== Alpha level is 0.05. ==

------------------------------
```

```
 group   n     Mean    S.D.
------------------------------
     A   5   7.5807  2.4331
     B   4  10.1681  3.9548
     C   6  13.5469  1.9483
------------------------------


-----------------------------------------------------------
 Pair     Diff  t-value  df       p   adj.p
-----------------------------------------------------------
  A-C  -5.9662   3.6062  12  0.0036  0.0108  A < C *
  B-C  -3.3789   1.9159  12  0.0795  0.0795  B = C
  A-B  -2.5873   1.4117  12  0.1834  0.1834  A = B
-----------------------------------------------------------


output is over --------------------///
```

出力結果は大きく分けて記述統計<< DESCRIPTIVE STATISTICS >>と，分散分析表<< ANOVA TABLE >>，下位検定<< POST ANALYSES >>に分けられる。記述統計はデータが正しく読み込めているかどうかのチェックに使おう。

一番のメインは分散分析表であり，平方和 sum of squares を自由度 df で割った，1 自由度あたりのデータの散らばりを，群間と群内 (誤差) との比で検証しているのが見て取れる。群間平方和が 98.38，群内平方和が 89.58 であり，それぞれ自由度 2(3 水準 -1) と 12($\sum_{j=1}^{3} n_j - 1$) から生じているので，平均平方 Mean Squares がそれぞれ 49.19 と 7.47 である。この比が 6.5897 で，自由度 $F(2, 12)$ の F 分布においてこの値以上の極端な数字が出る確率が 5% を下回っている (実に $p = 0.0117$ である) ため，統計的に有意であると判断できる。分散分析表の Total のところで，全体の SS が群間 SS+ 群内 SS に一致していること，自由度も全体 df=群間 df+ 群内 df になっていることを確認しておこう。

また，anovakun 関数の引数として peta = TRUE を指定したが，これは偏 η^2(partial eta) と呼ばれる効果量を出力するためのオプションである。

今回は分散分析の時点で統計的な有意差が認められたため ($F(2, 12) = 6.59, p < 0.05, \eta^2 = 0.52$)，続いて下位検定が表示されている。ANOVA 君は下位検定についても複数のオプションを持っているが，デフォルトでは Shaffer の修正 Bonferroni 検定が行われる。詳しくは専門書 (永田・吉田，1997) を参照してほしいが，概略を説明すると，検証すべき仮説の数で有意水準を分割するという Bonferroni の方法を，競合する仮説の数も考慮して分母を調整するというものである。

この計算の結果，A 群と C 群の間にのみ統計的な有意差が確認された ($t(12) = 3.61, p < 0.05$) と言える。

9.4.2 2way-ANOVA

二要因の場合も見ておこう。ANOVA 君の表記方法は要因計画の型が変わるだけで大きな変更はないが，交互作用 interaction を考える必要があるところがポイントである。これも仮想データの組成を見ることでその意義がわかりやすくなるだろう。間 2× 間 2 の実験デザインを例に，まずは各水準の理論的平均値がどのようにつくられるかをみておこう。

```
set.seed(123)
# 各群のサンプルサイズ
n <- 10
# 全体平均，効果量，母SD
mu <- 10
delta1 <- 1
delta2 <- 0 # ここではあえて要因Bの効果を0にしている
delta3 <- 2
sigma <- 3
# 効果の計算
effectA <- delta1 * sigma # Factor A
effectB <- delta2 * sigma # Factor B
effectAB <- delta3 * sigma # interaction
# 各群の平均
mu11 <- mu + effectA + effectB + effectAB
mu12 <- mu + effectA - effectB - effectAB
mu21 <- mu - effectA + effectB - effectAB
mu22 <- mu - effectA - effectB + effectAB
```

効果の現れ方は相対的だから，要因 A が第一水準に `+effectA` の形で現れたら，第二水準には`-effectA` の形で現れる。要因 B についても同様である。交互作用については組み合わせにおいて生じるから，要因 A の第一水準と要因 B の第一水準の組み合わせのところに `+effectAB` を充てる。ここでも効果は相対的に現れるという条件を守るために，要因 A の第一水準の中で `+effectAB` の効果を相殺するために，要因 A の第一水準と要因 B の第二水準の組み合わせの符号が反転する。同様に，要因 B の第一水準の中で相殺するために要因 A の第二水準と要因 B の第一水準には`-effectAB` が加わる。

このようにして考えられる理論的平均値に対して，外乱要因である誤差が生じて実現値が得られる。組み上げて得られたデータを確認しておこう。

```
X11 <- rnorm(n, mean = mu11, sd = sigma)
X12 <- rnorm(n, mean = mu12, sd = sigma)
X21 <- rnorm(n, mean = mu21, sd = sigma)
X22 <- rnorm(n, mean = mu22, sd = sigma)
dat <- data.frame(
  ID = 1:(n * 4),
```

```
  FactorA = rep(1:2, each = n * 2),
  FactorB = rep(rep(1:2, each = n), 2),
  value = c(X11, X12, X21, X22)
)
dat
```

```
   ID FactorA FactorB      value
1   1       1       1 17.3185731
2   2       1       1 18.3094675
3   3       1       1 23.6761249
4   4       1       1 19.2115252
5   5       1       1 19.3878632
6   6       1       1 24.1451950
7   7       1       1 20.3827486
8   8       1       1 15.2048163
9   9       1       1 16.9394414
10 10       1       1 17.6630141
11 11       1       2 10.6722454
12 12       1       2  8.0794415
13 13       1       2  8.2023144
14 14       1       2  7.3320481
15 15       1       2  5.3324766
16 16       1       2 12.3607394
17 17       1       2  8.4935514
18 18       1       2  1.1001485
19 19       1       2  9.1040677
20 20       1       2  5.5816258
21 21       2       1 -2.2034711
22 22       2       1  0.3460753
23 23       2       1 -2.0780133
24 24       2       1 -1.1866737
25 25       2       1 -0.8751178
26 26       2       1 -4.0600799
27 27       2       1  3.5133611
28 28       2       1  1.4601194
29 29       2       1 -2.4144108
30 30       2       1  4.7614448
31 31       2       2 14.2793927
32 32       2       2 12.1147856
33 33       2       2 15.6853770
34 34       2       2 15.6344005
```

```
35 35       2       2 15.4647432
36 36       2       2 15.0659208
37 37       2       2 14.6617530
38 38       2       2 12.8142649
39 39       2       2 12.0821120
40 40       2       2 11.8585870
```

もちろん実際には，計画に応じたデータセットが得られているはずであり，各群のサンプルサイズが異なるなどの事情もあるだろう。しかしこうして，理論的にデータの組成を見ておくことで，サンプルサイズを変えたり効果量を変えたりしながら，どのように結果が変わってくるかを確認しながら進めることができる[*2]。

それではこの仮想データを分析してみよう。

```
anovakun(dat, "ABs", long = TRUE, peta = TRUE)
```

```
[ ABs-Type Design ]

This output was generated by anovakun 4.8.9 under R version 4.6.1.
It was executed on Wed Sep 16 15:26:32 2026.


<< DESCRIPTIVE STATISTICS >>

----------------------------------------
 FactorA  FactorB   n     Mean    S.D.
----------------------------------------
       1        1  10  19.2239  2.8614
       1        2  10   7.6259  3.1142
       2        1  10  -0.2737  2.7924
       2        2  10  13.9661  1.5819
----------------------------------------


<< ANOVA TABLE >>

------------------------------------------------------------------------------
           Source        SS  df        MS  F-ratio  p-value      p.eta^2
```

[*2] かつては分散分析は手計算でできる分析モデルであり，得られたデータを平方和に分解していくプロセスをたどりながら分散分析のメカニズムが体得されるという教育が多く見られた。ただしその方法は計算に時間がかかること，ミスが混在しやすいことに加え，手元のデータが唯一無二のものであるという印象を強くすることが懸念される。推測統計学においては，手元のデータはあくまでも実現値に過ぎないと考えるのであり，乱数を生成して幾つでも自在に作り出せる経験を得た方が教育効果として良いのではないか，と筆者は考えている。

```
------------------------------------------------------------------------------
          FactorA  432.7854   1  432.7854  61.4190   0.0000 ***   0.6305
          FactorB   17.4478   1   17.4478   2.4761   0.1243 ns    0.0644
FactorA x FactorB 1668.9825   1 1668.9825 236.8545   0.0000 ***   0.8681
            Error  253.6721  36    7.0464
------------------------------------------------------------------------------
            Total 2372.8878  39   60.8433
                          +p < .10, *p < .05, **p < .01, ***p < .001


<< POST ANALYSES >>

< SIMPLE EFFECTS for "FactorA x FactorB" INTERACTION >

------------------------------------------------------------------------
      Source        SS  df        MS  F-ratio  p-value      p.eta^2
------------------------------------------------------------------------
FactorA at 1 1900.7730   1 1900.7730 269.7492   0.0000 ***   0.8823
FactorA at 2  200.9950   1  200.9950  28.5243   0.0000 ***   0.4421
FactorB at 1  672.5693   1  672.5693  95.4480   0.0000 ***   0.7261
FactorB at 2 1013.8610   1 1013.8610 143.8826   0.0000 ***   0.7999
       Error  253.6721  36    7.0464
------------------------------------------------------------------------
                         +p < .10, *p < .05, **p < .01, ***p < .001

output is over --------------------///
```

基本的な結果の見方については，一要因のときと同じである。今回は要因 A と交互作用の効果を作り，正しく検出されている。下位検定については，要因 A が 2 水準であったためこちらの主効果の検証は必要なく (記述統計を見て群平均比較をすればよい)，交互作用についての単純効果の検証が行われている。

9.5　Within デザイン

Within デザインは対応のある t 検定の時と同じように，多次元正規分布からの生成として考えよう。すなわち各個体から得られるデータが相関しているという仮定をおくのである。以下のサンプルコードを読んで，データ生成過程を確認しよう。なお共分散は $\rho_{xy} = \frac{s_{xy}}{s_x s_y}$ より $s_{xy} = \rho_{xy} s_x s_y$ として整形している。

```
pacman::p_load(tidyverse)
pacman::p_load(MASS)
set.seed(42)
# 各群のサンプルサイズ
```

```
n <- 10
# 全体平均，効果量，母SD
mu <- 10
delta <- 1
s1 <- s2 <- s3 <- 1
rho12 <- 0.1
rho13 <- 0.3
rho23 <- 0.8
mus <- c(mu, mu + s1 * delta, mu - s1 * delta)
# 共分散行列の生成
Sigma <- matrix(NA, ncol = 3, nrow = 3)
Sigma[1, 1] <- s1^2
Sigma[2, 2] <- s2^2
Sigma[3, 3] <- s3^2
Sigma[1, 2] <- Sigma[2, 1] <- rho12 * s1 * s2
Sigma[1, 3] <- Sigma[3, 1] <- rho13 * s1 * s3
Sigma[2, 3] <- Sigma[3, 2] <- rho23 * s2 * s3
# データの生成
X <- mvrnorm(n, mus, Sigma) %>% as.data.frame()
# データの確認
X
```

```
          V1        V2       V3
1  10.625304  9.418518 7.493325
2  12.437964 11.249993 8.806719
3   8.604481 11.182418 8.722798
4   9.390742 10.181310 8.786312
5   9.567609 10.147592 9.194809
6  10.651739 11.005419 8.917299
7   9.125913  9.805634 7.511082
8   7.770294 12.462671 8.790231
9   6.909722  9.863405 7.429485
10 11.267590 10.798088 8.754522
```

```
# Long型に整形
X <- X %>%
  rowid_to_column("ID") %>%
  pivot_longer(-ID) %>%
  print()
```

```
# A tibble: 30 x 3
```

```
      ID name  value
   <int> <chr> <dbl>
 1     1 V1    10.6
 2     1 V2     9.42
 3     1 V3     7.49
 4     2 V1    12.4
 5     2 V2    11.2
 6     2 V3     8.81
 7     3 V1     8.60
 8     3 V2    11.2
 9     3 V3     8.72
10     4 V1     9.39
# i 20 more rows
```

```
# 分析の実行
anovakun(X, "sA", long = TRUE, peta = TRUE, GG = TRUE)
```

```
[ sA-Type Design ]

This output was generated by anovakun 4.8.9 under R version 4.6.1.
It was executed on Wed Sep 16 15:26:32 2026.


<< DESCRIPTIVE STATISTICS >>

------------------------------
 name   n     Mean    S.D.
------------------------------
   V1  10   9.6351  1.6609
   V2  10  10.6115  0.9057
   V3  10   8.4407  0.6777
------------------------------


<< SPHERICITY INDICES >>

== Mendoza's Multisample Sphericity Test and Epsilons ==

------------------------------------------------------------------------
 Effect  Lambda  approx.Chi  df      p         LB     GG     HF     CM
```

```
-------------------------------------------------------------------------
   name  0.0068      8.8720   2 0.0118 *   0.5000 0.5988 0.6392 0.5547
-------------------------------------------------------------------------
                              LB = lower.bound, GG = Greenhouse-Geisser
                             HF = Huynh-Feldt-Lecoutre, CM = Chi-Muller


<< ANOVA TABLE >>

-------------------------------------------------------------
  Source      SS  df      MS  F-ratio  p-value      p.eta^2
-------------------------------------------------------------
       s 16.4609   9  1.8290
-------------------------------------------------------------
    name 23.6422   2 11.8211  10.7022   0.0009 ***   0.5432
s x name 19.8819  18  1.1045
-------------------------------------------------------------
   Total 59.9849  29  2.0684
                 +p < .10, *p < .05, **p < .01, ***p < .001


<< POST ANALYSES >>

< MULTIPLE COMPARISON for "name" >

== Shaffer's Modified Sequentially Rejective Bonferroni Procedure ==
== The factor < name > is analysed as dependent means. ==
== Alpha level is 0.05. ==

-----------------------------
 name   n     Mean    S.D.
-----------------------------
   V1  10   9.6351  1.6609
   V2  10  10.6115  0.9057
   V3  10   8.4407  0.6777
-----------------------------

-----------------------------------------------------------
  Pair     Diff  t-value  df       p   adj.p
-----------------------------------------------------------
 V2-V3   2.1708   9.5342   9  0.0000  0.0000  V2 > V3 *
```

```
 V1-V3   1.1945   2.3896   9  0.0406  0.0406  V1 > V3 *
 V1-V2  -0.9764   1.6250   9  0.1386  0.1386  V1 = V2
-----------------------------------------------------------


output is over --------------------///
```

上記コードについていくつか解説をしておこう。今回，各群の分散は同じにしつつ，変数間相関に大きな違いを持たせた。あえて球状性の仮定が成立しないような例を得たかったからで，出力の`<< SPHERICITY INDICES >>`をみると統計量 λ のあとの p 値が 5% を下回っており，「球状性が成立している」という帰無仮説が棄却されていることがわかる。この時いくつかの補正法があるが，今回は Greenhouse-Geisser の補正を当てることにしている。それが `anovakun` 関数のなかの `GG=TRUE` の箇所である。

これを踏まえて分散分析表が示されている。ここでも全体平方和 `SS`，自由度 `df` が各行の要素の和になっていることが確認できるが，その因子名のところに `s` が含まれていることが確認できる。これが個体ごとの変動を表しており，誤差から個人差を取り除いて効果の検証ができていることがわかる。

分散分析は加法的，線型的な分解であるからわかりやすく，要因が複雑に組み合わさることがあっても基本的に今回のパーツを組み上げることで理解できる。言い換えると，データが先にある実践的な場合には平方和をひとつひとつ丁寧に紐解いていくことで理解できる。実に anovakun の前進である anova4 では 4 要因，anovakun では 26 要因までのデザインを分析することが可能である。もっとも 4 要因計画にもなると 3 次の交互作用まで考えられ，主効果と合わせてこれらの交互作用効果を解釈するのは困難である。anoakun は 2 次以上の交互作用が見られた場合，自動的に下位検定を行ってくれないので，要因の水準ごとにデータを分割して，分散分析表を解体しつつ分析する必要がある。[*3]

しかしもちろん，これには検定の多重性の問題が関わってくるから，あまり推奨される手法ではない。ごく少ない要因で，主効果の有無を検証することを主眼においた丁寧な実験デザインを組み立てることを試みるべきである。

またここでは，仮想データを作ることで，得られたデータの背後にある生成メカニズムに注目した。「与えられたデータを分解する」のが分散分析であるのに対し，リバースエンジニアリングからアプローチしたのである。こうすることで，分散分析の見えない仮定に注意が向くことを期待している。簡便のために，いくつかのパラメータを均質化するなどしたが，実践的には群ごとのサンプルサイズが異なることも少なくないだろうし，群間の分散や共分散が均質であることを前提とするのは難しいだろう。これを考慮した細かい作り込みも，リバースエンジニアリングによって生成メカニズムがわかっている場合には応用が可能である。さらに，どこの水準間にどのような効果があると仮定されるか，といった精緻な仮説があるのなら，そこだけをターゲットにした分析を行うことも可能である。

分散分析は，あくまでも大雑把な全体的傾向を見るためのものであることに留意しよう。心理学のデータがより精緻な仮定に耐えうる精度を持つものになれば，分散分析は過去の遺物となるかもしれない。

[*3] anovakun の補助関数 anovatan を用いることで，注目したい要因ごとにデータを分割してくれる。詳しくは公式サイトのマニュアルを参照してほしい。

9.6 課題

1. 以下のデータセットは一要因 4 水準 Between 計画で得られたものです。分散分析を行って，要因の効果があるかどうか，水準間に差があるとすればどこに見られるかを報告してください。なおこのデータセットはこちら ex_anova1.csv からダウンロード可能です。

	ID	group	value
1	1	A	14.37
2	2	A	15.11
3	3	A	16.11
4	4	A	11.17
5	5	A	14.51
6	6	A	7.85
7	7	A	10.65
8	8	B	16.45
9	9	B	11.76
10	10	B	19.11
11	11	B	19.62
12	12	C	2.92
13	13	C	6.27
14	14	C	1.82
15	15	C	-0.10
16	16	C	5.30
17	17	C	1.57
18	18	D	8.33
19	19	D	2.71
20	20	D	5.97
21	21	D	4.97
22	22	D	1.65
23	23	D	8.73
24	24	D	5.93
25	25	D	4.27

2. 以下のデータセットは一要因 4 水準 Within 計画で得られたものです。分散分析を行って，要因の効果があるかどうか，水準間に差があるとすればどこに見られるかを報告してください。なおこのデータセットはこちら ex_anova2.csv からダウンロード可能です。

	V1	V2	V3	V4
1	11.32	12.99	9.34	-0.14
2	10.77	13.84	14.74	3.52
3	9.86	12.26	12.56	2.60

```
4   8.74 11.59 14.27  0.68
5  11.12 12.93 12.92  1.13
6   9.65 16.55 12.60  2.32
7   9.72 14.64  9.69 -1.34
8  12.02 11.18 14.43  2.64
9  10.00 10.79  9.19 -1.09
10 10.04 15.53 13.38  1.82
11 10.20 11.56 11.02 -0.05
12  7.81  9.29 12.20 -3.25
```

3. 間 (3)× 間 (3) の分散分析モデルの仮想データセットを作りましょう。そのデータに分散分析を適用し，仮定した要因の効果がみられるか (あるいは効果がないと仮定した場合に正しく検出されないか) を確認しましょう。
4. 【発展課題】二要因混合計画分散分析 (間 × 内) の仮想データセットを作りましょう。そのデータに分散分析を適用し，仮定した要因の効果がみられるか (あるいは効果がないと仮定した場合に正しく検出されないか) を確認しましょう。

第 10 章

疑わしき研究実践とサンプルサイズ設計

ここまでシミュレーションを通じて仮想データを生成し，帰無仮説検定のステップをリバースエンジニアリングしながら検定を「モデル」の観点から確認してきた。

シミュレーションは仮想世界を作ることであり，いかようにもデータを作ることができるのだから，たとえば実践的にタブーとされていることを仮想的に検証してみることができる。このアプローチで，QRPs が具体的にどのように問題になるのかを体験してみよう。

10.1 疑わしき研究実践 Questionable Research Practicies

10.1.1 検定の繰り返し

帰無仮説検定は確率を伴った判断なので，「差がないのにあると判断してしまった (タイプ 1 エラー)」とか，「差があるのに検出できなかった (タイプ 2 エラー)」といった問題が生じうる。すでに述べたように，タイプ 2 エラーの方は本質的に知り得ないので (差がどの程度あるか，事前にわかっていることがない)，せめてタイプ 1 エラーは制御することを目指すことになる。

こうした検定は合理的に行われるべきもので，なんとか有意に「したい」といった研究者のお気持ちとは独立しているはずである。しかし (もしかすると) 意図せぬところで，この制御に失敗してしまっている可能性がある。

ひとつは検定の繰り返しに関する問題である。たとえば分散分析において，「主効果が出てから下位検定で各ペアの検証をするんだから，最初から各ペアの t 検定を繰り返せばいいじゃないか」と考える人がいるかもしれない。これで本当に問題ないのか，シミュレーションで確認してみよう。

以下のコードは，有意差のないデータセットを作り，1. 分散分析を行なって有意になるかどうか，2. 各ペアについて繰り返し t 検定を行い，どこかに有意差が検出されるかどうか，を比較している。分散分析は ANOVA 君ではなく，R 固有の `aov` 関数を用いた[*1]。また，「どこかに有意差が検出される」を `if` 文を使って書いているところを，注意深く確認しておいてほしい。

[*1] ANOVA 君は結果をコンソールに直接出力し，戻り値を持たない。ここでは結果の p 値が必要だったので，このようにした。

```
pacman::p_load(tidyverse)
pacman::p_load(broom) # 分析結果をtidyに整形するパッケージ。ない場合はinstallしておこう
```

```
alpha <- 0.05 # 有意水準を0.05に設定
n1 <- n2 <- n3 <- 10 # 各グループのサンプルサイズを10に設定
mu <- 10 # 平均値を10に設定
sigma <- 2 # 標準偏差を2に設定

mu1 <- mu2 <- mu3 <- mu # 各グループの平均値を同じに設定

set.seed(12345) # 乱数のシードを設定して再現性を確保
iter <- 1000 # シミュレーションの繰り返し回数を1000に設定

anova.detect <- rep(NA, iter) # ANOVA検出結果の保存用ベクトルを初期化
ttest.detect <- rep(NA, iter) # t検定検出結果の保存用ベクトルを初期化

for (i in 1:iter) { # 1000回のシミュレーションを繰り返すループ
  X1 <- rnorm(n1, mu1, sigma) # グループ1のデータを生成
  X2 <- rnorm(n2, mu2, sigma) # グループ2のデータを生成
  X3 <- rnorm(n3, mu3, sigma) # グループ3のデータを生成

  dat <- data.frame( # データフレームを作成
    group = c(rep(1, n1), rep(2, n2), rep(3, n3)), # グループ番号を追加
    value = c(X1, X2, X3) # データを追加
  )
  result.anova <- aov(value ~ group, data = dat) %>% tidy() # ANOVAを実行し結果を整形
  anova.detect[i] <- ifelse(result.anova$p.value[1] < alpha, 1, 0) # 有意差があるかを判定して保存

  # t検定を繰り返す
  ttest12 <- t.test(X1, X2)$p.value # グループ1と2のt検定
  ttest13 <- t.test(X1, X3)$p.value # グループ1と3のt検定
  ttest23 <- t.test(X2, X3)$p.value # グループ2と3のt検定

  ttest.detect[i] <- ifelse(ttest12 < alpha | ttest13 < alpha | ttest23 < alpha, 1, 0) # いずれかのt検
}

ttest.detect %>% mean() # t検定で有意差が検出された割合を計算
```

```
[1] 0.109
```

```
anova.detect %>% mean() # ANOVAで有意差が検出された割合を計算
```

```
[1] 0.04
```

結果を見ると，t 検定で有意差が検出された確率が 0.109 であり，設定した α 水準を大きく上回っていることがわかる。有意でないところに有意差を見出しているのだから，これはタイプ 1 エラーのインフレである。分散分析で検出された結果は 0.04 であり，正しく α 水準がコントロールできている。

検定を繰り返すことの問題は，確率的判断にある。5% の水準でタイプ 1 エラーが起こるということは，95% の確率で正しく判断できるということだが，2 回検定を繰り返すとその精度は $(1-0.05)^2 = 0.9025$ であり，3 回検定を繰り返すと $(1-0.05)^3 = 0.857375$ と，どんどん小さくなっていってしまう。検定はタイプ 1 エラーのハンドリングが目的であったことを忘れてはならない。

10.1.2 ボンフェローニの方法

一つの論文のなかに複数の研究 (Study1, Study2,...) があり，それぞれで検定による確率的判断を行っているとしよう。それぞれ別のデータセットに対する検定であっても，一つの論文の中で確率的判断が繰り返されていることに違いはない。このような場合は，どのようにして有意水準をコントロールすれば良いのだろうか。

最も単純明快な方法のひとつは，分散分析の下位検定でもみられた Bonferroni の補正である。すなわち，検定の回数で有意水準を割ることで，検定を厳しくするのである。5% 水準の検定を 5 回繰り返すのなら，$0.05/5 = 0.01$ とすることで全体的なタイプ 1 エラー率を抑制するのである。これが正しく機能するかどうか，シミュレーションで確認してみよう。

反復してデータを生成することになるので，仮想データ生成関数を別途事前に準備しておこう。

```
# シミュレーション用の関数を定義
studyMake <- function(n, mu, sigma, delta) {
  X1 <- rnorm(n, mu, sigma) # グループ1のデータを生成
  X2 <- rnorm(n, mu + sigma * delta, sigma) # グループ2のデータを生成（平均値が異なる）
  dat <- data.frame( # データフレームを作成
    group = rep(1:2, each = n), # グループ番号を追加
    value = c(X1, X2) # データを追加
  )
  result <- t.test(X1, X2)$p.value # グループ間のt検定を実行
  return(result) # p値を返す
}
```

この関数は，引数としてサンプルサイズ `n`，平均値 `mu`，標準偏差 `sigma`，効果量 `delta` をとり，2 群の t 検定の結果である p 値を返す関数である。

```
# 使用例；t検定の結果のp値が戻ってくる
studyMake(n = 10, mu = 10, sigma = 1, delta = 0)
```

```
[1] 0.9444895
```

これ一回で 1 分析するので，これを複数回行って一つの研究とし，一つの論文のなかで num_studies 回の研究を行ったとしよう。今回は num_studies = 3 としている。R の replicate 関数で研究回数繰り返した p 値ベクトルを得て，どこかに差が検出されるかどうかをチェックする。「どこかに」を表現するために any 関数を使って判定する。判定する有意水準として，α と補正をかけた α_{adj} の 2 つを用意した。

```
set.seed(12345) # 乱数のシードを設定して再現性を確保
iter <- 1000 # シミュレーションの繰り返し回数を1000に設定
alpha <- 0.05 # 有意水準を0.05に設定
num_studies <- 3 # 研究の数を3に設定
alpha_adjust <- alpha / num_studies # 多重検定補正後の有意水準を計算

FLG.detect <- rep(NA, iter) # 検出結果を保存するベクトルを初期化
FLG.detect.adj <- rep(NA, iter) # 補正後の検出結果を保存するベクトルを初期化
for (i in 1:iter) { # 1000回のシミュレーションを繰り返すループ
  p_values <- replicate(num_studies, studyMake(n = 10, mu = 10, sigma = 1, delta = 0)) # 各研究のp値を
  FLG.detect[i] <- ifelse(any(p_values < alpha), 1, 0) # 補正前の有意差検出を判定して保存
  FLG.detect.adj[i] <- ifelse(any(p_values < alpha_adjust), 1, 0) # 補正後の有意差検出を判定して保存
}

FLG.detect %>% mean() # 補正前の有意差検出率を計算
```

```
[1] 0.145
```

```
FLG.detect.adj %>% mean() # 補正後の有意差検出率を計算
```

```
[1] 0.049
```

結果を見ると，α 水準のまま検定を行うと，論文全体でのタイプ 1 エラー率が 0.145 と 5% を上回っており，3 つの研究のどこかで間違った判断をしていることがわかる。補正すると 0.049 と正しく制御されている。

一連の研究をまとめた一つの論文に，複数の研究が含まれていることは少なくない。各検定結果をまとめて総合考察とすることも一般的である。総合考察は各分析結果から全体的な結論を導くのだが，その要素のどこかに間違いがあると，全体の論立てが崩れてしまうことにもなりかねない。いわば腐った支柱が紛れ込んでいる土台の上に家屋を建てるようなもので，研究の積み重ねを目的とする科学活動の一環である以上，正しく制御されていることは重要である。

10.1.3　N 増し問題

人間を対象にした研究を行って，データを一生懸命取る。その結果，効果があると見られた操作/介入から統計的な有意差が検出されなければ，「悔しい」という心情になることは理解できる。もう少し実験を工夫すれば，もう少しデータが違えばよかったのでは，と思うかもしれない。ではもう少し**頑張って**データを増やしてみればどうだろうか。

実はこの考え方は QRPs のひとつである。検定は真偽判定をする競技のようなものなので，ゲームの途中でプレイヤーの人数が変わるのはよろしくない。このことをシミュレーションで確認してみよう。

以下のコードは，t 検定のデータを最初 `n1=n2=10` で作成して行っている。タイプ 1 エラーの検証をするので，効果量は 0 である。ここで t 検定を行い，もしその p 値が α よりも大きかったら，つまり有意であると判断されなかったら，同じ方法でデータを 1 件追加する。そしてまた t 検定を行う。このサンプルの追加は，効果量 0 なので，偶然のお許しが出るまでいつまで経っても終わることがないため，上限を 100 にしてある。上限に達したら流石に諦めてもらうとして，さてそうした QRPs な努力の結果，α 水準はどれぐらいに保たれているだろうか。

```
iter <- 1000 # シミュレーションの繰り返し回数を1000に設定
alpha <- 0.05 # 有意水準を0.05に設定
p <- rep(0, iter) # p値を保存するベクトルを初期化
add.vec <- rep(0, iter) # 増やした人数を保存するベクトルを初期化

set.seed(123) # 乱数のシードを設定して再現性を確保

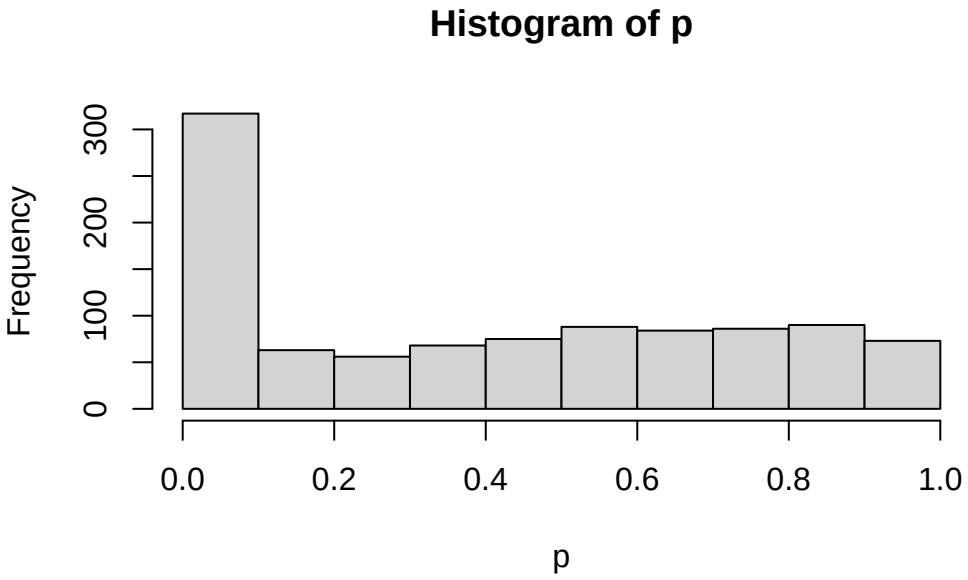
n1 <- n2 <- 10 # 各グループのサンプルサイズを10に設定
mu <- 10 # 平均値を10に設定
sigma <- 2 # 標準偏差を2に設定
delta <- 0 # 平均の差を0に設定

## シミュレーション本体
for (i in 1:iter) { # 1000回のシミュレーションを繰り返すループ
  # 最初のデータを生成
  Y1 <- rnorm(n1, mu, sigma) # グループ1のデータを生成
  Y2 <- rnorm(n2, mu + sigma * delta, sigma) # グループ2のデータを生成
  p[i] <- t.test(Y1, Y2)$p.value # t検定を実行しp値を保存
  # データを追加する
  count <- 0 # 追加したデータの数をカウント
  ## p値が5%を下回るか、データが100になるまでデータを増やし続ける
  while (p[i] >= alpha && count < 100) { # 条件を満たすまでループを繰り返す
    # 有意でなかった場合、変数ごとに1つずつデータを追加
    Y1_add <- rnorm(1, mu, sigma) # グループ1に新しいデータを1つ追加
    Y2_add <- rnorm(1, mu + sigma * delta, sigma) # グループ2に新しいデータを1つ追加
    Y1 <- c(Y1, Y1_add) # グループ1のデータを更新
    Y2 <- c(Y2, Y2_add) # グループ2のデータを更新
    p[i] <- t.test(Y1, Y2)$p.value # 新しいデータでt検定を再度実行しp値を更新
    count <- count + 1 # データを追加した回数をカウント
  }
  add.vec[i] <- count
}
```

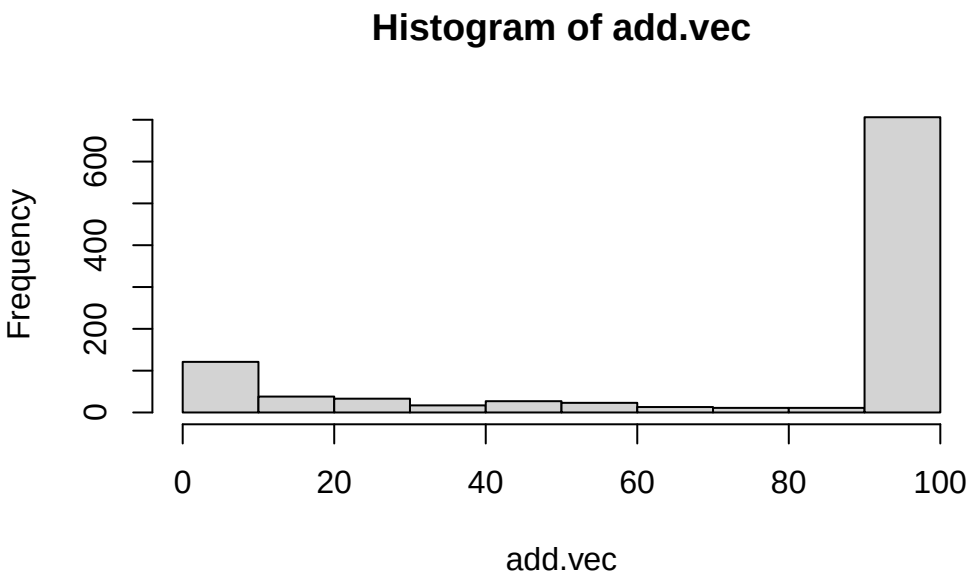
```
## 結果
ifelse(p < 0.05, 1, 0) |> mean() # p値が5%未満の割合を計算
```

```
[1] 0.306
```

```
hist(p)
```




```
hist(add.vec)
```



結果をみると，0.306 とかなり逸脱して，誤った結論に辿り着いていることがわかる。努力の結果得られた有意差は，偶然の賜物でもあり，誤った研究実践による幻想にすぎない。加えたデータのヒストグラムからわかるように，悲しいかな，75% もの割合で上限 100 まで達してしまう。百害あって一利なしとはこのことである。

10.1.4 サンプルサイズを事前に決めないことの問題

サンプルサイズを事前に決めずに検定する，という状況を別の角度から見てみよう。クルシュケ（2017）は「コインフリップを 24 回して，うち 7 回表が出た」というシーンを例に挙げて説明している。7/24 は半分を下回っているから，やや裏が出やすいコインであるように思える。帰無仮説として，このコインは公平である (表と裏が出る確率が半々である)，というのを検証したいとする。

この 24 回中 7 回成功，という話の背後に「24 施行する」ということを決めていたかどうか (サンプルサイズを事前に決めていたか) ということを考えてみよう。

まずは正直に，最初から 24 回コインフリップすることを決めていたとする。コインフリップはベルヌーイ試行[*2]であり，それを繰り返すので二項分布に従うと考えられる。そこで，二項検定として次のように計算できるだろう。

```
N <- 24
# 7回表が出る確率
pbinom(7, N, 0.5) * 2
```

```
[1] 0.06391466
```

二項分布の p 値を出すには pbinom を使いった。また帰無仮説として，このコインフリップは公平であると考えているのだから，$\theta = 0.5$ が帰無仮説の状態である。この $\theta = 0.5$ とした時に，$N = 24, k = 7$ という結果になる確率を計算し，かつ両側検定 (公平でない，が対立仮説なので裏が 7 回でもよい) であることを考えて確率を 2 倍した。p 値は 0.0639147 であるから，5% 水準では有意であると判定できない。これぐらいの確率はあるということだ。

しかしここで第二の状況を考えてみよう。24 回コインフリップすることを決めていたのではなくて，7 回成功するまでコインフリップを続けたところ，結果的に 24 回で終わったということだった，とするのである。このようなシーンの確率分布は負の二項分布と呼ばれ，pnbinom で次のように計算できる。

```
k <- 7
# 24回以上必要な確率
pnbinom(k, 24, 0.5) * 2
```

```
[1] 0.003326893
```

この結果から，$\theta = 0.5$ の時に 7 回表がでるまでに 24 施行も必要とする確率は，0.0033269 だから，5% 水準で有意である。つまり，滅多にこんなことが起きないので，$\theta = 0.5$ という帰無仮説が疑わしいことになる。ここではシーンが異なると p 値が違っている，ということに注意してほしい。

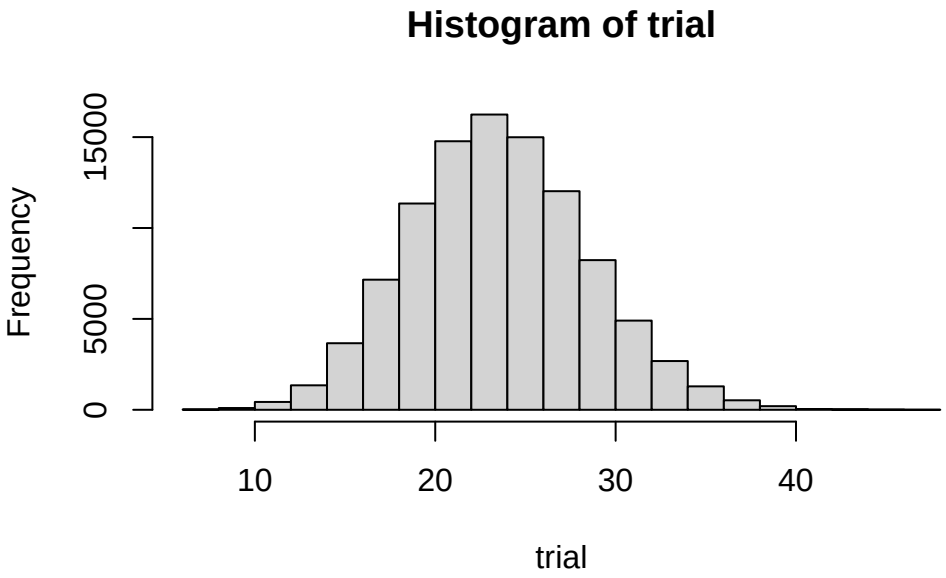
さらに第 3 のシーンを考えよう。これは「何回やるかは決めてないけど，まあ 5 分ぐらいかな」と試行にかける時間だけ決めていたという状況である。結果的に 24 回になったけど，もしかすると 23 回だったかもしれないし，25 回や 20 回，30 回だったかもしれない。これをシミュレーションするために，「24 がピークになるような頻度の

[*2] 表 (1) が出るか，裏 (0) が出るか，という 2 値の結果変数だけを持つ施行のことで，この確率変数がベルヌーイ分布にし違う。ベルヌーイ分布は，表が出る確率 θ をパラメータに持つ。$P(X = k) = \theta^k(1-\theta)^{1-k}$, ただし$k = \{0, 1\}$ という確率変数である。1/0 というのが生死，男女，成功失敗などさまざまなメタファに適用できるので応用範囲が広い。

分布」をポアソン分布を使って生成する[*3]。

[*4]ポアソン分布は正の整数を実現値に取る分布で，カウント変数の確率分布として用いられる。パラメータは λ だけであり，期待値と分散が λ に一致する，非常にシンプルな分布である。

```
set.seed(12345)
iter <- 100000 # 発生させる乱数の数
## 24回がピークに来るトライアル回数
trial <- rpois(iter, 24)
hist(trial)
```



この各トライアルにおいて，二項分布で成功した回数を計算し，トライアル回数で割ることによって，表が出る確率のシミュレーションができる。その時の割合は，7/24 よりもレアな現象だろうか?

```
result <- rep(NA, iter)
for (i in 1:iter) {
  result[i] <- rbinom(1, trial[i], 0.5) / trial[i]
}
## 7/24よりも小さい確率で起こった?
length(result[result < (7 / 24)]) / iter
```

```
[1] 0.02262
```

これを見ると，両側検定にしても 0.04524 なので，ギリギリ有意になるかどうか，というところだろうか。

さて判断にこまった。「24 回やる」と決めていたのであれば $\theta = 0.5$ は棄却されないし，「7 回成功するまで」と決めていたのであれば $\theta = 0.5$ は棄却される。「5 分間」と決めていても棄却されるが，そもそもこうした実験者の

[*3] もちろん基準となる有意水準 α，検出力 $1-\beta$ も定める必要があるが，慣例的に $\alpha = 0.05$ であり，$1-\beta = 0.8$ ぐらいが必要とされている。

[*4] もちろん基準となる有意水準 α，検出力 $1-\beta$ も定める必要があるが，慣例的に $\alpha = 0.05$ であり，$1-\beta = 0.8$ ぐらいが必要とされている。

意図によって判断が揺らいで良いものだろうか?というのがクルシュケ（2017）の指摘する疑問点である。

問題は，「24 回中 7 回成功」という事実に，二項分布，負の二項分布，あるいは組み合わさった分布のような，確率分布の情報が含まれていないことにある。この確率分布はデータが既知で母数が未知だから尤度関数であり，データ生成メカニズムであるとも言えるだろう。想定するメカニズムが明示されない検定は，ともすれば事後的に「実は負の二項分布を想定していたんですよ，へへ」ということも可能になってしまう。こうした点からも，研究者の自由度をなるべく少なくする研究計画の**事前登録**制度が必要であることがわかる。

10.2　サンプルサイズ設計

どのようにデータを取り，どのように分析・検定し，どのような基準で判断するかを事前に決めることに加え，事前にサンプルサイズを見積もっておく必要があるだろう。サンプルはとにかく多ければ多いほど良いか，というとそうではなく，過剰にサンプルを集めることは研究コストの増大であり，回答者の負担増でしかない。またサンプルサイズが大きくなると有意差を検出しやすくなるが，必要なのは有意差ではなく実質的に効果を見積もることであり，有意差が見つかれば良いというものではないことに注意が必要である。もちろん上で見てきたように，有意差が検出できるかどうかを指標にしてサンプルサイズを変動させてしまうのは，明らかに誤った研究実践である。

事前にサンプルサイズを決定するのに必要なのは，これまでのリバースエンジニアリングの演習例からもわかるように，効果量の見積もりである。[*5] これをどのように定めるかについては，先行研究を考えるとか，研究領域で「これぐらい差がないと意味がないよね」とコンセンサスが取れている程度で決めることになる。[*6]

10.2.1　対応のない t 検定

サンプルサイズの設計には，これまで使ってきた検定統計量に**非心度** non-centrality parameter というパラメータを加えて考える必要がある。

具体的に，対応のない t 検定を例にサンプルサイズ設計の方法を見てみよう。t 検定は言葉の通り，t 分布を用いて帰無仮説の元での検定統計量の実現値が問題になるのであった。帰無仮説は $\mu_0 = \mu_1 - \mu_2 = 0$ であり，検定統計量は次式で表されるのであった。

$$T = \frac{d - \mu_0}{\sqrt{U_p^2 / \frac{n_1 n_2}{n_1 + n_2}}}$$

この分子において，$d - \mu_0 = (\bar{X}_1 - \bar{X}_2) - (\mu_1 - \mu_2)$ の第二項を,$\mu_1 - \mu_2 = 0$ と仮定するから，0 が中心の標準化された t 分布が用いられたのである。帰無仮説はこのように理論的に特定できる比較点をもとに置かれているのであって，帰無仮説下ではない現実の世界では，検定統計量は母平均の差 $\mu_1 - \mu_2$ に応じた分布から生じている。このように中心がずれている t 分布のことを**非心分布**といい，ズレの程度が非心度パラメータである。t 検定における非心度 λ は，以下の式で表される。

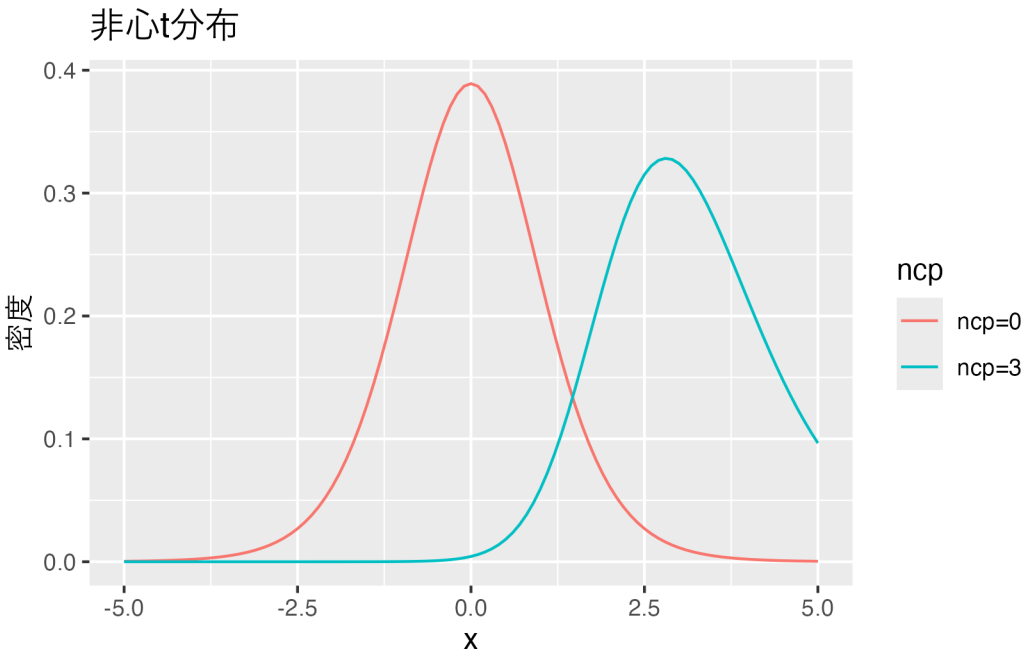
[*5] もちろん基準となる有意水準 α，検出力 $1 - \beta$ も定める必要があるが，慣例的に $\alpha = 0.05$ であり，$1 - \beta = 0.8$ ぐらいが必要とされている。

[*6] この「最低限検出したい効果」のことを Smallest Effect Size of Interest, SESOI と呼ぶ。小杉他（2023）も参照。

$$\lambda = \frac{(\mu_1 - \mu 2) - \mu_0}{\sigma\sqrt{n}}$$

この非心度の分だけ，非心 t 分布は t 分布からズレていることになる。R では dt 関数に ncp パラメータがあり，デフォルトでは ncp=0 になっていた。これを変えて描画してみよう。

```
# データの準備
df <- 10 # 自由度を指定
# ggplotでプロット
ggplot(data.frame(x = c(-5, 5)), aes(x = x)) +
  stat_function(fun = dt, args = list(df = df, ncp = 0), aes(color = "ncp=0")) +
  stat_function(fun = dt, args = list(df = df, ncp = 3), aes(color = "ncp=3")) +
  labs(
    title = "非心t分布",
    x = "x",
    y = "密度",
    color = "ncp"
  )
```



ncp=0 の時は，中心が 0 にある帰無仮説の世界であり，これを使ってタイプ 1 エラー，つまり α が算出されたのであった。ncp を効果量で表現すれば，母平均の差がゼロでない時の分布が描けるのだから，タイプ 2 エラー，つまり β が計算できる。

自由度 df = 10，非心度 ncp = 3 の例で考えてみよう。タイプ 1 エラーになるのは，自由度 10 の t 分布で上 2.5% の臨界値以上の実現値が得られた時である。

```
qt(0.975, df = 10, ncp = 0)
```

```
[1] 2.228139
```

このとき，実際は ncp = 3 ほどずれていたのだから，タイプ 2 エラーが生じる確率は次のとおりである。

```
qt(0.975, df = 10, ncp = 0) %>% pt(df = 10, ncp = 3)
```

```
[1] 0.2285998
```

当然，ncp = 0 から離れるほどタイプ 2 エラーは生じにくくなる。非心度は母効果量 $\delta = \frac{(\mu_1 - \mu_2) - \mu_0}{\delta}$ を使って，$\lambda = \delta\sqrt{n}$ で表すことができる。

これを使って，t 検定のサンプルサイズを設計してみよう。話を簡単にするために，サンプルサイズは 2 群で等しいものとする。

検定統計量の式を思い出して，$\sqrt{n}$ にあたるところは 2 群のサンプルサイズから計算される，プールされた標本サイズから得られることに注意しよう[*7]。

```
alpha <- 0.05
beta <- 0.2
delta <- 0.5

for (n in 10:1000) {
  df <- n + n - 2
  lambda <- delta * (sqrt((n * n) / (n + n)))
  cv <- qt(p = 1 - alpha / 2, df = df) # Type1errorの臨界値
  er <- pt(cv, df = df, ncp = lambda) # Type2errorの確率
  if (er <= beta) {
    break
  }
}
print(n)
```

```
[1] 64
```

ここでは，サンプルサイズを 10 から徐々に増やしていき，1000 までの間で目標とする β まで抑えられたところで，カウントしていく for ループを break で脱出する，というかたちで組んでいる。結果的に，各群 64 名，合計 128 名のサンプルがあれば，目標が達成できることがわかる。サンプルサイズが 2 群で異なる場合など，詳細は @kosugi2023 に詳しい。

[*7] t 統計量の実現値の式にある分母，$\sqrt{U_p^2/\frac{n_1 n_2}{n_1+n_2}}$ に見られる，プールされた不偏分散を割るための標本サイズであり，2 群の母分散が等しいと仮定して計算するなら，$\sigma^2(\frac{1}{n_1}+\frac{1}{n_2}) = \sigma^2(\frac{n_1+n_2}{n_1 n_2}) = \sigma^2/\frac{n_1 n_2}{n_1+n_2}$ から得られる。

10.2.2　シミュレーションによるサンプルサイズ設計

非心 F 分布を使えば分散分析でもサンプルサイズができるし，そのほかの検定についても同様に非心分布を活用すると良い。しかし，非心分布の理解や非心度の計算など，ケースバイケースで学ぶべきことは多い。

そこで，電子計算機の演算力をたのみに，データ生成のシミュレーションを通じて設計していくことを考えてみよう。サンプルサイズや効果量を定めれば，仮想データを作ることができるし，それ対して検定をかけることもできる。仮想データの生成と検定を反復し，タイプ 2 エラーがどの程度生じるかを相対度数で近似することもできるだろう。であれば，その近似をサンプルサイズを徐々に変えることで繰り返してサンプルサイズを定めることもできる。

以下は，母相関が $\rho = 0.5$ とした時に，検出力が 80% になるために必要なサンプルサイズを求めるシミュレーションコードである。

```
pacman::p_load(MASS)
set.seed(12345)
alpha <- 0.05
beta <- 0.2
rho <- 0.5
sd <- 1
Sigma <- matrix(NA, ncol = 2, nrow = 2)
Sigma[1, 1] <- Sigma[2, 2] <- sd^2
Sigma[1, 2] <- Sigma[2, 1] <- sd * sd * rho

iter <- 1000

for (n in seq(from = 10, to = 1000, by = 1)) {
  FLG <- rep(0, iter)
  for (i in 1:iter) {
    X <- mvrnorm(n, c(0, 0), Sigma)
    cor_test <- cor.test(X[, 1], X[, 2])
    FLG[i] <- ifelse(cor_test$p.value > alpha, 1, 0)
  }
  t2error <- mean(FLG)
  print(paste("n=", n, "のとき，betaは", t2error, "です。"))
  if (t2error <= beta) {
    break
  }
}
```

```
[1] "n= 10 のとき，betaは 0.681 です。"
```

```
[1] "n= 11 のとき，betaは 0.639 です。"
[1] "n= 12 のとき，betaは 0.612 です。"
[1] "n= 13 のとき，betaは 0.566 です。"
[1] "n= 14 のとき，betaは 0.563 です。"
[1] "n= 15 のとき，betaは 0.471 です。"
[1] "n= 16 のとき，betaは 0.462 です。"
[1] "n= 17 のとき，betaは 0.419 です。"
[1] "n= 18 のとき，betaは 0.402 です。"
[1] "n= 19 のとき，betaは 0.385 です。"
[1] "n= 20 のとき，betaは 0.353 です。"
[1] "n= 21 のとき，betaは 0.344 です。"
[1] "n= 22 のとき，betaは 0.312 です。"
[1] "n= 23 のとき，betaは 0.285 です。"
[1] "n= 24 のとき，betaは 0.256 です。"
[1] "n= 25 のとき，betaは 0.265 です。"
[1] "n= 26 のとき，betaは 0.21 です。"
[1] "n= 27 のとき，betaは 0.227 です。"
[1] "n= 28 のとき，betaは 0.176 です。"
```

```
print(n)
```

```
[1] 28
```

ここではシミュレーション回数 1000，上限 1000，刻み幅を 1 にしているが，状況に応じて変更すると良い。

10.3 課題

1. 一要因 3 水準の Between デザインの分散分析において、1. 分散分析で有意差が見られる場合と、2. 任意の 2 水準の組み合わせのどこかで有意差が見られる場合を考えたとき、タイプ 1 エラーはどのように異なるかをシミュレーションで確かめてみましょう。設定として、`n1=n2=n3=10`、標準偏差も各群等しく `sigma = 1` としてみましょう。
2. N 増し問題は相関係数の検定の時も生じるでしょうか。母相関が $\rho = 0.0$ のとき、サンプルサイズを 10 から始めて、有意になるまでデータを追加する仮想研究を 1000 回行ってみましょう。データ追加の上限は 100、有意水準は $\alpha = 0.05$ として、最終的に有意になる割合を計算してみましょう。
3. $\alpha = 0.05, \beta = 0.2$ とし、効果量 $\delta = 1$ とした時の対応のない t 検定のサンプルサイズ設計をしたいです。1. 非心 t 分布を使った解析的な方法と、2. シミュレーションによる近似的な方法の両方で、同等の結果が出ることを確認しましょう。

第 11 章

重回帰分析の基礎

11.1　回帰分析の基礎

ここでは回帰分析を扱う。説明変数 x と被説明変数 y の関数関係 $y = f(x)$ に，次の一次式を当てはめるのが単回帰分析である。

$$y_i = \beta_0 + \beta_1 x_i + e_i = \hat{y}_i + e_i$$

一次式を $\hat{y}$ とまとめたものを予測値といい，予測値と実測値 y の差分 e_i を残差 residuals という。

空間上の一次直線の切片，傾きを求めるというのが基本的な問題であり，二点であれば一意に定めることができるが，データ分析の場面では 3 点以上の多くのデータセットの中に直線を当てはめることになるので，なんらかの外的な基準が必要になる。この時，「残差の分散が最も小さくなるように」と考えるのが**最小二乗法**の考え方であり，「残差が正規分布に従っていると考え，その尤度が最も大きくなるように」と考えるのが**最尤法**の考え方である。前者は記述統計的な，後者は確率モデルとしての感が過多になっていることに注意してほしい。また確率モデルの推定方法としては，事前分布を用いた**ベイズ推定**が用いられることもある。

最小二乗法による推定値は，次の式で表される。証明は他書 (小杉，2018; 西内，2017) に譲るが，ロジックとして残差の二乗和 $\sum e_i^2 = \sum (y_i - (\beta_0 + \beta_1 x_i))^2$ を最小にすることを考え，この式を展開するか偏微分を用いて極小値を求めることで算出できるとだけ伝えておこう。いずれにせよ，平均値 $\bar{x}, \bar{y}$ や分散・共分散 s_x, s_y, r_{xy} など標本統計量から推定できるのはありがたいことである。

$$\beta_0 = \bar{y} - \beta_1 \bar{x}, \quad \beta_1 = r_{xy} \frac{s_y}{s_x}$$

また，ここでは x, y ともに連続変数を想定しているが，説明変数 x が二値，あるいはカテゴリカルなものであれば y の平均値を通る直線を探すことになる。直線の傾きが 0 であれば「平均値が同じ」という線型モデルであり，これは平均値差の検定における帰無仮説と同等である。このように，t 検定や ANOVA は回帰分析の特殊ケースとも考えられ，まとめて**一般線型モデル**と呼ばれる。一般線型モデルは，被説明変数が連続的で，線型モデルによる平均値に正規分布に従う残差が加わったものとして考えるという意味で統一的に表現される。

ANOVA の場合は，二つ以上の要因による効果を考えることもあった。交互作用項を考慮しなければ，2 要因のモデルは次のように表現することができる。

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + e_i$$

このように説明変数が複数ある回帰分析を特に重回帰分析 Multiple Regression Analysis と呼ぶ。一次式なので，ある変数に限れば線型性が担保されているから，これも線型モデルの仲間である。重回帰分析を用いる場合は，説明変数同士を比較して「どちらの説明変数の方が影響力が大きいか」ということが論じられることが多いが，係数は当然 x_n, y の単位に依存するため，素点の回帰係数は使い勝手が悪い。そこですべての変数を標準化した標準化係数が用いられることが多い。

11.2　回帰分析の特徴

以下，具体的なデータを用いて回帰分析の特徴を見てみよう

11.2.1　パラメータリカバリ

回帰分析のモデル式にそってデータを生成し，分析によってパラメータリカバリを行ってみよう。

説明変数については制約がないので一様乱数から生成し，平均 0，標準偏差 σ の誤差とともに被説明変数を作る。

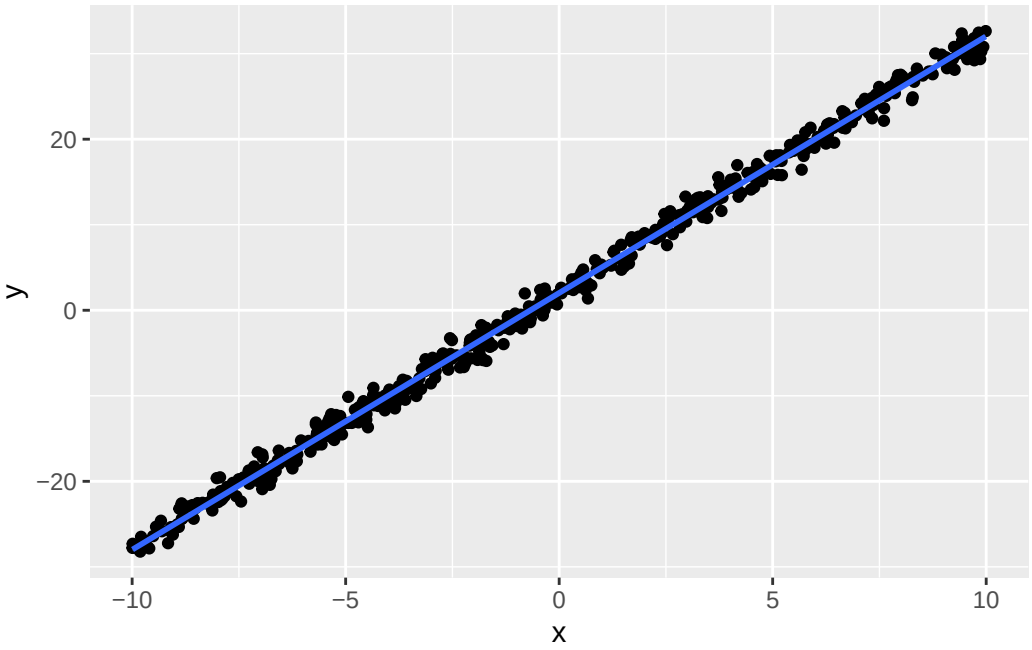
```
pacman::p_load(tidyverse)
set.seed(123)
n <- 500
beta0 <- 2
beta1 <- 3
sigma <- 1
# データの生成
x <- runif(n, -10, 10)
e <- rnorm(n, 0, sigma)
y <- beta0 + beta1 * x + e

dat <- data.frame(x, y)
# データの確認
head(dat)
```

```
          x          y
1 -4.248450 -11.120952
2  5.766103  18.736432
3 -1.820462  -3.805302
4  7.660348  25.071541
5  8.809346  30.026546
```

```
6 -9.088870 -25.355175
```

```
dat %>% ggplot(aes(x = x, y = y)) +
  geom_point() +
  geom_smooth(method = "lm", formula = "y~x") # 線型モデルの描画
```



このデータに基づいて回帰分析を実行した結果が以下のとおりである。

```
result.lm <- lm(y ~ x, data = dat)
summary(result.lm)
```

```
Call:
lm(formula = y ~ x, data = dat)

Residuals:
     Min       1Q   Median       3Q      Max
-2.82796 -0.61831  0.03553  0.69367  2.68062

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2.021928   0.045010   44.92   <2e-16 ***
x           3.002194   0.007919  379.09   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.006 on 498 degrees of freedom
```

```
Multiple R-squared:  0.9965,    Adjusted R-squared:  0.9965
F-statistic: 1.437e+05 on 1 and 498 DF,  p-value: < 2.2e-16
```

ここでは $\beta_0 = 2, \beta_1 = 3, \sigma = 1$ と設定しており，ほぼ理論通りの係数がリカバリーできていることを出力から確認しておこう。もちろんリカバリの精度は，データの線型性の強さに依存するから，残差の分散が大きかったりサンプルサイズが小さくなると，必ずしもうまくリカバリできないことがあることは想像に難くないだろう。

11.2.2　残差の正規性と相関関係

lm 関数が返した結果オブジェクトには，表示されていない多くの情報が含まれている。例えば予測値や残差も含まれているので，これを使って回帰分析の特徴を見てみよう。

```
dat <- bind_cols(dat, yhat = result.lm$fitted.values, residuals = result.lm$residuals)
summary(dat)
```

```
       x                   y                yhat             residuals
 Min.   :-9.99069   Min.   :-28.216   Min.   :-27.9721   Min.   :-2.82796
 1st Qu.:-5.08007   1st Qu.:-13.074   1st Qu.:-13.2294   1st Qu.:-0.61831
 Median :-0.46887   Median :  0.301   Median :  0.6143   Median : 0.03553
 Mean   :-0.09433   Mean   :  1.739   Mean   :  1.7387   Mean   : 0.00000
 3rd Qu.: 4.65795   3rd Qu.: 15.963   3rd Qu.: 16.0060   3rd Qu.: 0.69367
 Max.   : 9.98809   Max.   : 32.638   Max.   : 32.0081   Max.   : 2.68062
```

予測値 $\hat{y}$ は fitted.values として保存されている。これの平均値が被説明変数 y の平均値に一致していることが確認できる。回帰分析は説明変数 x を伸ばしたり (β_1 倍する) ズラしたり (β_0 を加える) しながら，被説明変数 y に当てはめるのであり，位置合わせがなされた予測値の中心が被説明変数の中心と一致することは理解しやすいだろう[*1]。

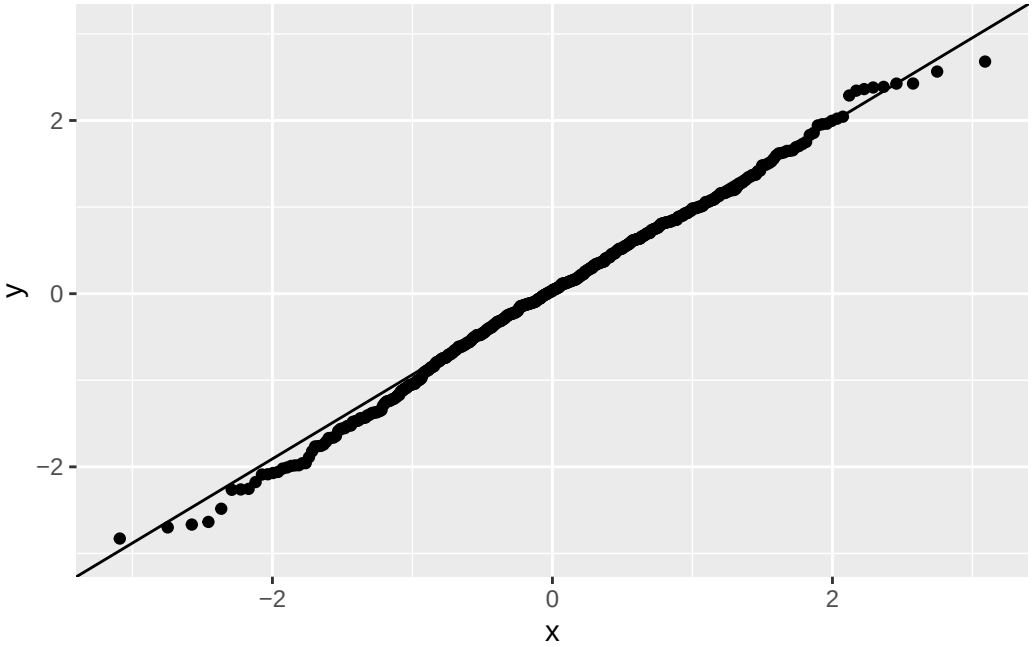
次に，残差の平均が 0 になっていることも確認しておこう。これが 0 でない c であれば，回帰係数が常に c だけズレていることになるので，そのような系統的ズレは最適な線型の当てはめにおいて除外されているべきだからである[*2]。

また，回帰分析において残差は正規分布に従うことが仮定されていた。これを検証するには Q-Q プロットを見ると良い。

```
dat %>%
  ggplot(aes(sample = residuals)) +
  stat_qq() +
  stat_qq_line()
```

[*1] もちろん証明できる。$\beta_0 = \bar{y} - \beta_1 \bar{x}, \beta_1 = r_{xy}\frac{s_y}{s_x}$ より，$\bar{\hat{y}} = \frac{1}{n}\sum(\bar{y} - \beta_1\bar{x} + \beta_1 x_i) = \bar{y} - \beta_1\bar{x} + \beta_1\frac{1}{n}\sum x_i = \bar{y}$ である。

[*2] もちろん証明できる。$\bar{e} = \frac{1}{n}\sum e_i = \frac{1}{n}\sum(y_i - \hat{y}_i) = \bar{y} = \bar{\hat{y}} = 0$ である。



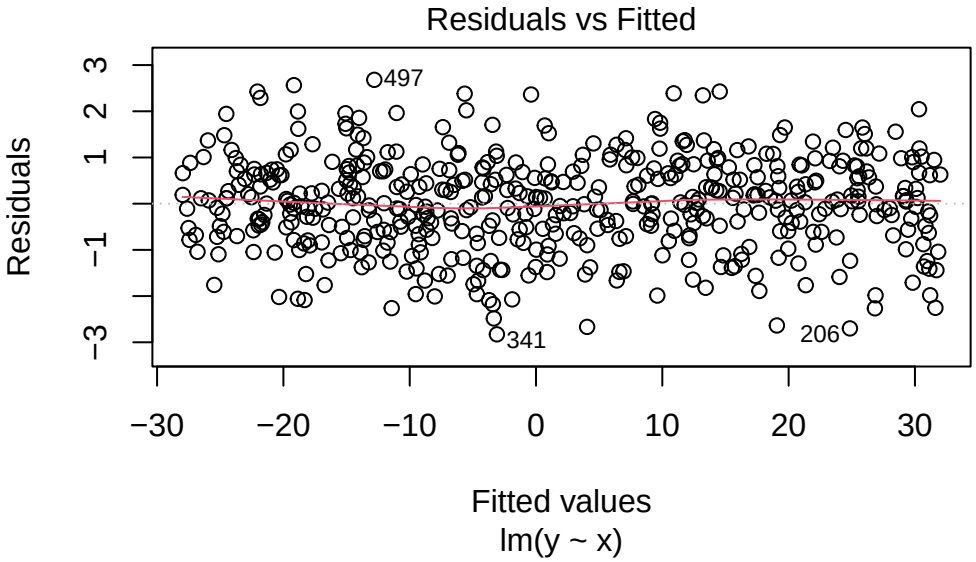
Q-Q プロットとは 2 つの確率分布を比較するためのグラフであり，横軸には理論的分布の分位点が，縦軸に実データが並ぶもので，右上がりの直線上にデータが載っていれば分布に従っている，と判断するものである。直線から逸脱している点は理論的分布からの逸脱と考えられる。今回の結果はほとんどが正規分布の直線上にあることから，大きな逸脱がないことが認められる。

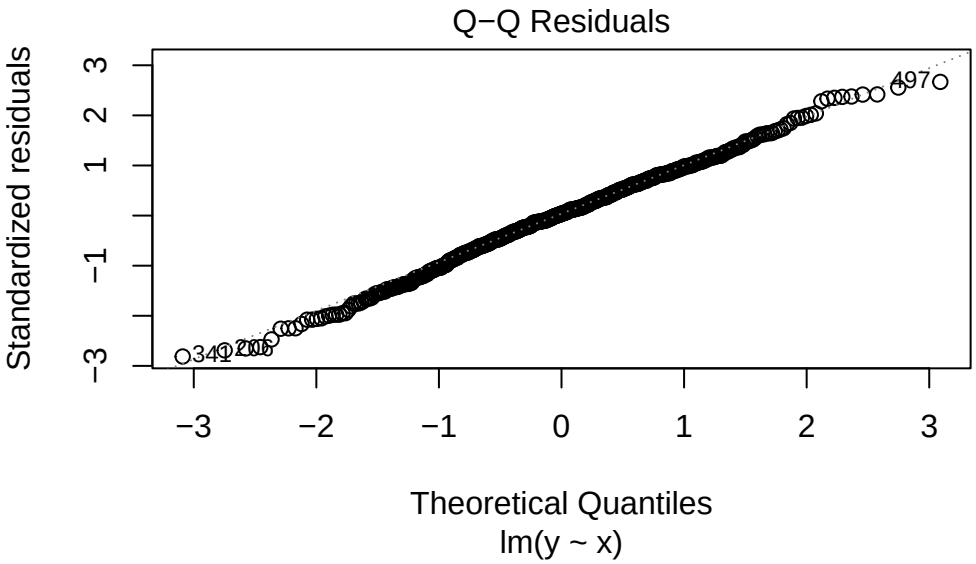
データ生成メカニズムによっては，被説明変数が二値的であったり，順序的であったり，カウント変数であったり，と正規分布がそぐわないものもあるだろう。そのようなデータに無理やり回帰分析を当てはめることは適切ではない。いかなる時もデータは可視化して，モデルを当てはめることの適切さをチェックすることを忘れたはならない。

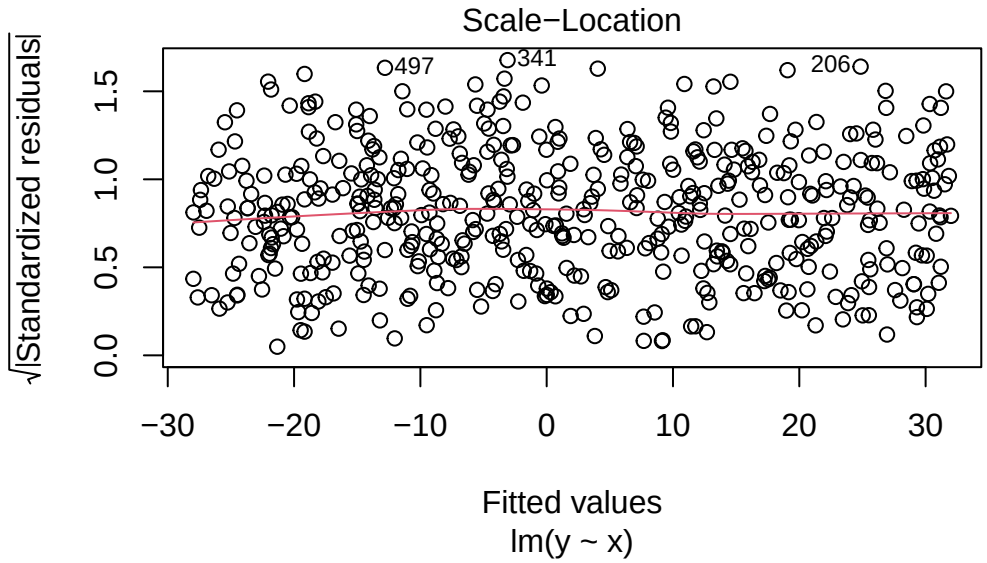
ちなみに出力結果を直接 `plot` 関数に入れてもよい。ここから残差と予測値の相関関係や，Q-Q プロット，標準化残差のスケールロケーションプロット，レバレッジと標準化残差[*3]などがプロットされる。

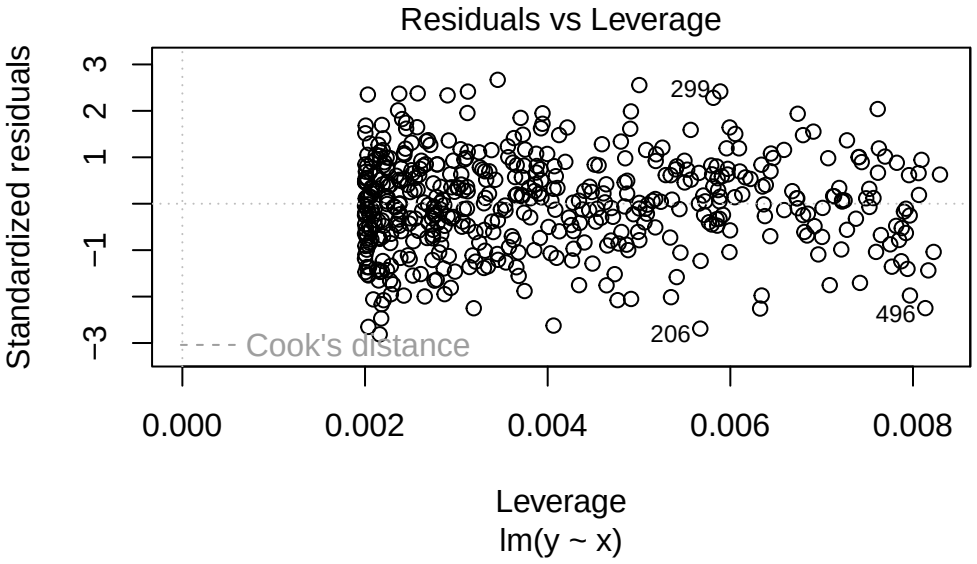
```
plot(result.lm)
```

[*3] 縦軸の標準化された残差の大きな値は解釈に注意が必要な外れ値である可能性が高い。レバレッジも同様に回帰係数に大きな影響を与える値の指標であり，この図の端に位置する変数は注意が必要，と考える。


Residuals vs Fitted
Residuals
497
341
206
−30
−20
−10
0
10
20
30
−3
−1
1
2
3
Fitted values
lm(y ~ x)

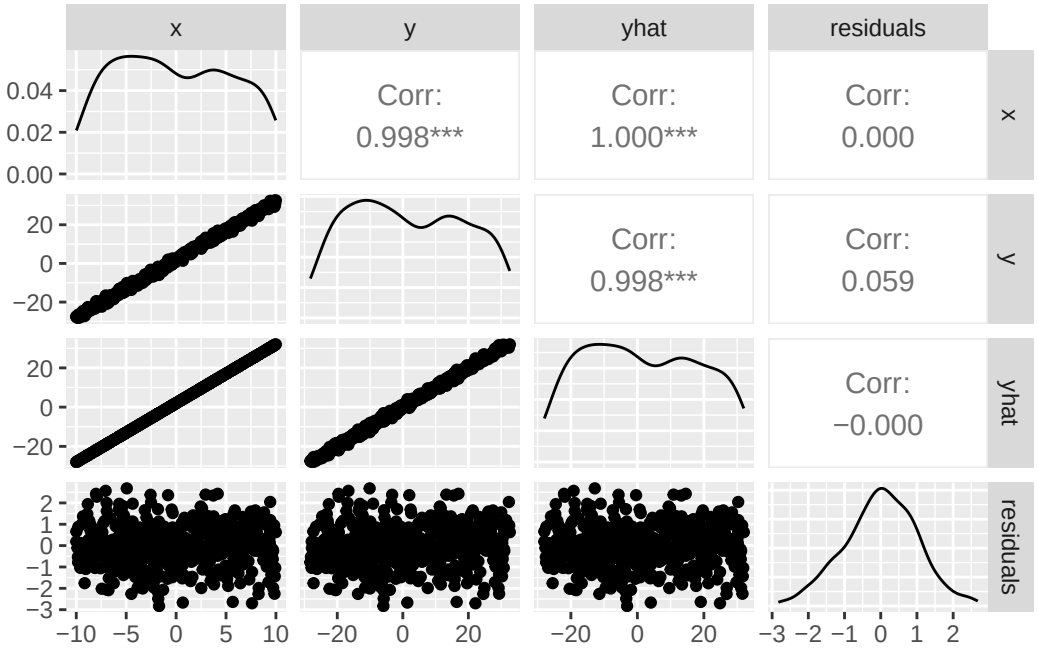

Q−Q Residuals
Standardized residuals
497
341
206
−3
−2
−1
0
1
2
3
Theoretical Quantiles
lm(y ~ x)


Scale−Location
√|Standardized residuals|
497
341
206
0.0
0.5
1.0
1.5
−30
−20
−10
0
10
20
30
Fitted values
lm(y ~ x)



残差と予測値のプロットからも想像できるが，両者の相関はゼロである。図で確認しておこう。

```
pacman::p_load(GGally) # 必要ならインストールしよう
ggpairs(dat)
```



この関係から明らかなように，残差は説明変数や予測値と相関しない[*4]。説明変数と残差に相関関係があるとすると，説明変数でまだ説明できていない分散が残っていることになるし，予測値と残差に相関がないことは予測値が高いか低いかにかかわらず，残差が一様に分布していることを意味する。このことを踏まえて，重回帰分析の特徴を理解していこう。

[*4] もちろん証明できる。詳細は小杉（2018）を参照すること。

11.3　重回帰分析の特徴

重回帰分析においては，回帰係数は偏回帰係数 partial regression coefficients と呼ばれる。この「偏」の一文字が意味することを考えていこう。

11.3.1　回帰係数と偏回帰係数

単回帰分析の回帰係数は，説明変数 x が一単位上昇した時の被説明変数の変化量，と解釈すればよい。これに対して重回帰分析の偏回帰係数を，「説明変数 x_1 が一単位上昇した時の被説明変数の変化量」とすることはできない。というのも，説明変数が複数 $(x_2, x_3, ...)$ あり，他の説明変数の次元についての変化を考慮していない変化量になっているからである。

重回帰分析において，説明変数が完全に無相関で直交しているのであれば，x_1 の変化と x_2 の変化を独立して説明できるが，往々にしてそのようなことはない。偏回帰係数は当該変数以外の変動を**統制した**回帰係数である。

上で単回帰係数において，説明変数と残差が相関しないことを確認した。言い換えれば，説明変数で説明でき分散は全て説明し尽くされており，残差は説明変数で説明できない被説明変数の分散，つまり説明変数の影響を除外した被説明変数の分散と考えることができる (被説明変数の分散=説明変数が説明する分散 + 残差の分散)。

ここで第二の変数 x_2 があったとする。第一の変数 x_1 で y を説明した残差 e_y と，第一の変数で第二の変数を説明した残差 e_{x2} との相関を**偏相関 partial correlation** という。これは第一の変数 x_1 からの影響を両者から取り除いているので，x_1 で統制した相関係数ということができる。偏相関は単純な相関が「見せかけの関係」でないことを検証するための重要な指標である。

偏相関係数を計算してみよう。

```
pacman::p_load(MASS)
pacman::p_load(psych)
Sigma <- matrix(c(1, 0.3, 0.5, 0.3, 1, 0.8, 0.5, 0.8, 1), ncol = 3)
X <- mvrnorm(1000, c(0, 0, 0), Sigma, empirical = TRUE) %>% as.data.frame()
## 相関行列
cor(X)
```

```
    V1  V2  V3
V1 1.0 0.3 0.5
V2 0.3 1.0 0.8
V3 0.5 0.8 1.0
```

```
## 回帰分析をして残差を求める
result.lm1 <- lm(V2 ~ V1, data = X)
result.lm2 <- lm(V3 ~ V1, data = X)
cor(result.lm1$residuals, result.lm2$residuals)
```

```
[1] 0.7867958
```

```
## 偏相関を求めるR関数で確認
psych::partial.r(X)[2, 3]
```

```
[1] 0.7867958
```

最後は psych パッケージの偏相関行列を求める関数で検証した。確かに残差同士の相関係数が偏相関係数になっていることが確認できたと思う。

そして，ここでは残差同士の相関係数として算出しているが，残差をつかった回帰分析の係数が偏回帰係数になるのである。このデータセットの第一変数を従属変数にした重回帰分析の結果から，これを確認してみよう。

```
result.mra <- lm(V1 ~ V2 + V3, data = X)
# 回帰係数を取り出す
result.mra$coefficients
```

```
  (Intercept)            V2            V3
-5.218050e-17 -2.777778e-01  7.222222e-01
```

```
# 残差をつかって偏回帰係数を確認する
result.lm3 <- lm(V1 ~ V3, data = X)
result.lm4 <- lm(V2 ~ V3, data = X)
result.lm5 <- lm(result.lm3$residuals ~ result.lm4$residuals)
#
result.lm5$coefficients
```

```
          (Intercept) result.lm4$residuals
       -7.925711e-18        -2.777778e-01
```

重回帰分析の結果 result.mra の V2 から V1 への回帰係数は-0.2778 である。また，V3 で V1,V2 を統制した残差同士をつかい，回帰係数を求めた結果は-0.2778 と，同じ値になっていることが確認できただろう。

V3 から V1 への偏回帰係数も同様で，V2 で両者を統制した残差同士による回帰係数になっている。このように，重回帰分析の回帰係数は，他の説明変数で統制した値になっており，日本語で説明するなら「他の変数の値が同じであると想定した条件つきの，当該変数の影響力」とでもいうべき値になっている。

なぜこのような持って回った説明をするかというと，つい「条件付きの」という話を忘れて報告，解釈してしまうことが多いからで，吉田・村井（2021）の論文での指摘は議論を呼んだのは記憶に新しい[*5]。たとえば今回の例でも，回帰係数が-0.2778 であったのに対し，V1 と V2 の単相関が 0.3 であったことを思い出そう。符号が反転しているため，解釈は真逆になってしまう。実際の単相関は正の関係であるから，条件付きであることを忘れて「V2 は負の影響，V3 は正の影響」と表現してしまうと，ミスリーディングなことになるからである。

[*5] 論文が早期公開された後，心理学会が主催するオンラインシンポジウムでは著者とこの論文で取り上げられた論文の著者が登場して，議論が交わされた (日本心理学会 YouTube ライブ・話題の論文について著者と語るシリーズ,2021 年 7 月 2 日 20 時-21 時 40 分)。平日の夜という設定，早期公開版における議論であったにも関わらず，1700 名近い視聴者が参加した。

また，豊田（2017）は重回帰分析のこうした誤用を避けるために，独立変数を事前に直交化したデザインで行うコンジョイント分析の積極的な利用を提案している。我々が重回帰分析をうまく使いこなせないのであれば，そうした手法も有用であるだろう。

11.3.2　多重共線性

偏回帰係数の解釈が難しい理由の一つは，説明変数同士に相関関係がみられることにある。特に，説明変数間の相関関係が高くなることは**多重共線性** Multicollinearity の問題という。この問題は，回帰係数の標準誤差がインフレを起こすことを指す。

例えば先ほどの例で，説明変数 V2 と V3 は相関係数 0.8 を持っていた。この時の回帰係数の標準誤差を確認しておこう。

```
summary(result.mra)
```

```
Call:
lm(formula = V1 ~ V2 + V3, data = X)

Residuals:
     Min       1Q   Median       3Q      Max
-2.59118 -0.54717 -0.03692  0.55044  2.90735

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -5.218e-17  2.690e-02   0.000        1
V2          -2.778e-01  4.486e-02  -6.192 8.65e-10 ***
V3           7.222e-01  4.486e-02  16.100  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.8507 on 997 degrees of freedom
Multiple R-squared:  0.2778,    Adjusted R-squared:  0.2763
F-statistic: 191.7 on 2 and 997 DF,  p-value: < 2.2e-16
```

標準誤差は 0.0449 であり，それほど大きくない標準誤差で問題がないようである。しかし両者の相関係数がより高くなり，一方が他方に線型的に従属してしまうと係数の推定値が不安定になるため，注意が必要である。

このインフレを確認するための指標が Variance Infration Factor: VIF である。R では car パッケージにある vif 関数に重回帰モデルを入れることでこの指標が算出される。一般に VIF が 3，あるいは 10 を超えると多重共線性が生じており，解釈に注意が必要と言われている[*6]。

[*6] 2 変数重回帰分析モデルで，VIF が 3 であれば説明変数間の相関は $r = 0.81$ 程度である。VIF が 10 であれば $r = 0.97$ にもなる。詳しくは小杉他（2023）を参照。

```
pacman::p_load(car) # なければ入れておこう
vif(result.mra)
```

```
      V2       V3
2.777778 2.777778
```

幸い，今回の値はこれらの基準を下回っていたので許容範囲内である。

11.3.3　変数の投入順序

重回帰分析の場合は複数の説明変数があるが，これを投入するときに全ての変数を同時に投入するか，順番をつけて投入するかといった手法の違いがある。前者を強制投入法と呼ぶこともある。順番をつけて投入する方法は，逐次投入と呼ばれる。この場合は，適合度指標などを参考に変数を追加あるいは削除して，適合度が統計的に有意に向上するかどうかを考えながら進めていく。

重回帰係数の予測値 $\hat{y}$ と，被説明変数 y の相関係数 $R_{y\hat{y}}$ は**重相関係数**と呼ばれ，予測がうまくいっているかどうかを表す適合度の一つである。相関係数なので -1 から $+1$ までの値を取りうるが，-1 は逆に完全に合致していることになるので，この相関係数の符号は大して情報を持たない。そこでこれを二乗した $R^2_{y\hat{y}}$ を考える。これは**決定係数**とも呼ばれ，説明変数の分散のうち予測値の分散が占める割合を表している。[*7]

説明変数の逐次投入は，説明変数を持たないヌルモデルから一つずつ追加していく Forward Selection，全ての変数を投入してから一つずつ減らしていく Backward Selection がある。Forward のほうは追加することによって R^2 が有意に増加するか，Backward のほうは削除しても有意に R^2 が減らないか，を確認しながら進めることになる。この方法は手元のデータに最も適した説明変数のペアを選出できる方法ではあるが，検定を繰り返していることの問題と，手元のデータ以外に一般化する時の根拠の乏しさから，用いられないこともある。

逐次投入法には別の観点からの手法もある。それが階層的回帰分析である。この手法は，重回帰分析における交互作用項の投入を検討する文脈で発展した。重回帰分析では，説明変数同士の相関がない，もしくは小さい方が望ましい。しかし，交互作用とは分散分析における組み合わせの効果を表すものであり，実験デザインによっては交互作用効果が重要な変動であることも少なくない。回帰分析と分散分析は，一般線型モデルという形で統一的に理解されるが，回帰分析でも連続的に変化する組み合わせの効果を考えることができる。交互作用があるということは説明変数間に相関があることを意味するため，回帰分析の大前提に抵触する可能性があり，その投入には慎重を期する必要がある。

こうした文脈から，まずは要因の効果を投入し，次に交互作用項を投入してモデル適合度の有意な改善がみられるかどうかを検証する手順が推奨されている。この逐次投入法を特に階層的回帰分析と呼ぶ。ここでの「階層」とは，手順が重要度順に進められていることを意味し，データの特徴に関するものではないことに注意が必要である[*8]。

[*7] もちろん証明できる。小杉（2018）を参照。

[*8] これに対して，データの階層性 (ex. 学級 $\subset$ 市区町村 $\subset$ 都道府県) を考慮する線型モデルのことを，階層線型モデル Hierarchical Linear Model:HLM という。

11.4　係数の標準誤差と検定

11.4.1　係数の検定

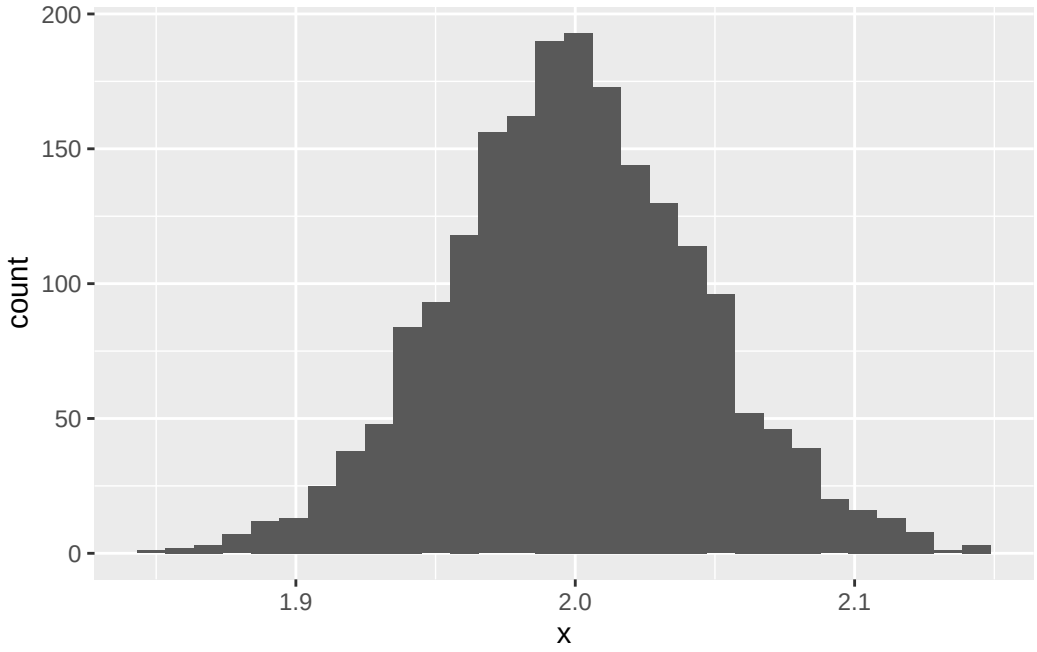
サンプルが母集団から得られた確率変数であるのだから，(偏) 回帰係数もまた確率変数である。すなわち，サンプルが変わるごとに変化し，その揺らぎがある確率分布に従うと考えられる。これを確認するためには，データ生成過程をモデリングし，反復することで近似させて理解するのがいいだろう。

```
set.seed(123)
n <- 500
beta0 <- 2
beta1 <- 3
sigma <- 1
# データ生成関数
dataMake <- function(n, beta0, beta1, sigma) {
  x <- runif(n, -10, 10)
  e <- rnorm(n, 0, sigma)
  y <- beta0 + beta1 * x + e
  dat <- data.frame(x, y)
  return(dat)
}

# 結果オブジェクトの準備
iter <- 2000
beta0.est <- rep(NA, iter)
beta1.est <- rep(NA, iter)
# simulation
for (i in 1:iter) {
  sample <- dataMake(n, beta0, beta1, sigma)
  result.lm <- lm(y ~ x, data = sample)
  beta0.est[i] <- result.lm$coefficients[1]
  beta1.est[i] <- result.lm$coefficients[2]
}

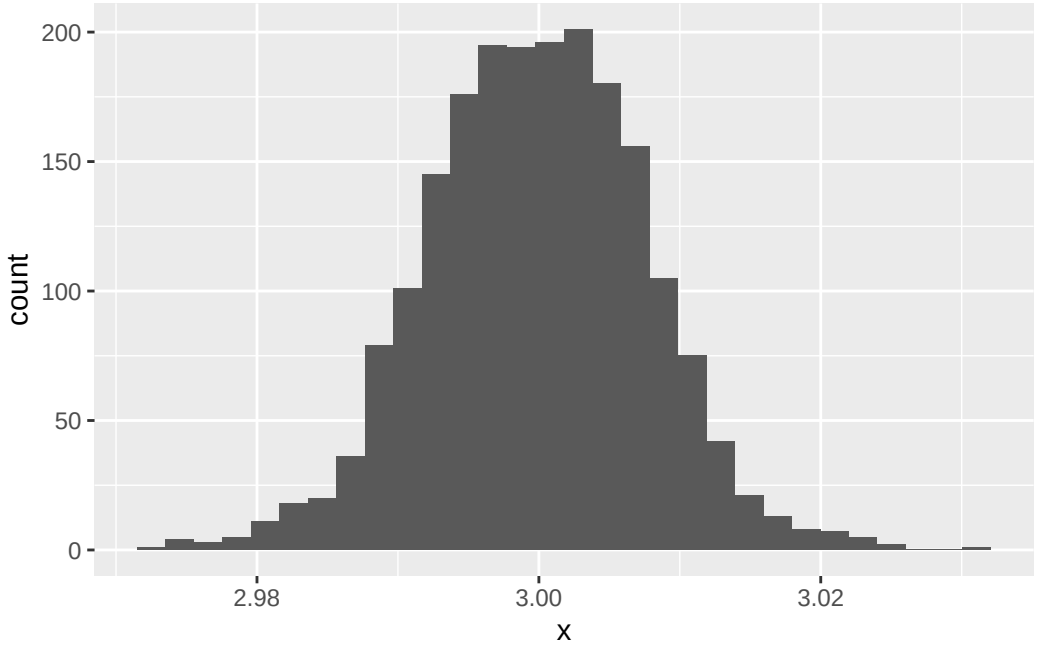
data.frame(x = beta0.est) %>% ggplot(aes(x = x)) +
  geom_histogram()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```




```
data.frame(x = beta1.est) %>% ggplot(aes(x = x)) +
  geom_histogram()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



図から明らかなように，回帰係数も確率的に分布する。ただしその平均は理論値に近似している。

```
mean(beta0.est)
```

```
[1] 1.999257
```

```
mean(beta1.est)
```

```
[1] 2.999798
```

この分布の幅が回帰係数の標準誤差である。

```
sd(beta0.est)
```

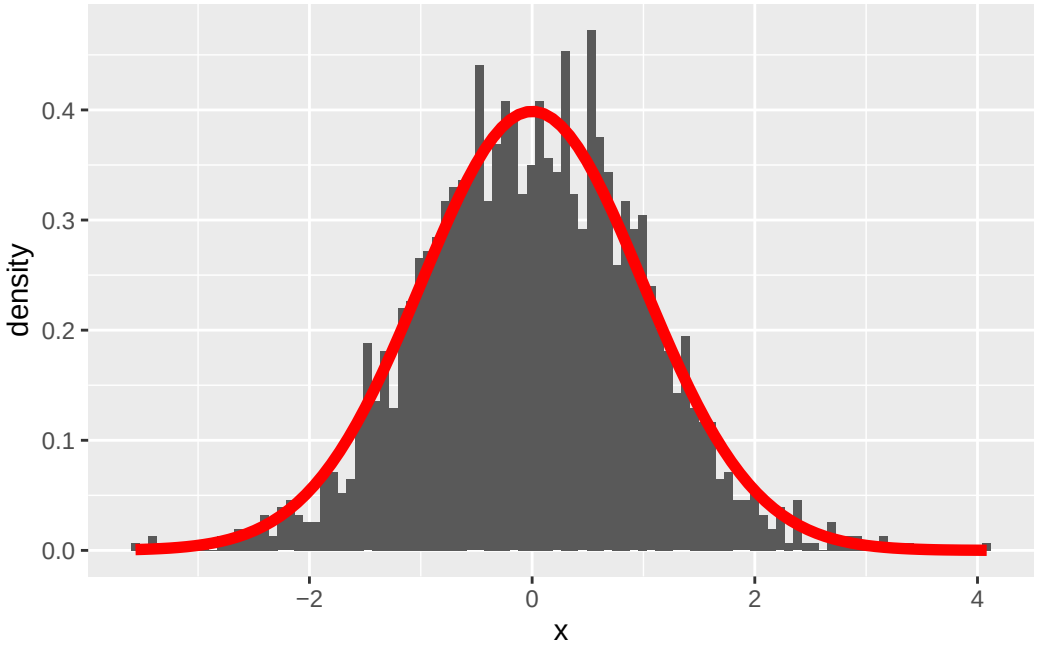
```
[1] 0.04580387
```

```
sd(beta1.est)
```

```
[1] 0.007659277
```

回帰係数は t 分布に従い，その自由度はサンプルサイズからモデルで用いる係数の数を引いたものになる。先ほどのヒストグラムを基準化し，理論分布を重ねて描画してみることで確認しておこう。

```
data.frame(x = beta1.est) %>%
  scale() %>%
  ggplot(aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 100) +
  stat_function(fun = function(x) dt(x, df = n - 2), color = "red", linewidth = 2)
```



この t 分布を用いて，係数が 0 の母集団から得られたサンプルなのかどうかの検定が行われる。

11.4.2　モデル適合度の検定

一方で，出力の最後には F 統計量による検定も行われていたことを確認しておこう。次に示すのは重回帰分析の例である。

```r
set.seed(123)
n <- 500
beta0 <- 2
beta1 <- 0
beta2 <- 0
sigma <- 1
x1 <- runif(n, -10, 10)
x2 <- runif(n, -10, 10)
e <- rnorm(n, 0, sigma)
y <- beta0 + beta1 * x1 + beta2 * x2 + e
sample <- data.frame(y, x1, x2)
result.lm <- lm(y ~ x1 + x2, data = sample)
summary(result.lm)
```

```
Call:
lm(formula = y ~ x1 + x2, data = sample)

Residuals:
     Min       1Q   Median       3Q      Max
-2.85235 -0.68275 -0.01436  0.67809  2.70488

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.996999   0.045263  44.120   <2e-16 ***
x1          -0.006453   0.007970  -0.810    0.418
x2          -0.003928   0.007795  -0.504    0.615
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.012 on 497 degrees of freedom
Multiple R-squared:  0.001893,  Adjusted R-squared:  -0.002124
F-statistic: 0.4713 on 2 and 497 DF,  p-value: 0.6245
```

上の例では，統計量 F が，自由度 $F(\ 2{,}497\ )$ のもとで，0.4713 であり，統計的に有意ではないと判断される (p=0.6245,n.s.)。

これは重相関係数に対する検定であり，母集団においてモデル全体としての説明力が 0 である，という帰無仮説を検証しているものである。この有意性検定には，説明変数の数 p，サンプルサイズ n，重相関係数 R^2 を用いて，以下の式で用いられる検定統計量 F を利用する (南風原，2014)。

$$F = \frac{R^2}{1-R^2} \cdot \frac{n-p-1}{p}$$

ここで右辺第一項目は Cohen の効果量 ($f^2 = \frac{R^2}{1-R^2}$) といわれ，サンプルサイズ設計においてはこの指標が利用される。

11.5　サンプルサイズ設計

重回帰分析のサンプルサイズ設計は，変数の効果の大きさ (回帰係数) が事前にわかっているのであれば，n を徐々に増やしていくシミュレーションによって行える。しかしそのようなケースは稀であり，実際には R^2 の検定を用いて，ある効果量と検出力の下で，正しく検出できるサイズを算出することになる。

サンプルサイズの算出には，非心 F 分布を用いる。この時の非心度は，効果量 f^2 に n をかけたものになる。これを使ってサンプルサイズ設計をする例は以下のとおりである。

```
f2 <- 0.15 # 効果量
alpha <- 0.05 # タイプ1エラー率
beta <- 0.2 # タイプ2エラー率
p <- 5 # 説明変数の数

for (n in 10:500) {
  lambda <- f2 * n
  df1 <- p
  df2 <- n - p - 1
  cv <- qf(p = 1 - alpha, df1, df2)
  t2error <- pf(q = cv, df1, df2, ncp = lambda)
  if (t2error < beta) {
    break
  }
}

print(n)
```

```
[1] 92
```

この設定では，n = 92 以上であればモデルとして影響力がないとは言えない，ということがわかる。

11.6　まとめ

重回帰分析は，人文社会科学で多用される技術ではあるが，技術が先行して理解が伴わないまま利用されているケースも少なくない。繰り返しになる点もあるが，以下に注意点をまとめておく。

- **偏回帰係数の意味**；重回帰分析における偏回帰係数は，ほかの変数を統制した上での値であり，あたかも各係数が独立直交しているかのように解釈するのは適切ではない。
- **誤差の正規性**；誤差は正規分布に従っているという仮定があり，二値データや整数しかとらないカウントデータなどに盲目的にモデルを適用してはならない。誤差の正規性が満たされているかどうかは，分析後に Q-Q プロットを用いて確認する。
- **誤差の均質性**；誤差はモデル全体にわたって同じ正規分布に従っているというのもモデルの仮定である。すなわち，独立変数に応じて誤差分散が変わるといった均質でないデータの場合は，正しく推定されない。誤差の均質性については，分析後の Q-Q プロットを用いて確認する。
- **誤差間の独立性**；誤差はモデル全体にわたって同じ正規分布から独立に生成されている (i.i.d) というのがモデルの仮定である。時系列データのように，誤差間に対応 (自己回帰) がみられるデータの場合は回帰分析は適切な手法とならない。状態空間モデルなど，誤差間関係を適切にモデリングしたものを当てはめる必要がある。
- **モデルの適切な定式化**；モデルには被説明変数に影響を与えるすべての変数が正しく含まれている必要がある。例えば，影響を与えることがわかっている変数 X_o を意図的に除外して分析をしたとする。そのモデルに含まれる変数 X_a が被説明変数 y に影響を与えていたとしても，X_a と X_o に相関があれば，X_o の影響力が X_a を通じて y に伝播する k から，X_a の影響力が課題に評価されることになる。自らの仮説のために，意図的に変数を選択するのは QRPs に該当する。
- **説明変数間の相関関係**；説明変数のうちにあまりにも相関関係が高い変数ペアがあれば，多重共線性の疑いが生じる。多重共線性は推定値の不安定さとなって現れる。このような場合は，説明変数を主成分分析で合成変数にまとめるといった対応が考えられる。また，高い相関ではないが交互作用効果が見たいといった場合は，逐次投入など慎重に個々の影響を考えながら投入するようにする (階層的重回帰分析)。なお交互作用項は，各変数の平均からの偏差をかけ合わせたものにすることが一般的である。

11.7 課題

1. 以下のデータセットは被説明変数 y，説明変数 $x1, x2$ からなる重回帰分析のサンプルデータです。画面には一部しか表示しておらず，全体 ($n = 100$) はこちら ex_regression1.csv からダウンロード可能です。このデータセットを用いて重回帰分析を行い，結果を出力してください。

```
           y         x1          x2
1  1.8685595 -4.248450  1.9997792
2 -0.5728781  5.766103 -3.3435292
3  1.0321850 -1.820462 -0.2277393
4 10.0468488  7.660348  9.0894765
5 -1.1968078  8.809346 -0.3419521
6  9.6719213 -9.088870  7.8070044
```

2. 以下のデータセットは被説明変数 y，説明変数 $x1, x2$ からなる重回帰分析のサンプルデータです。画面には一部しか表示しておらず，全体 ($n = 300$) はこちら ex_regression2.csv からダウンロード可能です。このデータセットを用いて重回帰分析を行ってください。結果のプロットから，上に挙げた重回帰分析の仮定に反しているところを指摘してください。

```
          y         x1           x2
1  3.586304 -0.4248450  0.132767341
2  8.599252  0.5766103  0.922713561
3  2.397115 -0.1820462  0.053684622
4  3.505236  0.7660348  0.007801881
5  6.517720  0.8809346  0.633076091
6 -1.394231 -0.9088870 -0.895346802
```

3. $R^2 = 0.3$ を目標として，説明変数の数 $p = 10$ の重回帰分析を行う際に，必要なサンプルサイズはいくつになるか，計算してみましょう。ここで，$\alpha = 0.05, \beta = 0.2$ とします。

第 12 章

ベイズ統計の活用

ここまで心理統計の基本的な話題を R で実践しながら見てきたが，ここでベイズ統計の話題を導入しよう。わざわざこのように断る理由は，ベイズ統計学が推測統計学の枠組みの一種であるにもかかわらず，その歴史や解釈によって様々な誤解が生じているからである。一説にはベイズ統計学には 100 を超える派閥・解釈のスタイルがあるとも言われており，迂闊に議論をすることが憚られるような言説が交わされることも少なくない。

筆者はファッションベイジアンを自認しており，主義主張を戦わせることはあまり好ましいことではないと考えている。そのような調子の良い立場の人間が語ることである，という事前情報を十分理解した上で，以下の解説を読んでほしい。

12.1　ベイズ統計の位置付け

ベイズ統計学の生まれは古く，ベイズの定理にまで遡れば実に 300 年近く前になる。ベイズの定理を見出したとされるトーマス・ベイズは，18 世紀の牧師であり，1763 年に死んでいる。その古い人物の名を冠した定理が，現在は「ベイズ統計学」と学問全体を覆うほどになっているのは，それがこれまで紹介してきた統計モデルと考え方や解釈の仕方を大幅に変えているからである。

また，ベイズ統計学はその長い歴史に対して，応用的な価値が見出されたのが比較的最近である。筆者の感覚で言えば，2010 年以降になって，ベイズ統計学の応用が急速に広まったように思う。古くから知られていた理論が長らくその名をひそめていたのは，ベイズ統計学を応用できるシーンが限定的であったこと，その研究が軍事的な機密を含むことで公開されることが少なかったこと，そして冗談のように聞こえるかもしれないが，アメリカ統計学会の重鎮がベイズ統計学を嫌っていたこと，がその理由と考えられる。このあたりの事情については (シャロン・バーチュ・マグレイン，2018) を参考にしてほしい。

そして近年になって改めてベイズ統計学が注目されるようになったのには，2 つの理由がある。第一は社会心理学における再現性の危機に対する対応として，従来の統計学ではない新しいアプローチとして期待されたこと。第二は計算機科学の発展により，非常にパワフルな推定方法が開発され，統計モデルを非常に柔軟に扱うことができるようになったこと，である。

再現性の危機に対する対応としてのベイズ統計学は，従来の統計学を「頻度主義的」なものとして，ベイズ統計学を「ベイズ主義的」なものとして区別することで，その違いを強調するきらいがある。しかし，頻度主義的な方法

に問題があるからという単純な理由でベイズ主義的な方法に乗り換えると，そもそもの問題であった統計法の悪用や誤用の種類が変わるだけに過ぎない。

筆者は教育的観点から，ベイズ主義的な統計学の方が初学者には優しく，より理解しやすいのではないかと考えている。しかし，心理学のこの 100 年の歴史の中で積み重ねられてきた「頻度主義的」な研究のお作法は，誤用悪用を招く恐れもあるとは言え，非常にリッチな教育コンテンツ，分析ツールを提供してきた。またこれまでの心理学的文献が「頻度主義的」なものであったことを考えると，これらを捨て去って心機一転，一気にベイズに乗り換えようというのは現実的ではない。ベイズ統計学の弱点としては，教育コンテンツ，分析ツール，そしてベイズ統計学の教育者がそもそも少ないこと，が挙げられる。もちろんこれらの問題点が，近い将来のうちに解決されることを望むものである。

第二の理由はベイズ統計学のポジティブな未来を予感させる。統計モデルが複雑になるにつれ，最尤推定法は実質的な限界を迎えるときでも，ベイズ推定の新しいアプローチは対応できる。これは，ベイズ統計学のパワーを具現化する計算機手法，すなわち MCMC 法の発展によるところが大きい。統計モデルを非常に柔軟に，自らの研究データにカスタマイズした分析モデルを構築でき，その他のモデルともベイズファクターという一元的な指標で評価できることは，ベイズ統計学の大きな魅力である。しかし反面，細かく統計モデルをカスタマイズできるということは，研究者にプログラマとしての技量を要求することになる。心理学者はあくまでも統計をツールとして使いたいユーザなのだから，ソフトウェア的なエンジニアリングにあまりエフォートをかけていられないということもあるだろう。しかし，平均値の差の検定や要因計画にとらわれない，自由なモデリングができる魅力はおおきく，心理学の中でも統計モデリングによるアプローチをとる人も年々増加傾向にある。

12.2　MCMC 法

ベイズ統計を広く応用できるようにした，革新的な技術である MCMC 法についてみていこう。その前に，ベイズの定理を使った考え方について基本的な説明をしておく。

12.2.1　ベイズの定理

ベイズの定理は，条件付き確率の定理である。ある事象 A が起きたときに，事象 B が起きる確率を $P(B|A)$ と表すと，ベイズの定理は以下のように表される。

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

ここで，$P(A)$ は事象 A が起きる確率，$P(B)$ は事象 B が起きる確率，$P(A|B)$ は事象 B が起きたときに事象 A が起きる確率，$P(B|A)$ は事象 A が起きたときに事象 B が起きる確率を表す。ここでの重要な点は，この式の右辺と左辺とで，条件付き確率の位置が変わっていることである。

この式は，事象 A が起きたときに事象 B が起きる確率 $P(B|A)$ を，事象 B が起きたときに事象 A が起きる確率 $P(A|B)$ と，事象 B が起きる確率 $P(B)$ と，事象 A が起きる確率 $P(A)$ を用いて表している。ここで A をデータ，B をモデルのパラメータと考えると，ベイズの定理は以下のように表される。

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

この式の左辺にあるのは，データ D が与えられた時のパラメータ θ が得られる確率であり，データに基づいてパラメータを推論するという推測統計学の基本的な考え方である。これを構成する右辺は，$P(D|\theta)$ はパラメータ θ が与えられた時のデータ D が得られる確率，$P(\theta)$ はパラメータ θ が得られる確率と言える。パラメータ θ がある値になったときに，データ D が得られる確率 $P(D|\theta)$ は尤度と考えることができる。それにかけられる $P(\theta)$ は，パラメータ θ の事前確率と呼ばれるが，これがあることによってベイズ統計学の特徴がより鮮明になる。

データを取る前にパラメータがどのようにあるのか，この不確実さをベイズ統計学では事前確率，あるいは一般に事前分布として表現する。それと尤度を掛け合わせたものが (周辺尤度 $P(D)$ で除されるとは言え) 統計的な推測の結果である事後確率，事後分布 $P(\theta|D)$ となっている。

古典的には，データを取る前にパラメータのあり方を想定するのは，科学的な態度として不適切であるとして，ベイズ統計学は批判されてきた。この点を認めたとして，改めて式を見てみると，確率分布とデータの関係を表す尤度に，事前分布と周辺尤度の比をかけた結果，事後分布が得られていることになる。尤度を計算してその最も高い値を持つパラメータを推定値としよう，というのが最尤推定法だったわけだが，ベイズ推定はその拡張で，尤度から得られる結果を分布として考えるために事前分布と周辺尤度の比を掛け合わせた，とも言える。尤度関数は確率関数ではないので，そのピークになるところを推定値として用いることしかできなかったが，ベイズの定理を経由すると結果的に得られるのが確率分布になるのだから，パラメータがどのあたりにありそうか，その分布として定量的に評価することも可能である。

12.2.2 ベイズの定理の実践史

ベイズ統計学では，未知なるものを確率で表現する。心理統計では平均値パラメータや，複数の平均値パラメータ間の差をすることが目的であり，データを取っただけでは母数が未知なものだったわけだから，これを確率分布として表現することがベイズ統計の最初のステップである。

ベイズ統計は，「未知なるパラメータを確率で表現する（事前分布)」「データによって事前分布を事後分布にアップデートする」というたった 2 つのステップを繰り返すだけである。事前分布も，データの得られた状況に応じて適切に選択することができる。例えば男性と女性の平均身長の差を知りたい，という状況を考えてみよう。男性の身長の平均値，女性の身長の平均値が未知なるパラメータであるが，長さであること，人間であることを考えると，どれほど低く見積もっても 100cm，どれほど高く見積もっても 300cm という幅で考えていれば十分であろう。ならば，この区間のどこかにピークが来るような広い正規分布 (例えば平均 100，標準偏差 10) を事前分布として用いれば十分である。

尤度，すなわちデータが得られるメカニズムとして，確率分布としての正規分布を考えたとしよう。このとき，ベイズの定理の分子に乗るのはいずれも正規分布であるから，正規分布と正規分布の組み合わせから得られる事後分布の形は正規分布になる。

このように，事後分布の形が明確であれば，ベイズ統計学は従来の統計学と同じように，パラメータの推定値を求めることができる。しかし問題は，このように事後分布の形が明確でないことが少なくない，ということである。

複雑なモデルになればなるほど，複数の確率分布がいろいろ組み合わさった形になり，結果的に事後分布がどのような確率分布になるのか，その形がわからないということになる。

これまでのベイズ統計学は，事後分布の形がわかるような，あるいは計算しやすい形になるような (分子の) 確率分布の組み合わせを見出す，というのが中心的な問題であった。非常に限定的なものに感じられるかもしれない。まさにその理由で，ベイズ統計学は絵に描いた餅だったのである。

計算機科学が発達するにつれて，複雑怪奇な事後分布の形であっても，そのピークを探索する方法が考えられた。これは確率分布のパラメータを少しずつ変えていくことで，事後分布の確率密度がより高い位置に動くように変化させていく，つまりあらゆる組み合わせを考えていくグリッドサーチの方法である。グリッドサーチは計算量が膨大になるが，頑張ればなんとかなる。その意味で，ベイズ統計学が少しは実用的になってきた。

しかしもちろん，まだまだ普段使いできるほどのレベルではない。そのレベルにまで達したのが MCMC という技術である。

12.2.3　Markov Chain Monte Carlo 法

MCMC 法は，マルコフ連鎖モンテカルロ法の略で，マルコフ連鎖とモンテカルロ法という 2 つの技術を合わせたものである。前者は確率過程のモデル，後者はシミュレーションによる確率分布のサンプリングを行う方法である。一言で言ってしまえば，マルコフ連鎖によってどんな形の確率分布でも機械の中に作り出すことができるようになり，モンテカルロ法によってその確率分布から乱数を取り出すことができるようになったのである。

確率分布と乱数の関係はこれまでに見てきた通りで，乱数ひとつひとつは確率分布の実現値であり，ありえる状態の一つでしかないが，これが大量になってくると全体的な傾向，すなわち確率分布の形状を目に見える形にしてくれるのである。数式では表現できない，複雑な事後分布の形であっても，そこから乱数を生成することはできる。そしてその乱数が大量に集まり、ヒストグラムをかいて稜線を眺めてみれば，それが事後分布の形である，と考えることができる。

この方法の第 1 の利点は，確率の計算を集計の問題に置き換えるところである。計算機能力の発達によって，数千，数万程度のデータの集計は一瞬でできるようになった昨今，パラメータについて数千の代表値を得て，その平均値を計算するのは容易いことである。確率分布による平均値，すなわち期待値の近似値として，この平均値を用いることができる。その近似値の精度は MCMC のサンプルサイズに依存するが，サンプルを一桁上げるとその精度も一桁上がるわけだから，乱数を大量に作ることでその精度は一気に向上させられる。

この方法の第 2 の利点は，積分計算が容易になることである。複数のパラメータを持つ確率分布は，多次元空間における確率分布である。そのなかで特定のパラメータに注目する場合，それパラメータ以外のパラメータは不要なので積分によって周辺化する必要がある。数式でこれを行うと，多重積分になって非常に面倒が生じるが，乱数生成アプローチの場合は当該パラメータについてのみ代表値を計上すれば良いので，非常に簡単である。

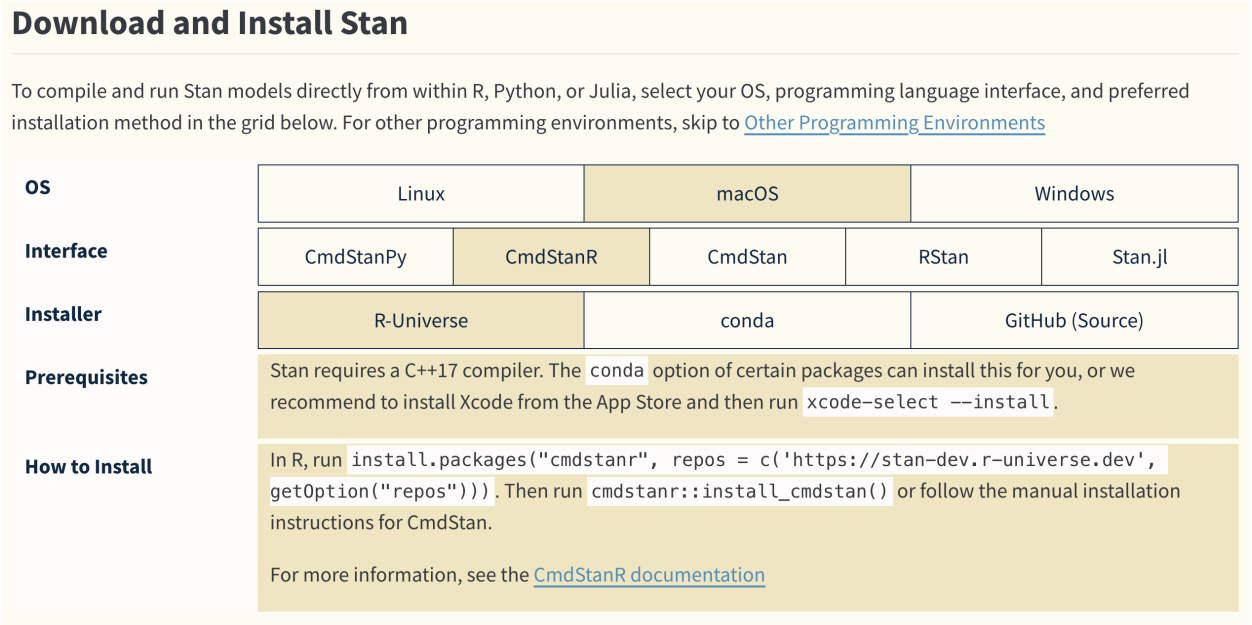
あくまでも近似に過ぎないと言われればそうだが，この方法はかなり強力で，事前分布や尤度など考えるべき確率分布とその組み合わせ方を適切に設定すれば，事後分布の形を計算できなくても事後分布からの乱数は得られるのである。この組み合わせの設定は，確率型プログラミング言語によって実装される。確率型プログラミング言語では，事前分布と尤度 (データがどの確率分布から出てきているか) を記述するだけで，そのまま事後分布からの乱数を生成することができる。ユーザは自分の好きな確率分布を好きな形で組み合わせることができるから，

モデルの表現力が飛躍的に上がったのである。

確率型プログラミング言語として，代表的なものは JAGS と Stan である。現在は Stan が主流である。Stan は R だけでなく，Python など他の言語からも利用可能である。R で Stan を利用するには，パッケージとして `cmdstanr` を用いるのが一般的である。このパッケージを導入するにあたっては，`cmdstanr` のホームページを参照してほしい。

12.3　ツールの導入；Stan と brms

Stan の導入には，環境によって若干の違いがあるので，公式のホームページを参照してほしい。2025 年 3 月現在公式ページから Get Started へと進むと，OS とインターフェイス，インストーラをどこにするかを選択すると，導入に必要なもの (Requirements) と導入方法が表示される。



画面では OS として MacOS を選んでいるが，ここは各自の環境に合わせてもらいたい。インターフェイスは `CmdStanR` を選択して欲しい。Stan を実行するには C コンパイラが必要であり，また `CmdStanR` はコマンドラインから Stan を実行して R に繋げるという代物で，`cmdstanr::install_cmdstan` 関数を実行した後，インストール先のパスを設定する必要がある。

`CmdStanR` は stan 言語で書いた確率モデルを実行し，計算機内部で事後分布を作ってその代表値 (MCMC サンプル) を出力させることができる。自ら確率モデルを書くことができるので自由度が高いが，線型モデルに限定して実行するのであれば，Stan を開発しているのと同じチームが提供する `brms` パッケージが便利である。

`brms` パッケージを用いれば，R の `formula` の指定の仕方で一般化混合線型モデル，階層線型モデルなどが表現できる。これらの非ベイズ推定版である `lmer` パッケージとその書式が同じなので，非常に使いやすい。このパッケージの導入は，一般的な `CRAN` からのインストールでも可能である。詳しくは brms のサイトを参照してほしい。

```
install.packages("brms")
```

これらの環境の準備ができたものとして，使い方を見ていこう。

12.4　ベイズ法による推定の例

12.4.1　パラメータリカバリによる確認と結果の読み取り

前回に倣って，回帰分析のモデル式にそってデータを生成し，分析によってパラメータリカバリを行ってみよう。

説明変数については制約がないので一様乱数から生成し，平均 0，標準偏差 σ の誤差とともに被説明変数を作り，従来のやり方で推定してみよう。

```
pacman::p_load(tidyverse)
set.seed(123)
n <- 500
beta0 <- 2
beta1 <- 3
sigma <- 1
# データの生成
x <- runif(n, -10, 10)
e <- rnorm(n, 0, sigma)
y <- beta0 + beta1 * x + e

dat <- data.frame(x, y)
result.lm <- lm(y ~ x, data = dat)
summary(result.lm)
```

```
Call:
lm(formula = y ~ x, data = dat)

Residuals:
     Min       1Q   Median       3Q      Max
-2.82796 -0.61831  0.03553  0.69367  2.68062

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2.021928   0.045010   44.92   <2e-16 ***
x           3.002194   0.007919  379.09   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.006 on 498 degrees of freedom
Multiple R-squared:  0.9965,    Adjusted R-squared:  0.9965
F-statistic: 1.437e+05 on 1 and 498 DF,  p-value: < 2.2e-16
```

結果は切片 2.0219277，傾き 3.0021943 であるから，設定した $\beta_0 = 2, \beta_1 = 3$ が正しく復元できた，という話であった。

これは最尤法による推定であったが，brms パッケージを使ってベイズ推定に変えてみよう。方法は次のとおりである。

```
pacman::p_load(brms)
# brmsのバックエンドをcmdstanrに統一する（rstan非依存）
options(brms.backend = "cmdstanr")
result.bayes <- brm(y ~ x, data = dat)
```

```
Start sampling

Running MCMC with 4 sequential chains...

Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1 Iteration:  100 / 2000 [  5%]  (Warmup)
Chain 1 Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 1 Iteration:  300 / 2000 [ 15%]  (Warmup)
Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 1 Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 1 Iteration:  700 / 2000 [ 35%]  (Warmup)
Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 1 Iteration:  900 / 2000 [ 45%]  (Warmup)
Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1 Iteration: 1100 / 2000 [ 55%]  (Sampling)
Chain 1 Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 1 Iteration: 1300 / 2000 [ 65%]  (Sampling)
Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 1 Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 1 Iteration: 1700 / 2000 [ 85%]  (Sampling)
Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 1 Iteration: 1900 / 2000 [ 95%]  (Sampling)
Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1 finished in 0.0 seconds.
```

```
Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2 Iteration:  100 / 2000 [  5%]  (Warmup)
Chain 2 Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 2 Iteration:  300 / 2000 [ 15%]  (Warmup)
Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 2 Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 2 Iteration:  700 / 2000 [ 35%]  (Warmup)
Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 2 Iteration:  900 / 2000 [ 45%]  (Warmup)
Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2 Iteration: 1100 / 2000 [ 55%]  (Sampling)
Chain 2 Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 2 Iteration: 1300 / 2000 [ 65%]  (Sampling)
Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 2 Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 2 Iteration: 1700 / 2000 [ 85%]  (Sampling)
Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 2 Iteration: 1900 / 2000 [ 95%]  (Sampling)
Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2 finished in 0.0 seconds.
Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 3 Iteration:  100 / 2000 [  5%]  (Warmup)
Chain 3 Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 3 Iteration:  300 / 2000 [ 15%]  (Warmup)
Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 3 Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 3 Iteration:  700 / 2000 [ 35%]  (Warmup)
Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 3 Iteration:  900 / 2000 [ 45%]  (Warmup)
Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 3 Iteration: 1100 / 2000 [ 55%]  (Sampling)
Chain 3 Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 3 Iteration: 1300 / 2000 [ 65%]  (Sampling)
Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 3 Iteration: 1600 / 2000 [ 80%]  (Sampling)
```

```
Chain 3 Iteration: 1700 / 2000 [ 85%]  (Sampling)
Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 3 Iteration: 1900 / 2000 [ 95%]  (Sampling)
Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 3 finished in 0.0 seconds.
Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 4 Iteration:  100 / 2000 [  5%]  (Warmup)
Chain 4 Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 4 Iteration:  300 / 2000 [ 15%]  (Warmup)
Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
Chain 4 Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 4 Iteration:  700 / 2000 [ 35%]  (Warmup)
Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 4 Iteration:  900 / 2000 [ 45%]  (Warmup)
Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 4 Iteration: 1100 / 2000 [ 55%]  (Sampling)
Chain 4 Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 4 Iteration: 1300 / 2000 [ 65%]  (Sampling)
Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
Chain 4 Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 4 Iteration: 1700 / 2000 [ 85%]  (Sampling)
Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 4 Iteration: 1900 / 2000 [ 95%]  (Sampling)
Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 4 finished in 0.0 seconds.

All 4 chains finished successfully.
Mean chain execution time: 0.0 seconds.
Total execution time: 0.6 seconds.
```

要求されたパッケージ rstan をロード中です

実行に際して，Compiling Stan program... との文字が表示されるが，これは brms パッケージが内部で stan 言語を書き，それを C 言語に書き換えてコンパイルしていることを意味する。他にもいろいろ出力されているが解説は後述する。簡単なモデルなので，すぐにプロンプトが待機状態に戻るはずである。

さて，summary 関数で結果の要約を見てみよう。

```
summary(result.bayes)
```

```
 Family: gaussian
  Links: mu = identity
Formula: y ~ x
   Data: dat (Number of observations: 500)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     2.02      0.04     1.93     2.11 1.00     4187     2968
x             3.00      0.01     2.99     3.02 1.00     4547     2980

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     1.01      0.03     0.95     1.07 1.00     3106     2736

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

まず Regression Coefficients のところを見てみよう。とりあえずリカバリーが上手くいってるか見てみたいからだ。推定値 Estimate のところに，2.0206785 と 3.0021181 とあるから，なるほど正しく推定できているようである。その後に標準誤差 (SE) があるのはいいとして，その次にあるのが l-95%CI と u-95%CI，すなわち lower-upper で表される区間である。

ベイズ統計学ではわからないものを確率分布として表現する。今回わからなかったものは回帰係数だから，切片 β_0 と傾き β_1 はそれぞれ確率として表現され，結果も確率分布 (事後分布) として得られている。つまり，ここに表されているのは事後分布の 95% 区間であり，β_0 は 1.9330752, 2.1095861 の間に分布しているもの，という結果になっている。

最尤推定が点推定であったのに対し，ベイズ推定はこのように分布で表現されるから，表示されている推定値もその代表値である。代表値の取り方はご存知の通り，平均値，最頻値，中央値などが考えられ，そのいずれもが推定値として用いられる。平均値による代表値は Expectation A Posteriori，EAP 推定値と呼ばれる。中央値による代表値は MEDian A Posteriori，そして最頻値というより分布のピークを取る推定値のことを Maximum A Posteriori，MAP 推定値という。ここで得られた事後分布は分布の関数形によるものではなく，事後分布の代表値の集積による稜線で形を見ているに過ぎないから，MAP 推定値を得る方法は 1. ヒストグラムを書いてその最頻値である級数の平均値をとる，2. ヒストグラムにフィットする関数を近似し，そのピークを算出する，といった方法が考えられる。計算そのものはパッケージに含まれる関数を利用すれば良い。ここで大事なことは，分布の形状に応じてその代表値を選ぶことである。すなわち，正規分布のような左右対称の分布であれば，平均値，中央値，最頻値は同じ値になるが，ここが異なるようであれば歪んだ分布をしていることが考えられるから，事後分布

のヒストグラムを描いて確認した上で，中央値や MAP 推定値などを用いるといいだろう。

12.4.2 MCMC を評価する

結果はその他にもいろいろな情報を提供してくれているので，見ていこう。summary 関数によって表示された出力の 5 行目，Draws のところに MCMC サンプリングの情報が表示されている。これによると，4 つのチェインがあり，そのそれぞれが 2000 回反復 (iter) されたこと，そのうちの最初の 1000 回は warmup と呼ばれるステップであったこと，最後に thin というのがあるが，これは MCMC サンプルを間引きするためのもので，ここでは 1 回毎にサンプルを取っている，すなわち毎回のサンプルを用いたことがわかる。

MCMC は事後分布を作り，そこから乱数を取り出すステップであったこをと思い出そう。乱数を取り出すステップは，まず適当な初期値から始め，次に高次元同時確率空間の中である方向に移動する。その場所からまた次の場所を選ぶ，と次々とステップを進めていく形で事後分布の代表値を拾い集めてゆく。今回の例で言うと，β_0, β_1, σ という 3 つのパラメータを推定しているので 3 次元空間を探索する。この空間のある座標は，この 3 つのパラメータが取りうる可能性のある値の組み合わせである。この空間で，初期値 t_0 の座標に立ち，近傍の別の座標 t_1 に移動する。この t_1 の座標も，この 3 つのパラメータが取りうる別の可能性の組み合わせである。同様に t_1 の近傍 t_2,t_2 の近傍 t_3，とステップを踏むことで，それぞれのステップが MCMC サンプルの 1 つとして記録されていく。この記録の集約が事後分布の近似になる。

このような形でサンプリングが進むことを考えると，まず懸念されるのが「初期値によって結果が変わるのではないか」ということである。実際，初期値によっては，サンプリングがうまくいかないことはある。うまく推定できるときは，どんな初期値から発生しても，事後分布が作る空間の密度の濃いところからサンプリングが進むので，同じような値を集めてくることができるだろう。

MCMC がうまくいってるかどうかを評価するために，MCMC では一般的に複数の初期値から始め，別々のステップを踏んでログを取る。このステップのログをチェインと呼ぶ。つまり，それぞれのチェインは異なる初期値から始まる一連の代表値の連なりなのである。今回の結果では，4 つのチェイン，つまり 4 つの初期値から始まったことがわかる。

さて，もう一度サンプリングの進み方を思い返してみよう。初期値から始めてステップしている最初のうちは，最終的に目的としている事後分布の代表値からは大きく外れているかもしれない。ステップを繰り返すことで，より事後分布の密度の濃いところに近づいていくのだから，最初のうちはうまくいってなくても当然である。この最初のうちのステップは代表値として信用できないので，「バーンイン burn in 期間だった」ということで切り捨てることが一般的である。brms が採用している MCMC アルゴリズムの stan は，この初期の探索時に，効率よくサンプリングできるようにステップサイズ (どれぐらい遠くの近傍まで考えるか) 等アルゴリズムを自動で調整する期間を設定している。これを特にウォームアップ期間という。

結果に戻ってみてみると，4 つのチェインで 2000 ステップ (iter) を踏んでいるが，ウォームアップ期間が 1000 あるので，実際にサンプリングが行われたのは 1000 ステップ以降である。なので total post-warmup draws =4000 となっている。

thin は，サンプリングの間引き間隔を指定するものであった。ステップ毎に代表値のログを取っていくとしたが，原理的にはそれぞれのステップ・代表値は事後分布から独立にサンプリングされたものであるはずだ。MCMC の

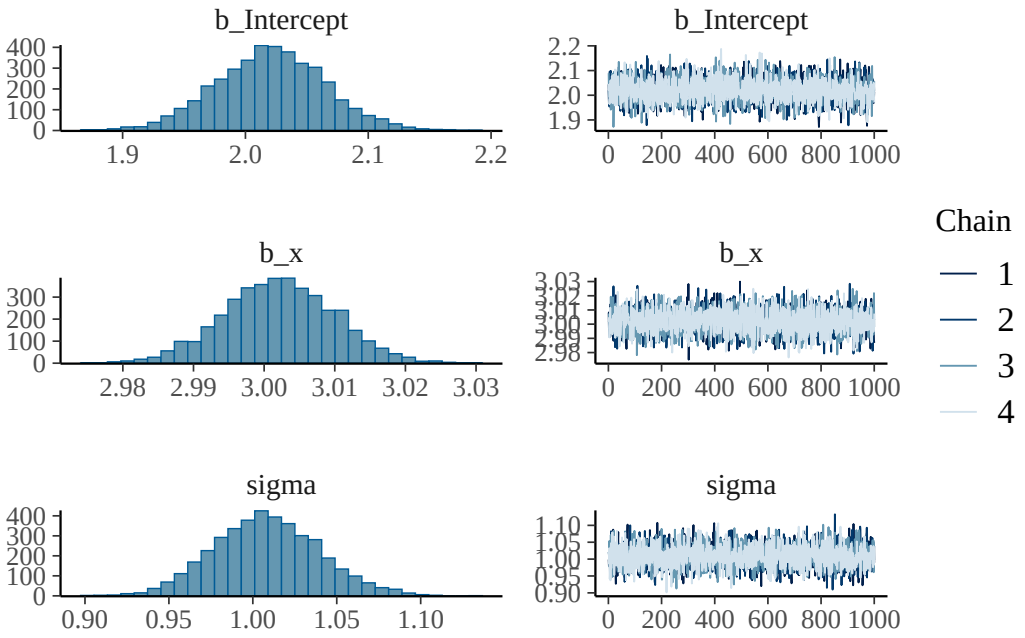
結果を見て，ステップ毎の自己相関を確認し，もし $t-1$ 時点の代表値が t 時点の代表値に影響を与えているようであればよろしくない。そこで間引きをすることで，そのような影響を与えるようなステップを省略することで，より独立性の高いサンプルを得ることができる，と考えるのである。間引きをすると事後分布からのサンプルの数が減ることになる。また，自己相関が高いような MCMC サンプルしか得られないのは，モデルやパラメタライゼーションが不適切である場合が多く，`thin` オプションを使うのは結果オブジェクトのサイズを減らす目的である，と考えた方がいいだろう。

推定値の出力結果に，`Rhat` と `Bulk-ESS` という指標がある。`Rhat` は，チェイン間の自己相関を表す指標で，1 に近いほどよいとされる。基準として，全てのパラメータにおける `Rhat` が 1.1 未満であれば，複数のチェインが絡み合った，良いサンプリングであったと評価される。逆に，`Rhat` が 1.1 以上であれば，初期値によって異なるパラメータ空間を探索していたことになり，事後分布からの適切な代表値ではない可能性があるので，モデルの見直しなどが必要になる。`Bulk-ESS` は，有効サンプルサイズ Effective Sample Size の略であり，実際に独立したサンプルが何個分の情報を持っているかを推定する指標である。これの量的目安は大まかにいって，3 桁以上の数字があれば良いと言われる。逆に 1，2 桁の数字しか示されないのであれば，有効なサンプリングができていないと考えて，モデルの見直しなどが必要になる。

12.4.3　可視化してモデルの評価をみる

結果を可視化してみるとわかりやすい。次のコードは，推定されたパラメータの事後分布のヒストグラムと，トレースプロットと呼ばれるものを描画する。

```
plot(result.bayes)
```



トレースプロットとは，チェイン毎のステップログを描画したものである。チェインがうまく混合しているかどうかを確認するためのものであり，今回は 4 つのチェインが混じり合った状態であるから，事後分布からのサンプリングとしてはうまくいっていると考えられる。

12.4.4　MCMC サンプルを確認する

これら今回の出力はすでに要約されたものになっているが，具体的にどのような MCMC サンプルが得られているか確認してみよう。結果オブジェクトから，brms パッケージに含まれる as_draws_df 関数を用いて，MCMC サンプルをデータフレームとして取り出すことができる。

```
mcmc_samples <- brms::as_draws_df(result.bayes)
mcmc_samples %>%
  as.data.frame() %>%
  head()
```

```
  b_Intercept      b_x     sigma Intercept    lprior      lp__ .chain
1    2.043952 3.008112 0.9987509  1.760208 -7.430641 -719.4657      1
2    1.977235 2.997184 0.9795715  1.694522 -7.430310 -720.1073      1
3    2.009560 3.004681 1.0159795  1.726140 -7.430547 -719.1872      1
4    2.052064 3.008638 1.0013805  1.768271 -7.430683 -719.6048      1
5    2.051816 2.993277 1.0075065  1.769471 -7.430706 -719.9127      1
6    1.983225 3.012142 1.0038503  1.699101 -7.430399 -720.2288      1
  .iteration .draw
1          1     1
2          2     2
3          3     3
4          4     4
5          5     5
6          6     6
```

いま取り出した mcmc_samples は，as_draws_df 関数の出力によって，data.frame 型の特殊系になっているので，改めて as.data.frame 関数を用いてデータフレームに変換し，最初の数行を表示させている。これの後ろの 3 列に，.chain,.iteration,.draw という列があるが，これはそれぞれ MCMC サンプルの chain 番号，サンプルの通し番号，サンプルの通し番号である。

すでに述べたように，デフォルトでは 4 つのチェインからサンプリングを行う。画面に表示されているのは，第 1 チェインの 1，2，…,6 番目のステップ＝サンプルである。つまり，各行が 3 次元同時確立空間の代表値であることを指している。

b_Intercept は切片 β_0 のサンプル，b_x は傾き β_1 のサンプルである。また今回は単回帰分析であるから，$Y \sim N(\beta_0 + \beta_1 x, \sigma)$ というモデルを推定していたわけで，sigma はこの σ のサンプルである。Intercept は $\beta_0 + \beta_1 x$，すなわち正規分布の位置を表している。

lprior は Log Prior の略で，パラメータの事前分布の対数を表している。lp__は Log Posterior の略で，パラメータの事後分布の対数を表している。いずれも，モデルの推定に関係する情報として提供されているが，今ここは気にしなくてもいいだろう。

次のコードは，MCMC サンプルの要約統計量を使って事後分布を記述したものである。MAP 推定値については，R の density 関数を使って，観測データからカーネル密度推定（Kernel Density Estimation, KDE）を計算し，そのピークの度数を取ることで，MAP 推定値を算出している。

```
# MAP推定用の関数を定義
find_map <- function(x) {
  density_obj <- density(x)
  return(density_obj$x[which.max(density_obj$y)])
}

mcmc_samples %>%
  as.data.frame() %>%
  select(b_Intercept, b_x, sigma) %>%
  rowid_to_column("iter") %>%
  pivot_longer(-iter) %>%
  group_by(name) %>%
  summarise(
    EAP = mean(value),
    MAD = median(value),
    MAP = find_map(value),
    SD = sd(value),
    L95 = quantile(value, probs = 0.025),
    U95 = quantile(value, probs = 0.975),
    .groups = "drop"
  )
```

```
# A tibble: 3 x 7
  name          EAP   MAD   MAP      SD   L95   U95
  <chr>       <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl>
1 b_Intercept  2.02  2.02  2.02 0.0446  1.93   2.11
2 b_x          3.00  3.00  3.00 0.00784 2.99   3.02
3 sigma        1.01  1.01  1.00 0.0314  0.949  1.07
```

今回は，ベイズ統計の位置付けと MCMC 法によるベイズ推定の実際を，パッケージを用いて説明した。brms パッケージは線型モデルやその応用において非常に強力であり，ベイズ統計の実践においては，これを用いることが多いだろう。ただし，線型モデルでないモデルについては，自分で stan のコードを書いて推定することも多い。線型モデルの限界に囚われず，自由な統計モデリングの世界があることも視野に入れておいてほしい。

12.4.5　brms パッケージのオプション

ベイズ推定には事前分布が必要である。しかし brm 関数を実行した時に，事前分布は特段指定しなかった。これはパッケージがデフォルトで用意した事前分布を用いたからである。どのような事前分布が用いられたかを確認

するには，以下のようにする。

```
brms::get_prior(result.bayes)
```

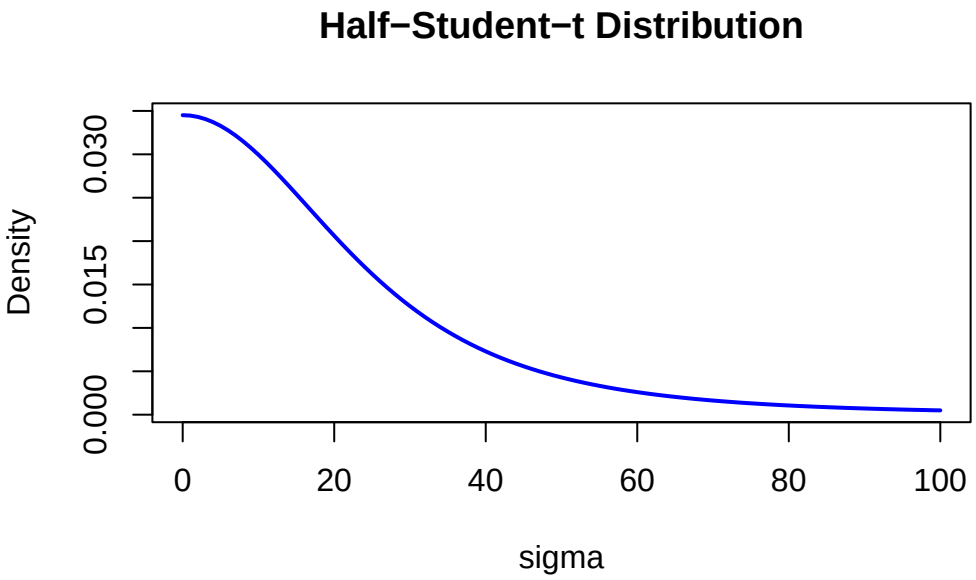
```
                  prior     class coef group resp dpar nlpar lb ub tag
                 (flat)         b
                 (flat)         b    x
 student_t(3, 0.3, 21.3) Intercept
   student_t(3, 0, 21.3)     sigma                              0
       source
      default
 (vectorized)
      default
      default
```

これをみると，回帰係数には flat な事前分布が用いられていることがわかる。これはデータから考えられた取りうる範囲が全て等確率な，一様分布を次全部ぷとしたことを表している。これは無情報時全部分布と呼ばれる。

残差の分散 σ については，`student_t(3, 0, 21.3)`，すなわち t 分布が使われていることがわかる。具体的にこの分布がどのような形状か，描いてみてみよう。

```
df <- 3
mu <- 0
sigma <- 21.3

curve(2 * dt((x - mu) / sigma, df) / sigma,
  from = 0, to = 100,
  col = "blue", lwd = 2, main = "Half-Student-t Distribution",
  xlab = "sigma", ylab = "Density"
)
```



分散は正の値しか取らないので，brms は student_t(3, 0, 21.3) を半分に折りたたみ、半 t 分布（half-Student-t）に変換して利用している。t 分布は正規分布より裾の重い分布であり，大きな値が出る可能性も考慮されている。分散の事前分布には他にも，cauchy 分布や exponential 分布などが用いられることもある。

12.4.6　MCMC サンプリングの設定

brms パッケージでは，MCMC サンプリングの設定もできる。例えばチェインの数を増やすとか，warmup 期間を変えるとか，間引きを変えるとか，いろいろな設定が可能である。また，再現性を確保するために乱数のシードを固定することもできる。これらの設定をしたコードの例を示す。必要に応じて，いろいろな設定を試してみるといいだろう。特に事前分布については，いろいろ変えても結果が大きく変わらない方が，推定値の信頼性も高まると考えられる。事前分布の変更によって，事後分布がどの程度影響されるかを分析することを感度分析というが，このためにも事前分布をデフォルトに任せず設定したり，デフォルトを利用したとしてもどの設定にしているのかを確認できるようになっておこう。

```
# 事前分布の設定
priors <- c(
  set_prior("uniform(0, 100)", class = "Intercept"), # 切片: 一様分布(0, 100)
  set_prior("normal(0, 10)", class = "b"), # 回帰係数: N(0, 10)
  set_prior("cauchy(0, 5)", class = "sigma") # 標準偏差: Cauchy(0, 5)
)

fit <- brm(y ~ x,
  data = dat,
  prior = priors,
  iter = 3000,
  warmup = 2000,
  chains = 3,
```

```
  seed = 12345,
)
```

12.5 課題

以下のデータセットは被説明変数 y，説明変数 $x1, x2$ からなる重回帰分析のサンプルデータです。データセット全体 ($n = 100$) は ex_regression3.csv からダウンロード可能です。このデータセットを用いて，`brms` パッケージによる重回帰分析を実行してください。

分析結果について，以下の項目を順に報告してください：

1. 回帰係数（切片，$x1$，$x2$）の推定値と 95% 信用区間
2. MCMC の収束診断（`Rhat` と `Bulk-ESS` の値に基づいて判断）
3. 各パラメータの事後分布のヒストグラムとトレースプロット
4. デフォルトで使用された事前分布の確認と説明
5. MCMC サンプルからの MAP 推定値の算出（本章で示した `find_map` 関数を利用）
6. MCMC サンプルに基づく 90% 信用区間と 75% 信用区間の算出

なお，レポートには使用した R コードと，各項目の結果に対する簡潔な解釈を含めてください。

第 13 章

線型代数の基礎

多変量解析の章 (14 章，15 章) では，因子分析，主成分分析，多次元尺度構成法，クラスター分析などさまざまな手法を紹介した。これらの手法に共通する数学的基盤が**線型代数**——ベクトルや行列の演算体系——である。統計パッケージが瞬時に結果を出してくれる時代ではあるが，その仕組みを知らなければ結果の意味を正しく理解できないし，出力の妥当性を判断することもできない。

行列を使う最大の利点は，**多くの数字のセットを簡潔な記号で一般的に表現できること**にある。心理学のデータは「n 人の回答者 $\times$ m 個の変数」という長方形の構造をしているが，これはまさに行列そのものである。行列の言葉で記述すると，平均や分散の計算も，回帰分析の推定も，因子分析の原理も，統一的な枠組みで表現できる。

この章では，行列の基本的な計算規則と R での操作方法を学ぶ。

```
pacman::p_load(tidyverse)
```

13.1 スカラー・ベクトル・行列

13.1.1 スカラー

行列でもベクトルでもない，ただの 1 つの数値を**スカラー** (scalar) と呼ぶ。普段の計算で使っている $1, 2, 3.14$ などはすべてスカラーである。

13.1.2 ベクトル

複数の数値を一列にまとめたものを**ベクトル**と呼ぶ。横に並べたものを**行ベクトル** (row vector)，縦に並べたものを**列ベクトル** (column vector) という。

$$\text{行ベクトル: } a = \begin{pmatrix} 1 & 3 & 5 \end{pmatrix}, \quad \text{列ベクトル: } b = \begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix}$$

a はサイズ 1×3 の行ベクトル，b はサイズ 3×1 の列ベクトルである。行列やベクトルは慣例として太字 (A, x) で書く。

R のベクトルには行・列の区別がないが，`matrix()` で明示できる。

```
# Rのベクトル（行・列の区別なし）
a <- c(1, 3, 5)
a
```

```
[1] 1 3 5
```

```
# 行ベクトル（1×3の行列として）
matrix(a, nrow = 1)
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
```

```
# 列ベクトル（3×1の行列として）
matrix(a, ncol = 1)
```

```
     [,1]
[1,]    1
[2,]    3
[3,]    5
```

13.1.3　行列

行列 (matrix) とは，数を長方形に並べたものである。横の並びを**行** (row)，縦の並びを**列** (column) と呼ぶ。

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix}$$

これは n 行 m 列の行列で，サイズを $n \times m$ と表す。成分 a_{ij} の添え字は，最初が行番号，次が列番号である。心理学のデータ行列では，行が回答者 (ケース)，列が変数 (項目) に対応するのが一般的である。

R では `matrix()` 関数で行列を作る。デフォルトでは**列優先** (column-major order) で要素が埋められる。

```
# 列優先（デフォルト）：1列目→2列目の順に埋まる
A <- matrix(1:6, nrow = 2)
A
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
# 行優先：byrow = TRUE で行方向に埋める
B <- matrix(1:6, nrow = 2, byrow = TRUE)
```

```
B
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

要素へのアクセスは **[行，列]** で行う。

```
A[2, 3]    # 2行3列目の要素
```

```
[1] 6
```

```
A[1, ]    # 1行目全体（ベクトル）
```

```
[1] 1 3 5
```

```
A[, 2]    # 2列目全体（ベクトル）
```

```
[1] 3 4
```

13.2　特殊な行列

データ分析でよく登場する特殊な行列を紹介する。

正方行列: 行数と列数が等しい行列 $(n \times n)$。

対称行列: $a_{ij} = a_{ji}$ を満たす正方行列。左上から右下の対角線を軸に対称で，**相関行列**や**分散共分散行列**はその代表例である。

3 変数 x_1, x_2, x_3 の相関行列は次のような形をしている。

$$R = \begin{pmatrix} 1 & r_{12} & r_{13} \\ r_{12} & 1 & r_{23} \\ r_{13} & r_{23} & 1 \end{pmatrix}$$

対角成分は 1(自分自身との相関)，非対角成分は対称 $(r_{ij} = r_{ji})$ である。分散共分散行列は対角成分が各変数の分散，非対角成分が共分散の対称行列となる。

$$S = \begin{pmatrix} s_1^2 & s_{12} & s_{13} \\ s_{12} & s_2^2 & s_{23} \\ s_{13} & s_{23} & s_3^2 \end{pmatrix}$$

対角行列: 対角成分以外がすべて 0 の正方行列。**単位行列** I は対角成分がすべて 1 の対角行列で，「どんな行列にかけても変わらない」という，スカラーにおける 1 のような存在である。

```
# 3×3の単位行列
diag(3)
```

```
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
```

```
# ベクトルから対角行列を作成
diag(c(2, 5, 3))
```

```
     [,1] [,2] [,3]
[1,]    2    0    0
[2,]    0    5    0
[3,]    0    0    3
```

```
# 行列から対角成分を取り出す
M <- matrix(1:9, nrow = 3)
diag(M)
```

```
[1] 1 5 9
```

diag() はスカラーを渡すと単位行列，ベクトルを渡すと対角行列を作り，行列を渡すと対角成分を取り出す。引数の型によって動作が変わる多機能な関数である。

13.3　行列の演算

13.3.1　加法・減法

行列の足し算・引き算は，対応する位置にある成分どうしを加える (引く) だけである。**サイズが同じ行列でないと計算できない**。

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

```
A <- matrix(c(1, 3, 2, 4), nrow = 2)
B <- matrix(c(5, 7, 6, 8), nrow = 2)
A + B
```

```
     [,1] [,2]
[1,]    6    8
[2,]   10   12
```

```
A - B
```

```
     [,1] [,2]
[1,]   -4   -4
[2,]   -4   -4
```

13.3.2　スカラー倍

行列にスカラーをかけると，各成分がスカラー倍される。

```
2 * A
```

```
     [,1] [,2]
[1,]    2    4
[2,]    6    8
```

13.3.3　行列の積

行列の掛け算は普通の掛け算とは異なる。**掛けて足す**という操作が入る。まず行ベクトルと列ベクトルの積 (内積) から見てみよう。

$$\begin{pmatrix} 1 & 2 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \\ 2 \end{pmatrix} = 1 \times 3 + 2 \times 4 + 1 \times 2 = 13$$

掛け算なのに足し算が入る。これが行列計算の特徴である。

行列 $A(n \times m)$ と行列 $B(m \times l)$ の積は，$n \times l$ の行列になる。**前の行列の列数と後ろの行列の行数が一致していないと計算できない**。結果の (i, j) 要素は，A の i 行と B の j 列の内積である。

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \times 2 + 2 \times 1 \\ 3 \times 2 + 4 \times 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 10 \end{pmatrix}$$

スカラーの計算と違い，**かける順番を入れ替えると結果が変わる** $(AB \neq BA)$。R では %*% が行列積，*は要素ごとの積である。この区別は非常に重要である。

```
A <- matrix(c(1, 3, 2, 4), nrow = 2)
b <- c(2, 1)

# 行列積（%*%）
A %*% b
```

```
     [,1]
[1,]    4
[2,]   10
```

```
# 要素ごとの積（*）…行列積とはまったく異なる
A * b
```

```
     [,1] [,2]
```

```
[1,]    2    4
[2,]    3    4
```

*を使うと各行に b の要素がリサイクルされてかかるだけで，行列積とは意味が異なる。

13.3.4　転置

行列の行と列を入れ替える操作を**転置** (transpose) と呼び，A' や A^T と書く。

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad \Rightarrow \quad A' = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

2×3 の行列は転置すると 3×2 になる。対称行列は転置しても変わらない ($A' = A$)。

```
A <- matrix(1:6, nrow = 2)
A
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
t(A)
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
```

転置にはいくつか重要な性質がある。特に積の転置 $(AB)' = B'A'$ は，順番が逆になることに注意してほしい。

13.3.5　逆行列

逆行列は，行列における「割り算」のような存在である。ある正方行列 A に対して，$AA^{-1} = I$（単位行列）となる行列 A^{-1} を逆行列と呼ぶ。スカラーでいえば「逆数をかけると 1 になる」のと同じ考え方である。

2×2 行列の場合，逆行列は次の公式で求められる。

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \text{のとき}, \quad A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

分母の $ad - bc$ は**行列式**と呼ばれ，これが 0 のとき逆行列は存在しない。R では `solve()` で計算する。

```
A <- matrix(c(2, 5, 1, 3), nrow = 2)
A
```

```
     [,1] [,2]
[1,]    2    1
[2,]    5    3
```

```
# 逆行列
solve(A)
```

```
     [,1] [,2]
[1,]    3   -1
[2,]   -5    2
```

```
# AA^{-1} = I の確認（丸め誤差が出るので round する）
round(A %*% solve(A), 10)
```

```
     [,1] [,2]
[1,]    1    0
[2,]    0    1
```

逆行列は**連立方程式の解法**に直結する。$Ax = b$ の両辺に左から A^{-1} をかけると $x = A^{-1}b$ が得られる。回帰分析の推定量 $\hat{\beta} = (X'X)^{-1}X'y$ も，この仕組みで成り立っている。

13.3.6　トレース

正方行列の対角成分の総和を**トレース** (trace) と呼び，$\mathrm{tr}(A) = \sum_{i=1}^{n} a_{ii}$ と表す。後述する固有値と密接な関係がある。

```
A <- matrix(c(1, 2, 6, 5), nrow = 2)
# トレース = 対角成分の和
sum(diag(A))
```

```
[1] 6
```

13.4　データの行列表現

心理学のデータを行列の言葉で扱ってみよう。n 人の回答者に m 個の変数を測定したとすると，データは $n \times m$ の行列 X になる。

13.4.1　平均ベクトルと平均偏差行列

1 をすべて 1 のベクトルとすると，各変数の平均ベクトルは次のように書ける。

$$m = \frac{1}{n} X'1$$

各成分から平均を引いた**平均偏差行列**は $V = X - 1m'$ であり，これを使って分散共分散行列は次の行列積で表される。

$$S = \frac{1}{n} V'V$$

さらに各変数を標準偏差で割って標準化した行列 Z を使えば，相関行列は

$$R = \frac{1}{n} Z'Z$$

と表される。記述統計量が行列の積で統一的に記述できるのが，行列を使う利点のひとつである。

13.4.2　R での確認

`iris` データセットの数値 4 変数を使って，行列演算と組み込み関数の結果が一致することを確かめてみよう。

```
# データ行列
X <- as.matrix(iris[, 1:4])
n <- nrow(X)

# 平均ベクトル
one <- rep(1, n)
m <- t(X) %*% one / n
as.vector(m)
```

```
[1] 5.843333 3.057333 3.758000 1.199333
```

```
colMeans(X)
```

```
Sepal.Length  Sepal.Width Petal.Length  Petal.Width
    5.843333     3.057333     3.758000     1.199333
```

```
# 平均偏差行列
V <- X - one %*% t(m)

# 分散共分散行列（nで割る = 標本分散）
S <- t(V) %*% V / n

# 標準偏差の対角行列
Q <- diag(sqrt(diag(S)))

# 標準化スコア行列
Z <- V %*% solve(Q)
```

```
# 相関行列
R_manual <- t(Z) %*% Z / n
round(R_manual, 6)
```

```
          [,1]      [,2]      [,3]      [,4]
[1,]  1.000000 -0.117570  0.871754  0.817941
[2,] -0.117570  1.000000 -0.428440 -0.366126
[3,]  0.871754 -0.428440  1.000000  0.962865
[4,]  0.817941 -0.366126  0.962865  1.000000
```

```
round(cor(X), 6)
```

```
             Sepal.Length Sepal.Width Petal.Length Petal.Width
Sepal.Length     1.000000   -0.117570     0.871754    0.817941
Sepal.Width     -0.117570    1.000000    -0.428440   -0.366126
Petal.Length     0.871754   -0.428440     1.000000    0.962865
Petal.Width      0.817941   -0.366126     0.962865    1.000000
```

13.5 距離行列

ケース (行) 間の距離を正方行列にまとめたものが**距離行列**である。ユークリッド距離を使うのが標準的で，距離行列は対称行列 (行き帰りの距離は同じ)，対角成分は 0(自分自身との距離) という性質を持つ。

距離行列は多次元尺度構成法 (MDS) やクラスター分析の入力として用いられる。R では `dist()` で計算する。

```
# 最初の5ケースだけで確認
iris5 <- iris[1:5, 1:4]
d <- dist(iris5)
d
```

```
          1         2         3         4
2 0.5385165
3 0.5099020 0.3000000
4 0.6480741 0.3316625 0.2449490
5 0.1414214 0.6082763 0.5099020 0.6480741
```

```
# 正方行列として表示
as.matrix(d)
```

```
          1         2        3         4         5
1 0.0000000 0.5385165 0.509902 0.6480741 0.1414214
2 0.5385165 0.0000000 0.300000 0.3316625 0.6082763
3 0.5099020 0.3000000 0.000000 0.2449490 0.5099020
```

```
4 0.6480741 0.3316625 0.244949 0.0000000 0.6480741
5 0.1414214 0.6082763 0.509902 0.6480741 0.0000000
```

13.6　固有値と固有ベクトル

正方行列 A に対して，次の関係を満たすスカラー λ とベクトル x を，それぞれ**固有値** (eigenvalue) と**固有ベクトル** (eigenvector) と呼ぶ。

$$Ax = \lambda x$$

左辺は「行列をかける」操作だが，右辺は「スカラーをかける」だけ。つまり，行列をかけても方向が変わらず，伸び縮みだけするような特別なベクトルがあるということである。λ がその伸び率を表す。

具体例で確認しよう。

$$\begin{pmatrix} 1 & 6 \\ 2 & 5 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 7 \\ 7 \end{pmatrix} = 7 \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

確かに $\lambda = 7$，$x = (1, 1)'$ で成り立っている。

```r
A <- matrix(c(1, 2, 6, 5), nrow = 2)
eig <- eigen(A)
eig$values    # 固有値
```

```
[1]  7 -1
```

```r
eig$vectors   # 固有ベクトル（各列が各固有値に対応）
```

```
           [,1]       [,2]
[1,] -0.7071068 -0.9486833
[2,] -0.7071068  0.3162278
```

13.6.1　固有値の重要な性質

$n \times n$ の正方行列からは n 個の固有値が得られる。固有値にはいくつかの重要な性質がある。

トレースとの関係: 固有値の総和は行列の対角成分の総和 (トレース) に一致する。

$$\sum_{i=1}^{n} \lambda_i = \mathrm{tr}(A) = \sum_{i=1}^{n} a_{ii}$$

```r
sum(eig$values)    # 固有値の総和
```

```
[1] 6
```

```
sum(diag(A))       # トレース
```

```
[1] 6
```

対称行列の場合: 対称行列の固有値はすべて実数であり，異なる固有値に対応する固有ベクトルは互いに直交する。分散共分散行列や相関行列は対称行列なので，この性質が保証される。

13.6.2 スペクトル分解

対称行列 A は，固有値を対角に並べた行列 Λ と固有ベクトルを列に並べた行列 X を使って，次のように分解できる。

$$A = X\Lambda X'$$

これを**スペクトル分解** (spectral decomposition) あるいは固有値分解という。固有値分解は元の行列の情報を「方向 (固有ベクトル)」と「大きさ (固有値)」に分離する操作であり，主成分分析や因子分析の数学的な基盤である。

13.6.3 固有値分解と主成分分析

相関行列を固有値分解するとどうなるか。相関行列の対角成分はすべて 1 なので，トレースは変数の数 m に一致する。固有値分解はこの情報量を重要度順に再分配する操作である。

```
# irisの相関行列を固有値分解
R <- cor(iris[, 1:4])
eig_R <- eigen(R)

# 固有値
eig_R$values
```

```
[1] 2.91849782 0.91403047 0.14675688 0.02071484
```

```
# 寄与率（各固有値が全体に占める割合）
eig_R$values / sum(eig_R$values)
```

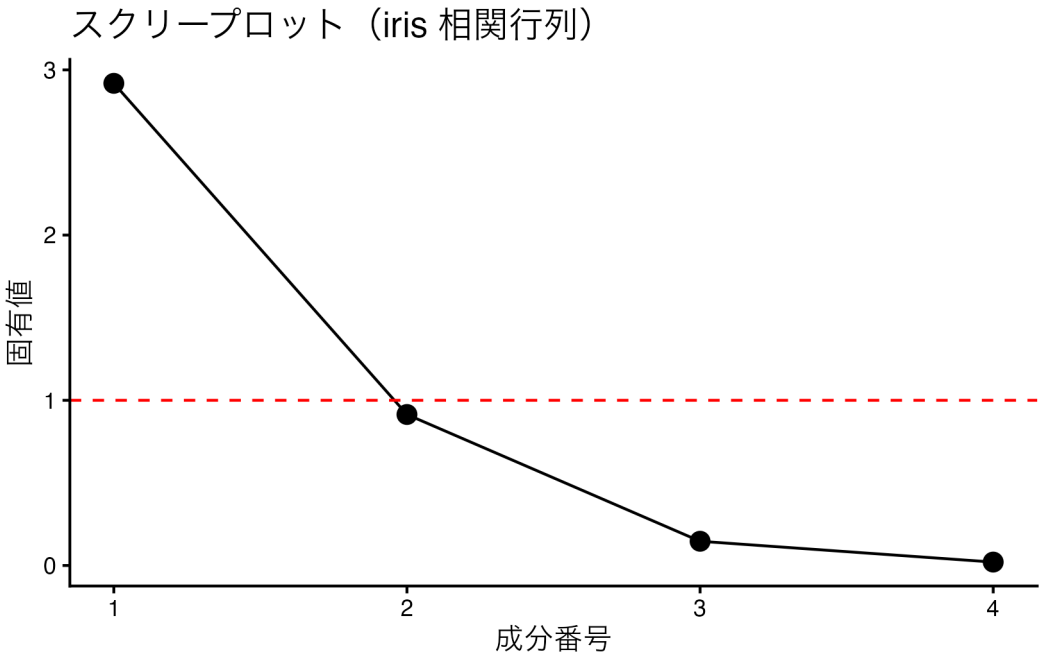
```
[1] 0.729624454 0.228507618 0.036689219 0.005178709
```

```
# 累積寄与率
cumsum(eig_R$values) / sum(eig_R$values)
```

```
[1] 0.7296245 0.9581321 0.9948213 1.0000000
```

第 1 固有値が全体の約 73% を説明している。つまり 4 変数の情報の大部分が 1 つの主成分 (次元) でまとめられるということである。これが主成分分析 (PCA) の原理であり，`prcomp()` の内部で行われている計算そのものである。

```r
# 固有値の大きさを可視化（スクリープロット）
data.frame(
  Component = 1:4,
  Eigenvalue = eig_R$values
) %>%
  ggplot(aes(x = Component, y = Eigenvalue)) +
  geom_point(size = 3) +
  geom_line() +
  geom_hline(yintercept = 1, linetype = "dashed", color = "red") +
  scale_x_continuous(breaks = 1:4) +
  labs(
    title = "スクリープロット（iris 相関行列）",
    x = "成分番号",
    y = "固有値"
  ) +
  theme_classic()
```



赤い破線は固有値=1 のラインで，これを下回る成分は変数 1 つ分の情報量にも満たないことを意味する。固有値の大きさから成分数を決めるのがスクリープロットの読み方である。

因子分析では，相関行列 R の構造を $R \approx AA' + D^2$（A は因子負荷行列，D^2 は独自分散）というモデルで近似する。この推定にも固有値分解が基礎となっている。

13.7　用語のまとめ

用語	意味	R の関数/演算子
スカラー	ただの数値	—
ベクトル	数値の 1 次元の並び	`c()`
行列	数値の 2 次元の並び	`matrix()`
転置	行と列を入れ替える	`t()`
行列積	掛けて足す乗法	`%*%`
要素ごとの積	対応する位置の掛け算	`*`
逆行列	かけると単位行列になる	`solve()`
対角成分	正方行列の a_{ii}	`diag()`
トレース	対角成分の総和	`sum(diag())`
固有値分解	$Ax = \lambda x$ を求める	`eigen()`
相関行列	変数間の相関係数の行列	`cor()`
分散共分散行列	分散と共分散の行列	`cov()`
距離行列	ケース間の距離の行列	`dist()`

13.8 課題

1. `iris` データセットの数値 4 変数について，相関行列と分散共分散行列を計算し，それぞれの対角成分に何が入っているか確認しよう。また，数値 4 変数を標準化 (`scale()`) したデータの分散共分散行列が相関行列と一致することを確かめよう。

2. 次の 2 つの行列 A，B について，AB と BA を計算し，結果が異なることを確認しよう。

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

   さらに，積の転置 $(AB)' = B'A'$ が成り立つことも確認しよう。

3. `iris` の数値 4 変数のうち最初の 5 ケースについて，中心化 (各列から平均を引く) した行列 V を `scale(center = TRUE, scale = FALSE)` で作成し，$\frac{1}{n}V'V$ が `cov()` の結果と近い値になることを確認しよう。ヒント: `cov()` は n ではなく $n-1$ で割る不偏分散を返す。

4. `iris` の相関行列を固有値分解し，各成分の寄与率を求めよう。次に `prcomp(iris[, 1:4], scale. = TRUE)` を実行し，その `sdev` の二乗 (各主成分の分散) が固有値と一致することを確かめよう。

5. 距離行列と相関行列のどちらも対称行列だが，対角成分の意味が異なる。`iris` の数値 4 変数の最初の 10 ケースについて距離行列を求め，対角成分がすべて 0 であること，相関行列の対角成分がすべて 1 であることを確認しよう。その違いがなぜ生じるか考察してみよう。

第 14 章

線型モデルの展開

この章では，線型モデルの展開を説明する。線型モデルは，説明変数と目的変数の関係を線型すなわち一次式で表現するモデルである。線型モデルは，一般化線型モデル (GLM)，一般化線型混合モデル (GLMM)，階層線型モデル (HLM) と拡張していくが，まずは一般線型モデル (LM) についてみておこう。

14.1 一般線型モデル (General Linear Model)

線型モデルの基本は回帰分析モデルである。単回帰モデルは次の式で表される。

$$y_i = \beta_0 + \beta_1 x + e_i$$

ここで，y_i は i 番目の観測値，β_0 は切片，β_1 は回帰係数，e_i は誤差項である。この式を拡張したのが重回帰分析で，次の式で表される。

$$y_i = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + e_i$$

ここで，$x_1, x_2, \cdots, x_p$ は説明変数，$\beta_1, \beta_2, \cdots, \beta_p$ は回帰係数である。

この式をベクトルと行列で表現すると，次のようになる。

$$y = X\beta + e$$

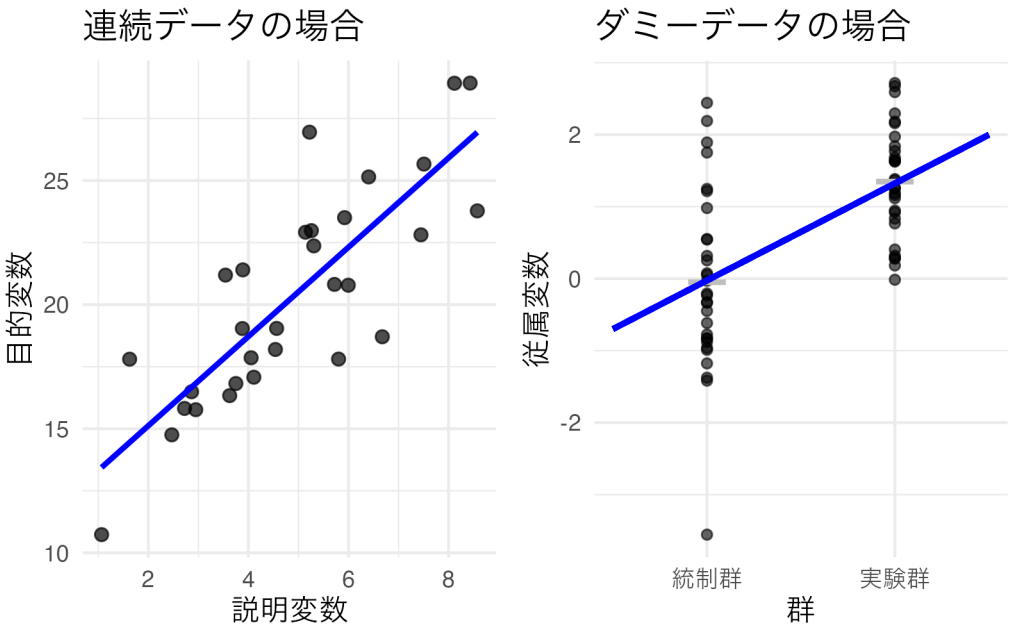
ここで，y は n 次元のベクトル，X は $n \times p$ の**計画行列** (design matrix)，β は p 次元のベクトル，e は n 次元のベクトルである。

計画行列は，説明変数のデータをまとめた行列であり，次のようになる。

$$X = \begin{pmatrix} 1 & x_1 & x_2 & \cdots & x_p \\ 1 & x_1 & x_2 & \cdots & x_p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1 & x_2 & \cdots & x_p \end{pmatrix}$$

第一列目にあるのは切片にかかる係数であり，第二列目以降にあるのは説明変数にかかる係数である。ここに係数ベクトル $\beta = (\beta_0, \beta_1, \beta_2, \cdots, \beta_p)^T$ をかけることで，説明変数の線型結合が得られる。

この係数ベクトルにおいて，x の値がバイナリ (0/1) の場合，その説明変数はダミー変数と呼ばれる。線型回帰モデルにおいて説明変数 X がダミー変数である場合は，横軸が 2 つの値しかとらないため散布図を描くと奇妙な形になることがわかるだろう。



この散布図に対して回帰直線を引けば，2 群の平均値を通る直線が引かれる。回帰分析においては，予測式 $\hat{y}$ を中心とした正規分布に従って誤差が生じると仮定されているのであった。ダミーデータの場合も，両群の平均値を中心に正規分布に従って誤差が生じると仮定したことになる。これはいわゆる 2 群の平均値の差を検定する時の仮定と同じであり，このことから平均値差の検定は説明変数が名義尺度水準の変数である回帰分析と同じ (一般) であることがわかる。回帰分析と平均値差の検定は数学的には同じ式で表現できるから，これを総称して**一般線型モデル** General Linear model という。

14.2　一般化線型モデル (GLM)

かつて心理学実践においては，要因計画法と帰無仮説検定の組み合わせによって研究を行うことが主流であった。平均値差の検定に落とし込むように研究をデザインし，帰無仮説検定で YES か NO か決着がつく。帰無仮説検定はデータの特性や生成メカニズムに依存せず，統計パッケージを用いれば誰にでも扱うことができ，p 値という手垢のついてない数値だけで判断できると考えられてきた。

このことに関する問題については置くとして，ともかく「正規分布の平均値の差」に持ち込めさえすれば良い，と

いうことが当時の研究者の共通認識であった。そのため，例えば比率のデータやカウントデータなど，正規分布を想定することができないデータに対しても，対数変換・角変換などを行って分布を正規分布のそれに近づけ，検定を行うということが行われていた。

正規分布は理論的に $\pm\infty$ の範囲を取るのに対し，比率のデータは 0 から 1 の範囲にしかとらない。カウントデータは正の整数しかとらないデータであるから，こうしたデータに対して一般線型モデルをあてがうのは重大な仮定違反である[*1]。

このような背景から，正規分布以外の確率分布を用いた統計モデルが考えられた。これが**一般化線型モデル** (Generalized Linear Model, GLM) である[*2]。

確率分布の多くは，位置パラメータとスケールパラメータを持つ。統計モデルは平均的な挙動について考えるモデルだから，位置パラメータに線型モデルが当てがえるように，数式を変形してやれば良い。以下に例として，ベルヌーイ分布とポアソン分布を GLM で表現したものを考えてみよう。

14.2.1　ベルヌーイ分布に対する線型モデル；ロジスティック回帰分析

ベルヌーイ分布はコイントスの表/裏のような 2 値をとるデータに対して用いられる。ベルヌーイ分布の確率関数は次のように表される。

$$P(Y = y) = p^y(1-p)^{1-y}$$

ここで，p は成功確率であり，0 から 1 の範囲の値を取る。こうしたデータは現実にも少なくない。生死，病気の有無，テストの正答/誤答のようなデータが代表的である。こうしたデータを目的変数にして，直線回帰を行うと当然おかしなことが生じる。すなわち，予測値が 0 と 1 の範囲を超えることがありえるからである。

そこで，線型予測子と確率を適切に結びつけるリンク関数を用いる。ロジスティック回帰では**ロジット関数**（logit function）をリンク関数として使用する：

$$\mathrm{logit}(p) = \log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x$$

この関係を p について解いてみよう。$\eta = \beta_0 + \beta_1 x$ とおくと：

$$\log\left(\frac{p}{1-p}\right) = \eta$$

両辺の指数を取ると：

[*1] エディタがその言語に対応していたら，おかしな記述に下線が引かれるなど注意を促してくる機能もある。また RStudio で Stan 言語を書いていると，コンパイルの前に文法のチェックをする機能もある。

[*2] とはいえ，コードに原因がない場合もある。それは Stan を導入する際のシステム的なエラーであり，書かれた内容ではなく動かす環境全体の問題である。解決策としては，表示されるエラーを解読して問題を解決するか，環境を再構築する (Stan を再インストールする，最新バージョンに入れ替える等) 必要がある。この場合も，生成 AI が助力してくれるだろう。

$$\frac{p}{1-p} = e^{\eta}$$

両辺に $(1-p)$ をかけると：

$$p = (1-p) \cdot e^{\eta}$$

展開すると：

$$p = e^{\eta} - p \cdot e^{\eta}$$

p について整理すると：

$$p + p \cdot e^{\eta} = e^{\eta}$$

$$p(1 + e^{\eta}) = e^{\eta}$$

$$p = \frac{e^{\eta}}{1 + e^{\eta}}$$

分子分母を e^{η} で割ると：

$$p = \frac{1}{e^{-\eta} + 1} = \frac{1}{1 + e^{-\eta}}$$

$\eta = \beta_0 + \beta_1 x$ を代入すると，**ロジスティック関数**（逆リンク関数）が得られる：

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

このようにして得られた確率を用いて，ベルヌーイ分布の確率関数を次のように表現できる。

$$y \sim \text{Bernoulli}(p)$$

サンプルデータを作って，線型回帰モデルとロジスティック回帰モデルの予測値を比較してみよう。

```
pacman::p_load(tidyverse, patchwork)

# データ生成
set.seed(17)
```

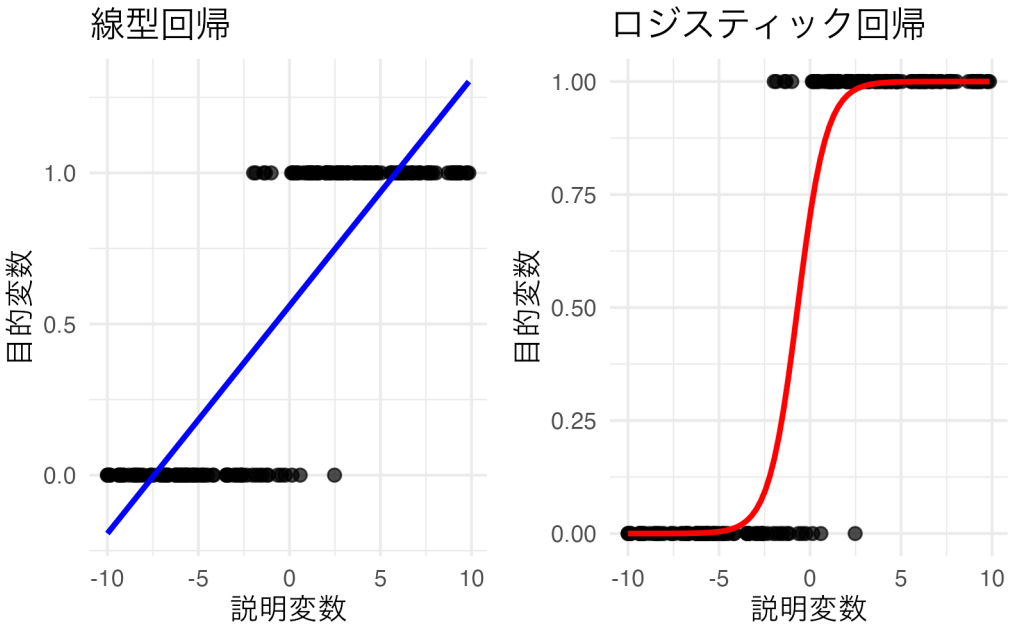
```r
n <- 200
x <- runif(n, min = -10, max = 10)
beta_0 <- 1
beta_1 <- 2
p <- beta_0 + x * beta_1
prob <- 1 / (1 + exp(-p))
y <- rbinom(n, size = 1, prob = prob)

df_logistic <- data.frame(x = x, y = y)

# p1: 線型回帰
p1 <- ggplot(df_logistic, aes(x = x, y = y)) +
  geom_point(alpha = 0.7, size = 2) +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "線型回帰") +
  theme(text = element_text(family = "IPAexGothic"))

# p2: ロジスティック回帰
p2 <- ggplot(df_logistic, aes(x = x, y = y)) +
  geom_point(alpha = 0.7, size = 2) +
  geom_smooth(method = "glm", method.args = list(family = "binomial"), se = FALSE, color = "red") +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "ロジスティック回帰") +
  theme(text = element_text(family = "IPAexGothic"))

# パッチワークで結合
p1 + p2
```



線型回帰の予測値が不適切な範囲にまで延伸するのに対し，ロジスティック回帰はうまく適合していることがわかるだろう。

データからロジスティック回帰モデルを推定するには，glm 関数を用いることができるが，ここではすでに学んだ brms パッケージによるベイズ推定でアプローチしてみよう。brm 関数の書き方は，glm 関数の書き方とほぼ同じであり，推定法を ML からベイズに変えることできる。

```
pacman::p_load(brms)
# brmsのバックエンドをcmdstanrに統一する（rstan非依存）
options(brms.backend = "cmdstanr")
result.bayes.logistic <- brm(
  y ~ x,
  family = bernoulli(),
  data = df_logistic,
  seed = 12345,
  chains = 4, cores = 4, backend = "cmdstanr",
  iter = 2000, warmup = 1000,
  refresh = 0
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.0 seconds.
Chain 2 finished in 0.0 seconds.
Chain 3 finished in 0.0 seconds.
Chain 4 finished in 0.0 seconds.
```

```
All 4 chains finished successfully.
Mean chain execution time: 0.0 seconds.
Total execution time: 0.2 seconds.
```

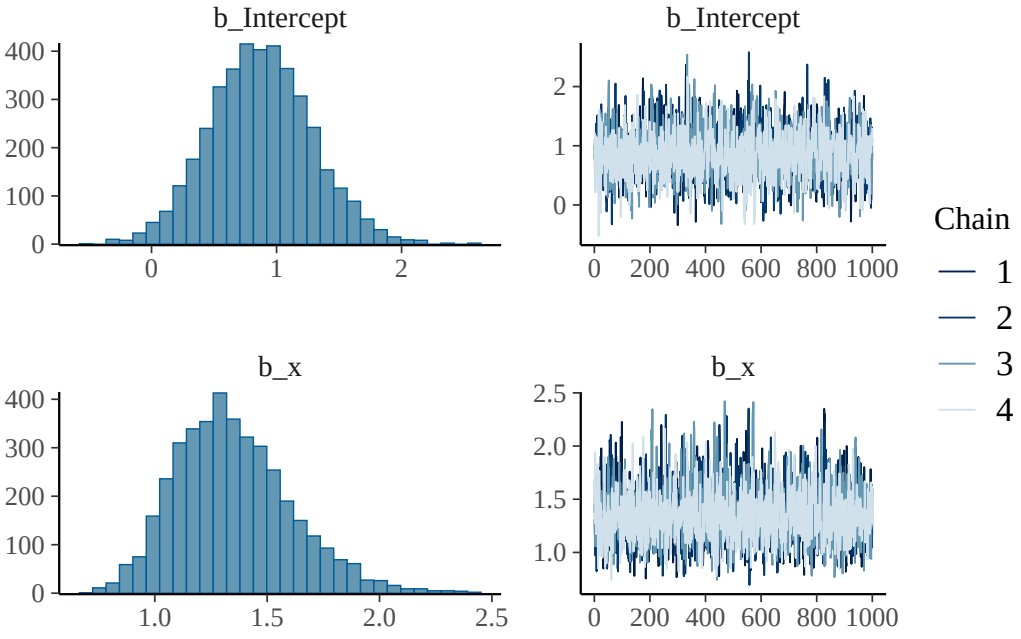
```
summary(result.bayes.logistic)
```

```
 Family: bernoulli
  Links: mu = logit
Formula: y ~ x
   Data: df_logistic (Number of observations: 200)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     0.88      0.41     0.10     1.70 1.00     2314     2331
x             1.35      0.26     0.91     1.92 1.00     2164     2080

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(result.bayes.logistic)
```




```
## 比較のためにML推定も行っておく
result.ml <- glm(y ~ x, family = binomial(), data = df_logistic)
summary(result.ml)
```

```
Call:
glm(formula = y ~ x, family = binomial(), data = df_logistic)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.8673     0.4058   2.137   0.0326 *
x             1.2661     0.2413   5.248 1.54e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 273.869  on 199  degrees of freedom
Residual deviance:  45.112  on 198  degrees of freedom
AIC: 49.112

Number of Fisher Scoring iterations: 8
```

最尤推定の結果とベイズ推定の結果に多少のズレが見られるが，これはサンプルサイズの小ささによるものである。アウトプット変数が 2 値しか持たないため，分散がどうしても小さくなりがちであり，正確な推定値を得るためにはより多くのデータが必要である。

また，回帰係数の解釈には注意が必要である。普通の回帰分析であれば，説明変数が 1 単位変わると目的変数がどれだけ変わるかを表すが，ロジスティック回帰ではこのような直接的な解釈はできない。ロジスティック関数によって変換されたものが意味を持つからである。

線型モデルが表すのは次の関係なのであった。

$$\beta_0 + \beta_1 x = \log \frac{p}{1-p}$$

この $\log \frac{p}{1-p}$ は**ロジット** (logit) と呼ばれる。ロジスティック回帰では，確率 p をロジットに変換することで線型関係を表現している。逆に言えば，線型モデルで表現されているのはログを取った確率の比であり，この比が説明変数の線型関係によって決まるということである。であるから，結果を解釈するには係数を指数関数 e を取り，確率の比として理解する必要がある。

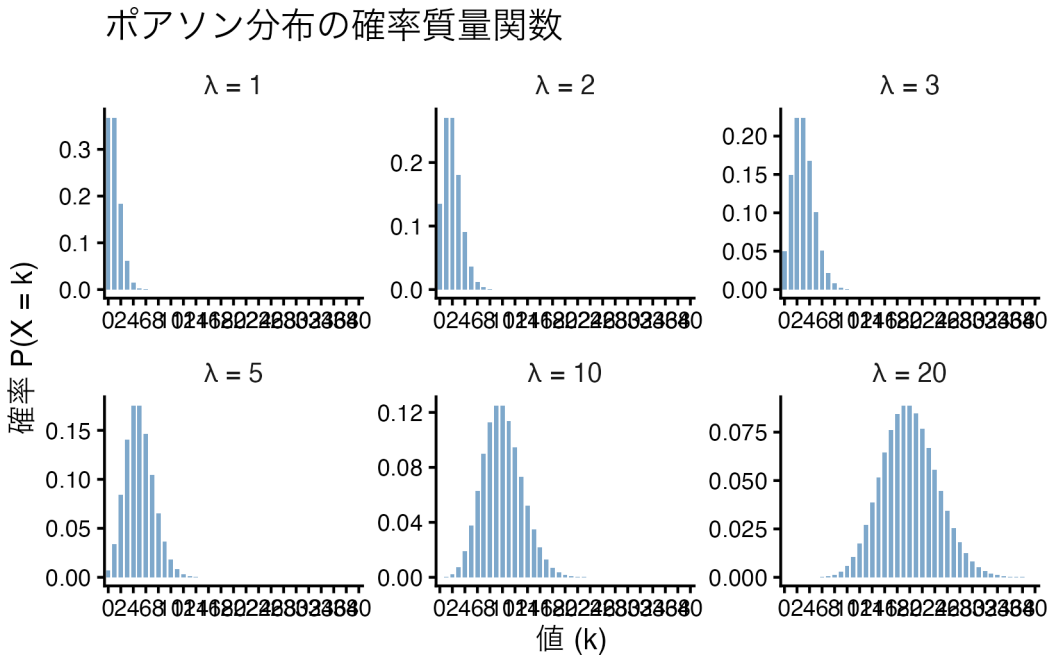
今回のデータでは説明変数の係数が 1.36 と推定されたから，$e^{1.36} = 3.89$ である。これは，説明変数が 1 単位増加すると，成功確率が 3.89 倍になる，ということを意味する。

14.2.2　ポアソン分布に対する線型モデル；ポアソン回帰

今度はカウント変数に対するモデルを考えてみよう。カウント変数は正の整数をとるデータであり，ポアソン分布を用いることができる。ポアソン分布の確率関数は次のように表される。

$$P(Y = y) = \frac{\lambda^y e^{-\lambda}}{y!}$$

ここで，λ は平均である。ポアソン分布の形状も確認しておこう。



このような正の整数しかとらないデータに対してはポアソン分布で回帰した方がよい。ポアソン回帰では**対数関数**をリンク関数として使用する：

$$\log(\lambda_i) = \beta_0 + \beta_1 x_i$$

この関係を λ_i について解くと，**指数関数**（逆リンク関数）が得られる：

$$\lambda_i = \exp(\beta_0 + \beta_1 x_i)$$

として

$$y_i \sim \mathrm{Pois}(\lambda_i)$$

を考えるのである。

以下にサンプルデータを作ったポアソン回帰の例を示す。

```
# データ生成
set.seed(17)
n <- 200
```

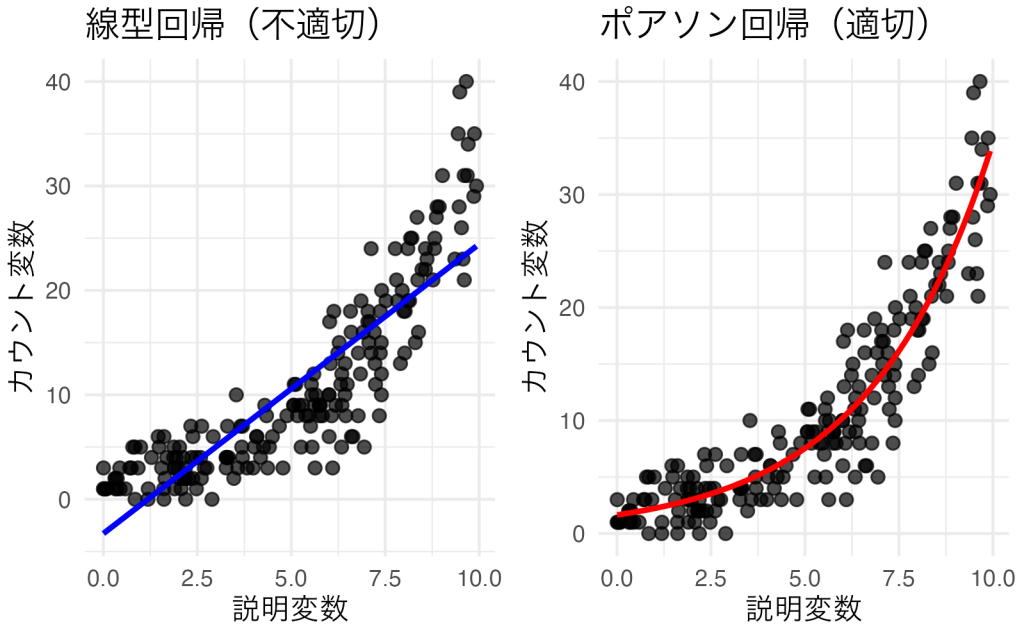
```r
x <- runif(n, min = 0, max = 10)
beta_0 <- 0.5
beta_1 <- 0.3
lambda <- exp(beta_0 + beta_1 * x)
y <- rpois(n, lambda = lambda)

df_pois <- data.frame(x = x, y = y)

# p1: 線型回帰（不適切な例）
p1 <- ggplot(df_pois, aes(x = x, y = y)) +
  geom_point(alpha = 0.7, size = 2) +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  theme_minimal() +
  labs(x = "説明変数", y = "カウント変数", title = "線型回帰（不適切）") +
  theme(text = element_text(family = "IPAexGothic"))

# p2: ポアソン回帰（適切な例）
p2 <- ggplot(df_pois, aes(x = x, y = y)) +
  geom_point(alpha = 0.7, size = 2) +
  geom_smooth(method = "glm", method.args = list(family = "poisson"), se = FALSE, color = "red") +
  theme_minimal() +
  labs(x = "説明変数", y = "カウント変数", title = "ポアソン回帰（適切）") +
  theme(text = element_text(family = "IPAexGothic"))

# パッチワークで結合
p1 + p2
```



線型回帰では負の値の予測値が出てしまう可能性があるのに対し，ポアソン回帰は指数関数による変換によって適切にカウントデータの特性を捉えていることがわかる。

ポアソン回帰を実行する R コードの例を次に示す。

```
result.bayes.pois <- brm(
  y ~ x,
  family = poisson(),
  data = df_pois,
  seed = 12345,
  chains = 4, cores = 4, backend = "cmdstanr",
  iter = 2000, warmup = 1000,
  refresh = 0
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.0 seconds.
Chain 2 finished in 0.0 seconds.
Chain 3 finished in 0.1 seconds.
Chain 4 finished in 0.0 seconds.

All 4 chains finished successfully.
Mean chain execution time: 0.0 seconds.
Total execution time: 0.2 seconds.
```

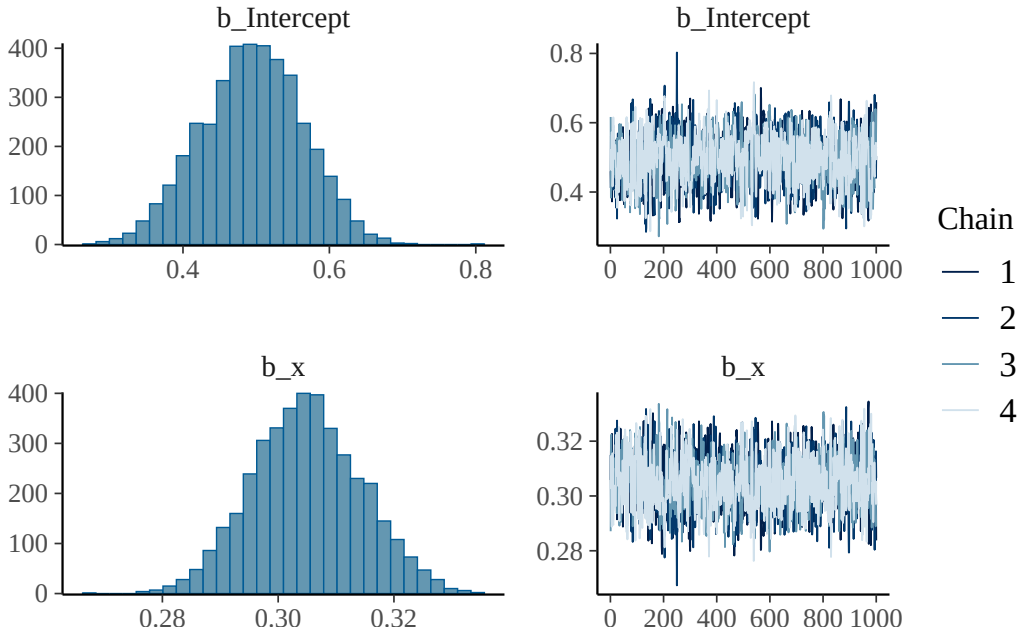
```
summary(result.bayes.pois)
```

```
 Family: poisson
  Links: mu = log
Formula: y ~ x
   Data: df_pois (Number of observations: 200)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     0.49      0.07     0.36     0.63 1.00     1275     1675
x             0.30      0.01     0.29     0.32 1.00     1471     1943

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(result.bayes.pois)
```




```
## 比較のためにML推定も行っておく
result.ml.pois <- glm(y ~ x, family = poisson(), data = df_pois)
summary(result.ml.pois)
```

```

Call:
glm(formula = y ~ x, family = poisson(), data = df_pois)
```

```
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) 0.495962   0.071166   6.969 3.19e-12 ***
x           0.304829   0.009555  31.902  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 1457.6  on 199  degrees of freedom
Residual deviance:  214.3  on 198  degrees of freedom
AIC: 975.04

Number of Fisher Scoring iterations: 4
```

14.3 一般化線型混合モデル (GLMM)

線型モデルのさらなる拡張として，**一般化線型混合モデル (GLMM)**Generalized Linear Mixed Model がある。ここまでの回帰モデルは，説明変数が目的変数に対して一貫した効果を持つと仮定してきた。この効果を特に**固定効果** fixed effect というが，GLMM では固定効果に加えて**変量効果** random effect を考慮することができる。

変量効果とは，説明変数が個体ごとに異なる値を持つことを意味する。たとえば Within デザインの要因計画は個人差を考慮することができるが，これは変量効果を考慮したモデルであるといえる。すなわち，研究として見たい効果は Within 要因の水準ごとの固定効果であり，これとは別に個人の平均値が異なることを想定しているから，個人ごとの平均値を変量効果として考慮しているモデルといえる。

このように変量効果は個人ごとに異なる影響を考えることであるが，仮定としてこうした個人差が確率分布，特に正規分布に従っていると考えるのである。個人差は確率分布からランダムに生じるものであり，個人同士の平均的な違いは交換可能であると考えられる。またこうした個人差の分散は，個人の平均値の分散として捉えられる。このように個人差を表す確率分布も混ぜ込むので，混合 (Mixed) モデルと呼ばれるのである。

14.3.1 個人差の分布を「混ぜる」

分散分析の Between モデルと Within モデルの違いについて考えてみよう。一要因の場合，Between モデルは次のように表される。

$$SS_T = SS_A + SS_e$$

ここで SS_T とは全体の平方和 (Sum of Squares) であり，これを右辺の要因 A で説明する平方和 SS_A と誤差 SS_E に分解するのであった。

このとき Within モデルは，

$$SS_T = SS_A + SS_s + SS_e$$

と表され，ここで SS_s が個人差の平方和である。検定に際しては SS_e に対する SS_A の比率を考える[*3]ことになるから，効果の大きさが同じであれば Within デザインの方が有利である。Between デザインは誤差から個人差を分離できていないからで，これを分離して誤差を小さくできる方が効果 vs 誤差の比は大きくなるからである。

ここでは分散での記述であったが，個別の値 Y_{ij} を考えると，Between デザインは

$$Y_{ij} = \beta_0 + \beta_1 x_{ij} + e_{ij}$$

であり，Within デザインは

$$Y_{ij} = \mu + \beta_{0i} + \beta_1 x_{ij} + e_{ij}$$

である。ここで μ は全平均であり，β_{01} は全平均から個人平均の差，つまり個人差を表している。個人 i に対して反復測定がなされているからその平均を計算することができ，相対的な切片の違いを個人の効果として取り出していると言える。

ここで確率変数を考えると，

$$e_{ij} \sim N(0, \sigma_e)$$
$$\beta_{0i} \sim N(0, \sigma_s)$$

となるから，一つの値に対して複数の確率分布が混合 (mix) されていることがわかる。

ここでは個人差が平均値，すなわち切片が異なるものとして説明したが，傾きに対して個人差を考えることもできる。変量効果がどこにあるかによって，ランダム切片モデル，ランダム傾きモデル，ランダム切片ランダム傾きモデルなどと呼ばれることがある。

14.3.2　ランダム切片モデル

ランダム切片モデルは，切片が個人ごとに異なるモデルである。切片が個人ごとに異なるということは，切片の個人差が正規分布に従うと考えることである。

$$\beta_{0i} = \beta_0 + u_{0i}$$

ここで，β_0 は全体の切片，u_{0i} は個人 i の切片の個人差である。個人差は正規分布に従うと考えるから，

$$u_{0i} \sim N(0, \sigma_u)$$

と表現できる。モデル全体としては

[*3] 詳しくは (浜田他，2019) を参照してほしい

$$y_{ij} = (\beta_0 + u_{0i}) + \beta_1 x_{ij} + e_{ij}$$

となる。ここで，e_{ij} は誤差項であり，個人 i の j 番目の観測値に対する誤差を表す。

具体的なデータを作って見てみよう。

```
# データ生成
set.seed(17)
n_person <- 10 # 個人数
n_obs <- 20 # 各個人の観測数
beta_0 <- 1
beta_1 <- 2
sigma_u <- 1 # 個人差の標準偏差
sigma_e <- 0.5 # 誤差の標準偏差

# 個人ごとのランダム切片
person_intercepts <- rnorm(n_person, mean = 0, sd = sigma_u)

# データフレーム作成
df_random_intercept <- expand_grid(
  person = 1:n_person,
  obs = 1:n_obs
) %>%
  mutate(
    x = runif(n(), min = 0, max = 10),
    u_0 = person_intercepts[person],
    y = beta_0 + u_0 + beta_1 * x + rnorm(n(), mean = 0, sd = sigma_e),
    person_factor = factor(person)
  )

## データの確認
df_random_intercept %>% head()
```

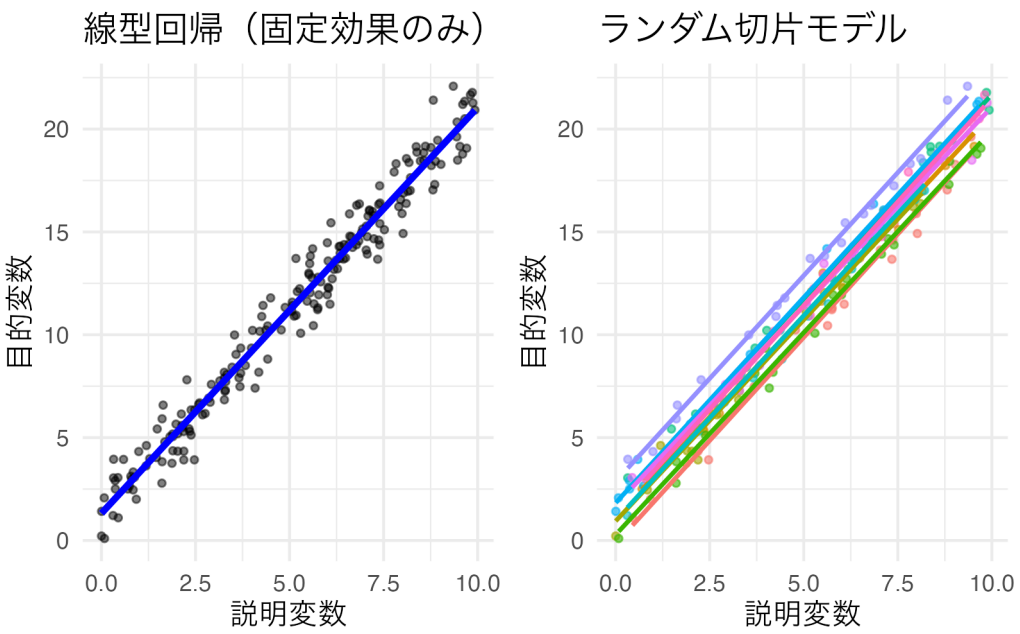
```
# A tibble: 6 x 6
  person   obs     x   u_0     y person_factor
   <int> <int> <dbl> <dbl> <dbl> <fct>
1      1     1  8.81 -1.02 17.0  1
2      1     2  6.07 -1.02 11.5  1
3      1     3  7.40 -1.02 15.4  1
4      1     4  8.03 -1.02 16.9  1
5      1     5  9.02 -1.02 18.3  1
```

```
6      1      6 0.927 -1.02  2.00 1
```

```
# p1: 線型回帰（全体で一つの回帰線）
p1 <- ggplot(df_random_intercept, aes(x = x, y = y)) +
  geom_point(alpha = 0.5, size = 1) +
  geom_smooth(method = "lm", se = FALSE, color = "blue", linewidth = 1.2) +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "線型回帰（固定効果のみ）") +
  theme(text = element_text(family = "IPAexGothic"))

# p2: ランダム切片モデル（個人ごとに異なる切片の回帰線）
p2 <- ggplot(df_random_intercept, aes(x = x, y = y, color = person_factor)) +
  geom_point(alpha = 0.6, size = 1) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "ランダム切片モデル") +
  theme(
    text = element_text(family = "IPAexGothic"),
    legend.position = "none"
  )

p1 + p2
```



ランダム効果を含むモデルを推定するコード例を以下に示す。

```
result.bayes.random_intercept <- brm(
  y ~ x + (1 | person),
```

```
  family = gaussian(),
  data = df_random_intercept,
  seed = 12345,
  chains = 4, cores = 4, backend = "cmdstanr",
  iter = 2000, warmup = 1000,
  refresh = 0
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.4 seconds.
Chain 2 finished in 0.4 seconds.
Chain 3 finished in 0.4 seconds.
Chain 4 finished in 0.5 seconds.

All 4 chains finished successfully.
Mean chain execution time: 0.4 seconds.
Total execution time: 0.7 seconds.
```

```
summary(result.bayes.random_intercept)
```

```
 Family: gaussian
  Links: mu = identity
Formula: y ~ x + (1 | person)
   Data: df_random_intercept (Number of observations: 200)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

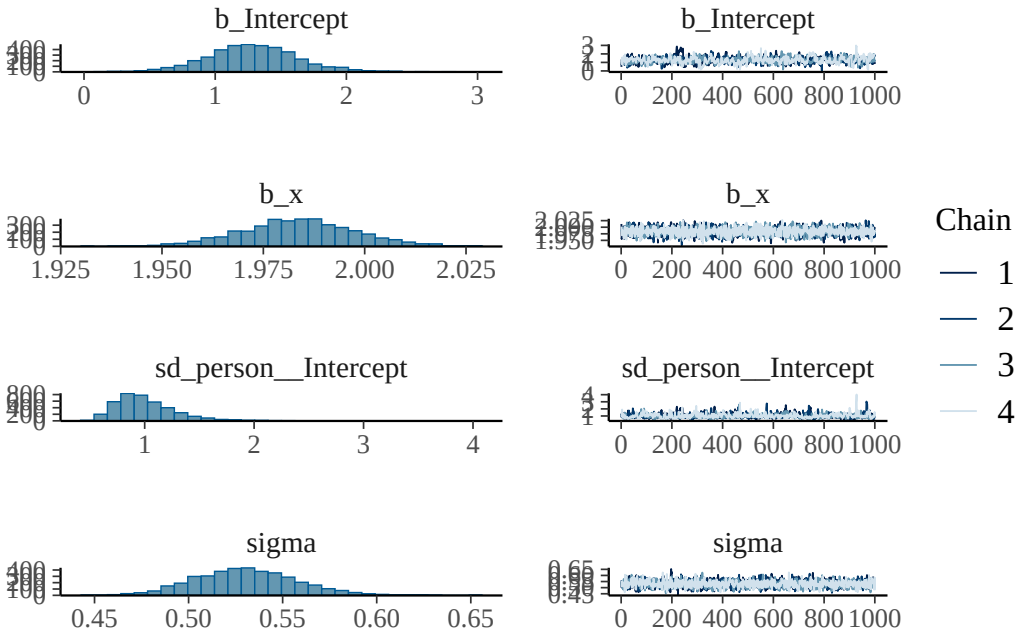
Multilevel Hyperparameters:
~person (Number of levels: 10)
              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(Intercept)     1.01      0.30     0.60     1.73 1.01      408      786

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     1.29      0.35     0.61     2.01 1.01      620      461
x             1.98      0.01     1.96     2.01 1.00     2380     2052

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     0.53      0.03     0.48     0.59 1.00     1790     2245
```

```
Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(result.bayes.random_intercept)
```




```
## 比較のためにML推定も行っておく
pacman::p_load(lmerTest)
result.ml.random_intercept <- lmer(y ~ x + (1 | person), data = df_random_intercept)
summary(result.ml.random_intercept)
```

```
Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]
Formula: y ~ x + (1 | person)
   Data: df_random_intercept

REML criterion at convergence: 356.6

Scaled residuals:
    Min      1Q  Median      3Q     Max
-3.3167 -0.5745 -0.1189  0.6343  2.6464

Random effects:
 Groups   Name        Variance Std.Dev.
 person   (Intercept) 0.7462   0.8638
 Residual             0.2773   0.5266
```

```
Number of obs: 200, groups:  person, 10

Fixed effects:
             Estimate Std. Error        df t value Pr(>|t|)
(Intercept)   1.26699    0.28414  10.15076   4.459  0.00117 **
x             1.98396    0.01361 189.35597 145.774  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
  (Intr)
x -0.242
```

```
## さらに比較のために，普通の回帰分析も行ってみる
result.ml.random_intercept.ordinal <- lm(y ~ x, data = df_random_intercept)
summary(result.ml.random_intercept.ordinal)
```

```
Call:
lm(formula = y ~ x, data = df_random_intercept)

Residuals:
     Min       1Q   Median       3Q      Max
-2.27275 -0.59838  0.08193  0.50855  2.67596

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.31764    0.14235   9.256   <2e-16 ***
x            1.97394    0.02464  80.124   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9771 on 198 degrees of freedom
Multiple R-squared:  0.9701,    Adjusted R-squared:  0.9699
F-statistic:  6420 on 1 and 198 DF,  p-value: < 2.2e-16
```

変量効果のモデル表記は，固定効果に加えて (1 | person) のように表記する。ここで，1 は切片を表し，person は個人を表す。このようにすることで，個人ごとに異なる切片を考慮することができる。

設定したパラメタ（個人差の標準偏差 $\sigma_u = 1$，残差の標準偏差 $\sigma_e = 0.5$）に対して，ベイズ推定では個人差の標準偏差が 1.012，残差の標準偏差が 0.53 と推定されている。ML 推定では個人差の標準偏差が 0.864，残差の標準偏差が 0.527 と推定された。両手法ともに設定値に近い値が得られており，適切にパラメタが推定されていること

がわかる。

このデータに対して，固定効果のみで推定すると切片が 1.318，傾きが 1.974 と推定される。傾きに対しては同じ値になるが切片の変動を考慮するかどうかで推定値が異なっていることがわかる。固定効果のみのモデルの場合，切片に想定された正規分布の平均値のみ推定したことになっている。実践的な意味としては，Within 計画を Between 計画と皆して推定しているようなものであり，せっかく綺麗に取り分けられる個人差分散を無視しているようなものである。

14.3.2.1　個人レベルの推定値の取得

ベイズ推定の利点の一つは，個人ごとの推定値とその不確実性を定量化できることである。MCMC サンプルから個人の変量効果を取り出し，事後分布を可視化してみよう。

brms パッケージの関数 ranef を使うと面倒なことをしなくても直接取り出してくれるが，MCMC サンプルを取り出して加工して確認することは，MCMC による推定の理解を深めるのに良い訓練となる。

```
# 個人ごとの変量効果を取得して表示してみる
random_effects <- ranef(result.bayes.random_intercept)
print(random_effects)
```

```
$person
, , Intercept

     Estimate Est.Error       Q2.5      Q97.5
1  -1.2842573 0.3604082 -2.0328410 -0.6093268
2  -0.2815002 0.3612608 -1.0005543  0.4143774
3  -0.4328320 0.3614324 -1.1734025  0.2534430
4  -1.0881607 0.3642192 -1.8473431 -0.4056352
5   0.5388313 0.3636371 -0.2012680  1.2376421
6  -0.2689177 0.3633187 -1.0316210  0.4175833
7   0.5917790 0.3634519 -0.1563298  1.3063219
8   1.6501281 0.3597518  0.9219949  2.3568713
9   0.1666085 0.3624218 -0.5813913  0.8600218
10  0.2276495 0.3628401 -0.5134036  0.9184449
```

```
# 個人の切片の変量効果の事後分布(MCMCサンプル)を取得
posterior_samples <- as_draws_df(result.bayes.random_intercept) %>%
  select(starts_with("r_person")) %>%
  rowid_to_column("iter") %>%
  pivot_longer(-iter) %>%
  mutate(person = str_extract(name, pattern = "\\d+")) %>%
  mutate(person = factor(person, levels = as.character(1:10))) %>%
  select(-name)
```

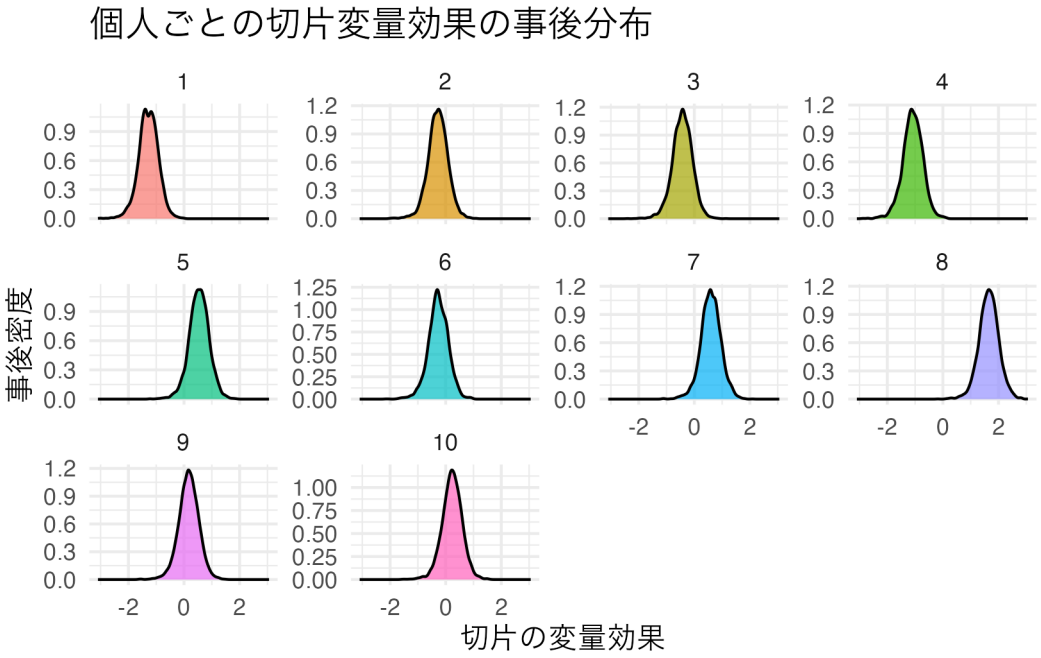
```
Warning: Dropping 'draws_df' class as required metadata was removed.
```

```
# MCMCサンプルを使った要約
posterior_samples %>%
  group_by(person) %>%
  summarise(
    EAP = mean(value),
    median = quantile(value, 0.5),
    q025 = quantile(value, 0.025),
    q975 = quantile(value, 0.975),
    sd = sd(value),
    .groups = "drop"
  )
```

```
# A tibble: 10 x 6
   person    EAP median   q025   q975    sd
   <fct>   <dbl>  <dbl>  <dbl>  <dbl> <dbl>
 1 1      -1.28  -1.28  -2.03  -0.609 0.360
 2 2      -0.282 -0.273 -1.00   0.414 0.361
 3 3      -0.433 -0.425 -1.17   0.253 0.361
 4 4      -1.09  -1.08  -1.85  -0.406 0.364
 5 5       0.539  0.545 -0.201  1.24  0.364
 6 6      -0.269 -0.270 -1.03   0.418 0.363
 7 7       0.592  0.593 -0.156  1.31  0.363
 8 8       1.65   1.66   0.922  2.36  0.360
 9 9       0.167  0.171 -0.581  0.860 0.362
10 10      0.228  0.235 -0.513  0.918 0.363
```

```
# 事後分布の描画
p1 <- ggplot(posterior_samples, aes(x = value, fill = person)) +
  geom_density(alpha = 0.7) +
  facet_wrap(~person, scales = "free_y") +
  theme_minimal() +
  labs(x = "切片の変量効果", y = "事後密度", title = "個人ごとの切片変量効果の事後分布") +
  theme(text = element_text(family = "IPAexGothic"), legend.position = "none")

print(p1)
```



ここで抜き出された個人の効果は，全体平均からの偏差であり，実際の切片は固定効果＋変量効果の形で得られている。これについても coef 関数あるいは MCMC サンプルを加工することで，より具体的にイメージしながら利用できるだろう。

```
# 各個人の実際の切片（固定効果+変量効果）を取得
individual_coefs <- coef(result.bayes.random_intercept)$person

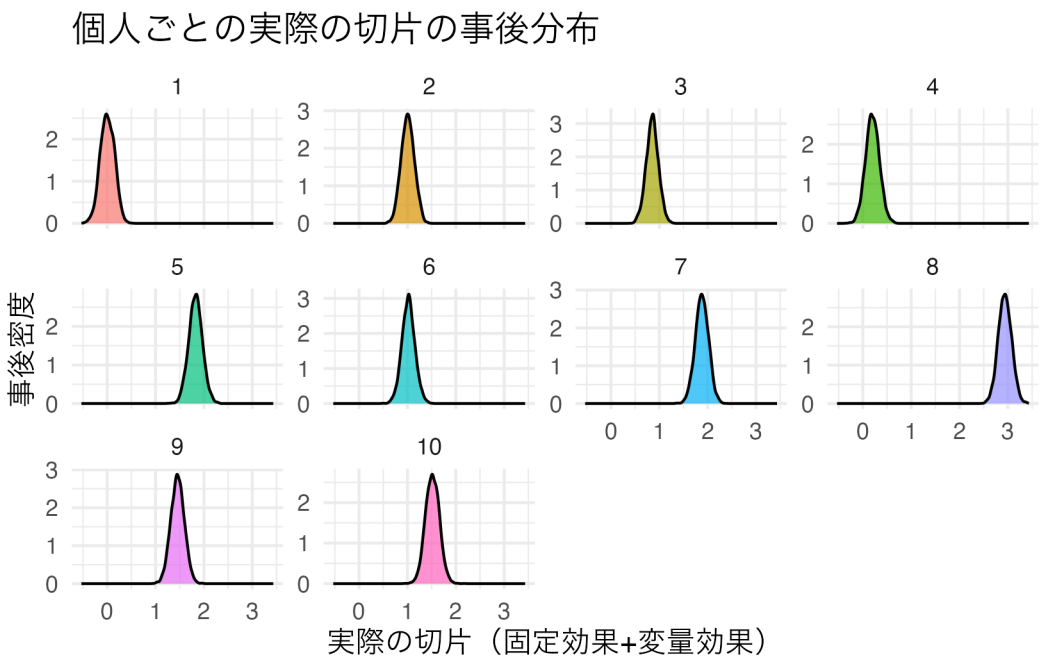
# 実際の切片の事後分布（固定効果+変量効果）
total_intercept_samples <- as_draws_df(result.bayes.random_intercept) %>%
  select(b_Intercept, starts_with("r_person")) %>%
  rowid_to_column("iter") %>%
  pivot_longer(-c(iter, b_Intercept)) %>%
  mutate(person = str_extract(name, pattern = "\\d+")) %>%
  mutate(person = factor(person, levels = as.character(1:10))) %>%
  mutate(total_intercept = b_Intercept + value) %>%
  select(iter, person, total_intercept)
```

```
Warning: Dropping 'draws_df' class as required metadata was removed.
```

```
# 実際の切片の事後分布の描画
p2 <- ggplot(total_intercept_samples, aes(x = total_intercept, fill = person)) +
  geom_density(alpha = 0.7) +
  facet_wrap(~person, scales = "free_y") +
  theme_minimal() +
  labs(
    x = "実際の切片（固定効果+変量効果）", y = "事後密度",
```

```
    title = "個人ごとの実際の切片の事後分布"
  ) +
  theme(text = element_text(family = "IPAexGothic"), legend.position = "none")

print(p2)
```




```
# 実際の切片の信頼区間
total_intercept_summary <- total_intercept_samples %>%
  group_by(person) %>%
  summarise(
    EAP = mean(total_intercept),
    median = quantile(total_intercept, 0.5),
    q025 = quantile(total_intercept, 0.025),
    q975 = quantile(total_intercept, 0.975),
    sd = sd(total_intercept),
    .groups = "drop"
  )

print(total_intercept_summary)
```

```
# A tibble: 10 x 6
   person      EAP  median    q025  q975    sd
   <fct>     <dbl>   <dbl>   <dbl> <dbl> <dbl>
 1 1       0.00382 0.00210 -0.284  0.279 0.146
 2 2       1.01    1.01     0.746  1.27  0.135
```

```
 3  3      0.855   0.857    0.597   1.10  0.128
 4  4      0.200   0.197   -0.0805  0.484 0.143
 5  5      1.83    1.83     1.55    2.11  0.142
 6  6      1.02    1.02     0.753   1.29  0.136
 7  7      1.88    1.88     1.61    2.14  0.137
 8  8      2.94    2.94     2.67    3.21  0.137
 9  9      1.45    1.45     1.18    1.73  0.139
10 10      1.52    1.52     1.24    1.80  0.144
```

このように，ベイズ推定では個人ごとの推定値とその不確実性を詳細に分析することができる。

14.3.3　ランダム傾きモデル

ランダム傾きモデルは，傾きが個人ごとに異なるモデルである。傾きが個人ごとに異なるということは，傾きの個人差が正規分布に従うと考えることである。

$$\beta_{1i} = \beta_1 + u_{1i}$$

ここで，β_1 は全体の傾き，u_{1i} は個人 i の傾きの個人差である。個人差は正規分布に従うと考えるから，

$$u_{1i} \sim N(0, \sigma_u)$$

と表現できる。モデル全体としては

$$y_{ij} = \beta_0 + (\beta_1 + u_{1i})x_{ij} + e_{ij}$$

となる。

これについても具体的なデータを作って見てみよう。

```r
# データ生成
set.seed(17)
n_person <- 10 # 個人数
n_obs <- 20 # 各個人の観測数
beta_0 <- 1
beta_1 <- 2
sigma_u <- 0.5 # 傾きの個人差の標準偏差
sigma_e <- 1.5 # 誤差の標準偏差

# 個人ごとのランダム傾き
person_slopes <- rnorm(n_person, mean = 0, sd = sigma_u)
```

```
# データフレーム作成
df_random_slope <- expand_grid(
  person = 1:n_person,
  obs = 1:n_obs
) %>%
  mutate(
    x = runif(n(), min = 0, max = 10),
    u_1 = person_slopes[person],
    y = beta_0 + (beta_1 + u_1) * x + rnorm(n(), mean = 0, sd = sigma_e),
    person_factor = factor(person)
  )

## データの確認
df_random_slope %>% head()
```

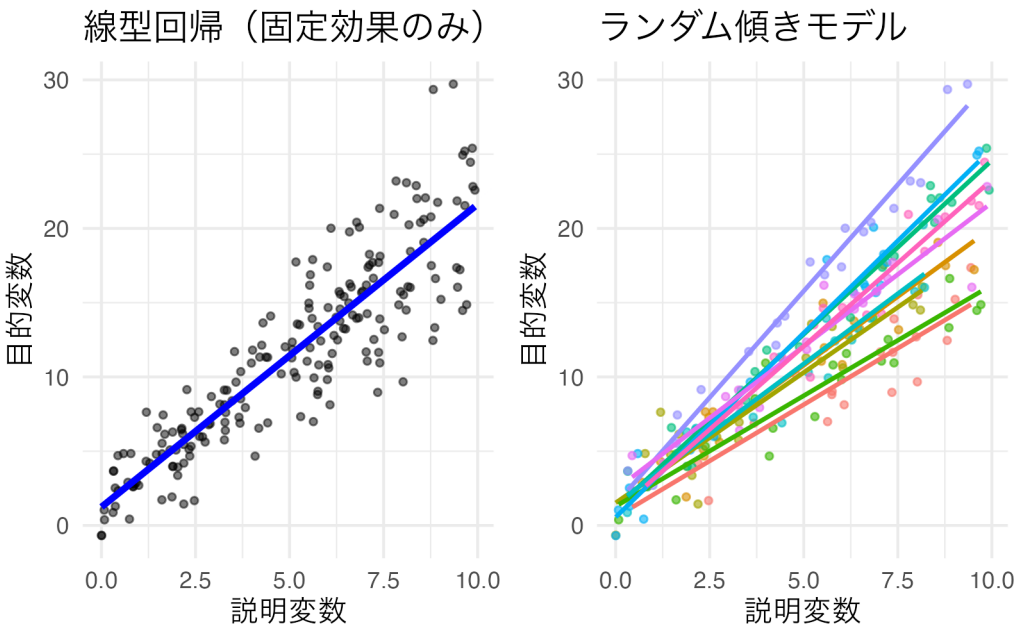
```
# A tibble: 6 x 6
  person   obs     x    u_1     y person_factor
   <int> <int> <dbl>  <dbl> <dbl> <fct>
1      1     1 8.81  -0.508 12.5  1
2      1     2 6.07  -0.508  8.12 1
3      1     3 7.40  -0.508 13.9  1
4      1     4 8.03  -0.508 15.5  1
5      1     5 9.02  -0.508 15.2  1
6      1     6 0.927 -0.508  2.88 1
```

```
# p1: 線型回帰（全体で一つの回帰線）
p1 <- ggplot(df_random_slope, aes(x = x, y = y)) +
  geom_point(alpha = 0.5, size = 1) +
  geom_smooth(method = "lm", se = FALSE, color = "blue", linewidth = 1.2) +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "線型回帰（固定効果のみ）") +
  theme(text = element_text(family = "IPAexGothic"))

# p2: ランダム傾きモデル（個人ごとに異なる傾きの回帰線）
p2 <- ggplot(df_random_slope, aes(x = x, y = y, color = person_factor)) +
  geom_point(alpha = 0.6, size = 1) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "ランダム傾きモデル") +
  theme(
```

```
    text = element_text(family = "IPAexGothic"),
    legend.position = "none"
  )

p1 + p2
```



ランダム傾きモデルでは，個人ごとに回帰直線の傾きが異なることがわかる。これは，説明変数の効果が個人によって異なることを表している。

ランダム傾きモデルを推定するコード例を以下に示す。

```
result.bayes.random_slope <- brm(
  y ~ x + (0 + x | person),
  family = gaussian(),
  data = df_random_slope,
  seed = 12345,
  chains = 4, cores = 4, backend = "cmdstanr",
  iter = 2000, warmup = 1000,
  refresh = 0
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.4 seconds.
Chain 3 finished in 0.3 seconds.
Chain 2 finished in 0.4 seconds.
Chain 4 finished in 0.4 seconds.
```

```
All 4 chains finished successfully.
Mean chain execution time: 0.4 seconds.
Total execution time: 0.5 seconds.
```

```
summary(result.bayes.random_slope)
```

```
 Family: gaussian
  Links: mu = identity
Formula: y ~ x + (0 + x | person)
   Data: df_random_slope (Number of observations: 200)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

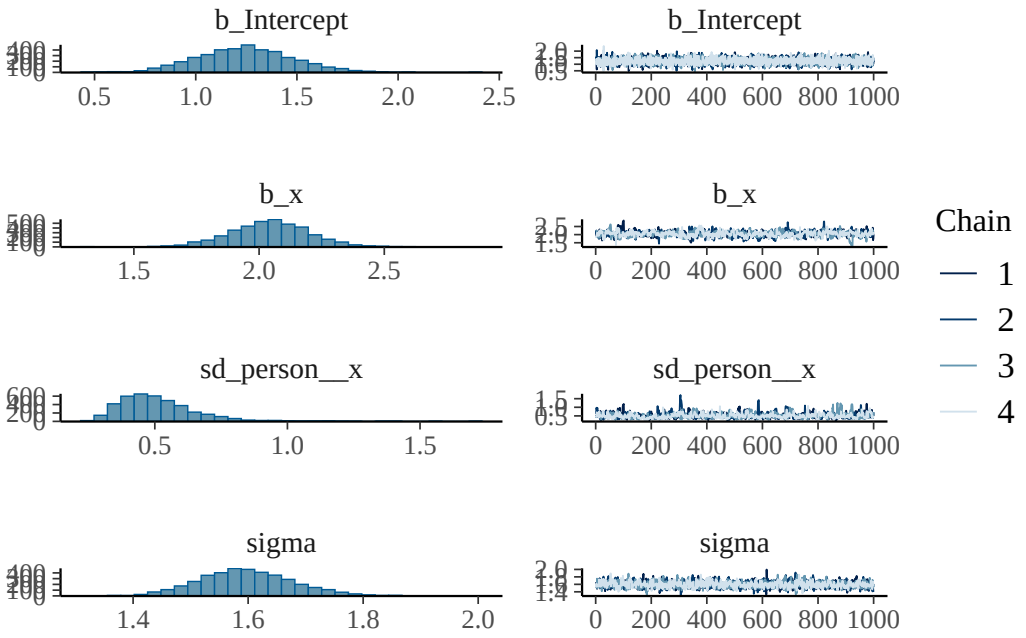
Multilevel Hyperparameters:
~person (Number of levels: 10)
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(x)     0.51      0.15     0.30     0.86 1.00      527     1003

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     1.24      0.24     0.79     1.71 1.00     5674     3203
x             2.04      0.16     1.73     2.36 1.00      605      743

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     1.60      0.08     1.45     1.76 1.00     1642     1684

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(result.bayes.random_slope)
```




```
## 比較のためにML推定も行っておく
result.ml.random_slope <- lmer(y ~ x + (0 + x | person), data = df_random_slope)
summary(result.ml.random_slope)
```

```
Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]
Formula: y ~ x + (0 + x | person)
   Data: df_random_slope

REML criterion at convergence: 792.3

Scaled residuals:
    Min      1Q  Median      3Q     Max
-3.2526 -0.6127 -0.0716  0.6858  2.6584

Random effects:
 Groups   Name Variance Std.Dev.
 person   x    0.1894   0.4352
 Residual      2.5139   1.5855
Number of obs: 200, groups:  person, 10

Fixed effects:
            Estimate Std. Error       df t value Pr(>|t|)
(Intercept)   1.2415     0.2335 189.2290   5.317 2.96e-07 ***
x             2.0451     0.1436  10.2643  14.240 4.31e-08 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
  (Intr)
x -0.250
```

変量効果のモデル表記で，(0 + x | person) は傾きのみにランダム効果を考慮することを表す。0 によって切片の固定効果を除き，x で傾きにランダム効果を指定している。それぞれの出力が，理論値のどこを反映したものであるか確認しておこう。

また出力結果はランダム切片モデルの時と同様に，関数あるいは MCMC サンプルから推定値やその分布情報を得ることもできる。自分なりに加工して出力結果を確認してみてほしい。

14.3.4 ランダム切片ランダム傾きモデル

ランダム切片ランダム傾きモデルは，切片と傾きの両方が個人ごとに異なるモデルである。これは最も複雑な変量効果モデルで，個人ごとに異なる切片と傾きを同時に考慮する。

$$\beta_{0i} = \beta_0 + u_{0i}$$
$$\beta_{1i} = \beta_1 + u_{1i}$$

ここで，u_{0i} は個人 i の切片の個人差，u_{1i} は個人 i の傾きの個人差である。このランダム効果はいずれも個人に係るものであるから相関することが仮定されるため，それぞれに正規分布を仮定するのではなく多変量正規分布から得られるものと仮定する。当然のことながら，この相関係数も推定対象である。

$$\begin{pmatrix} u_{0i} \\ u_{1i} \end{pmatrix} \sim MVN\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \sigma_{u0}^2 & \sigma_{u01} \\ \sigma_{u01} & \sigma_{u1}^2 \end{pmatrix} \right)$$

モデル全体としては

$$y_{ij} = (\beta_0 + u_{0i}) + (\beta_1 + u_{1i})x_{ij} + e_{ij}$$

となる。

```
# データ生成
set.seed(17)
n_person <- 10 # 個人数
n_obs <- 20 # 各個人の観測数
beta_0 <- 1
beta_1 <- 2
sigma_u0 <- 1.0 # 切片の個人差の標準偏差
sigma_u1 <- 0.5 # 傾きの個人差の標準偏差
```

```r
rho <- 0.3 # 切片と傾きの相関
sigma_e <- 0.5 # 誤差の標準偏差

# 切片と傾きの共分散行列
Sigma <- matrix(
  c(
    sigma_u0^2, rho * sigma_u0 * sigma_u1,
    rho * sigma_u0 * sigma_u1, sigma_u1^2
  ),
  nrow = 2
)

# 個人ごとのランダム効果（切片と傾き）
pacman::p_load(MASS)
random_effects <- mvrnorm(n_person, mu = c(0, 0), Sigma = Sigma)
person_intercepts <- random_effects[, 1]
person_slopes <- random_effects[, 2]

# データフレーム作成
df_random_both <- expand_grid(
  person = 1:n_person,
  obs = 1:n_obs
) %>%
  mutate(
    x = runif(n(), min = 0, max = 10),
    u_0 = person_intercepts[person],
    u_1 = person_slopes[person],
    y = (beta_0 + u_0) + (beta_1 + u_1) * x + rnorm(n(), mean = 0, sd = sigma_e),
    person_factor = factor(person, levels = as.character(1:10))
  )

## データの確認
df_random_both %>% head()
```

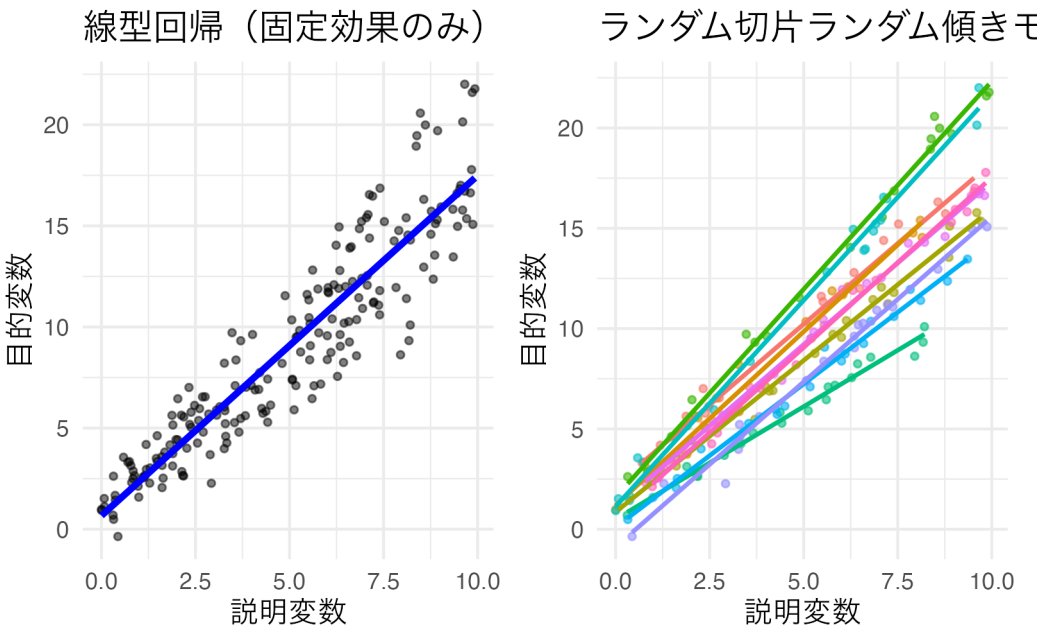
```
# A tibble: 6 x 7
  person   obs     x   u_0    u_1     y person_factor
   <int> <int> <dbl> <dbl>  <dbl> <dbl> <fct>
1      1     1  7.52  1.12 -0.351 15.2  1
2      1     2  6.34  1.12 -0.351 12.8  1
3      1     3  2.48  1.12 -0.351  6.12 1
```

```
4       1     4  5.51  1.12 -0.351 11.3  1
5       1     5  2.35  1.12 -0.351  5.18 1
6       1     6  2.59  1.12 -0.351  5.38 1
```

```r
# p1: 線型回帰（全体で一つの回帰線）
p1 <- ggplot(df_random_both, aes(x = x, y = y)) +
  geom_point(alpha = 0.5, size = 1) +
  geom_smooth(method = "lm", se = FALSE, color = "blue", linewidth = 1.2) +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "線型回帰（固定効果のみ）") +
  theme(text = element_text(family = "IPAexGothic"))

# p2: ランダム切片ランダム傾きモデル（個人ごとに異なる切片と傾きの回帰線）
p2 <- ggplot(df_random_both, aes(x = x, y = y, color = person_factor)) +
  geom_point(alpha = 0.6, size = 1) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.8) +
  theme_minimal() +
  labs(x = "説明変数", y = "目的変数", title = "ランダム切片ランダム傾きモデル") +
  theme(
    text = element_text(family = "IPAexGothic"),
    legend.position = "none"
  )

p1 + p2
```



プロットを見ると，個人ごとに色分けしていない図 (左；固定効果のみの線型回帰) だけ見て単純な線型回帰でも

いいように思えるが，色分けした図 (右；ランダム切片ランダム傾きモデル) をみると，より個体差をうまく捉えていることがわかる。データはまず可視化することが重要であることがわかるだろう。

さて，これについての推定コードの書き方も見ておこう。切片と傾きの両方が変量効果として () の中に記述される。

```
result.bayes.random_both <- brm(
  y ~ x + (1 + x | person),
  family = gaussian(),
  data = df_random_both,
  seed = 12345,
  chains = 4, cores = 4, backend = "cmdstanr",
  iter = 2000, warmup = 1000,
  refresh = 0
)
```

```
Running MCMC with 4 parallel chains...

Chain 2 finished in 2.3 seconds.
Chain 3 finished in 2.3 seconds.
Chain 1 finished in 2.5 seconds.
Chain 4 finished in 2.4 seconds.

All 4 chains finished successfully.
Mean chain execution time: 2.4 seconds.
Total execution time: 2.6 seconds.
```

```
summary(result.bayes.random_both)
```

```
Warning: There were 7 divergent transitions after warmup. Increasing
adapt_delta above 0.8 may help. See
http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup

 Family: gaussian
  Links: mu = identity
Formula: y ~ x + (1 + x | person)
   Data: df_random_both (Number of observations: 200)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Multilevel Hyperparameters:
~person (Number of levels: 10)
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
```

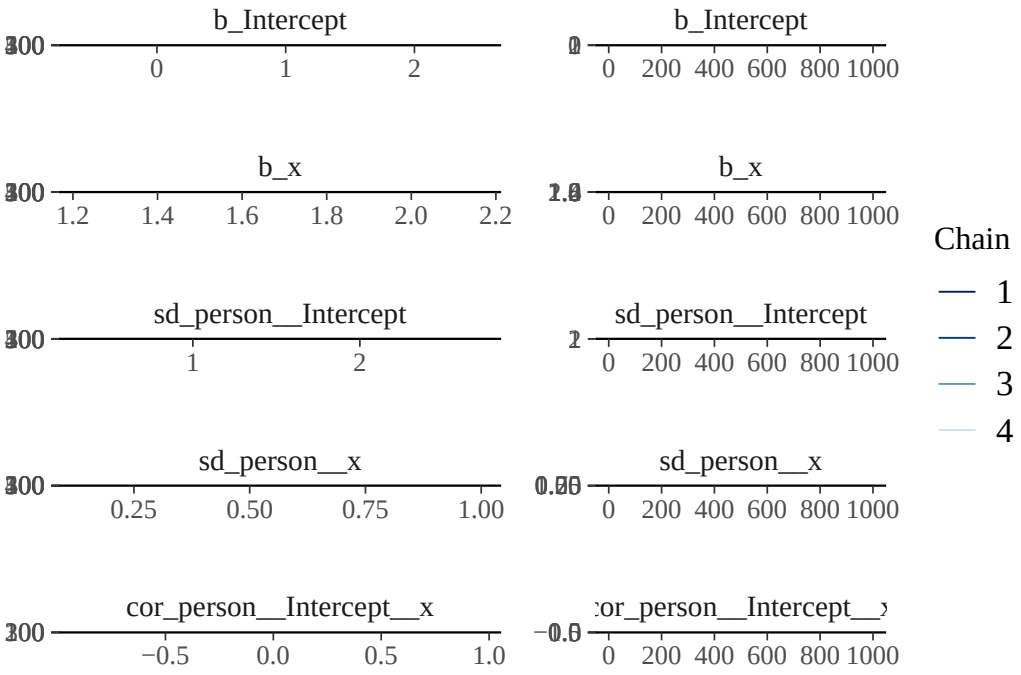
```
sd(Intercept)         0.96      0.29      0.55      1.70 1.00      1083      2237
sd(x)                 0.34      0.10      0.20      0.59 1.00      1244      1640
cor(Intercept,x)      0.33      0.30     -0.34      0.80 1.00      1492      2408

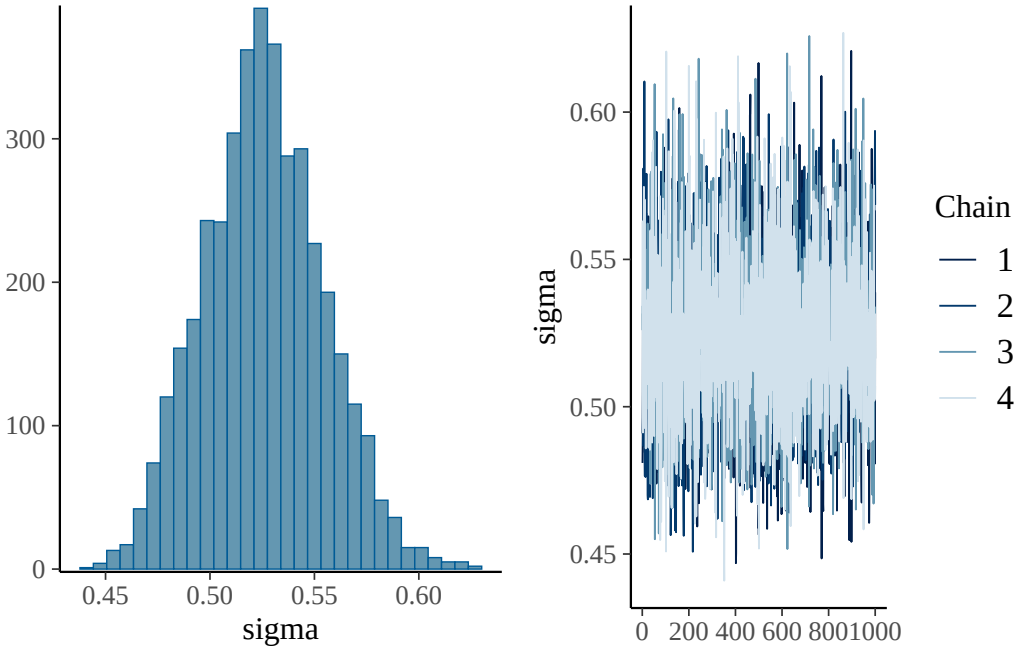
Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     0.83      0.33     0.14     1.47 1.00     1360     1763
x             1.65      0.11     1.43     1.89 1.00     1064     1454

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     0.53      0.03     0.47     0.58 1.00     4200     2756

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
plot(result.bayes.random_both)
```






```
## 比較のためにML推定も行っておく
result.ml.random_both <- lmer(y ~ x + (1 + x | person), data = df_random_both)
summary(result.ml.random_both)
```

```
Linear mixed model fit by REML. t-tests use Satterthwaite's method [
lmerModLmerTest]
Formula: y ~ x + (1 + x | person)
   Data: df_random_both

REML criterion at convergence: 385

Scaled residuals:
    Min      1Q  Median      3Q     Max
-3.4362 -0.6324 -0.0689  0.5880  2.7101

Random effects:
 Groups   Name        Variance Std.Dev. Corr
 person   (Intercept) 0.62433  0.7901
          x           0.07568  0.2751   0.43
 Residual             0.27260  0.5221
Number of obs: 200, groups:  person, 10

Fixed effects:
            Estimate Std. Error      df t value Pr(>|t|)
(Intercept)  0.83552    0.26122 8.76380   3.199   0.0112 *
```

```
x            1.65027    0.08804 8.98000  18.745 1.65e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:
  (Intr)
x 0.369
```

変量効果のモデル表記で，(1 + x | person) は切片と傾きの両方にランダム効果を考慮することを表す。1 は切片，x は傾きのランダム効果を指定している。

このモデルでは切片と傾きの相関も推定される。設定した相関（$\rho = 0.3$）がどの程度再現されているか確認してみよう。また，個人レベルの推定値も前述の方法で取り出すことができる。

14.4 階層線型モデル (HLM)

階層線型モデルは，さらに確率分布を Mix していくモデルである。個人差変数を変量効果として考えた時，ある個人 i から複数の反応 j を得てデータとしているのであった。これを個人に「ネストされた」データ (入子状のデータ。マトリョーシカのようなもの) として考えると，階層線型モデルのイメージに近い。

階層線型モデルが扱うのは，階層構造を持つデータである。例えばある学級での教育効果を考えるとして，調査研究の範囲がひろがれば学級間比較，学校間比較となっていくだろう。学級は学校にネストされているし，学校は地区にネストされているし，地区は市区町村に，市区町村は都道府県にネストされている，という形で階層を考えることができる。

また，学級の中の個人，個人の中の課題種別・・・と内側にネストしていくこともできる。ポイントはネストのレベルに対応した確率分布を仮定するから，そのネストレベルに含まれる各要素は質が同じで相互に交換可能なものであると考える点である。例えば記憶実験などにおいて，同程度の「無意味さ」を持った単語を覚えると言ったとき，「めぬそ」「ぬきは」といっった言葉はどちらを刺激として与えられても三文字の無意味な綴りであるという点では等質と皆しているのである。

また階層線型モデルの面白いところは，上位の変数が下位の変数に影響を与えるようなモデルや，その逆のようなレベル間関係をモデリングできるところである。たとえば市区町村レベルの住民の数が，学級レベルの教育水準に影響をしているかもしれない。こういったレベル間関係も (理論的には) 表現できるのである。

どこにレベルを想定するか，そのレベルを想定する意味があるかどうかについても検証する必要がある。過度に複雑なモデルにしても意味がないので，そのレベルで一旦まとめる必要があるかどうかを，事前にチェックする必要がある。レベルでまとめることの意義は，級内相関 (Interclass Correlation Coefficient) を見ることで判断する。

14.4.1 2 レベル階層線型モデル

最も基本的な階層線型モデルは 2 レベルモデルである。ここでは学級（クラスター）にネストされた生徒の学習データを例に，2 レベル階層線型モデルについて説明する。

14.4.1.1　レベル 1（個人レベル）のモデル

個人 i（$i = 1, 2, \ldots, n_j$）が学級 j（$j = 1, 2, \ldots, J$）に属している場合，レベル 1 のモデルは以下のように表される：

$$Y_{ij} = \beta_{0j} + \beta_{1j} X_{ij} + r_{ij}$$

ここで：

- Y_{ij}：学級 j の生徒 i の学習成績
- X_{ij}：学級 j の生徒 i の説明変数（例：学習時間）
- β_{0j}：学級 j の切片（学級 j の平均的な学習成績）
- β_{1j}：学級 j の傾き（学級 j における説明変数の効果）
- r_{ij}：個人レベルの誤差項，$r_{ij} \sim N(0, \sigma^2)$

14.4.1.2　レベル 2（学級レベル）のモデル

学級レベルでは，レベル 1 の係数が学級レベルの変数の関数として表される：

$$\beta_{0j} = \gamma_{00} + \gamma_{01} W_j + u_{0j}$$

$$\beta_{1j} = \gamma_{10} + \gamma_{11} W_j + u_{1j}$$

ここで：

- W_j：学級 j の説明変数（例：学級規模）
- γ_{00}：全体の切片（全学級の平均的な学習成績）
- γ_{01}：学級レベル変数 W_j の切片への効果
- γ_{10}：全体の傾き（全学級の平均的な効果の大きさ）
- γ_{11}：学級レベル変数 W_j の傾きへの効果
- u_{0j}, u_{1j}：学級レベルの誤差項（変量効果）

変量効果は多変量正規分布に従うと仮定される：

$$\begin{pmatrix} u_{0j} \\ u_{1j} \end{pmatrix} \sim MVN\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \tau_{00} & \tau_{01} \\ \tau_{01} & \tau_{11} \end{pmatrix} \right)$$

14.4.1.3　統合モデル

レベル 1 とレベル 2 のモデルを統合すると：

$$Y_{ij} = \gamma_{00} + \gamma_{01} W_j + \gamma_{10} X_{ij} + \gamma_{11} W_j X_{ij} + u_{0j} + u_{1j} X_{ij} + r_{ij}$$

この式は以下のように分解できる：

固定効果（Fixed Effects）：

- γ_{00}：全体切片
- γ_{01}：学級レベル変数の主効果
- γ_{10}：個人レベル変数の主効果
- γ_{11}：クロスレベル交互作用効果

変量効果（Random Effects）：

- u_{0j}：学級の切片における変量効果
- $u_{1j}X_{ij}$：学級の傾きにおける変量効果
- r_{ij}：個人レベルの誤差

14.4.1.4 級内相関係数（ICC）

階層構造の意義を評価するために，級内相関係数（Intraclass Correlation Coefficient: ICC）を計算する。ICC は同じクラスター内の観測値間の相関を表す：

$$\text{ICC} = \frac{\tau_{00}}{\tau_{00} + \sigma^2}$$

ここで：

- τ_{00}：学級間分散（between-group variance）
- σ^2：学級内分散（within-group variance）

ICC が 0 に近い場合，階層構造を考慮する必要性は低く，ICC が大きい場合（一般的に 0.05 以上），階層モデルの適用が推奨される。

14.4.2 モデルの図解

数式で表現することでわかりにくくなったかもしれないので，図解しておく。この図は下から順にみていくとよい。データ Y_{ij} が図の底にあり，これをある正規分布が生成していると考える。この正規分布の幅 σ_e^2 に応じて個人レベルの誤差 r_{ij} が生成される。この正規分布の平均は会期モデルで表現されるが，その時の回帰係数が多次元正規分布によって生成される。多次元正規分布はそれぞれの位置 (平均) と幅をもち，さらに両者の相関関係 ρ も含まれる。切片 β_{0j} および傾き β_{1j} に生じる誤差 u_{0j}, u_{1j} が幅 τ_{00}, τ_{11} に応じて生成され，またその平均は学級レベルの変数で説明される会期式になっている。ここでの切片と傾きについては事前情報がないので，無情報分布 (一様分布) で表している。誤差の事前分布は負の数を取らないように，切断された分布をもちいる。cauchy 分布や t 分布など，裾の重い分布が用いられることが多い。

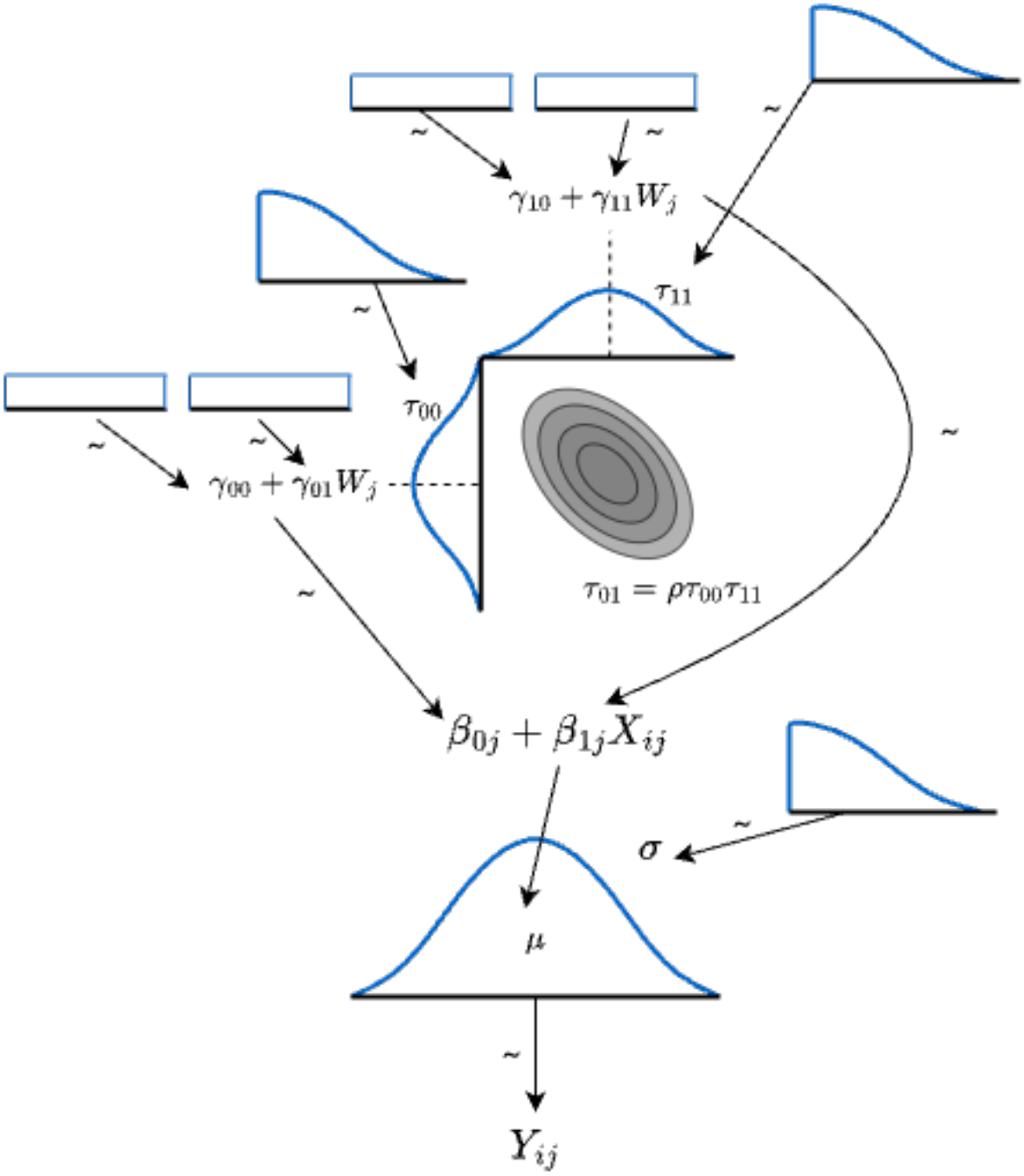


Figure14.1: HLM のモデル図

14.4.3　具体例

具体的な応用例を見ておこう。ここでは，これまで用いてきた野球のデータを用いる。このデータには選手の個人成績（身長，体重，安打数など）と，選手が所属するチーム，守備位置といった階層構造が含まれている。

野球データの特徴として，選手は特定のチームに所属し，そのチーム内で特定の守備位置を担当するという階層構造がある。つまり，守備位置はチームにネストされた構造となっている。チームレベルでは戦術や指導方法が共通し，同じチーム内の守備位置レベルでは求められる技能や体格が類似する傾向がある。このような階層構造を無視して分析すると，個人差を過大評価したり，統計的推論に誤りが生じる可能性がある。

本例では 2020 年度のデータに限定し，投手を除く野手のデータのみを使用する。投手は他の野手とは明らかに異なる特性を持つため，分析から除外することで，より一貫性のあるデータセットとする。

まずは階層構造の意義を確認するため，チームおよび守備位置の ICC（級内相関係数）を計算し，階層モデルの適用が適切かどうかを検証する。ICC の計算には `multilevel` パッケージを用いた。

```
pacman::p_load(tidyverse, brms, bayesplot, multilevel)
# brmsのバックエンドをcmdstanrに統一する（rstan非依存）
options(brms.backend = "cmdstanr")
dat <- read_csv("Baseball.csv") %>%
  filter(Year == "2020年度") %>%
  mutate(
    position = as.factor(position)
  ) %>%
  filter(position != "投手")
```

```
Rows: 7944 Columns: 17
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (6): Year, Name, team, bloodType, UniformNum, position
dbl (11): salary, height, weight, Games, AtBats, Hit, HR, Win, Lose, Save, Hold

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Calculate ICC using multilevel package
# チームレベルのICC
icc_team <- multilevel::ICC1(aov(Hit ~ team, data = dat))

# ネスト構造を考慮したICC計算
# チーム内ポジションのICC
dat$team_position <- paste(dat$team, dat$position, sep = "_")
icc_team_position <- multilevel::ICC1(aov(Hit ~ team_position, data = dat))

# 参考：ポジション単独のICC
icc_position <- multilevel::ICC1(aov(Hit ~ position, data = dat))

print(paste("Team ICC1:", round(icc_team, 3)))
```

```
[1] "Team ICC1: -0.031"
```

```
print(paste("Team:Position ICC1:", round(icc_team_position, 3)))
```

```
[1] "Team:Position ICC1: -0.029"
```

```
print(paste("Position ICC1:", round(icc_position, 3)))
```

```
[1] "Position ICC1: 0.046"
```

ICC の値が負になっているので，群内のばらつきの方が群間のばらつきよりも大きいことがわかる。つまり群に分けた意味がないことを意味している。本来はある程度の正の大きさの ICC を持つことが望ましい[*4]。ただし，チーム × ポジションの組み合わせで群をつくると，群内の類似性が現れるため，これを群わけ変数として用いることにしよう。

従属変数を安打数 (Hit)，説明変数をゲーム出場回数 (Games) ならびに選手の体躯 (height と weight) とする。これらがチームの中でのポジションでネストされていて切片が異なるとしたモデルを構成する。なお安打数は正の整数しか取らないから，ポアソン分布を用いた。

```
# ネスト構造を考慮したポアソンモデル
# (1 | team) + (1 | team:position) でポジションをチーム内にネスト
model_hit <- brm(
  Hit ~ Games + height + weight + (1 | team:position),
  data = dat,
  family = poisson(),
  chains = 4,
  iter = 10000,
  cores = 4
)
```

```
Start sampling
```

```
Running MCMC with 4 parallel chains...

Chain 1 Iteration:    1 / 10000 [  0%]  (Warmup)
Chain 1 Iteration:  100 / 10000 [  1%]  (Warmup)
Chain 1 Iteration:  200 / 10000 [  2%]  (Warmup)
Chain 2 Iteration:    1 / 10000 [  0%]  (Warmup)
Chain 2 Iteration:  100 / 10000 [  1%]  (Warmup)
Chain 4 Iteration:    1 / 10000 [  0%]  (Warmup)
Chain 4 Iteration:  100 / 10000 [  1%]  (Warmup)
Chain 3 Iteration:    1 / 10000 [  0%]  (Warmup)
```

[*4] 正規分布の幅のパラメータは，テキストによっては分散 σ^2 で記載されていることが多いが，ここでは標準偏差 σ で記述する。Stan では標準偏差で記述するようになっているので，それに合わせたいからである。標準偏差を二乗したものが分散であるから，意味するところは本質的に変わりがない。

```
Chain 3 Iteration:  100 / 10000 [  1%]  (Warmup)
Chain 2 Iteration:  200 / 10000 [  2%]  (Warmup)
Chain 4 Iteration:  200 / 10000 [  2%]  (Warmup)
Chain 1 Iteration:  300 / 10000 [  3%]  (Warmup)
Chain 3 Iteration:  200 / 10000 [  2%]  (Warmup)
Chain 4 Iteration:  300 / 10000 [  3%]  (Warmup)
Chain 2 Iteration:  300 / 10000 [  3%]  (Warmup)
Chain 2 Iteration:  400 / 10000 [  4%]  (Warmup)
Chain 4 Iteration:  400 / 10000 [  4%]  (Warmup)
Chain 2 Iteration:  500 / 10000 [  5%]  (Warmup)
Chain 3 Iteration:  300 / 10000 [  3%]  (Warmup)
Chain 4 Iteration:  500 / 10000 [  5%]  (Warmup)
Chain 2 Iteration:  600 / 10000 [  6%]  (Warmup)
Chain 3 Iteration:  400 / 10000 [  4%]  (Warmup)
Chain 4 Iteration:  600 / 10000 [  6%]  (Warmup)
Chain 2 Iteration:  700 / 10000 [  7%]  (Warmup)
Chain 4 Iteration:  700 / 10000 [  7%]  (Warmup)
Chain 2 Iteration:  800 / 10000 [  8%]  (Warmup)
Chain 3 Iteration:  500 / 10000 [  5%]  (Warmup)
Chain 4 Iteration:  800 / 10000 [  8%]  (Warmup)
Chain 1 Iteration:  400 / 10000 [  4%]  (Warmup)
Chain 2 Iteration:  900 / 10000 [  9%]  (Warmup)
Chain 3 Iteration:  600 / 10000 [  6%]  (Warmup)
Chain 4 Iteration:  900 / 10000 [  9%]  (Warmup)
Chain 2 Iteration: 1000 / 10000 [ 10%]  (Warmup)
Chain 3 Iteration:  700 / 10000 [  7%]  (Warmup)
Chain 3 Iteration:  800 / 10000 [  8%]  (Warmup)
Chain 4 Iteration: 1000 / 10000 [ 10%]  (Warmup)
Chain 2 Iteration: 1100 / 10000 [ 11%]  (Warmup)
Chain 2 Iteration: 1200 / 10000 [ 12%]  (Warmup)
Chain 3 Iteration:  900 / 10000 [  9%]  (Warmup)
Chain 4 Iteration: 1100 / 10000 [ 11%]  (Warmup)
Chain 4 Iteration: 1200 / 10000 [ 12%]  (Warmup)
Chain 2 Iteration: 1300 / 10000 [ 13%]  (Warmup)
Chain 2 Iteration: 1400 / 10000 [ 14%]  (Warmup)
Chain 3 Iteration: 1000 / 10000 [ 10%]  (Warmup)
Chain 4 Iteration: 1300 / 10000 [ 13%]  (Warmup)
Chain 2 Iteration: 1500 / 10000 [ 15%]  (Warmup)
Chain 3 Iteration: 1100 / 10000 [ 11%]  (Warmup)
Chain 4 Iteration: 1400 / 10000 [ 14%]  (Warmup)
Chain 4 Iteration: 1500 / 10000 [ 15%]  (Warmup)
```

```
Chain 2 Iteration: 1600 / 10000 [ 16%]  (Warmup)
Chain 2 Iteration: 1700 / 10000 [ 17%]  (Warmup)
Chain 3 Iteration: 1200 / 10000 [ 12%]  (Warmup)
Chain 3 Iteration: 1300 / 10000 [ 13%]  (Warmup)
Chain 4 Iteration: 1600 / 10000 [ 16%]  (Warmup)
Chain 2 Iteration: 1800 / 10000 [ 18%]  (Warmup)
Chain 3 Iteration: 1400 / 10000 [ 14%]  (Warmup)
Chain 4 Iteration: 1700 / 10000 [ 17%]  (Warmup)
Chain 4 Iteration: 1800 / 10000 [ 18%]  (Warmup)
Chain 1 Iteration:  500 / 10000 [  5%]  (Warmup)
Chain 2 Iteration: 1900 / 10000 [ 19%]  (Warmup)
Chain 2 Iteration: 2000 / 10000 [ 20%]  (Warmup)
Chain 2 Iteration: 2100 / 10000 [ 21%]  (Warmup)
Chain 3 Iteration: 1500 / 10000 [ 15%]  (Warmup)
Chain 3 Iteration: 1600 / 10000 [ 16%]  (Warmup)
Chain 4 Iteration: 1900 / 10000 [ 19%]  (Warmup)
Chain 4 Iteration: 2000 / 10000 [ 20%]  (Warmup)
Chain 1 Iteration:  600 / 10000 [  6%]  (Warmup)
Chain 2 Iteration: 2200 / 10000 [ 22%]  (Warmup)
Chain 2 Iteration: 2300 / 10000 [ 23%]  (Warmup)
Chain 3 Iteration: 1700 / 10000 [ 17%]  (Warmup)
Chain 4 Iteration: 2100 / 10000 [ 21%]  (Warmup)
Chain 4 Iteration: 2200 / 10000 [ 22%]  (Warmup)
Chain 4 Iteration: 2300 / 10000 [ 23%]  (Warmup)
Chain 1 Iteration:  700 / 10000 [  7%]  (Warmup)
Chain 2 Iteration: 2400 / 10000 [ 24%]  (Warmup)
Chain 2 Iteration: 2500 / 10000 [ 25%]  (Warmup)
Chain 3 Iteration: 1800 / 10000 [ 18%]  (Warmup)
Chain 3 Iteration: 1900 / 10000 [ 19%]  (Warmup)
Chain 3 Iteration: 2000 / 10000 [ 20%]  (Warmup)
Chain 4 Iteration: 2400 / 10000 [ 24%]  (Warmup)
Chain 4 Iteration: 2500 / 10000 [ 25%]  (Warmup)
Chain 1 Iteration:  800 / 10000 [  8%]  (Warmup)
Chain 2 Iteration: 2600 / 10000 [ 26%]  (Warmup)
Chain 2 Iteration: 2700 / 10000 [ 27%]  (Warmup)
Chain 2 Iteration: 2800 / 10000 [ 28%]  (Warmup)
Chain 3 Iteration: 2100 / 10000 [ 21%]  (Warmup)
Chain 3 Iteration: 2200 / 10000 [ 22%]  (Warmup)
Chain 4 Iteration: 2600 / 10000 [ 26%]  (Warmup)
Chain 4 Iteration: 2700 / 10000 [ 27%]  (Warmup)
Chain 4 Iteration: 2800 / 10000 [ 28%]  (Warmup)
```

```
Chain 1 Iteration:  900 / 10000 [  9%]  (Warmup)
Chain 2 Iteration: 2900 / 10000 [ 29%]  (Warmup)
Chain 2 Iteration: 3000 / 10000 [ 30%]  (Warmup)
Chain 2 Iteration: 3100 / 10000 [ 31%]  (Warmup)
Chain 3 Iteration: 2300 / 10000 [ 23%]  (Warmup)
Chain 3 Iteration: 2400 / 10000 [ 24%]  (Warmup)
Chain 4 Iteration: 2900 / 10000 [ 29%]  (Warmup)
Chain 4 Iteration: 3000 / 10000 [ 30%]  (Warmup)
Chain 1 Iteration: 1000 / 10000 [ 10%]  (Warmup)
Chain 1 Iteration: 1100 / 10000 [ 11%]  (Warmup)
Chain 2 Iteration: 3200 / 10000 [ 32%]  (Warmup)
Chain 2 Iteration: 3300 / 10000 [ 33%]  (Warmup)
Chain 3 Iteration: 2500 / 10000 [ 25%]  (Warmup)
Chain 3 Iteration: 2600 / 10000 [ 26%]  (Warmup)
Chain 3 Iteration: 2700 / 10000 [ 27%]  (Warmup)
Chain 4 Iteration: 3100 / 10000 [ 31%]  (Warmup)
Chain 4 Iteration: 3200 / 10000 [ 32%]  (Warmup)
Chain 4 Iteration: 3300 / 10000 [ 33%]  (Warmup)
Chain 1 Iteration: 1200 / 10000 [ 12%]  (Warmup)
Chain 1 Iteration: 1300 / 10000 [ 13%]  (Warmup)
Chain 2 Iteration: 3400 / 10000 [ 34%]  (Warmup)
Chain 2 Iteration: 3500 / 10000 [ 35%]  (Warmup)
Chain 3 Iteration: 2800 / 10000 [ 28%]  (Warmup)
Chain 3 Iteration: 2900 / 10000 [ 29%]  (Warmup)
Chain 4 Iteration: 3400 / 10000 [ 34%]  (Warmup)
Chain 4 Iteration: 3500 / 10000 [ 35%]  (Warmup)
Chain 1 Iteration: 1400 / 10000 [ 14%]  (Warmup)
Chain 2 Iteration: 3600 / 10000 [ 36%]  (Warmup)
Chain 2 Iteration: 3700 / 10000 [ 37%]  (Warmup)
Chain 2 Iteration: 3800 / 10000 [ 38%]  (Warmup)
Chain 3 Iteration: 3000 / 10000 [ 30%]  (Warmup)
Chain 3 Iteration: 3100 / 10000 [ 31%]  (Warmup)
Chain 4 Iteration: 3600 / 10000 [ 36%]  (Warmup)
Chain 4 Iteration: 3700 / 10000 [ 37%]  (Warmup)
Chain 1 Iteration: 1500 / 10000 [ 15%]  (Warmup)
Chain 1 Iteration: 1600 / 10000 [ 16%]  (Warmup)
Chain 2 Iteration: 3900 / 10000 [ 39%]  (Warmup)
Chain 2 Iteration: 4000 / 10000 [ 40%]  (Warmup)
Chain 3 Iteration: 3200 / 10000 [ 32%]  (Warmup)
Chain 3 Iteration: 3300 / 10000 [ 33%]  (Warmup)
Chain 3 Iteration: 3400 / 10000 [ 34%]  (Warmup)
```

```
Chain 4 Iteration: 3800 / 10000 [ 38%]  (Warmup)
Chain 4 Iteration: 3900 / 10000 [ 39%]  (Warmup)
Chain 4 Iteration: 4000 / 10000 [ 40%]  (Warmup)
Chain 1 Iteration: 1700 / 10000 [ 17%]  (Warmup)
Chain 2 Iteration: 4100 / 10000 [ 41%]  (Warmup)
Chain 2 Iteration: 4200 / 10000 [ 42%]  (Warmup)
Chain 2 Iteration: 4300 / 10000 [ 43%]  (Warmup)
Chain 3 Iteration: 3500 / 10000 [ 35%]  (Warmup)
Chain 3 Iteration: 3600 / 10000 [ 36%]  (Warmup)
Chain 4 Iteration: 4100 / 10000 [ 41%]  (Warmup)
Chain 4 Iteration: 4200 / 10000 [ 42%]  (Warmup)
Chain 1 Iteration: 1800 / 10000 [ 18%]  (Warmup)
Chain 1 Iteration: 1900 / 10000 [ 19%]  (Warmup)
Chain 2 Iteration: 4400 / 10000 [ 44%]  (Warmup)
Chain 2 Iteration: 4500 / 10000 [ 45%]  (Warmup)
Chain 2 Iteration: 4600 / 10000 [ 46%]  (Warmup)
Chain 3 Iteration: 3700 / 10000 [ 37%]  (Warmup)
Chain 3 Iteration: 3800 / 10000 [ 38%]  (Warmup)
Chain 3 Iteration: 3900 / 10000 [ 39%]  (Warmup)
Chain 4 Iteration: 4300 / 10000 [ 43%]  (Warmup)
Chain 4 Iteration: 4400 / 10000 [ 44%]  (Warmup)
Chain 4 Iteration: 4500 / 10000 [ 45%]  (Warmup)
Chain 1 Iteration: 2000 / 10000 [ 20%]  (Warmup)
Chain 1 Iteration: 2100 / 10000 [ 21%]  (Warmup)
Chain 1 Iteration: 2200 / 10000 [ 22%]  (Warmup)
Chain 2 Iteration: 4700 / 10000 [ 47%]  (Warmup)
Chain 2 Iteration: 4800 / 10000 [ 48%]  (Warmup)
Chain 3 Iteration: 4000 / 10000 [ 40%]  (Warmup)
Chain 3 Iteration: 4100 / 10000 [ 41%]  (Warmup)
Chain 4 Iteration: 4600 / 10000 [ 46%]  (Warmup)
Chain 4 Iteration: 4700 / 10000 [ 47%]  (Warmup)
Chain 1 Iteration: 2300 / 10000 [ 23%]  (Warmup)
Chain 1 Iteration: 2400 / 10000 [ 24%]  (Warmup)
Chain 2 Iteration: 4900 / 10000 [ 49%]  (Warmup)
Chain 2 Iteration: 5000 / 10000 [ 50%]  (Warmup)
Chain 2 Iteration: 5001 / 10000 [ 50%]  (Sampling)
Chain 3 Iteration: 4200 / 10000 [ 42%]  (Warmup)
Chain 3 Iteration: 4300 / 10000 [ 43%]  (Warmup)
Chain 3 Iteration: 4400 / 10000 [ 44%]  (Warmup)
Chain 4 Iteration: 4800 / 10000 [ 48%]  (Warmup)
Chain 4 Iteration: 4900 / 10000 [ 49%]  (Warmup)
```

```
Chain 4 Iteration: 5000 / 10000 [ 50%]  (Warmup)
Chain 4 Iteration: 5001 / 10000 [ 50%]  (Sampling)
Chain 1 Iteration: 2500 / 10000 [ 25%]  (Warmup)
Chain 1 Iteration: 2600 / 10000 [ 26%]  (Warmup)
Chain 2 Iteration: 5100 / 10000 [ 51%]  (Sampling)
Chain 2 Iteration: 5200 / 10000 [ 52%]  (Sampling)
Chain 2 Iteration: 5300 / 10000 [ 53%]  (Sampling)
Chain 3 Iteration: 4500 / 10000 [ 45%]  (Warmup)
Chain 3 Iteration: 4600 / 10000 [ 46%]  (Warmup)
Chain 4 Iteration: 5100 / 10000 [ 51%]  (Sampling)
Chain 4 Iteration: 5200 / 10000 [ 52%]  (Sampling)
Chain 4 Iteration: 5300 / 10000 [ 53%]  (Sampling)
Chain 1 Iteration: 2700 / 10000 [ 27%]  (Warmup)
Chain 1 Iteration: 2800 / 10000 [ 28%]  (Warmup)
Chain 1 Iteration: 2900 / 10000 [ 29%]  (Warmup)
Chain 2 Iteration: 5400 / 10000 [ 54%]  (Sampling)
Chain 2 Iteration: 5500 / 10000 [ 55%]  (Sampling)
Chain 2 Iteration: 5600 / 10000 [ 56%]  (Sampling)
Chain 3 Iteration: 4700 / 10000 [ 47%]  (Warmup)
Chain 3 Iteration: 4800 / 10000 [ 48%]  (Warmup)
Chain 3 Iteration: 4900 / 10000 [ 49%]  (Warmup)
Chain 4 Iteration: 5400 / 10000 [ 54%]  (Sampling)
Chain 4 Iteration: 5500 / 10000 [ 55%]  (Sampling)
Chain 1 Iteration: 3000 / 10000 [ 30%]  (Warmup)
Chain 1 Iteration: 3100 / 10000 [ 31%]  (Warmup)
Chain 2 Iteration: 5700 / 10000 [ 57%]  (Sampling)
Chain 2 Iteration: 5800 / 10000 [ 58%]  (Sampling)
Chain 3 Iteration: 5000 / 10000 [ 50%]  (Warmup)
Chain 3 Iteration: 5001 / 10000 [ 50%]  (Sampling)
Chain 3 Iteration: 5100 / 10000 [ 51%]  (Sampling)
Chain 4 Iteration: 5600 / 10000 [ 56%]  (Sampling)
Chain 4 Iteration: 5700 / 10000 [ 57%]  (Sampling)
Chain 4 Iteration: 5800 / 10000 [ 58%]  (Sampling)
Chain 1 Iteration: 3200 / 10000 [ 32%]  (Warmup)
Chain 1 Iteration: 3300 / 10000 [ 33%]  (Warmup)
Chain 1 Iteration: 3400 / 10000 [ 34%]  (Warmup)
Chain 2 Iteration: 5900 / 10000 [ 59%]  (Sampling)
Chain 2 Iteration: 6000 / 10000 [ 60%]  (Sampling)
Chain 2 Iteration: 6100 / 10000 [ 61%]  (Sampling)
Chain 3 Iteration: 5200 / 10000 [ 52%]  (Sampling)
Chain 3 Iteration: 5300 / 10000 [ 53%]  (Sampling)
```

```
Chain 3 Iteration: 5400 / 10000 [ 54%]  (Sampling)
Chain 4 Iteration: 5900 / 10000 [ 59%]  (Sampling)
Chain 4 Iteration: 6000 / 10000 [ 60%]  (Sampling)
Chain 4 Iteration: 6100 / 10000 [ 61%]  (Sampling)
Chain 1 Iteration: 3500 / 10000 [ 35%]  (Warmup)
Chain 1 Iteration: 3600 / 10000 [ 36%]  (Warmup)
Chain 2 Iteration: 6200 / 10000 [ 62%]  (Sampling)
Chain 2 Iteration: 6300 / 10000 [ 63%]  (Sampling)
Chain 3 Iteration: 5500 / 10000 [ 55%]  (Sampling)
Chain 3 Iteration: 5600 / 10000 [ 56%]  (Sampling)
Chain 4 Iteration: 6200 / 10000 [ 62%]  (Sampling)
Chain 4 Iteration: 6300 / 10000 [ 63%]  (Sampling)
Chain 1 Iteration: 3700 / 10000 [ 37%]  (Warmup)
Chain 1 Iteration: 3800 / 10000 [ 38%]  (Warmup)
Chain 1 Iteration: 3900 / 10000 [ 39%]  (Warmup)
Chain 2 Iteration: 6400 / 10000 [ 64%]  (Sampling)
Chain 2 Iteration: 6500 / 10000 [ 65%]  (Sampling)
Chain 2 Iteration: 6600 / 10000 [ 66%]  (Sampling)
Chain 3 Iteration: 5700 / 10000 [ 57%]  (Sampling)
Chain 3 Iteration: 5800 / 10000 [ 58%]  (Sampling)
Chain 3 Iteration: 5900 / 10000 [ 59%]  (Sampling)
Chain 4 Iteration: 6400 / 10000 [ 64%]  (Sampling)
Chain 4 Iteration: 6500 / 10000 [ 65%]  (Sampling)
Chain 4 Iteration: 6600 / 10000 [ 66%]  (Sampling)
Chain 1 Iteration: 4000 / 10000 [ 40%]  (Warmup)
Chain 1 Iteration: 4100 / 10000 [ 41%]  (Warmup)
Chain 2 Iteration: 6700 / 10000 [ 67%]  (Sampling)
Chain 2 Iteration: 6800 / 10000 [ 68%]  (Sampling)
Chain 2 Iteration: 6900 / 10000 [ 69%]  (Sampling)
Chain 3 Iteration: 6000 / 10000 [ 60%]  (Sampling)
Chain 3 Iteration: 6100 / 10000 [ 61%]  (Sampling)
Chain 3 Iteration: 6200 / 10000 [ 62%]  (Sampling)
Chain 4 Iteration: 6700 / 10000 [ 67%]  (Sampling)
Chain 4 Iteration: 6800 / 10000 [ 68%]  (Sampling)
Chain 4 Iteration: 6900 / 10000 [ 69%]  (Sampling)
Chain 1 Iteration: 4200 / 10000 [ 42%]  (Warmup)
Chain 1 Iteration: 4300 / 10000 [ 43%]  (Warmup)
Chain 1 Iteration: 4400 / 10000 [ 44%]  (Warmup)
Chain 2 Iteration: 7000 / 10000 [ 70%]  (Sampling)
Chain 2 Iteration: 7100 / 10000 [ 71%]  (Sampling)
Chain 3 Iteration: 6300 / 10000 [ 63%]  (Sampling)
```

```
Chain 3 Iteration: 6400 / 10000 [ 64%]  (Sampling)
Chain 4 Iteration: 7000 / 10000 [ 70%]  (Sampling)
Chain 4 Iteration: 7100 / 10000 [ 71%]  (Sampling)
Chain 1 Iteration: 4500 / 10000 [ 45%]  (Warmup)
Chain 1 Iteration: 4600 / 10000 [ 46%]  (Warmup)
Chain 2 Iteration: 7200 / 10000 [ 72%]  (Sampling)
Chain 2 Iteration: 7300 / 10000 [ 73%]  (Sampling)
Chain 2 Iteration: 7400 / 10000 [ 74%]  (Sampling)
Chain 3 Iteration: 6500 / 10000 [ 65%]  (Sampling)
Chain 3 Iteration: 6600 / 10000 [ 66%]  (Sampling)
Chain 3 Iteration: 6700 / 10000 [ 67%]  (Sampling)
Chain 4 Iteration: 7200 / 10000 [ 72%]  (Sampling)
Chain 4 Iteration: 7300 / 10000 [ 73%]  (Sampling)
Chain 4 Iteration: 7400 / 10000 [ 74%]  (Sampling)
Chain 1 Iteration: 4700 / 10000 [ 47%]  (Warmup)
Chain 1 Iteration: 4800 / 10000 [ 48%]  (Warmup)
Chain 1 Iteration: 4900 / 10000 [ 49%]  (Warmup)
Chain 2 Iteration: 7500 / 10000 [ 75%]  (Sampling)
Chain 2 Iteration: 7600 / 10000 [ 76%]  (Sampling)
Chain 2 Iteration: 7700 / 10000 [ 77%]  (Sampling)
Chain 3 Iteration: 6800 / 10000 [ 68%]  (Sampling)
Chain 3 Iteration: 6900 / 10000 [ 69%]  (Sampling)
Chain 3 Iteration: 7000 / 10000 [ 70%]  (Sampling)
Chain 4 Iteration: 7500 / 10000 [ 75%]  (Sampling)
Chain 4 Iteration: 7600 / 10000 [ 76%]  (Sampling)
Chain 1 Iteration: 5000 / 10000 [ 50%]  (Warmup)
Chain 1 Iteration: 5001 / 10000 [ 50%]  (Sampling)
Chain 1 Iteration: 5100 / 10000 [ 51%]  (Sampling)
Chain 2 Iteration: 7800 / 10000 [ 78%]  (Sampling)
Chain 2 Iteration: 7900 / 10000 [ 79%]  (Sampling)
Chain 3 Iteration: 7100 / 10000 [ 71%]  (Sampling)
Chain 3 Iteration: 7200 / 10000 [ 72%]  (Sampling)
Chain 4 Iteration: 7700 / 10000 [ 77%]  (Sampling)
Chain 4 Iteration: 7800 / 10000 [ 78%]  (Sampling)
Chain 4 Iteration: 7900 / 10000 [ 79%]  (Sampling)
Chain 1 Iteration: 5200 / 10000 [ 52%]  (Sampling)
Chain 1 Iteration: 5300 / 10000 [ 53%]  (Sampling)
Chain 1 Iteration: 5400 / 10000 [ 54%]  (Sampling)
Chain 2 Iteration: 8000 / 10000 [ 80%]  (Sampling)
Chain 2 Iteration: 8100 / 10000 [ 81%]  (Sampling)
Chain 2 Iteration: 8200 / 10000 [ 82%]  (Sampling)
```

```
Chain 3 Iteration: 7300 / 10000 [ 73%]  (Sampling)
Chain 3 Iteration: 7400 / 10000 [ 74%]  (Sampling)
Chain 3 Iteration: 7500 / 10000 [ 75%]  (Sampling)
Chain 4 Iteration: 8000 / 10000 [ 80%]  (Sampling)
Chain 4 Iteration: 8100 / 10000 [ 81%]  (Sampling)
Chain 1 Iteration: 5500 / 10000 [ 55%]  (Sampling)
Chain 1 Iteration: 5600 / 10000 [ 56%]  (Sampling)
Chain 1 Iteration: 5700 / 10000 [ 57%]  (Sampling)
Chain 2 Iteration: 8300 / 10000 [ 83%]  (Sampling)
Chain 2 Iteration: 8400 / 10000 [ 84%]  (Sampling)
Chain 3 Iteration: 7600 / 10000 [ 76%]  (Sampling)
Chain 3 Iteration: 7700 / 10000 [ 77%]  (Sampling)
Chain 4 Iteration: 8200 / 10000 [ 82%]  (Sampling)
Chain 4 Iteration: 8300 / 10000 [ 83%]  (Sampling)
Chain 4 Iteration: 8400 / 10000 [ 84%]  (Sampling)
Chain 1 Iteration: 5800 / 10000 [ 58%]  (Sampling)
Chain 1 Iteration: 5900 / 10000 [ 59%]  (Sampling)
Chain 2 Iteration: 8500 / 10000 [ 85%]  (Sampling)
Chain 2 Iteration: 8600 / 10000 [ 86%]  (Sampling)
Chain 3 Iteration: 7800 / 10000 [ 78%]  (Sampling)
Chain 3 Iteration: 7900 / 10000 [ 79%]  (Sampling)
Chain 3 Iteration: 8000 / 10000 [ 80%]  (Sampling)
Chain 4 Iteration: 8500 / 10000 [ 85%]  (Sampling)
Chain 4 Iteration: 8600 / 10000 [ 86%]  (Sampling)
Chain 1 Iteration: 6000 / 10000 [ 60%]  (Sampling)
Chain 1 Iteration: 6100 / 10000 [ 61%]  (Sampling)
Chain 1 Iteration: 6200 / 10000 [ 62%]  (Sampling)
Chain 2 Iteration: 8700 / 10000 [ 87%]  (Sampling)
Chain 2 Iteration: 8800 / 10000 [ 88%]  (Sampling)
Chain 2 Iteration: 8900 / 10000 [ 89%]  (Sampling)
Chain 3 Iteration: 8100 / 10000 [ 81%]  (Sampling)
Chain 3 Iteration: 8200 / 10000 [ 82%]  (Sampling)
Chain 4 Iteration: 8700 / 10000 [ 87%]  (Sampling)
Chain 4 Iteration: 8800 / 10000 [ 88%]  (Sampling)
Chain 4 Iteration: 8900 / 10000 [ 89%]  (Sampling)
Chain 1 Iteration: 6300 / 10000 [ 63%]  (Sampling)
Chain 1 Iteration: 6400 / 10000 [ 64%]  (Sampling)
Chain 2 Iteration: 9000 / 10000 [ 90%]  (Sampling)
Chain 2 Iteration: 9100 / 10000 [ 91%]  (Sampling)
Chain 2 Iteration: 9200 / 10000 [ 92%]  (Sampling)
Chain 3 Iteration: 8300 / 10000 [ 83%]  (Sampling)
```

```
Chain 3 Iteration: 8400 / 10000 [ 84%]  (Sampling)
Chain 3 Iteration: 8500 / 10000 [ 85%]  (Sampling)
Chain 4 Iteration: 9000 / 10000 [ 90%]  (Sampling)
Chain 4 Iteration: 9100 / 10000 [ 91%]  (Sampling)
Chain 4 Iteration: 9200 / 10000 [ 92%]  (Sampling)
Chain 1 Iteration: 6500 / 10000 [ 65%]  (Sampling)
Chain 1 Iteration: 6600 / 10000 [ 66%]  (Sampling)
Chain 1 Iteration: 6700 / 10000 [ 67%]  (Sampling)
Chain 2 Iteration: 9300 / 10000 [ 93%]  (Sampling)
Chain 2 Iteration: 9400 / 10000 [ 94%]  (Sampling)
Chain 2 Iteration: 9500 / 10000 [ 95%]  (Sampling)
Chain 3 Iteration: 8600 / 10000 [ 86%]  (Sampling)
Chain 3 Iteration: 8700 / 10000 [ 87%]  (Sampling)
Chain 3 Iteration: 8800 / 10000 [ 88%]  (Sampling)
Chain 4 Iteration: 9300 / 10000 [ 93%]  (Sampling)
Chain 4 Iteration: 9400 / 10000 [ 94%]  (Sampling)
Chain 4 Iteration: 9500 / 10000 [ 95%]  (Sampling)
Chain 1 Iteration: 6800 / 10000 [ 68%]  (Sampling)
Chain 1 Iteration: 6900 / 10000 [ 69%]  (Sampling)
Chain 1 Iteration: 7000 / 10000 [ 70%]  (Sampling)
Chain 2 Iteration: 9600 / 10000 [ 96%]  (Sampling)
Chain 2 Iteration: 9700 / 10000 [ 97%]  (Sampling)
Chain 3 Iteration: 8900 / 10000 [ 89%]  (Sampling)
Chain 3 Iteration: 9000 / 10000 [ 90%]  (Sampling)
Chain 3 Iteration: 9100 / 10000 [ 91%]  (Sampling)
Chain 4 Iteration: 9600 / 10000 [ 96%]  (Sampling)
Chain 4 Iteration: 9700 / 10000 [ 97%]  (Sampling)
Chain 1 Iteration: 7100 / 10000 [ 71%]  (Sampling)
Chain 1 Iteration: 7200 / 10000 [ 72%]  (Sampling)
Chain 2 Iteration: 9800 / 10000 [ 98%]  (Sampling)
Chain 2 Iteration: 9900 / 10000 [ 99%]  (Sampling)
Chain 2 Iteration: 10000 / 10000 [100%]  (Sampling)
Chain 3 Iteration: 9200 / 10000 [ 92%]  (Sampling)
Chain 3 Iteration: 9300 / 10000 [ 93%]  (Sampling)
Chain 4 Iteration: 9800 / 10000 [ 98%]  (Sampling)
Chain 4 Iteration: 9900 / 10000 [ 99%]  (Sampling)
Chain 4 Iteration: 10000 / 10000 [100%]  (Sampling)
Chain 2 finished in 6.2 seconds.
Chain 4 finished in 6.1 seconds.
Chain 1 Iteration: 7300 / 10000 [ 73%]  (Sampling)
Chain 1 Iteration: 7400 / 10000 [ 74%]  (Sampling)
```

```
Chain 1 Iteration: 7500 / 10000 [ 75%]  (Sampling)
Chain 3 Iteration: 9400 / 10000 [ 94%]  (Sampling)
Chain 3 Iteration: 9500 / 10000 [ 95%]  (Sampling)
Chain 3 Iteration: 9600 / 10000 [ 96%]  (Sampling)
Chain 1 Iteration: 7600 / 10000 [ 76%]  (Sampling)
Chain 1 Iteration: 7700 / 10000 [ 77%]  (Sampling)
Chain 1 Iteration: 7800 / 10000 [ 78%]  (Sampling)
Chain 3 Iteration: 9700 / 10000 [ 97%]  (Sampling)
Chain 3 Iteration: 9800 / 10000 [ 98%]  (Sampling)
Chain 3 Iteration: 9900 / 10000 [ 99%]  (Sampling)
Chain 3 Iteration: 10000 / 10000 [100%]  (Sampling)
Chain 3 finished in 6.1 seconds.
Chain 1 Iteration: 7900 / 10000 [ 79%]  (Sampling)
Chain 1 Iteration: 8000 / 10000 [ 80%]  (Sampling)
Chain 1 Iteration: 8100 / 10000 [ 81%]  (Sampling)
Chain 1 Iteration: 8200 / 10000 [ 82%]  (Sampling)
Chain 1 Iteration: 8300 / 10000 [ 83%]  (Sampling)
Chain 1 Iteration: 8400 / 10000 [ 84%]  (Sampling)
Chain 1 Iteration: 8500 / 10000 [ 85%]  (Sampling)
Chain 1 Iteration: 8600 / 10000 [ 86%]  (Sampling)
Chain 1 Iteration: 8700 / 10000 [ 87%]  (Sampling)
Chain 1 Iteration: 8800 / 10000 [ 88%]  (Sampling)
Chain 1 Iteration: 8900 / 10000 [ 89%]  (Sampling)
Chain 1 Iteration: 9000 / 10000 [ 90%]  (Sampling)
Chain 1 Iteration: 9100 / 10000 [ 91%]  (Sampling)
Chain 1 Iteration: 9200 / 10000 [ 92%]  (Sampling)
Chain 1 Iteration: 9300 / 10000 [ 93%]  (Sampling)
Chain 1 Iteration: 9400 / 10000 [ 94%]  (Sampling)
Chain 1 Iteration: 9500 / 10000 [ 95%]  (Sampling)
Chain 1 Iteration: 9600 / 10000 [ 96%]  (Sampling)
Chain 1 Iteration: 9700 / 10000 [ 97%]  (Sampling)
Chain 1 Iteration: 9800 / 10000 [ 98%]  (Sampling)
Chain 1 Iteration: 9900 / 10000 [ 99%]  (Sampling)
Chain 1 Iteration: 10000 / 10000 [100%]  (Sampling)
Chain 1 finished in 7.2 seconds.

All 4 chains finished successfully.
Mean chain execution time: 6.4 seconds.
Total execution time: 7.2 seconds.
```

```
# Model summary for Hit model
summary(model_hit)
```

```
 Family: poisson
  Links: mu = log
Formula: Hit ~ Games + height + weight + (1 | team:position)
   Data: dat (Number of observations: 329)
  Draws: 4 chains, each with iter = 10000; warmup = 5000; thin = 1;
         total post-warmup draws = 20000

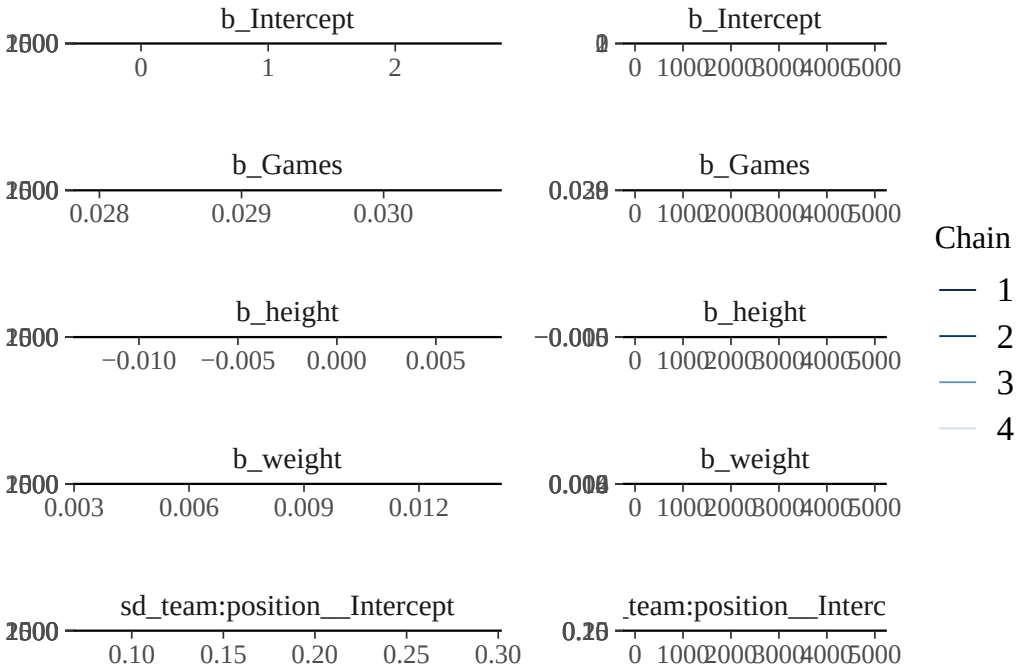
Multilevel Hyperparameters:
~team:position (Number of levels: 36)
              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(Intercept)     0.14      0.02     0.11     0.19 1.00     5955     8908

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     1.10      0.38     0.37     1.85 1.00    26157    17243
Games         0.03      0.00     0.03     0.03 1.00    23745    16787
height       -0.00      0.00    -0.01     0.00 1.00    26603    16077
weight        0.01      0.00     0.01     0.01 1.00    26479    17223

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

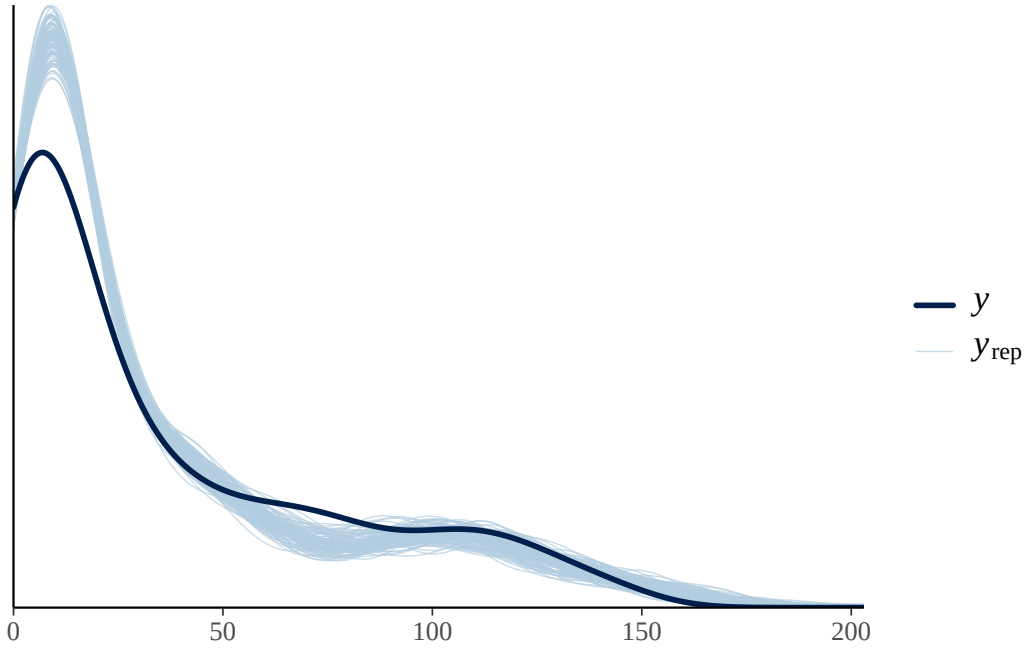
MCMC のチェックのプロットを見ておく。Rhat などの数値も良好である。

```
plot(model_hit)
```



モデルがデータにあっているかどうかを確認する一つの方法が，**事後予測分布**による可視化である。モデルから生成されるデータの分布を実際のデータに重ねてみて，同じような分布が得られているかを見ておく。`bayesplot` パッケージの `pp_check` 関数は，事後予測分布の一部とデータとの重なり具合を可視化してくれる。これを見ると，安打数がごく少ないあたりの分布はうまく適合していない。モデルとして改良の余地があるところだろう。

```
# Posterior predictive check for Hit model
pp_check(model_hit, ndraws = 100)
```

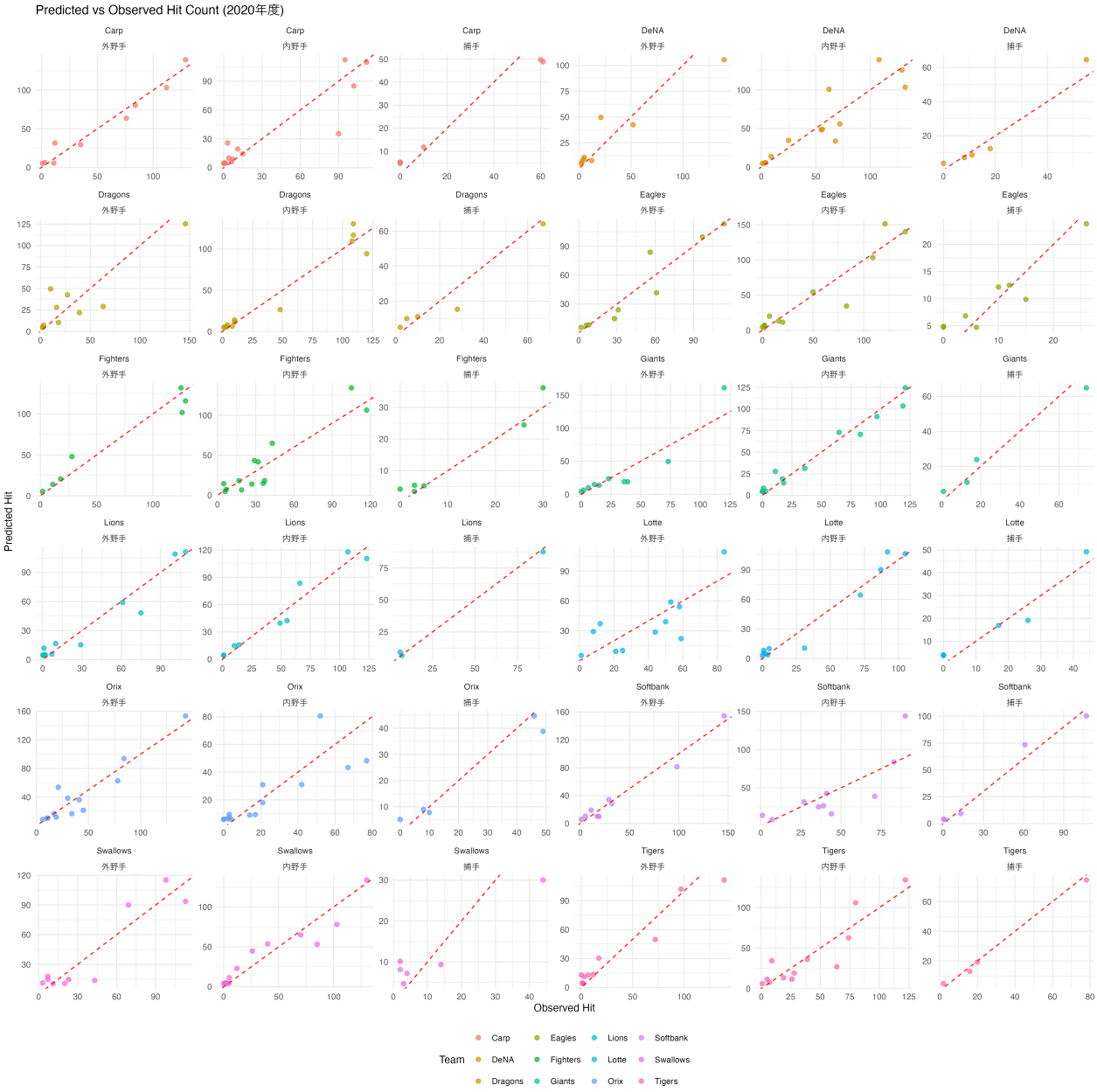


最後に，グループごとのプロットを見てみよう。チーム差，ポジションの違いといった細かい調整により，よりうまくデータに適合している様子が見てとれるだろう。

```r
predicted_hit <- fitted(model_hit,
  newdata = dat,
  allow_new_levels = TRUE
) %>% as.data.frame()


plot_data <- data.frame(
  observed = dat$Hit,
  predicted = predicted_hit$Estimate,
  team = dat$team,
  position = dat$position
)

ggplot(plot_data, aes(x = observed, y = predicted, color = team)) +
  geom_point(alpha = 0.7, size = 2) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(
    x = "Observed Hit", y = "Predicted Hit",
    title = "Predicted vs Observed Hit Count (2020年度)",
    color = "Team"
  ) +
  facet_wrap(team ~ position, scales = "free") +
  theme_minimal() +
  theme(legend.position = "bottom")
```



14.5　課題

14.5.1　基本問題：パラメータリカバリ

brms パッケージを使用してベイズ統計モデリングにおけるパラメータリカバリを実践してください。パラメータリカバリとは、既知のパラメータで生成したデータから、そのパラメータを正しく推定できるかを確認する手法のことです。

14.5.1.1 課題 1-1: ロジスティック回帰のパラメータリカバリ

```
# ロジスティック回帰のパラメータリカバリ
# Step 1: 既知のパラメータでデータを生成
set.seed(123)
n <- 200
true_intercept <- 0.5
true_slope <- 1.2
x <- rnorm(n, mean = 0, sd = 1)
p <- plogis(true_intercept + true_slope * x) # logistic function
y <- rbinom(n, size = 1, prob = p)

# データフレーム作成
df_logistic <- data.frame(x = x, y = y)

# Step 2: brmsでベイズ推定
model_logistic <- brm(
  y ~ x,
  family = bernoulli(),
  data = df_logistic,
  prior = c(
    prior(normal(0, 2.5), class = Intercept),
    prior(normal(0, 2.5), class = b)
  ),
  chains = 4,
  iter = 2000,
  cores = 4,
  refresh = 0
)

# Step 3: 結果の確認
summary(model_logistic)
plot(model_logistic)

# Step 4: パラメータリカバリの評価
# 真の値との比較
posterior_summary(model_logistic)
```

課題: 上記のコードを実行し、真の切片 (0.5) と傾き (1.2) が適切に推定されているかを確認してください。また、事後分布の 95% 信頼区間に真の値が含まれているかを確認してください。

14.5.1.2　課題 1-2: ポアソン回帰のパラメータリカバリ

```
# 課題1-2: ポアソン回帰のパラメータリカバリ
set.seed(456)
n <- 300
true_intercept <- 1.0
true_slope <- 0.8
x <- runif(n, min = 0, max = 3)
lambda <- exp(true_intercept + true_slope * x)
y <- rpois(n, lambda = lambda)

df_poisson <- data.frame(x = x, y = y)

# ベイズ推定
model_poisson <- brm(
  y ~ x,
  family = poisson(),
  data = df_poisson,
  prior = c(
    prior(normal(0, 2.5), class = Intercept),
    prior(normal(0, 2.5), class = b)
  ),
  chains = 4,
  iter = 2000,
  cores = 4,
  refresh = 0
)

summary(model_poisson)
plot(model_poisson)
```

課題: 真の切片 (1.0) と傾き (0.8) が適切に推定されているかを確認してください。

14.5.2　応用問題：野球データの階層モデリング

野球データ Baseball.csv を用いて、Year、Team でネストされた階層構造を持つモデルを構築します。2018 年度から 2020 年度の 3 年間のデータを使用し、投手のデータに限定します。変数には年俸 (salary)、身長 (height)、体重 (weight) のほか、投手の成績として勝利数 (Win)、負け数 (Lose)、ホールドポイント (Hold)、セーブポイント (Save) があります。これらの変数を用いて、年度・チームの階層効果を考慮したモデリングを行います。

```
dat <- read_csv("Baseball.csv") %>%
  filter(Year %in% c("2018年度", "2019年度", "2020年度")) %>%
  filter(position == "投手") %>%
  filter(!is.na(Win), !is.na(Lose), !is.na(Games), !is.na(salary), !is.na(height), !is.na(weight)) %>%
  mutate(
    Year = as.factor(Year),
    team = as.factor(team)
  )
```

14.5.2.1 モデル A: ポアソン回帰モデル

次のモデルを実行してください。

```
model1 <- brm(
  Win ~ height + weight + Games + (1 | team) + (1 | Year),
  family = poisson(),
  data = dat,
  iter = 2000,
  cores = 4
)
```

課題 1: このモデルの構造を説明してください。以下の点について述べてください：

- 従属変数と確率分布
- 固定効果 (説明変数) の意味
- 変量効果 (階層構造) の意味
- なぜこの確率分布を選んだのか

課題 2: モデルの結果を解釈してください。以下の観点が含まれるといいでしょう。：

- 各固定効果の係数の意味と統計的有意性
- 変量効果の分散の大きさとその解釈

課題 3: 以下の可視化を行ってください：

- `plot(model1)` で MCMC の収束診断
- `pp_check(model1)` で事後予測チェック
- チーム別・年度別の予測値と実測値の比較プロット

14.5.2.2 モデル B: 対数正規モデル

次のモデルを実行してください。

```
model2 <- brm(
  log(salary) ~ Win + Lose + (1 + Games | team) + (1 | Year),
```

```
  family = gaussian(),
  data = dat,
  iter = 2000,
  cores = 4
)
```

課題 1: このモデルの構造を説明してください。以下の点について述べてください：

- 従属変数の対数変換の理由
- 固定効果 (Win, Lose) の経済的意味
- 変量効果の構造 (ランダム切片・ランダム傾き) の意味

課題 2: モデルの結果を解釈してください。以下の観点が含まれるといいでしょう。：

- 勝利数・敗戦数が年俸に与える影響（% 変化として）
- チーム間の年俸格差と Games 効果の違い
- 年度効果の大きさと経済的解釈

課題 3: 以下の可視化を行ってください：

- `plot(model2)` で MCMC の収束診断
- チーム別の Games 効果の違いを可視化
- 実際の年俸と予測年俸の散布図

第 15 章

多変量解析 (その 1)

ここでは心理系で最もよく使われる分析法のひとつ，因子分析を中心に多変量解析の全体的な解説を行う。

15.1　多変量解析の包括的な説明

多変量解析はその名のとおり，変数が多く含まれるデータの解析であり，その目的は情報の要約にある。多くの変数が含まれる時にひとつ一つの変数を解釈していくのは大変な労力であるから，要領よくまとめることができればそれに越したことはないからである。この目的から派生して，多変量解析にはさまざまな意味解釈が付随する。以下にモデルの解釈と対応する多変量解析技術を列記してみた。

- 要約する＝すべての情報を使わず，いくつかの情報を捨象することでもある。
- 1 つの合成変数にまとめ上げる：総合評価として一次元化することでもある (主成分分析)。
- 2 つ 3 つなど少数の次元に写像する：多次元空間に可視化することでもある (多次元尺度法)。
- 少数のグループに変数を分類する：グループごとの特徴を記述したり分析することで解釈を深める (クラスター分析)。
- 観測変数の背後にある少数の潜在変数にまとめ上げる：構成概念を測定して数値を割り振る (因子分析)。
- 観測変数が 0/1 のバイナリデータに対して 1 つの潜在変数からの影響を考える：正答誤答を意味するテストと考える (項目反応理論)
- 観測された回答パターンから潜在変数を仮定したグループ分類を行う：隠れた購買層の発見などに使われる (潜在クラス分析)
- 潜在変数間の構造的な関係もモデル化する：回帰分析や因子分析などの線型関数関係をデータ全体に当てはめて考察する (構造方程式モデリング)
- 潜在変数を仮定せず，変数同士の結びつきの強さを可視化する：ノード node とタイ tie で位相的な関係を描画したり，数学的構造を含めたモデリングを行ったりする (ネットワーク分析)

これらのモデルに共通する本質的な特徴は「変数間関係をデータから見出すこと」である。変数間関係をどのようなもので表現するかによって，多少モデルの扱い方は変化する。一般的には分散共分散 (相関係数) を変数間関係とするが，この場合は間隔尺度水準以上のデータが得られていることが必要になる。もし順序尺度水準でしか得られていないのであれば，ピアソンの相関係数の代わりにポリコリック相関係数やポリシリアル相関係数と呼ばれる相関係数を用いることになる。0/1 のバイナリデータの場合はさらに特殊で，ピアソンの相関係数の代わりに

テトラコリック相関係数を用いることになる。

変数間関係は必ずしも相関係数だけではない。カテゴリカル変数の場合は，あるカテゴリと他のカテゴリが同時に選択される・発生する頻度 (共頻関係) を，その変数間関係の指標として捉えることができる。共頻関係を分析する手法としては双対尺度法 (西里，2010) (対応分析や数量化 III 類と原理的に同じ) などが知られている。このようなカテゴリデータの分析は，自由記述などの自然言語を形態素解析し，多変量解析で分析するテキストマイニングなどで応用されている。

また，変数間関係を変数同士の類似度，すなわち距離であると考えることもできる。距離の公理を満たすデータがあれば，それを元に多次元尺度構成法 (岡太・今泉，1994; 高根，1980) で可視化したり，クラスター分析で分類 (足立，2006; 新納，2007) したりすることができる。

さらに，相関係数は変数間の直線的な関係の強さを意味するが，隠れた変数による擬似相関の可能性も含まれるため，偏相関係数を使って周囲の変数からの影響を統制した変数間関係を考えることもある。この手法はグラフィカルモデリング (宮川，1997) やネットワーク分析 (アデラ＝マリア他，2024) で用いられるものである。

いずれにせよ，こうした変数間関係をもとに，外的な基準があればそこに対するフィッティングを目的として未知数を推定するし，外的な基準がなければモデル的仮定に基づいてデータから構造化していくことになる。またその目的の基本は情報の要約であるが，可視化に重点をおいたモデルや潜在得点の推定に重点を置いたモデルなど，各種モデルによって得意とする場所や理論的に強調されるところは異なる。

やや本筋から外れるが，こうした多変量を同時に扱うための数学的基盤として，線型代数の知識が必要となることが少なくない。線型代数はベクトルや行列の演算体系であり，その利点は「計算」と「可視化」を統合する観点を得られるところにある。より深く理解したいものにとっては，これらの学習も合わせて行うことを期待する。

15.1.1　データキューブと断面

Cattel.R.B は共変図 covariation chart，あるいは data cube と呼ばれる図を描いてデータの分類をしている。彼は，どんな観察あるいは測定事象でも，

- 観察が行われる時 time あるいは場所 occasion
- 観測される変数 variable
- その変数が属せられる場所 reference point あるいは個人 person

の 3 つの属性で規定できるとしている。

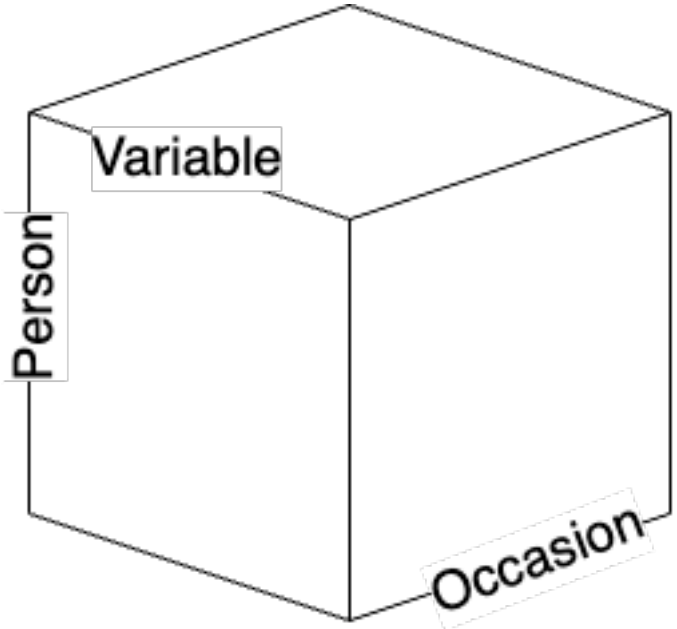


Figure15.1: Cattel のデータキューブ

心理学におけるデータというと，行に人，列に変数のある 2 次元行列 (あるいは表計算ソフトのスプレッドシートと言えばいいだろうか) を想定するが，それはデータの時間軸でカットした一時点のものである。しかしこのキューブは他の軸でも断面を切り出すことができるし，行・列の入れ替えを考えると 6 種類の共変動を計算できる。

	R	Q	O	P	S	T
技法	変数 × 人	人 × 人	時・所 × 時・所	人 × 時・所	時・所 × 変数	変数 × 時・所
実験計画	変数 × 人	人 × 時・所	時・所 × 変数	変数 × 時・所	人 × 変数	時・所 × 人
共変動 (取り出す因子)	変数	人	時・所	人	時・所	変数

Cattel は各段面における因子分析を「technique」として分類した。一般的な因子分析は，変数と人の組み合わせから，変数同士の相関行列を計算して変数の背後にある因子を抽出する。これは R 技法である。これに対し，人と人の相関行列を計算して，人因子を抽出することもできる。これは Q 技法として知られている。あるいは，人と時間・場所の断面で切り出して，時間の因子を出すこともできるし (O 技法)，この面から見た人の因子を取り出しても良い (P 技法)。同様に，S 技法，T 技法を考えることもできる。S 技法を用いた実践例は確認されていないが，それ以外の方法についてはそれぞれ対応する研究もわずかながら存在する。

変数間関係は距離行列 (類似度行列) でもいいし，共頻関係でも良い。例えば距離行列を使って変数をクラスター分析で分類しても良いし，人を分類しても良い。どの共変動，どの側面で分析するかは自由なのである。

昨今では 3 次元という制約を外してデータキューブを一般化し，含まれるデータの種類 (相 mode) とその組み合わせ回数 (元 way) で分類する。例えば，ある集団においてメンバーが相互に評定し合うデータを，複数の集団に

対して取るとすると，含まれるデータはメンバーと集団の 2 種類で，メンバー × メンバーの関係が並ぶので，2 相 3 元データ (2-mode 3-way data) などと呼ぶ。家族システム論ではこうしたデータを扱うことになる。

多変量解析はこのように，どの組み合わせにおいて，どの側面からアプローチするか，またどの要素を要約するかといったということを考える一般的な技法なのである。繰り返すが，「この側面で切り取ってはいけない」といった制約はなく，分析者はデータの全体像をイメージしながら，実践的に意味のある側面で自由に切り取ると良い。それができるような数学的・統計的モデル上の制約はなく，R をはじめとするパッケージを駆使すればいかなる分析も可能である。

多変量解析の全体像を俯瞰したところで，心理学で最も愛されてよく使われている因子分析について詳しく見ていくことにしよう。

15.2　因子分析

ここでは心理学で最もよく用いられる手法の一つである，因子分析法について概説する。因子分析法は測定についての統計モデルである。類似の手法として主成分分析があげられるが，主成分分析は測定のモデルというより要約のモデルというべきである。

因子分析のモデルは次の式で表される。

$$z_{ij} = a_{j1}f_{i1} + a_{j2}f_{i2} + \cdots + a_{jm}f_{im} + d_jU_{ij}$$

ここで z_{ij} は個人 i の項目 j に対するスコアを標準化したものである。$a_{j.}$ は項目 j の因子負荷量 (factor loadings)，$f_{i.}$ は個人 i の因子得点 (factor score)，d_j は項目 j の独自因子負荷量，U_{ij} は項目 j に伴う個人 i の独自因子得点である。$a_{j.}$ で表される m 個の因子を共通因子と呼ぶ。一般に m は項目数 M よりもかなり小さい。例えば性格検査の BIG-five は $M = 25$ で $m = 5$ である。YG 正確検査は $M = 120$ で $m = 12$ である。この意味で多変量解析の目的の一つ，情報圧縮のモデルであるということもできる。

これに対して主成分分析は次のように表される。

$$P_i = w_1X_{i1} + w_2X_{i2} + \cdots + w_MX_{iM}$$

ここで $X_{i.}$ は個人 i の項目 j に対する反応を表し，この重み付き線型結合で主成分 P を形成する。ここでの未知数は w_j であるから，一つの合成変数に作るための最適な重みを見つけることが目的となる。その基準の一つが，生成される合成変数 P が個人 i の特徴を最大限際立たせるように，すなわち P_i の分散を最大にすることと考える。もちろん一つの合成変数で M 個の変数が持つ情報をすべて反映させることは難しいので，第二，第三の合成得点 (主成分) を作ることも可能である。

そうすると，情報圧縮という観点から見た m 個の共通因子，m 個の主成分の違いは何だろうか。これはすでに述べたように，因子分析は測定のモデルなので，得られたデータ $z_{ij} = \frac{X_{ij} - \bar{X}_j}{\sigma_j}$ には誤差 d_jU_{ij} が含まれていると考えているのに対し，主成分分析では X_{ij} にそれを仮定せず，得られた値をそのまま用いているところが異なる。

実践的な面では，心理尺度のような反応に誤差が仮定されるものには因子分析を用い，公的な記録など値に誤差が想定されないものには主成分分析を用いることが相応しい。因子分析が心理学やテスト理論の領域で広まり，主成分分析が経済学，商学，社会学の領域で広まったのはそうした背景による。

計算論的には，いずれも変数間関係を元に最大限説明できる要素を抽出するというところで，行列の固有値分解を用いるという点が同じであるから，統計パッケージによっては同じメニューで異なる出力になっているものも少なくない。しかし上で述べたように，モデルの設計上の違いがあることは知っておいて損はないだろう。また，因子分析は変数間関係として相関行列を，主成分分析は分散共分散行列を用いることが多い。これは因子分析を用いる心理学的な領域では，測定値に絶対的な意味がなく相対的な意味 (ex. より外向的，より内向的) しかないことに対し，他の社会科学領域では絶対的な意味がある (ex. 国家間の貿易黒字・赤字の額など) 場合が多いからである。また，主成分分析は多くの変数を情報圧縮する目的で第一主成分のみに注目することが多いのに対し，因子分析は測定しているものの考え方から複数の因子を考えることが多い。因子分析において単因子で考えるか多因子で考えるかについては，知能検査において知能を一般的な単一の因子で考えるのか，各領域に対応する複数の因子があるのかといった，理論的な相違がその黎明期にみられたことを反映している。

類似の手法ではあるが，こうした背景を知っておくことで適切な手法を用いることができるようになるだろう。

15.2.1 探索的因子分析

特に断りなく単に因子分析というとき，探索的因子分析 (Exploratory Factor Analysis) を指すことが多い。探索的というのは，因子負荷量 (因子から項目へのパス係数，因子と項目の関係の強さを反映したもの) はもちろん共通因子の数についても事前に定めず，データから因子構造を探ることを目的とするものだからである。

探索的因子分析は次のステップで進められる。

1. 因子数の決定
2. 因子負荷量の推定 (因子軸の回転)
3. 因子得点の推定

もちろん分析に入る前に，分析対象となるデータの記述統計や可視化を通じて基本的な項目属性を把握していることが前提である。

15.2.1.1 因子数の決定

因子分析を数学的に語れば，M 個の項目相互の相関係数を表した相関行列 $\mathbb{R}$ を固有値分解することに尽きる。相関係数はピアソンの積率相関係数を用いることが一般的であるが，項目が順序尺度水準であるとかバイナリ変数であるとかいった場合は，それに応じた相応しい相関係数を用いる。

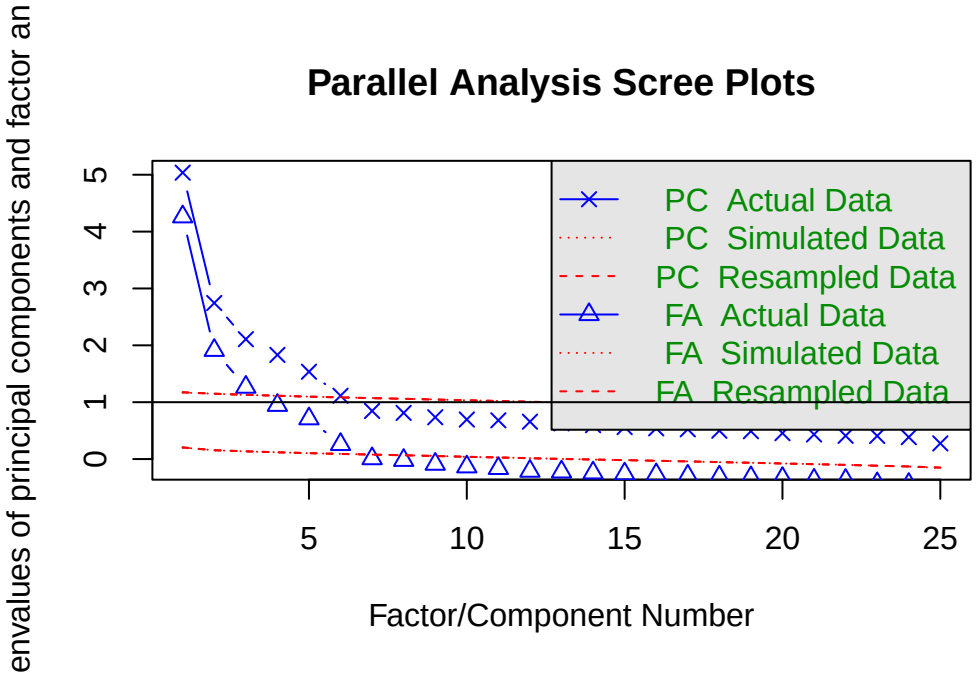
固有値分解とは相関行列の次元性を見ることでもある。M 個の項目もつ情報は M 次元あると考える。例えば 2 変数 X, Y があれば，変数 X を x 軸，変数 Y を y 軸に取った 2 次元空間に核反応をプロットすることでデータ全体の関係を表現できるだろう。しかしこれらの二変数が相関しているなら，変数 X, Y を直交させた空間で表現する必要は必ずしもなく，より分散の大きくとれる二次元基底を見つけることができるに違いない。これが因子分析，主成分分析に共通する考え方であり，最大の分散を持つ次元に注目するのが主成分分析，多次元のなかで有用な次元を共通因子，それ以外を誤差因子と区別して多因子 (多次元) で考えるのが因子分析である。

ここで共通因子の数を決めるのは分析者であり，「有用な次元」の決定は主観的な側面を含むことに注意しよう。もちろんデータの構造から適した次元数を考える手法はいろいろ提案されており，昨今はより客観的基準で因子数を決定するのが一般的であるが，数学的な特徴から実践的な意味合いをもつ共通次元とみなすのは，あくまで

も分析者の責任において行われるものである。

因子数を決定する手法として，スクリープロットをつかった平行分析がある。次のコードを見ながら具体的に見ていこう。分析には psych パッケージを用い，データは psych パッケージの持つサンプルデータ，bfi を用いる。これは性格テストのビッグファイブ因子それぞれについて 5 項目で測定したデータである。

```
pacman::p_load(tidyverse, psych)
dat <- psych::bfi |> select(-gender, -education, -age)
# 並行分析
fa.parallel(dat)
```




```
Parallel analysis suggests that the number of factors =  6  and the number of components =  6
```

データ行列から得られる固有値の大きい順に折れ線グラフを描いたものを，スクリープロットという。デフォルトでは PC すなわち主成分分析 Principle Component Analysis のスクリープロットと，FA すなわち因子分析 Factor Analysis のスクリープロットが表示されている。この違いは上で述べたように，データに誤差を仮定するかどうかの違いにある。因子分析はこれを仮定するため 1 つの項目のもつ情報量が 1 単位以下になる (相関係数 r_{jj} が 1.0 より小さくなる。正確には，$r_{jj} = 1 - h_j^2 = u^2 < 0$ であり，ここで h_j^2 は共通性と呼ばれる共通因子負荷量の二乗和，u_j^2 は独自性因子負荷量の二乗和) ため，主成分分析のそれより必ず低くなる。

プロットされているのは Actual Data, Simulated Data, Resmapled Data となっているのがわかるだろう。実際のデータは何らかの意味構造を有しているだろうから，その相関関係にも偏りが生じ，よく説明できる次元とそうでない次元とが生まれるため，徐々に減衰するカーブで表示される。これに対して Simulated Data は同じサイズの乱数データから，Resampled Data は実際のデータをごちゃ混ぜにした行列を作って得られた固有値構造を表している。乱数や撹拌したデータは実際の意味構造を持たず，どの次元も均等に無意味になるため，フラットな線で表示されるだろう。このフラットな線と実データの線を比べ，フラットなラインよりも大きな意味がある次元は無意味ではない，と考えて因子数を決めるのが平行分析の考え方である。この考え方に基づくと，因子分析解

も主成分分析解も 6 因子 (6 成分) がてきせつであるということになる。

なお図中には固有値が 1.0 のところにもラインが引かれている。これはかつて使われていたガットマン基準というもので，項目 1 つ分の分散も持たないような因子は共通因子たり得ない，という考え方である。この考え方によると 3 因子が妥当ということになる。ただし判断の基準として，共通因子で分散全体の何 % を説明したか，というのもあり，たとえば 3 因子までで 50% も説明しないようであれば半分以上の情報を捨てることになるから，4，5 因子まで採用するという考え方もあり得る。

15.2.1.2 因子負荷量の推定

因子の数が決まると，その過程のもとで因子負荷量の推定に入る。例えば次のようにして結果を得る。

```
result.fa <- fa(dat, nfactors = 6, fm = "ML", rotate = "geominQ")
```

```
要求されたパッケージ GPArotation をロード中です
```

psych パッケージの fa 関数は実に多くのオプションを持っているが，ここでは因子数 (nfactors)，推定法 (fm)，回転法 (rotate) の 3 つを指定した。因子数はすでに述べたので，推定法と回転法について解説する。

推定法は，ここでは**最尤法 (ML)** を指定した。サンプルサイズが 200 を超えるような大きなデータであれば，多変量正規分布のもとからデータが得られたと仮定して因子負荷量を推定するのが最も適切だろう。サンプルサイズが小さい場合は，**最小二乗法系列 (ULS,OLS,WLS,GLS など)** の推定法を指定し，データとモデルのずれを最も小さくするような手法にするのが良い。特段の指定がなければ**最小残差法 (minres)** が選ばれる。これは最小二乗法と同じだが，アルゴリズムが改善されていて収束しやすいという特徴がある。推定法として主成分解 (pa) を選べば，残差を推定しないモデルとなる。アルゴリズムの違い，仮定の違いなどでいろいろ変えうるが，基本的にこれで大きく変化が出るようなものではない。

回転法は因子負荷量を推定した後で，さらに解釈をしやすくするためのものである。因子分析や主成分分析は，データの持ってる空間的特徴の軸を見つけ直すという説明はすでにした通りだが，この軸は原点こそ決まっているが，線型代数的変換によって軸を任意の方向に回転させることができる。であれば最も解釈がやりやすい方向に回転させるのが実践上便利である。この解釈がやりやすい方向というのを数学的に言い換えるならば，一つは項目と因子の関係が**単純構造**にあることだろう。単純構造とは，ある項目が特定の因子に寄与しているのなら，そのほかの因子には寄与していないということである。例えば，外向性を測定する項目が第 1 因子に重く負荷しているのであれば，第 2,3,4,5 因子には負荷していないほうが解釈しやすい。因子はデータの空間的特徴を表す軸 (次元) なのだから，事後的にその軸がの意味であったかを考察する必要があるので，「この因子はこの項目にもあの項目にも影響している」という状況は悩みの種だからである。

この基本方針のもと，いくつかの計算法が考えられている。もっとも古典的なバリマックス回転は，因子負荷量の二乗和の分散が最大になるように回転角を定める。ほかにも，オブリミン回転やジオミン回転などさまざまな回転方法が考えられており，これらについて詳しくは小杉 (2018) などを参照されたい。また，回転方法は大きく分けて**斜交回転**と**直交回転**とに分けられる。直交回転は回転後の軸が直交する，すなわち因子間相関を仮定しない方法であり，斜交回転は因子間相関を仮定する回転方法である。後者の方が数学的な仮定が緩いため，分析の手順としてはまず斜交回転を行い，因子間相関が十分にひくく直交をかていできるなら直交回転をやり直す，という方法をとるべきである。ちなみにここでは geominQ というジオミン回転の斜交版を適用して結果を出力している。

Bernaards & Jennrich (2005) パッケージには多くの回転法が含まれており，回転法を rotate オプションで選択することができるので，ヘルプなどを見て理解を深めて欲しい。

因子軸の回転についても，推定法と同じように絶対的な基準はなく，それぞれの考え方や仮定に基づくアルゴリズムの違いがあるだけである。推定法と違って，因子負荷量は異なる回転法を施すと大きく変わることがある。因子軸の回転は解釈を容易にするためのものであるから，分析者にとって都合の良い回転方法を指定していいが，その回転方法が何で，どういう仮定があるのかについては，自身の言葉で説明できるようになっていた方がいい。

15.2.1.3　出力結果を確認する

推定法，回転法についての概略を踏まえた上で，結果を見てみよう。

```
print(result.fa, sort = T, cut = 0.3)
```

```
Factor Analysis using method =  ml
Call: fa(r = dat, nfactors = 6, rotate = "geominQ", fm = "ML")
Standardized loadings (pattern matrix) based upon correlation matrix
   item   ML3   ML1   ML4   ML6   ML2   ML5   h2   u2 com
E4   14  0.68                             0.56 0.44 1.2
E3   13  0.67                             0.48 0.52 1.2
A5    5  0.56                             0.48 0.52 1.7
E2   12 -0.55              0.45            0.55 0.45 1.9
E1   11 -0.49              0.37            0.39 0.61 2.3
A3    3  0.48                        0.46 0.51 0.49 2.0
E5   15  0.43                             0.40 0.60 3.0
N2   17         0.83                      0.69 0.31 1.0
N1   16         0.82                      0.71 0.29 1.0
N3   18         0.60                      0.52 0.48 1.4
N5   20         0.37        0.34          0.34 0.66 2.4
C2    7               0.69                0.50 0.50 1.1
C4    9              -0.60  0.38          0.55 0.45 2.0
C3    8               0.55                0.31 0.69 1.1
C1    6               0.55                0.35 0.65 1.2
C5   10              -0.52                0.43 0.57 1.8
N4   19         0.35        0.42          0.48 0.52 2.6
O3   23  0.44                     0.56     0.48 0.52 1.9
O5   25                          -0.55     0.37 0.63 1.4
O1   21  0.33                     0.47     0.34 0.66 2.0
O2   22                          -0.46     0.29 0.71 1.6
O4   24                           0.38     0.25 0.75 2.3
A2    2  0.31                           0.58 0.50 0.50 1.6
A1    1                             -0.56 0.33 0.67 1.2
```

```
A4    4                                    0.28 0.72 3.7

                        ML3  ML1  ML4  ML6  ML2  ML5
SS loadings            2.88 2.24 1.92 1.43 1.37 1.23
Proportion Var         0.12 0.09 0.08 0.06 0.05 0.05
Cumulative Var         0.12 0.20 0.28 0.34 0.39 0.44
Proportion Explained   0.26 0.20 0.17 0.13 0.12 0.11
Cumulative Proportion  0.26 0.46 0.64 0.76 0.89 1.00

 With factor correlations of
      ML3   ML1   ML4   ML6   ML2   ML5
ML3  1.00 -0.09  0.26 -0.11 -0.03  0.14
ML1 -0.09  1.00 -0.13  0.30  0.04 -0.11
ML4  0.26 -0.13  1.00 -0.07  0.14  0.16
ML6 -0.11  0.30 -0.07  1.00 -0.03 -0.03
ML2 -0.03  0.04  0.14 -0.03  1.00  0.02
ML5  0.14 -0.11  0.16 -0.03  0.02  1.00

Mean item complexity =  1.8
Test of the hypothesis that 6 factors are sufficient.

df null model =  300  with the objective function =  7.23 with Chi Square =  20163.79
df of  the model are 165  and the objective function was  0.36

The root mean square of the residuals (RMSR) is  0.02
The df corrected root mean square of the residuals is  0.03

The harmonic n.obs is  2762 with the empirical chi square  330.64  with prob <  3.7e-13
The total n.obs was  2800  with Likelihood Chi Square =  1013.79  with prob <  4.6e-122

Tucker Lewis Index of factoring reliability =  0.922
RMSEA index =  0.043  and the 90 % confidence intervals are  0.04 0.045
BIC =  -295.88
Fit based upon off diagonal values = 0.99
Measures of factor score adequacy
                                                 ML3  ML1  ML4  ML6  ML2  ML5
Correlation of (regression) scores with factors  0.88 0.89 0.85 0.77 0.82 0.78
Multiple R square of scores with factors         0.77 0.79 0.71 0.59 0.67 0.61
Minimum correlation of possible factor scores    0.53 0.57 0.43 0.19 0.35 0.21
```

この出力では sort オプションと cut オプションを指定した。sort オプションは因子負荷量の大きい順に並べ替

えてくれるものであり，cut オプションは因子負荷量の表示を抑制するものである。あくまで表示上のオプションであり，実際は各因子から各項目へのパス (5×25 本) が計算されている。

まず表示されているのが因子負荷行列であり，項目の因子ごとの負荷量に加え，共通性 h_j^2 と独自性 $u_j^2 = 1 - h_j^2$，複雑度 complexity が示されている[*1]。なお，ここで表示されている因子負荷量などは回転後のパターン行列であり，斜交回転の場合は，因子軸の負荷量をどう考えるかによって因子パターンと因子構造とに分かれる。因子パターンは変数を斜交座標系に直交に投影した影のようなものであり，変数から因子への直接的な効果を表すと考えられる。因子構造は因子構造は変数を各因子軸に平行に投影した影のようなものであり，変数と因子の間の単純相関を表している。

その下には負荷量の平方和 SS loagings があり，これが説明する分散の大きさである。それを比率にしたもの (Proportion Var)，累積比率にしたもの (Cumulative Var) がある。今回は累積して 44% の説明しかしていないことになるから，56% もの情報をカットしているので，情報圧縮の観点から言えば少し捨てすぎている危険性もある。

続いて，回転行列に斜交回転を指定しているから，因子間相関が出力されている。これを見ると絶対値最大で-0.36がみられる。全ての因子間相関が ± 0.3 に収まるようであれば，直交回転を考えても良い。

その後に出力されているのは適合度に関する指標である。各指標に関する解説は割愛する。

15.2.1.4　因子得点の推定

ここまでで因子と項目の関係を探索的に求める方法について見てきたが，心理学的な研究としては因子と回答者の関係についても興味関心をもつだろう。すなわち，「外向性が高い人は誰か」「情緒不安定性が低い人はどういう特徴を持つか」といった，人に対する理解を深めることである。

数学的には，行列の固有値分解のときに固有値と同時にえらえる固有ベクトルが因子負荷量になる。この相関行列などを構成する時点で，すでに個人の相がもつ情報は要約されて欠落している。なので，因子の構造が明らかになってから，逆算的に個人の得点を考えることになる。因子分析のモデル式で見たように，因子得点は $f_{i.}$ であるが，右辺の因子負荷量がすでに定まっているのなら，左辺も実測値から与えられているので，方程式を解くように答えを求めることができるのである。

psych::fa 関数はデフォルトで因子得点を返すようになっており，以下のコードで確認できる。

```
head(result.fa$scores, 10)
```

```
              ML3         ML1         ML4         ML6         ML2         ML5
61617 -0.67490929 -0.06370731 -1.50533078 -0.4445837 -1.44973997 -0.5355337
61618  0.34631693  0.04818735 -0.61043488 -0.1182062 -0.30201114 -0.3599542
61620 -0.08464855  0.76215875 -0.02892983 -0.1497537  0.30013873 -0.7672156
61621  0.07563257 -0.17026870 -0.97981214  0.5719536 -1.19682554 -0.5298213
61622 -0.03390682 -0.26750789 -0.23091937 -0.4502843 -0.66490684 -0.8116890
61623  1.05989453  0.33788499  1.39368006 -0.5902346  0.28367728 -0.2424507
```

[*1] エディタがその言語に対応していたら，おかしな記述に下線が引かれるなど注意を促してくる機能もある。また RStudio で Stan 言語を書いていると，コンパイルの前に文法のチェックをする機能もある。

```
61624  0.02153406 -1.16850234  0.08502100 -0.6578682  0.78510402  0.2247226
61629 -2.15403379  0.67041389 -1.12937534  0.1900900 -0.04178055 -1.3772793
61630          NA          NA          NA         NA          NA         NA
61633  0.63848897  1.07066826  1.06204540 -0.4196472  0.16316662  0.3949592
```

ここで一部 NA が返されているところがある (例えば ID 61630)，これは回答の中に欠測値が含まれていた場合におこる。

```
head(dat, 10)
```

```
      A1 A2 A3 A4 A5 C1 C2 C3 C4 C5 E1 E2 E3 E4 E5 N1 N2 N3 N4 N5 O1 O2 O3 O4
61617  2  4  3  4  4  2  3  3  4  4  3  3  3  4  4  3  4  2  2  3  3  6  3  4
61618  2  4  5  2  5  5  4  4  3  4  1  1  6  4  3  3  3  3  5  5  4  2  4  3
61620  5  4  5  4  4  4  5  4  2  5  2  4  4  4  5  4  5  4  2  3  4  2  5  5
61621  4  4  6  5  5  4  4  3  5  5  5  3  4  4  4  2  5  2  4  1  3  3  4  3
61622  2  3  3  4  5  4  4  5  3  2  2  2  5  4  5  2  3  4  4  3  3  3  4  3
61623  6  6  5  6  5  6  6  6  1  3  2  1  6  5  6  3  5  2  2  3  4  3  5  6
61624  2  5  5  3  5  5  4  4  2  3  4  3  4  5  5  1  2  2  1  1  5  2  5  6
61629  4  3  1  5  1  3  2  4  2  4  3  6  4  2  1  6  3  2  6  4  3  2  4  5
61630  4  3  6  3  3  6  6  3  4  5  5  3 NA  4  3  5  5  2  3  3  6  6  6  6
61633  2  5  6  6  5  6  5  6  2  1  2  2  4  5  5  5  5  5  2  4  5  1  5  5
      O5
61617  3
61618  3
61620  2
61621  5
61622  3
61623  1
61624  1
61629  3
61630  1
61633  2
```

因子得点はモデル式から逆算的に推定するので，一箇所でも値が見つからなければ答えが出ないのである。また，この推定法による因子得点は標準化されたスコアなので単位がなく，相対的に比較することしかできない。加えて，推定された相対的なスコアであるから，以下の研究プロセスにおいて差の検定などをすることは不適切である，という考え方もある。

実践的には簡便的因子得点と呼ばれる因子得点の計算方法がある。これは因子分析の結果，当該因子に関係する項目に着目し，その評定値を平均することで算出するものである。先の具体的にみていこう。第一因子は E2,E1,N4,E4,E5 から構成されていると考えたとする。ここで E4,E5 は因子負荷量が負であるから，評定値を逆転して考える必要がある。これを踏まえて，たとえば以下のように計算する。

```r
Fscore1.raw <- dat |>
  # 第一因子に該当しそうな項目だけ抜き出す
  select(E2, E1, N4, E4, E5) |>
  # 逆転項目の評定値を反転する
  mutate(
    E4 = 7 - E4,
    E5 = 7 - E5
  )

# 行ごとに欠損値を除いた平均値を計算する
Fscore1 <- apply(Fscore1.raw, 1, function(x) mean(x, na.rm = TRUE))
summary(Fscore1)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.000   2.000   2.800   2.892   3.600   6.000
```

ここでは 6 件法の評定値を逆転させるために，7 から引くという操作をして，欠測を除いて平均するようにしている。こうすることで，尺度値の持つ意味 (中点以上が賛成，未満が反対といったような) を踏まえて考えることができるし，全てが欠測でない限りスコアの算出ができるという利点がある。

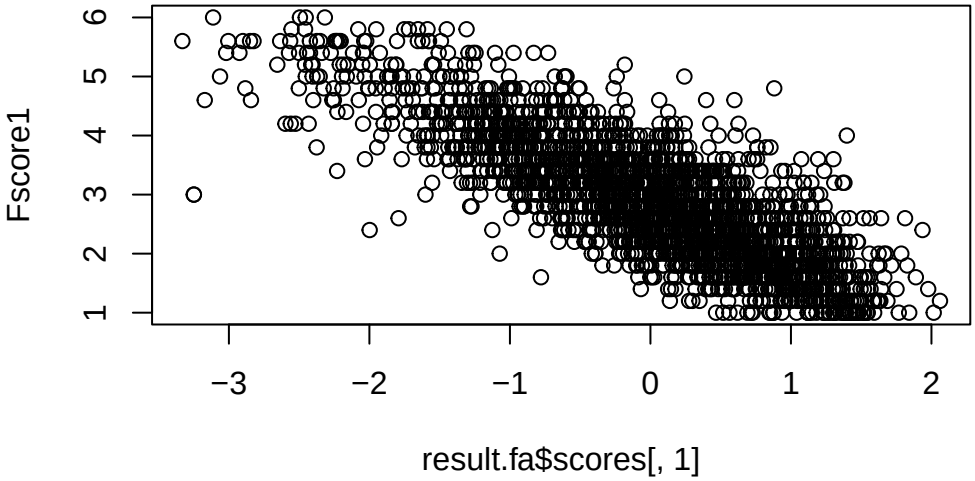
ただしこの方法は，因子負荷量による項目ごとの重みづけを考えないこと，因子分析法によって除外したはずの誤差分散の情報を含んだスコアにしていること，といった短所をもつ。また，そもそも評定値が尺度構成法で正しくスコアリングされたものであるべきだが，実践的にそのような工夫をしている例はほとんど見られないため，非常に精度の低い，荒い推定値になっていると言わざるを得ない。

とはいえ，推定法でもとめたものと簡便法で求めたものは，非常に高く相関するので，心理学のデータがそれほどの精度を持つものでないと割り切れるのであれば，簡便法でも十分だろう。

```r
cor(result.fa$scores[, 1], Fscore1, use = "pairwise")
```

```
[1] -0.8101041
```

```r
plot(result.fa$scores[, 1], Fscore1)
```



15.2.2 確認的因子分析

ここまで探索的因子分析について詳しく解説してきた。そこでは因子数や因子負荷量は事前の情報がなく，データによって語らせる方法で後付け的に解釈を行なっていくのであった。数値例にもあるように，第一因子は主に E 因子の項目 (外向性,Extraversion) から構成されているが，中には N 因子 (情緒不安定性,Neuroticism) の項目も一部含まれており (N4)，解釈に頭を悩ませることも少なくない。そもそも Big5 の名前にあるように理論的には 5 因子なのだが，データは 6 因子を示す，ということもある。

このように探索的因子分析はデータに沿った解釈をするしかないのだが，性格検査のような理論的背景や仮定があるのなら，そちらを重視したいということもあるだろう。このような場合は，因子の構造や仮定を盛り込んだモデルをデータに当てはめるという，確認的因子分析 (Confirmatory Factor Analysis, CFA) を用いる。これは構造方程式モデリング (Structural Equation Modeling) の枠組みで因子分析をとらえたものであり，項目と潜在変数の関係を方程式で表して推定する。

構造方程式モデリングは，分散共分散構造/相関構造の要素に潜在変数を含んだ方程式を当てはめた時の係数を推定するモデルである。方程式はパス図と呼ばれる表現方法で図示されることが多い。パス図では相関的関係を双方向の矢印で，回帰的関係を単方向の矢印で表現し，観測変数を矩形で，潜在変数を楕円で表す。パス図の表現を使うと，因子分析と主成分分析の違いは一目瞭然である。

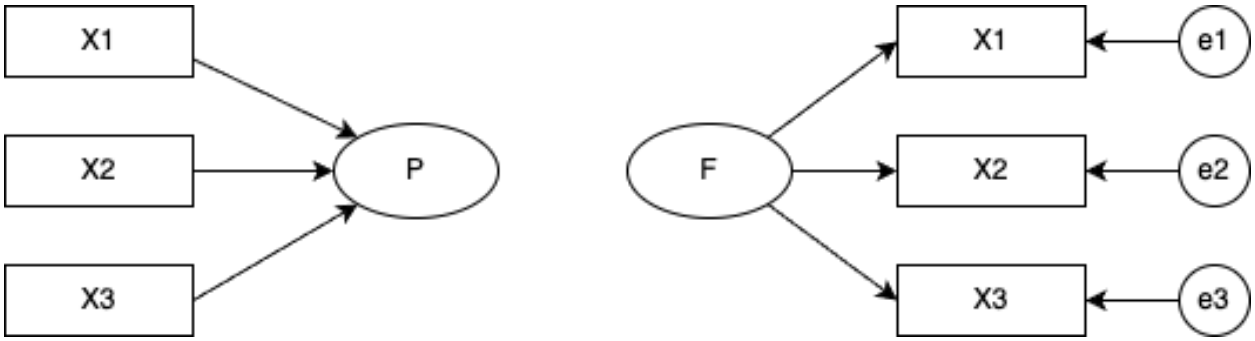


Figure15.2: 主成分分析 (左) と因子分析 (右) のパス図表現

また，探索的因子分析と確認的因子分析の違いも明白である。

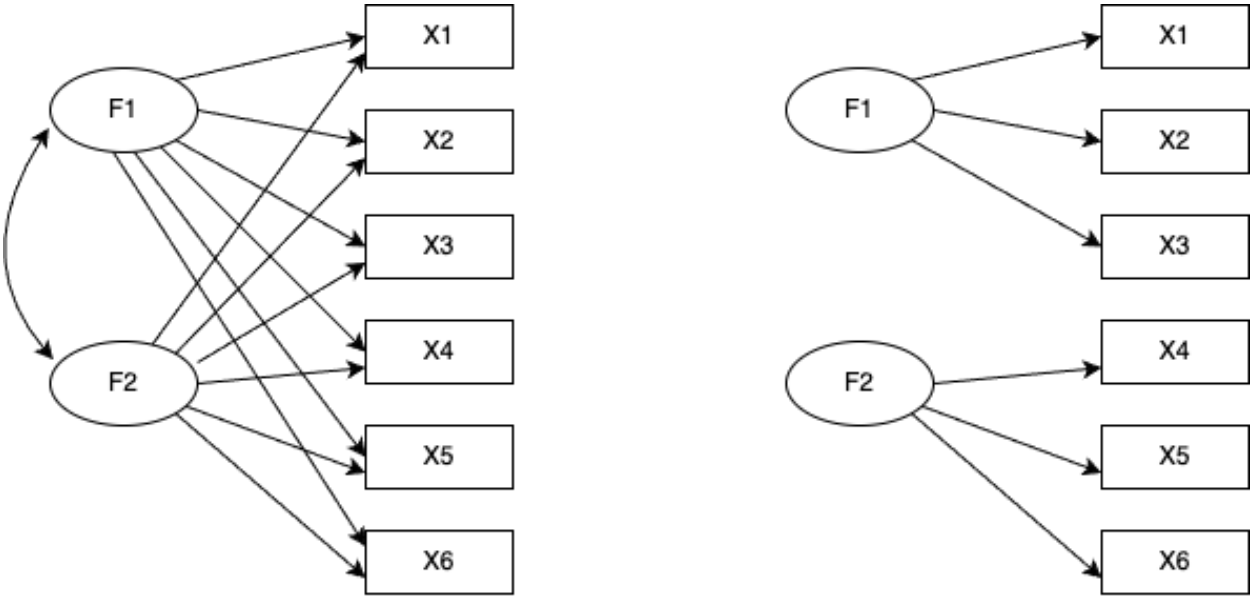


Figure15.3: EFA(左) と CFA(右) のパス図表現。簡略化するために誤差因子は描画しない。

確認的因子分析では，どの因子がどの項目に影響しているかを個別に指定している。言い換えるなら，影響がないと仮定するパスの係数を 0 に固定しているともいえる。この図では確認的因子分析モデルの因子間相関が 0 であると仮定しているため，パスを引いていない (引くこともちろん可能である)。

この方法ではモデルの方が先にあり，このモデルから考えられる分散共分散行列の式を実際の分さ共分散行列に当てはめることになる。当てはめる，すなわち係数を推定する方法は，大きく分けて最小二乗法，最尤法，ベイズ法であるが，実際は各種推定法のアルゴリズム名まで把握しておくといいだろう。さらに推定後，モデルと実際の分散共分散行列がどれほど一致しているか，即ち**適合度** Model Fit Indices をみてその評価を行うことになる。適合度指標も複数あるため，それらを見ながら総合的に評価することになる。

R での具体例を見ておこう。パッケージ `lavvaan` を用いて[*2]以下のようにモデルを与える[*3]。

```
pacman::p_load(lavaan)
# モデル指定
model <- "
Neuroticism =~ N1 + N2 + N3 + N4 + N5
Agreeableness =~ A1 + A2 + A3 + A4 + A5
Extraversion =~ E1 + E2 + E3 + E4 + E5
Openness =~ O1 + O2 + O3 + O4 + O5
Conscientiousness =~ C1 + C2 + C3 + C4 + C5
```

[*2] とはいえ，コードに原因がない場合もある。それは Stan を導入する際のシステム的なエラーであり，書かれた内容ではなく動かす環境全体の問題である。解決策としては，表示されるエラーを解読して問題を解決するか，環境を再構築する (Stan を再インストールする，最新バージョンに入れ替える等) 必要がある。この場合も，生成 AI が助力してくれるだろう。

[*3] 詳しくは (浜田他，2019) を参照してほしい

```
"
```

ここで model オブジェクトは文字列として入力されていることに注意しよう。シングル，あるいはダブルクォーテーションでモデル記述を囲むのである。また，潜在変数名を左辺に置き，それを構成する観測変数を右辺に置く**測定方程式**は，=~という演算子で繋ぐ。変数同士の相関的関係は~~という演算子を，回帰的関係は~という演算子を用いる。特に潜在変数同士の関係を記述する方程式は**構造方程式**と呼ばれる。

ここでは因子間相関に関する記述はないが，デフォルトで 0 と指定しないところには相関のパスが仮定される。係数をゼロに指定したい場合は，Neuroticism ~~ 0 * Openness のように記述するといいだろう。

さて，モデルの指定が終われば，データと推定法を指定して推定させよう。ここでは推定法 (estimator) オプションを最尤法 (ML) とした。また，要約を出力するときに，適合度指標 (fit.meassures) と標準化係数 (standardized) も表示するよう指定してある。

```
# モデル推定
model.fit <- sem(model, estimator = "ML", data = dat)
summary(model.fit, fit.measures = TRUE, standardized = TRUE)
```

```
lavaan 0.6-21 ended normally after 55 iterations

  Estimator                                         ML
  Optimization method                           NLMINB
  Number of model parameters                        60

                                                  Used       Total
  Number of observations                          2436        2800

Model Test User Model:

  Test statistic                              4165.467
  Degrees of freedom                           265
  P-value (Chi-square)                          0.000

Model Test Baseline Model:

  Test statistic                             18222.116
  Degrees of freedom                           300
  P-value                                       0.000

User Model versus Baseline Model:

  Comparative Fit Index (CFI)                   0.782
```

```
  Tucker-Lewis Index (TLI)                       0.754

Loglikelihood and Information Criteria:

  Loglikelihood user model (H0)             -99840.238
  Loglikelihood unrestricted model (H1)     -97757.504

  Akaike (AIC)                              199800.476
  Bayesian (BIC)                            200148.363
  Sample-size adjusted Bayesian (SABIC)     199957.729

Root Mean Square Error of Approximation:

  RMSEA                                          0.078
  90 Percent confidence interval - lower         0.076
  90 Percent confidence interval - upper         0.080
  P-value H_0: RMSEA <= 0.050                    0.000
  P-value H_0: RMSEA >= 0.080                    0.037

Standardized Root Mean Square Residual:

  SRMR                                           0.075

Parameter Estimates:

  Standard errors                             Standard
  Information                                 Expected
  Information saturated (h1) model          Structured

Latent Variables:
                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
  Neuroticism =~
    N1                1.000                               1.300    0.825
    N2                0.947    0.024   39.899    0.000    1.230    0.803
    N3                0.884    0.025   35.919    0.000    1.149    0.721
    N4                0.692    0.025   27.753    0.000    0.899    0.573
    N5                0.628    0.026   24.027    0.000    0.816    0.503
  Agreeableness =~
    A1                1.000                               0.484    0.344
    A2               -1.579    0.108  -14.650    0.000   -0.764   -0.648
    A3               -2.030    0.134  -15.093    0.000   -0.983   -0.749
```

```
    A4                -1.564    0.115  -13.616    0.000   -0.757   -0.510
    A5                -1.804    0.121  -14.852    0.000   -0.873   -0.687
  Extraversion =~
    E1                 1.000                             0.920    0.564
    E2                 1.226    0.051   23.899    0.000    1.128    0.699
    E3                -0.921    0.041  -22.431    0.000   -0.847   -0.627
    E4                -1.121    0.047  -23.977    0.000   -1.031   -0.703
    E5                -0.808    0.039  -20.648    0.000   -0.743   -0.553
  Openness =~
    O1                 1.000                             0.635    0.564
    O2                -1.020    0.068  -14.962    0.000   -0.648   -0.418
    O3                 1.373    0.072   18.942    0.000    0.872    0.724
    O4                 0.437    0.048    9.160    0.000    0.277    0.233
    O5                -0.960    0.060  -16.056    0.000   -0.610   -0.461
  Conscientiousness =~
    C1                 1.000                             0.680    0.551
    C2                 1.148    0.057   20.152    0.000    0.781    0.592
    C3                 1.036    0.054   19.172    0.000    0.705    0.546
    C4                -1.421    0.065  -21.924    0.000   -0.967   -0.702
    C5                -1.489    0.072  -20.694    0.000   -1.012   -0.620

Covariances:
                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
  Neuroticism ~~
    Agreeableness     0.141    0.018    7.712    0.000    0.223    0.223
    Extraversion      0.292    0.032    9.131    0.000    0.244    0.244
    Openness         -0.093    0.022   -4.138    0.000   -0.112   -0.112
    Conscientisnss   -0.250    0.025  -10.117    0.000   -0.283   -0.283
  Agreeableness ~~
    Extraversion      0.304    0.025   12.293    0.000    0.683    0.683
    Openness         -0.093    0.011   -8.446    0.000   -0.303   -0.303
    Conscientisnss   -0.110    0.012   -9.254    0.000   -0.334   -0.334
  Extraversion ~~
    Openness         -0.265    0.021  -12.347    0.000   -0.453   -0.453
    Conscientisnss   -0.224    0.020  -11.121    0.000   -0.357   -0.357
  Openness ~~
    Conscientisnss    0.130    0.014    9.190    0.000    0.301    0.301

Variances:
                   Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
   .N1               0.793    0.037   21.575    0.000    0.793    0.320
```

.N2	0.836	0.036	23.458	0.000	0.836	0.356
.N3	1.222	0.043	28.271	0.000	1.222	0.481
.N4	1.654	0.052	31.977	0.000	1.654	0.672
.N5	1.969	0.060	32.889	0.000	1.969	0.747
.A1	1.745	0.052	33.725	0.000	1.745	0.882
.A2	0.807	0.028	28.396	0.000	0.807	0.580
.A3	0.754	0.032	23.339	0.000	0.754	0.438
.A4	1.632	0.051	31.796	0.000	1.632	0.740
.A5	0.852	0.032	26.800	0.000	0.852	0.528
.E1	1.814	0.058	31.047	0.000	1.814	0.682
.E2	1.332	0.049	26.928	0.000	1.332	0.512
.E3	1.108	0.038	29.522	0.000	1.108	0.607
.E4	1.088	0.041	26.732	0.000	1.088	0.506
.E5	1.251	0.040	31.258	0.000	1.251	0.694
.O1	0.865	0.032	27.216	0.000	0.865	0.682
.O2	1.990	0.063	31.618	0.000	1.990	0.826
.O3	0.691	0.039	17.717	0.000	0.691	0.476
.O4	1.346	0.040	34.036	0.000	1.346	0.946
.O5	1.380	0.045	30.662	0.000	1.380	0.788
.C1	1.063	0.035	30.073	0.000	1.063	0.697
.C2	1.130	0.039	28.890	0.000	1.130	0.650
.C3	1.170	0.039	30.194	0.000	1.170	0.702
.C4	0.960	0.040	24.016	0.000	0.960	0.507
.C5	1.640	0.059	27.907	0.000	1.640	0.615
Neuroticism	1.689	0.073	23.034	0.000	1.000	1.000
Agreeableness	0.234	0.030	7.839	0.000	1.000	1.000
Extraversion	0.846	0.062	13.693	0.000	1.000	1.000
Openness	0.404	0.033	12.156	0.000	1.000	1.000
Conscientisnss	0.463	0.036	12.810	0.000	1.000	1.000

出力として，まずモデルの要約 (推定法など) が示され，続いて適合度指標 (CFI, TFI, AIC, BIC, RMSEA, SRMR) などが表示される。これらの適合度指標は大きく 3 つのカテゴリーに分類できる：

まずは，飽和モデルとヌルモデルの間の比較指標である。CFI や TFI がこれに該当する。ヌルモデルを 0，飽和モデルを 1 としたとき，今回のモデルがどこに位置づくかを示す。ここで飽和モデルとはデータに完全に適合するモデルであり，すべての観測変数間にすべての可能なパスが引かれたものを指す。逆にヌルモデル (独立モデル) は観測変数間に関連性がまったくないと仮定する最も制約の強いモデルである。

次に，尤度に基づく相対指標である。AIC，BIC，SABIC がこれに該当する。尤度はデータが確率モデルにどれほど近いかを表したものであるから，データやモデルが異なれば比較はできない。当該データに対する相対比較として用いる。AIC(赤池情報量規準，Akaike Information Criterion) は対数尤度とパラメータの数で計算されており，対数尤度が大きくパラメータ数が少ない方が良いモデルだと考える規準である。-2LL + 2p(LL は対数尤

度，p はパラメータ数) で算出され，小さければ小さいほど当てはまりが良いと判断する。BIC(ベイジアン情報量規準) は，AIC よりもサンプルサイズに対して強いペナルティを与えている。SABIC はサンプルサイズを調整した BIC の亜形である。

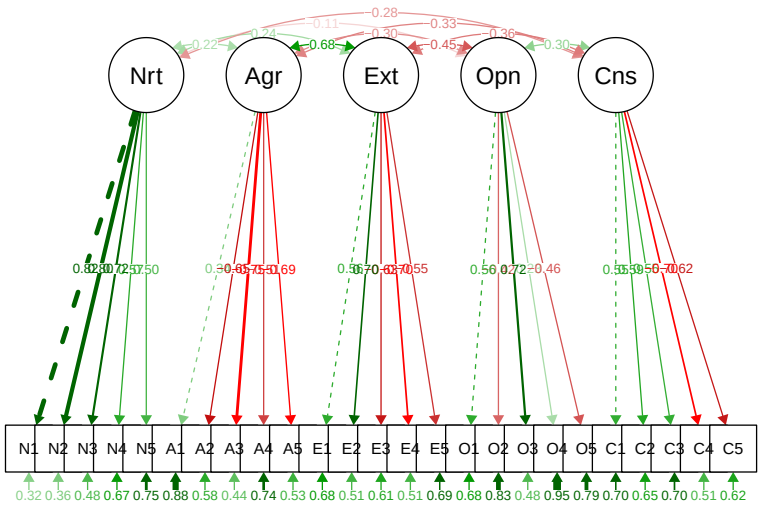
最後に，実データの分散説明量の残量に関する指標がある。RMSEA と SRMR がこれに該当する。RMSEA(Root Mean Square Error of Approximation) はモデルの近似誤差を評価しており，0.05 未満が良好とされる。SRMR(Standardized Root Mean Square Residual) は実データと予測データの残差の標準化された平方根で，0.08 未満が良好とされる。

モデル評価の際は、これらの指標を総合的に検討することが推奨される。単一の指標だけではなく、複数の指標を参照し、それらが一貫して良好な適合を示すかを確認することが重要である。

続く出力で，推定値 Estimator や検定統計量が表示される。心理学の場合は，全ての変数を標準化した推定値 `Std.all` を参照することが多い[*4]。ここでパス係数が小さかったり，統計的に有意にならないものを削除することでモデルの適合度は上げることができる。ただし，適合度を上げることが研究の目的になってはならない。仮定を当てはめるのだから，適当とされる目安を達成できない場合は仮定を省みて改良することはあるだろうが，適合度を上げるために仮定に合わないパスを引くようなことは，「頑張って有意にする」のと同じ QRPs である。

ところで，パッケージを使えば，パス図も自動で描いてくれる。他にも，`lavaanExtra`，`tidySEM`，`lavaanPlot`,`lavaanPlot2` など開発中のものも含めて様々なものがあるが，ここでは古典的な `semPlot` パッケージによる出力例を示す。

```
pacman::p_load(semPlot)
semPaths(model.fit, what = "stand", style = "lisrel")
```



以上が因子分析法の概略である。

因子分析法は測定に関するモデルであり，心理尺度を作成する場合は非常によく用いられるものである。しかしあくまでもデータや項目間の相関関係から共通次元を見出すものであるから，構成概念を直接測定したとか，構

[*4] 正規分布の幅のパラメータは，テキストによっては分散 σ^2 で記載されていることが多いが，ここでは標準偏差 σ で記述する。Stan では標準偏差で記述するようになっているので，それに合わせたいからである。標準偏差を二乗したものが分散であるから，意味するところは本質的に変わりがない。

成概念の存在が証明されたかのような利用・解釈は適切ではない。例えば，何らかの話題についての文言，極端な話「ラーメンに関する記述」を用意して，そこに量的な評価を加えれば，ラーメン因子だろうが豚骨因子だろうが，何らかの解釈ができる因子を抽出することはできる。そのことと，人が心理的に豚骨因子を内在化しているということにはならない。

また構造方程式モデリングによって潜在変数間に回帰や相関のパスを仮定することはできるが，そのことが実際どのような形で顕現化するかについて考えておく必要がある。モデルが非常に適合していて，潜在変数間に強い影響関係があったとしても，測定方程式のパス係数が低かったりすると，結局一方の因子得点が 1 単位増えたことで，従属する潜在変数がどのように変化し，それがどのように行動・測定値に反映されるかを意識しよう。そのような実態的な影響がない，つまり妥当性のない統計モデルは机上の空論に過ぎないからである。

測定とその実際的影響については，因子分析モデルの一種とも言える項目反応理論と問題意識を軌を一にする。

15.2.3　SEM の応用

構造方程式モデリングは確認的因子分析の枠組みにとどまらず，潜在変数間の構造的関係を柔軟に表現できる枠組みであるから，以下のような応用的分析が可能である。

15.2.3.1　媒介分析
15.2.3.2　多母集団同時分析
15.2.3.3　成長曲線モデル

15.3　項目反応理論

つづいて項目反応理論 (Item Response Theory, IRT) を取り上げる。項目反応理論は古典的テスト理論 (Classical Test Theory, CTT) との対比で，現代テスト理論と呼ばれることもある。テスト理論を背景に持つものであるから，従属変数としてバイナリ変数を前提としている (0 が誤答，1 が正答を表す)。これは言い換えれば因子分析において従属変数がバイナリであるものといってもよく，実際にカテゴリカル因子分析との数学的等価性が明らかになっている。

もう一つ因子分析と異なる側面としては，因子分析が性格心理学を背景に因子構造を探索することに重点が置かれていることに対して，項目反応理論は因子得点をより精緻にすることに重点が置かれている。また性格心理学の場合は何因子構造であるかということがすでに学術的な問いであるが，項目反応理論を一とするテスト理論においては「学力」の一因子構造であることが望ましいとされる。このことから，因子分析を使った心理尺度の構成は「単純構造」が良いものであると考え項目を洗練 (取捨選択) することが多いのに対し，項目反応理論は第一因子の負荷量が十分大きければ (一般に 30% 程度の分散説明率があれば良い) 一因子構造であると考えるし，いかなる項目であっても何らかの情報をもたらすものと考えて項目をプールする (捨てない) という方針で進められることが多い点である。

項目反応理論の各種モデルを用いた，コンピュータ適応型テスト (Computer Adaptive Test, CAT) が現在のテスト理論の主流である。これは受験者の回答パターンに応じて項目プールから動的に次々問題を提供し，効率よく受験者の能力を推定していくものである。CAT に必要なのは IRT を背景にしたモデルはもちろんのこと，各能力

水準を測定するのに適した項目プールであり，また項目プールというデータベースとの連携システムである[*5]。

15.3.1　ロジスティックモデル

IRT はバイナリデータに対する単因子モデルである。バイナリデータであるから，連続値を前提とするピアソンの積率相関係数ではなく，テトラコリック相関係数を用いてその次元性を解析する。また因子に該当する被験者母数 θ によって項目反応が回帰されるモデルでもあるから，ロジスティック回帰分析のようなモデル化をすることになる。

被験者母数は標準正規分布が仮定されるが，これを累積正規分布の形で表現するとロジスティックカーブがよくあてまるし，項目の特徴を表現するための項目母数を線型モデルの中に組み込むためには，ロジスティックモデルで表現する方がわかりやすいという側面もある[*6]。以下に標準正規分布と累積正規分布，並びにロジスティック関数による正規累積分布の近似を示す。なお，ロジスティックモデルで扱われる関数は，以下のように係数 (1.702) を用いると，よりよく近似することが知られている。

$$f(x) = \frac{1}{1 + exp(-1.702x)}$$

```
pacman::p_load(ggplot2, patchwork)

# データの準備
x <- seq(-4, 4, length.out = 1000)
normal_df <- data.frame(
  x = x,
  density = dnorm(x),
  cdf = pnorm(x),
  logistic = 1 / (1 + exp(-1.702 * x))
)

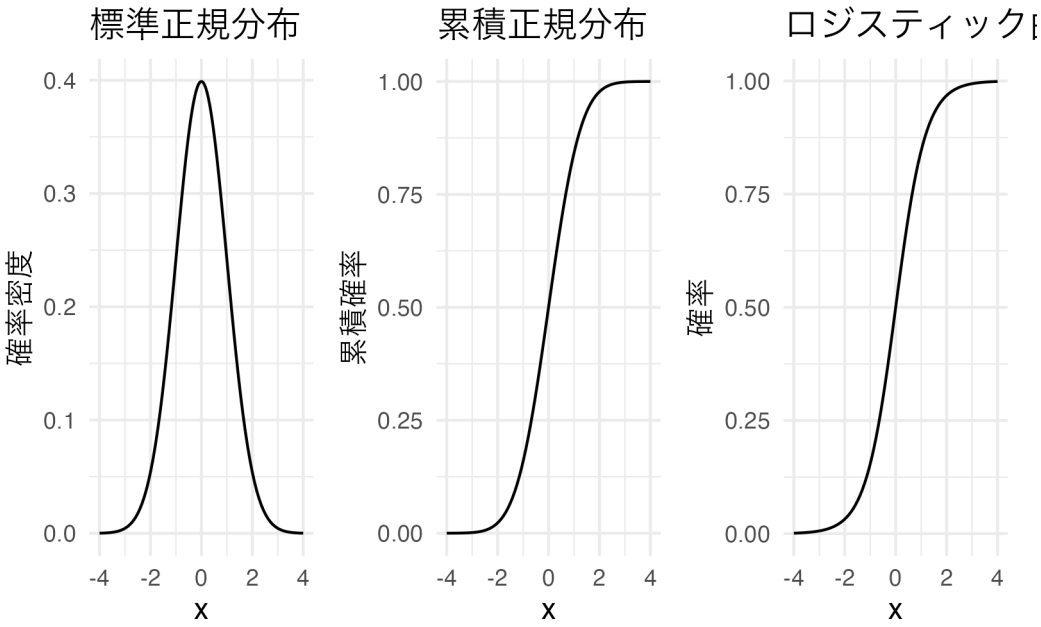
# 1. 標準正規分布
p1 <- ggplot(normal_df, aes(x = x, y = density)) +
  geom_line() +
  labs(
    title = "標準正規分布",
    x = "x",
    y = "確率密度"
  ) +
  theme_minimal()
```

[*5] このカバーストーリーとモデルはリー・ワゲンメーカーズ（2017）の「七人の科学者」P.48–49 を参考に作ったものである。
[*6] たとえば紀ノ定（2018）が参考になる。

```
# 2. 累積正規分布
p2 <- ggplot(normal_df, aes(x = x, y = cdf)) +
  geom_line() +
  labs(
    title = "累積正規分布",
    x = "x",
    y = "累積確率"
  ) +
  theme_minimal()

# 3. ロジスティック曲線
p3 <- ggplot(normal_df, aes(x = x, y = logistic)) +
  geom_line() +
  labs(
    title = "ロジスティック曲線による近似",
    x = "x",
    y = "確率"
  ) +
  theme_minimal()

# 3つのプロットを横に並べて表示
p1 + p2 + p3
```



このロジスティック関数を用いて，項目母数を使って項目の特徴を描画することを考えよう。IRT のロジスティックモデルには，パラメータが 1 つのもの，2 つのもの…とさまざまなものが考えられているが，パラメータ数が多

いモデルはパラメータ数が少ないモデルに含まれる (特殊形) である。

15.3.1.1 1PL モデル

まずは 1 パラメータロジスティックモデル (1PL モデル) を考えよう。このモデルは，項目母数 b を用いて，以下のように表現される。

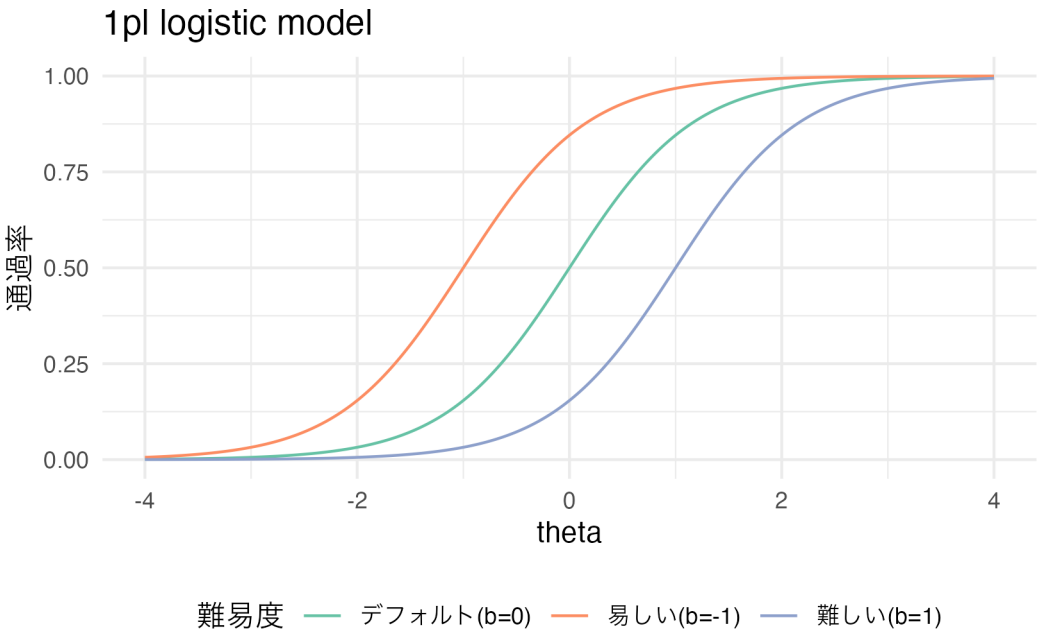
$$P(Y_{ij} = 1|\theta_i, b_j) = \frac{1}{1 + exp(-1.702(\theta_i - b_j))}$$

ここで，Y_{ij} は被験者 i が項目 j に正答したかどうかを表すバイナリ変数であり，θ_i は被験者 i の能力を表す被験者母数である。また，b_j は項目 j の**困難度 (difficulty)** を表す項目母数である。というのも，この b_j が大きくなるとロジスティック曲線は右に寄る，また小さくなると左に寄るからである。横軸は被験者母数 θ であり，縦軸は通過率であるから，曲線が右にシフトすることはより能力が高くなければ通過率が上昇しないことを表すからである。

```
logistic_1pl <- function(theta, b) {
  1 / (1 + exp(-1.702 * (theta - b)))
}

x <- seq(-4, 4, length.out = 1000)
normal_df <- data.frame(
  x = x,
  default = logistic_1pl(x, 0), # default
  easy = logistic_1pl(x, -1),
  hard = logistic_1pl(x, 1)
)

ggplot(normal_df) +
  geom_line(aes(x = x, y = default, color = "デフォルト(b=0)")) +
  geom_line(aes(x = x, y = easy, color = "易しい(b=-1)")) +
  geom_line(aes(x = x, y = hard, color = "難しい(b=1)")) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "1pl logistic model",
    x = "theta",
    y = "通過率",
    color = "難易度"
  ) + # 凡例のタイトルを追加
  theme_minimal() +
  theme(legend.position = "bottom") # 凡例を下部に配置
```



15.3.1.2　2PL モデル

2 パラメータロジスティックモデル (2PL モデル) は，1PL モデルに加えて項目母数 a を含める。この母数は**識別力**と呼ばれる。

$$P(Y_{ij} = 1|\theta_i, a_j, b_j) = \frac{1}{1 + exp(-1.702a_j(\theta_i - b_j))}$$

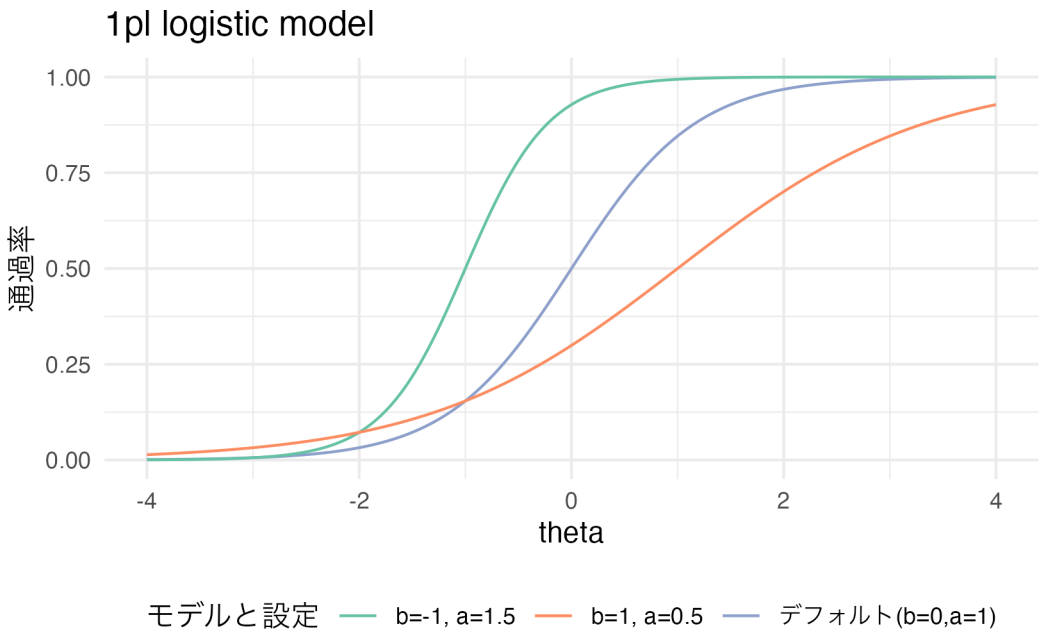
これが識別力と呼ばれるのは，ロジスティック曲線の傾きを変えるからである。傾きが強くなって急激に上昇することは，ある θ の値で急に正誤の確率が変わることを意味し，逆に傾きが緩くなることは特定の θ の値でも正誤の違いが大きくないことを意味するからである。ちなみにカテゴリカル因子分析の文脈では，困難度 b_j が閾値に，識別力 a_j が因子負荷量に相当する。

```
logistic_2pl <- function(theta, a, b) {
  1 / (1 + exp(-1.702 * a * (theta - b)))
}

x <- seq(-4, 4, length.out = 1000)
normal_df <- data.frame(
  x = x,
  default = logistic_2pl(x, 1, 0), # default
  easy = logistic_2pl(x, 1.5, -1),
  hard = logistic_2pl(x, 0.5, 1)
)

ggplot(normal_df) +
```

```
  geom_line(aes(x = x, y = default, color = "デフォルト(b=0,a=1)")) +
  geom_line(aes(x = x, y = easy, color = "b=-1, a=1.5")) +
  geom_line(aes(x = x, y = hard, color = "b=1, a=0.5")) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "1pl logistic model",
    x = "theta",
    y = "通過率",
    color = "モデルと設定"
  ) + # 凡例のタイトルを追加
  theme_minimal() +
  theme(legend.position = "bottom") # 凡例を下部に配置
```



15.3.1.3　3,4,5PL モデル

実践的には 2PL モデルが最もよく用いられるが，理論的には 3，4，5PL モデルまで提案されており，それぞれ次のように表現される。

$$P(Y_{ij}=1|\theta_i,a_j,b_j,c_j)=c_j+\frac{1-c_j}{1+exp(-1.702a_j(\theta_i-b_j))}$$

$$P(Y_{ij}=1|\theta_i,a_j,b_j,c_j,d_j)=c_j+\frac{d_j-c_j}{1+exp(-1.702a_j(\theta_i-b_j))}$$

$$P(Y_{ij}=1|\theta_i,a_j,b_j,c_j,d_j,e_j)=c_j+\frac{d_j-c_j}{\{1+exp(-1.702a_j(\theta_i-b_j))\}^{e_j}}$$

ここで c_j は下方漸近線母数，d_j は上方漸近線母数，e_j は非対称性母数と呼ばれている。このように，モデルとしては徐々にパラメータ数を増やして表現しているが，推定すべきパラメータの数が増えるとより大きい標本サイズ (受験者数) が必要となるし，等価など運用シーンでも複雑になることから，あまり用いられるものではない。

15.3.1.4　ロジスティックモデルと分析の実際

項目の特徴を表現するロジスティック曲線は，**項目反応関数** (Item Response Function) あるいは**項目特性曲線** (Item Characteristic Curve) と呼ばれる。`ltm` パッケージや `exametrika` パッケージなど，IRT モデルを実行するための R パッケージは複数あり，これを使って実践例を見てみよう。

ここでは著者が開発した `exametrika` パッケージとそのサンプルデータを用いて，実際に IRT を実行してみよう。`exametrika` パッケージにはサンプルデータが複数含まれている。今回用いる J15S500 は，500 人の被験者が 15 問の項目に回答したサンプルデータである。

```
pacman::p_load(exametrika)
result.2pl <- IRT(J15S500, model = 2, verbose = FALSE)
print(result.2pl)
```

```
Item Parameters
       slope location PSD(slope) PSD(location)
Item01 0.698   -1.684     0.1093         0.266
Item02 0.810   -1.553     0.1166         0.221
Item03 0.559   -1.839     0.0988         0.338
Item04 1.416   -1.179     0.1569         0.113
Item05 0.681   -2.242     0.1152         0.360
Item06 0.997   -2.163     0.1499         0.273
Item07 1.084   -1.040     0.1281         0.130
Item08 0.694   -0.558     0.1002         0.153
Item09 0.347    1.629     0.0766         0.427
Item10 0.492   -1.421     0.0906         0.306
Item11 1.122    1.020     0.1314         0.124
Item12 1.216    1.030     0.1385         0.117
Item13 0.875   -0.720     0.1111         0.133
Item14 1.200   -1.232     0.1407         0.134
Item15 0.823   -1.204     0.1127         0.180

Item Fit Indices
       model_log_like bench_log_like null_log_like model_Chi_sq null_Chi_sq
Item01       -263.526       -240.190      -283.343       46.673      86.307
Item02       -252.912       -235.436      -278.949       34.952      87.025
Item03       -281.083       -260.906      -293.598       40.353      65.383
Item04       -205.839       -192.072      -265.962       27.534     147.780
```

```
Item05       -232.073       -206.537       -247.403       51.072      81.732
Item06       -173.933       -153.940       -198.817       39.987      89.755
Item07       -252.037       -228.379       -298.345       47.317     139.933
Item08       -313.756       -293.225       -338.789       41.061      91.127
Item09       -325.691       -300.492       -327.842       50.397      54.700
Item10       -309.450       -288.198       -319.850       42.503      63.303
Item11       -250.830       -224.085       -299.265       53.488     150.360
Item12       -240.231       -214.797       -293.598       50.869     157.603
Item13       -291.822       -262.031       -328.396       59.582     132.730
Item14       -224.331       -204.953       -273.212       38.756     136.519
Item15       -273.122       -254.764       -302.847       36.717      96.166
       model_df null_df   NFI   RFI   IFI   TLI   CFI RMSEA    AIC    CAIC
Item01       12      13 0.459 0.414 0.533 0.488 0.527 0.076 22.673 -39.902
Item02       12      13 0.598 0.565 0.694 0.664 0.690 0.062 10.952 -51.623
Item03       12      13 0.383 0.331 0.469 0.414 0.459 0.069 16.353 -46.223
Item04       12      13 0.814 0.798 0.886 0.875 0.885 0.051  3.534 -59.042
Item05       12      13 0.375 0.323 0.440 0.384 0.432 0.081 27.072 -35.503
Item06       12      13 0.554 0.517 0.640 0.605 0.635 0.068 15.987 -46.589
Item07       12      13 0.662 0.634 0.724 0.699 0.722 0.077 23.317 -39.258
Item08       12      13 0.549 0.512 0.633 0.597 0.628 0.070 17.061 -45.515
Item09       12      13 0.079 0.002 0.101 0.002 0.079 0.080 26.397 -36.179
Item10       12      13 0.329 0.273 0.405 0.343 0.394 0.071 18.503 -44.073
Item11       12      13 0.644 0.615 0.700 0.673 0.698 0.083 29.488 -33.087
Item12       12      13 0.677 0.650 0.733 0.709 0.731 0.081 26.869 -35.706
Item13       12      13 0.551 0.514 0.606 0.569 0.603 0.089 35.582 -26.993
Item14       12      13 0.716 0.692 0.785 0.765 0.783 0.067 14.756 -47.820
Item15       12      13 0.618 0.586 0.706 0.678 0.703 0.064 12.717 -49.858
           BIC
Item01 -27.902
Item02 -39.623
Item03 -34.223
Item04 -47.042
Item05 -23.503
Item06 -34.589
Item07 -27.258
Item08 -33.515
Item09 -24.179
Item10 -32.073
Item11 -21.087
Item12 -23.706
Item13 -14.993
```

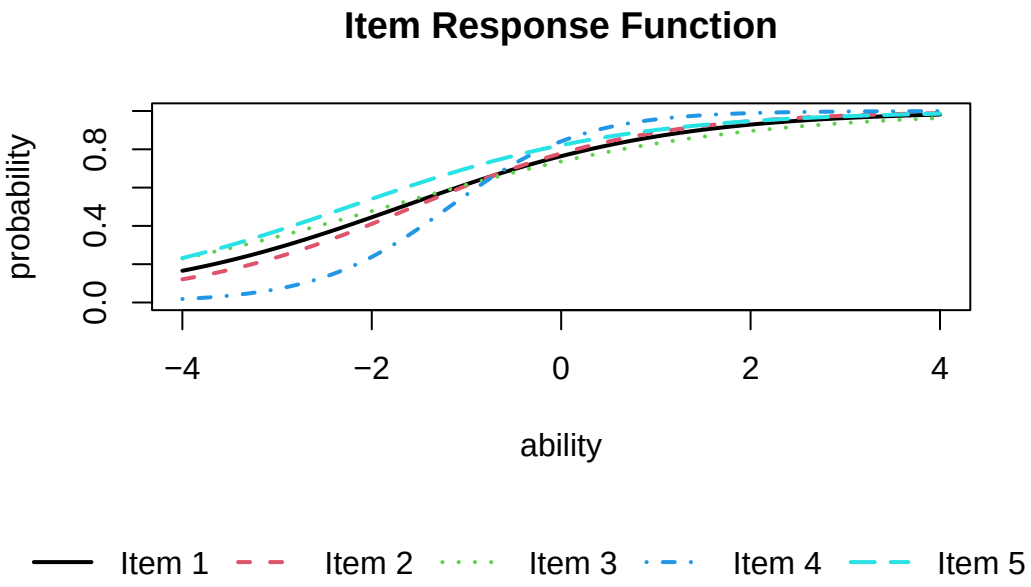
```
Item14 -35.820
Item15 -37.858

Model Fit Indices
                   value
model_log_like -3890.635
bench_log_like -3560.005
null_log_like  -4350.217
model_Chi_sq     661.260
null_Chi_sq     1580.424
model_df         180.000
null_df          195.000
NFI                0.582
RFI                0.547
IFI                0.656
TLI                0.624
CFI                0.653
RMSEA              0.073
AIC              301.260
CAIC            -637.369
BIC             -457.369
```

数値的な特徴としては，`Item Parameters` のところに `slope`(識別力)，`location`(困難度) が示されており，またそれぞれの標準誤差が示されている。続く `Item Fit Indices` は項目ごとの適合度，`Model Fit Indices` はテスト全体のモデル適合度であるが，IRT モデルは SEM の適合度の観点から言えば非常に当てはまりは悪い。これはバイナリデータに対するモデリングであることなどを考えると，ある程度は仕方がないことであるとも言える。

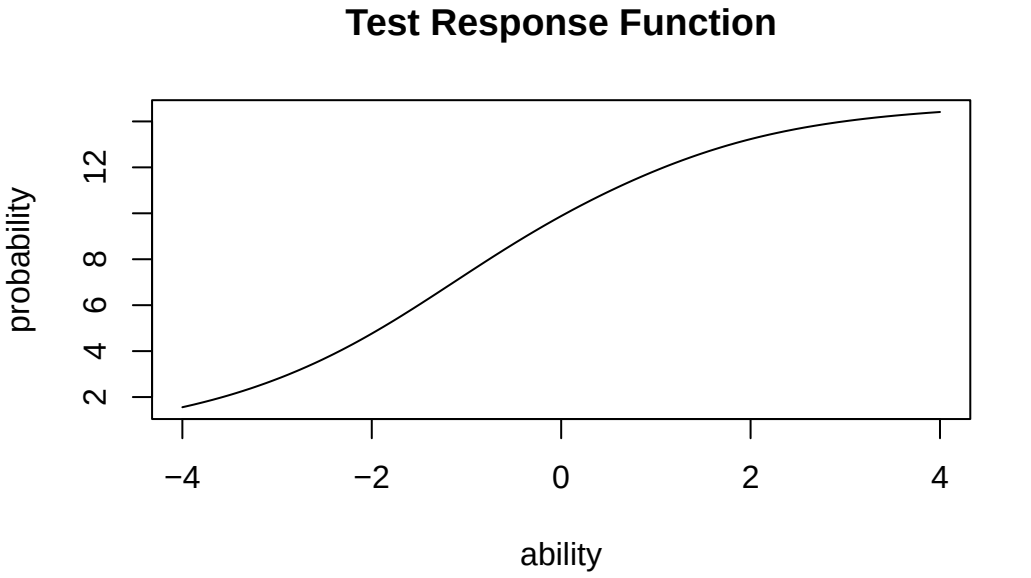
IRT の良さは，こうした数値的特徴というよりも，項目分析の時における IRT の可視化のしやすさにあると言えるだろう。`exametrika` パッケージでは，`plot` 関数を用いて項目特性曲線を描画することができる。

```
plot(result.2pl, item = 1:5, type = "IRF", overlay = TRUE)
```



また，IRF 関数をテストの全項目に対して加算したテスト反応関数 (Test Response Function) を描画することもできる。

```
plot(result.2pl, type = "TRF")
```

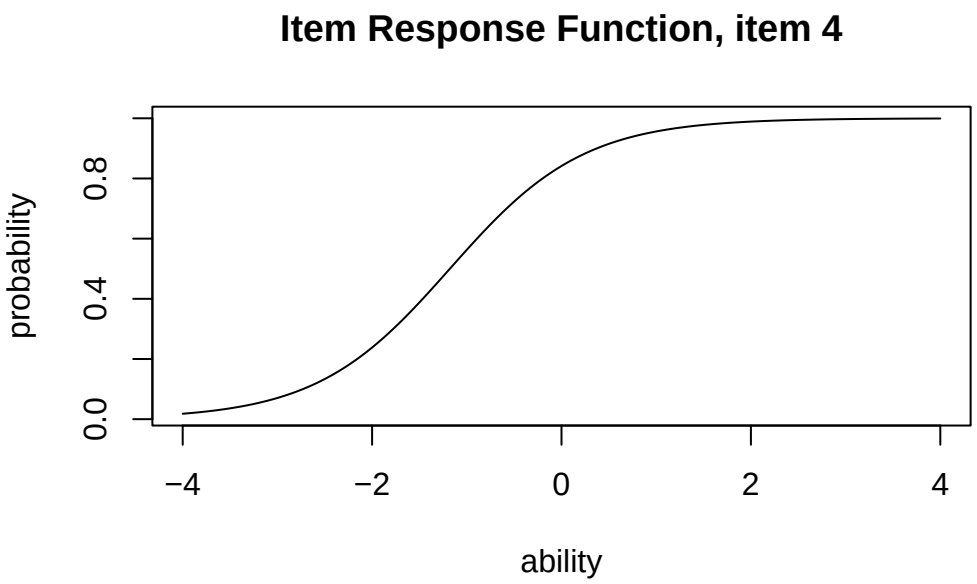


さらに項目反応関数を変換した**項目情報関数** (Item Information Function) を描画することもできる。項目情報関数は，その項目において最も分散が大きくなるところ，すなわち $\theta = 0.5$ をピークにする関数であり，以下のように定義される。

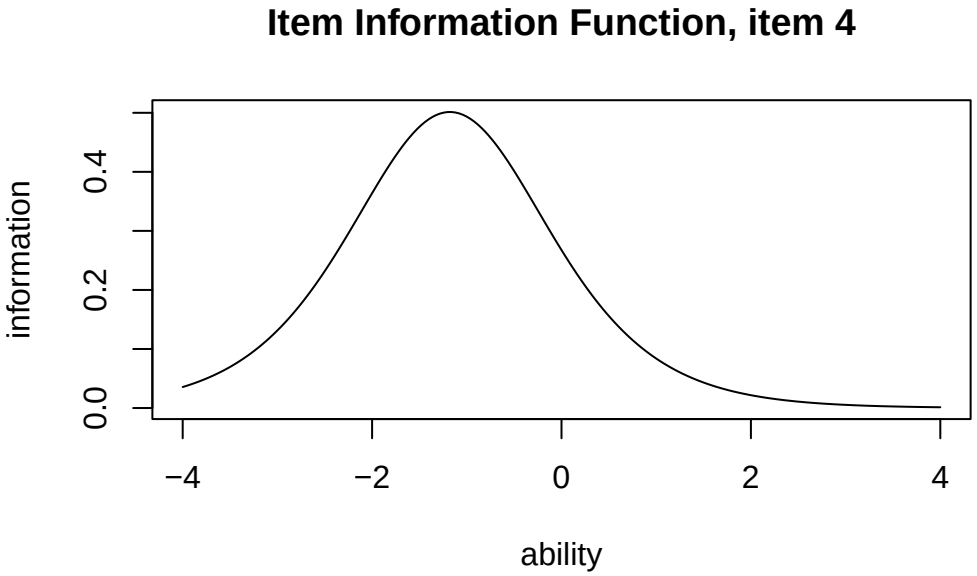
$$I_j(\theta) = \frac{P_j'(\theta)^2}{P_j(\theta)(1 - P_j(\theta))}$$

要するに，正答と誤答の確率の差が大きいほど，その項目の情報量が大きくなるということである。この関数を `exametrika` パッケージでプロットするには次のように `type = "IIF"`と指定する。

```
plot(result.2pl, item = 4, type = "IRF")
```




```
plot(result.2pl, item = 4, type = "IIF")
```



IIF が示すのは，項目反応理論における信頼性の概念であるとも言える。すなわち，IRT においては信頼性が θ の関数として表現され，どの領域匂いってその項目が最も効率的に機能するかを評価するのである。

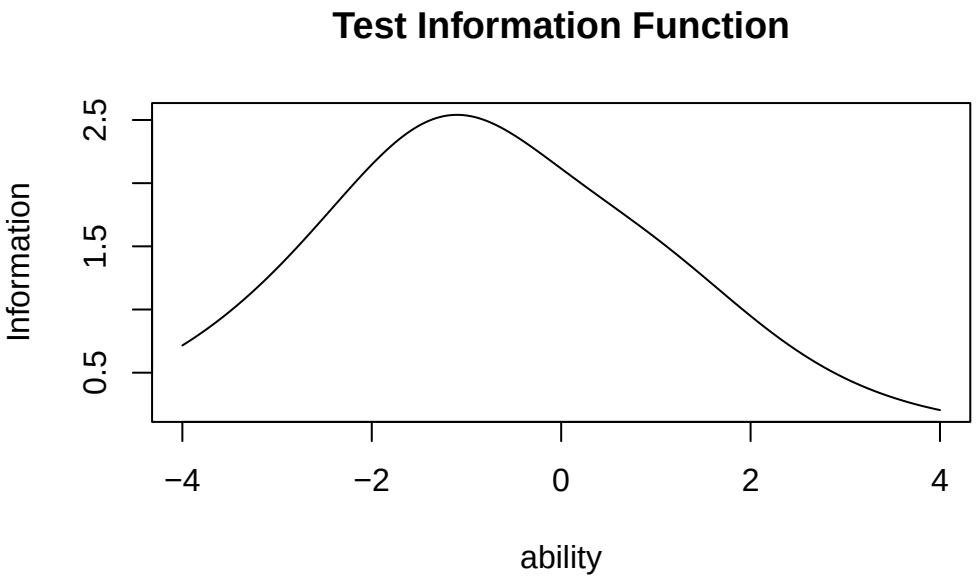
確認しておくと，古典的テスト理論においては，テストの全体に対する真分散の割合で信頼性をとらえていたのであった。また因子分析においては項目における共通性 h_j^2 で信頼性をとらえていた。つまりテスト全体から各項目へと進んで行ったのだが，現代テスト理論においては関数・項目の機能性を評価するようにと発展してきたのである。

すでに述べたように，IRT の観点からは，難易度が高すぎる・低すぎる項目であっても，削除するようなことはない。そうした項目は，高い能力・低い能力を査定する時に必要なのである。実践に際してそうした項目は大きな分

散を持ち得ないから，共通性も低くなりがちであるが，だからと言ってそうした項目を削除するようなことはしない。この辺りに，テスト理論と因子分析との思想的な違いがあると言える。

テスト全体の情報関数は，テストに含まれる項目情報関数の総和で表現される。この関数を `exametrika` パッケージでプロットするには次のように `type = "TIF"`と指定する。

```
plot(result.2pl, type = "TIF")
```



これを見ると，この 15 項目からなるテストは全体として $\theta = -1$ のあたりをピークとしており，相対的にやや θ が低い受験者に対して精緻な情報を提供するようになっていることがわかる。事前に項目母数がわかっている多くの項目プールがあり，それらを組み合わせてテストを作成する場合には，このような情報関数を用いて事前にテストの精度をデザインして適用することができる。

被験者母数 θ の推定については，受験者の回答パターンから推定される。一般に標準正規分布を事前分布としたベイズ推定が用いられる[*7]。`exametrika` パッケージでは，分析と同時に被験者母数の推定も行われている。

```
head(result.2pl$ability)
```

```
                   ID          EAP       PSD
Student001 Student001 -0.66529696 0.5457069
Student002 Student002 -0.14923429 0.5627007
Student003 Student003  0.01293751 0.5699788
Student004 Student004  0.58723568 0.6012928
Student005 Student005 -0.97854256 0.5415545
Student006 Student006  0.85828059 0.6187242
```

[*7] このカバーストーリーとモデルはリー・ワゲンメーカーズ（2017）の「飛行機を再捕獲する」P.63–66 を参考に作ったものである。なおこの本の Stan コードはこちらにあるので参考にしてほしい。書き写すだけでもかなりの勉強になることは間違いない。

15.3.2　IRT モデルの展開

IRT モデルは基本的にバイナリデータに対するモデルであるが，多段階反応，多値反応に対するモデルも提案されている。たとえばリッカート尺度のような多段階反応に対しては，**段階反応モデル** (Graded Response Model) や**部分採点モデル** (Partial Credit Model) が提案されている。これらを用いることの利点は，心理尺度データに対して順序尺度水準を仮定できるところにある。

心理学では基本的に，段階評定はせいぜい順序尺度水準の精度しか持ち得ないと考えられていながら，その数学的な利便性から (あるいはそこまでの精度がないとそもそも信用されていなかったから)，間隔尺度水準とみなしてピアソンの積率相関係数を算出し，一般的な因子分析を行ってきた。こうした「みなし」が必要だった理由のひとつが，統計パッケージに GRM や PCM が実装されていなかったからである。昨今では，GRM と 2PL モデルとの数学的等価性から同じ潜在変数モデルとして推定する統計ソフトウェア (Mplus など) もあるし，R には `ltm` パッケージなど GRM，PCM を提供するものもある。つまり，ツールがないからという言い訳はもう通用しない時代である。

多段階モデルで分析できるさらなる利点は，適切な反応段階を考えられる点である。リッカート法といえば 5 件法，7 件法であるというのが一般的に考えられているが，このことに特段の理論的根拠はない。それよりも，回答者が 5，7 段階のカテゴリ反応をしっかりと弁別できるのかどうかを考えるべきである[*8]。

具体例で見てみよう。`ltm` パッケージの `grm` 関数を用いて，サンプルデータ `Science` を分析してみる。このデータは科学態度に対するデータであり，4 段階評定になっている。

```
pacman::p_load(ltm)
data(Science)
result.grm <- grm(Science)
print(result.grm)
```

```

Call:
grm(data = Science)

Coefficients:
             Extrmt1  Extrmt2  Extrmt3  Dscrmn
Comfort      -10.768   -5.645    3.097   0.411
Environment   -2.154   -0.790    0.627   1.570
Work           32.102    9.261  -24.402  -0.074
Future        -30.602  -11.806   10.455   0.108
Technology     -2.462   -0.885    0.642   1.650
```

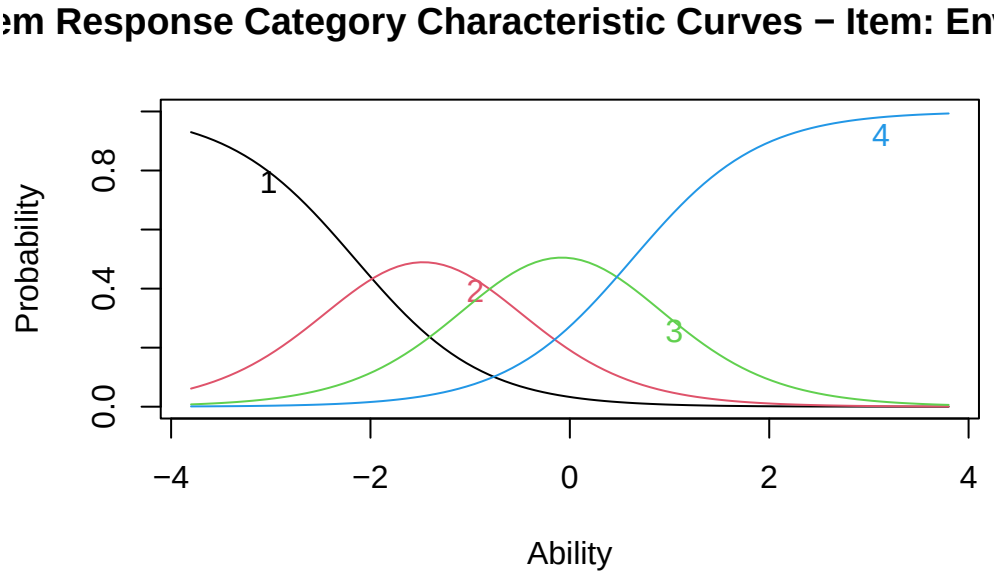
[*8] ここでは Stan の Hypergeometric 関数の書き方に従った。テキストによっては (例えばリー・ワゲンメーカーズ（2017）)`Hypergeometric(n,x,t)` とかかれることもあるが，Stan では `k \sim Hypergeometric(n, a, b)` で，`n` が 2 回目のサンプルサイズを指し，`a` が 1 回目のサンプルサイズ，`b` が 1 回目のサンプルで母集団から取り出せなかった数を表す。母集団全体の大きさは `a+b` であり，ここでの表記では `x + t-x` なので `t` である。

```
Industry      -2.870   -1.529    0.286   1.642
Benefit      -21.232   -5.982   10.297   0.136

Log.Lik: -2998.129
```

結果で示されているのは 3 つの閾値 (Extrmt) と，それぞれの閾値に対する識別力 (Dcrmn) である。段階反応モデルも，ロジスティックモデル同様 IRF，IIF，TIF を描画できるから，IRF(`ltm` パッケージでは `ICC` という引数を用いる) を描いてみよう。

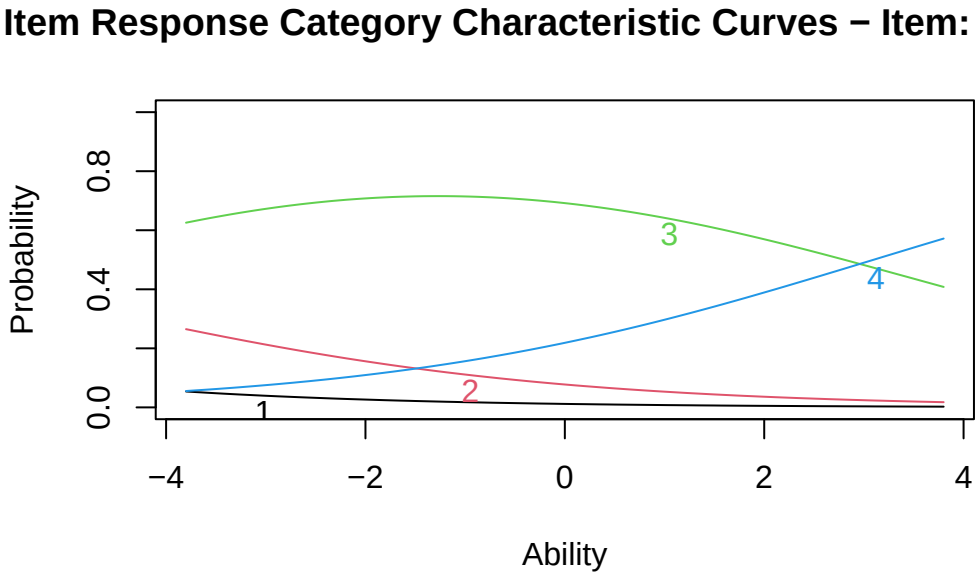
```
plot(result.grm, items = 2, type = "ICC")
```



この図には各カテゴリに対する反応確率が，θ の関数として表示されている。これを見ると，科学的態度の θ が高くなるにつれて，回答する確率が最も高いカテゴリが 1 から 2，2 から 3 へと変わっていくことが見て取れる。

しかし次の項目はどうだろうか。

```
plot(result.grm, items = 1, type = "ICC")
```



これを見ると，反応カテゴリ 1，2 のピークが存在せず，ほとんど 3 の反応で被覆されており，$\theta = 3$ を超えたところでやっと反応カテゴリ 4 がでてくることになる。プロットは $-4 \leq \theta \leq +4$ の範囲であるから，この幅を負の方向に広げればピークが出てくるのかもしれないが，実際的ではない。データによってはカテゴリ反応のピークが出てこないものもあり，そうしたものは適切な反応段階の設計になっていないことが疑われる。

昨今では心理尺度の設計に対しても，IRT のアプローチを取ることが推奨されている。因子分析アプローチとのもう一つの違いである，単因子を仮定する点についても拡張され，多次元 IRT モデルも提案されている (`mirt` パッケージなどが提供されている)。もはや，IRT のアプローチを取らない理由がないのである。

15.4　まとめ

ここでは探索的因子分析，検証的因子分析 (構造方程式モデリング)，項目反応理論のそれぞれについて，その基本的な考え方と実践的な使い方を紹介した。

これらに共通する考え方は，分散共分散行列あるいは相関行列をもとに，潜在変数を仮定した測定モデルを構築するという点である。相関行列が順序尺度水準に対するテトラコリック/ポリコリック/ポリシリアル相関係数であれば，カテゴリカルな因子分析をしていることになるし，それは (多段階) 項目反応理論をしていることでもある。つまり尺度水準に対応した相関係数が算出できるアプリケーションであれば，モデルの適用は同じ手順で行うことができる (`lavaan` でも変数の尺度水準の設定ができる。そのほかのアプリケーションとしては Mplus が有名である)。

それぞれのモデルの持つ歴史的背景や理論的系譜を知ることは，モデルの適用に有益な知見をもたらすが，ユーザ視点でいえば使えるものはなんでも使うべきであり，とくに調査協力者 (=回答者，受験生) の回答のしやすさといった観点から調査研究を設計することが肝要であろう。数学的限界やソフトウェアの都合によって，ましてや研究者の無理解や怠慢によって，回答しにくい調査デザインを適用することは，調査研究の質を低下させることになることを忘れてはならない。

15.5 課題

このチャプターで学んだ多変量解析の手法について、以下の課題に取り組んでください。各課題は、実践的なデータ分析を通じて、理論と実践の両面から理解を深めることを目的としています。例として，psych パッケージの small.msq データセット (気分状態質問紙) を使用します。このデータセットは 14 の変数，200 ケースです。[*9]

14 の変数は，エネルギー的覚醒状態変数 (活動的 active, 注意深さ alert，覚醒 arousal，ねむさ sleepy，疲れ tired，ウトウト drowsy)，緊張的覚醒変数 (不安 anxious，落ち着かない jittery，神経質な nervous，穏やかな calm，リラックスした relaxed，気楽な at.ease) と，性別 gender，薬物条件 drug からなります。性別と薬物条件を除いた 2 因子構造であると考えられ,

15.5.1 課題 1：探索的因子分析の実践

探索的因子分析を実施してください。

分析の手順：

1. 因子数の決定
2. 因子分析の実行 (斜交回転)
3. 結果の解釈（因子負荷量、共通性、独自性）

15.5.2 課題 2：確認的因子分析の実践

lavaan パッケージを使って，2 因子モデルを推定してください。モデル指定の際に，ordered = TRUE オプションを入れることで，観測変数を順序尺度水準として推定します。

分析の手順：

1. 理論的モデルの構築 (パス図の作成)
2. lavaan パッケージを用いたモデルの推定
3. モデルのプロット
4. 適合度指標の評価
5. モデルの修正 (必要な場合)

15.5.3 課題 3：多段階項目反応理論の実践

ltm パッケージの段階反応モデル (GRM) を適用してください。一次元性を仮定したモデルなので，エネルギー的覚醒項目セット，緊張的覚醒項目セットに分け，それぞれに GRM モデルを適用します。

[*9] これは各ケースが独立に同じ分布 (独立同分布，independent and identically distributed,i.i.d.) から得られているという仮定が成り立つときになされる計算である。サイコロの目が 2 回連続で 1 が出る場合などは，$1/6 \times 1/6$ と計算するが，これは 1/6 の確率が独立しているから掛け合わせよう，ということに他ならない。

分析の手順：

1. データの分割
2. GRM の適用
3. 項目特性曲線の描画
4. 項目情報関数の描画

15.5.4　課題 4: 多次元多段階項目反応理論の実践

`mirt` パッケージ (Multidimensional IRT) を用いて，多次元モデルを実行します。コードは次のようになります。

```
pacman::p_load(mirt)
# 2因子(model = 2)，段階反応(itemtype = 'graded')を指定
result.mirt <- mirt(dat, model = 2, itemtype = "graded")
# 出力に際して斜交回転
summary(result.mirt, rotate = "geominQ")

# 多次元ICCの描画
plot(result.mirt, type = "trace", which.items = 1)
# 多次元IICの描画
plot(result.mirt, type = "info")
```

第 16 章

多変量解析 (その 2)

先の章では分散共分散行列 (相関行列) に基づいた線型モデルを中心に紹介した。しかし，多変量解析はそれだけではない。むしろ測定モデルとして潜在変数を仮定する手法だけに固執するあまり，心理測定の仮定に違反するようなデータであっても因子分析を適用したり，モデルの適合度を優先しすぎて不自然な設定に走ったりするような誤用が多く見られる。

心理学においては因子分析が構成概念を測定していると「純粋に」信じられて多用されてきたが，かつてはさまざまな多変量解析技法が必要に応じて開発，使用されていたのである。ここでは分析のスタートになる行列の種類で区分し，いくつかの多変量解析モデルを紹介する。

16.1 距離行列を用いるもの

距離とは，次の四つの公理を満たす数字のことをいう。

1. **非負性**: $d(x, y) \geq 0$ (距離は 0 以上)
2. **同一性**: $d(x, y) = 0 \Leftrightarrow x = y$ (距離が 0 なのは同じ点の場合のみ)
3. **対称性**: $d(x, y) = d(y, x)$ (距離は方向に依存しない)
4. **三角不等式**: $d(x, z) \leq d(x, y) + d(y, z)$ (遠回りは最短距離以上)

距離行列とは行列の要素が距離を表しているもので，一般に正方・対称行列になる。この点は分散共分散行列や相関行列と同じで，この行列演算によって分散共分散を用いたモデルとは別の解釈が成立する分析を作ることができる。

ちなみに R では距離行列を作るのに `dist` 関数を用いる。オプションとして特段の指定がなければユークリッド距離が用いられるが，他にも以下のようなオプションがある。

- `"euclidean"`: ユークリッド距離。$d(x, y) = \sqrt{\sum_{i=1}^{N}(x_i - y_i)^2}$ で表される。
- `"maximum"`: チェビシェフ距離。$d(x, y) = max(|x_i - y_i|)$ で表される。
- `"manhattan"`: マンハッタン距離。$d(x, y) = \sum(|x_i - y_i|)$ で表される。
- `"canberra"`: キャンベラ距離。$d(x, y) = \sum \frac{|x_i - y_i|}{|x_i + y_i|}$ で表される
- `"binary"`: バイナリ距離。ジャッカード距離ともいう。0/1 のデータに対する距離で，$d(x, y) = \frac{b+c}{a+b+c+d}$

で表される (a は両方 1，b は x が 1 で y が 0，c は x が 0 で y が 1，d は両方 0)。両方に共通して 1 である要素が多いほど距離が小さくなる。

- `"minkowski"`: ミンコフスキー距離。一般化された距離とも言われ，係数 p でさまざまな距離を表現できる。$d(x,y) = (\sum |x_i - y_i|^p)^{\frac{1}{p}}$ で表される。例えば $p = 1$ ならばマンハッタン距離，$p = 2$ ならユークリッド距離である。$p = \infty$ の時を特にチェビシェフの距離，または優勢次元距離という。

16.1.1　心理学における距離データ

数字を何とみなすか，によって心理学でも距離データを扱うことはできる。尺度評定の差分を (得点間の) 距離とみなすこともできるし，相関係数も $1.0 - |r_{jk}|$ のようにすれば距離とみなすことができる。社会心理学におけるソシオメトリックデータは，対人関係の選好評定だが，これも対人間の距離とみなすことができるだろう。実験心理学における刺激の混同率や汎化勾配，2 つの刺激が同じか違うかを判断する課題への反応潜時，刺激の代替価・連想価は類似性と考えられるから，これも距離データと言えるだろう (高根，1980)。距離行列は一個体からの評定や反応からでも生成できるから，小サンプルの実験計画であっても距離行列を得ることができる。

このように，類似性あるいは非類似性を距離と見做して用いることができる。この利点は，回答者の自然な判断に任せられること，つまり「総合的に判断して，似ているか，似ていないか」といった回答をデータにできることである。研究者はついつい複数の類似した項目で多角的に聞かねばならない，と思いがちだが，下位の評定次元を実験者が準備することは回答者の自由度を束縛している側面もあり，また回答者の負担を考えると必ずしもいいことばかりではない。さらに項目によっては社会的な望ましさバイアスなども含まれるから，「総合的に評価してもらいたい」というのはそういったバイアスから逃れられる側面もある。

距離を対象に考える多変量解析モデルも，そのほかのモデルと同様，要約や分類を目的にしている。また，多次元データを少数の次元に要約するために可視化する手法として用いられることもある。まずは分類を目的としたモデルから見ていこう。

16.1.2　クラスター分析

クラスター分析は類似したものをまとめてクラスター (塊) を形成する分析方法である。クラスター分析の中でも多くの手法・モデルが考えられており，いくつかの側面から分類することができる。

16.1.2.1　モデルの階層性

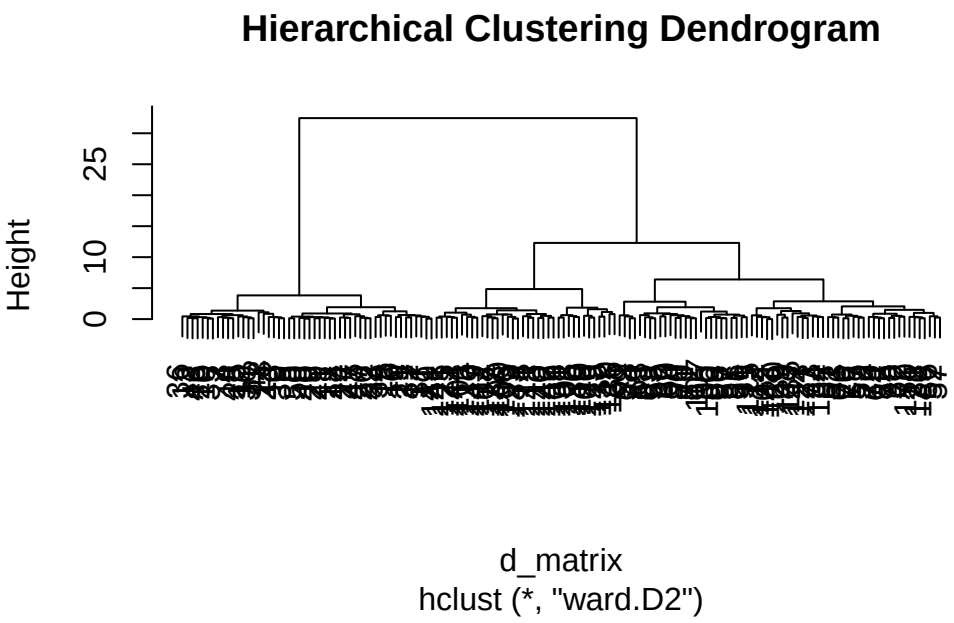
■16.1.2.1.1　階層的クラスター分析

まずはクラスターが階層性を持つかどうか。階層的クラスター分析は距離の短いものから順にまとめていき，クラスターのクラスター，クラスターのクラスターのクラスター，といったように順次大きなグループにまとめ上げていく。

結果は**デンドログラム**と呼ばれるツリー状のプロットで表されることが一般的で，適当なところで分割して利用する。適切なクラスター数に関する一般的な基準はほとんどなく，実用性に応じてツリーをカットすることが多い。

`iris` データによる実行例を示す。

```
data(iris)
d_matrix <- dist(iris[, -5], method = "euclidean")
result.h <- hclust(d_matrix, method = "ward.D2")
plot(result.h, main = "Hierarchical Clustering Dendrogram")
```



階層的クラスター分析のクラスター同士を上位クラスターにまとめていく方法がいくつか考案されており，R では hclust 関数の method オプションで指定することができる。

- "ward.D" / "ward.D2": ウォード法。クラスター内の分散を最小化する
- "single": 最短距離法。クラスター間の最短距離でリンク
- "complete": 最長距離法。クラスター間の最長距離でリンク
- "average": 平均法 (群平均法)。クラスター間の平均距離でリンク
- "mcquitty": McQuitty 法。重み付き群平均法の一種
- "median": メディアン法。重み付き群中心法の一種
- "centroid": 重心法。

最もよく使われるのがウォード法で，実践的にもこの手法による分類が最も解釈しやすい。なお，ward.D オプションはバグがあるので用いないことが望ましく，バグを修正した ward.D2 を用いること[*1]。

得られたクラスターの結果は cutree 関数で任意のクラスター数に分割できる。今回，iris データは 3 種類の iris があることがわかっているので，クラスター数 3 にしてその分類精度を確認してみよう。

```
clusters <- cutree(result.h, k = 3)
table(clusters, iris$Species)
```

[*1] エディタがその言語に対応していたら，おかしな記述に下線が引かれるなど注意を促してくる機能もある。また RStudio で Stan 言語を書いていると，コンパイルの前に文法のチェックをする機能もある。

```
clusters setosa versicolor virginica
       1     50          0         0
       2      0         49        15
       3      0          1        35
```

■16.1.2.1.2　非階層的クラスター分析

非階層的クラスター分析として有名なのは，k-means 法による分類である。アルゴリズムは次のとおりである。

1. 指定されたクラスター数の重心ベクトルをランダムに生成する。
2. 各データ点を最も近い重心のクラスターに所属させる。
3. クラスターごとに，重心を再計算する。
4. 2.-3. のステップを繰り返し，変動がなくなると推定終了とする。

この方法は任意のクラス数に分類できること，大規模なデータであっても比較的早く収束することが利点である。R によるサンプルは以下のとおりである。

```
result.k <- kmeans(d_matrix, centers = 3)
table(result.k$cluster, iris$Species)
```

```
   setosa versicolor virginica
1      23          0         0
2       0         49        50
3      27          1         0
```

16.1.2.2　クラスタリングの境界

ここまでのクラスタリングは，各個体がどのクラスターに所属するかが明確に定まっていたが，境界がそこまで明確でない中間的なデータ点もあるかもしれない。各データ点が 1 つのクラスターにのみ所属するという明確なクラスタリングのことを，ハードクラスタリングとかクリスプクラスタリングという。これに対して，各データ点が複数のクラスターに部分的に所属している，あるいは所属度が例えば 0-1 などの連続値で表現されるような，緩やかな所属をゆるするクラスタリングもあり，これらを総称してファジィクラスタリングとかソフトクラスタリングと呼ぶ。

ファジィクラスタリングの例として，fuzzy c-means 法を挙げる。

```
pacman::p_load(e1071)
result.c <- cmeans(d_matrix, centers = 3, m = 2)
head(result.c$membership)
```

```
          1           2           3
1 0.9961597 0.002333733 0.001506532
2 0.9942338 0.003508504 0.002257731
3 0.9893045 0.006430515 0.004265034
```

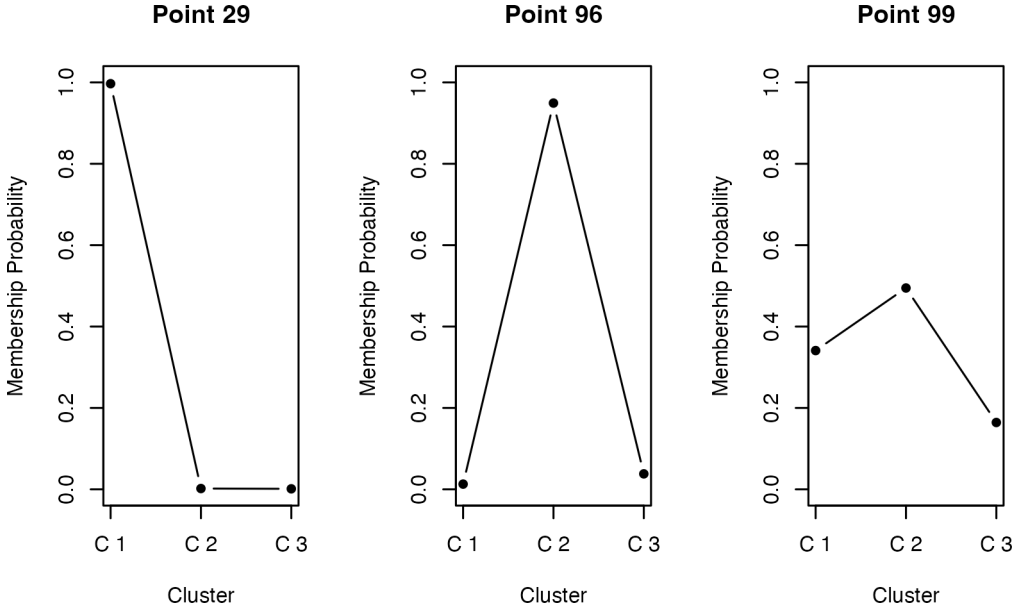
```
4 0.9916600 0.005064746 0.003275304
5 0.9959249 0.002468187 0.001606896
6 0.9753719 0.015283915 0.009344178
```

```
table(result.c$cluster, iris$Species)
```

```
   setosa versicolor virginica
1      50          0         0
2       0         48        13
3       0          2        37
```

fuzzy c-means は e1071 パッケージに含まれている。モデルの指定の時に，ファジィ度パラメータ m を指定する。通常 1.5 から 3 程度で，大きいほど曖昧さをゆるす。出力として membership という所属確率が返される。この所属確率が最大のものをハードな分類として使用することができる。

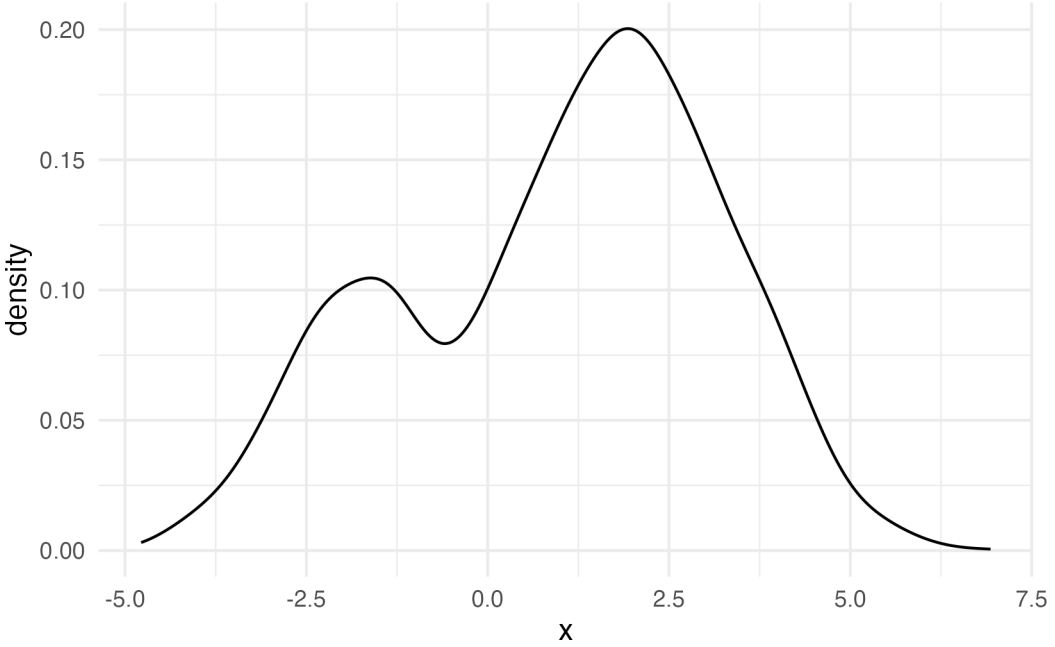
membership をプロットしてみると，明らかに所属するクラスが明確なものと，曖昧なデータもあることがわかる。心理学的応用としては，パーソナリティの分類や症状の分類などが考えられるだろう。



16.1.2.3　モデルベースか否か

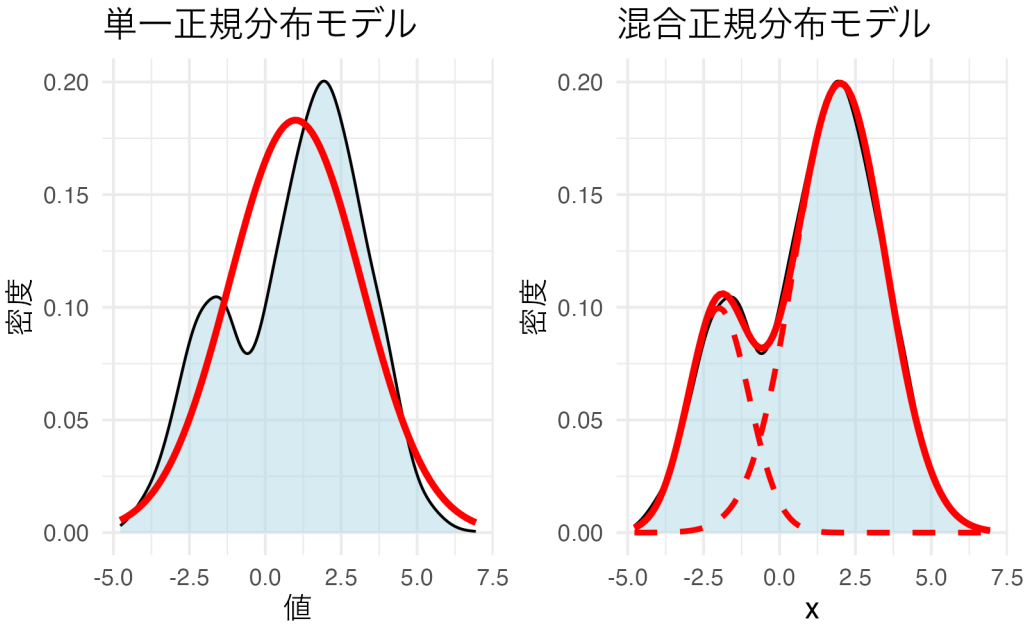
階層的クラスタリングや，k-means, fuzzy c-means の非階層的クラスタリングモデルでは，クラスター数の決定について客観的な指標がなかった。そこで，確率モデルとしてクラスター分析を考え，モデル適合度の観点から評価することを考える。

ある変数についてヒストグラムを描き，次のような出力を得たとしよう。



このようなデータに対して，正規分布モデルを当てはめるのは適切だろうか。正規分布は単峰で左右対称であることが特徴だから，無理やり当てはめるとおかしなことになるだろう。ここには隠れた二つの正規分布があると考え，それぞれが混ざり合って出てきたものと考えたほうが良い。

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```



統計モデルとしては，混合正規分布モデル (Gaussian Mixture Model, GMM) と呼ばれるのだが，これは異なる 2 つの群を生成モデルとしているクラスター分析であると考えることもできる。GMM では観測されたデータが複数の正規分布を含んでいると考え，各群に想定される正規分布の平均，分散および群の混合率を推定することでデータの潜在的な特徴を明らかにする。今回は簡便のために 1 変数でのモデルにしたが，複数の変数がある場合

は多変量正規分布で考えることになる。

確率モデルになっているので，尤度を用いてデータとの適合度を計算することができる。潜在的な分布がいくつあるのかを BIC などを基準に選定することで，客観的にクラス数を決定することができるのが利点である。

パッケージを使って具体的なデータを分類してみよう。

```
pacman::p_load(mclust)

# irisデータから数値変数のみを取得
iris_data <- iris[, 1:4]

# mclustによるクラスタリング
gmm_result <- Mclust(iris_data)

# 結果の表示
summary(gmm_result, parameters = TRUE)
```

```
----------------------------------------------------
Gaussian finite mixture model fitted by EM algorithm
----------------------------------------------------

Mclust VEV (ellipsoidal, equal shape) model with 2 components:

 log-likelihood   n df       BIC       ICL
       -215.726 150 26 -561.7285 -561.7289

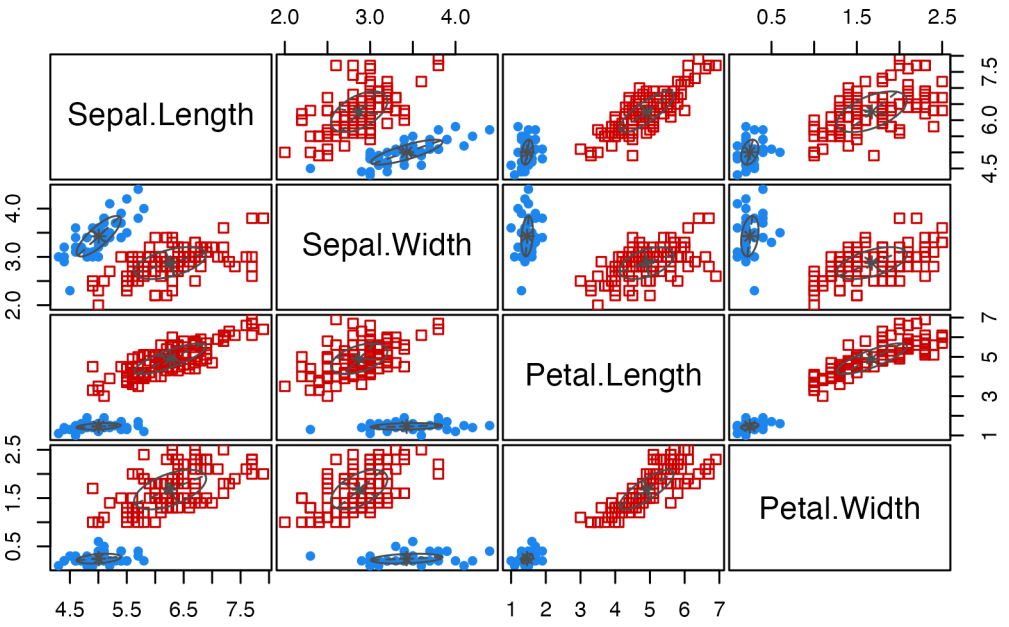
Clustering table:
  1   2
 50 100

Mixing probabilities:
        1         2
0.3333319 0.6666681

Means:
                  [,1]     [,2]
Sepal.Length 5.0060022 6.261996
Sepal.Width  3.4280049 2.871999
Petal.Length 1.4620007 4.905992
Petal.Width  0.2459998 1.675997
```

```
Variances:
[,,1]
             Sepal.Length Sepal.Width Petal.Length Petal.Width
Sepal.Length   0.15065114  0.13080115   0.02084463  0.01309107
Sepal.Width    0.13080115  0.17604529   0.01603245  0.01221458
Petal.Length   0.02084463  0.01603245   0.02808260  0.00601568
Petal.Width    0.01309107  0.01221458   0.00601568  0.01042365
[,,2]
             Sepal.Length Sepal.Width Petal.Length Petal.Width
Sepal.Length    0.4000438  0.10865444    0.3994018  0.14368256
Sepal.Width     0.1086544  0.10928077    0.1238904  0.07284384
Petal.Length    0.3994018  0.12389040    0.6109024  0.25738990
Petal.Width     0.1436826  0.07284384    0.2573899  0.16808182
```

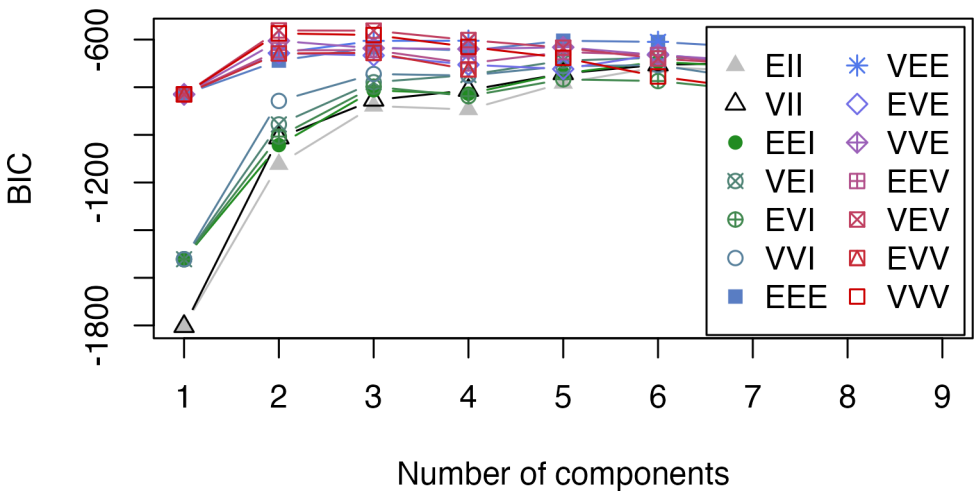
```r
# 分類結果の可視化
plot(gmm_result, what = "classification")
```




```r
# 真の種と比較
table(iris$Species, gmm_result$classification)
```

```
            1  2
  setosa     50  0
  versicolor  0 50
  virginica   0 50
```

```
# BICによるモデル選択結果
plot(gmm_result, what = "BIC")
```



mclust では，各クラスターの共分散行列の構造によって異なるモデルが考慮される。モデル名は 3 文字のコードで表現され，それぞれの文字が以下の意味を持つ：

- **1 文字目 (Volume)**: 各クラスターの大きさ (体積)
  - E = Equal(等しい)
  - V = Variable(異なる)
- **2 文字目 (Shape)**: 各クラスターの形状
  - E = Equal(等しい)
  - V = Variable(異なる)
- **3 文字目 (Orientation)**: 各クラスターの向き
  - E = Equal(等しい)
  - V = Variable(異なる)
  - I = Identity(単位行列，球形)

例えば：- **EII**: 全クラスターが同じ大きさの球形 (等分散球形) - **VII**: 各クラスターが異なる大きさの球形 (異分散球形) - **EEE**: 全クラスターが同じ大きさ・形状・向きの楕円 - **VVV**: 各クラスターが異なる大きさ・形状・向きの楕円 (最も一般的)

これらの組み合わせにより，14 種類の共分散構造から最適モデルを BIC を参考に自動的に選定される。今回は VEV (ellipsoidal, equal shape) の 2 クラスターモデルが最適として判断された (実際は 3 種あるが，データからは 2 種類が最適と判断されている)。

潜在的に分類する手法は，例えばマーケティング業界では購買層を探索的に見出す手法として使われる。潜在的なクラスが順序尺度水準であることを想定すれば，潜在ランクモデルと呼ばれ，テスト理論の応用モデルとして提案されている。Shojima (2022) では IRT のような精緻な θ_i の推定よりも段階的な推定の方が実践的意義が高いことから，潜在ランクモデルを活用する利点が論じられている。

16.1.2.4　バイクラスタリング

Cattel のデータキューブのところで触れたように，Observation × Variables のデータセットがあったとき，変数の共変動から個人を分類することも，個人の共変動から変数を分類することもできる。そしてまた，両者を同時に分類するバイクラスタリング (Biclustering) という手法も提案されている。

Biclustering は Two-Mode Clustering とも呼ばれ，様々なモデルが提案されているが，ここでは Shojima (2022) のテスト理論の観点から見てみよう。

テスト理論はデータがバイナリであり，この分析の確率モデルとしてはベルヌーイ分布を置いた IRT が一般的である。しかし上で述べたように，IRT で推定されるような潜在得点 θ の精度は実質的な意味が見えにくいことがある。すなわち，θ が 0.01 ポイント違うことが，どのような違いに相当するのか。θ を 0.5 ポイント上昇させるために受検者はどのような努力をすれば良いのか。また実用上も，数段階の診断結果や，単純な合否の 2 段階に分割してのフィードバックをするのであれば，そこまで細かい分類は必要ないかもしれない。

Shojima (2022) のバイクラスタリングでは，テストデータにおける項目を複数のフィールドに，受検者を複数のクラスに分類する。受験者の分類は正答率に応じて順序づけることができ，ランクとして表現することができる (ランクで表現されるモデルは特にランクラスタリングとよばれる)。

荘島のバイクラスタリングモデルを形式化するために，主要な行列を定義しよう。J を項目数，S を受検者数，C を潜在クラス/ランク数，F を潜在フィールド数とする。

バイクラスター参照行列 Π_B は次のように定義される：

$$\Pi_B = \begin{bmatrix} \pi_{11} & \cdots & \pi_{1F} \\ \vdots & \ddots & \vdots \\ \pi_{C1} & \cdots & \pi_{CF} \end{bmatrix} = \left\{ \pi_{fc} \right\}$$

ここで各要素 π_{fc} は，クラス/ランク c の受検者がフィールド f の項目に正答する確率を表す。

クラス所属行列 $\mathbf{M}_C$ とフィールド所属行列 $\mathbf{M}_F$ は次のように定義される：

$$\mathbf{M}_C = \begin{bmatrix} m_{11} & \cdots & m_{1C} \\ \vdots & \ddots & \vdots \\ m_{S1} & \cdots & m_{SC} \end{bmatrix}, \quad \mathbf{M}_F = \begin{bmatrix} m_{11} & \cdots & m_{1F} \\ \vdots & \ddots & \vdots \\ m_{J1} & \cdots & m_{JF} \end{bmatrix}$$

ランククラスタリングにおけるランク所属行列 $\mathbf{M}_R$ は，クラス所属行列 $\mathbf{M}_C$ に，前後のクラスのつながりを緩やかに繋げるフィルター行列 $\mathbf{F}$ をかけることで得られる。

$$\mathbf{M}_R = \mathbf{M}_C \mathbf{F}$$

フィルタ行列は，例えばランク数が 6 の場合，次のような行列になる。

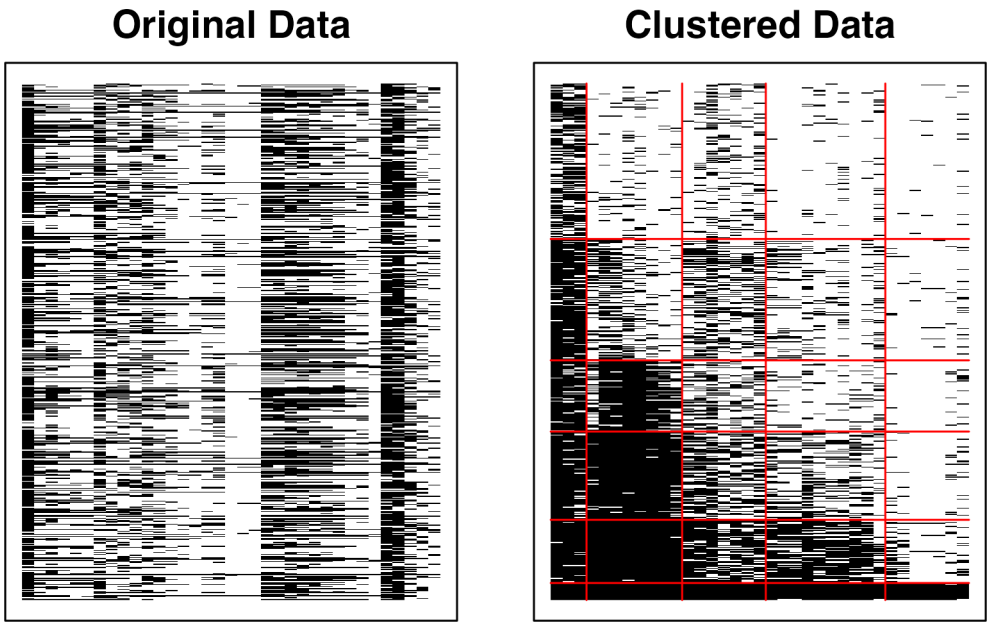
$$\mathbf{F} = \begin{bmatrix} 0.864 & 0.120 & & & & & \\ 0.136 & 0.760 & 0.120 & & & & \\ & 0.120 & 0.760 & 0.120 & & & \\ & & 0.120 & 0.760 & 0.120 & & \\ & & & 0.120 & 0.760 & 0.120 & \\ & & & & 0.120 & 0.760 & 0.136 \\ & & & & & 0.120 & 0.864 \end{bmatrix}$$

これらを踏まえて，尤度関数は次のように定義され，EM アルゴリズムによって推定される。

$$l(\mathbf{U} \mid \Pi_B) = \prod_{s=1}^{S} \prod_{j=1}^{J} \prod_{f=1}^{F} \prod_{c=1}^{C} \left(\pi_{fc}^{u_{sj}} (1 - \pi_{fc})^{1-u_{sj}} \right)^{z_{sj} m_{sc} m_{jf}}$$

これを実装したパッケージ `exametrika` とそのサンプルコードをみて，実践例を見てみよう。

```
pacman::p_load(exametrika)
result.Ranklustering <- Biclustering(J35S515,
  nfld = 5, ncls = 6,
  method = "R", verbose = F
)
plot(result.Ranklustering, type = "Array")
```



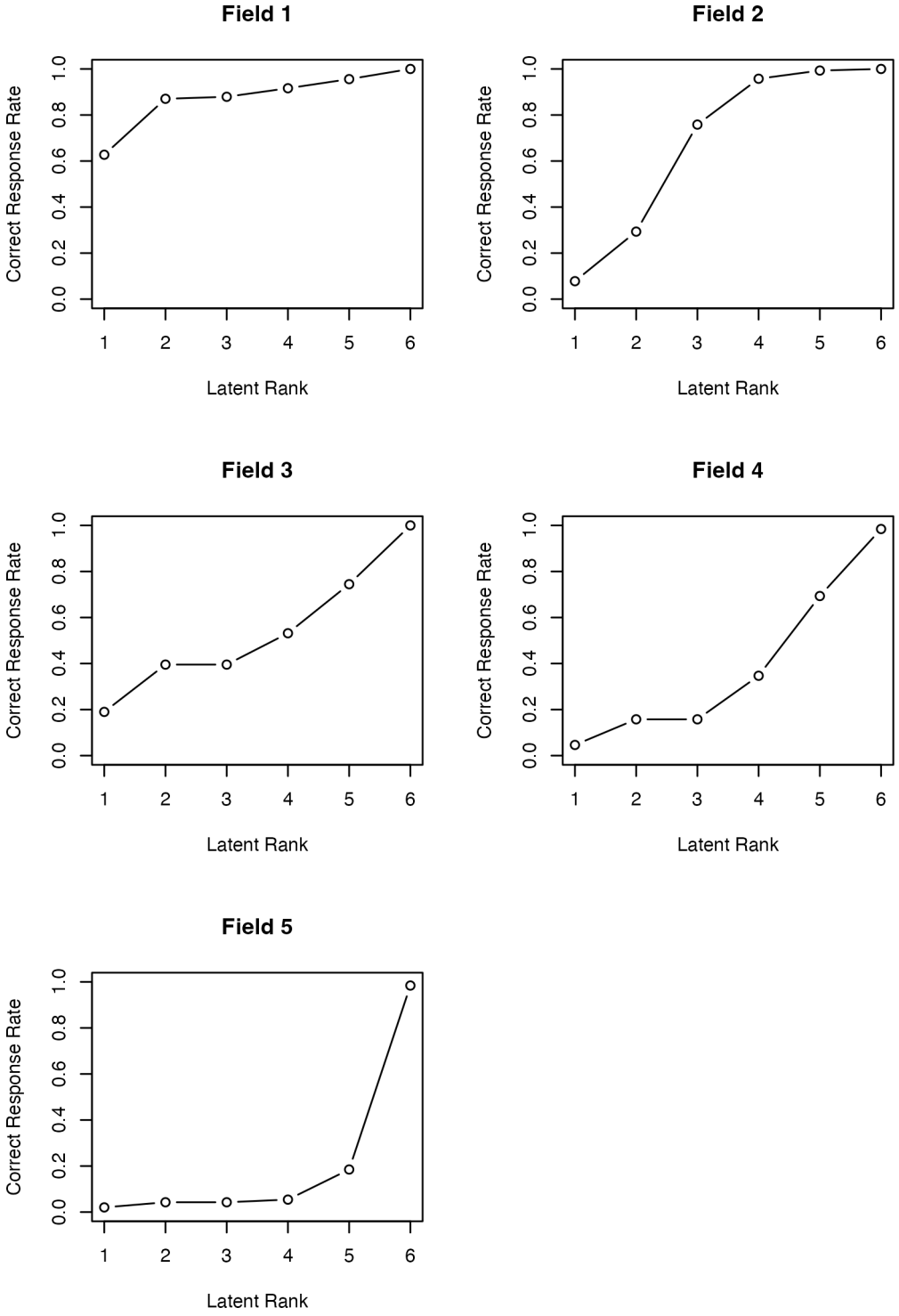
`exametrika` パッケージのバイクラスタリングは，引数にデータ，フィールド数 `nfld`，クラス (ランク) 数 `ncls`，および手法 (`B` ならバイクラスタリング，`R` ならランクラスタリング) をとる。ここではパッケージに含まれているサンプルデータ `J35S515` を使っているが，これは 35 項目からなるテストで 515 人の受検者からの回答を得たものである。

分析結果としてアレイプロットが示されている。この図の左は，行ごとに受検者，列ごとに項目からなり，正答を黒い四角 (■)，誤答を白い四角 (□) で表現したローデータである。右に示されているのは分析結果による同様の表示で，ランクごと，フィールドごとに類似したパターンがまとめられていることがわかる。

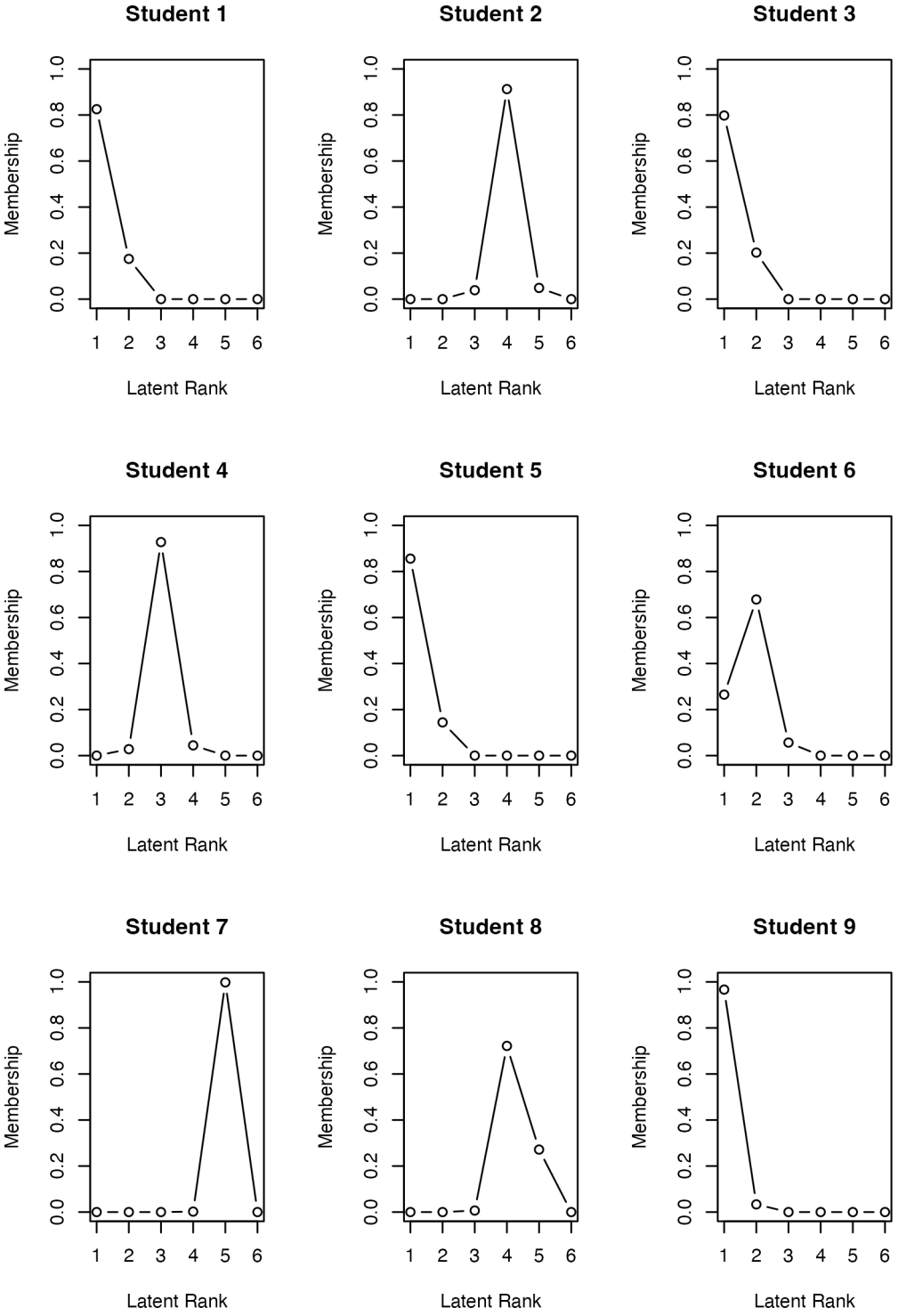
フィールドの分類とランクをみることで，受検者には次のランクに進むにはどの領域の項目に正答すれば良いか，といった情報を提供することができる。また，フィールドやランクへの所属は確率で表現され (ファジィクラスタリング)，受検者にはランクアップ Odds, ランクダウン Odds を提供することができる。

フィールド所属行列，ランク所属行列を可視化したプロファイルの出力を以下に示す。

```
plot(result.Ranklustering, type = "FRP", nc = 2, nr = 3)
```




```
plot(result.Ranklustering, type = "RMP", students = 1:9, nc = 3, nr = 3)
```



バイクラスタリングは多値モデルも開発されており，心理尺度の新しい分析手法として期待されている。というのも，クラスタリングは表層的な反応パターンによる分類で，因子分析法や IRT(GRM) のようなデータ生成メカニズムを仮定しないことから，潜在変数の意味解釈といった理論的問題を避けることができるからである。また，項目についてのクラスタリングは得点もしくは変化得点を質的な意味に割り当てることが容易であることもその理由として挙げられる。

なお，パッケージ `exametrika` には Shojima (2022) に含まれる 12 のモデル全てを実装している。詳しくはサイトを参照のこと。

16.1.3　多次元尺度構成法

多次元尺度構成法 (Multidimensional Scaling,MDS) は，一言で言えば距離行列から地図を復元する手法である。MDS は大きく分けて計量 (metric)MDS と非計量 (non-metric)MDS がある。前者は距離行列の固有値分解から得られる固有ベクトルを座標とみなす，ストレートな表現方法であり，データが比率尺度水準以上の誤差のない数値であることが求められる。これによって，どうしてそのような解釈が可能なのかについては，Young-Householder の定理という MDS の基礎になった数学定理があるので，興味がある人は調べて見てほしい。

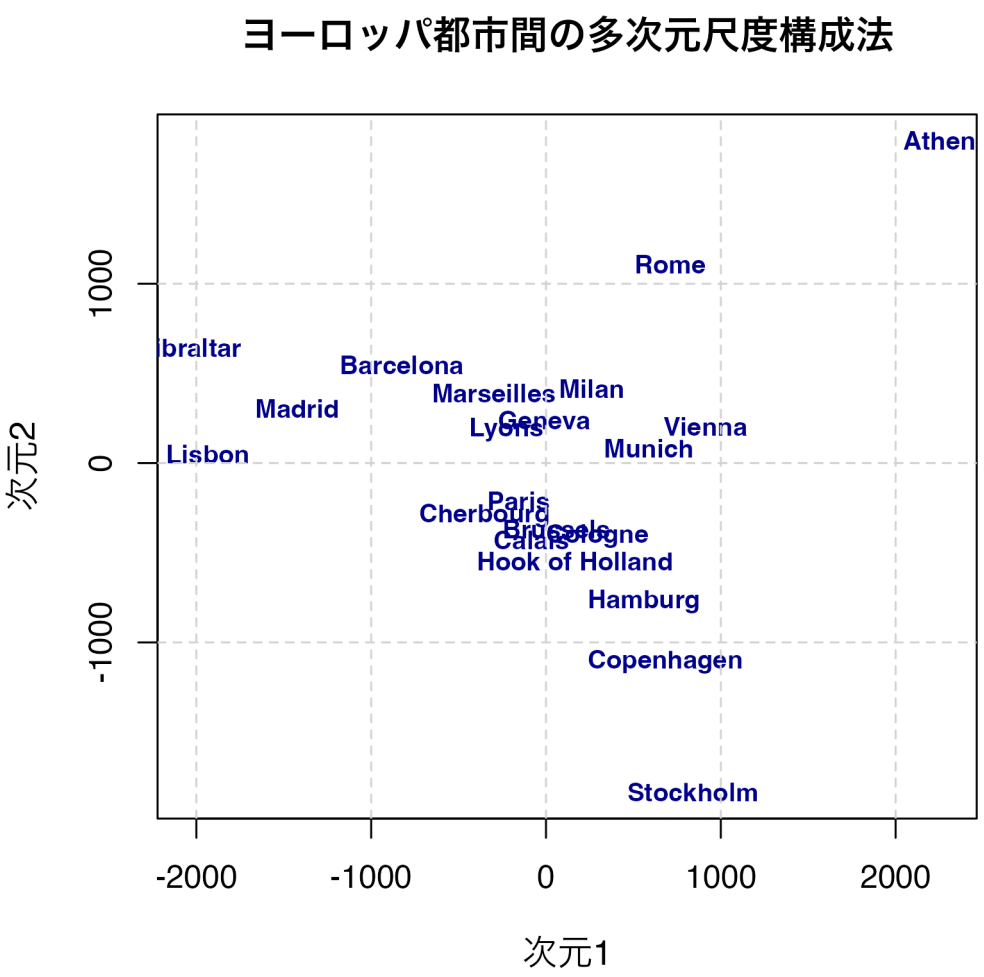
以下に計量 MDS の例を見てみよう。R は `eurodist` というヨーロッパ各都市の距離についてのサンプルデータを持っており，これを使って基本関数 `cmdscale` で MDS を実行してみよう。

```
result <- cmdscale(eurodist, k = 2)

# 美しいプロットの作成
plot(result[, 1], result[, 2],
  type = "n", # 点は描かずにプロット領域だけ作成
  xlab = "次元1", ylab = "次元2",
  main = "ヨーロッパ都市間の多次元尺度構成法",
  cex.main = 1.2,
  cex.lab = 1.1
)

# 都市名をラベルとして表示
text(result[, 1], result[, 2],
  labels = rownames(result),
  cex = 0.8,
  col = "darkblue",
  font = 2
)

# グリッドを追加
grid(lty = 2, col = "lightgray")
```



アテネ (Athens) が右上，ストックホルム (Stockholm) が下に来ていることから，南北が反転していると思われるが，それぞれの相対的な位置は大体復元できていることがわかるだろう。関数 `cmdscale` は引数 `k` で次元数を指定でき，今回は 2 次元を指定したが，実際は地球が球体だから `k=3` とすることが正しいが，事前知識がない場合は可視化のために低次元を指定することが一般的である。

`eudodist` は実際の距離データであるから，比率尺度水準の誤差のないデータと見做せるだろう。しかし，心理学的な応用場面においては，比率尺度水準のデータや誤差のないデータは考えにくい。このとき用いられるのが，非計量 MDS である。これを簡単に言えば，データの持つ大小関係を反映した多次元空間に対象を付置する (地図の座標を与える) ものである。すなわち，対象 i と j の距離を d_{ij} と表すとすると，

$$d_{ij} < d_{kl} \rightarrow \delta_{ij} < \delta_{kl}$$

という関係が保持されるような座標 δ_{ij} を求めるものである。非計量 MDS のはしりであった Kruscal の方法は，データとの適合を表す stress 値として，

$$\sum_{i>j} e_{ij}^2 = \sum_{ij} (\delta_{ij} - d_{ij})^2$$

という最適化関数を最小化するように座標を求める。最適化関数は多くの研究者によって様々に提唱されており，R では便利なパッケージ `smacof` のアルゴリズム (Scaling by Majorizing a COmplicated Function) による実行がいいだろう。

実例で見てみよう。smacof パッケージの持つ FaceExp データセットを用いる。これは顔表情の類似度評定データで、異なる表情間の知覚的類似性を測定したものである。

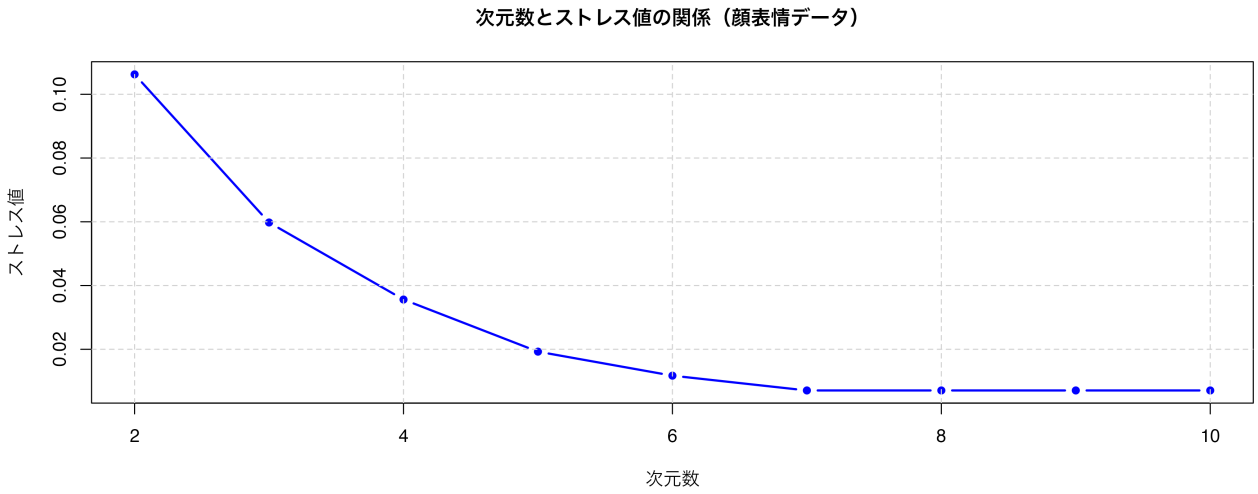
```
pacman::p_load(smacof)

# 次元数2から10でストレス値を計算
dimensions <- 2:10
stress_values <- numeric(length(dimensions))

for (i in seq_along(dimensions)) {
  result_temp <- mds(FaceExp, ndim = dimensions[i], type = "ordinal")
  stress_values[i] <- result_temp$stress
}
stress_values
```

```
[1] 0.106248100 0.059778016 0.035605225 0.019262481 0.011722294 0.007084647
[7] 0.007084647 0.007084647 0.007084647
```

```
# ストレス値のプロット
plot(dimensions, stress_values,
  type = "b", pch = 16,
  xlab = "次元数", ylab = "ストレス値",
  main = "次元数とストレス値の関係（顔表情データ）",
  cex.main = 1.1,
  cex.lab = 1.0,
  col = "blue", lwd = 2
)
grid(lty = 2, col = "lightgray")
```



ここでは次元数を様々に変えて，適合度指標であるストレス値を得てプロットしている。一般的に，ストレス値が

0.05 以下なら優秀，0.1 以下なら良好，0.2 以下なら普通とされる。

プロットや値を見てみると，3 次元でも十分な解が得られそうだ。改めて 3 次元であることを指定して分析し，プロットしてみよう。なお，mds 関数の type 引数に順序尺度水準であることを明記することで，適切な分析が行われる。

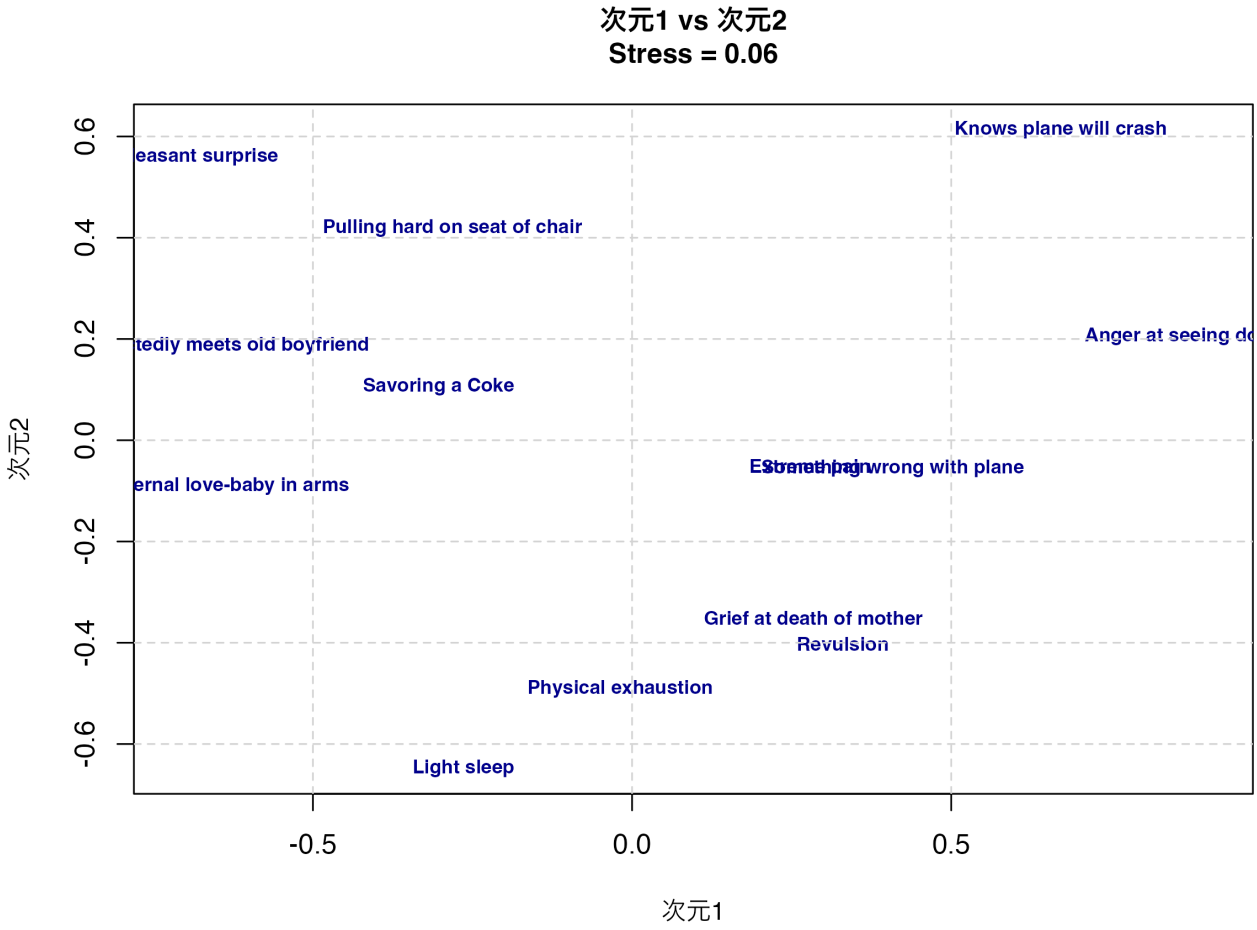
```
# 3次元でのMDS結果
result <- mds(FaceExp, ndim = 3, type = "ordinal")
result
```

```
Call:
mds(delta = FaceExp, ndim = 3, type = "ordinal")

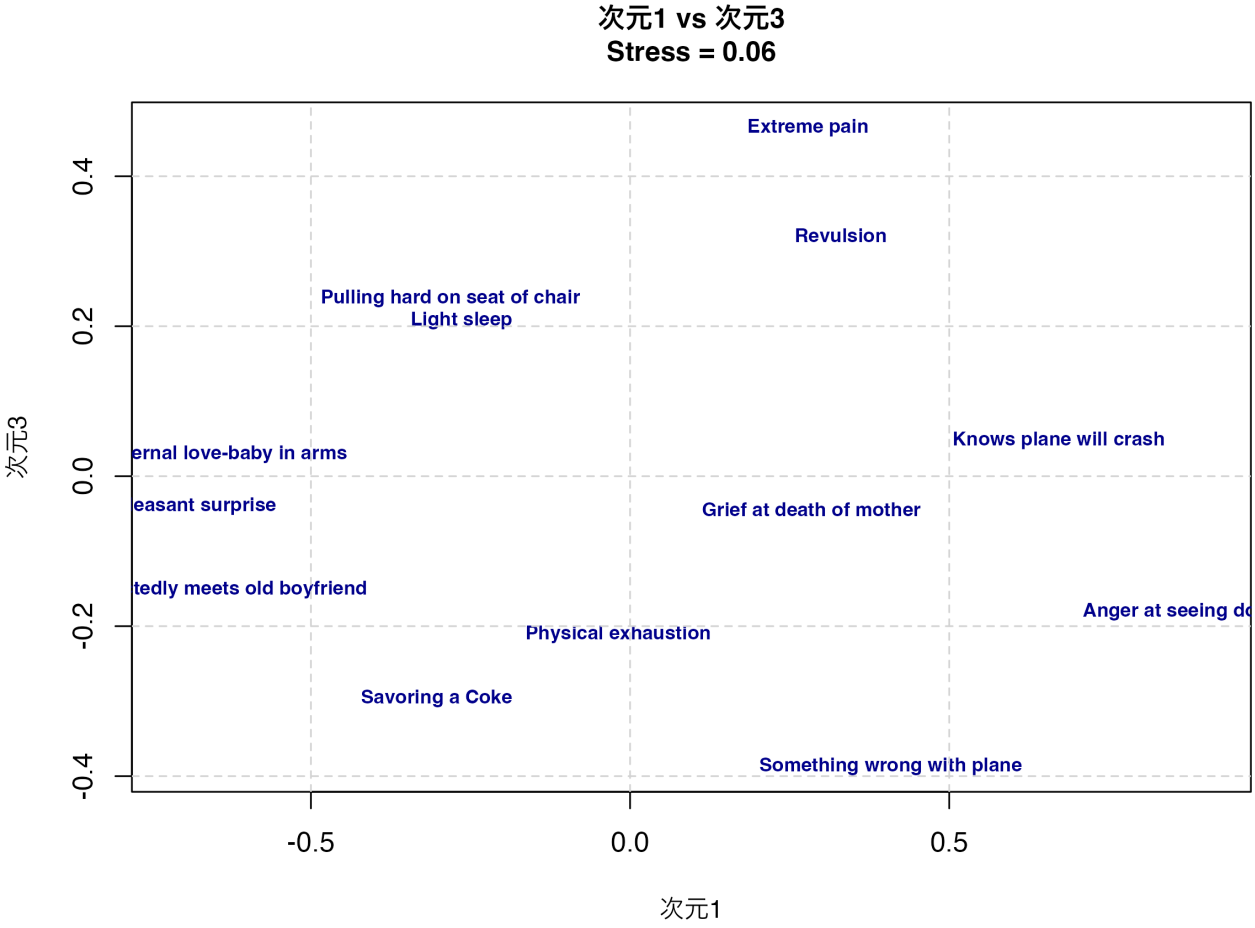
Model: Symmetric SMACOF
Number of objects: 13
Stress-1 value: 0.06
Number of iterations: 73
```

```
# 3次元MDSの可視化（3つの2次元プロット）

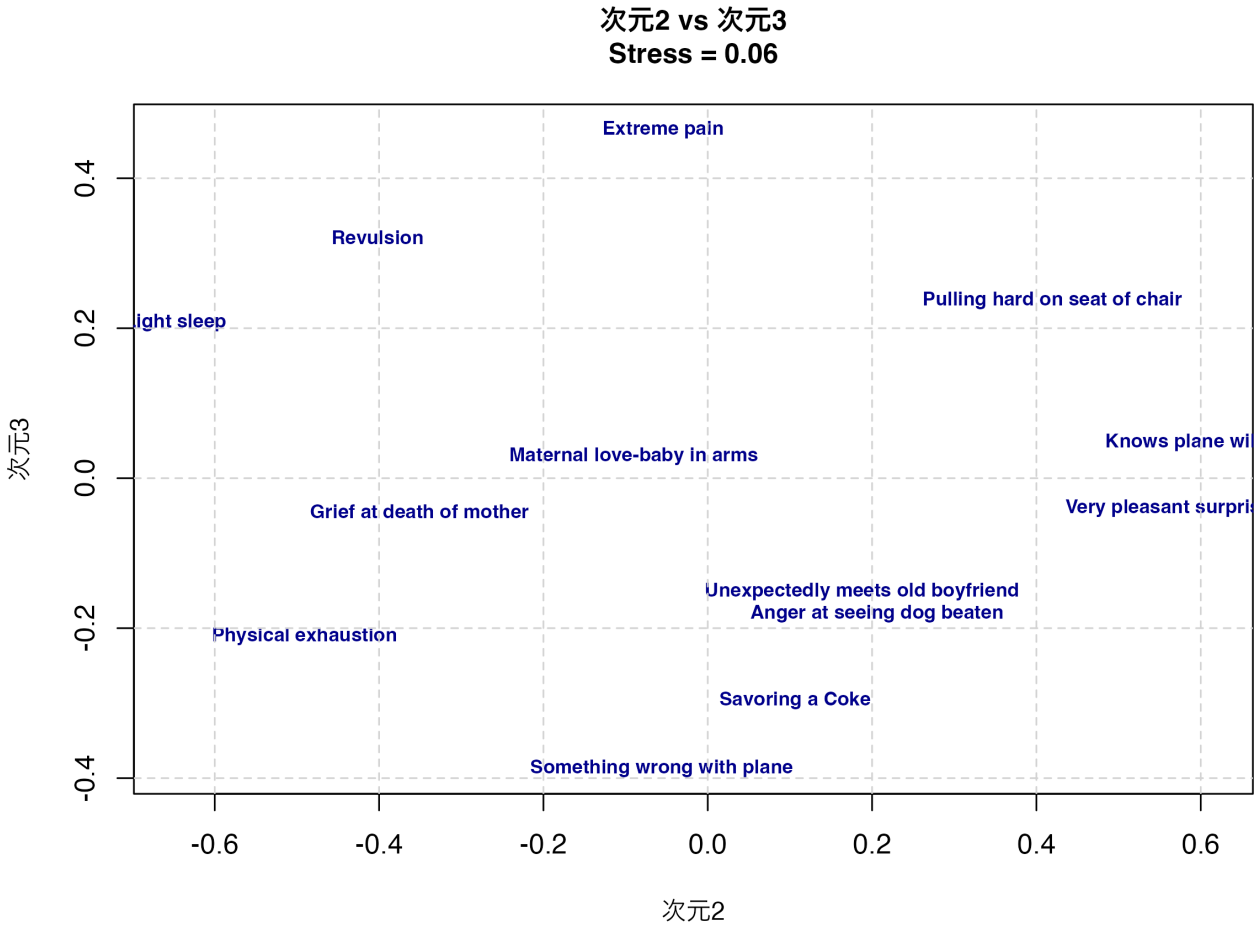
# 次元1 vs 次元2
plot(result$conf[, 1], result$conf[, 2],
  type = "n",
  xlab = "次元1", ylab = "次元2",
  main = paste("次元1 vs 次元2\nStress =", round(result$stress, 3)),
  cex.main = 1.0,
  cex.lab = 0.9
)
text(result$conf[, 1], result$conf[, 2],
  labels = rownames(result$conf),
  cex = 0.7,
  col = "darkblue",
  font = 2
)
grid(lty = 2, col = "lightgray")
```




```r
# 次元1 vs 次元3
plot(result$conf[, 1], result$conf[, 3],
  type = "n",
  xlab = "次元1", ylab = "次元3",
  main = paste("次元1 vs 次元3\nStress =", round(result$stress, 3)),
  cex.main = 1.0,
  cex.lab = 0.9
)
text(result$conf[, 1], result$conf[, 3],
  labels = rownames(result$conf),
  cex = 0.7,
  col = "darkblue",
  font = 2
)
grid(lty = 2, col = "lightgray")
```




```
# 次元2 vs 次元3
plot(result$conf[, 2], result$conf[, 3],
  type = "n",
  xlab = "次元2", ylab = "次元3",
  main = paste("次元2 vs 次元3\nStress =", round(result$stress, 3)),
  cex.main = 1.0,
  cex.lab = 0.9
)
text(result$conf[, 2], result$conf[, 3],
  labels = rownames(result$conf),
  cex = 0.7,
  col = "darkblue",
  font = 2
)
grid(lty = 2, col = "lightgray")
```




```
print(result$conf)
```

```
                                  D1          D2          D3
Grief at death of mother          0.28389211 -0.35088341 -0.04383041
Savoring a Coke                  -0.30306573  0.10620276 -0.29595552
Very pleasant surprise           -0.71552867  0.56031573 -0.04008618
Maternal love-baby in arms       -0.63827700 -0.09006026  0.02975313
Physical exhaustion              -0.01879485 -0.49089080 -0.21059225
Something wrong with plane        0.40829233 -0.05617713 -0.38685372
Anger at seeing dog beaten        0.90768476  0.20575384 -0.18027858
Pulling hard on seat of chair    -0.28180980  0.41937543  0.23764181
Unexpectedly meets old boyfriend -0.65807066  0.18760808 -0.15097801
Revulsion                         0.32995517 -0.40195569  0.32170945
Extreme pain                      0.27856073 -0.05436048  0.46450844
Knows plane will crash            0.67130146  0.61301964  0.04768550
Light sleep                      -0.26413984 -0.64794771  0.20727634
```

このプロットから，顔表情間の知覚的類似性の構造を読み取ることができる。第一次元は快不快（valence)、第二次元は覚醒度（arousal)、第三次元は自然性（naturalness）を表していると解釈できる。

より客観的な命名がしたい場合，座標と外部変数との相関などを求めて考える。相関係数はベクトルの角度に基づく $\cos(\theta)$ であり，軸上に外部変数の補助線を引くなどすればわかりやすい。こうした方法については，グリム・ヤーノルド（2016）に詳しい。

16.2　共頻行列を用いるもの

続いて共頻行列に基づく分析について考えてみよう。共頻行列とはカテゴリカルなデータのクロス表であり，カテゴリが同時に生起していることを表す。このことが当該カテゴリの近さを表す共変量だとして分析を行う。

カテゴリカルなデータであるから，応用範囲は広い。よく知られているのがテキストマイニングにおける応用例である。

16.2.1　テキストマイニングとは

テキストマイニングとは自然言語データを統計的に処理するための，一連の技法の名称である。小説や Web 上の記事，自由記述回答や逐語録など自由に書かれた自然言語表現を統計的に処理し，そこから意味のある知見を引き出す。

日本語のテキストマイニングは，基本的に形態素解析＋多変量解析の技術の総称であると言える。日本語は英語をはじめとする単語で分かち書きされる文章とは違い，一連の文字列から品詞ごとに文章を区分する必要がある。この文章から品詞の切り分け，活用前の原型を抽出するなどの操作を形態素解析という。形態素解析には日本語辞書に基づく形態素解析エンジンが必要で，MECAB や JANOME，ChaSen などが知られている。これらを R や Python で扱うためのパッケージも存在する。

形態素解析によって，文章の中に当該単語が何回出現したかを表す文書単語行列を作成する。これは一般に，単語の数が非常に多くなるから疎な矩形行列になる。この矩形行列を対象に特異値分解を行なったり，単語 $\times$ 単語の (正方) 共頻行列にすることで，多変量解析の対象となる。

多変量解析モデルはどのようなものでもよく，ここで紹介したクラスター分析や多次元尺度法などがよく用いられる。行列のサイズが大きくなりがちなので，一般的な多変量解析では少ない次元での適合度が悪くなりがちである。ある程度は適合度を諦めて可視化のために低次元にするか，自己組織化マップ (Self-Organization Mapping)[*2]などのアプローチで強制的に分類・可視化する方法などがとられることが多い。

多変量解析で分析することももちろん可能だが，ビッグデータと機械学習の組み合わせにより，形態素ではなくトークン[*3]を単位に用いることも多い。

ここでは形態素解析によるテキストマイニングの例を見てみよう。R で形態素解析をするには，`RMeCab` パッケージを用いることが多い。最新版は作者の Github サイト[*4]から導入するのがいいだろう。まずは形態素解析エン

[*2] とはいえ，コードに原因がない場合もある。それは Stan を導入する際のシステム的なエラーであり，書かれた内容ではなく動かす環境全体の問題である。解決策としては，表示されるエラーを解読して問題を解決するか，環境を再構築する (Stan を再インストールする，最新バージョンに入れ替える等) 必要がある。この場合も，生成 AI が助力してくれるだろう。

[*3] 詳しくは (浜田他，2019) を参照してほしい

[*4] 正規分布の幅のパラメータは，テキストによっては分散 σ^2 で記載されていることが多いが，ここでは標準偏差 σ で記述する。Stan では標準偏差で記述するようになっているので，それに合わせたいからである。標準偏差を二乗したものが分散であるから，意味すると

ジン Mecab をインストールし，その後で RMecab をインストールする[*5]。詳しくは作者のサイト[*6]を参照して欲しい。

最近はこの一連の手間を省いてくれる，gibasa というパッケージがある。gibasa は CRAN に登録されており，内部で MeCab のバイナリファイルを含んでいるので，外部ファイルを準備する必要がない仕組みになっている[*7]。ここではこのパッケージを使った例を見てみよう。

パッケージを読み込んで，適当な文章を形態素解析してみよう。

```
pacman::p_load(tidyverse, gibasa)
text <- "私は昨日，カフェでコーヒーを飲んだ。"
dat <- gibasa::tokenize(text)
dat
```

```
# A tibble: 11 x 5
   doc_id sentence_id token_id token    feature
   <fct>        <int>    <int> <chr>    <chr>
 1 1                1        1 私       名詞,代名詞,一般,*,*,*,私,ワタクシ,ワタクシ~~
 2 1                1        2 は       助詞,係助詞,*,*,*,*,は,ハ,ワ
 3 1                1        3 昨日     名詞,副詞可能,*,*,*,*,昨日,キノウ,キノー
 4 1                1        4 ,        記号,読点,*,*,*,*,, ,, ,,
 5 1                1        5 カフェ   名詞,一般,*,*,*,*,カフェ,カフェ,カフェ
 6 1                1        6 で       助詞,格助詞,一般,*,*,*,で,デ,デ
 7 1                1        7 コーヒー 名詞,一般,*,*,*,*,コーヒー,コーヒー,コーヒー~~
 8 1                1        8 を       助詞,格助詞,一般,*,*,*,を,ヲ,ヲ
 9 1                1        9 飲ん     動詞,自立,*,*,五段・マ行,連用タ接続,飲む,ノン,ノン~~
10 1                1       10 だ       助動詞,*,*,*,特殊・タ,基本形,だ,ダ,ダ
11 1                1       11 。       記号,句点,*,*,*,*,。 ,。 ,。
```

tokenize 関数によって文字列が品詞ごとに区分され，またその特徴が feature 列に入っていることがわかるだろう。この feature 列に入っているのは gibasa が内蔵する MeCab の戻り値であり，このラベルを整理するためには prettify 関数を用いる。

また，実際のテキストマイニングにおいては助詞や記号は使いにくいし，動詞も活用する前の原型で考えるほうがいいだろう。これらをパイプ演算子で繋ぎながら処理すると次のようになる。

```
gibasa::prettify(dat, col_select = c("POS1", "Original")) |>
  dplyr::filter(POS1 %in% c("名詞", "動詞", "形容詞"))
```

ころは本質的に変わりがない。

[*5] このカバーストーリーとモデルはリー・ワゲンメーカーズ（2017）の「七人の科学者」P.48–49 を参考に作ったものである。

[*6] 正規分布の幅のパラメータは，テキストによっては分散 σ^2 で記載されていることが多いが，ここでは標準偏差 σ で記述する。Stan では標準偏差で記述するようになっているので，それに合わせたいからである。標準偏差を二乗したものが分散であるから，意味するところは本質的に変わりがない。

[*7] たとえば紀ノ定（2018）が参考になる。

```
# A tibble: 5 x 6
  doc_id sentence_id token_id token    POS1  Original
  <fct>        <int>    <int> <chr>    <chr> <chr>
1 1                1        1 私       名詞  私
2 1                1        3 昨日     名詞  昨日
3 1                1        5 カフェ   名詞  カフェ
4 1                1        7 コーヒー 名詞  コーヒー
5 1                1        9 飲ん     動詞  飲む
```

ここでは 1 つの文だけで分析を行ったが，次に実際のテキストマイニングの例として，京都ラーメンに関する感想文を使った分析例を見てみよう。

ここでの処理の流れは次のとおりである。

1. 文章を形態素に分割する
2. 文章ごとに単語の出現度数を書いた文書 × 単語行列をつくる (dtm)
3. 文書単語行列から単語 × 単語の距離行列を作る
4. 距離行列を対象に計量 MDS を使って可視化

```
# 京都ラーメンに関する感想文データの作成
ramen_reviews <- c(
  "京都ラーメンは意外とコッテリしたのが多いんだよね。",
  "濃厚なスープと細麺の組み合わせが絶妙で美味しかった。",
  "ドロドロとした鶏ガラベースのスープが京都らしくて好み。",
  "老舗のラーメンは深いコクがあり，麺との絡みが良くて好き。",
  "京都駅近くの店で食べた醤油ラーメンのチャーシューが柔らかくて最高。",
  "京都ラーメンといえば天下一品のコッテリが魅力的。",
  "背脂がたっぷりのラーメンも京都で食べると格別だった。",
  "細麺とスープのバランスが絶妙で，また食べたくなる味。",
  "京都らしい濃い醤油ラーメンに九条ネギがよく合っていた。",
  "老舗の店主が作る丁寧なラーメンは心に残る味わいだった。"
)

# トークン化して，品詞を選び出し，文章ごとにカウントする
dat_count <- ramen_reviews |>
  # テキストを形態素に分解（単語・品詞・活用形など）
  gibasa::tokenize() |>
  # 品詞情報と原型を選択・整理
  gibasa::prettify(col_select = c("POS1", "Original")) |>
  # 分析対象を名詞・形容詞に限定
  dplyr::filter(POS1 %in% c("名詞", "形容詞")) |>
  # データ変換処理
```

```
  dplyr::mutate(
    doc_id = forcats::fct_drop(doc_id),
    # 原型がある場合は原型を、ない場合は現在の形を使用
    token = dplyr::if_else(is.na(Original), token, Original)
  ) |>
  # 文書ID別・単語別に出現回数をカウント
  dplyr::count(doc_id, token)

# 文章単語行列の作成
dtm <- dat_count |>
  tidyr::pivot_wider(
    id_cols = doc_id,
    names_from = token,
    values_from = n,
    values_fill = 0 # 欠損値（その文書にその単語が出現しない）を0で埋める
  )

dtm
```

```
# A tibble: 10 x 45
   doc_id     の     ん コッテリ ラーメン   京都   多い スープ   濃厚     細
   <fct>  <int> <int>    <int>    <int> <int> <int>  <int> <int> <int>
 1 1          1     1        1        1     1     1      0     0     0
 2 2          0     0        0        0     0     0      1     1     1
 3 3          0     0        0        0     1     0      1     0     0
 4 4          0     0        0        1     0     0      0     0     0
 5 5          0     0        0        1     1     0      0     0     0
 6 6          0     0        1        1     1     0      0     0     0
 7 7          0     0        0        1     1     0      0     0     0
 8 8          0     0        0        0     0     0      1     0     1
 9 9          0     0        0        1     1     0      0     0     0
10 10         0     0        0        1     0     0      0     0     0
# i 35 more variables: 組み合わせ <int>, 絶妙 <int>, 美味しい <int>, 麺 <int>,
#   ガラベース <int>, 好み <int>, 鶏 <int>, コク <int>, 好き <int>, 深い <int>,
#   老舗 <int>, 良い <int>, チャーシュー <int>, 店 <int>, 最高 <int>,
#   柔らかい <int>, 近く <int>, 醤油 <int>, 駅 <int>, 天下一品 <int>, 的 <int>,
#   魅力 <int>, 格別 <int>, 背 <int>, 脂 <int>, バランス <int>, 味 <int>,
#   ネギ <int>, 九 <int>, 条 <int>, 濃い <int>, 丁寧 <int>, 味わい <int>,
#   店主 <int>, 心 <int>
```

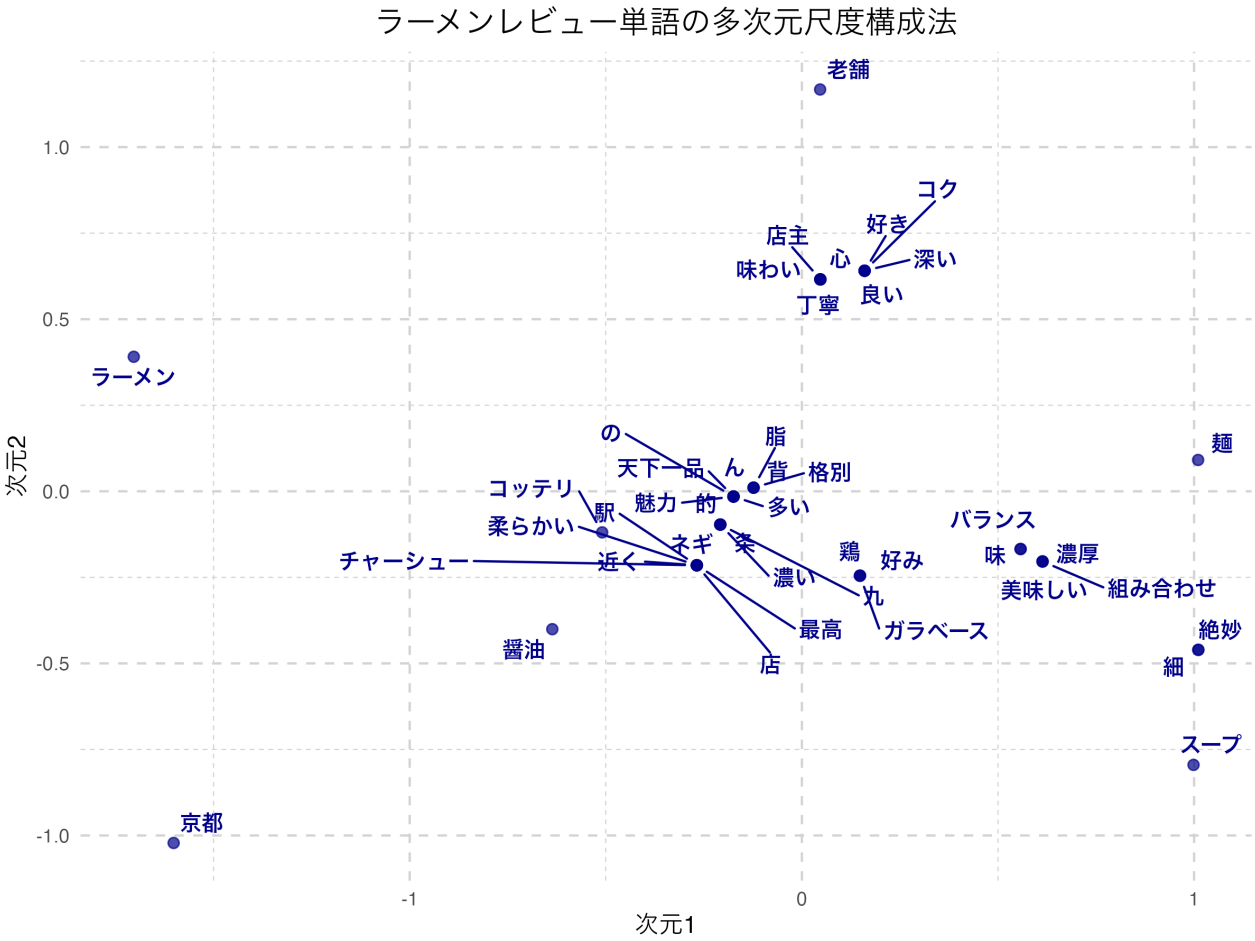
```r
distance_matrix <- dist(t(dtm[, -1]))

# 多次元尺度構成法（MDS）の実行
mds_result <- cmdscale(distance_matrix, k = 2) # 2次元での古典的MDS

# ggplotで可視化（ggrepelを使用してラベル重複を回避）
pacman::p_load(ggrepel) # ラベル重複回避のためのパッケージ

# MDS結果をデータフレームに変換
mds_df <- data.frame(
  dim1 = mds_result[, 1], # 第1次元の座標
  dim2 = mds_result[, 2], # 第2次元の座標
  word = rownames(mds_result) # 単語名
)

# ggplotで散布図を作成
ggplot(mds_df, aes(x = dim1, y = dim2)) +
  geom_point(color = "darkblue", size = 2, alpha = 0.7) + # 点をプロット
  geom_text_repel( # ggrepelを使用してラベル重複を回避
    aes(label = word), # 単語をラベルとして表示
    size = 3.5, # ラベルサイズ
    color = "darkblue", # ラベル色
    fontface = "bold", # 太字
    box.padding = 0.3, # ラベル周りの余白
    point.padding = 0.3, # 点周りの余白
    max.overlaps = Inf # 重複制限なし
  ) +
  labs(
    title = "ラーメンレビュー単語の多次元尺度構成法",
    x = "次元1",
    y = "次元2"
  ) +
  theme_minimal() + # シンプルなテーマ
  theme(
    plot.title = element_text(hjust = 0.5, size = 14), # タイトル中央寄せ
    panel.grid = element_line(linetype = "dashed", color = "lightgray") # グリッド線
  )
```



ここでプロットされているのは，単語同士の類似度が近いものは近くに，遠いものは遠くにプロットされる地図である。これをみると，「麺」と「スープ」や「組み合わせ」「バランス」といった言葉が近くに付置されており，関係が深そうなことが読み取れる。

ただし，「九条ネギ」が「九」と「条」に別れているように，機械的に分解することの限界もあり，こうした場合は辞書を変更したり，特定の単語・専門用語などを強制的に取り出すような工夫をする必要がある。形態素解析を経由したテキストマイニングは，多変量解析に入る前の事前クリーニングにかなりの労力を要することが少なくない。

また，今回は 10 件程度の例であったが，基本的にテキストマイニングは膨大なデータや学習された辞書をもちいることが主流になってきている。これは生成 AI との相性もよく (生成 AI はすでに自然言語についてある程度学習済みのものとも言える)，形態素解析や統計モデルを経ないテキストマイニングがこれから主流になってくるかもしれない。

16.3　偏相関行列を用いるもの

最後に偏相関行列を用いる統計モデルを紹介しておこう。

(ピアソンの) 相関行列は多くの線形モデルで多用されているが，変数 X と Y の背後に，両者に影響する Z があ

る場合，見掛け上相関が高くなっているが，Z の影響を除外する (統計的に統制する) とそれほど相関が高くない，ということもあり得る。

この統計的に統制するというのは，X を Z で回帰して得られた残差 R_x と，Y を Z で回帰して得られた残差 R_y との相関であり，偏相関と呼ばれる。この相関の方が，本来的な関係を捉えていると言えるかもしれない。ここでは X, Y の 2 変数の例であったが，多変量の相関行列に関しても同様に当該 2 変数以外の変数で (重) 回帰をし，その残差同士の相関を出した偏相関行列を考えることができる。これは逐一回帰分析をしなくても，次のような行列演算で一括して求めることができる。

相関行列を $\mathbf{R}$ とすると，偏相関行列 $\mathbf{P}$ の要素は次の式で計算される：

$$p_{ij} = -\frac{r^{ij}}{\sqrt{r^{ii} r^{jj}}}$$

ここで r^{ij} は相関行列の逆行列 $\mathbf{R}^{-1}$ の (i, j) 要素である。

さて，因子分析は相関行列から始めて関係の強いところを因子にまとめるというイメージだが，この手順を反転させて，偏相関行列の関係の弱いところの繋がりをカットする，という方法で，純粋な変数間関係だけ残るようにして可視化する方法がある。

この分析方法は**グラフィカルモデリング**と呼ばれる。量的な変数の場合は偏相関行列から，質的な変数の場合でも多元分割行列から，変数間の条件付き独立性を検証する方法で進められる。可視化の手段として，変数間関係をノードとエッジからなるネットワークで表現することから，最近では**ネットワーク分析**と呼ばれることもある。

心理学においては，因子が心理学的構成概念だと捉えられることが多い。しかし，構成概念とはそもそも単体でそこに「ある」ものではなく，いろいろな現象の総合的な全体である，というシステム論的な観点によれば，このネットワーク分析的なアプローチの方が適しているかもしれない。この理論的な体系と方法論の合体については，アデラ＝マリア他 (2024) を参考にしてほしい。

具体的な例で見てみよう。例えば，性格検査の Big5 データを使って相関行列と，その偏相関行列を見てみよう。

```
pacman::p_load(psych, corrplot, RColorBrewer)

# Big5性格データの読み込み（最初の25項目のみ使用）
bfi <- psych::bfi[, 1:25]
bfi_clean <- na.omit(bfi) # 欠損値を除去

# 1. 相関行列の計算
cor_matrix <- cor(bfi_clean)
# 2. 偏相関行列の計算（行列演算で実行）
R_inv <- solve(cor_matrix) # 相関行列の逆行列を計算

# 偏相関行列の各要素を計算
partial_cor_matrix <- matrix(0, nrow = nrow(R_inv), ncol = ncol(R_inv))
for (i in 1:nrow(R_inv)) {
```

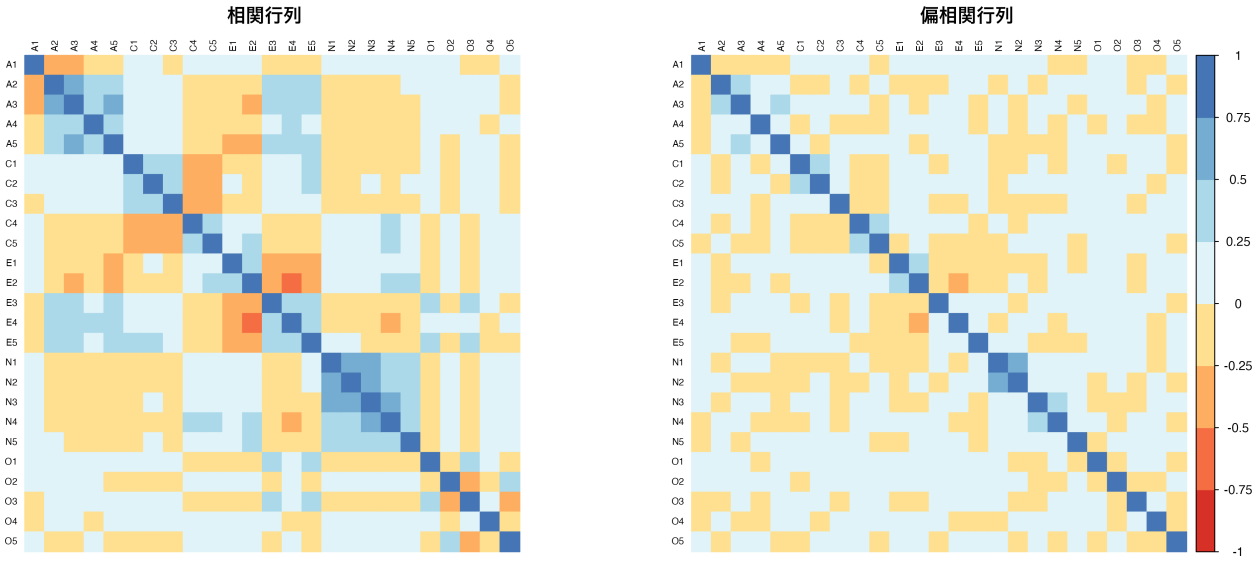
```
  for (j in 1:ncol(R_inv)) {
    if (i != j) {
      # 偏相関の公式: p_ij = -r^ij / sqrt(r^ii * r^jj)
      partial_cor_matrix[i, j] <- -R_inv[i, j] / sqrt(R_inv[i, i] * R_inv[j, j])
    }
  }
}
# 対角要素は1に設定
diag(partial_cor_matrix) <- 1

# 行名・列名を設定
rownames(partial_cor_matrix) <- rownames(cor_matrix)
colnames(partial_cor_matrix) <- colnames(cor_matrix)
```

```
# 3. 相関行列と偏相関行列の比較表示
par(mfrow = c(1, 2))

# 相関行列ヒートマップ
corrplot(cor_matrix,
  method = "color",
  order = "alphabet", # 同じ順序で比較
  tl.cex = 0.6,
  tl.col = "black",
  col = brewer.pal(n = 8, name = "RdYlBu"),
  title = "相関行列",
  mar = c(0, 0, 2, 0),
  cl.pos = "n"
) # カラーバーを非表示

# 偏相関行列ヒートマップ
corrplot(partial_cor_matrix,
  method = "color", # 色で偏相関の強さを表示
  order = "alphabet", # アルファベット順で表示（相関行列と同じ順序で比較）
  tl.cex = 0.6, # 変数名のテキストサイズ
  tl.col = "black", # 変数名の色
  col = brewer.pal(n = 8, name = "RdYlBu"), # 青-黄-赤のカラーパレット
  title = "偏相関行列", # 図のタイトル
  mar = c(0, 0, 2, 0)
) # 図の余白設定（下，左，上，右）
```




```
par(mfrow = c(1, 1)) # 環境を元に戻す
```

相関行列と偏相関行列を並べてみると，所々相関のパターンが違うところが見える。

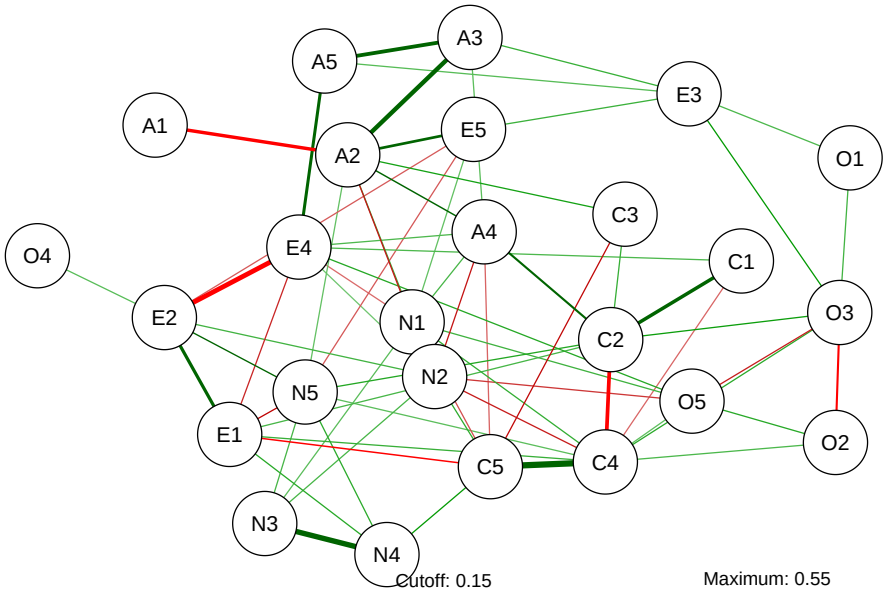
さて，ネットワークの表現をするには相関行列でも偏相関行列でも構わないのだが，一応ここでは偏相関行列を用いたネットワークを見てみよう。

ここでは qgraph パッケージを用いている。この関数のオプションとして，推定に glasso を用いているが，ガウシアングラフィカルモデルの一種で，L1 正則化 (lasso) を行い重要でない変数間の関係を 0 にするモデルである。これで BIC を基準に不要なパスをカットして描画される。描画のレイアウトオプションは spring だが，これはなるべく自然な配置にしてくれる方法であり，他にもいろいろなオプションがあり得る。

```
pacman::p_load(qgraph)
BICgraph <- qgraph(
  partial_cor_matrix, # 偏相関行列を入力データとして使用
  graph = "glasso",
  sampleSize = nrow(bfi), # サンプルサイズを指定（統計的有意性の判定に使用）
  tuning = 0, # 正則化パラメータ（0=BIC基準で自動選択、大きいほどスパース）
  layout = "spring",
  title = "BIC", # グラフのタイトル
  threshold = TRUE, # 弱い結合を除去してスパースなネットワークを作成
  details = TRUE # 詳細な出力情報を表示
)
```

```
Note: Network with lowest lambda selected as best network: assumption of sparsity might be violated.
```

BIC

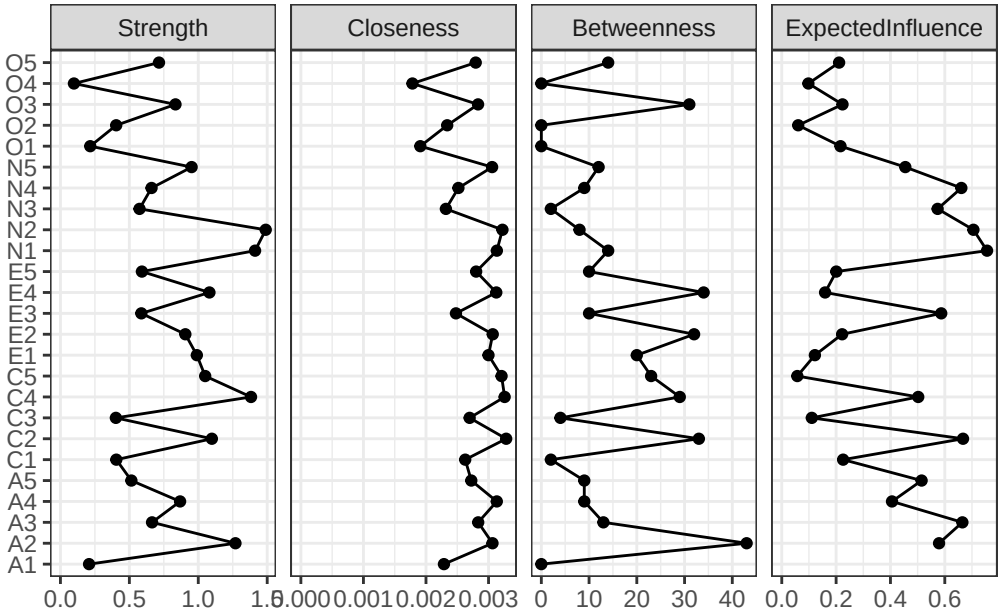


さて，ネットワーク分析は「これが因子！」のような特定の何かに帰着するのではなく，全体像をそのまま見て捉えることが特徴的である。しかしそれでは何がなんだか分かりにくいということもあろう。ということで，ネットワークの中心がどこにあるのか，どのノードとどのノードの結びつきが強いのか，といったことを指標とする。これらは中心性指数と呼ばれ，次のコードで可視化される。

```
centralityPlot(
  list(BIC = BICgraph),
  include = "all"
)
```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_path()`).
```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_point()`).
```



ここで示されている各指標の意味は次のとおりである。

1. Strength（強度）

- ノードに接続しているエッジの重みの合計
- ネットワークでは、そのノードが他のノードとどれだけ強く関連しているかを表す
- 次の式で計算される。$strength = \sum |w_{ij}|$（絶対値の和）

2. Closeness（近接中心性）

- 他の全てのノードまでの最短距離の逆数
- そのノードが他の全てのノードにどれだけ「近い」かを測定
- 情報伝播や影響の広がりやすさを表すといえる
- 距離の合計の逆数であり，次の式で計算される。$closeness = \frac{1}{\sum d_{ij}}$

3. Betweenness（媒介中心性）

- 他のノード間の最短経路上にそのノードが位置する頻度であり，「橋渡し」の役割を果たすノードを特定する。
- ネットワークでは症状間の連鎖反応の「ハブ」を表す

4. Expected Influence（期待影響力）

- strength の改良版で、正と負のエッジを区別したもの
- 正のエッジは加算、負のエッジは減算。ここでは抑制関係も考慮した真の「影響力」を測定していると考えられる。

ネットワークモデルは，推定法もいろいろ工夫されているし，動的なモデル，時系列モデルなども扱える。今後の発展が楽しみな分析方法の一つと言えるだろう。

16.4 課題

この章で紹介した多変量解析手法（クラスター分析，多次元尺度構成法，ネットワーク分析）を使って，以下の課題に取り組んでみよう。

16.4.1 課題 1: クラスター分析による消費者セグメンテーション

100 人の消費者の購買行動データ（5 つの商品カテゴリへの支出額）を用いて，階層的クラスター分析と k-means 法の両方でクラスター分析を実行し，結果を比較せよ。画面には一部しか表示しておらず，全体データはこちら consumer_data.csv からダウンロード可能です。

	消費者ID	食品	衣料品	書籍	電子機器	旅行
1	C001	7.159287	2.6886403	0.7544844	1.4639778	2.8421412
2	C002	7.654734	2.5539177	0.1000000	0.8720764	1.6853630
3	C003	10.338062	1.9380883	1.5028693	0.1656465	0.1134270
4	C004	8.105763	1.6940373	0.6453996	1.1958188	0.1823988
5	C005	8.193932	1.6195290	0.6559957	2.2351973	0.1000000
6	C006	10.572597	1.3052930	1.5127857	1.0397224	1.3629122
7	C007	8.691374	1.7920827	0.8576135	1.9863715	0.2458933
8	C008	6.102408	0.7346036	0.3896411	0.2056938	2.8255001
9	C009	6.969721	4.1689560	1.0906517	1.4555504	4.5201307
10	C010	7.331507	3.2079620	0.9305543	1.9155258	0.4555634

分析手順: 1. データを読み込み，標準化を行う 2. 階層的クラスター分析（ウォード法）でデンドログラムを作成 3. k-means 法で 3 クラスターに分類 4. 両手法の結果を比較し，各クラスターの特徴を解釈する

16.4.2 課題 2: 多次元尺度構成法による都市間類似性の可視化

日本の主要 10 都市間の類似度評定データを用いて，多次元尺度構成法で都市の関係性を 2 次元平面上に可視化せよ。データはこちら city_similarity.csv からダウンロード可能です。

	東京	大阪	名古屋	札幌	仙台
東京	10	6	7	2	5
大阪	6	10	6	2	3
名古屋	7	6	10	2	4
札幌	2	2	2	10	7
仙台	5	3	4	7	10

分析手順: 1. 類似度データを距離行列に変換する 2. 古典的 MDS で 2 次元解を求める 3. ggplot と ggrepel で結果を可視化する 4. 都市の配置パターンから地理的・文化的特徴を考察する

16.4.3　課題 3: ネットワーク分析による心理尺度の構造解析

性格特性の下位尺度間の関係をネットワーク分析で可視化し，中心性指標を用いて重要な特性を特定せよ。データはこちら personality_data.csv からダウンロード可能です。

	参加者ID	外向性1	外向性2	協調性1	協調性2	誠実性1	誠実性2	神経症傾向1
1	P001	2	2	4	4	2	2	1
2	P002	5	5	2	3	3	2	5
3	P003	5	5	4	4	4	5	2
4	P004	5	4	2	2	6	5	2
5	P005	3	6	4	5	6	7	3
6	P006	3	4	3	4	5	5	6
7	P007	1	2	5	5	6	6	1
8	P008	5	7	4	4	7	7	7
9	P009	2	1	3	4	7	7	1
10	P010	4	6	4	5	3	2	4

	神経症傾向2	開放性1	開放性2
1	1	1	1
2	3	3	4
3	2	1	3
4	2	3	4
5	6	3	4
6	4	5	5
7	4	4	5
8	7	5	3
9	4	4	5
10	4	2	1

分析手順: 1. データを読み込み，下位尺度間の偏相関行列を計算する 2. glasso 法でスパースなネットワークを推定する 3. 中心性指標（強度，近接性，媒介性）を算出する 4. 最も中心的な特性と周辺的な特性を特定し，性格構造について考察する

第 17 章

ベイジアンモデリング

ここまでは定型的な統計モデルをいろいろ紹介してきた。定型的といったのは，モデルの形や求めるパラメータの数，その解釈の仕方が決まっていて，データの種類や型に合えば適用できるモデルという意味である。これに対してベイジアンモデリングは，データに合う形のモデルを形作る＝モデリングするということであり，その推定方法としてベイズ法をつかうというものである。推定法は必ずしもベイズ法である必要はなく，最尤法でも最小二乗法でも良いのだが，これらの手法による推定は推定手順も自らで開発しなければならない。これに対し，すでにみた確率的プログラミング言語によるベイズ推定は，確率モデルさえ記述できれば推定結果が得られる。これにより，研究者は自らのデータとその背景に合ったモデルを考えて記述するだけでよく，テクニカルな推定手順を考える必要がなくなる。確率的プログラミング言語は，その言葉にあるようにプログラミングの知識を必要とするが，逆に言えばこの知識・技能さえ習得しておけば，あとは分析者のアイデア次第で、自身のオリジナルな分析ができる。

以下では Stan によるプログラミングと，その特徴的な利用例についてみていくが，その前にベイジアンモデリングを学ぶ上での指針を示しておく。

17.1　ベイジアンモデリングの学習方法

17.1.1　学習のステップ 1; 確率的プログラミング言語 Stan の学習

ここでは R を使った統計分析を扱ってきたので，改めてプログラミングとは，という話をする必要はないだろう。ただ，確率的プログラミング言語としてここで取り上げる Stan は，R よりもやや上級者向けの，C++ と呼ばれる言語に基づいたものである。初学者にとって大きな違いは，「インタプリタ型とコンパイル型」，および「型宣言」の 2 点だろう。

17.1.1.1　インタプリタ型とコンパイル型

R はインタプリタ型言語と呼ばれる。個人的には「一問一答型」と呼んでいる。コマンドプロンプト>が表示されている時，R は入力を待って聞き耳を立てているのであった。ここに計算式や命令文を入れると，結果を計算して返す。つまり，問いに対して答えが返ってくる，という形式の繰り返しである。

これに対してコンパイル型言語というのがある。C 言語や Java，Python，そして Stan はこの形である。すなわ

ち，命令文全体をまず書いて，その文書 (スクリプトファイル) 全体を機械語に翻訳する。この作業を**コンパイル**という。コンパイルされたものを実行すると，その文書の内容が実行される。ここで命令文に誤りがある場合，1. コンパイルできないというエラーが表示される，2. コンパイルはできるが，実行時にエラーが表示される，という 2 つのケースがある。エラーは大抵，XX 行目がおかしい，という形で表示される。インタプリタ型であれば，書いて実行した行でエラーだと言われるので気づきやすいが，コンパイル型は一旦書き切ってからでないとエラーかどうかわからないので[*1]，不便に感じるかもしれない。

コンパイル型の利点は，一旦機械語に翻訳し，計算機は計算機自身の母語 (機械語) で計算をするので，計算速度が速いという点にある。この利点のために必要なこととして理解して欲しい。また，コンパイルは専用のツールを使い，そのツールによってコンパイルされたものは，そのツールの環境でしか動かないという制約がある。Windows の場合は Rtools，Mac の場合は command line tools を導入する必要がある。これらは計算機のより根源的なところにアクセスする。一般的なアプリケーションを使うのとは違い，むしろ MCMC サンプリングを行うアプリケーションを作るようなものだから，ウィルス対策ソフトがその実行を妨げるようなことがある。環境の構築はすでに済んでいるものとして話を進めるが，その準備に一苦労する可能性があることは覚えておくと良い。困ったことがあれば，自身で検索するなどして対応する必要があるだろう。

さて，Stan を使った分析では，R ファイルとは別に命令文全体を Stan の言語で書いた Stan ファイルを準備することになる。このファイルを R の命令文で「Stan を使ってコンパイルせよ」と指示する。コンパイルが終われば，これまた R 側から，「そのコンパイルされたオブジェクトを使って MCMC サンプリングをせよ」と指示する。計算結果は R のオブジェクトとして環境に保存されるから，あとは R によるデータハンドリングの作業になってくる。Stan ファイルも R ファイルも RStudio のエディタ機能を利用すれば良いが，両者を混ぜるようなことのないよう，この仕組みを理解して進めてほしい。

17.1.1.2　型宣言

聞きなれない言葉かもしれないが，型宣言とは，変数の型を宣言することである。例えば，`int x;` というコードがあったとき，`int` は整数型を表している。整数型は整数であり，`x` に代入可能なのは `1.0`(実数) でも `1+0i`(複素数) でもなく，`1`(整数) である。

このように，変数を使う前にその変数がどの型なのかを宣言することを**型宣言**という。このような型宣言は，コンパイル型言語では必須である。このように宣言しておくことで，本来整数しか入らないところに実数を入れてしまう，といったエラーが生じないように工夫されている。R では変数を事前に宣言する必要がなく，ただ `x <- 1` と書き始めると，`x` が整数であれ実数であれ，自由に扱うことができた。このことに慣れていると，事前に宣言しなければならないことが非常に不便に思えるかもしれないが，型宣言をすることで言語の堅牢性を高めているという利点がある。

Stan はこの型宣言をブロックごとに行う必要がある。ブロックとは，中括弧`{}`で囲われる領域のことであり，次の 6 つのブロックがある。

1. data ブロック
2. transformed data ブロック

[*1] エディタがその言語に対応していたら，おかしな記述に下線が引かれるなど注意を促してくる機能もある。また RStudio で Stan 言語を書いていると，コンパイルの前に文法のチェックをする機能もある。

3. parameters ブロック
4. transformed parameters ブロック
5. model ブロック
6. generated quantities ブロック

もっともよく使われるのは 1.data ブロックと，3.parameters ブロック，5.model ブロックである。data ブロックは Stan 外部とのやりとり，すなわち Stan が外部から受け取るデータを宣言，記述するところである。ここで型が異なるデータが与えられるとエラーになる。すなわち，Stan の側で `int x;` と宣言してあるのに対し，R 側から `x <- 1.2` のような実数が与えられると，実行時にエラーになる。このように，型宣言をすることで，エラーを防ぐものであると理解してほしい。

parameters ブロックは推定したいパラメータを宣言するものであり，ここで宣言されたパラメータについて，Stan はサンプリング結果を返すことになる。model ブロックは確率モデルを記述するところ (尤度関数を記述するところ) であるので，もっとも重要なブロックであると言えるだろう。

そのほかのブロックは補捉的なものであり，必ずしも使う必要があるわけではない。transformed data ブロックは，data ブロックで宣言されたデータを変換するところであり，transformed parameters ブロックは，parameters ブロックで宣言されたパラメータを変換するところである。なぜそのような変換をするかといえば，内部で以後の計算をやりやすくするためである。例えば複数のパラメータを組み合わせて，確率分布に与える場合は一旦返還しておいた方が可読性が高い。具体例として回帰分析のことを考えると，パラメータは切片 β_0 と傾き β_1 であり，これが説明変数 x_i と組み合わさって予測値 $\hat{y}_i$ を作るのであった。パラメータブロックには β_0 と β_1 を宣言するが，transformed parameters ブロックで `yhat` を宣言して

$$yhat = \beta_0 + \beta_1 x$$

とかいておくと，モデルブロックでは `yhat` を使って記述できる。このように，あるパラメータがほかのパラメータの組み合わせで作られる場合などは，一旦その置き換えられる形を書いておいた方がわかりやすだろう。

generated quantities ブロックは，サンプリングされた値を加工して使う場合に用いる。サンプリングされたものの加工は，結果を受け取った R の側でも可能なので，このブロックは必ずしも必要ではない。しかし，サンプリングが終わった時に自分に必要な加工された値も (コンパイルして高速で) 計算しておいてくれると便利である。他にも色々な用途があるので，このブロックに関しては続く実践例のところでみていこう。

17.1.1.3 そのほかの細かな違い

あとは，行の終わりにセミコロンをつける必要があるとか，コメントを書くときに//を使うとか，そういった細かなところが違うだけである。

プログラミングの基本は**思った通りに動くのではなく，書いた通りに動く**ことである。もし思い通りにいかず，エラーが表示されれば，それもあなたが書いたコードに原因がある[*2]。そのためエラーがでたら恐れ慄くのではな

[*2] とはいえ，コードに原因がない場合もある。それは Stan を導入する際のシステム的なエラーであり，書かれた内容ではなく動かす環境全体の問題である。解決策としては，表示されるエラーを解読して問題を解決するか，環境を再構築する (Stan を再インストールする，最新バージョンに入れ替える等) 必要がある。この場合も，生成 AI が助力してくれるだろう。

く，**解決のためのヒントが表示された**ぐらいに理解すれば良い。問題点を一つ一つ解決していけば，必ず望むところに到達できるはずである。最近は生成 AI が発達しているので，エラーメッセージを丸ごと生成 AI に与えて，どこにどのような問題があるかを聞くという方法があるので，それを利用すると良い。

17.1.2　学習のステップ 2; これまでの分析方法を書き直してみる

ここまで様々な統計モデルを見てきた。ベイジアンモデリングの学習のステップ 2 は，これまでの分析方法を Stan に書き直してみることである。

例えば回帰分析を，重回帰分析を，階層線形モデルを Stan の言葉で書いたらどうなるだろうか。もちろん `brms` パッケージを使うとこうした苦労は必要ないのだが，改めて自分で既知のモデルを描いてみると，どのようなモデルがどのように記述されるかがわかるだろう。

この時のポイントは，分析に際して**データ生成メカニズム**という視点を持つことである。我々はつい，データがあってそれに合う分析方法を探す，という発想になってしまう。あるいは分析方法のバリエーションがないばあい，分析方法に合うようなデータを取る，という考え方になってしまう。これはおかしなことだとは思わないだろうか。自分の購入した統計ソフトが回帰分析しかできないので，離散変数は諦めて研究計画を練り直そう，というのは大変貧しい話である。

本来，自然な人間の振る舞いや反応の仕方を数値におとして，そこから意味を読み取ることが統計学であり，予算や環境の問題で人間の振る舞いの方を変えさせるというのはおかしいのである。なるべくデータは生のままで，これをどのように分析するかを考えるべきである。その時，「このデータはどのようなメカニズムで生まれてきたのか」という視点からアプローチする。ここでのメカニズムは確率分布と言い換えても良いかもしれない。すなわち，個々の反応は確定した一つの値しか取らないわけがないので，その反応のあり得るほかの値をかんがえ，その総覧を確率分布として表現するのである。その上で，その確率分布が持つパラメータが，どのような仕組みを持っているかを数式で記述する。

回帰分析は，個々の値 y_i が，本来取りうる値 $\hat{y}_i$ に誤差 e_i がついて生じた，と考えている。この誤差は正規分布に従うから，$y_i \sim N(\hat{y}_i, \sigma^2)$ と記述される。この $\hat{y}_i$ は説明変数 x_i の線形結合で表現されるから，$y_i \sim N(\beta_0 + \beta_1 x_i, \sigma^2)$ とする，といった具合である。

回帰分析がこのようなメカニズムであったように，t 検定や分散分析なども同様に記述することができる。こうした既存のモデルを改めて記述すると，これまで意識していなかったモデルの性質が見えてくる。例えば，確率分布として何を仮定していたのか，パラメータの制約として何をおいていたのか，事前分布として何を考えていたのか，といったことが，Stan の言語で逐一記述することでわかるようになる。これが，既存のモデルを Stan の言語で書き直すことによる学習の利点である。

3. 様々なモデルを試してみる

既存のモデルが確率的プログラミング言語で表現できることがわかれば，つぎは確率的プログラミング言語でないと表現できないことに目を向けてみよう。

例えば t 検定や分散分析は「平均値の差の検定」である。ここで行われていたことは，正規分布に従うデータの，平均値の差があるかどうか，全ての群間に差がないと言って間違える可能性はどれぐらいあるか，ということで

あった。データ生成メカニズムの観点から見ると，この手法はごく限定的な一部分しか見ていなかったことに気づく。

平均値以外のパラメータを考えることはできないのだろうか。特定の群間の差だけを考えることはできないのだろうか。差があるかないかだけではなく，どれぐらい差があるのかとか，一方が他方より大きい確率はどれぐらいか，といったことを考えることはできないのだろうか。

これらの疑問に対して，ベイジアンモデリングは答えを与える。実験計画法によって得られたデータであっても，これまで以上に多角的な視点，様々な仮説を持って考えることができる。

もちろん実験計画法による要因効果の特定だけがベイジアンモデリングではない。正規分布ではないデータに対しても，特定の離散的な区別をしていないデータに対しても，データ生成の観点からモデルを組み込んでいくことができる。以下ではこうした例をいくつか見ていくが，これらを見ることで統計分析の視点が一変することを感じてほしい。統計分析は，与えられたデータに既存の分析を制約の中で考えるのではなく，データの生成メカニズムをクリエイティブに考える楽しい営みなのである。

4. 限界について知っておこう

ベイジアンモデリングの自由さ，創造性に目覚めてしばらくすると，その限界に気づくこともあるだろう。まずは楽しんでいってほしいというところだが，先にどのような壁に直面しがちなのか，みておこう。

一つはモデル評価の問題である。例えば帰無仮説検定の場合，これは評価・判断をするための技術であるから，「設定した有意水準を下回る p 値を得れば差があると言って良い」といった評価基準が明確であった。これに対して，ベイジアンモデリングを行うと，こうした「Yes か No か」といった答えは出しにくい。帰無仮説と対立仮説というモデル，あるいは自分が開発したモデルが既存のモデルに比べて，良いのか悪いのかといった判断基準をどう持てば良いのか。

これについての答えは明確で，**ベイズファクター (Bayes Factor)** をみよ，というのがそれである。ベイズ的モデル評価はこの BF に一元化できるといっても過言ではない。BF はモデルとデータの当てはまりの良さを，モデル同士の相対比較で表現するものであるから，対立仮説よりも帰無仮説のほうが良い，という結論を出すこともできる。ただし，この「当てはまりのよさ」(周辺対数尤度) を計算するプロセスが少し複雑で，またモデルによっては解析的に計算できず推定するしかないこともある[*3]。この点については，今後の計算機科学の発展が望まれる。

t 検定や分散分析など，定型的なモデルについては BF を自動的に算出してくれるパッケージやアプリケーションがある。JASP(JASP Team, 2025) はその代表的なもので，GUI を備えた統計ソフトウェアでありながら，既存の分析結果と同時にベイズ推定の結果も出力し，BF も自動的に計算してくれる。

ただし，BF も「3.0 より大きければ優っていると判断して良い」という数値基準もあるが，こうした「Yes か No か！」という二値判断が，過大な解釈を許したり基準を超えるための不正を生んだりしてきたという歴史を鑑みると，使い方には注意が必要である。また BF は事前分布の置き方によっては同じモデルでも大きく値を変えることが知られており，客観的な事前分布の置き方については様々な議論がある。

BF を離れてモデルを評価するのであれば，得られた事後分布やパラメータを見て色々判断するしかないだろう。Kruschke (2018) は事前に判断するパラメータの領域を宣言しておく方法を考えているし，豊田 (2020) は事後分

[*3] 詳しくは (浜田他，2019) を参照してほしい

布の関数の形で判断する方法を提示したりしている。これらは帰無仮説検定に対する代案として提示されているものである。今後どのような形で議論が進むのか，まだ確定していないというのが現状である。

モデルの評価は BF でできるとして，次に初学者が直面する問題は，「自分でモデルを作るのが難しい」というものである。以下に続く様々なモデルはどれも魅力的であるが，自分では思いつかないよ，と思って挫けそうになるという相談をよく耳にすることがある。これについては特効薬があるわけではないが，そもそもゼロから全てのモデルを作り上げよう，とするのが大きすぎる野望のように思われる。まずは色々なモデルを知って，このモデルを自分のデータにはこのように応用できそうだ，と想像力を働かせるところから進めよう。あるいは非常に限定的な，小さなおもちゃのようなモデル (Toy モデル) を作って，それを徐々に発展させていくことで大きなモデルに育てる，という観点を持つことである。浜田（2018）や 浜田（2020）を読むと，このステップの重要さがよくわかるだろう。

そもそも，線形モデルでも十分なシーンというのも結構あるものである。ベイジアンモデリングで自分似合ったデータをカスタマイズする，と豪語しておいた後で言うのもおかしいが，はっきりした傾向があるのであれば線形モデルで大体うまくいく。線形モデルはピッタリとは言わないが大体当てはまっていて理解できるモデルであり，モデリングは細かな違いに当てはめていこうとする「作り込み」の技術であるから，実践的にはそこまで必要のないことも少なくない。もちろん線形モデルといってもただの単回帰分析で良い，といってるのではなく，データの生成メカニズムにあった一般化線形モデル，混合モデルなど工夫できるところは色々あるのだから。

こうした問題やぶつかりそうな壁があると知ってもなお，ベイジアンモデリングはおすすめできる。この自由で創造的な世界を知らずして，統計分析が苦手だと思ってしまうのは非常にもったいないからである。以下の用例で，ベイジアンモデリングの様々な可能性を味わっていただきたい。

17.2　正規分布を使ったモデル

まずはこれまでもよく用いられてきた，正規分布を使ったモデルについて見てみよう。

17.2.1　分散を推定する

正規分布を使ったモデルといえば，一般線形モデルのような平均値に関するモデルがほとんどである。正規分布は位置パラメータ μ と，スケールパラメータすなわち幅のパラメータ σ でその形状が定まるが[*4]，後者はブレの大きさ，誤差の大きさに関するものと考えられるから，そうした指標としてモデルを考えてみよう。

カバーストーリーとして次のようなシーンを考える[*5]。ある牛丼チェーン店では，並盛り一人前につき 150g の肉を載せて提供すること，という決まりになっている。ここである店舗で社員による抜き打ち検査があり，提供係 10 名に牛丼を作らせ，その肉の量を計測した。10 名のうち 2 名はまだ日の浅いアルバイトであることがわかっている。計測した肉の量は以下の通りであった。

[*4] 正規分布の幅のパラメータは，テキストによっては分散 σ^2 で記載されていることが多いが，ここでは標準偏差 σ で記述する。Stan では標準偏差で記述するようになっているので，それに合わせたいからである。標準偏差を二乗したものが分散であるから，意味するところは本質的に変わりがない。

[*5] このカバーストーリーとモデルはリー・ワゲンメーカーズ（2017）の「七人の科学者」P.48–49 を参考に作ったものである。

```
y <- c(151, 149, 152, 150, 151, 148, 151, 150, 221, 245)
```

ここから次のようなモデルを考えよう。全員平均 $\mu = 150$ のつもりで提供しているが，誤差が個人ごとに異なるものとする。これを数式で次のように表現する。

$$y_i \sim N(\mu, \sigma_i)$$

添字を注意深く見るとわかるが，個人を識別する i がデータ y_i と標準偏差 σ_i についていて，平均値 μ にはついていない。つまり平均値は全員で共通していると仮定し，そこからのブレが個人ごとに違うというモデルになっている。

これがデータ y_i を生成するデータ生成メカニズムである。ここでの未知数 μ, σ_i をデータから推測するために，Stan による MCMC 法を用いる。Stan のコードは次のようになる。このコードを，例えば gyudon10.stan とでも名前をつけて保存しておこう。

```
data{
    int N;
    array[N] real Y;
}
parameters{
    real mu;
    array[N] real<lower=0> sigma;
}
model{
    // likelihood
    for(i in 1:N){
        Y[i] ~ normal(mu, sigma[i]);
    }
    // prior
    mu ~ uniform(0, 200);
    sigma ~ cauchy(0,5);
}
```

コードが data と parameters，model の 3 つのブロックに分かれていることに注意してほしい。また，各ブロックで int や array などの型宣言があることに注意してほしい。

まずは data ブロックを見よう。int は整数型で，まずデータのサイズを外部から入力するようにしている。これで，例えばデータが 7 件とか 50 件と変わった時でも同じコードが使えるようにしている。また array は配列の型宣言であり，同じ変数名で複数の値が入るようになっている。ここでは N 人分のデータを扱うために array[N] real Y; というように宣言している。

続く paramters ブロックでは，知りたい未知数の μ と σ_i をそれぞれ real mu; と array[N] real<lower=0> sigma; としている。real は実数型，array はすでに述べたように配列で，σ_i の i によって異なる σ である様を

表現している。<lower=0>としているのは変数に対する制約で，この σ_i は下限が 0，すなわち正の数しかとらないことにしている。分散は負になることがないので，こうしておくと Stan が MCMC サンプリングにおいて可能な値の候補を探す領域が適切に制限されることになる。

最後の model ブロックは，確率モデルを記述する。ベイズ的確率モデルは尤度と事前分布が必要で，まず尤度を記述している。Y[i] ~ normal(mu, sigma[i]) のところがそれで，i が for 文によって繰り返されている。数式で言えば，次の計算と同じである。

$$\prod_{i=1}^{N} N(Y_i|\mu, \sigma_i)$$

実際の計算は対数尤度をとって，次の計算を行っているが，Stan では「データが次の確率分布に従う」という形で書けるので，確率分布を使ったモデルさえかければ誰でも推定ができる。

$$\sum_{i=1}^{N} \log\{N(Y_i|\mu, \sigma_i)\}$$

また今回はそれぞれの事前分布として，μ に 0 から 200 までの一様分布を，σ にコーシー分布を置いた。どのような事前分布をおくかは自由だし，特段指定がなければ Stan は十分にひろい一様分布を自動的に設定する。分散 (標準偏差) パラメータの事前分布には裾の重い分布をおくことが一般的で，コーシー分布や Student の t 分布，指数分布などがよく用いられる。

さてこのコードを，次の R コードから呼び出して実行する。手順は (必要なライブラリを読み込んだ上で)，stan ファイルをコンパイルし，できたオブジェクトにサンプリングの設定を与えて出力する，というものである。

サンプリングに際してはデータを外部から与える必要があるため，dataSet オブジェクトを作って渡すようにした。与えるデータはリスト型であり，stan ファイル側の data ブロックと同じ変数名をつける必要がある。サンプリングのその他の設定は次のとおりである。

- data…データを与える
- chains…MCMC チェインを走らせる数
- parallel_chains…MCMC チェインのうち，並列で走らせるチェインの数。実行環境の CPU が持っているコア数-1 程度にするのが良い。
- iter_warmup…サンプリングに入る前の調整期間。詳しくは@sec-mcmc-evaluation を参照
- iter_sampling…サンプリングの数。詳しくは@sec-mcmc-evaluation を参照
- reflesh…サンプリングの途中経過をどの程度の頻度で出力するかの設定。必ずしも設定しなくとも良い。

```
pacman::p_load(cmdstanr)
model <- cmdstan_model("gyudon10.stan")
dataSet <- list(
  N = length(y),
  Y = y
)
```

```
fit <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 2000,
  iter_sampling = 5000,
  refresh = 0,
  save_cmdstan_config = TRUE
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.2 seconds.
Chain 2 finished in 0.2 seconds.
Chain 3 finished in 0.2 seconds.
Chain 4 finished in 0.3 seconds.

All 4 chains finished successfully.
Mean chain execution time: 0.2 seconds.
Total execution time: 0.4 seconds.
```

```
print(fit)
```

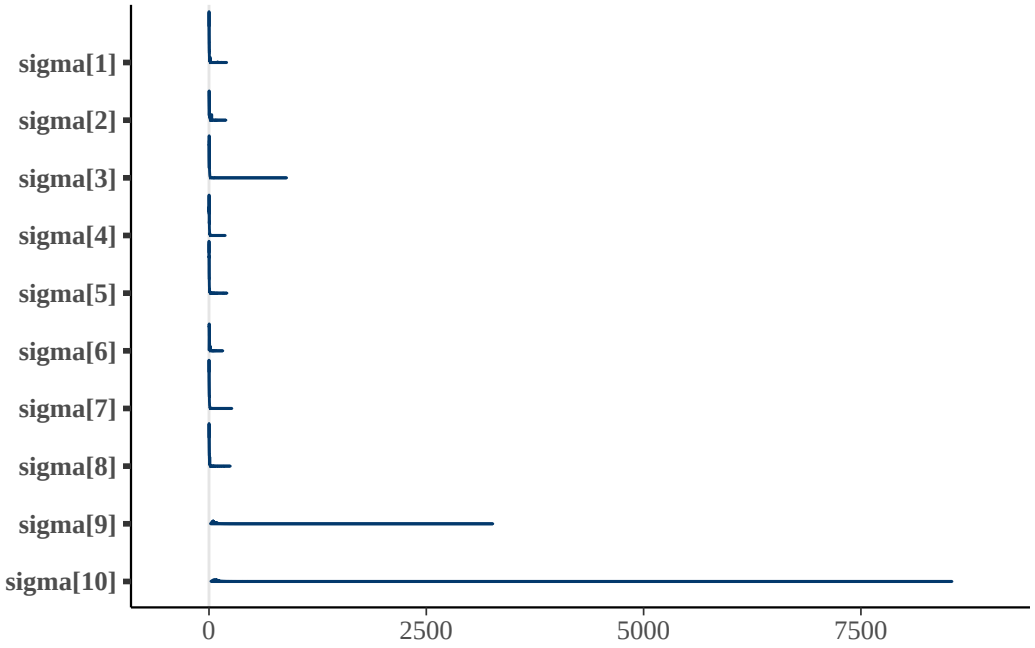
```
 variable   mean median    sd  mad     q5    q95 rhat ess_bulk ess_tail
 lp__     -18.84 -18.53  2.23 2.35 -22.79 -15.72 1.01      392     1361
 mu       150.53 150.69  0.64 0.60 149.43 151.34 1.01      908     4445
 sigma[1]   4.04   1.53  9.86 1.78   0.13  15.89 1.05       64      358
 sigma[2]   5.81   3.20  7.85 2.61   0.74  29.20 1.04       77       16
 sigma[3]   4.50   2.63 12.07 2.05   0.71  12.31 1.04      418      555
 sigma[4]   3.17   1.99  5.17 1.79   0.13   9.72 1.01      533      274
 sigma[5]   2.71   1.51  5.36 1.66   0.03   8.93 1.04       64       16
 sigma[6]   5.97   3.92  6.81 2.71   1.38  15.87 1.06      232     1457
 sigma[7]   3.06   1.60  6.63 1.88   0.07   9.43 1.04       75       20
 sigma[8]   3.30   1.71  6.65 1.79   0.03   9.62 1.06       49       20

 # showing 10 of 12 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

結果を見ると，μ は平均して 150 程度であり，店舗マニュアルに沿った結果が出ているといえよう。注目すべきは個々人の誤差であり，可視化してみるとその特徴がわかりやすい。

```
pacman::p_load(bayesplot)
draws <- fit$draws()
# sigmaパラメータのみを可視化
```

```
bayesplot::mcmc_areas(draws, regex_pars = "sigma")
```



これをみると明らかなように，最後の 2 人が大きな値であり，明らかに未熟であることがわかる。数値で出力するために，bayestestR パッケージの力も借りてみよう。

```
pacman::p_load(bayestestR)
# sigmaパラメータのみを選択
sigma_draws <- draws[, , grepl("sigma", dimnames(draws)$variable)]
# EAP(平均), MAP(最頻値), 中央値, 95%HDI
bayestestR::describe_posterior(sigma_draws,
  centrality = c("mean", "median", "MAP"),
  ci = 0.95, ci_method = "hdi", test = NULL
)
```

```
Summary of Posterior Distribution

Parameter | Median |   Mean |   MAP |          95% CI
-----------------------------------------------------
sigma[1]  |   1.53 |   4.04 |  0.23 | [ 0.07,  15.89]
sigma[2]  |   3.20 |   5.81 |  1.56 | [ 0.03,  29.20]
sigma[3]  |   2.63 |   4.50 |  1.81 | [ 0.06,  12.31]
sigma[4]  |   1.99 |   3.17 |  1.11 | [ 0.02,   9.72]
sigma[5]  |   1.51 |   2.71 |  0.22 | [ 0.01,   8.94]
sigma[6]  |   3.92 |   5.97 |  3.94 | [ 0.43,  16.04]
sigma[7]  |   1.60 |   3.06 |  0.28 | [ 0.03,   9.44]
sigma[8]  |   1.71 |   3.30 |  0.73 | [ 0.01,   9.62]
```

```
sigma[9]  |  59.34 |  83.99 | 45.22 | [18.76, 201.42]
sigma[10] |  81.55 | 117.74 | 71.84 | [24.96, 295.40]
```

ここにあるように，結果は確率分布の形で得られるので，平均値 (Mean, EAP) でみるのか，中央値 (Median) でみるのか，密度が最大になるところ (MAP) でみるのかによって大きく値が異なる。特に分散パラメータは左右対称ではなく歪んだ分布になるので，EAP 推定値は適切ではない。また，確信区間は今回 HDI を指定した。HDI は Highest Density Intervals の略で，最高密度を含む 95% の領域を指す。いわゆるパーセンタイルのように，上下 2.5% を除外した領域を取る方法は ETI(Equal-Tailed Intervals) と呼ばれるが，これだと歪んだ分布に対してやや偏った結果になることがある。詳しくはクルシュケ（2017）や Makowski et al. (2019)，あるいは単に bayestestR のサイトを参照して欲しい。

さて，ともあれこのように幅のパラメータを推定するモデルを描くことができた。今回はたった 10 件の数字でもモデリングができること，平均パラメータ以外もモデリングの対象になることを確認してもらいたい。心理学において，特に反復測定を行う場合の個人の分散は，その人の持っている精度・誤差の大きさを表すと言える。今回のようにこの幅が，熟練度と解釈できるようなシーンであれば，立派に解釈可能なパラメータである。例えばこの σ_i に，経験日数 z_i をつかって $\sigma_i = \beta_0 + \beta_1 z_i$ のようなモデリングをすれば，どの程度経験によって誤差がなくなっていくかといった意味のある考察もできるだろう。このように，心理学において考察できるパラメータは平均だけではなく，我々は自由にその発想で考えることができるということを味わってもらいたい。

17.2.2 欠測のあるデータを有効に使う

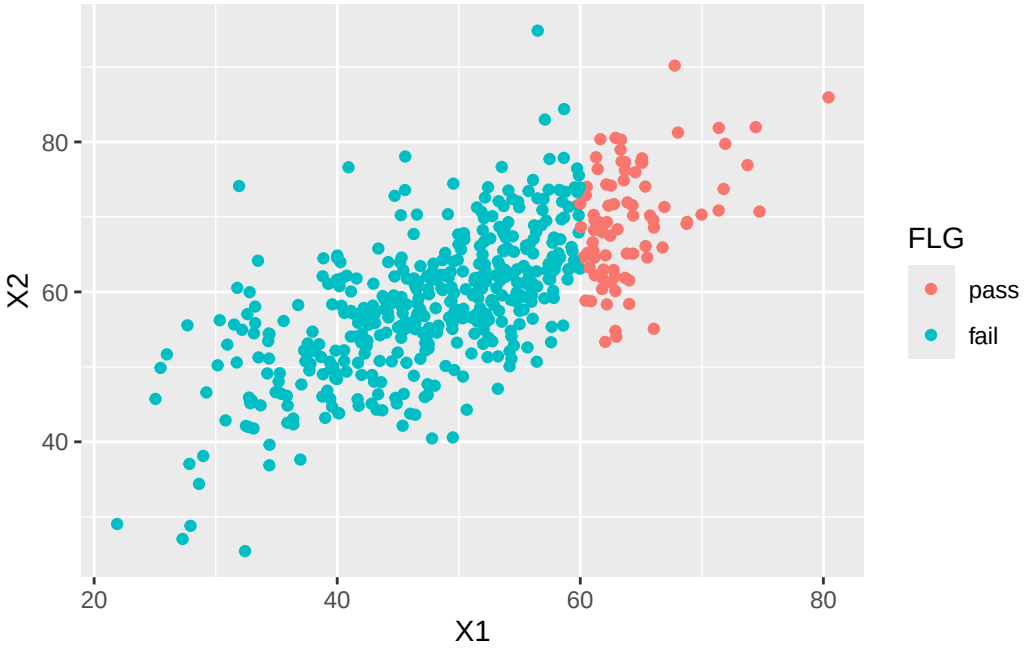
続いては相関係数のモデリングを見てみよう。例えば大学の入試の成績と，入学後の成績の間にはそれほど高い相関が見られない，という現象がある。これは大学の入試が適切に学力を測定していない，という問題ではなく，いわゆる「選抜効果」とよばれるものだ。大学に入学できているのは一定のスコアを超えた者だけなので，得られたデータが本来の「入試の成績」の情報を含んでいないという問題である。

具体的に数字で見てみよう。まずある程度の相関関係を持つデータを作り，その一部を欠損させてみよう。

```
pacman::p_load(MASS, tidyverse)
N <- 500
mu <- c(50, 60)
sd <- c(10, 10)
rho <- 0.7
Sig <- matrix(nrow = 2, ncol = 2)
Sig[1, 1] <- sd[1] * sd[1]
Sig[1, 2] <- sd[1] * sd[2] * rho
Sig[2, 1] <- sd[2] * sd[1] * rho
Sig[2, 2] <- sd[2] * sd[2]
# 乱数の発生
set.seed(17)
X <- mvrnorm(N, mu, Sig, empirical = T)

dat <- data.frame(X)
```

```
dat$FLG <- factor(ifelse(dat$X1 > 60, 1, 2), labels = c("pass", "fail"))
# 描画
g <- ggplot(dat, aes(x = X1, y = X2, group = FLG, color = FLG)) +
  geom_point()
g
```



ここでは 60 点以上の者が入学したとして切断してみた。この場合の相関係数を，欠測値を除く use=complete.obs のオプションをつけて確認しておく。

```
# フルデータの場合
cor(dat$X1, dat$X2)
```

```
[1] 0.7
```

```
# 欠測値に置き換える
dat[dat$FLG == "fail", ]$X2 <- NA
# 改めて相関係数を算出
cor(dat$X1, dat$X2, use = "complete.obs")
```

```
[1] 0.4284751
```

不合格者が欠測値だとすると，相関係数は 0.428 にまで落ちてしまった。

さてこれをベイジアンモデリングでできるだけ補正してみよう。コードは次のようなものになる。

```
data{
    int<lower=0> Nobs;
    int<lower=0> Nmiss;
```

```
    array[Nobs] vector[2] obsX;
    array[Nmiss] real missX;
}

parameters{
    vector[2] mu;
    real<lower=0> sd1;
    real<lower=0> sd2;
    real<lower=-1,upper=1> rho;
}

transformed parameters{
    cov_matrix[2] Sig;
    Sig[1,1] = sd1 * sd1;
    Sig[1,2] = sd1 * sd2 * rho;
    Sig[2,1] = Sig[1,2];
    Sig[2,2] = sd2 * sd2;
}

model{
    //likelihood
    obsX ~ multi_normal(mu, Sig);
    missX ~ normal(mu[1], sd1);
    //prior
    mu[1] ~ uniform(0,100);
    mu[2] ~ uniform(0,100);
    sd1 ~ cauchy(0,5);
    sd2 ~ cauchy(0,5);
    rho ~ uniform(-1,1);
}
```

このコードでは，まずデータブロックで Nobs と Nmiss という二つの整数値をとっている。これは 2 つの変数が両方とも観測されたケースの数と，一方が欠測値であったケースの数である。Stan では NA を直接扱うことができず，有効なデータの数だけ渡す必要があり，冒頭にその数を明示しておいた。

次に vector 型で obsX[Nobs] を宣言している。vector[2] とあるのは 2 つの要素を持つベクトルで 1 セットの変数であることを意味し，それが Nobs 個あることを表している。最後に，一方の変数が欠落していたデータも活用するために，Nmiss 個の配列変数を用意した。こちらはベクトルではなく，サイズをしていした実数の配列の扱いである。

今回推定したいのは相関係数 rho だが，これは 2 次元の多変量正規分布を想定した時の分散共分散行列の中に現れる。尤度のところにあるように，平均ベクトル μ と，分散共分散行列 Σ からデータは生成されている。

$$obsX \sim MN(\mu, \Sigma)$$

ここで分散共分散行列の要素を紐解くと，

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{pmatrix} = \begin{pmatrix} \sigma_1^2 & \sigma_1\sigma_2\rho \\ \sigma_1\sigma_2\rho & \sigma_2^2 \end{pmatrix}$$

となる。ここで σ_1, σ_2 はそれぞれの変数の標準偏差であり，ρ は相関係数である。

`parameters` ブロックでは，この σ_1, σ_2, ρ を宣言しておき，これを `transformed parameters` ブロックで分散共分散行列に書き直している。なお `cov_matrix` 型は Stan のもつ分散共分散行列専用の型である。

`model` ブロックも注意深くみて欲しい。まず 2 つの変数が揃っている `obsX` は多変量正規分布から生成されている。そして 1 つの変数が欠如している `missX` も，`X1` 側のデータは残っているので，これをいわゆる普通の (単変量の) 正規分布から生成されたものとしている。`mu[1]` と `sd1` の尤度を追加することで補正を試みているのだ。

平均値の事前分布には，テストなので 0 点から 100 点の間のどこかにあるから，`uniform(0,1)` を置いた。標準偏差の事前分布には裾の重いコーシー分布を，相関係数の事前分布には，Stan がもつ相関係数用の分布である `lkj_corr` を置いた。これのパラメータが `1` であれば，相関係数に適した無情報分布を置いたことと同義である。

さて，このモデルをコンパイルし，データを与えて `rho` がどうなるかみてみよう。データの与え方に注意して欲しい。

```
model <- cmdstan_model("missing_corr.stan")

dataSet <- list(
  Nobs = sum(!is.na(dat$X2)),
  Nmiss = NROW(dat$X1) - sum(!is.na(dat$X2)),
  obsX = dat[!is.na(dat$X2), c(1, 2)],
  missX = dat[is.na(dat$X2), 1]
)

fit <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  refresh = 0,
  save_cmdstan_config = TRUE
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.8 seconds.
Chain 2 finished in 0.8 seconds.
Chain 3 finished in 0.7 seconds.
```

```
Chain 4 finished in 0.7 seconds.

All 4 chains finished successfully.
Mean chain execution time: 0.7 seconds.
Total execution time: 0.8 seconds.
```

```
bayestestR::describe_posterior(fit$draws("rho"),
  centrality = c("mean", "median", "MAP"), ci = 0.95, ci_method = "hdi", test = NULL
)
```

```
Summary of Posterior Distribution

Parameter | Median | Mean |  MAP |       95% CI
-------------------------------------------------
rho       |   0.73 | 0.71 | 0.76 | [0.50, 0.88]
```

今回推定された rho は 0.711 と，ほぼ理論通りの値を復元できている。多変量正規分布は，各変数に限って見てみると一変数の正規分布と同じであり，今回欠測のない X1 の情報を尤度に追加することで，持ちうるデータの情報全てを活用することができた。このように，得られたデータが限定的であってもそのデータ生成メカニズムを考えることで補正できることは少なくない。たとえば上限や下限の値が決まっているデータに対し，そうした打ち切りがなければ本当は何点だったのか，といったこともモデルの工夫で推定することができるようになる。[*6]

17.3　正規分布以外の分布を使う

ベイジアンモデリングは，確率分布のパラメータに数式的構造を仮定することでもある。ここでは正規分布以外の確率分布を使った例を見てみよう。

17.3.1　項目反応理論

すでに Section 15.3 で見たように，項目反応理論はまさに正規分布以外のパラメータに数式的構造を持たせたものであった。すなわち，結果変数が 0/1 のバイナリデータであるから，確率分布としてはベルヌーイ分布を用い，パラメータ θ は 1 が出る確率 (正答する確率) と考えられるから，この θ に潜在的な学力を仮定したロジスティックモデルを入れるのであった。

この観点で数式を書き表すと，ある項目 j に個人 i が回答したパターン Y_{ij} は次のように表現できる。

$$Y_{ij} \sim Bernoulli(p_{ij})$$

$$p_{ij} = \frac{1}{1 + \exp(-a_j(\theta_i - b_j))}$$

これを Stan によるモデルで実装してみよう。コードは次のようになる。

[*6] たとえば紀ノ定 (2018) が参考になる。

```
data{
  int<lower=0> L; // data length
  int<lower=0> N; // number of persons
  int<lower=0> M; // number of questions
  array[L] int<lower=0> Pid;  // personal ID
  array[L] int<lower=0> Qid;  // question ID
  array[L] int<lower=0,upper=1> resp; // response
}

parameters{
  array[M] real<lower=0> a;
  array[M] real<lower=-5,upper=5> b;
  array[N] real theta;
}

model{
  //likelihood
  for(l in 1:L){
    resp[l] ~ bernoulli_logit(1.7*a[Qid[l]]*(theta[Pid[l]]-b[Qid[l]]));
  }

  //prior
  a ~ lognormal(0,sqrt(0.5));
  b ~ normal(0,3);
  theta ~ normal(0,1);
}
```

ここではデータが Long 型の整然データになっていることが前提である。すなわち，一行には受験者 ID，項目番号 (項目 ID) と正答誤答の反応が入った 3 列のデータである。

これを受けて `data` ブロックでは，まずデータ全体の長さ `L` と，受験者数 `N`，項目数 `M` の値を整数型で受け取り，これらの数字に応じた配列の変数を用意している。受験者の ID は `Pid`，項目の ID は `Qid`，反応は `resp` であり，それぞれ長さ `L` の配列である。受け取る数値の範囲にも注意してもらいたい。

`parameters` ブロックでは，項目パラメータの `a_j` と `b_j` をサイズ `M` で，受験者パラメータの `theta` をサイズ `N` で用意している。

`model` ブロックでは `bernoulli_logit` という関数で表しているが，これはロジスティック関数とベルヌーイ関数を組み合わせたものが Stan に最初から用意されているからである。`l` 行目の反応 `resp[l]` に対し，項目パラメータと受験者パラメータを使って数式が組み込まれている。`theta[Pid[l]]` のように二重の代入になっている。こうすることで `l` 行目の `Pid[l]` が整数値を返し，`l` 行目の `theta_i` を表している。事前分布に関して，`a_j` と `b_j` それぞれに対数正規分布と正規分布を，`theta_i` に標準正規分布をおいた。

実際に推定してみよう。今回も `exametrika` パッケージの `J15S500` データセットを使うが，Long 型に加工して用いていることに注意してほしい。

```
pacman::p_load(exametrika, tidyverse)
dat <- J15S500$U
dat_long <- dat %>%
  as.data.frame() %>%
  rowid_to_column("ID") %>%
  pivot_longer(-ID)

model <- cmdstan_model("irt2pl.stan")
dataSet <- list(
  L = NROW(dat_long),
  M = 15,
  N = 500,
  Pid = dat_long$ID,
  Qid = as.numeric(as.factor(dat_long$name)),
  resp = dat_long$value
)

fit <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  refresh = 0,
  save_cmdstan_config = TRUE
)
```

```
Running MCMC with 4 parallel chains...

Chain 4 finished in 43.1 seconds.
Chain 2 finished in 52.2 seconds.
Chain 1 finished in 57.7 seconds.
Chain 3 finished in 60.9 seconds.

All 4 chains finished successfully.
Mean chain execution time: 53.5 seconds.
Total execution time: 61.1 seconds.
```

```
fit$draws(c("a", "b")) %>%
  describe_posterior(
    centrality = c("mean", "median", "MAP"), ci = 0.95,
```

```
    ci_method = "hdi", test = NULL
  )
```

```
Summary of Posterior Distribution

Parameter | Median |  Mean |   MAP |         95% CI
-------------------------------------------------
a[1]      |   0.40 |  0.41 |  0.41 | [ 0.25,  0.58]
a[2]      |   0.47 |  0.48 |  0.47 | [ 0.30,  0.66]
a[3]      |   0.31 |  0.31 |  0.29 | [ 0.17,  0.46]
a[4]      |   0.87 |  0.88 |  0.83 | [ 0.61,  1.18]
a[5]      |   0.39 |  0.39 |  0.39 | [ 0.23,  0.55]
a[6]      |   0.58 |  0.59 |  0.57 | [ 0.36,  0.84]
a[7]      |   0.65 |  0.65 |  0.65 | [ 0.45,  0.86]
a[8]      |   0.40 |  0.41 |  0.41 | [ 0.25,  0.56]
a[9]      |   0.18 |  0.18 |  0.17 | [ 0.09,  0.29]
a[10]     |   0.27 |  0.27 |  0.26 | [ 0.15,  0.40]
a[11]     |   0.68 |  0.69 |  0.66 | [ 0.47,  0.92]
a[12]     |   0.74 |  0.75 |  0.74 | [ 0.50,  0.99]
a[13]     |   0.51 |  0.52 |  0.49 | [ 0.35,  0.70]
a[14]     |   0.73 |  0.73 |  0.69 | [ 0.48,  0.96]
a[15]     |   0.48 |  0.49 |  0.48 | [ 0.32,  0.66]
b[1]      |  -1.73 | -1.79 | -1.65 | [-2.56, -1.11]
b[2]      |  -1.56 | -1.61 | -1.50 | [-2.22, -1.09]
b[3]      |  -1.95 | -2.05 | -1.82 | [-3.08, -1.14]
b[4]      |  -1.16 | -1.17 | -1.12 | [-1.48, -0.90]
b[5]      |  -2.33 | -2.41 | -2.20 | [-3.51, -1.56]
b[6]      |  -2.18 | -2.26 | -2.08 | [-3.07, -1.54]
b[7]      |  -1.03 | -1.05 | -1.00 | [-1.38, -0.75]
b[8]      |  -0.57 | -0.59 | -0.56 | [-0.95, -0.23]
b[9]      |   1.85 |  1.95 |  1.64 | [ 0.86,  3.20]
b[10]     |  -1.52 | -1.60 | -1.45 | [-2.53, -0.85]
b[11]     |   1.00 |  1.01 |  0.98 | [ 0.72,  1.35]
b[12]     |   1.01 |  1.02 |  0.98 | [ 0.75,  1.35]
b[13]     |  -0.73 | -0.74 | -0.68 | [-1.06, -0.43]
b[14]     |  -1.22 | -1.23 | -1.20 | [-1.59, -0.89]
b[15]     |  -1.21 | -1.23 | -1.15 | [-1.71, -0.84]
```

`exametrika` やそのほかの IRT モデルでは一般に，数が多くなる受験者パラメータを積分消去することで計算を効率化し，EM アルゴリズムなどで解析的に解く。一方ここで示した MCMC によるベイズ推定は，全てのパラメータが同時に成立しうる可能な空間を探索しながら代表値 (MCMC サンプル) を得るため，計算効率は非常に

悪く，結果の出力まで少し時間を要する環境もあるだろう。しかし Stan のコードを書くことによって，どのような計算がなされているかが確認できるし，データ生成メカニズムがわかれば，様々な加工，工夫もやりやすくなるだろう。

17.3.2　再捕獲法による全体の推論

正規分布以外の分布を用いた例として，今度は少し特殊な例を見てみよう。その前に，どういうことを目的としているかのカバーストーリーを考える[*7]。

ある時，あなたの友人はいつも同じような服を着ていることに気がついた。そこである一週間の服装をしっかり観察してみると，いちおう毎日違うコーディネートのようで，7 種類のパターンがあることがわかった。同じく翌週もみてみると，5 日間の観察のうち，4 回は先週と同じコーディネートで着まわしていることがわかった。さて，この友人は全体で何パターンぐらいの服を着まわしているだろうか。

これはあくまでも身近な例で考えてみたものであって，同じような問いの構造を別の例で考えてみよう。

- ある池に魚を釣りに行き，`X` 匹釣れたとする。その `X` 匹に印をつけリリースする。別の機会に魚を釣りに行くと今度は `N` 匹釣り上げることができて，そのうち `K` 匹は前回つけた印のある魚であった。このとき，この池には何匹ぐらいの魚がいるだろうか。
- ある調査では，大学のイメージを自由記述で回答してもらうことにした。調査結果をまとめると，`X` 件のカテゴリに分類できるようだ。また第二回調査を同様に行うと，今度は `N` 件のカテゴリに分類することができ，第一回調査との重複は `K` 件であった。このとき，この調査を続けて「回答の全体像」を考えるとすると，どれぐらいのカテゴリが得られるだろうか。

第 2 の例，池の魚の総数を推定する方法は実際にも行われる調査方法であり，第 3 の例は質問紙調査における理論的飽和を参考にしたサンプルサイズ設計 (豊田他，2013) などに応用可能であることが議論されている。これらの手法は応用シーンが全く異なるものであるが，確率モデルとしては同じで一般に捕獲 – 再捕獲法と呼ばれている。

さて，では捕獲 – 再捕獲問題を確率モデルを確率モデルとして考えよう。ここで用いるのは超幾何分布 (Hypergeometric Distribution) である。超幾何分布は有限母集団から，非復元抽出を行う際の確率分布であり，次のように定式化される。

$$P(X = k) = \frac{{}_xC_k \times {}_{(t-k)}C_{n-k}}{{}_tC_n}$$

ここで `t` は母集団のサイズ，`x` は最初のサンプルサイズ，`n` が 2 回目のサンプルサイズで，`k` が再捕獲された標本の数である。また，`C` は組み合わせの数を表す。2 回目のサンプル `n` は，全体 `t` からとりうる組み合わせのうち，1 回目のサンプル `x` から `k` が得られる組み合わせ，かつ，残りの `t-k` から印のない `n-k` を取り出す組み合わせ，から構成されていると言える。

[*7] このカバーストーリーとモデルはリー・ワゲンメーカーズ（2017）の「飛行機を再捕獲する」P.63–66 を参考に作ったものである。なおこの本の Stan コードはこちらにあるので参考にしてほしい。書き写すだけでもかなりの勉強になることは間違いない。

確率分布を使って表現すると，次のようになる[*8]。

$$k \sim Hypergeometric(n, x, t - x)$$

Stan モデルで見てみよう。

```
data {
  int<lower = 0> x;  // size of first sample (captures)
  int<lower = 0> n;  // size of second sample
  int<lower = 0, upper = n> k;  // number of recaptures from n
  int<lower = x> maxT;  // maximum population size
}

transformed data {
  int<lower=x> minT;
  minT = x + n - k;
}

transformed parameters {
  vector[maxT] lp;
  for (i in 1:maxT){
    if (i < minT)
      lp[i] = log(1.0 / maxT) + negative_infinity();
    else
      lp[i] = log(1.0 / maxT) + hypergeometric_lpmf(k | n, x, i - x);
  }
}

model {
  target += log_sum_exp(lp);
}

generated quantities {
  int<lower = minT, upper = maxT> t;
  simplex[maxT] tp;
  tp = softmax(lp);
  t = categorical_rng(tp);
```

[*8] ここでは Stan の Hypergeometric 関数の書き方に従った。テキストによっては (例えばリー・ワゲンメーカーズ（2017）)`Hypergeometric(n,x,t)` とかかれることもあるが，Stan では `k \sim Hypergeometric(n, a, b)` で，`n` が 2 回目のサンプルサイズを指し，`a` が 1 回目のサンプルサイズ，`b` が 1 回目のサンプルで母集団から取り出せなかった数を表す。母集団全体の大きさは `a+b` であり，ここでの表記では `x + t-x` なので `t` である。

```
}
```

この Stan モデルには色々なテクニックが内包されているので，順に確認していこう。

17.3.2.1　事前分布について

データとしては，最初のサンプルサイズ x，2 度目のサンプルサイズ n，再捕獲した個体数 t と，推定に必要なおおよその最大数 maxT を与える。たとえば例 1 のコーディネートパターンであれば，30 もあれば十分だろう。

事前分布としては，1 から maxT のどこにでもありうるのだから，無情報事前分布として一様に 1.0/maxT を置くといいだろう。

17.3.2.2　target 記法について

尤度について説明する前に，Stan の target 記法について説明しておかなければならない。これまで，尤度関数を例えば次のように記載してきた。

```
Y[i] ~ normal(mu, sigma)
```

この記法は Sampling 記法と呼ばれるが，これと同じモデルを別の記述で表現することができる。それが target 記法で，上と同じモデルを target 記法で書くと次のようになる。

```
target += normal_lpdf(Y[i]|mu,sigma)
```

MCMC の内部では尤度ではなく対数尤度を用いて計算することはすでに述べた。尤度は数字が小さくなりすぎるため，対数を取ることで計算上の安定化を図っているのである。target 記法の normal_lpdf についている lpdf は log-probability density function で対数確率密度関数を表している。モデル全体としては，ケースごとの尤度を掛け合わせる (総積) が[*9]，対数で計算する場合は足し合わせる (総和) ことになる。

足し合わせたモデル全体の対数尤度を target と呼ぶ。また，+=はプログラミング言語特有の表記法の一種で，x = x + y のように同じ変数 x に y を加えてまた x に代入するという作業をするときに，x += y のように記載するのである。

つまり target += normal_lpdf(Y[i]|mu,sigma) は，N(mu, sigma) から得られる Y[i] を loop などでモデル全体の対数尤度 target に追加していけ，という表現になっている。このような複雑な計算が存在する理由は 2 つある。ひとつは今回のように複雑なモデルで，対数尤度を直接記述したい場合や，確率的にモデルが異なるときに異なるモデルの可能性を分けて書いて後で足し合わせる，という工夫をする場合である。もうひとつは，Sampling 記法が計算の高速化のためにカットした部分も省略せずに計算する場合[*10]である。

[*9] これは各ケースが独立に同じ分布 (独立同分布，independent and identically distributed,i.i.d.) から得られているという仮定が成り立つときになされる計算である。サイコロの目が 2 回連続で 1 が出る場合などは，$1/6 \times 1/6$ と計算するが，これは 1/6 の確率が独立しているから掛け合わせよう，ということに他ならない。

[*10] ベイズの公式にある右辺の分母，すなわち周辺尤度 $P(D)$ は，結果を確率分布にするための定数であり，事後分布の形状を算出するのには不要な要素である。Sampling 記法はこの計算をしないことで高速化している。しかしベイズファクターなど周辺尤度をつかう必要がある時は，この計算をカットすることができないため，target 記法が必要になる。

17.3.2.3　尤度について

target 記法について理解したところで，もう一度コードに目を向けてみよう。

データの入力後に `transformed data` ブロックで，`minT` という値を定義してる。数式から，1 度目のサンプルサイズ +2 度目のサンプルサイズ-重複 (再捕獲) した個体の数であることがわかる。すなわちオリジナルな (重複しない) 個体が確認されている数を `minT` としている。

続いて `paramters` ブロックが来るはずだが，ここにそれは存在しない。`paramters` ブロックに書くのは推定対象のパラメータであるが，今回はパラメータが推定対象ではない。特定の `n,x,t` の値から，データ `k` が得られたとするモデルだからである。

`transformed paramters` ブロックで記述されているものが，実質的なモデルの対数尤度である。まず考えうる `1` から `maxT` までのベクトル `lp` を用意する。このベクトルのうち，少なくとも `minT` までは考えなくても良い。そこで

1. `i` を `1` から `maxT` までの for 文で回しつつ，
2. `i < minT` の間は確率ゼロとする。

この「確率をゼロにする」というのは，対数を取ると $\log(0) = -\infty$ だから，事前分布を加えて

```
lp[t] = log(1.0 / maxT) + negative_infinity();
```

とする (`negative_infinity()` は Stan の $-\infty$ 表記)。

また，

3. `i >= minT` の間は，事前分布 (`1.0/maxT`) と超幾何分布 `hypergeometric(n, x, i - x)` のなかから `k` という値が観測される尤度

の組み合わせである。ただし，対数を取るために事前分布に log をとり，尤度を `hypergeometric_lpdf` で表して，積を和に変えている。

このようにして `lp` を構成する要素はすでに準備したので，`model` ブロックではこれらを使うだけで良い。

ここでもう一工夫必要なのだが，`lp[t]` には考えるべき `t` のそれぞれ (ここでは 1,2,3,...,30) について，対数事後確率 (対数尤度＋対数事前確率) が計算されている。これは対数を取った確率だから，`exp` 関数 (指数関数) で事後確率に置き換え，各パターンを足し合わせる (`sum`) ことで全体の事後確率とし，計算の安定性を保ちつつ尤度を計算するために対数を取って `log` をする，という作業が必要である。これを一つの関数 `log_sum_exp` が担っている。

17.3.2.4　生成量について

最後にもうひとつ，`generated quantities` ブロックについて見ておく。

このブロックは，MCMC の推定が終わった後の変数を加工し，生成量とも呼ばれる出力を計算する箇所である。今回はここで，推定される総個体数 `t` を計算している。`t` は少なくとも `minT` で多くとも `maxT` な整数値であるか

ら，型宣言は理解できるだろう。

続いて，lp を softmax 関数に入れて tp を計算する。lp には対数事後確率が入っていたことを思い出そう。また，softmax 関数は数学的には次のような操作をしている。

$$\mathbb{x} = \frac{\exp(\mathbb{x})}{\sum \exp(\mathbb{x})}$$

ここで $\mathbb{x}$ はベクトルであり，今回はサイズ maxT の simplex 型で宣言した。simplex 型とは，全ての要素を足し合わせると 1.0 になるという制約付きベクトルである。総和が 1.0 なので確率として解釈でき，maxT 個のサイズであることから察せられるように，総数 t が 1,2,3,…,30 の各ケースそれぞれでどの程度起こりやすいかを表している。カテゴリカルな分布の確率パラメータのベクトルとして用いられる型である。

さて，総数 t がどういう値をとりやすいかは，対数事後確率に寄るところである。対数事後確率も 1,2,3,…,30 の各ケースの起こりやすさであるから，指数関数で対数をとり，総和を取ったうちの当該要素の確率，すなっわち各ケースの相対確率に置き直すのが softmax 関数の仕事である。

ここで計算された tp は確率に置き換えられた，各ケースの出やすさ (確率質量) である。また，categorical はカテゴリカル分布であり，categorical_rng は Stan の categorical 分布を使った乱数発生関数である。[*11]

例えばサイコロの目は 1,2,3,4,5,6 であり，各 1/6 の確率で生じるが，カテゴリカル分布では出目 x を

$$x \sim Categorical(1/6, 1/6, 1/6, 1/6, 1/6, 1/6)$$

のように表す。今回は tp で t がどのようになるか，乱数で生成することで，MCMC サンプルの後でどのような出目が出やすいかを取り出すことにしている。

さぁ，長かったコードの説明もこれで終わり。実際にコンパイルして推定してみよう。推定の際には parameters ブロックをおいていないことから，fixed_param = TRUE のオプションを入れるのを忘れずに。

```
model <- cmdstan_model("recapture.stan")

x <- 7 # 最初の観測
n <- 5 # 二度目の観測
k <- 4 # 同じ服だった回数
maxT <- 30 # 上限

dataSet <- list(x = x, k = k, n = n, maxT = maxT)

fit <- model$sample(
```

[*11] Stan はこのように確率分布の後ろに_をつけることで同じ確率分布の別の側面を表す。R で正規分布を norm と表し，確率密度を dnorm，累積確率を pnorm，累積確率の逆関数を qnorm，正規乱数を rnorm としているように，Stan では normal の対数確率密度を normal_lpdf，正規乱数を normal_rng とする。カテゴリカル分布の場合は，その対数確率質量を categorical_lpmf(log-probability mass function) とし，カテゴリカル乱数を categorical_rng としているのである。

```
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  fixed_param = TRUE,
  refresh = 0,
  save_cmdstan_config = TRUE
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.0 seconds.
Chain 2 finished in 0.0 seconds.
Chain 3 finished in 0.0 seconds.
Chain 4 finished in 0.0 seconds.

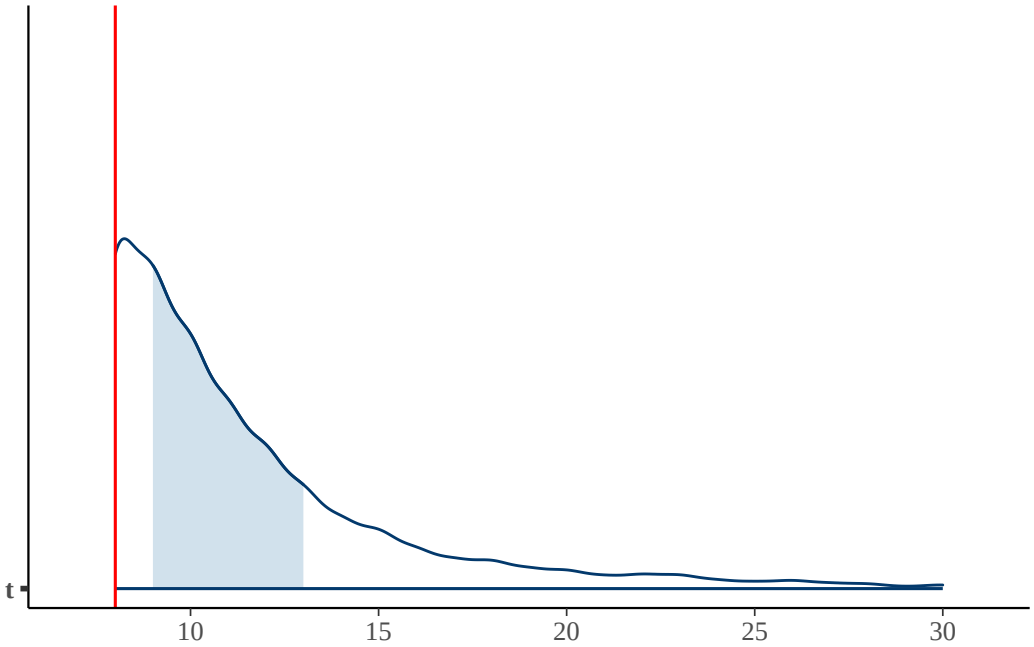
All 4 chains finished successfully.
Mean chain execution time: 0.0 seconds.
Total execution time: 0.2 seconds.
```

```
MAP_value <- bayestestR::describe_posterior(fit$draws("t"),
  centrality = c("mean", "median", "MAP"), ci = 0.95,
  ci_method = "hdi", test = NULL
)
MAP_value
```

```
Summary of Posterior Distribution

Parameter | Median |  Mean | MAP |        95% CI
-------------------------------------------------
t         |     10 | 11.44 |   8 | [8.00, 20.00]
```

```
bayesplot::mcmc_areas(fit$draws("t"), point_est = "none") +
  geom_vline(xintercept = MAP_value$MAP, color = "red")
```



今回の結果から，あなたの友人の持っている服装コーディネートパターンは，せいぜい 8 つであることがわかった。

いかがだっただろうか。

ここでのポイントは，超幾何分布を使ったということもさることながら，「アイデア次第で色々な分析ができる」という点にある。

考えるべき問題を数学的に定式化することで抽象度が上がる (同時に難易度も上がったように感じられる) が，そのことによって同型の問題を特定できること，すなわち様々な応用が可能になる。目の前の問題を考えるときに，データ生成メカニズムや確率モデルで考えることが，クリエイティブな営みであることがわかってもらえただろうか。

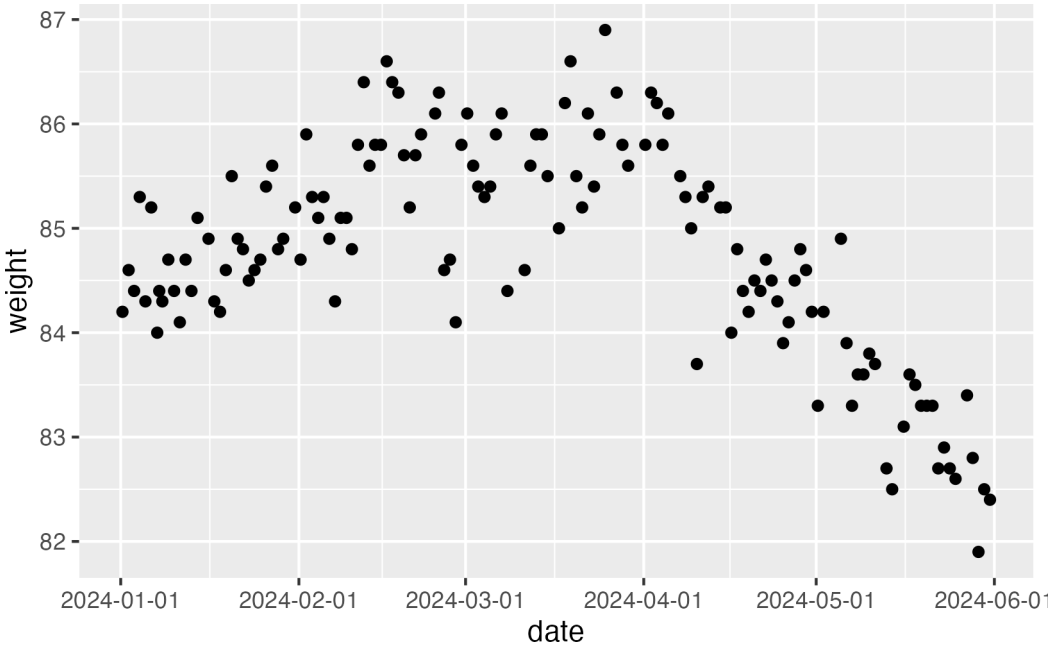
17.4　分布を混ぜる

今度は複数の分布を使うモデルを考えてみよう。

17.4.1　変化点を検出する

こちらのデータをみてもらいたい。読み込んでプロットしたのが次の図である。

```
w <- read_csv("myWeights.csv")
w |>  ggplot(aes(x=date,y=weight))+
  geom_point() +
  scale_x_datetime(date_labels = "%Y-%m-%d")
```



これは筆者の体重の推移で，2024 年の 1 月 1 日から同年 6 月 1 日までの変化である。これを見ると明らかにある時期から推移の傾向が変わっているのが見て取れる。おそらく 4 月の初旬から下降傾向に入ったようだ。このデータに対してなんらかの生成モデルを考える時，4 月までと 4 月からではメカニズムが異なるようなので，別のモデルが必要ではないだろうか。

そこで，ある時点を境に異なるメカニズムであることをモデル化してみよう。Stan のコードは次のようになる。

```
data{
    int<lower=0> L;
    array[L] real w;
}

parameters{
    real<lower=1, upper=L> tau;
    real beta0a;
    real beta0b;
    real beta1a;
    real<upper=0> beta1b;
    real<lower=0> sig;
}

model{
    //likelihood
    for(i in 1:L){
        if(i < tau){
            w[i] ~ normal(beta0a + beta1a * i, sig);
```

```
        }else{
            w[i] ~ normal(beta0b + beta1b * (i-tau+1), sig);
        }
    }
    //prior
    tau ~ uniform(1,L);
    beta0a ~ normal(80,10);
    beta0b ~ normal(80,10);
    beta1a ~ normal(0,10);
    beta1b ~ normal(0,10);
    sig ~ cauchy(0,5);

}
```

data ブロックはシンプルで，データの長さ L と体重 w の値だけをとるようになっている。parameters ブロックの最初に，τ というパラメータを用意した。データの長さ L の範囲内で，時点 tau で動きが変わったこと考えるのである。もちろんこの tau がいつなのかはわからない。わからないので，パラメータとして推定するのである (ベイズではわからないことを確率として表現する！)。

model ブロックの尤度には，i から L の間で，i < tau の時と i >= tau の時とで分けて書いている。データ自体の生成は単純な回帰モデルで，i < tau の時は beta0a + beta1a * i で平均が推移する。つまり 2024 年の i 日目から傾き beta1a で変化するというモデルである。次に変化点を超えた i>=tau のときは beta0b + beta1b * (i-tau+1)，すなわち変化点から何日経ったか (i-tau+1) という日数を説明変数に，傾き beta1b で変化する。

誤差に対してはいずれも同じ sig を置き，事前分布として tau には無情報分布を，beta0a,0b という切片は 80kg 付近の値を，beta1a,1b には正負どちらの値も取りうるような事前分布を置いた。

このモデルを次のコードで実行する。

```
model <- cmdstan_model("change_point.stan")

dataSet <- list(
  L = nrow(w),
  w = w$weight
)

fit <- model$sample(
  data = dataSet,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 3000,
  iter_sampling = 2000,
  refresh = 0,
```

```
  save_cmdstan_config = TRUE
)
```

```
Running MCMC with 4 parallel chains...

Chain 3 finished in 29.8 seconds.
Chain 1 finished in 29.9 seconds.
Chain 2 finished in 30.0 seconds.
Chain 4 finished in 30.1 seconds.

All 4 chains finished successfully.
Mean chain execution time: 29.9 seconds.
Total execution time: 30.2 seconds.
```

```
fit$summary()
```

```
# A tibble: 7 x 10
  variable    mean  median      sd     mad      q5     q95  rhat ess_bulk
  <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>    <dbl>
1 lp__      22.9    23.2    1.97    1.87    19.3    25.5     1.01    283.
2 tau       87.1    88.3    3.35    1.55    79.4    90.3     1.06     61.2
3 beta0a    84.5    84.5    0.118   0.112   84.3    84.7     1.11     36.8
4 beta0b    85.4    85.4    0.322   0.245   85.0    86.1     1.13     27.4
5 beta1a     0.0179  0.0180 0.00243 0.00237  0.0137  0.0217  1.09     55.0
6 beta1b    -0.0575 -0.0575 0.00566 0.00595 -0.0667 -0.0484  1.10     32.7
7 sig        0.509   0.508  0.0325  0.0301   0.456   0.562   1.04     92.8
# i 1 more variable: ess_tail <dbl>
```

```
# echo: false
MAPs <- bayestestR::describe_posterior(fit$draws(),
  centrality = c("mean", "median", "MAP"), ci = 0.95,
  ci_method = "hdi", test = NULL
)$MAP
```

分析の結果から，変化が生じた X デーである tau は 88.589，つまり 89 日目であり，それまでは傾き 0.018 で上昇傾向だったが，それ以後は-0.056 で下降傾向に減じたことがわかる。

データから MAP 推定値を取り出して，図とともに確認してみよう。

```
Xday <- bayestestR::describe_posterior(fit$draws("tau"),
  centrality = c("mean", "median", "MAP"), ci = 0.95,
  ci_method = "hdi", test = NULL
)$MAP |> round()
```

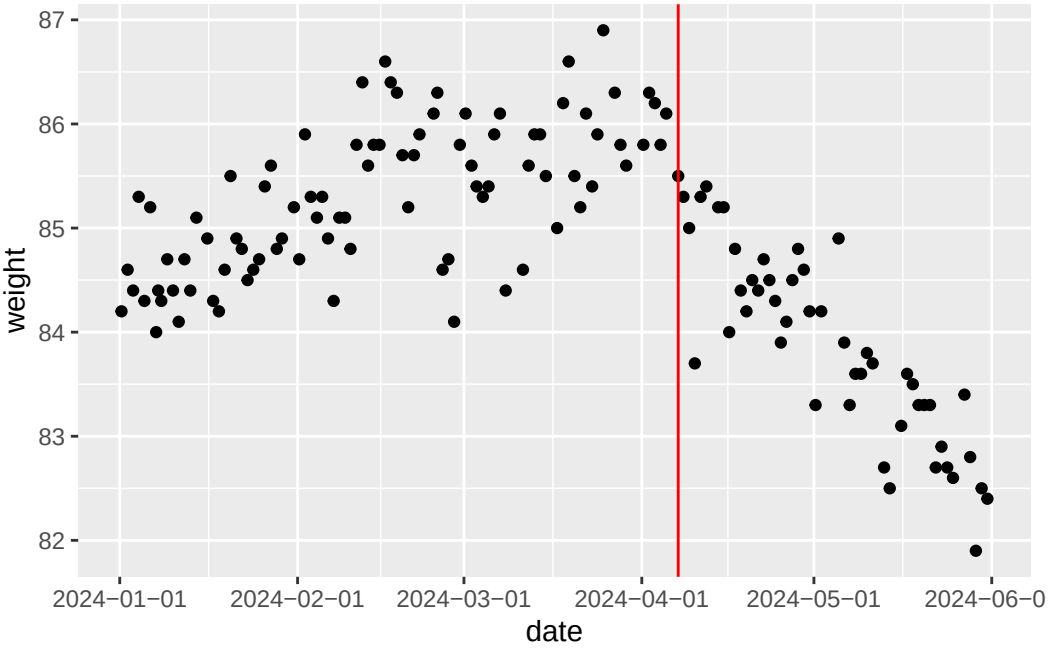
```
cat("変化したのは次の日から")
```

変化したのは次の日から

```
print(w[Xday,])
```

```
# A tibble: 1 x 2
  date                weight
  <dttm>               <dbl>
1 2024-04-07 08:21:32   85.5
```

```
w |>  ggplot(aes(x=date,y=weight))+
      geom_point()+
      geom_vline(xintercept = w$date[Xday],
      color = "red") +
      scale_x_datetime(date_labels = "%Y-%m-%d")
```



今回はデータの分布を見て，変化点が 1 点だろうと考えてモデリングを行った。もちろん分布を詳細に観察することで，違うモデルの立て方 (変化点が複数あり得る，など) を考えることができる。また，今回は変化点までの体重の推移に単回帰モデルを当てはめたり，変化点前後で体重のブレ (sig) が一定であろう，という仮定も含まれている。このように，モデルの数式をよく見てどのような仮定が含まれているか，それらは妥当であるかといったことを吟味したり，何か気付きがあれば，それが改良の余地になることもあるだろう。

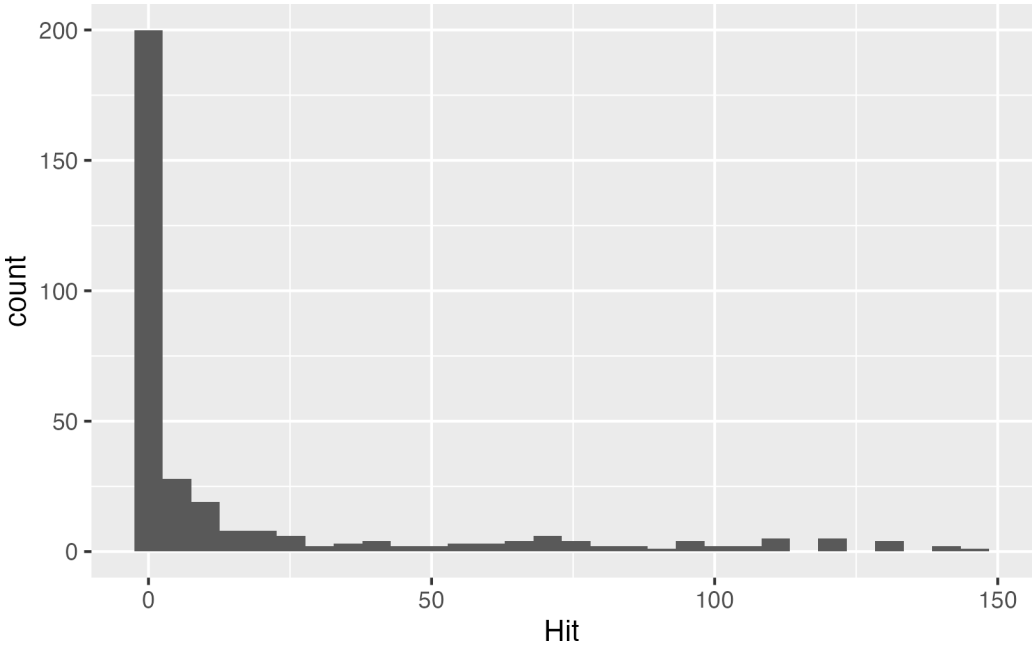
変化点検出はデータに語らせることができるという点で重要で，目で見て主観的に「この辺りからおかしい」と断ずるのではなく，(仮定は多く含まれているものの) より客観的な統計モデルを活用する例を示した。実際に，測定値の異常値を検出するとか，状況が変わったことを検出するなどの際に用いることができるし，皮膚電位反応による隠匿情報に対する反応 (一般に嘘発見器と呼ばれるもの) を診断するようなシーンにも使えるものである。

一点注意を促しておくと，今回は回帰モデルを適用したが，正しくは回帰モデルを用いるべきではない。というのも，回帰モデルの前提として全ての観測点が独立であるという仮定があるからである。時系列データは，観測点は前の時点のデータに依存することが多く，誤差間に相関を仮定する必要がある。時系列データに対してはそうした自己相関を仮定した状態空間モデルを適用するべきであった。状態空間モデルも Stan で記述することができるが，発展的な内容になるため今回は説明を省略する。興味がある人は小森（2022）や 馬場（2018），小杉（2022）などを参考にされたい。

17.4.2　異質な群を分けて考察する

続いて，またも野球のデータを使った例をみてみよう。このデータの中から，2022 年度のセリーグのチームを抜き出し，ヒットの本数をグラフにするのが次のコードである。

```
y <- read_csv("Baseball.csv") |>
  filter(Year == "2020年度") |>
  filter(team %in% c("Giants","Tigers","Carp","Swallows","DeNA","Dragons")) |>
  select(position, Hit, team) |>
  mutate(Hit = ifelse(is.na(Hit),0,Hit))
y |>  ggplot(aes(x=Hit)) + geom_histogram()
```




```
summary(y$Hit)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   0.00    0.00    0.00   17.94   14.25  146.00
```

これを見ると，ほとんどヒットを打たない人がかなりいることがわかる。これは当然で，2022 年のセ・リーグは投手も打席に立つからである。投手は投げることが仕事なので，バットを振らないことがほとんどであり，当然安

打数は基本的に 0 なのである[*12]。

このヒストグラムに対してモデリングすることを考える。安打数 (Hit) はカウント変数だから，理論的に言えばポアソン分布を当てはめるのがよかろう。しかし上で述べたように，打たない人もいるのだから，「打つ人なのかどうか」を判別してモデルを変えるべきである。もし安打数が 0 だというデータがあれば，それは「打たない人だから 0」なのか，「打つ人なんだけど 0」なのか，のどちらかであり，安打数が 0 でなければ「打つ人」であるはずだ。

こうした「構造的な 0 の可能性」を考える混合分布モデルを**ゼロ過剰ポアソン分布**という。今回はこの「ゼロ過剰」のモデルを見ていく。

その前に，ポアソン分布についてもう一つ。ポアソン分布のパラメータは λ だけである。これは平均値パラメータと言われるが，ポアソン分賦の平均値と分散は同じになるという特徴を持つ。そのため，単一のパラメータ λ だけでモデルを当てはめると，分散が大きい場合 (過分散という)，得てしておかしな推定値になる。幸にして，ポアソン分布の特徴を持ちつつ，平均パラメータと分散パラメータの両方を持つ分布があり，これを負の二項分布という。

負の二項分布は次のように定義される。

$$NegBinomial(n|\alpha,\beta) = \frac{\Gamma(\alpha+n)}{\Gamma(\alpha)\Gamma(n+1)}\left(\frac{\alpha}{\alpha+\beta}\right)^{\alpha}\left(\frac{\beta}{\alpha+\beta}\right)^{n}$$

この定義から導出される平均と分散は次のようになる。

$$E[N] = \frac{\alpha\beta}{\alpha} = \beta$$
$$Var[N] = \beta + \frac{\beta^2}{\alpha} = \beta\left(1+\frac{\beta}{\alpha}\right)$$

ただ，この表現はややこしいし直感的でない。そこで Stan では平均 μ と過分散パラメータ ϕ を用いた `neg_binomial_2` の書式が用意されている。

$$NegBinomial2(n|\mu,\phi) = \frac{\Gamma(\phi+n)}{\Gamma(\phi)\Gamma(n+1)}\left(\frac{\phi}{\phi+\mu}\right)^{\phi}\left(\frac{\mu}{\phi+\mu}\right)^{n}$$

この場合の平均と分散は次のようになる。

$$E[N] = \mu$$
$$Var[N] = \mu + \frac{\mu^2}{\phi} = \mu\left(1+\frac{\mu}{\phi}\right)$$

[*12] 野球に詳しくない人のためにコメントしておくと，まず日本のプロ野球は 129 段あるが，セントラルリーグ (セ・リーグ) とパシフィックリーグ (パ・リーグ) に 6 チームずつ分かれて普段のリーグ戦を行っている。両リーグの違いとして，パ・リーグでは指名打者制を取っており，投手の打順のところに別の選手が打者として入るため，投手は打席に立たない。セ・リーグでも 2027 年のシーズンから指名打者制を取ることが決まった。野球は 9 人でやるものだが，投手は打ったり走ったりして体力を使わないようにするため，打席に立っても仕事をしないものなのである。そう，大谷翔平選手が異常なのだ。

ϕ が大きいほど分散は平均に近づき、$\phi \to \infty$ でポアソン分布と一致する。

さて今回はゼロ過剰な負の二項分布モデルでアプローチしよう。Stan のコードは次のようになる。

```
data {
  int<lower=0> N;           // データ数
  array[N] int<lower=0> y;  // 観測値
}

parameters {
  real<lower=0, upper=1> theta;
  real<lower=0> mu;              // 負の二項分布の平均
  real<lower=0> phi;             // 負の二項分布の過分散パラメータ
}

model {
  // 事前分布
  theta ~ beta(1, 1);
  mu ~ gamma(5, 0.1);
  phi ~ gamma(2, 0.1);

  // 尤度関数（0過剰負の二項分布）
  for (i in 1:N) {
    if (y[i] == 0) {
      target += log_mix(theta,
                        0,
                        neg_binomial_2_lpmf(0 | mu, phi));
    } else {
      target += log(1 - theta) + neg_binomial_2_lpmf(y[i] | mu, phi);
    }
  }
}

generated quantities {
  array[N] int y_pred;     // 事後予測分布

  for (i in 1:N) {
    // 事後予測分布の生成
    if (bernoulli_rng(theta) == 1) {
      y_pred[i] = 0;  // 構造的ゼロ
    } else {
      y_pred[i] = neg_binomial_2_rng(mu, phi);  // 負の二項分布から生成
```

```
    }
  }
}
```

parameters ブロックには，「打つ人なのかどうか」を判断するパラメータ theta を用意した。これは 0 から 1 の間に入る実数だからベルヌーイ分布のイメージで，コイントスをして表が出れば「打たない人」，裏が出れば「打つ人だけど 0 だった人」という分岐をすると考える。また負の二項分布のパラメータとして，平均 mu と分散 phi についてのパラメータを置いた。

model ブロックをみると，この分岐がよくわかるだろう。もしデータ y_i が 0 なら，log_mix 関数を通る。Stan の log_mix 関数は混合率とそれぞれのケースの尤度を記載する。log_mix(theta,0,neg_binomial_2_lpmf(0 | mu, phi) の式が表すのは，

$$p(y_i|\theta,\mu,\phi) = \theta + (1-\theta)\cdot NegBinomial2(0|\mu,\phi)$$

であり，確率 θ で 0 が出ている[*13]か，$1-\theta$ で負の二項分布から 0 が出ている確率か，を表している。

もちろん y_i > 0 の場合は「打つ人」であるから，1-theta の確率で負の二項分布から出るコースになる。両者とも target 記法で，事後対数尤度に直接加えていることを確認しておこう。

また今回は，生成量としてこのモデルから生み出されるデータを生成させてみた。メカニズムは同じで，theta をベルヌーイ分布にいれて 1 が出たら「打たない人」だから 0 だし，そうでなければ負の二項分布からある値が生成されるだろう，という形である。このように，モデルが生み出す予測データのことを**事後予測分布**といい，これを視認することで「モデルから今回と同じようなデータが生成されるか」を確認することができる。

これを次の R コードで実行する。

```
model <- cmdstan_model("zero_inflated_negbinom.stan")

dataSet <- list(
  N = length(y$Hit),
  y = y$Hit
)

fit <- model$sample(
  data = dataSet,
  parallel_chains = 4,
  refresh = 0,
  save_cmdstan_config = TRUE
)
```

[*13] log_mix の第一引数は混合率であり，第 2・第 3 引数には対数確率密度を記述する。第 2 引数の 0 は対数確率密度なので，exp(0)=1，すなわち 100% の確率であることを意味する。第 3 引数は neg_binomial_2_lpmf であり，lpmf とあることから，これは負の二項分布 (第二形態) の対数確率密度であることがわかる。

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.6 seconds.
Chain 2 finished in 0.6 seconds.
Chain 3 finished in 0.6 seconds.
Chain 4 finished in 0.6 seconds.

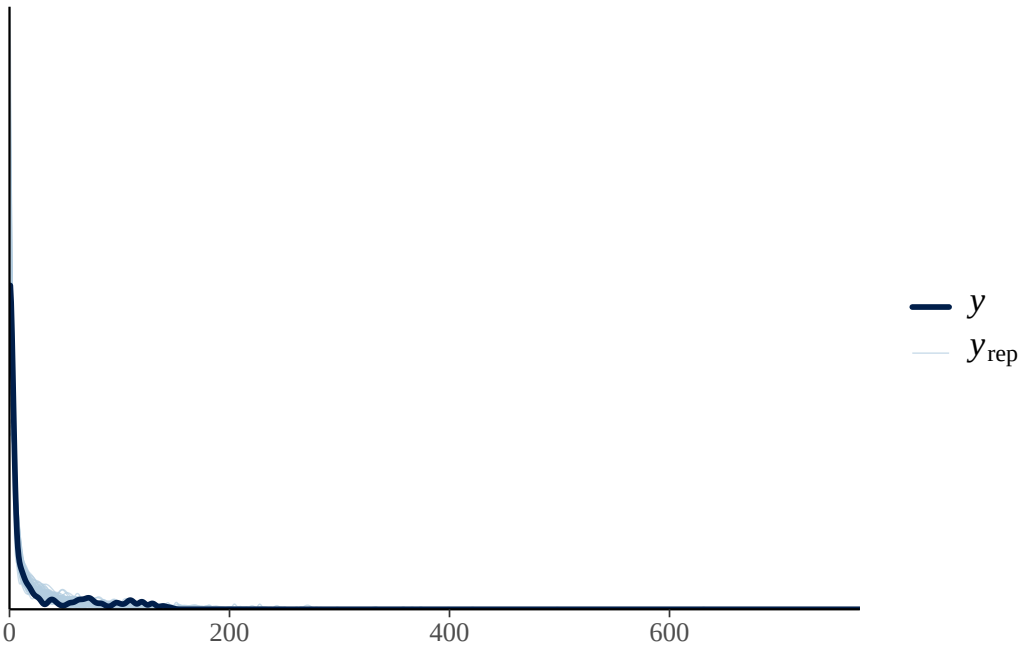
All 4 chains finished successfully.
Mean chain execution time: 0.6 seconds.
Total execution time: 0.7 seconds.
```

```
bayestestR::describe_posterior(fit$draws(c("theta","mu","phi")),
  centrality = c("mean", "median", "MAP"), ci = 0.95,
  ci_method = "hdi", test = NULL)
```

```
Summary of Posterior Distribution

Parameter | Median |  Mean |   MAP |         95% CI
-----------------------------------------------------
theta     |   0.45 |  0.44 |  0.45 | [ 0.35,  0.52]
mu        |  33.26 | 33.50 | 32.37 | [25.66, 41.85]
phi       |   0.43 |  0.43 |  0.42 | [ 0.27,  0.59]
```

```
y_pred <- fit$draws("y_pred", format = "matrix")
bayesplot::ppc_dens_overlay(y$Hit, y_pred[1:50, ])
```



出力結果から, 混合率 theta や平均, 分散の値が見て取れるだろう。加えて事後予測分布を見てみよう。bayesplot

パッケージには事後予測分布に関する関数が複数用意されており，モデルが生み出すデータと実際のデータを重ねて描画してくれる。ここでは `ppc_dens_overlay`(posterior predictive distribution の密度を重ね書きする関数) を使って，最初の 50 個のデータを重ね書きしてみた。実際のデータとモデルの予測値がしっかり重なっており，うまく適合していることがわかるだろう。

今回は「打つ人」と「打たない人」をデータから識別しながらモデルを適合するというモデルをみた。このモデルは例えば，「お店に初めて来る人とリピーターあるいは常連」で売上が異なるのではないかとか，例えば質問紙調査などで中心の回答 (「どちらともいえない」など) を選びがちな群とそうでない群を分けるとか，健常者群と臨床群との違いを反応から見つけてモデルを変える，など色々な研究的応用も可能である。

リー・ワゲンメーカーズ（2017）もいうように，**ベイズモデルの限界はユーザの想像力だけ**である。皆さんも色々なデータ生成メカニズムを想像し，面白いモデルを創造してもらいたい。

第 18 章

ベイズの観点から見た平均値差の検定

これまで，二群の平均値差を比べる t 検定，三群以上を比べる分散分析（ANOVA）を扱ってきた。これらはいずれも，「群の間で平均値に差があるとはいえない」という帰無仮説を立て，t 値や F 値といった検定統計量を計算し，その確率（p 値）が小さければ帰無仮説を棄却する，という頻度主義的な手続きであった。

この章では，まったく同じ問題をベイズ統計の観点から捉え直してみる。すなわち，「データはどのようなメカニズムで生まれたのか」というデータ生成の視点からモデルを組み立て，平均値や差の大きさを確率分布として直接見積もるのである。さらに，差があるかないかという判断（意思決定）について，ベイズ的なアプローチである**ROPE**，**ベイズファクター**，**事後予測分布**の三つの道具を紹介する。

なお本章では，ベイズ統計の活用の章で導入した `brms` と，ベイジアンモデリングの章で扱った Stan による確率モデルの記述の両方を用いる。これらの章の内容を前提とするので，必要に応じて読み返してほしい。

18.1　検定からモデリングへ

t 検定や分散分析は，「平均値の差の検定」と総称される。ここで行われていたことを改めて振り返ると，正規分布に従うと仮定されたデータについて，その平均値に差があるかどうか，より正確には「すべての群の母平均が等しい」と仮定したときに手元のデータ（が示す差）がどれほど生じやすいか，を評価していたのであった。

データ生成メカニズムの観点から見ると，この手続きはごく限定的な一面しか見ていない。平均値以外のパラメータを考えることはできないのか。特定の群間の差だけを取り出すことはできないのか。差があるかないかだけでなく，どれぐらいの差があるのか，一方が他方より大きい確率はどれぐらいか，といったことを直接問うことはできないのか。ベイズ推定は，こうした問いに自然な形で答えを与える。

18.1.1　多重比較の問題が生じない

頻度主義的な分散分析を思い出してほしい。三群以上を比較して有意差が得られた場合，帰無仮説 $H_0 : \mu_1 = \mu_2 = \mu_3$ が否定されただけで，「どこに差があるのか」はまだわからない。そこで多重比較へと進むのであったが，検定を繰り返すとタイプ 1 エラーが増大する。$\alpha = 0.05$ で検定していても，独立な検定を二回繰り返せば $1 - (0.95^2) = 0.0975$ と，全体としての誤り率は 5% を大きく上回ってしまう。そのため，テューキー法やボン

フェローニ法のように有意水準を調整する工夫が必要だったのである。

ところがベイズ推定の場合，こうした多重性の問題は生じない。なぜなら，確率を伴う二分判断（Yes か No か）を繰り返しているわけではないからである。頻度主義の p 値は「帰無仮説が正しいという条件のもとで」計算された検定統計量の生じやすさであって，パラメータがどこにあるかという確率ではない。これに対しベイズ推定で得られる事後分布は，パラメータがこのあたりにあるだろう，という確率そのものである。だから A 群と B 群の差，A 群と C 群の差，というように好きなだけ差の分布を眺めてよいし，「何度も比べたから誤りが増える」という心配は原理的に存在しない。複雑な要因計画を考えるとき，この性質は非常に助けになる。

18.2　分散分析の組成を思い出す

ベイズのモデルを組む前に，分散分析がデータの変動をどう分解していたかを思い出しておこう。これがそのままデータ生成モデルになるからである。

分散分析では，あるデータ x_{ij}（第 j 群の第 i 番目の観測値）を次のように考える。

$$x_{ij} = \mu + \delta_j + e_{ij}$$

ここで μ は全体平均，δ_j は第 j 群に属することの効果（全体平均からのずれ），e_{ij} は個体差・誤差である。三つのクラスの試験の平均点を比べる例でいえば，全体平均が 60 点で，あるクラスの平均が 68 点だったなら，そのクラスに属したことの効果 δ_j は $+8$ 点ということになる。効果は全体平均からの相対的なずれなので，すべての群の効果を足し合わせると 0 になる（$\sum_j \delta_j = 0$）。これが，水準数が 3 なのに効果の自由度が 2 になる理由であった。

頻度主義の分散分析では，この効果 δ_j の二乗を群のサイズ分だけ足し合わせた「群間変動」と，誤差 e_{ij} の二乗を足し合わせた「群内変動」を，それぞれの自由度で割って比べ，その比率 F が大きければ「差がある」と判断していた。

ベイズの観点では，この分解の式そのものを，データを生み出す確率モデルとして読み直す。すなわち，誤差 e_{ij} が平均 0，標準偏差 σ の正規分布に従うと考えれば，

$$x_{ij} \sim N(\mu + \delta_j, \sigma)$$

となる。あとは，未知数である全体平均 μ，効果 δ_j，誤差の大きさ σ を，データから推定すればよい。頻度主義が「変動を分解して比率を取る」ことに注力したのに対し，ベイズは「効果 δ_j そのものの分布」を直接求めにいくのである。

18.3　二群の平均値差をベイズで（t 検定）

まずは最も単純な，二群の平均値差，すなわち t 検定に対応するモデルから始めよう。

カバーストーリーとして，ある学習法の効果を考える。同じ範囲の内容を，異なる三つの学習法（方法 A，方法 B，方法 C）で学んだ受講者に，共通のテストを受けてもらった。各群 40 名である。まずはこのうち方法 A と方

法 C の二群だけを取り出して比較してみよう。

データは，分散分析の組成の式にもとづいて生成する。全体平均 60 点に，方法 A は +8 点，方法 B は +6 点，方法 C は −14 点の効果があり（合計 0），そこに標準偏差 8 点の誤差が乗るものとする。

```
pacman::p_load(tidyverse, cmdstanr, bayesplot, bayestestR, brms, logspline)
options(brms.backend = "cmdstanr")

set.seed(1234)
gm <- 60 # 全体平均
eff <- c(8, 6, -14) # 各群の効果（合計0）
sigma_true <- 8 # 誤差の標準偏差
N <- 40 # 各群のサンプルサイズ
labels <- c("方法A", "方法B", "方法C")

dat <- tibble(
  method = factor(rep(labels, each = N), levels = labels),
  score = c(
    rnorm(N, gm + eff[1], sigma_true),
    rnorm(N, gm + eff[2], sigma_true),
    rnorm(N, gm + eff[3], sigma_true)
  )
) |>
  mutate(idx = as.integer(method))

dat |>
  group_by(method) |>
  summarise(平均 = mean(score), 標準偏差 = sd(score), n = n())
```

```
# A tibble: 3 x 4
  method   平均 標準偏差     n
  <fct>   <dbl>    <dbl> <int>
1 方法A    64.7     7.30    40
2 方法B    65.4     8.67    40
3 方法C    46.4     6.32    40
```

群ごとの平均値を見ると，手元のデータでは方法 A と方法 B がほぼ同じ程度で，方法 C だけが低い，という結果になっている。母集団では方法 A と方法 B に 2 点の差をつけて生成したのだが，標本のばらつきによって，得られたデータ上ではほとんど差がなくなっている。このあたりの「標本だからこその揺らぎ」も，ベイズ推定がどう扱うか見どころである。

18.3.1　Stan によるモデルの記述

二群比較の Stan コードは次のようになる。これを `bayes_ttest.stan` という名前で保存しておく。

```
data {
  int<lower=0> N1;          // 第1群のサンプルサイズ
  int<lower=0> N2;          // 第2群のサンプルサイズ
  array[N1] real X1;        // 第1群の観測値
  array[N2] real X2;        // 第2群の観測値
}

parameters {
  real mu1;                 // 第1群の母平均
  real mu2;                 // 第2群の母平均
  real<lower=0> sigma;      // 共通の標準偏差（等分散の仮定）
}

model {
  // 尤度：各群が同じ散らばりを持つ正規分布から生じる
  X1 ~ normal(mu1, sigma);
  X2 ~ normal(mu2, sigma);
  // 事前分布
  mu1 ~ normal(50, 50);
  mu2 ~ normal(50, 50);
  sigma ~ cauchy(0, 5);
}

generated quantities {
  real diff;                // 平均値差そのもの
  real delta;               // 標準化した効果量（Cohen's d 相当）
  array[N1 + N2] real X_pred;  // 事後予測分布のための複製データ

  diff = mu1 - mu2;
  delta = (mu1 - mu2) / sigma;
  for (i in 1 : N1) {
    X_pred[i] = normal_rng(mu1, sigma);
  }
  for (i in 1 : N2) {
    X_pred[N1 + i] = normal_rng(mu2, sigma);
  }
```

```
}
```

data ブロックでは，二つの群それぞれのサンプルサイズと観測値を受け取る。parameters ブロックには，二つの母平均 mu1，mu2 と，共通の標準偏差 sigma を置いた。ここで標準偏差を共通にしているのは，t 検定でいう「等分散の仮定」に対応する。model ブロックでは，各群のデータがそれぞれの母平均を中心とする正規分布から生じたという尤度を記述し，事前分布を与えている。

注目すべきは generated quantities ブロックである。ここでは平均値差そのもの diff と，それを標準偏差で割って標準化した効果量 delta（Cohen's d に相当）を計算している。検定と違って，差の大きさを確率分布として直接得られるのがベイズの強みである。さらに X_pred として，推定されたモデルから新しいデータを生成しておく。これは後述する事後予測分布に用いる。

18.3.2　推定と結果の読み取り

このモデルをコンパイルし，方法 A と方法 C のデータを与えて推定する。

```
sub <- dat |> filter(method %in% c("方法A", "方法C"))
X1 <- sub$score[sub$method == "方法A"]
X2 <- sub$score[sub$method == "方法C"]

model_t <- cmdstan_model("bayes_ttest.stan")
fit_t <- model_t$sample(
  data = list(N1 = length(X1), N2 = length(X2), X1 = X1, X2 = X2),
  chains = 4, parallel_chains = 4,
  iter_warmup = 1000, iter_sampling = 2000,
  refresh = 0, save_cmdstan_config = TRUE
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.0 seconds.
Chain 2 finished in 0.0 seconds.
Chain 3 finished in 0.0 seconds.
Chain 4 finished in 0.0 seconds.

All 4 chains finished successfully.
Mean chain execution time: 0.0 seconds.
Total execution time: 0.2 seconds.
```

```
fit_t$summary(c("mu1", "mu2", "sigma", "diff", "delta"))
```

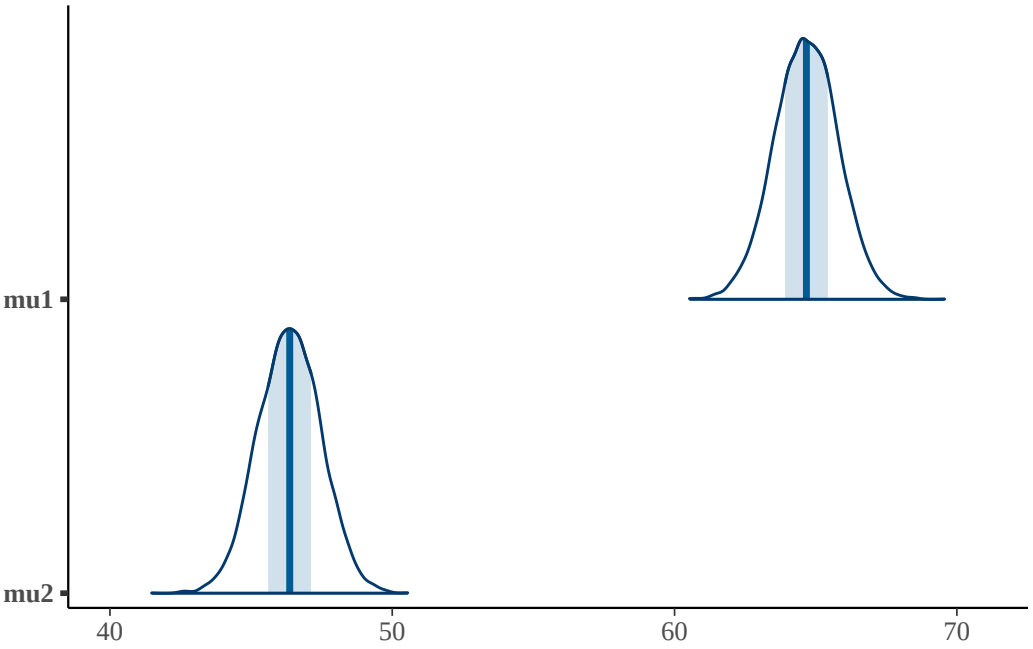
```
# A tibble: 5 x 10
  variable  mean median    sd   mad    q5   q95  rhat ess_bulk ess_tail
```

```
  <chr>     <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
1 mu1       64.7   64.7  1.13  1.12  62.8  66.5   1.00    8594.    5383.
2 mu2       46.4   46.4  1.11  1.13  44.6  48.2   1.00    7999.    6340.
3 sigma      6.87   6.85 0.556 0.556  6.03  7.83  1.00    6844.    5527.
4 diff      18.3   18.3  1.58  1.56  15.7  20.9   1.00    8728.    5529.
5 delta      2.68   2.68 0.314 0.312  2.17  3.22  1.00    7784.    5499.
```

Rhat はいずれも 1.00 付近，ess_bulk も十分大きく，サンプリングはうまくいっている。mu1，mu2 が方法 A，方法 C の母平均の事後分布の代表値である。注目すべきは diff で，これは平均値差の事後分布である。点推定としての差だけでなく，その差が 90% の確率でどの範囲にあるか（q5 から q95）まで一目でわかる。標準化効果量 delta も非常に大きく，方法 A と方法 C の間には実質的な差があると考えてよさそうである。

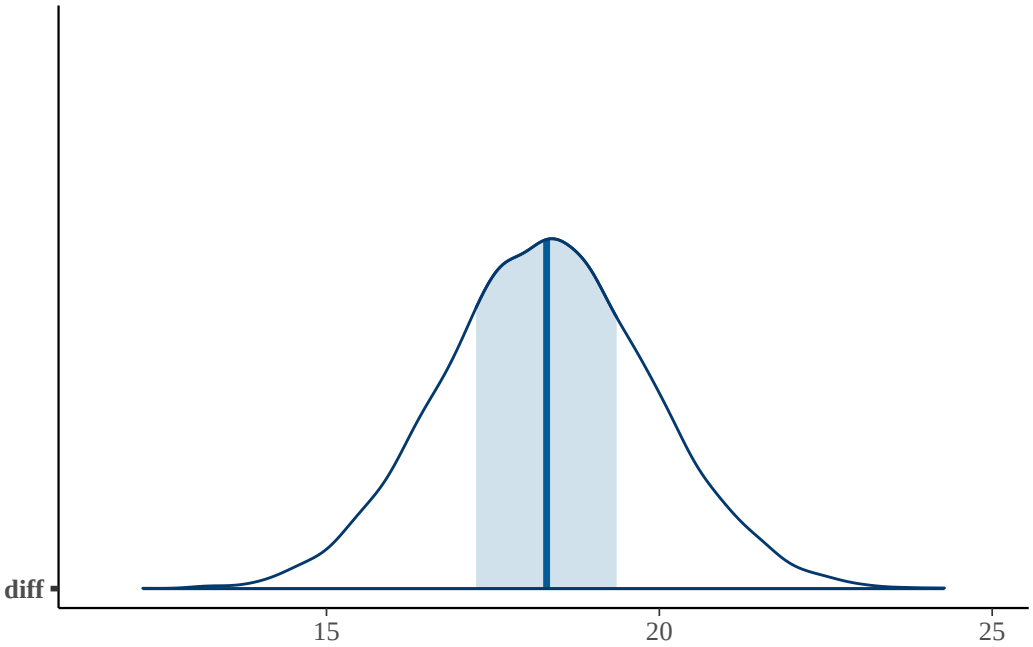
数値だけでなく，事後分布を絵にして眺めるとわかりやすい。bayesplot パッケージの mcmc_areas 関数は，パラメータの事後分布を山型の密度で描いてくれる。

```
# 二群の母平均の事後分布
bayesplot::mcmc_areas(fit_t$draws(c("mu1", "mu2")))
```




```
# 平均値差の事後分布（破線が差ゼロ）
bayesplot::mcmc_areas(fit_t$draws("diff")) +
  ggplot2::geom_vline(xintercept = 0, linetype = "dashed")
```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_vline()`).
```



二つの母平均の山がはっきり離れていること，そして差 `diff` の分布が 0（破線）から大きく右に離れていることが一目でわかる。差の山のほぼすべてが正の領域にあるので，先ほどの優越率がほぼ 1 だったことも納得できる。

差を確率分布として持っていることの利点は，さまざまな「データレベルの仮説」を後から自由に問える点にある。たとえば「方法 A の母平均が方法 C の母平均より大きい確率」は，差の事後分布のうち正の領域が占める割合として計算できる。これを優越率（probability of superiority）と呼ぶ。

```
draws_t <- fit_t$draws(format = "df")
# 方法Aが方法Cより大きい確率
mean(draws_t$diff > 0)
```

```
[1] 1
```

差の MCMC サンプルのうち正の値が占める割合を数えるだけである。ほぼ 1，すなわち「方法 A のほうが優れている」とほぼ確実にいえる。頻度主義の p 値が「帰無仮説のもとでの検定統計量の生じやすさ」という間接的な指標だったのに対し，これは知りたかったことそのものを確率で答えている点に注目してほしい。

18.4　多群の平均値差をベイズで（分散分析）

次に三群の比較，すなわち一要因分散分析に対応するモデルを考える。二群モデルを群の数だけ書き足してもよいが，それでは群の数が変わるたびにコードを修正してコンパイルし直すことになる。そこで，先に見た分散分析の組成の式を，群の数によらない一般的な形で書き下す。

すなわち全体平均 μ（コード中では `gm`）と，各群の効果 δ_j，誤差 σ でモデルを構成する。ただし効果には $\sum_j \delta_j = 0$ という制約があったから，推定するのは $\mathrm{Lv} - 1$ 個の効果だけでよく，最後の一つは「総和が 0 になるように」決まる。

18.4.1　Stan によるモデルの記述

整然データ（tidy data）の形でデータを与えることを前提にした，群の数によらないコードが次である。bayes_anova.stan として保存しておく。

```
data {
  int<lower=0> Lv;                        // 水準数（群の数）
  int<lower=0> L;                         // データの総数
  array[L] int<lower=1, upper=Lv> idx;    // 各データが属する群のID
  array[L] real X;                        // 観測値
}

parameters {
  real gm;                                // 全体平均（grand mean）
  array[Lv - 1] real raw_delta;           // 効果。自由度の関係で Lv-1 個
  real<lower=0> sigma;                    // 群内の散らばり（誤差）
}

transformed parameters {
  array[Lv] real delta;                   // 総和が0になるよう作り直した効果
  array[Lv] real mu;                      // 各群の母平均

  for (l in 1 : (Lv - 1)) {
    delta[l] = raw_delta[l];              // 最初の Lv-1 個はコピー
  }
  delta[Lv] = -sum(raw_delta);            // 最後の1つは「効果の総和=0」から決まる

  for (l in 1 : Lv) {
    mu[l] = gm + delta[l];                // 各群の平均 = 全体平均 + 効果
  }
}

model {
  // 尤度：l 番目のデータは，その群の平均 mu[idx[l]] を中心に散らばる
  for (l in 1 : L) {
    X[l] ~ normal(mu[idx[l]], sigma);
  }
  // 事前分布
  gm ~ normal(50, 50);
  raw_delta ~ normal(0, 50);
```

```
  sigma ~ cauchy(0, 5);
}

generated quantities {
  array[L] real X_pred;                    // 事後予測分布のための複製データ

  for (l in 1 : L) {
    X_pred[l] = normal_rng(mu[idx[l]], sigma);
  }
}
```

data ブロックでは，水準数 Lv，データ総数 L に加え，各データがどの群に属するかを示すインデックス idx と観測値 X を，いずれも長さ L の配列で受け取る。これにより，群ごとのサンプルサイズが異なっていても対応できる。

parameters ブロックには全体平均 gm，効果 raw_delta（Lv − 1 個），誤差 sigma を置いた。transformed parameters ブロックでは，raw_delta の最初の Lv − 1 個をそのままコピーし，最後の一つを-sum(raw_delta) として総和が 0 になるよう作り直すことで効果 δ_j を完成させ，各群の平均 mu[l] = gm + delta[l] を構成している。

model ブロックの尤度は，X[l] ~ normal(mu[idx[l]], sigma) の一行で，l 番目のデータがその群 idx[l] の平均を中心に散らばる，という関係を表している。mu[idx[l]] という入れ子の参照によって，どの群に属するデータかを指定しているのである。generated quantities ブロックでは，二群のときと同様に事後予測のための複製データ X_pred を生成しておく。

18.4.2　推定と全ペアの比較

三群すべてのデータを与えて推定する。

```
model_a <- cmdstan_model("bayes_anova.stan")
fit_a <- model_a$sample(
  data = list(Lv = 3, L = nrow(dat), idx = dat$idx, X = dat$score),
  chains = 4, parallel_chains = 4,
  iter_warmup = 1000, iter_sampling = 2000,
  refresh = 0, save_cmdstan_config = TRUE
)
```

```
Running MCMC with 4 parallel chains...

Chain 1 finished in 0.1 seconds.
Chain 2 finished in 0.1 seconds.
Chain 3 finished in 0.1 seconds.
```

```
Chain 4 finished in 0.1 seconds.

All 4 chains finished successfully.
Mean chain execution time: 0.1 seconds.
Total execution time: 0.3 seconds.
```

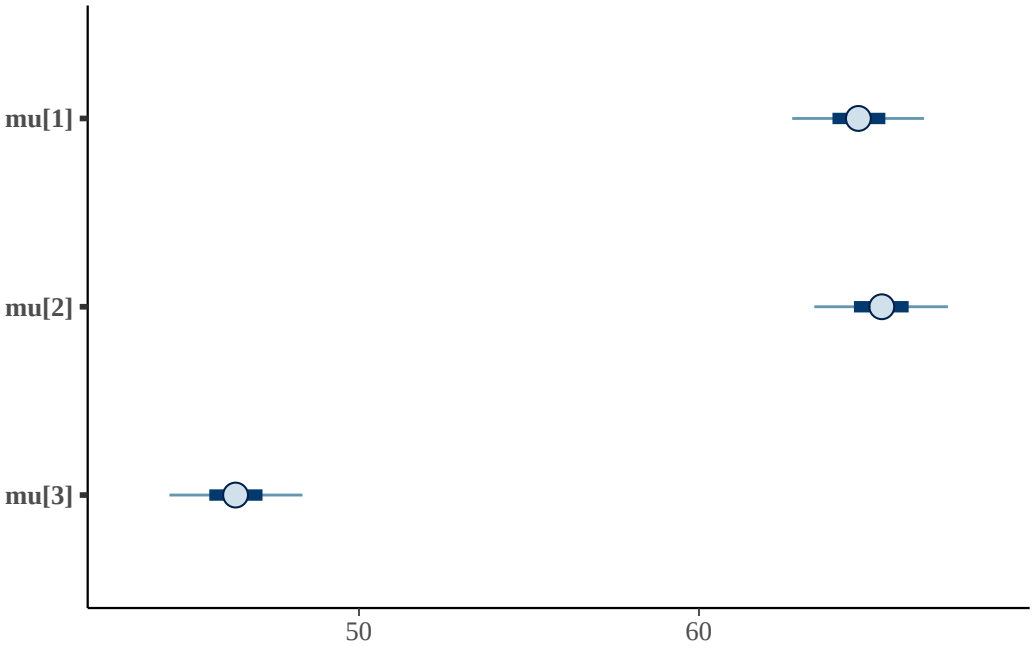
```
fit_a$summary(c("gm", "mu[1]", "mu[2]", "mu[3]",
  "delta[1]", "delta[2]", "delta[3]", "sigma"))
```

```
# A tibble: 8 x 10
  variable   mean median    sd   mad     q5    q95  rhat ess_bulk ess_tail
  <chr>     <dbl>  <dbl> <dbl> <dbl>  <dbl>  <dbl> <dbl>    <dbl>    <dbl>
1 gm         58.8   58.8  0.684 0.684  57.7   59.9   1.00    7525.    5741.
2 mu[1]      64.7   64.7  1.17  1.15   62.7   66.6   1.00    6765.    5555.
3 mu[2]      65.4   65.4  1.20  1.19   63.4   67.3   1.00    7005.    5621.
4 mu[3]      46.4   46.4  1.19  1.16   44.4   48.3   1.00    8793.    6423.
5 delta[1]    5.88   5.88 0.964 0.952   4.28   7.43  1.00    6157.    5834.
6 delta[2]    6.56   6.55 0.978 0.984   4.95   8.16  1.00    6594.    5307.
7 delta[3] -12.4  -12.4  0.970 0.979 -14.0  -10.9   1.00   10361.    6105.
8 sigma       7.54   7.51 0.499 0.493   6.77   8.39  1.00    7794.    5961.
```

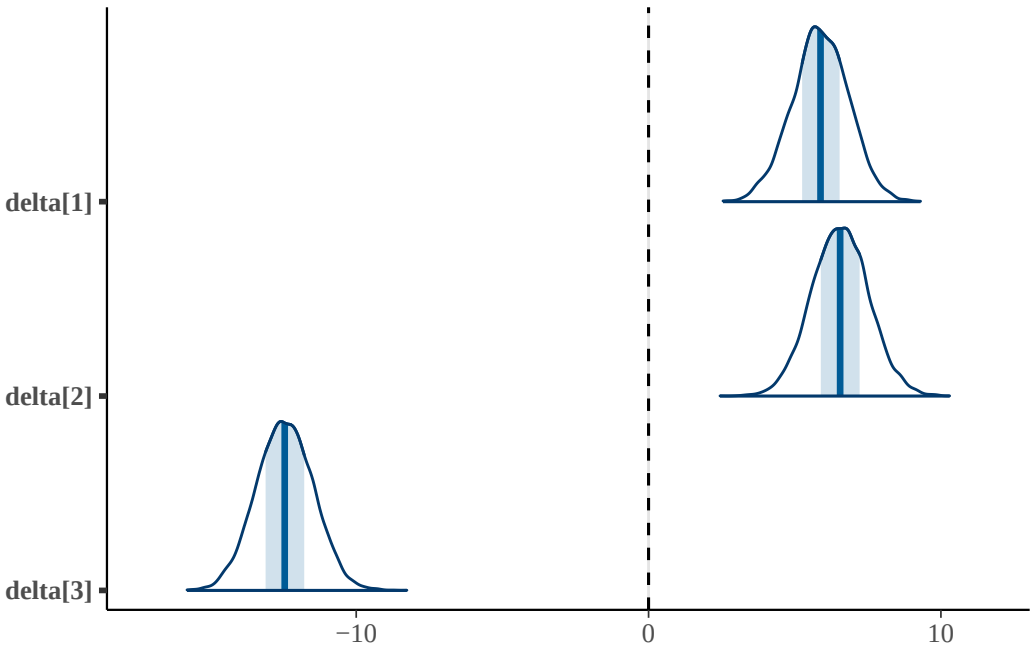
gm が全体平均，mu[1] から mu[3] が各群の母平均，delta[1] から delta[3] が各群の効果の事後分布である。効果を見ると，方法 A・方法 B が正，方法 C が大きく負であり，データの生成構造をおおむね復元できている。

これも可視化してみよう。各群の母平均は区間表示（mcmc_intervals），効果は密度（mcmc_areas）で描く。

```
# 各群の母平均（点と区間）
bayesplot::mcmc_intervals(fit_a$draws(c("mu[1]", "mu[2]", "mu[3]")))
```




```
# 各群の効果（破線が効果ゼロ）
bayesplot::mcmc_areas(fit_a$draws(c("delta[1]", "delta[2]", "delta[3]"))) +
  ggplot2::geom_vline(xintercept = 0, linetype = "dashed")
```



母平均では方法 A・方法 B が近い位置に並び，方法 C だけが下にずれている。効果の図でも，方法 C の効果が明確に負側にあるのに対し，方法 A・方法 B の効果は 0 付近で重なっていることが見て取れる。

ここで分散分析の後の多重比較に相当することをやってみよう。頻度主義では有意水準の調整が必要だったが，ベイズでは各群の母平均の事後分布（の MCMC サンプル）から，差を引き算するだけでよい。

```
draws_a <- fit_a$draws(format = "df")
pair <- tibble(
  `A-B` = draws_a$`mu[1]` - draws_a$`mu[2]`,
  `A-C` = draws_a$`mu[1]` - draws_a$`mu[3]`,
  `B-C` = draws_a$`mu[2]` - draws_a$`mu[3]`
)

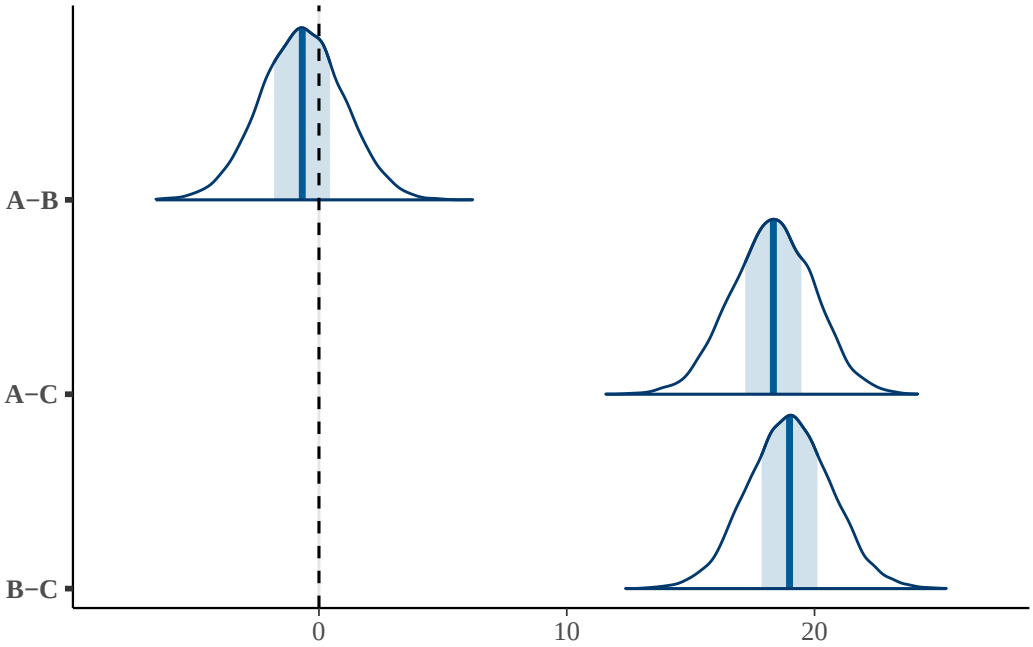
pair |>
  pivot_longer(everything(), names_to = "対", values_to = "差") |>
  group_by(対) |>
  summarise(
    EAP = mean(差),
    L95 = quantile(差, 0.025),
    U95 = quantile(差, 0.975),
    `P(差>0)` = mean(差 > 0)
  )
```

```
# A tibble: 3 x 5
  対       EAP   L95   U95 `P(差>0)`
  <chr>  <dbl> <dbl> <dbl>     <dbl>
1 A-B   -0.676 -3.99  2.62     0.346
2 A-C   18.3   15.1  21.5      1
3 B-C   19.0   15.7  22.3      1
```

結果を見ると，方法 A と方法 C の差，方法 B と方法 C の差は 95% 区間が 0 をまたがず，「差がある」とほぼ確実にいえる（優越率もほぼ 1）。一方で方法 A と方法 B の差は，95% 区間が 0 をはさんでおり，優越率も 0.5 に近い。つまり，この二つの方法には明確な差があるとはいえない，という結果である。母集団では 2 点差をつけて作ったのだが，40 名ずつのデータからはその差を見分けられなかった，ということになる。

全ペアの差の事後分布を重ねて描くと，どのペアが 0 をまたいでいるかがよくわかる。

```
# 全ペアの差の事後分布（破線が差ゼロ）
bayesplot::mcmc_areas(as.matrix(pair)) +
  ggplot2::geom_vline(xintercept = 0, linetype = "dashed")
```



方法 A と方法 C の差，方法 B と方法 C の差は，山全体が 0 より右に大きく離れている。一方で方法 A と方法 B の差の山は 0 をまたいでおり，差があるとはいいきれないことが視覚的に確認できる。

ここで重要なのは，「区間が 0 をまたぐから差がない」と即断してはいけない，という点である。これは「差があるとはいえない」のであって，「差がない（同等である）」とは違う。この区別こそが，次に述べる意思決定の道具が必要になる理由である。

18.5　意思決定の三つの道具

事後分布が得られたあと，「で，結局どう判断すればよいのか」という問いが残る。頻度主義には「$p < 0.05$ なら有意」という明快（だが乱用もされてきた）な基準があった。ベイズにはこれに代わる判断の道具がいくつかある。ここでは代表的な三つ，ROPE，ベイズファクター，事後予測分布を見ていく。

18.5.1　ROPE

ROPE は Region Of Practical Equivalence（実質的同等の領域）の略で，Kruschke (2018) によって提唱された判断方法である。考え方はシンプルで，「この範囲に入っていれば，差は実質的にゼロとみなしてよい」という幅をあらかじめ宣言しておく，というものである。

たとえば今回のテスト得点であれば，「±2 点くらいの差は，実用上は差がないも同然だ」と研究者が判断したとしよう。この $[-2, +2]$ が ROPE である。そのうえで，差の事後分布がこの ROPE にどれだけ入っているかを見る。

```
# 方法Aと方法Bの差：実質的同等か？
bayestestR::rope(pair$`A-B`, range = c(-2, 2), ci = 1)
```

```
# Proportion of samples inside the ROPE [-2.00, 2.00]:
```

```
Inside ROPE
-----------
72.60 %
```

```
# 方法Aと方法Cの差：実質的な差があるか？
bayestestR::rope(pair$`A-C`, range = c(-2, 2), ci = 1)
```

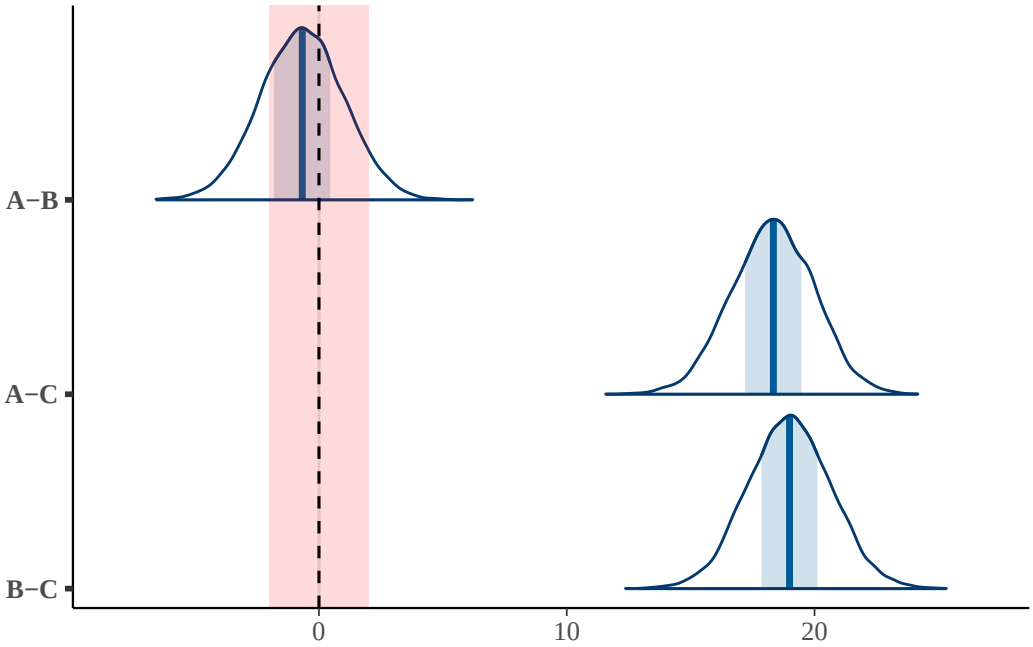
```
# Proportion of samples inside the ROPE [-2.00, 2.00]:

Inside ROPE
-----------
0.00 %
```

方法 A と方法 B の差は，その事後分布の大部分（7 割以上）が ROPE の内側に収まっている。つまり「両者の差は実質的に無視できる」と積極的に主張できる。これは頻度主義では原理的に言えなかったこと（帰無仮説の採択）であり，ベイズの大きな利点である。一方，方法 A と方法 C の差は ROPE に一切入っておらず，実質的にも明確な差があると判断できる。

この ROPE による判断も，差の事後分布に ROPE の帯を重ねて描くと直感的である。

```
# 差の事後分布に ROPE [-2, 2] の帯（赤）を重ねる
bayesplot::mcmc_areas(as.matrix(pair)) +
  ggplot2::annotate("rect",
    xmin = -2, xmax = 2, ymin = -Inf, ymax = Inf,
    alpha = 0.15, fill = "red"
  ) +
  ggplot2::geom_vline(xintercept = 0, linetype = "dashed")
```



赤い帯が ROPE である。方法 A と方法 B の差の山は，その大部分が赤い帯の中に収まっている（実質的に同等）。これに対し方法 A と方法 C，方法 B と方法 C の差の山は，赤い帯からはるか右に外れている（実質的に差がある）。区間が 0 をまたぐかどうかという見方に，ROPE の帯を重ねることで，「差がないとはいえない」のか「実質的に差がない」のかを描き分けられるのである。

ROPE の幅をどう決めるかは研究者の判断であり，分野の知見にもとづくべきものである。基準が思いつかない場合，Kruschke (2018) は標準化した尺度で ± 0.1 標準偏差を目安として提案している。`bayestestR` では `range = "default"`とすると，この標準化基準にもとづく ROPE を自動的に設定してくれる。

18.5.2　ベイズファクター

二つ目の道具はベイズファクター（Bayes Factor, BF）である。これは二つのモデルのどちらがデータをよく説明するかを，相対的な比で表すものである。ベイジアンモデリングの章でも触れたように，ベイズ的なモデル評価はこの BF に一元化できるといってよい。

ここでは，「効果はゼロである（差がない）」というモデルと「効果はゼロでない（差がある）」というモデルを比べる。BF を計算する一つの簡便な方法が，Savage-Dickey 密度比である。これは，事前分布と事後分布の，ゼロ点における密度の比を取るというもので，`brms` で事前分布もサンプリングしておけば `bayestestR` で計算できる。

そのためにまず，`brms` で同じ分散分析モデルを推定する。R の `formula` に `score ~ method` と書くだけで，内部では全体平均と各群の効果という形のモデルが組まれる。効果コーディング（`contr.sum`）を指定すると，先ほど Stan で推定した効果 δ_j とほぼ同じものが係数として得られ，答え合わせにもなる。

```
# 効果コーディング：全体平均からのずれ（効果）として係数を得る
contrasts(dat$method) <- contr.sum(3)

fit_brms <- brm(
```

```
  score ~ method,
  data = dat,
  prior = set_prior("normal(0, 50)", class = "b"),
  sample_prior = "yes", # BFのため事前分布もサンプリング
  chains = 4, iter = 11000, warmup = 1000,
  refresh = 0, seed = 1
)
```

```
fixef(fit_brms)
```

```
           Estimate Est.Error      Q2.5     Q97.5
Intercept 58.810085 0.6848841 57.460350 60.157594
method1    5.876573 0.9832975  3.958309  7.811276
method2    6.571917 0.9788995  4.638359  8.501470
```

method1，method2 が方法 A，方法 B の効果であり，先ほど Stan で得た delta[1]，delta[2] とほぼ一致している。自分で書いた Stan モデルと，brms が自動で組んだモデルが同じ結果を返すことが確認できた。brms は線型モデルの範囲なら手早く同じ推定ができるので，定型的な分析では便利である。

このモデルから BF を計算する。

```
bayestestR::bayesfactor_parameters(fit_brms, null = 0)
```

```
Sampling priors, please wait...

Bayes Factor (Savage-Dickey density ratio)

Parameter   |       BF
----------------------
(Intercept) | 9.89e+97
method1     | 1.84e+04
method2     | 3.61e+05

* Evidence Against The Null: 0
```

method1，method2 の BF はいずれも非常に大きな値であり，「効果がゼロ」というモデルに対して「効果がある」というモデルが圧倒的に支持される。一般に BF が 3 を超えれば一方のモデルが優っている目安とされるが，ここではそれをはるかに上回っている。

逆に，「差がない（同等である）」ことの証拠も BF で評価できる。方法 A と方法 B の差については，brms の hypothesis 関数で「$\mu_A - \mu_B = 0$」という仮説を検証してみよう。

```
brms::hypothesis(fit_brms, "method1 - method2 = 0")
```

```
Hypothesis Tests for class b:
```

```
             Hypothesis Estimate Est.Error CI.Lower CI.Upper Evid.Ratio
1 (method1-method2) = 0     -0.7       1.7    -4.02     2.66      38.12
  Post.Prob Star
1       0.97
---
'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
'*': For one-sided hypotheses, the posterior probability exceeds 95%;
for two-sided hypotheses, the value tested against lies outside the 95%-CI.
Posterior probabilities of point hypotheses assume equal prior probabilities.
```

出力の Evid.Ratio が，「差がない」という仮説を支持する BF である。1 をかなり上回っており，方法 A と方法 B については「差がない」ほうが支持される。ROPE の結論と整合的である。

18.5.2.1　サベージ・ディッキー法を絵で見る

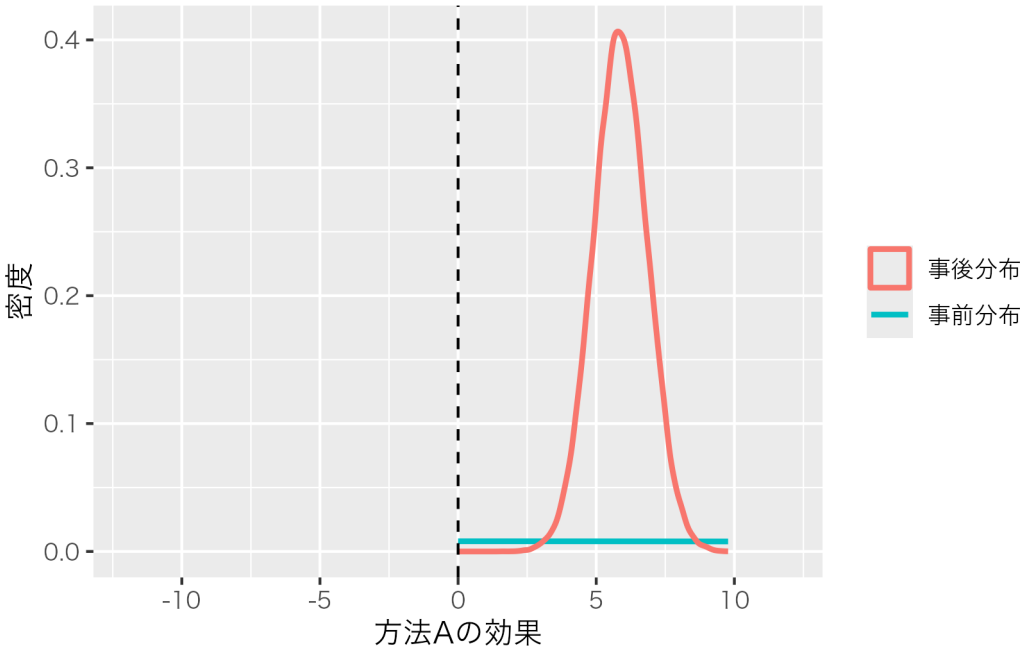
Savage-Dickey 密度比が何をしているのかは，事前分布と事後分布を重ねて描くとよくわかる。やっていることは単純で，注目するパラメータについて，事前分布と事後分布の，値ゼロにおける密度の高さを比べているだけである。データを見る前（事前分布）に比べて，データを見た後（事後分布）にゼロ点の密度がどれだけ減ったか（または増えたか）を見るのである。ゼロでの密度が減っていれば「効果はゼロでない」証拠，増えていれば「効果はゼロ」の証拠になる。

brms のフィット結果から事前分布と事後分布の MCMC サンプルを取り出して描いてみよう。事前分布には normal(0, 50) を置いたので，描画では解析的な正規分布をそのまま重ねる（二つの効果の差については，独立な二つの効果の差なので事前分布の標準偏差は $50\sqrt{2}$ になる）。

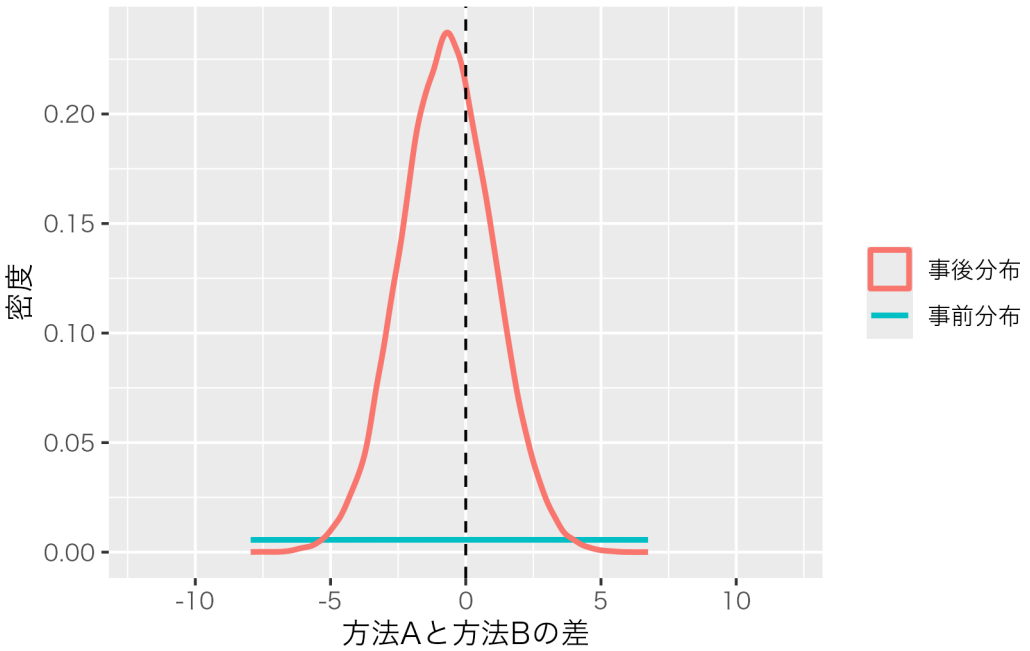
```
dd <- brms::as_draws_df(fit_brms)

# サベージ・ディッキー法を絵にする関数
sd_plot <- function(post, prior_sd, xlim, title) {
  tibble(value = post) |>
    ggplot(aes(value)) +
    # 事前分布（解析的な正規分布）
    stat_function(
      fun = dnorm, args = list(mean = 0, sd = prior_sd),
      aes(color = "事前分布"), linewidth = 1
    ) +
    # 事後分布（MCMCサンプルから推定した密度）
    geom_density(aes(color = "事後分布"), linewidth = 1) +
    geom_vline(xintercept = 0, linetype = "dashed") +
    coord_cartesian(xlim = xlim) +
    labs(x = title, y = "密度", color = NULL) +
    theme(text = element_text(family = "Hiragino Sans"))
}
```

```
# 方法Aの効果：ゼロ点の密度が大きく減る＝「効果はゼロでない」証拠
sd_plot(dd$b_method1, prior_sd = 50, xlim = c(-12, 12), title = "方法Aの効果")
```




```
# 方法Aと方法Bの差：ゼロ点に密度が集まる＝「差はゼロ」の証拠
sd_plot(dd$b_method1 - dd$b_method2,
  prior_sd = 50 * sqrt(2), xlim = c(-12, 12), title = "方法Aと方法Bの差")
```



一枚目の「方法 A の効果」では，平べったい事前分布に対して，事後分布は 0 からはっきり右に離れた位置に山を作っている。ゼロ点（破線）における事後分布の高さは，事前分布の高さよりもずっと低くなっている。この高

さの比が BF であり，事後分布がゼロ点をほとんど見捨てているので，「効果はゼロでない」という大きな BF になる。

二枚目の「方法 A と方法 B の差」は対照的である。事後分布の山はちょうど 0 の真上に乗っており，ゼロ点の高さは事前分布よりも高い。データを見たことで，かえってゼロ点の密度が増えたわけである。だからこの BF は「差はゼロ」を支持する向きになる。先ほど `bayesfactor_parameters` や `hypothesis` で得た数値が，この二つの絵の，破線上での高さの比に対応している。

なお，BF には注意も必要である。「3 を超えたら差がある」といった二値判断は，p 値と同じく過大解釈や基準超えのための不正を招きかねない。また BF は事前分布の置き方によって同じデータでも値が大きく変わることが知られており，事前分布の選択には議論がある。t 検定や分散分析のような定型的なモデルについては，`BayesFactor` パッケージや，それを GUI で扱える JASP(JASP Team, 2025) が BF を自動計算してくれるので，それらを併用するのもよいだろう。

```
# BayesFactorパッケージによる古典的なBF（JASPの計算エンジンと同じ）
pacman::p_load(BayesFactor)
anovaBF(score ~ method, data = as.data.frame(dat))
```

18.5.3　事後予測分布

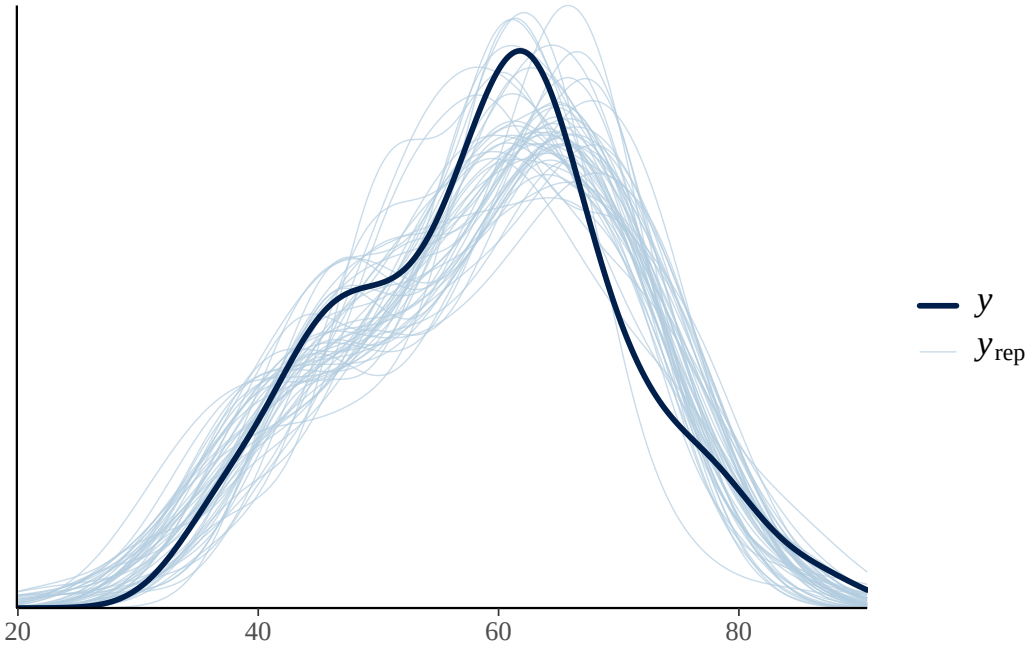
三つ目の道具は事後予測分布（posterior predictive distribution）である。事後予測分布には大きく二つの使い道がある。一つは「そもそも立てたモデルはデータをきちんと再現できているのか」というモデルの妥当性チェック，もう一つは効果の大きさを実際のデータのレベルで評価することである。順に見ていこう。

18.5.3.1　モデルの妥当性をチェックする

まずは妥当性チェックである。考え方はこうである。推定したパラメータ（の事後分布）を使って，モデルから人工的にデータを生成してみる。もしモデルが適切なら，この人工データ（予測データ）は，実際に観測されたデータとよく似た分布になるはずである。逆に，予測データが実データと食い違っていれば，モデルがデータの何らかの特徴を捉えそこねている，ということになる。

Stan モデルの `generated quantities` ブロックで生成しておいた `X_pred` が，まさにこの予測データである。MCMC のステップごとに一組のデータが生成されているので，その何本かを実データに重ねて描いてみる。`bayesplot` パッケージの `ppc_dens_overlay` 関数を使う。

```
y_obs <- dat$score
y_rep <- fit_a$draws("X_pred", format = "matrix")
# 予測データの中から50本を選んで実データに重ねる
bayesplot::ppc_dens_overlay(y = y_obs, yrep = y_rep[1:50, ])
```



濃い線が実際に観測されたデータの分布，薄い線が事後予測分布から生成された予測データの分布である。両者がよく重なっていれば，「正規分布を仮定したこのモデルは，データの全体的な形をうまく再現できている」と判断できる。今回はデータを正規分布から生成したのだから当然うまく重なるが，実データで分布が大きく歪んでいたり外れ値があったりすると，正規モデルの予測がそれを再現できず，重なりの悪さとして検出される。そのときは，より裾の重い t 分布を使うなど，モデルを見直す手がかりになる。

18.5.3.2　データのレベルで効果を測る

事後予測分布の使い道は，モデルの妥当性チェックだけではない。むしろ心理学の実践では，こちらの使い方のほうが重要になることも多い。

ここまで見てきた優越率や差の事後分布は，いずれも**母平均というパラメータのレベル**の話であった。たとえば「方法 A の母平均が方法 C の母平均より大きい確率」はほぼ 1 で，集団としての平均には明確な差があるといえる。しかし，現場で本当に知りたいのは「では，方法 A で学んだ一人と方法 C で学んだ一人を連れてきたら，どれくらいの割合で A の人のほうが高得点なのか」といった，**個々のデータのレベル**の問いであることも多い。

事後予測分布は，まさに予測される個々のデータの分布である。だから，パラメータではなくデータのレベルで効果を測ることができる。母平均の差だけでなく，個人ごとのばらつき σ も込みで「効果がどれほど体感されるか」を表現できるのである。

そのために，各事後サンプルから「無作為な一人」の予測得点を生成し，三つの量を計算してみよう。

```
set.seed(1)
# 各事後サンプルから，各群の「無作為な一人」の予測得点を生成する
xA <- rnorm(nrow(draws_a), draws_a$`mu[1]`, draws_a$sigma) # 方法A
xB <- rnorm(nrow(draws_a), draws_a$`mu[2]`, draws_a$sigma) # 方法B
xC <- rnorm(nrow(draws_a), draws_a$`mu[3]`, draws_a$sigma) # 方法C
```

```
# （1）データレベルの優越率：方法Aの一人が方法Cの一人を上回る確率
mean(xA > xC)
```

```
[1] 0.954
```

```
# 比較：母平均レベルの優越率（先に見たもの）
mean(draws_a$`mu[1]` > draws_a$`mu[3]`)
```

```
[1] 1
```

```
# （2）被覆率（重複度）：2群の予測分布がどれだけ重なっているか
bayestestR::overlap(xA, xC) # 方法A vs 方法C
```

```
# Overlap

24.0%
```

```
bayestestR::overlap(xA, xB) # 方法A vs 方法B
```

```
# Overlap

96.9%
```

```
# （3）閾上率：70点以上を取る確率
mean(xA >= 70)
```

```
[1] 0.250375
```

```
mean(xC >= 70)
```

```
[1] 0.0015
```

結果を順に読み取ろう。

優越率を見ると，母平均のレベルではほぼ 1（確実に A のほうが上）であったのに対し，データのレベルでは 0.95 前後にとどまっている。これは，平均としては A が明確に上でも，個人差があるために「たまたま C の人が A の人を上回る」ことが 5% ほど起こりうる，ということを意味する。集団の傾向と，目の前の一人ひとりに起こることは別だという当たり前の事実が，数値として表現されている。このデータレベルの優越率は，共通言語効果量（common language effect size）とも呼ばれ，効果の大きさを直感的に伝えるのに向いている。

被覆率（重複度）は，二つの群の予測分布がどれだけ重なっているかを表す。方法 A と方法 C はおよそ 2 割しか重なっておらず（8 割は分離している），効果が大きいことがわかる。一方，方法 A と方法 B は 9 割以上が重なっており，両者は実質的にほとんど同じ分布だといえる。重なりが小さいほど効果が大きい，という関係であり，ROPE やベイズファクターで得た「A・B は同等，C ははっきりちがう C」という結論が，分布の重なり具合としても確認できる。

閾上率は，ある基準値を超える確率である。たとえば合格ラインを 70 点とすると，方法 A では 2 割以上が合格圏に入るのに対し，方法 C ではほとんど誰も届かない。「平均が何点違うか」よりも「合格者が何割出るか」のほうが実務的な意思決定に直結する場面では，このように基準値を決めて閾上率を比べるのがわかりやすい。

このように，事後予測分布を使うと，平均値の差という一点だけでなく，個人のレベルでの優越率，分布の重なり，基準値の超過率といった，さまざまな角度から効果を検証できる。差があるかないかの二値判断にとどまらず，「その差が現場でどういう意味を持つのか」まで踏み込めるのが，データ生成メカニズムから出発するベイズ的アプローチの強みである。

18.6　まとめ

この章では，t 検定と分散分析という最も基本的な平均値差の分析を，ベイズの観点から組み直した。要点を整理しておこう。

- 分散分析の組成（データ＝全体平均＋効果＋誤差）は，そのままデータ生成モデルになる。これを Stan で記述すれば，効果 δ_j の分布を直接推定できる。
- ベイズでは差を確率分布として持つので，区間推定，優越率，全ペアの比較などを多重比較の調整なしで自由に行える。
- 判断の道具として，ROPE（実質的同等の領域による判断），ベイズファクター（モデル比較），事後予測分布（モデルの妥当性チェックに加え，データレベルの優越率・重複度・閾上率による効果の検証）がある。
- 「差があるとはいえない」と「差がない（同等である）」は別物であり，後者を積極的に主張できるのがベイズの強みである。

頻度主義の検定が悪いというわけではない。長い歴史の中で洗練された明快な手続きであり，多くの知見がその上に積み重ねられてきた。しかし，平均値差を「差の分布」として直接捉え，意思決定の幅を広げられるベイズのアプローチは，研究者により豊かな表現力を与えてくれる。どちらか一方ではなく，両方の視点を持っておくことが望ましい。

18.7　課題

以下のデータセットは，三つの群（`group`）について測定値（`y`）を記録したものです。データセットは ex_anova_bayes.csv からダウンロードできます。このデータに対して，本章で扱った方法を用いて以下を順に報告してください。

1. 本章の `bayes_anova.stan` を用いて，整然データの形で三群の平均値差をベイズ推定し，各群の母平均 `mu` と効果 `delta` の事後分布の代表値・95% 区間を示す。
2. MCMC サンプルから，三つの全ペアの差の事後分布を求め，それぞれの 95% 区間と優越率を報告する。
3. 各ペアの差について，ROPE を ± 0.5 として `bayestestR::rope` を計算し，「実質的に差がない」といえるペアがあるかどうかを論じる。
4. `brms` で同じモデルを推定し（効果コーディングを使うこと），Stan の結果と係数が一致することを確認する。さらにベイズファクターを計算し，効果がゼロでないといえるかを論じる。
5. 事後予測分布を `ppc_dens_overlay` で描き，正規分布を仮定したモデルがデータをうまく再現できている

かを評価する。

6. 事後予測分布を使って，群の組ごとに（a）データレベルの優越率，（b）`bayestestR::overlap` による予測分布の重複度（被覆率）を求める。さらに適当な基準値を一つ決めて，各群の閾上率を比較する。これらが母平均レベルの結果とどう違うかを論じる。

レポートには，使用した R コードと，各項目の結果に対する簡潔な解釈を含めてください。

第 19 章

演習問題

このコースでは，確率的に生じるデータを意識しながら，そのメカニズムから生成されるデータを乱数によって具体化し，そうしたサンプルデータに基づいて検定と分析のロジックを学んできました。

検定には，サンプルサイズ，有意水準，検出力，効果量が相互に関わっており，確率的判断がこれらの関数としてどのように現れるかを確認しました。

また仮想データをつくれるところから，QRPs のシミュレーションを行ったり，例数設計が行えることも見てきました。線型モデルやそのほか発展的なモデルについても，データ生成メカニズムの観点からアプローチできます。

また，一般線型モデル，一般化線型モデル，一般化線型混合モデル，階層線型モデルと，線型モデルの展開について見てきました。

いずれのモデルも，データによりフィットした，より適切なモデルにするための工夫です。もちろん複雑なモデルであればあるほど良い，というわけではありません。むしろモデルは単純な方がいいのですが，単純だからという理由でデータに合わないモデルを適用するのは，適切ではありません。

モデルが複雑になるにつれて，推定方法も最小二乗法，最尤法，ベイズ法と展開してきました。いずれも同じ母数を推定するための手法ですから，ここに優劣はなく，最終的な結論も同じになるべきです。確率の解釈の違いなどもありますので，主義主張の対立と思われることもありますが，ユーザとしては実用面での有用性から選ぶということでも良いかもしれません。

より発展的な内容として，確率的プログラミング言語を用いれば，データ生成メカニズムを記述することでモデルパラメータの推定が可能になることにも触れました。

確率的プログラミング言語を利用するには，プログラミングの技術，ベイズ統計の理論，MCMC による近似の理論と方法についての知識が必要です。ですが，これらの技術を身につけると，分析の可能性はもっと広がります。既存の統計モデルを当てはめるのではなく，あなた自身の考えたモデルを作り上げることもできます。その自由度は無限ともいえるほどで，想像力を働かせて分析モデルをデザインするのは大変楽しいことでもあります。また，そういう観点を得ることができれば，従来の統計モデルがどういう設計をされているのかについても理解が進みます。

MCMC は確率を乱数で具現化する手法です。MCMC に限らず，確率を乱数で具現化し，具体的なデータとして

分析することで，統計モデルを使いこなせるようになりましょう。ツールは使うべきもので，使われるべきものではないからです。

19.1 最終課題

- 無相関検定において，真の状態が母相関 $\rho = 0.4$ であったときに，サンプルサイズ $n = 20$ のデータをとって検定を行うとします。この時の帰無仮説の分布と，真の状態の分布を重ねて図示し，$\alpha = 0.05$ の臨界値，検出力を可視化する図を描くコードを書いてください。(参考；南風原（2002）,Pp.144)

- 2 要因 Between デザインの分散分析において，交互作用のみ有意になるようなサンプルデータを作るコードを書いてください。また，サンプルデータが正しくできているかどうかを確認するために，`anovakun` での分析結果も出力させてください。

- 2 つの変数 X,Y をもつ 3 つの群があり，群ごと X,Y の相関を見るとすべて $r = -0.3$ 程度の負の相関を持っているが，3 つの群をあわせて X,Y の相関を見ると正の相関を示すようなデータセットを作るコードを書いてください。なお，出来上がったデータは群ごとに色分けした散布図で図示するようにしてください。

  - ヒント：群ごとに回帰分析のサンプルデータを作ることを考え，傾きは一貫して $\beta_1 = -0.3$ であるのに対し，群ごとの切片 β_0 を適当に調整すると良いでしょう。
  - ねらい：このようなデータは，相関を見るときに可視化することの重要性を伝えるとともに，階層線型モデルの必要性を理解することに役立ちます。

- brms パッケージを用いて，ベイズ統計の観点から重回帰分析を実行してください。従属変数を自由に設定し，複数の説明変数を含むモデルを構築してください。事前分布の設定，MCMC の収束診断，事後分布の可視化，および最尤推定との比較を含めた包括的な分析を行ってください。データは何を用いても構いません。

  - ヒント：`brm()` 関数で `prior` オプションを使って事前分布を明示的に設定し，`plot()` と `pp_check()` を用いて結果を評価してください。サンプルデータとしては `mtcars` や `iris`，`psych::bfi` などが利用できます。
  - ねらい：ベイズ統計の実践的な応用と，頻度主義統計との違いを理解することを目的とします。

- 探索的因子分析と確認的因子分析の比較分析を行ってください。同一データセットに対して，まず探索的因子分析（EFA）を実行して因子構造を探索し，次にその結果を基に確認的因子分析（CFA）モデルを構築してください。両手法の結果を比較し，それぞれの利点と限界について考察してください。データは何を用いても構いません。

  - ヒント：`psych` パッケージの `fa()` 関数で EFA を実行し，`lavaan` パッケージの `cfa()` 関数で CFA を実行してください。適合度指標の比較も含めてください。サンプルデータとしては `psych::bfi`（ビッグファイブ性格検査），`psych::ability`（認知能力検査），`HolzingerSwineford1939`（Holzinger & Swineford の認知テスト）などが利用できます。
  - ねらい：測定モデルの探索と検証の違い，およびそれぞれの統計的手法の特徴を理解することを目的とします。

- 階層線型モデル（HLM）または GLMM を用いて，ネストされたデータ構造を持つ分析を実行してください。個人がグループにネストされている状況を想定し，ランダム切片モデルとランダム傾きモデルの両方を比較してください。また，ICC を計算してグループレベルの効果の大きさを評価してください。データは何を用いても構いません。
  - ヒント：`brms` パッケージの `(1|group)`（ランダム切片）と `(1+x|group)`（ランダム切片ランダム傾き）の記法を使い分けてください。サンプルデータとしては `lme4::sleepstudy`（睡眠研究データ），`nlme::Orthodont`（歯科矯正データ），`mlmRev::Exam`（学校別試験成績データ），`mlmRev::Exam`（機械データ）などが利用できます。
  - ねらい：階層構造を持つデータの適切な分析手法を理解し，個人レベルとグループレベルの変動を区別する重要性を学ぶことを目的とします。

References

足立 浩平（2006）．多変量データ解析法――心理・教育・社会系のための入門――　ナカニシヤ出版

アデラ＝マリア イスヴォラヌ・サシャ エプスカンプ・ローレンス ウォルドープ・デニー ボースブーム（2024）．心理ネットワークアプローチ入門: 行動科学者と社会科学者のためのガイド　勁草書房（Original work published 2022, Routledge）

馬場 真哉（2018）．時系列分析と状態空間モデルの基礎: R と Stan で学ぶ理論と実装　プレアデス出版

Bernaards, C. A. & Jennrich, R. I. (2005). Gradient Projection Algorithms and Software for Arbitrary Rotation Criteria in Factor Analysis. *Educational and Psychological Measurement, 65*, 676–696. https://doi.org/10.1177/0013164404272507

Gabry, J., Češnovar, R., & Johnson, A. (2023). *cmdstanr: R Interface to 'CmdStan'*. https://mc-stan.org/cmdstanr/, https://discourse.mc-stan.org.

グリム L.G.・ヤーノルド P.R.（2016）．研究論文を読み解くための多変量解析入門 基礎篇: 重回帰分析からメタ分析まで　北大路書房（Original work published 1994, American Psychological Association）

南風原 朝和（2002）．心理統計学の基礎――統合的理解のために――　有斐閣

南風原 朝和（2014）．心理統計学の基礎――続・統合的理解のために――　有斐閣

浜田 宏（2018）．その問題、数理モデルが解決します　ベレ出版

浜田 宏・石田 淳・清水 裕士（2019）．社会科学のためのベイズ統計モデリング　朝倉書店

浜田 宏（2020）．その問題、やっぱり数理モデルが解決します　ベレ出版

キーラン・ヒーリー（2021）．データ分析のためのデータ可視化入門　講談社（Original work published 2018, Princeton Univ Pr）

平岡 和幸・堀 玄（2009）．プログラミングのための確率統計　オーム社

池田 功毅・平石 界（2016）．心理学における再現可能性危機：問題の構造と解決策　心理学評論，*59*（1），3–14. https://doi.org/10.24602/sjpr.59.1_3

JASP Team (2025). JASP (Version 0.95.1)[Computer software].

小森 政嗣（2022）．R と Stan ではじめる　心理学のための時系列分析入門 (K S専門書)　講談社

河野 敬雄（1999）．確率概論　京都大学学術出版会

小杉 考司（2018）．言葉と数式で理解する多変量解析入門　北大路書房

小杉 考司（2022）．心理学データ解析応用: R と Stan で学ぶフリーで楽しい心理統計の世界　Independently published

小杉 考司・紀ノ定 保礼・清水 裕士（2023）．数値シミュレーションで読み解く統計のしくみ〜R でためしてわかる心理統計　技術評論社

クルシュケ J.K.（2017）．ベイズ統計モデリング: R, JAGS, Stan によるチュートリアル 原著第 2 版　共立出版（Original work published 2014, Elsevier）

Kruschke, J. K. (2018). Rejecting or accepting parameter values in Bayesian estimation. *Advances in Methods and Practices in Psychological Science, 1*(2), 270–280.

ランダー，J.P.（2018）．みんなの R 第 2 版　マイナビ出版（Original work published 2017, Addison-Wesley Professional）

リー M.D・ワゲンメーカーズ E-J.（2017）．ベイズ統計で実践モデリング: 認知モデルのトレーニング　北大路書房（Original work published 2013, Cambridge University Press）

Makowski, D., Ben-Shachar, M. S., & Lüdecke, D. (2019). bayestestR: Describing Effects and their Uncertainty, Existence and Significance within the Bayesian Framework. *Journal of Open Source Software, 4*(40), 1541. https://doi.org/10.21105/joss.01541

松村 優哉・湯谷 啓明・紀ノ定 保礼・前田 和寛（2021）．改訂 2 版 R ユーザのための RStudio[実践] 入門 ——tidyverse によるモダンな分析フローの世界——　技術評論社

シャロン・バーチュ・マグレイン（2018）．異端の統計学ベイズ　草思社（Original work published 2011, Yale University Press）

宮川 雅巳（1997）．グラフィカルモデリング (統計ライブラリー)　朝倉書店

永田 靖・吉田 道弘（1997）．統計的多重比較法の基礎　サイエンティスト社

西里 静彦（2010）．行動科学のためのデータ解析 – 情報把握に適した方法の利用　培風館

西内 啓（2017）．統計学が最強の学問である [数学編]——データ分析と機械学習のための新しい教科書——　ダイヤモンド社

岡太 彬訓・今泉 忠（1994）．パソコン多次元尺度構成法　共立出版

R プログラミング本格入門: 達人データサイエンティストへの道　共立出版（Original work published 2016, Packt Publishing）

Revelle, W. (2021). *psych: Procedures for Psychological, Psychometric, and Personality Research*. R package version 2.1.3. Northwestern University. Evanston, Illinois. from https://CRAN.R-project.org/package=psych

Rinker, T. W. & Kurkiewicz, D. (2018). *pacman: Package Management for R*. version 0.5.0. Buffalo, New York. from http://github.com/trinker/pacman

Rosseel, Y. (2012). lavaan: An R Package for Structural Equation Modeling. *Journal of Statistical Software, 48*(2), 1–36. https://doi.org/10.18637/jss.v048.i02

佐藤 坦（1994）．はじめての確率論——測度から確率へ——　共立出版

新納 浩幸（2007）．R で学ぶクラスタ解析　オーム社

Shojima, K. (2022). *Test data engineering: latent rank analysis, biclustering, and bayesian network*. Springer.

Stevens, S. S. (1946). On the theory of scales of measurement. *Science, 103*(2684), 677–680.

高橋 康介 石田 基広（編）(2018)．再現可能性のすゝめ　共立出版

高根 芳雄（1980）．多次元尺度法　東京大学出版会

豊田 秀樹（2009）．検定力分析入門——R で学ぶ最新データ解析ー——　東京図書

豊田 秀樹（2017）．もうひとつの重回帰分析　東京図書

豊田 秀樹（2020）．瀕死の統計学を救え！　朝倉書店

豊田 秀樹・大橋 洸太郎・池原 一哉（2013）．自由記述のカテゴリ化に伴う観点の飽和度としての捕獲率　データ分析の理論と応用，*3*(1)，49–61. https://doi.org/10.32146/bdajcs.3.49

Wickham, H. (2014). Tidy Data. *Journal of Statistical Software, 59*, 1–23. https://doi.org/10.18637/jss.v059.i10

R 言語徹底解説　共立出版（Original work published 2015, Taylor & Francis Group）

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., … & Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software, 4*(43), 1686. https://doi.org/10.21105/joss.01686

紀ノ定 保礼（2018）. 己の「歌唱力」を推定する 豊田 秀樹（編）　北大路書房

吉田 寿夫・村井 潤一郎（2021）．心理学的研究における重回帰分析の適用に関わる諸問題　心理学研究，*92*（3），178–187. https://doi.org/10.4992/jjpsy.92.19226

吉田 伸生（2021）．確率の基礎から統計へ 新装版　日本評論社

Zeileis, A. (2005). CRAN Task Views. *R News, 5*(1), 39–40.

シ（2016）．計算機言語のまとめノート　暗黒通信団

総務省（2020）．統計表における機械判別可能なデータ作成に関する表記方法

第 20 章

あとがき

この書き物は，2024 年から東京大学教養学部で担当させてもらうことになった，「心理統計実習」のためのテキストです。R の使い方からベイジアンモデリングまで，割と好き勝手に中身を詰め込んで書いてあります。自分の独断と偏見で中身を作っていますし，いちおうの章立てもありますがボリュームを気にすることなく書いてあります。1 コマで一章のような進め方をする，いわゆる「テキスト」とは違うので使い勝手が悪いかもしれません。実際，2025 年度の授業では見出しを提示して受講生が聞きたいテーマを中心に扱う，ということにしました。扱っていない章については自分で読んでおいてね，というスタイルで使っています。

演習の授業なので R を使うことになるのですが，生成 AI がある昨今，必ずしも自分の手で書く必要がないかもしれません。そこでこのテキストではコピーできるように Quarto で書き，Web ページで提供するというスタイルをとりました。生成 AI を使うこと自体は私は否定しませんし，このテキストも生成 AI によるサポートが入っています (誤字脱字のチェックや，課題にどれだけ正答するかをチェックするなどしています)。大事なのは生成 AI を使うことであって，使われる側に回らないこと。目的は単位を取ることではなく自分が成長することであり，自分が学ぶ，知る楽しみを機械に奪われてはならない，という心意気です。

また，オンライン資料にしているのは，R や Stan がフリーソフトウェアであり，発展し続けるものだからということでもあります。バージョンが上がる，リンク先が変わるというのは頻発しますので，オンライン教材にすることで随時アップデートできるようになります。紙媒体への憧れもあるので，PDF と ePub 形式も作りましたが (笑)

2024 年度はまだ全ての章がなく，特にベイズに関する章は 2025 年度に授業をしながら書き加えるというスタイルでした。本当は 2024 年度中に完成するはずだったんですが，2025 年度の夏休みいっぱいまでかかりました。でもそのおかげで，多変量解析やベイジアンモデリングなど，余裕を持って盛り込めましたけど。

ここに書いてある内容は，私が他のテキストや資料，書籍に書いたことと重複するところが少なくありません。同じ人間が書いているのですから，ご容赦ください。棲み分けする必要があるのかどうかもわかりませんが，このテキストはやや演習・実践寄りというところが違うでしょうか。ただベイズ統計を含め，心理統計の演習というシーンで必要な情報を統一的に扱っている資料があったほうがいいなと思っています。何かのお役に立てれば幸いです。

自分の授業テキストですから，校閲や査読が入ってるものではありません。もし誤りがありましたら，単に私の責任です。もしお気づきの点がありましたら，ご指摘いただけますとすぐに修正しますので，ご連絡ください。

2025 年 9 月 20 日　小杉　考司

第 21 章

インストールガイド

この教材の中で使われているパッケージを，一括インストールするためには次のコードを実行してください。

```
# ================================================================================
# PsyStatsPracticals 環境セットアップ
# https://kosugitti.github.io/PsyStatsPracticals/ で利用される全パッケージを
# 一括でインストール・読み込みする
# ================================================================================

# --------------------------------------------------------------------------------
# パッケージマネージャの導入
# --------------------------------------------------------------------------------
if (!requireNamespace("pacman", quietly = TRUE)) {
  install.packages("pacman")
}

# --------------------------------------------------------------------------------
# コア・データ操作
# --------------------------------------------------------------------------------
pacman::p_load(
  tidyverse,    # dplyr / ggplot2 / tidyr / forcats などのメタパッケージ
  broom         # 分析結果のtidy整形
)

# --------------------------------------------------------------------------------
# 可視化
# --------------------------------------------------------------------------------
pacman::p_load(
  ggplot2,      # tidyverseに含まれるが単独読み込み箇所もあり
  patchwork,    # 図の連結
```

```
  gridExtra,     # 図の配置
  RColorBrewer, # 配色
  ggrepel,       # ラベル重複回避
  corrplot,      # 相関行列の可視化
  GGally,        # 散布図行列
  bayesplot,     # MCMC可視化
  qgraph,        # ネットワーク図
  semPlot,       # SEMパス図
  lavaanPlot     # lavaan結果の図示
)

# ------------------------------------------------------------------------------
# 基礎統計・多変量
# ------------------------------------------------------------------------------
pacman::p_load(
  psych,         # 心理統計汎用
  car,           # 回帰診断（Anova等）
  MASS,          # 多変量正規乱数など
  effsize,       # 効果量
  pwr,           # 検定力分析
  e1071,         # 歪度・尖度等
  mclust         # 混合モデルクラスタリング
)

# ------------------------------------------------------------------------------
# 回帰・混合モデル
# ------------------------------------------------------------------------------
pacman::p_load(
  lmerTest,      # 線形混合モデル
  multilevel     # ICC等
)

# ------------------------------------------------------------------------------
# ベイズ
# ------------------------------------------------------------------------------
pacman::p_load(
  brms,          # Stanフロントエンド
  cmdstanr,      # CmdStanインターフェイス
  bayestestR     # 事後分布要約
)
```

```
# 注: cmdstanr はCRANではなくStan-devリポジトリから配布されているため，
#     未導入の場合は以下を実行する必要がある．
#     install.packages("cmdstanr",
#       repos = c("https://stan-dev.r-universe.dev", getOption("repos")))
#     インストール後 cmdstanr::install_cmdstan() でCmdStan本体を導入する．

# ------------------------------------------------------------------------------
# 構造方程式モデリング
# ------------------------------------------------------------------------------
pacman::p_load(
  lavaan        # SEM
)

# ------------------------------------------------------------------------------
# 項目反応理論・テスト理論
# ------------------------------------------------------------------------------
pacman::p_load(
  ltm,          # 一次元IRT
  mirt,         # 多次元IRT
  exametrika    # テスト理論統合
)

# ------------------------------------------------------------------------------
# 多次元尺度法
# ------------------------------------------------------------------------------
pacman::p_load(
  smacof        # MDS
)

# ------------------------------------------------------------------------------
# テキストマイニング
# ------------------------------------------------------------------------------
pacman::p_load(
  gibasa        # 日本語形態素解析
)
# 注: gibasa はMeCab(またはmecab-ipadic-neologd)等の形態素解析エンジン本体を
#     別途要する．macOSでは `brew install mecab mecab-ipadic` が一般的．
```